

Verilog FPGA Program 2

MicroBlaze

(Target Board : Arty A7-35T)

아이힐

Version 2.1

목차

1	개요	4
2	HW 구성	6
3	SW 설치	7
4	Hello World 구현	12
4.1	기본 Template 구현	12
4.1.1	프로젝트 생성	12
4.1.2	Create Block Design	15
4.1.3	Create HDL Wrapper	21
4.1.4	User Logic 구현	22
4.1.5	xdc 구현	27
4.1.6	Generate Bitstream	30
4.1.7	Export Hardware	31
4.2	Embedded SW 구현	34
4.3	Flash 에 프로그램 다운로드	45
4.3.1	vitis 에서 다운로드	45
4.3.2	elf 파일을 vivado 에서 추가 후 다운로드	50
5	MicroBlaze Peripheral 구현	59
5.1	Block Design	59
5.1.1	GPIO : LED_4bits	60
5.1.2	GPIO : btn0 (External Interrupt-1)	61
5.1.3	GPIO : btn1 (External Interrupt-2)	62
5.1.4	GPIO : btn3, 4(General Input, Toggle switch)	63
5.1.5	Timer	64
5.2	Application SW	71
5.2.1	xparameters.h file	72
5.2.2	소스 구조	75
5.2.3	코드 구현	77
5.3	다운로드 및 결과 확인	82
6	User Logic 인터페이스 구현	83
6.1	Block Design	84
6.2	User Logic 구현	88
6.2.1	mcu_pwm.v	89
6.2.2	pwm_top.v	90
6.2.3	spi_slave.v	91
6.2.3.1	통신 프로토콜	91

6.2.3.2 레지스터 맵	91
6.2.3.3 파형 분석	92
6.2.3.4 코드 구현	94
6.2.3.5 Simulation	102
6.2.4 BlazeTop.v	104
6.2.5 xdc 생성	107
6.3 Application SW	108
6.3.1 comm_task 복사	109
6.3.2 helloworld.c	109
6.3.3 통신 프로토콜	112
6.3.4 comm_task	113
6.4 다운로드 및 결과 확인	114
6.5 HW update	117
7 LightWeight IP (lwIP) Echo Server 구현	119
7.1 시스템 구성	119
7.2 lwIP HW 구현	121
7.2.1 프로젝트 생성	121
7.2.2 HW Block 생성	123
7.2.3 Create HDL Wrapper	145
7.2.4 Application SW	151
7.2.5 결과 확인	160
7.2.6 소스 분석	167
8 lwIP 활용	173
8.1 HW Design	173
8.2 User Logic 구현	178
8.3 Application SW 구현	185
9 W5500 을 이용한 TCP/IP 구현	195
9.1 프로젝트 생성	199
9.2 Block Design	200
9.3 Top Module 구현	210
9.4 응용 SW 구현	215
9.4.1 Memory Size	225
9.4.2 파일 구조	226
9.4.3 data_type.h	226
9.4.4 ax_common.h	227
9.4.5 w5500.c, w5500.h	227
9.4.6 w5500_task.c, w5500_task.h	227

9.4.7	w5500_socket.c, w5500_socket.h	228
9.4.8	w5500_loopback.c, w5500_loopback.h	228
9.4.9	helloworld.c	231
9.5	결과 확인	233
9.5.1	Build Project.....	233
9.5.2	PC Network 설정.....	234
9.5.3	프로그램 다운로드 및 결과 확인.....	235
9.5.4	외부 Flash 에 프로그램 다운로드.....	239
9.6	결론	240
10	Block Memory Interface - 1.....	241
10.1	프로젝트 생성	241
10.2	Block Design	243
10.3	Constraints 파일 추가.....	253
10.4	응용 SW 구현.....	256
11	Block Memory Interface - 2.....	261
11.1	프로젝트 생성	262
11.2	Block Design	263
11.3	User Logic Design	272
11.4	응용 SW 구현.....	282
11.5	다운로드 및 결과 확인.....	282
11.6	링크 스크립트 수정	288
12	참고 자료	290
13	Revision History.....	291

1. 개요

본서는 Xilinx FPGA 에서 MicroBlaze를 사용하는 방법을 설명합니다. MicroBlaze는 FPGA에서 IP 형태로 제공되는 프로세서입니다. MicroBlaze는 Processor Core 와 Peripheral 이 분리되어 있어서, 사용자가 목적에 맞게 Peripheral을 구성할 수 있습니다. MicroBlaze를 사용하는 방법은 크게 2가지가 있습니다. 2019 이전 버전의 Vivado(Vivado 내에 SDK 포함)를 사용하는 방법과 2019 이후 버전의 Vitis(Vitis 내에 Vivado 포함)를 사용하는 것입니다. 본서는 최신의 Vitis 2022.1을 기준으로 설명되어 있습니다. 본서는 임베디드 프로그램 경험과 FPGA 프로그램(Verilog) 경험이 있는 분들을 대상으로 하고 있습니다. MicroBlaze에 관심은 있었지만, 아직 경험해 보지 못한 분들에게 좋은 안내서가 될 것입니다. 본서에 설명된 내용만으로 실무에서 바로 적용할 수 있습니다. 본서를 구매하신 분들에게는 본서에서 설명된 모든 프로젝트 파일(소스 파일)을 제공하여 드립니다. 모든 소스들은 본서를 작성할 때 하나하나 테스트하고 검증된 소스들입니다. 또한 디버깅 및 개발에 필요한 Windows Application Program도 제공하여 드립니다. 본서를 통하여 MicroBlaze 세계에 입문하고자 하시는 분들께 많은 도움이 되시길 바랍니다.

본문의 구성은 다음과 같습니다.

2장에서는 HW 구성에 대해서 설명합니다.

3장에서는 Vitis 2022.1 을 설치하는 과정에 대해서 설명합니다. Vitis 는 매우 무거운 툴입니다. 설치시 주의해야할 사항과 현재 사용중인 Vivado와 충돌없이 설치하는 방법에 대해서 설명합니다.

4장에서는 MicroBlaze 를 이용하여 화면에 “Hello world”를 출력하는 과정을 설명합니다. 본장에서는 대략적인 흐름을 파악하는 것에 목적을 두고 설명합니다.

5장에서는 MicoBlaze 의 Peripheral 을 설명합니다. 주로 사용하는 GPIO, Timer, Uart, Interrupt 에 대해서 설명합니다. MicroBlaze에서 제공하는 Peripheral 이 비슷하기 때문에 본서에서 설명된 것을 익히시면 다른 Peripheral 도 쉽게 사용할 수 있습니다.

6장에서는 핵심적이고 중요한 내용을 다룹니다. 대부분의 자료가 Peripheral을 다루는 것에서 끝납니다. 그러나 FPGA에서 MicroBlaze를 사용하는 목적은 사용자가 설계한 Logic을 MicroBlaze를 통하여서 제어하는 것입니다. 궁극적으로는 UI(User Interface)를 통하여 사용자가 설계한 Logic을 제어하는 것입니다. 본서에서는 User Logic을 구성하기 위하여 PWM 모듈 4개를 추가하였습니다. User Logic을 제어하기 위한 Register Map을 구성하고, MicroBlaze, UI 를 통하여 User Logic을 제어합니다. 이 모든 과정을 설명하고 구현하고 결과를 보여줍니다. 본장에서 설명된 내용은 실무에 바로 적용할 수 있습니다. User Logic을 추가하고 Register Map을 추가하면 그 외의 모든 과정들은 구현되어 있는 그대로를 사용하면 됩니다.

7-8장은 버전 1.2에서 새롭게 추가된 내용으로 LwIP를 이용하여 TCP/IP를 구현하는 내용입니다.

7장에서는 lwIP를 이용하여 Echo Server를 구현하는 내용입니다. Microblaze에서 Cache(Instruction Cache, Data Cache)를 사용하기 위하여 DDR Controller를 구현합니다. 제공되는 lwip Echo Server Template(Sample Code)을 이용하여 결과를 확인합니다.

8장에서는 lwIP를 이용한 TCP/IP 통신에 User Logic을 추가하는 과정을 설명합니다. PC에서 TCP/IP를 이용하여 명령어를 전송하면, lwIP를 통하여 명령어를 받아서 User Logic에서 보드의 LED를 제어하는 과정을 설명합니다. 이를 통하여 lwIP와 User Logic의 인터페이스를 구현하고 결과를 확인합니다.

9장은 v1.4에서 추가된 내용입니다. wiznet 사의 w5500 모듈을 이용하여 TCP/IP를 구현하는 내용입니다. PC와 Network으로 연결해서 데이터 송수신을 구현합니다. 이를 응용하면 TCP/IP를 이용하는 분야에 다양하게 적용할 수 있을 것입니다.

10-11장은 v1.5에서 새롭게 추가된 내용으로 Block Memory Interface를 구현하고, 이를 이용하여 User Logic Register Map을 구현합니다.

10장에서는 Block Design상에서 기본적으로 제공하는 Block Memory Interface를 구현합니다. 응용 sw에서 구현된 Memory Access를 구현합니다.

11장에서는 사용자 로직에서 Block Memory를 추가하여 Block Memory Interface를 구현합니다. 이를 응용하여 User Logic Register Map을 구현하고, pwm의 frequency, duty를 제어하는 예제를 구현합니다.

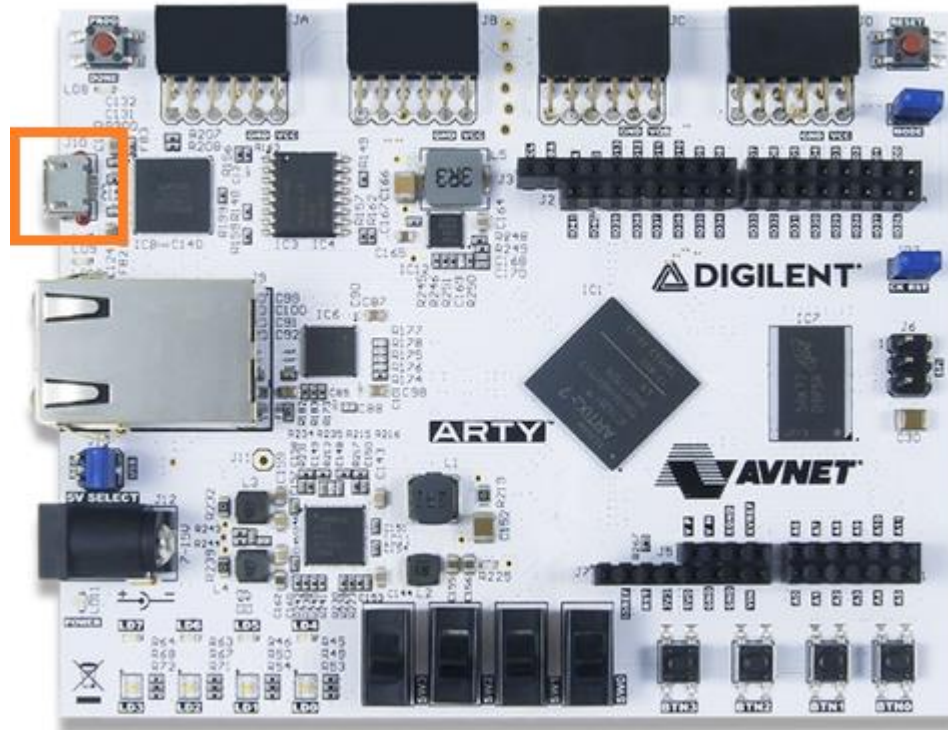
12장은 참고자료입니다.

이힐

2. HW 구성

본서에서 실습에 사용하는 보드는 Xilinx 사의 Arty A7-35T (100T도 가능)입니다. 보드와 USB 케이블(Micro-B) 만 있으면 모든 과정을 구현해 볼 수 있습니다. 전원 공급 및 프로그램 다운로드가 USB 케이블로 가능합니다.

[그림 2-1] Arty A7-35T



3. SW 설치

본서에서 사용하는 툴은 “Vitis 2022.1” 입니다. Vitis는 Xilinx사에서 2019년 발표한 통합 소프트웨어 개발 플랫폼입니다. Vitis 안에 HW 개발 툴 Vivado 와 SW 개발 툴 Vitis 가 포함되어 있습니다. MicroBlaze를 사용하기 위해서는 물론 이전 버전에서도 가능합니다. 저자는 주로 Vivado 2018.3 버전을 사용하여 개발을 하였습니다. Vivado 에서도 SW 개발을 위한 “SDK” 툴을 제공하기 때문에 MicroBlaze를 이용한 개발이 가능합니다. 그러나 본서를 통하여 MicroBlaze를 접하게 되는 분들에게 최신 툴을 사용하여 진행하는 것이 좋을 것이라 생각해서 “Vitis 2022.1” 버전을 설치하여 진행하게 되었습니다.

Vitis 툴은 용량이 어마무시합니다. 설치하는데 꼬박 하루는 투자하셔야 합니다. Default 로 선택된 툴들을 모두 설치하면 약 250G가 필요합니다. 저는 UltraScale 관련된 부분들은 빼고 설치를 진행하였는데 그래도 156G 입니다. 아래는 제 PC에서 설치에 소요된 시간을 대략적으로 나타내었습니다. 참고하시기 바랍니다.

제가 가진 PC 사양입니다.

프로세서	Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz	3.00 GHz
설치된 RAM	16.0GB(15.9GB 사용 가능)	

설치과정

1) 설치 파일 다운로드

- ✓ 용량 : 약 210 M, 수분 소요

2) 프로그램 다운로드

- ✓ 용량 : 50 G, 1시간 13분 소요

3) 프로그램 설치

- ✓ 용량 : 156 G, 1시간 41분 소요

4) Final Processing... (Optimize Disk Usage)

- ✓ 소요시간 측정 불가, 제가 2시간 넘게 기다리다가 포기하고 PC를 켜둔 채 퇴근하고(절전모드 OFF) 다음날 아침에 이어서 진행하였습니다. 뭔가 문제가 생겼나 싶어서 인터넷 찾아보니, “시간이 엄청 많이 소요됩니다. 결국은 끝납니다” 이런 내용들이었고, 실제로 xilinx 웹사이트에도 답변이 이렇게 되어 있습니다 “The time it takes to optimize disk space usage could take longer than expected, however if left running it will eventually complete.” Xilinx 사에서는 이 과정을 진행하면 디스크 용량을 20~30% Save 할 수 있다고 합니다. 설치과정에 생성된 불필요한 파일들을 삭제하는 것으로 보입니다. 아무튼 Vitis 를 설치하기 위해서는 인내심을 가지시고, 마지막 단계(Final Processing)가 나오면 퇴근하시고 다음날 마무리 작업(10분정도) 하시는 것을 추천해 드립니다.

5) 마지막으로 기존에 사용하던 툴들과의 호환성 문제입니다. 이 부분은 제 경험에 근거한 내용입니다. 혹 다른 결과가 나올 수 있음을 주의하시길 바랍니다. 저는 수년동안 “Vivado 2018.3”을 사용하여 프로젝트를 진행했습니다. 툴을 바꾸게 되면 이전에 진행했던 프로젝트들이 문제를 일으킬 수도 있습니다. 물론 “Vivado 2018.3” 버전에서도 MicroBlaze를 사용할 수 있습니다. 그러나 전자문서를 만들려고 하면서 고민을 많이 하였습니다. 이왕이며 최신 버전을 사용하여 진행하

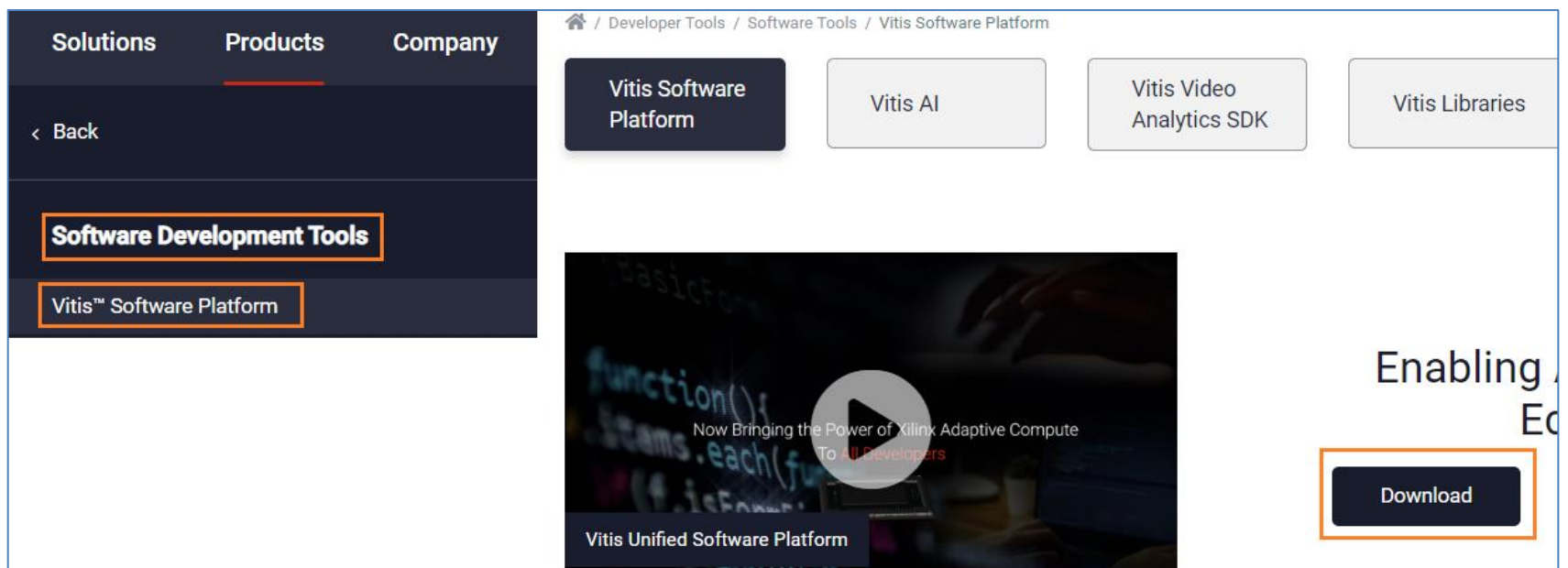
는 것이 이 문서를 통하여 MicroBlaze를 접하게 되는 분들에게 도움이 될 것이라 생각하였습니다. 그래서 “Vitis 2022.1”을 설치해서 진행하기로 하였습니다. 제 PC는 C(SSD, 512G), D(HDD, 2T) 2개의 드라이브가 있습니다. “Vivado 2018.3”은 C 드라이브에 설치되어 있습니다. 저는 여러가지 요인으로 인해서 “Vitis 2022.1”을 D 드라이브에 설치하기로 하였습니다. 대신 기존에 C 드라이브에 설치되어 있던 “Vivado 2018.3”은 그대로 두고 설치를 진행하였습니다. 결론적으로 “Vivado 2018.3”, “Vitis 2022.1” 버전 모두 충돌없이 잘 사용하고 있습니다. 혹 저와 같이 두개의 버전을 사용해야 하는 분들은 드라이브를 나누어서 설치하면 충돌없이 사용할 수 있을 것으로 생각합니다.

본격적으로 “Vitis 2022.1”을 설치하도록 하겠습니다.

Xilinx 웹사이트에서 Product - Software Development - Vitis Software Platform 을 선택합니다.

(링크 : <https://www.xilinx.com/products/design-tools/vitis/vitis-platform.html>)

다운로드를 클릭하면 다운로드 사이트로 이동합니다.



버전 “2022.1”을 확인하고 화면을 아래로 스크롤해서 “Xilinx Unified Installer 2022.1 : Windows ~” 을 클릭해서 설치 파일을 다운로드 합니다. Xilinx 계정이 없으면 생성해서 로그인을 하면 설치파일을 다운로드 받을 수 있습니다. Xilinx 의 많은 자료들이 로그인 정보를 요청하기 때문에 계정을 생성하시는 것이 좋습니다.

Version

2022.1
2021.2
2021.1
Vitis Archive
SDSoC Archive
SDAccel Archive
SDK/PetaLinux Archive

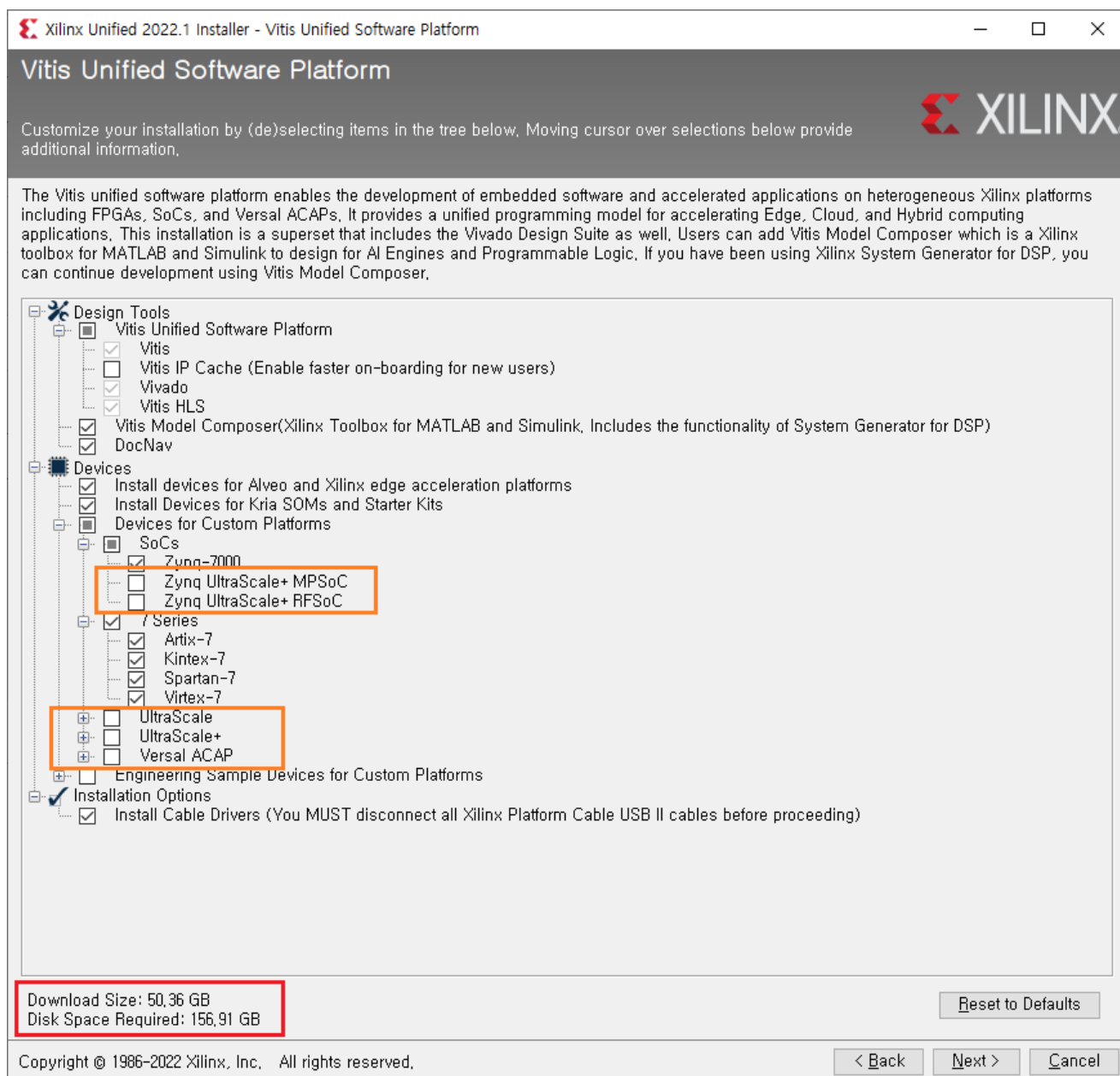
MD5 SUM Value : 76a6221ea8eed8c635c654cc100a1cae

Download Verification

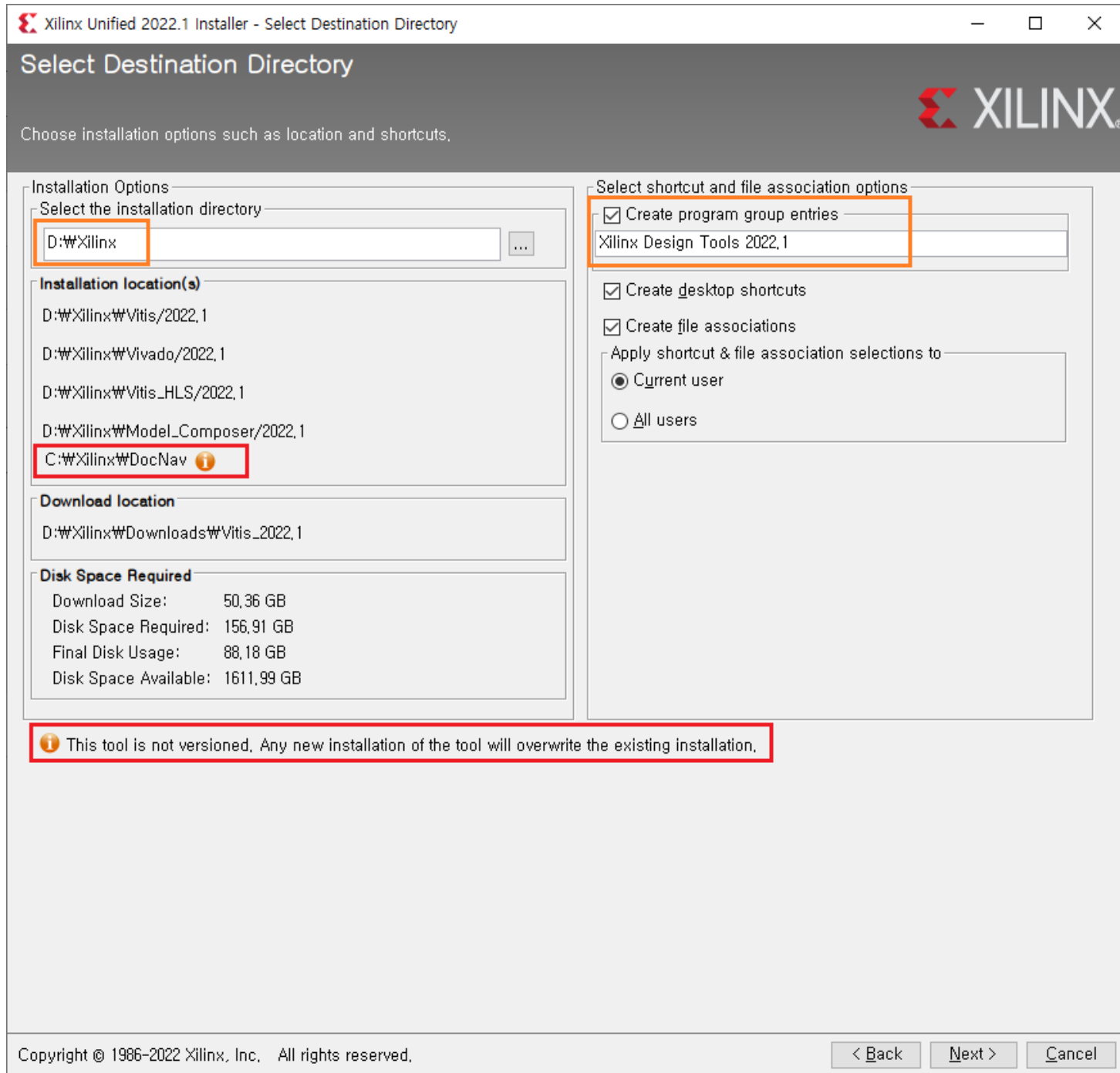
Digests
Signature
Public Key

다운로드가 다 되면 설치 파일을 실행해서 설치를 진행합니다. 여기서는 몇가지 중요한 부분만 설명하도록 하겠습니다.

- 1) 설치 파일 선택 : 용량과 시간을 절약하기 위해서 UltraScale 이상은 빼고 설치하는 것을 추천합니다. 필요하면 나중에 추가로 설치하면 됩니다.



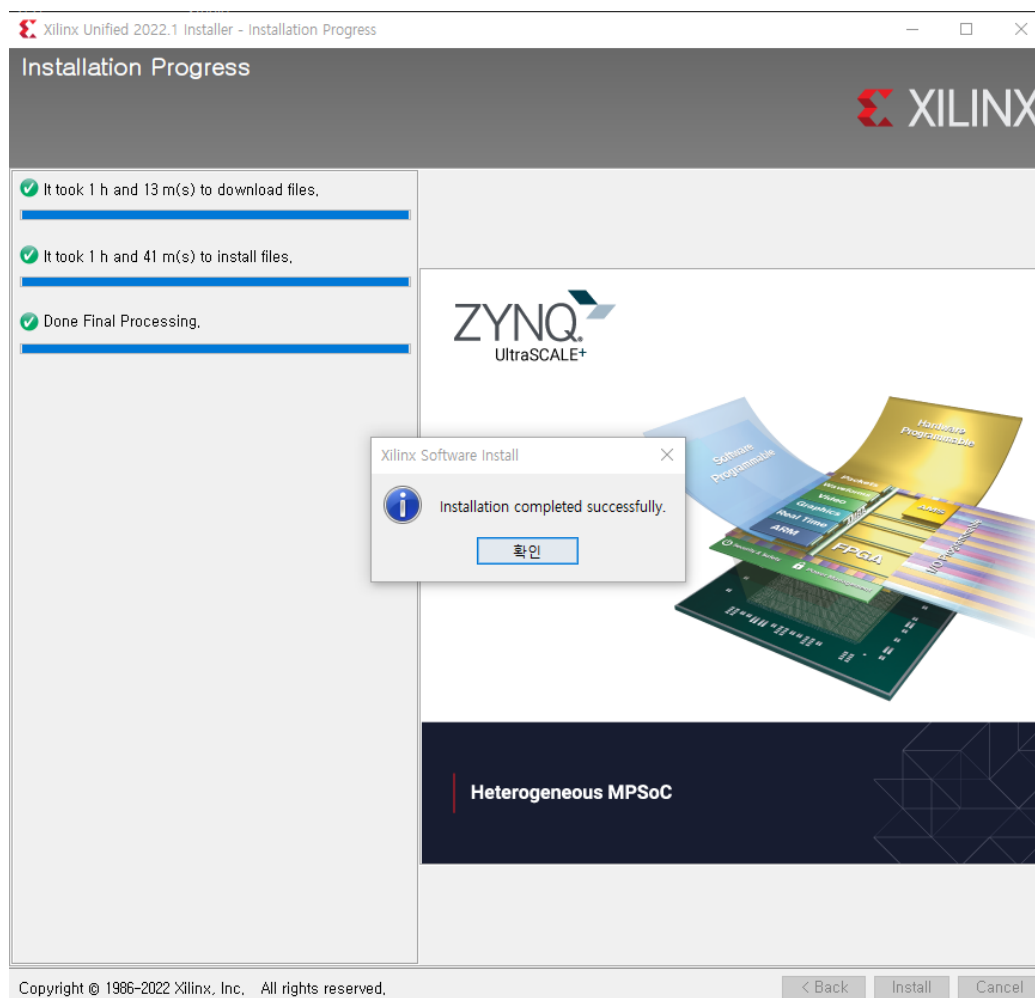
- 2) D 드라이브에 설치를 진행할 경우 참조하시길 바랍니다. 설치 폴더를 “D:\Xilinx” 로 하고, Program group 을 “Xilinx Design Tools 2022.1” 로 합니다. 이전 버전이 설치되어 있는 경우 “Xilinx Design Tools”로 되어 있어서 이를 구분하기 위해서 다른 이름으로 설정합니다. “DocNav” 파일은 D 드라이브로 잡히지 않고, C 드라이브에 설치됩니다. 이전버전의 데이터가 Overwrite 될 수 있다는 경고입니다. 무시하시고 진행하시면 됩니다.



- 3) 설치 과정입니다. 필요한 파일들을 다운로드하고, 압축파일을 풀어서 설치를 진행하고 마지막으로 “Final Processing...” 을 진행합니다. 마지막 단계에서는 진행바가 진행하지 않고 몇시간을 그대로 있습니다. 그냥 놔두고 퇴근해서 다음날 나머지 작업을 진행하시는 것을 권장합니다. 퇴근하실 때 PC의 절전모드를 반드시 OFF 하셔야 PC가 꺼지지 않고 진행하고 다음날 나머지 작업을 진행할 수 있습니다.



4) 마지막으로 드라이버를 설치하면 완료됩니다.



다음장부터는 MicroBlaze를 구현하는 것을 설명합니다.

4. Hello World 구현

이번 장에서는 가장 기본적인 MicroBlaze를 이용하여 Hello world 를 화면에 출력하고, 간단한 사용자 로직으로 LED를 On/Off 하는 것을 구현합니다. 또한 생성된 파일을 보드에 다운로드 하는 3가지 방법에 대해 설명합니다.

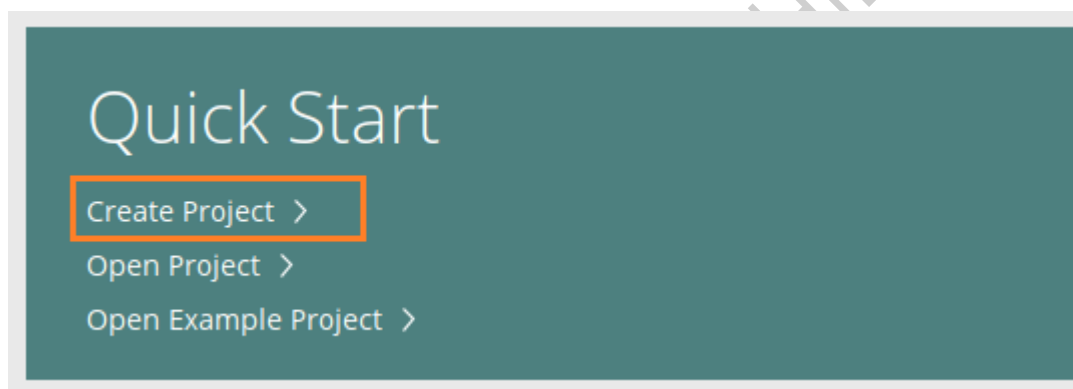
MicroBlaze는 Processor Core와 Peripheral이 분리되어 있습니다. 사용자는 원하는 Peripheral을 추가할 수 있고, 갯수도 필요한 만큼 추가하여 사용할 수 있습니다. 이번 장에서는 MicroBlaze Processor Core와 Peripheral 중에 Uart 를 추가하여 Uart 포트를 통하여 Hello world 를 출력하는 기능을 구현합니다. 이번장에서는 대략적인 흐름을 파악하는 것에 중점을 두시길 바랍니다. 처음에 툴을 사용하다 보면 에러도 많이 발생하고 익숙하지 않아 어렵게 느껴질 수도 있지만, 몇번 반복하다 보면 자연스럽게 툴에 익숙해지고 에러가 발생해도 바로 수정할 수 있게 됩니다. 마음에 여유를 가지시길 바랍니다~

4.1 기본 Template 구현

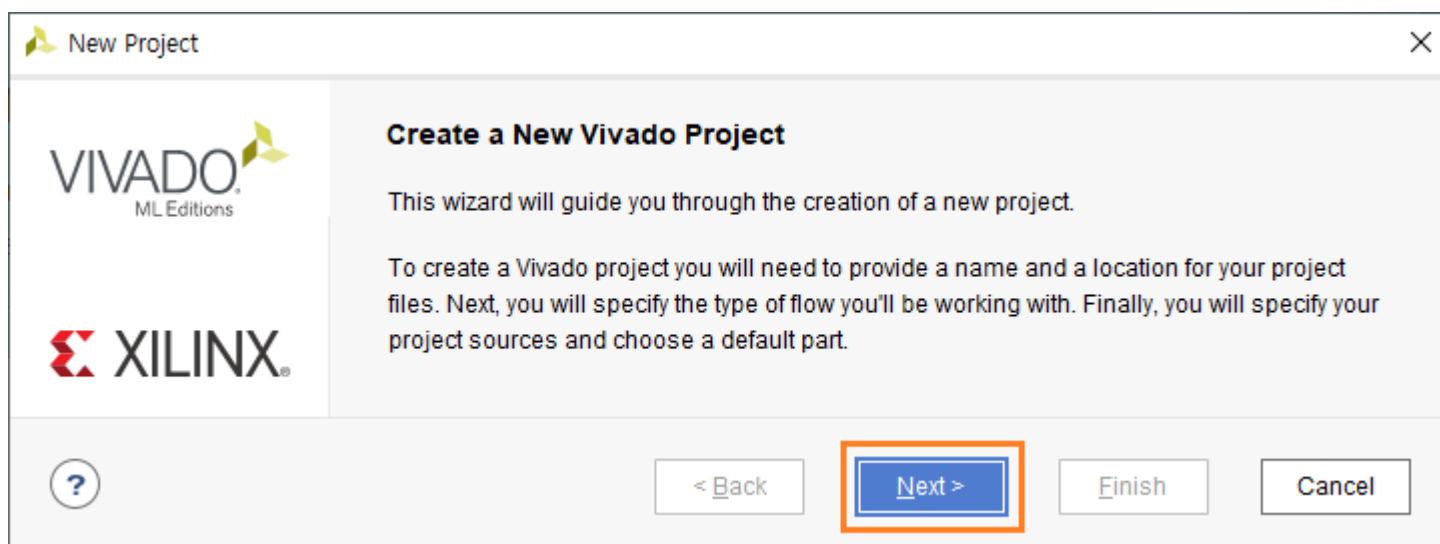
4.1.1 프로젝트 생성

vivado 2022.1에서 MicroBlaze를 사용할 때에는 반복적인 작업이 많이 있습니다. 매번 프로젝트를 생성할 때마다 반복적으로 하는 작업들이 있습니다. 이를 좀 더 간편하게 하기 위하여 공통적으로 진행되는 것을 미리 만들어 두고 다음번에는 복사해서 사용하도록 하겠습니다. 이를 위하여 이번 장에서는 기본 Template을 구현합니다.

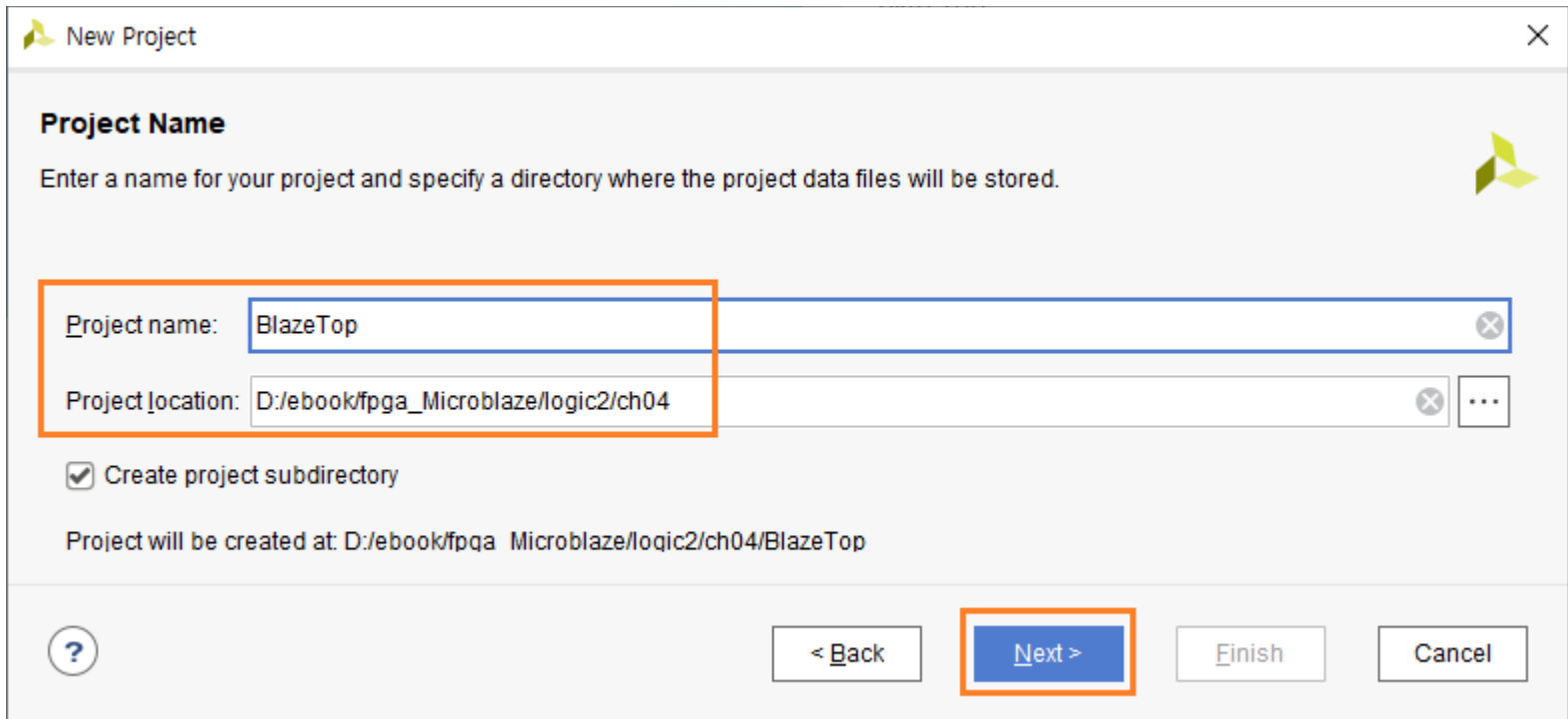
Vivado 2022.1을 실행하고, “Create Project”를 클릭합니다.



Next를 클릭합니다.



프로젝트 위치를 지정하고, 프로젝트 이름을 입력합니다. “BlazeTop”으로 하겠습니다. “Create project subdirectory”항목을 선택해서 새로운 폴더를 생성하도록 합니다. Next를 클릭합니다.



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

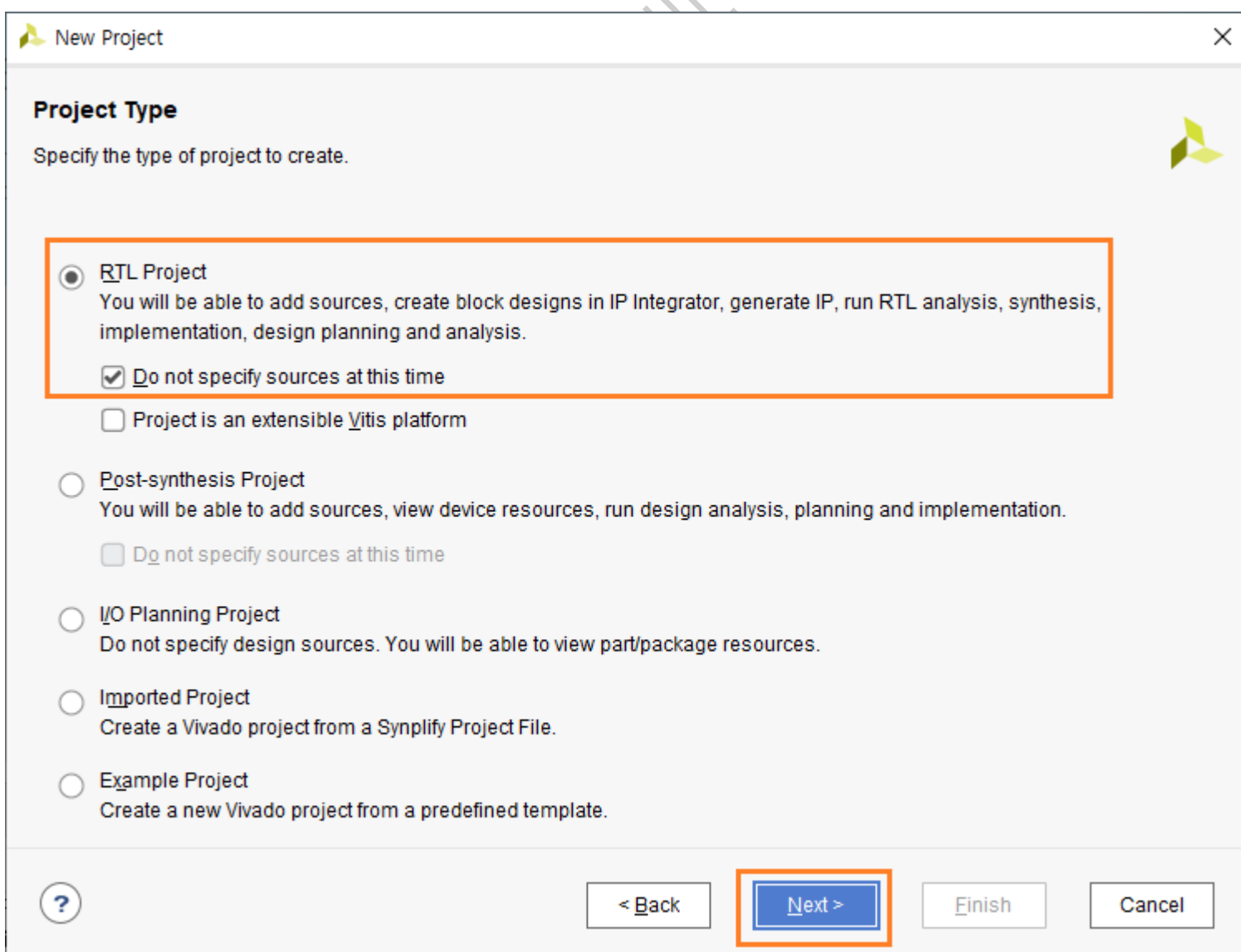
Project name:

Project location:

☒ Create project subdirectory

Project will be created at: D:/ebook/fpga_Microblaze/logic2/ch04/BlazeTop

프로젝트 타입은 RTL Project를 선택합니다. “Do not specify sources at this time”항목을 선택하면 소스 추가 윈도가 나타나지 않습니다. 소스추가는 나중에 진행합니다. Next를 클릭합니다.



New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time

☐ Project is an extensible Vitis platform

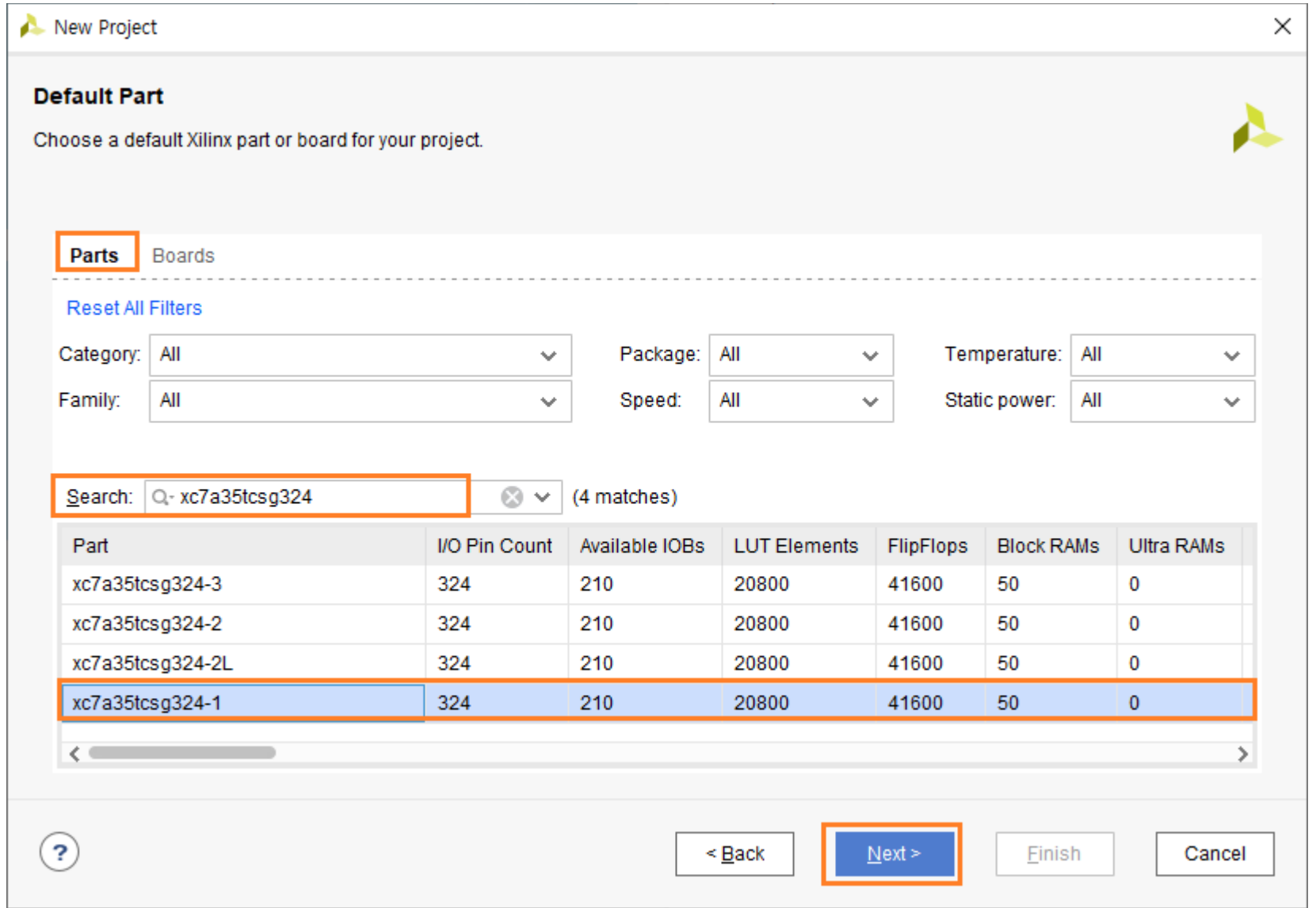
☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

Part를 선택합니다. Parts 탭을 선택하고, Search : xc7a35tcsg324 을 입력하고, 리스트 중에 “xc7a35tcsg324-1”을 선택하고 Next를 클릭합니다.



New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts Boards

[Reset All Filters](#)

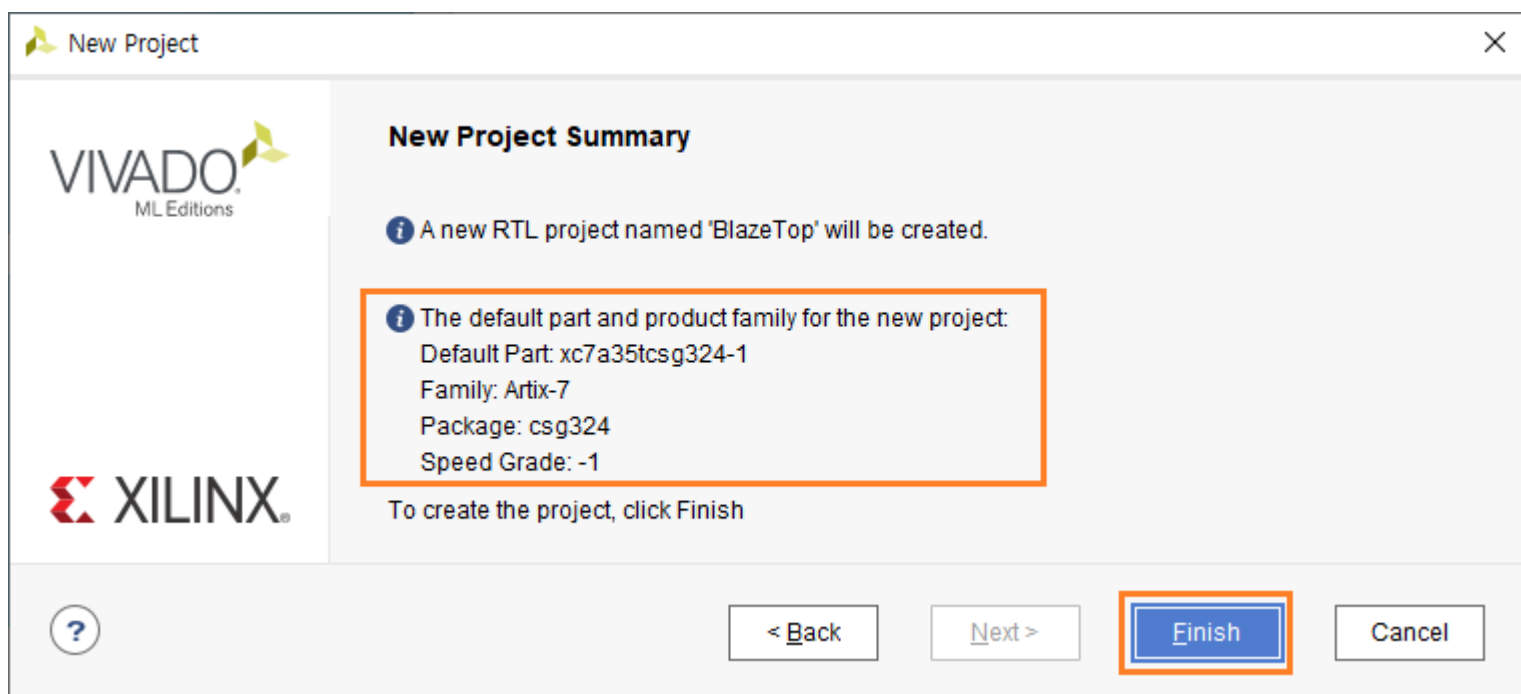
Category: All Package: All Temperature: All
Family: All Speed: All Static power: All

Search: Q- xc7a35tcsg324 (4 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs
xc7a35tcsg324-3	324	210	20800	41600	50	0
xc7a35tcsg324-2	324	210	20800	41600	50	0
xc7a35tcsg324-2L	324	210	20800	41600	50	0
xc7a35tcsg324-1	324	210	20800	41600	50	0

< Back Next > Finish Cancel

설정된 내용을 확인하고 Finish를 클릭하면 프로젝트가 생성됩니다.



New Project

VIVADO
ML Editions

XILINX

New Project Summary

i A new RTL project named 'BlazeTop' will be created.

i The default part and product family for the new project:
Default Part: xc7a35tcsg324-1
Family: Artix-7
Package: csg324
Speed Grade: -1

To create the project, click Finish

< Back Next > Finish Cancel

4.1.2 Create Block Design

프로젝트가 생성되었습니다. Flow Navigator 윈도우에서 IP INTEGRATOR - Create Block Design을 클릭합니다.



Design name : system 으로 입력하고 OK를 클릭하면 Diagram 윈도우가 생성됩니다.

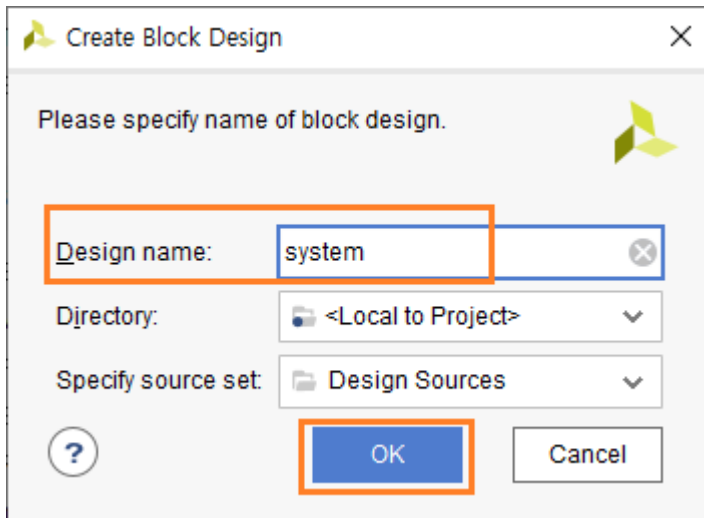


Diagram 윈도우에서 Processor를 추가하기 위하여 (+) 아이콘을 클릭합니다. Search : microblaze를 입력하고 나오는 리스트 중에서 MicroBlaze 을 더블 클릭해서 Processor를 추가합니다.

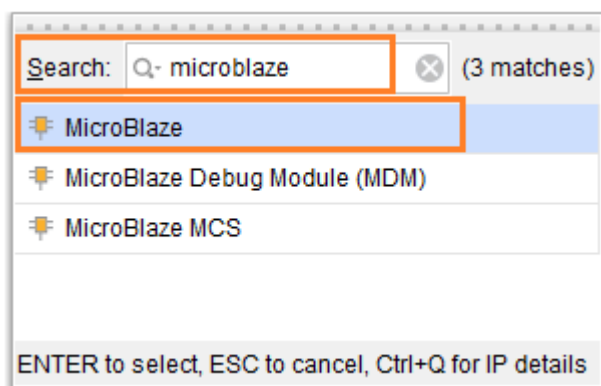
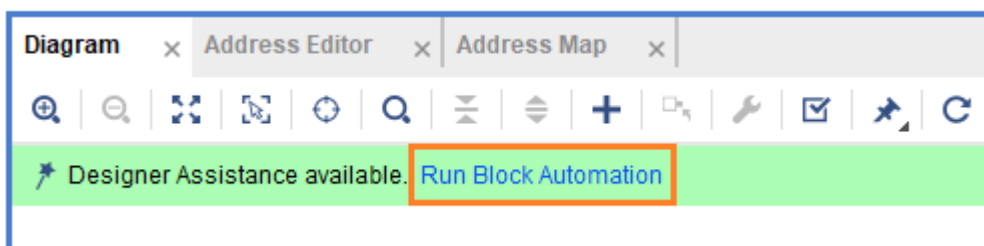
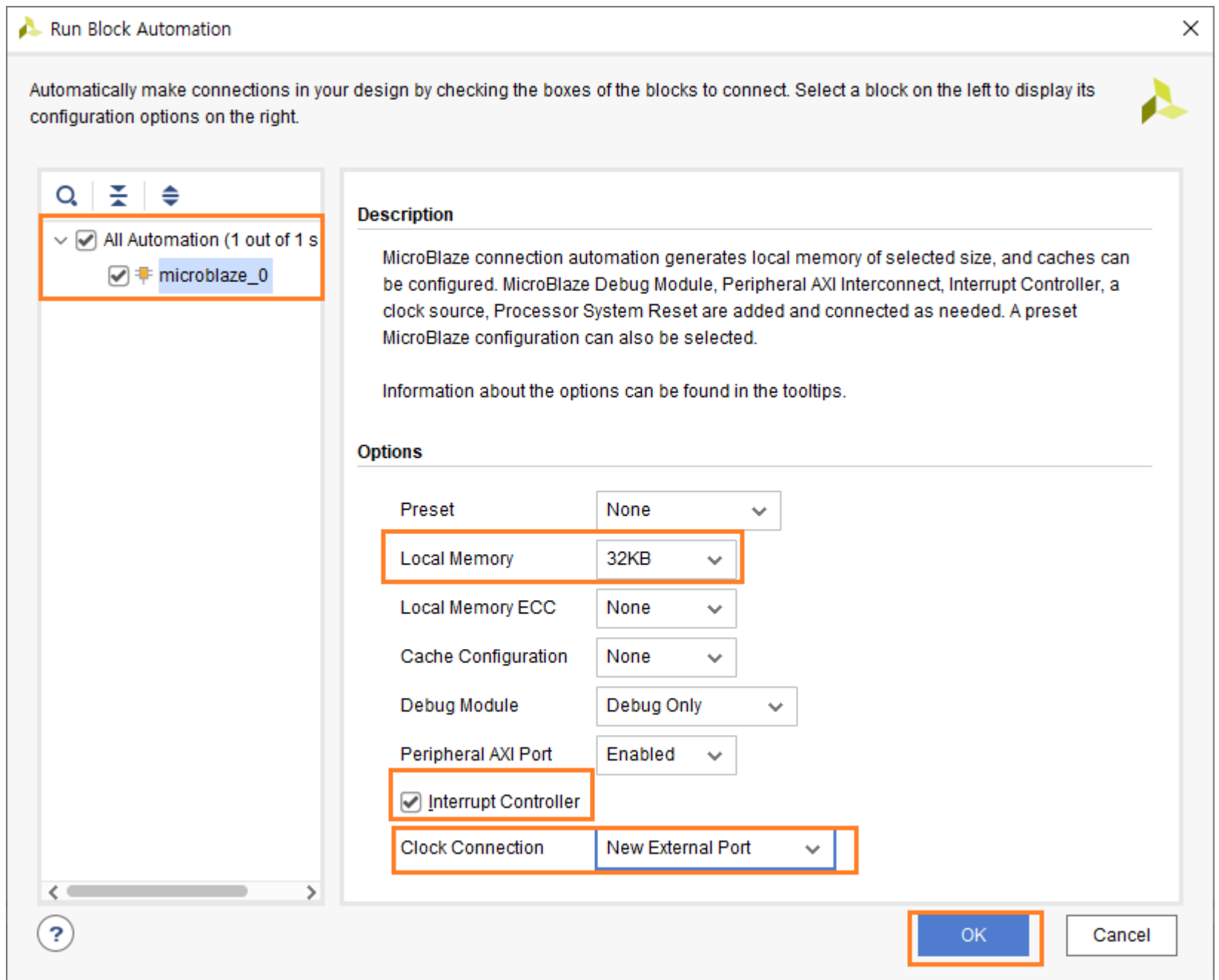


Diagram 윈도우 상단의 “Run Block Automation”을 클릭합니다.

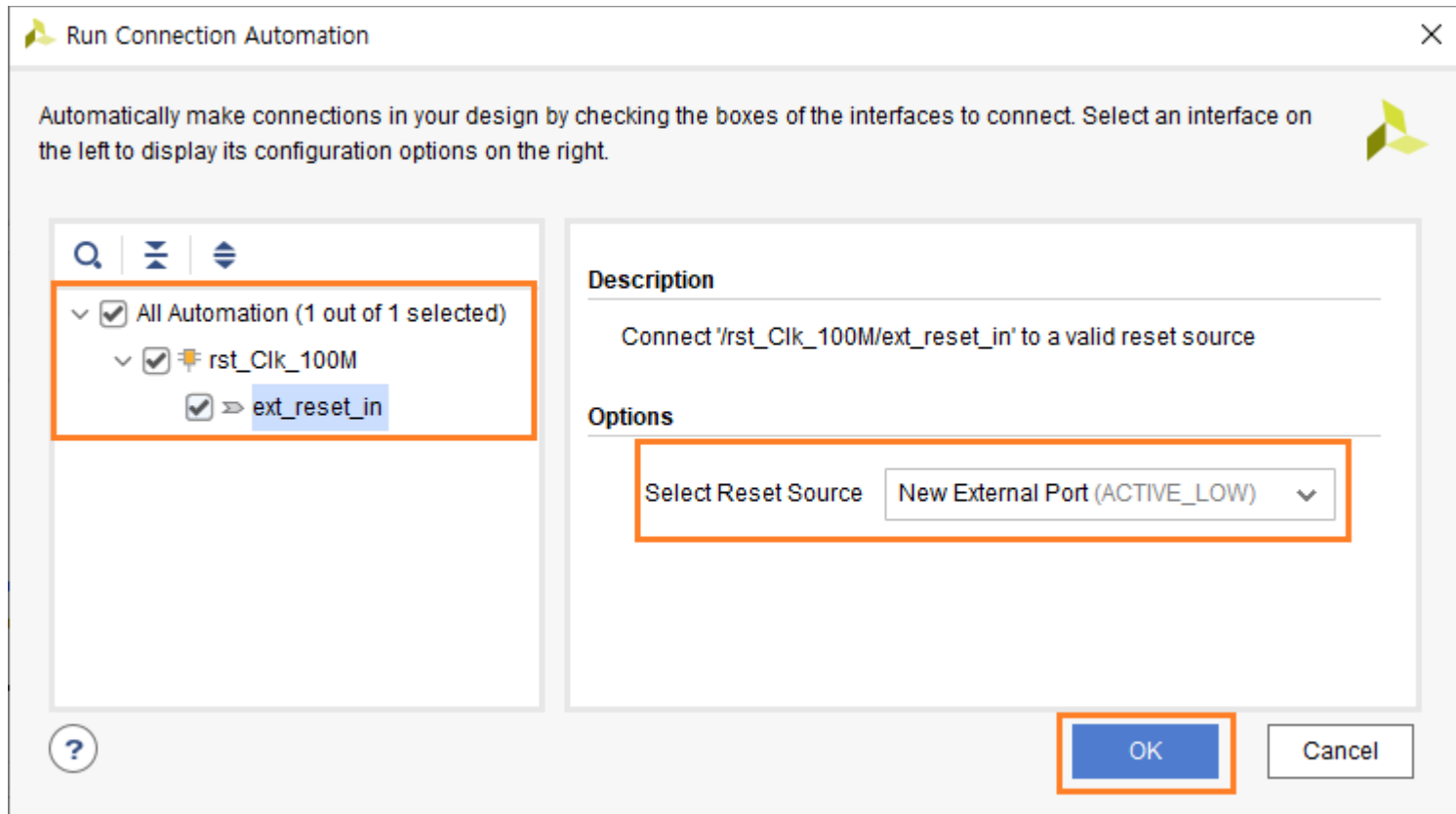


Run Block Automation 윈도우에서 속성을 아래와 같이 설정하고 OK 를 클릭합니다.

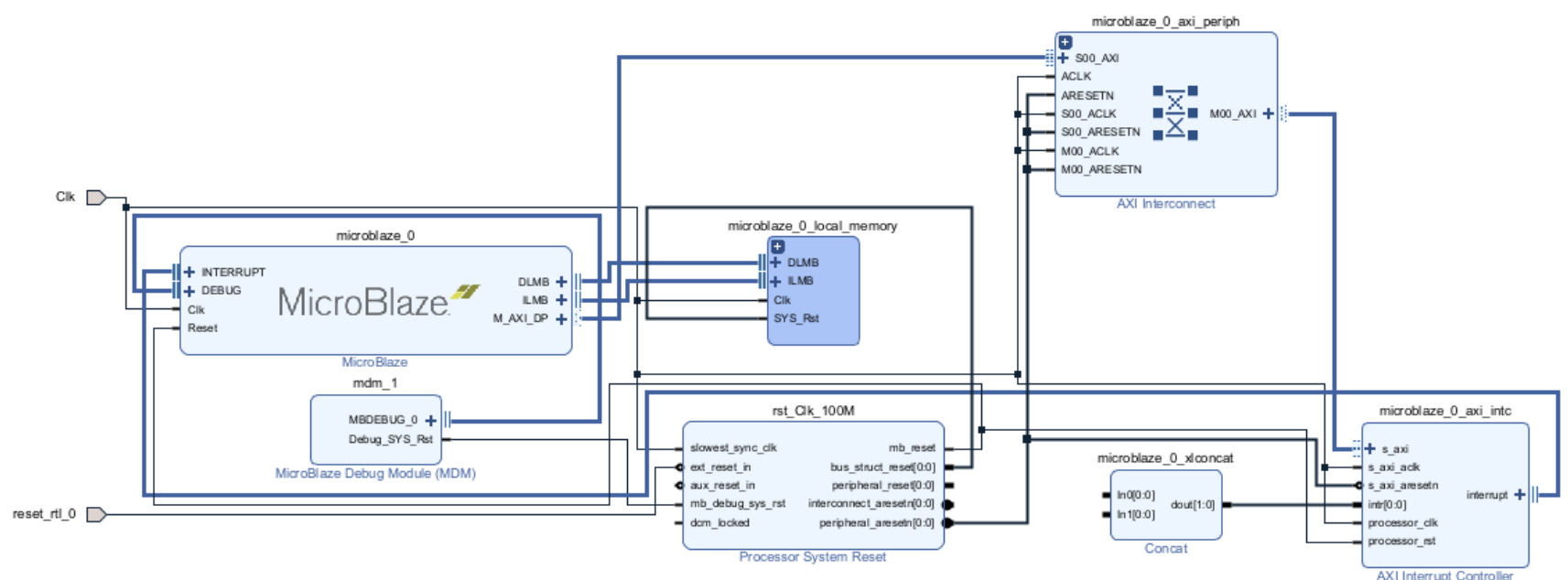
- ✓ Local Memory : 32KB, 프로그램 메모리를 나타냅니다. 기본값은 8KB 이지만, 32KB로 변경합니다.
- ✓ Interrupt Controller : checked, Interrupt Controller를 사용하기 위하여 선택합니다.
- ✓ Clock Connection : New External Port, 기본값은 New Clock Wizard로 설정되어 있지만, 보드에 있는 100Mhz clock을 그대로 사용하기 위하여 New External Port를 선택합니다.



설정된 내용대로 메모리와 Interrupt 등 기본적인 Block 들이 추가됩니다. 그러나 아직 연결해야 할 부분들이 남아 있습니다. Diagram 상단에 Run Connection Automation 이 활성화 됩니다. Run Connection Automation을 클릭합니다.

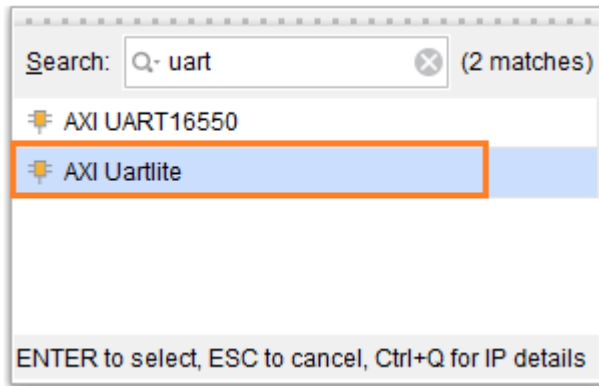


Run Connection Automation 윈도우에서는 왼쪽에는 아직 연결되지 않은 Block을 나타내고 해당 핀을 선택하면 오른쪽에 속성을 설정할 수 있습니다. rst_Clk_100M Block의 ext_reset_in 핀이 연결되지 않았음을 알 수 있습니다. ext_reset_in 핀을 선택하고 오른쪽에서 속성을 설정합니다. 보드에 있는 RESET 핀은 Active Low 이므로, “New External Port (Active LOW)”를 선택하고 OK를 클릭합니다. 아래는 지금까지 생성된 다이어그램을 보여줍니다.

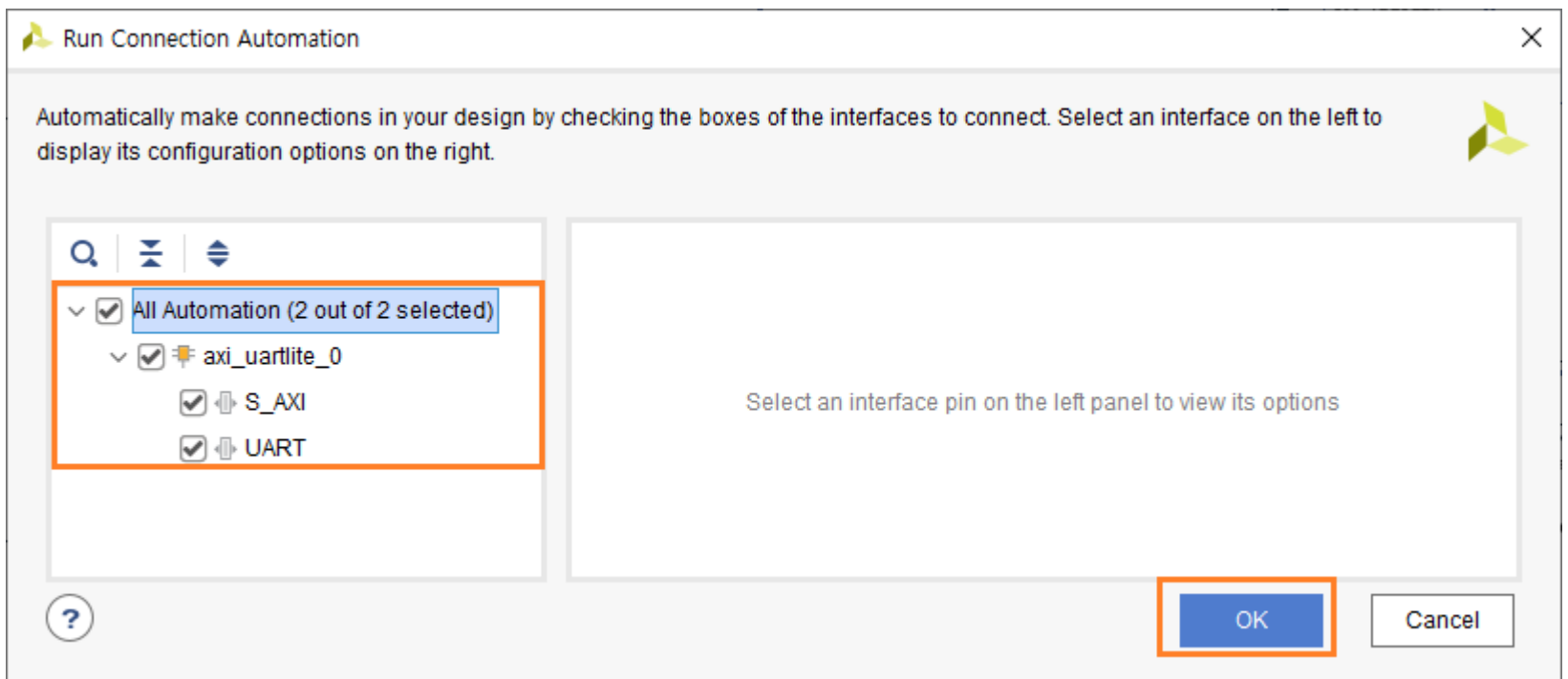


Clk, reset_rtl_0 포트가 추가된 것을 알 수 있습니다.

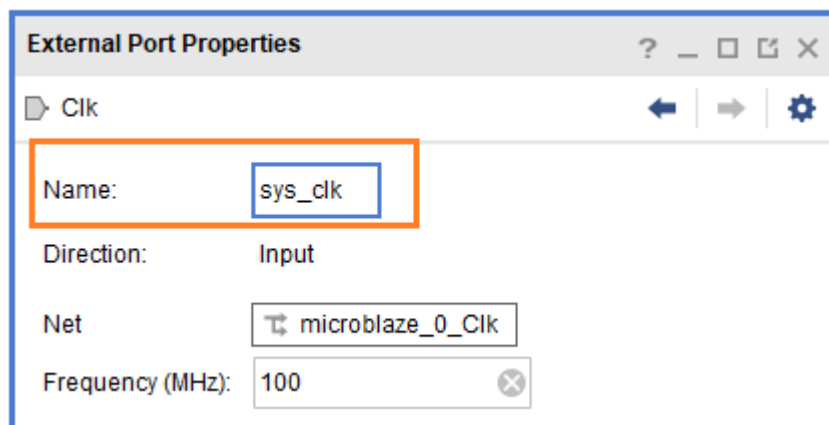
Add IP 아이콘 (+)을 클릭해서 UART Block을 추가합니다. uart 입력 후 나오는 IP 중에 “AXI Uartlite”를 Double Click 해서 추가합니다.



“Run Connection Automation”을 클릭해서 연결해 줍니다. All Automation을 선택해서 모든 Block을 선택하고 OK를 클릭합니다.

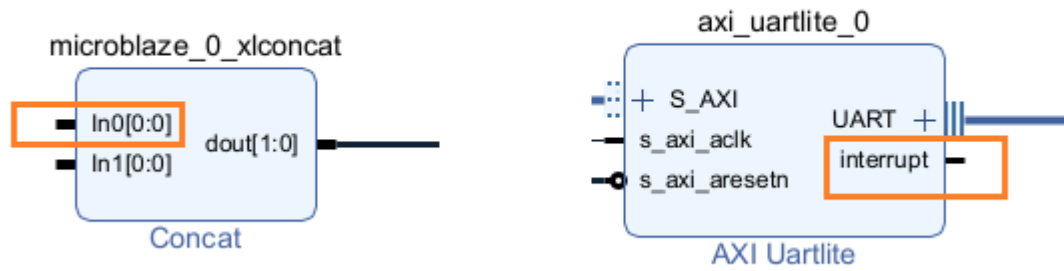


Port 이름을 아래와 같이 변경합니다. 해당 포트를 선택하면 Diagram 오른쪽에 해당 핀의 속성이 나타납니다.

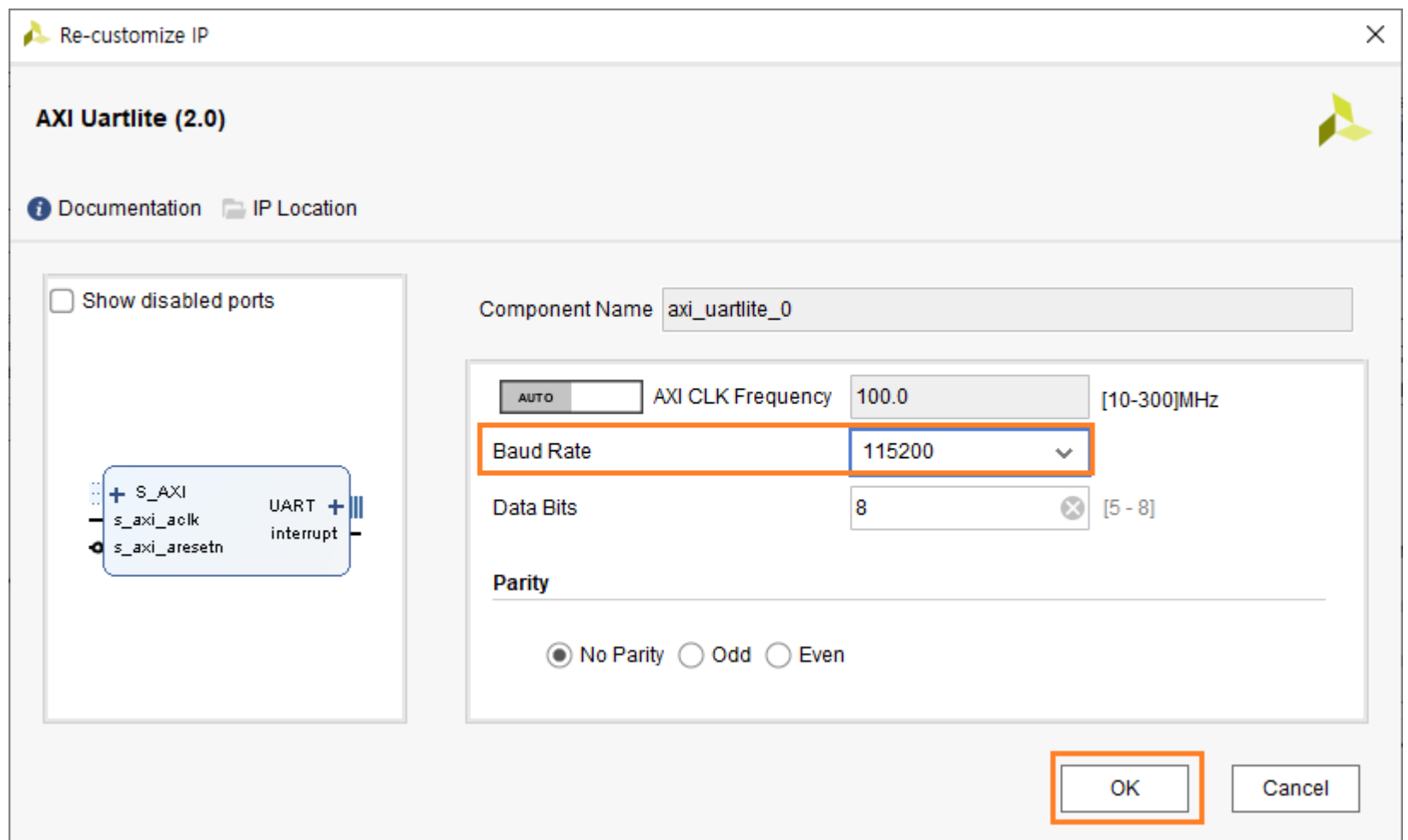


3개의 Port 이름을 변경합니다. Clk -> sys_clk, reset_rtl_0 -> reset, uart_rtl_0 -> uart

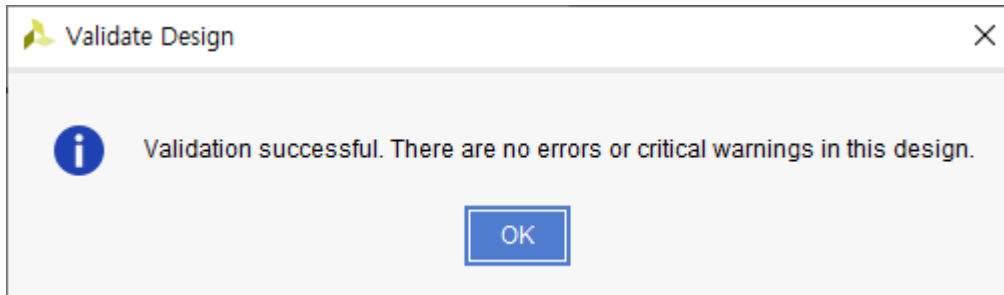
Interrupt 핀은 사용자가 수동으로 연결해야 합니다. “microblaze_0_xlconcat” block의 In0[0:0] 핀과 “axi_uartlite_0” block의 Interrupt 핀을 연결합니다. 해당 핀으로 마우스를 이동으로 연핀 모양으로 아이콘이 변경됩니다. 해당 핀을 클릭하고 드래그 해서 연결할 핀으로 이동하면 연결됩니다.



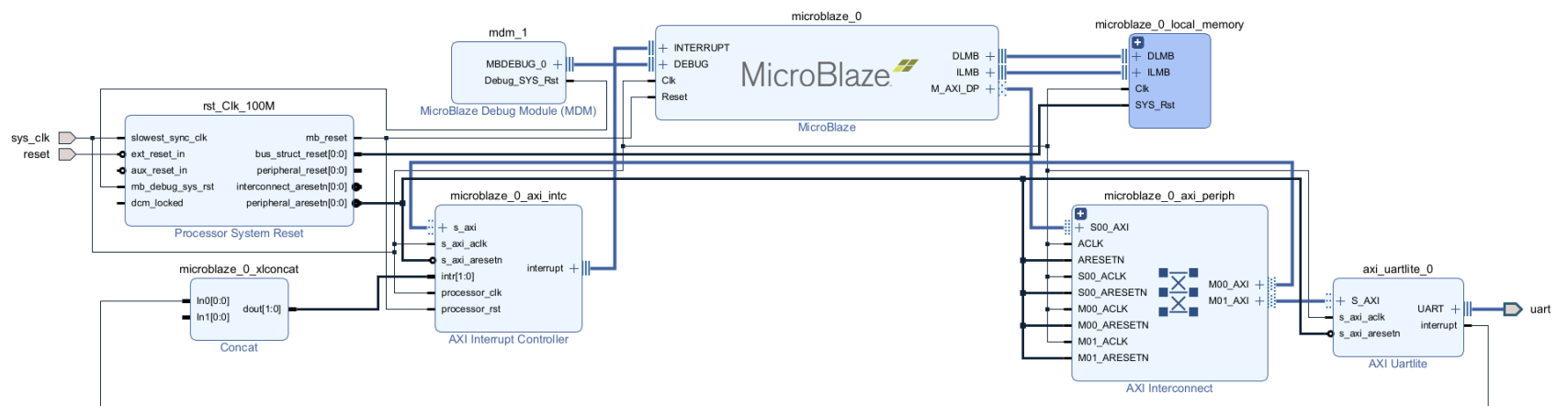
uart block의 buadrate를 수정합니다. axi_uartlite_0 block을 더블 클릭하면 속성창이 나타납니다. 아래와 같이 Baud Rate을 115200으로 수정하고 OK를 클릭합니다.



Block Design이 완료되었습니다. Block Design이 완료되면 에러가 없는지 확인을 해야 합니다. 상단 아이콘 중에 Validate Design () 을 클릭합니다. 에러가 없으면 아래와 같은 윈도가 나타납니다. 혹 에러가 발생하면 자료실에 있는 Diagram 파일과 비교하여 에러를 수정하고 다시 에러를 확인하시길 바랍니다.

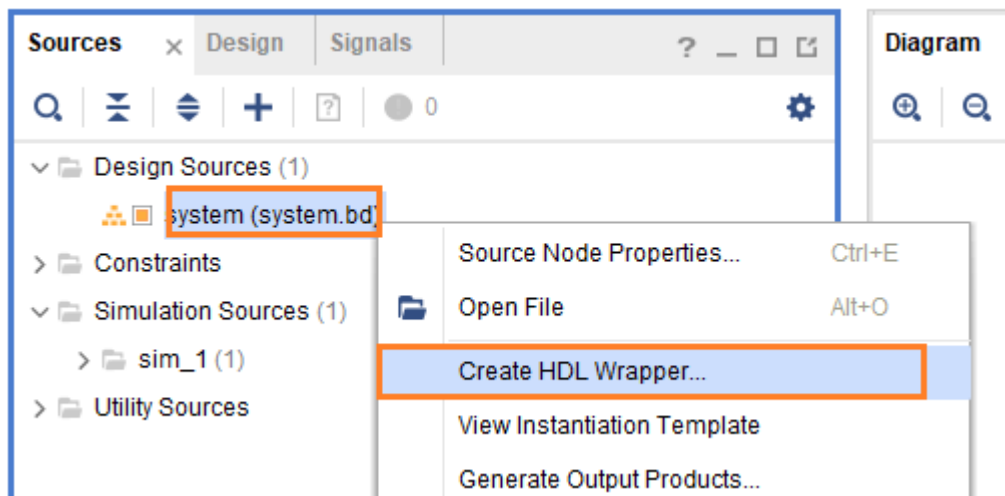


OK를 클릭하고, Regenerate Layout() 아이콘을 클릭하면 보기 좋게 Layout을 변경해 줍니다. Optimize Routing ()을 클릭해도 됩니다. 아래 그림은 지금까지 생성된 Block Design을 보여줍니다. (자료실의 diagram_ch4.1.png 파일을 참조하세요)

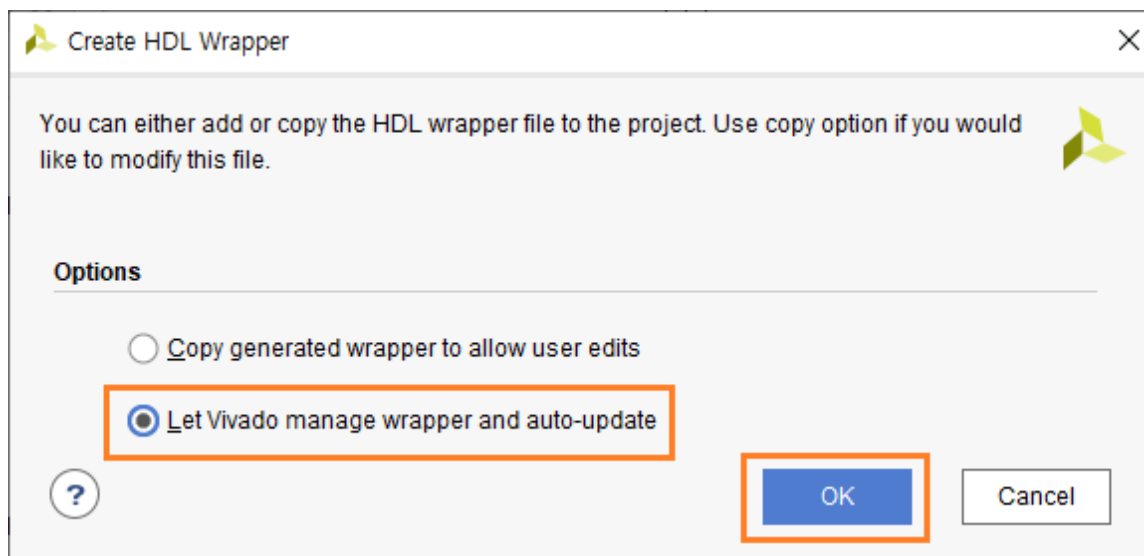


4.1.3 Create HDL Wrapper

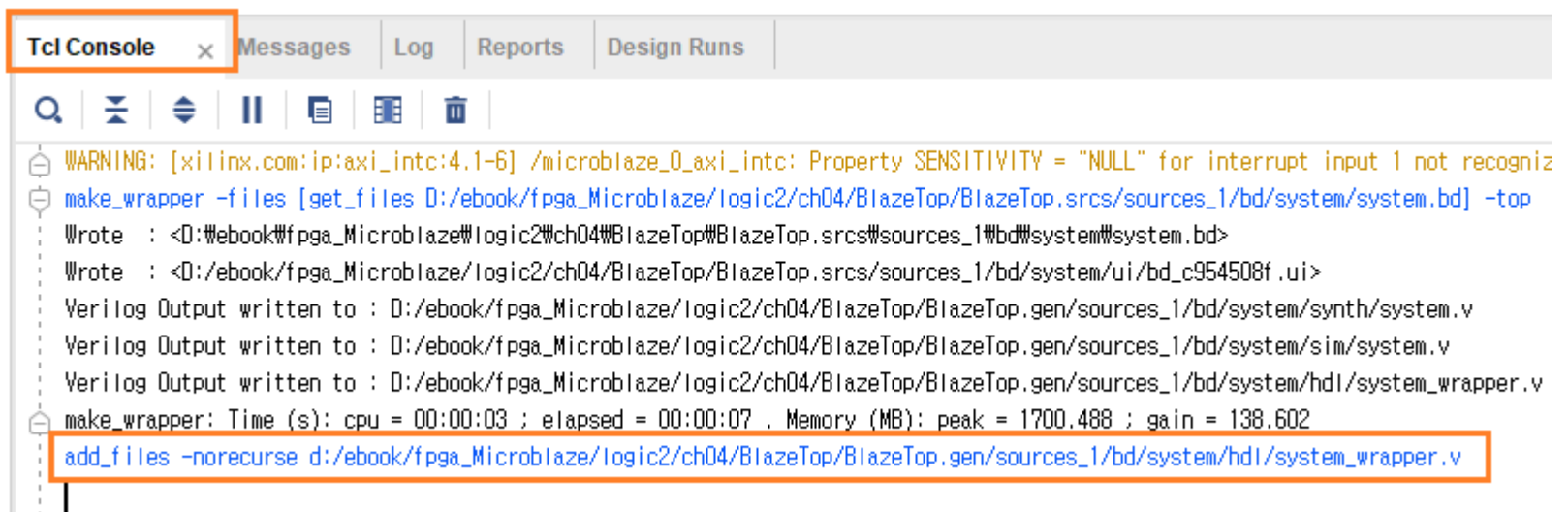
Block Design이 완료되었습니다. 이를 코드로 변환해 주어야 합니다. Design Sources - system - 우클릭 - Create HDL Wrapper... 를 클릭합니다.



“Let Vivado manage wrapper and auto-update”를 선택하고 OK를 클릭합니다.



system_wrapper.v 파일이 생성되었습니다.



아래는 system_wrapper.v 파일을 보여줍니다.

```

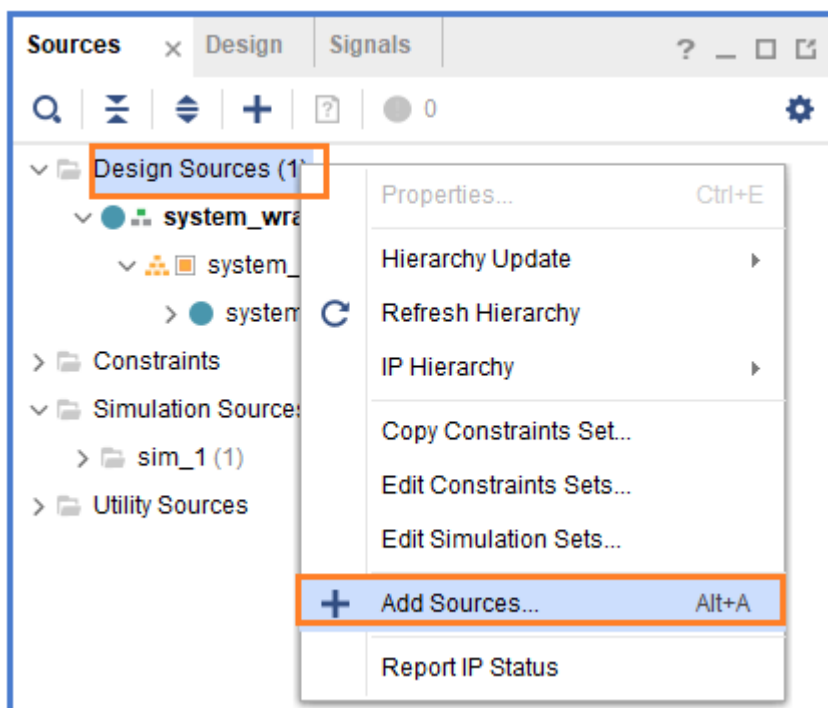
12 module system_wrapper
13     (reset,
14      sys_clk,
15      uart_rxd,
16      uart_txd);
17 input reset;
18 input sys_clk;
19 input uart_rxd;
20 output uart_txd;
21
22 wire reset;
23 wire sys_clk;
24 wire uart_rxd;
25 wire uart_txd;
26
27 system system_i
28     (.reset(reset),
29     .sys_clk(sys_clk),
30     .uart_rxd(uart_rxd),
31     .uart_txd(uart_txd));
32 endmodule

```

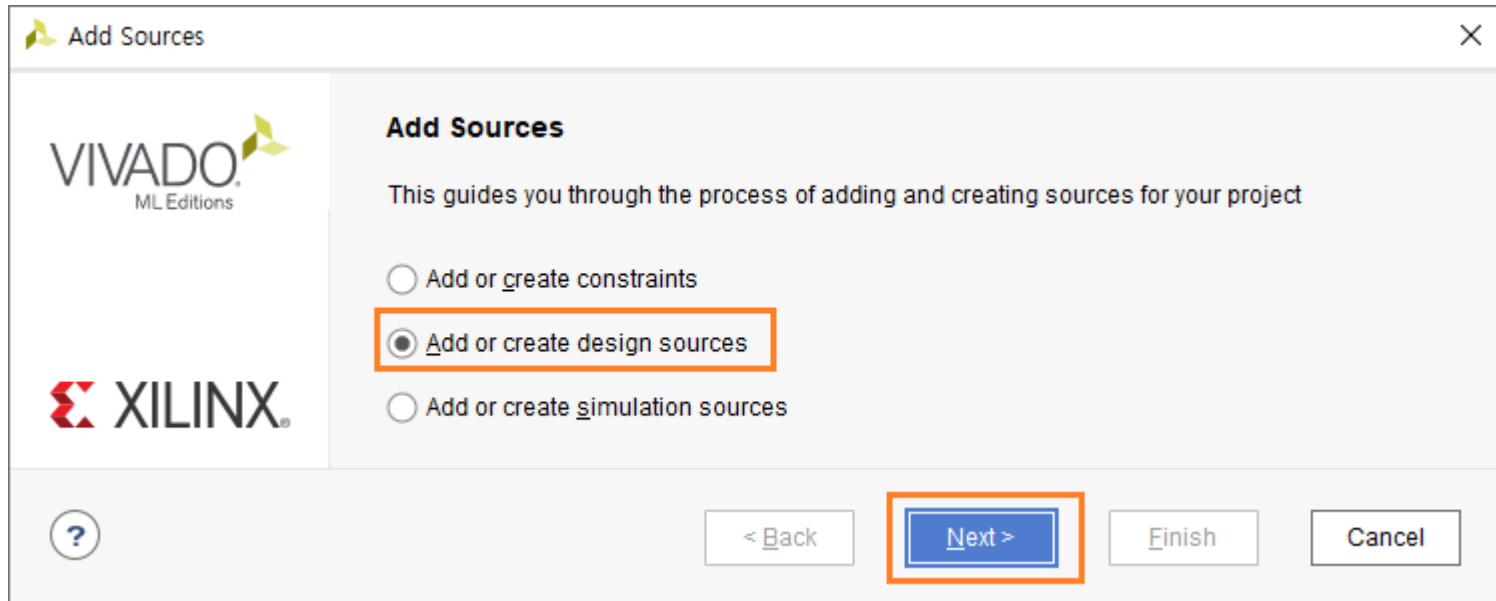
사용자가 추가한 포트 (reset, sys_clk, uart_rxd, uart_txd) 가 있고, 내부에 Block Design에서 구현한 system 모듈이 있습니다.

4.1.4 User Logic 구현

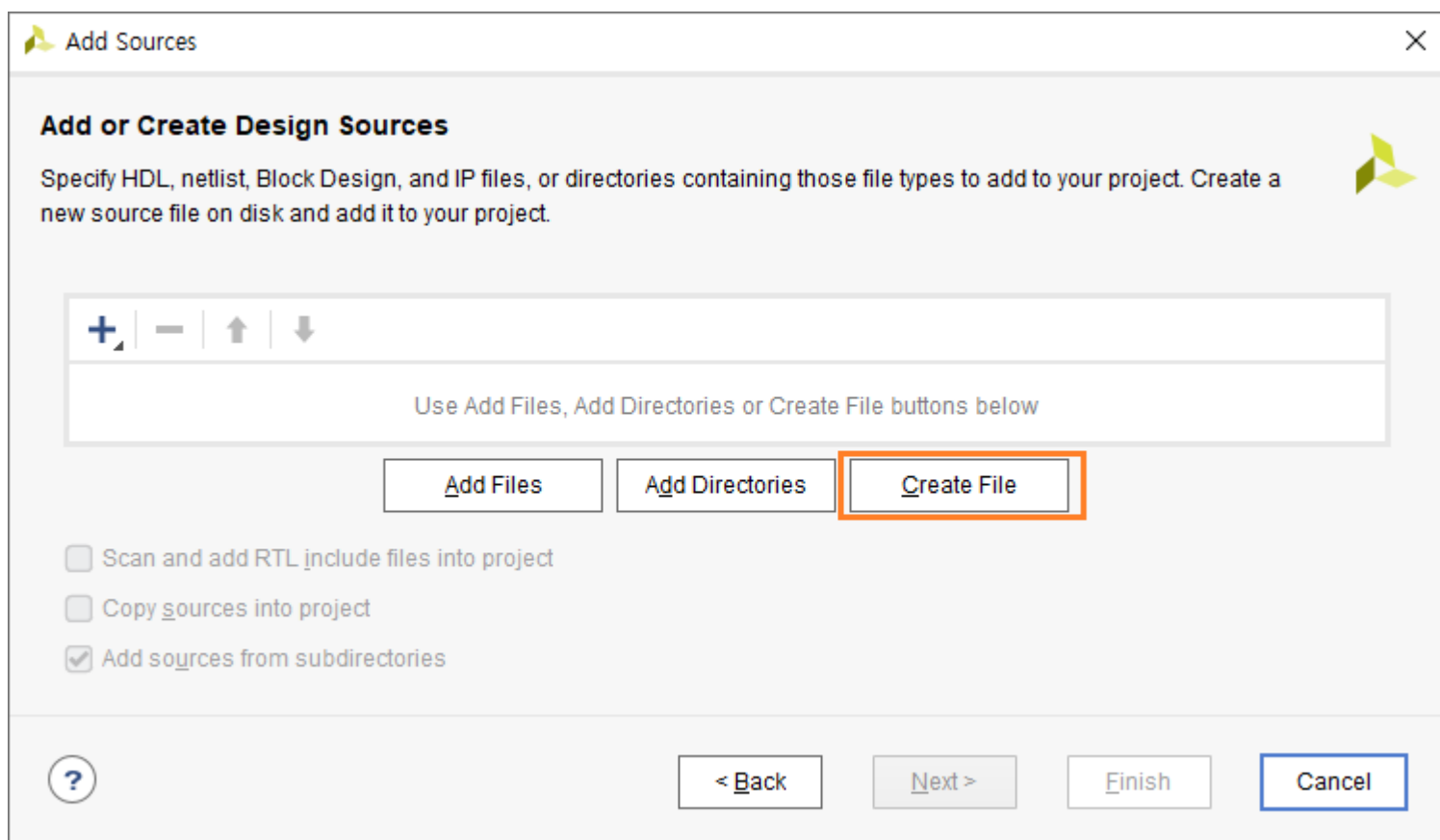
이번 장에서는 User Logic을 구현합니다. Led를 On/Off 하는 로직을 구현합니다. Design Sources - 우클릭 - Add Sources... 를 클릭합니다.

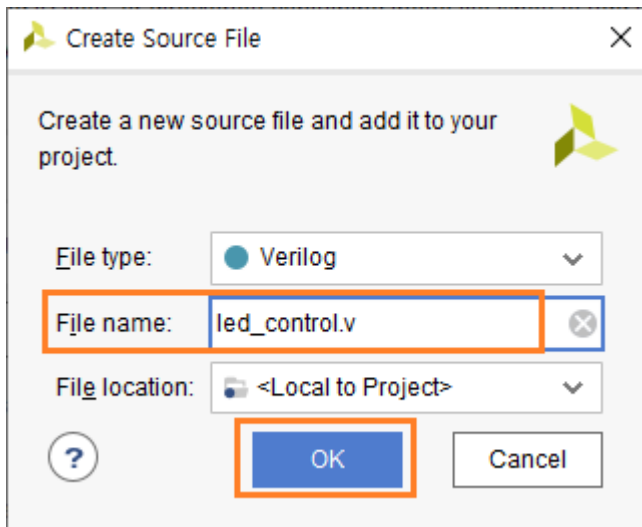


“Add or create design sources”을 선택하고 Next를 클릭합니다.

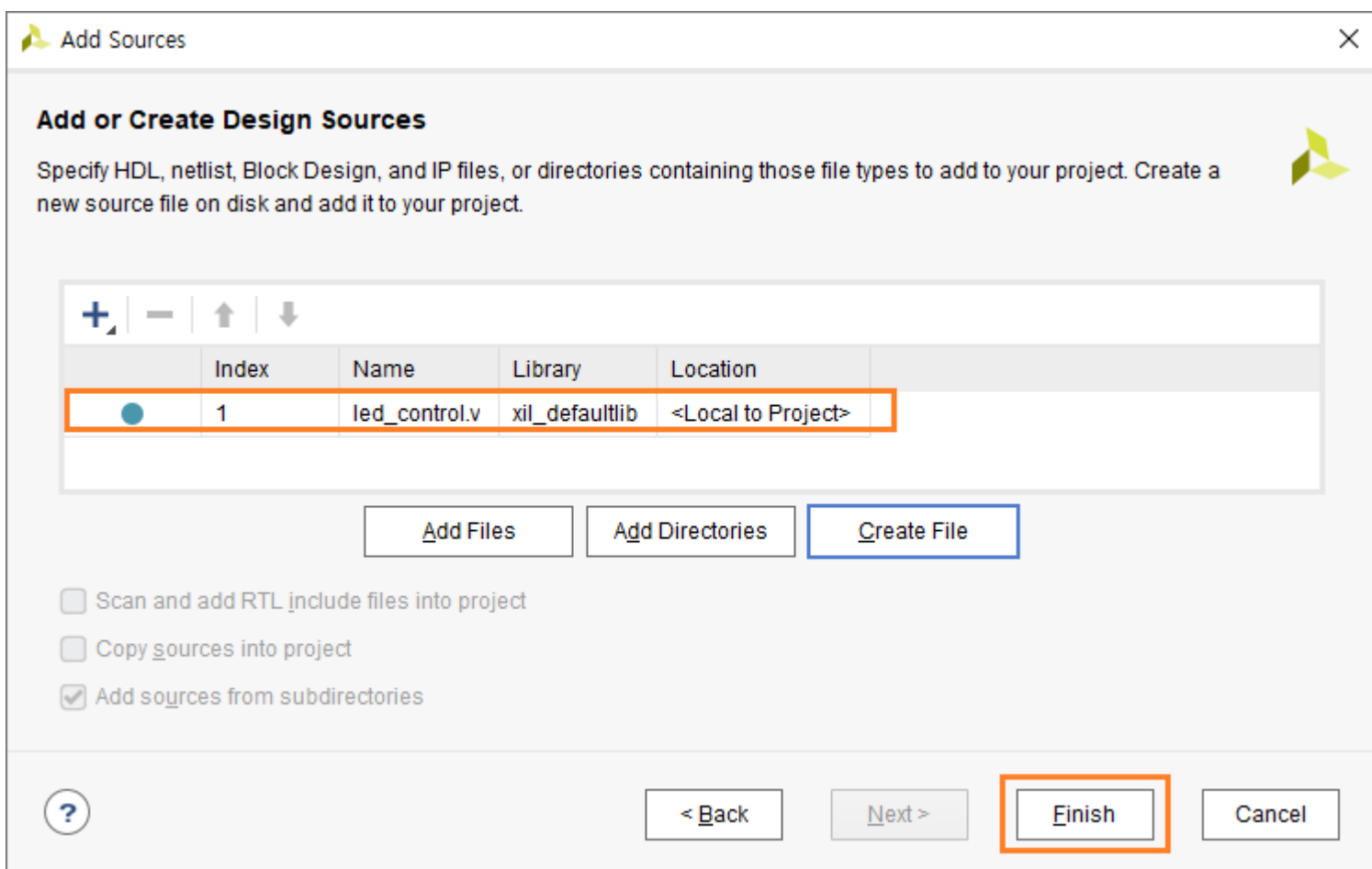


Create File 을 클릭하면, Create Source File 윈도우가 나타납니다. File name : led_control.v 을 입력하고 OK를 클릭합니다.

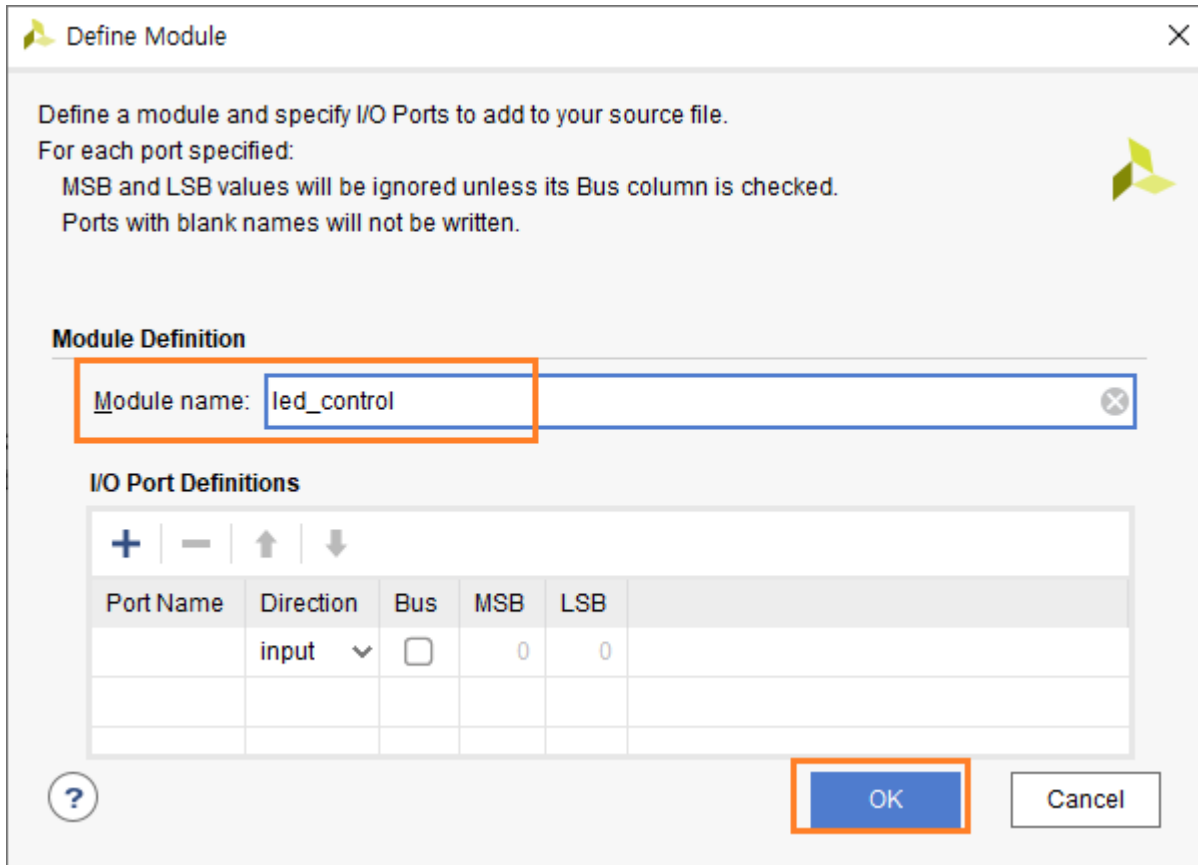




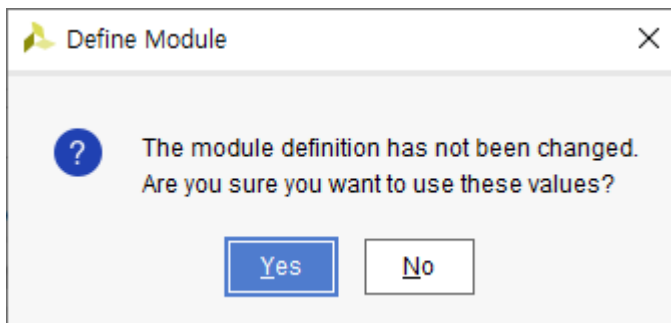
led_control.v 파일이 추가되었습니다. Finish를 클릭합니다. (이미 생성된 소스 파일을 프로젝트에 추가하려면 Add Files 버튼을 클릭하여 해당 파일을 추가합니다)



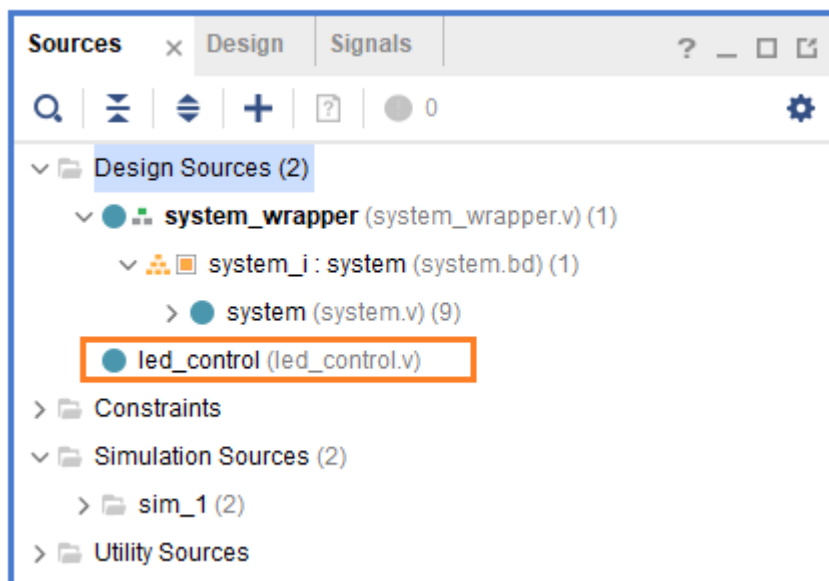
Define Module 윈도우에서는 모듈 이름과 핀들을 설정할 수 있습니다. 기본값을 사용합니다. OK를 클릭합니다.



YES를 클릭합니다.



아래 그림은 led_control 모듈이 추가된 것을 보여줍니다.



led_control.v 파일을 열어 아래와 같이 코드를 추가합니다. (참고로 Ultra Editor를 사용하고 Tap 수는 8을 사용합니다)
100Mhz Clock을 이용하여 0.5초마다 4개의 LED를 순차적으로 On/Off 하는 코드입니다.

```

23 module led_counter(
24     reset    ,
25     clock    ,
26     led
27 );
28 input      reset ;
29 input      clock ;
30 output [3:0] led ;
31
32 reg [25:0] cnt ;
33 always @(posedge clock or negedge reset)
34 begin
35     if(~reset)      cnt <= 26'd0;
36     else            cnt <= (cnt==26'd50000000) ? 26'd0 : cnt+1'b1;
37 end
38
39 reg [3:0] led ;
40 always @(posedge clock or negedge reset)
41 begin
42     if(~reset)      led <= 4'b0;
43     else            led <= (cnt==26'd50000000) ? led + 1'b1 : led ;
44 end
45
46 endmodule

```

Design Source에 led_counter, system_wrapper 모듈을 포함하는 Top 모듈을 추가합니다. led_control 모듈을 추가할 때와 동일한 방법으로 BlazeTop.v 모듈을 추가합니다. 그리고 아래와 같이 코드를 추가합니다.

```

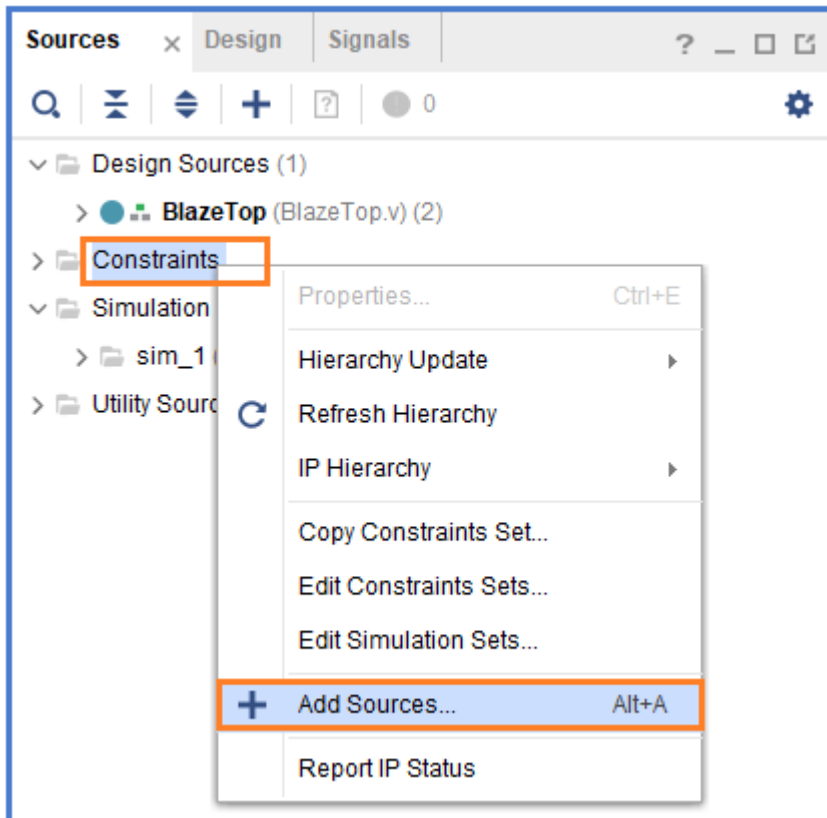
23 module BlazeTop(
24     reset    ,
25     sys_clk  ,
26     uart_rxd ,
27     uart_txd ,
28     led
29 );
30 input      reset ;
31 input      sys_clk ;
32 input      uart_rxd ;
33 output      uart_txd ;
34 output [3:0] led ;
35
36 system_wrapper system_wrapper (
37     .reset      (reset      ),
38     .sys_clk     (sys_clk    ),
39     .uart_rxd    (uart_rxd   ),
40     .uart_txd    (uart_txd   )
41 );
42
43 wire [3:0] led ;
44 led_counter led_counter(
45     .reset      (reset      ),
46     .clock       (sys_clk    ),
47     .led         (led        )
48 );
49
50 endmodule

```

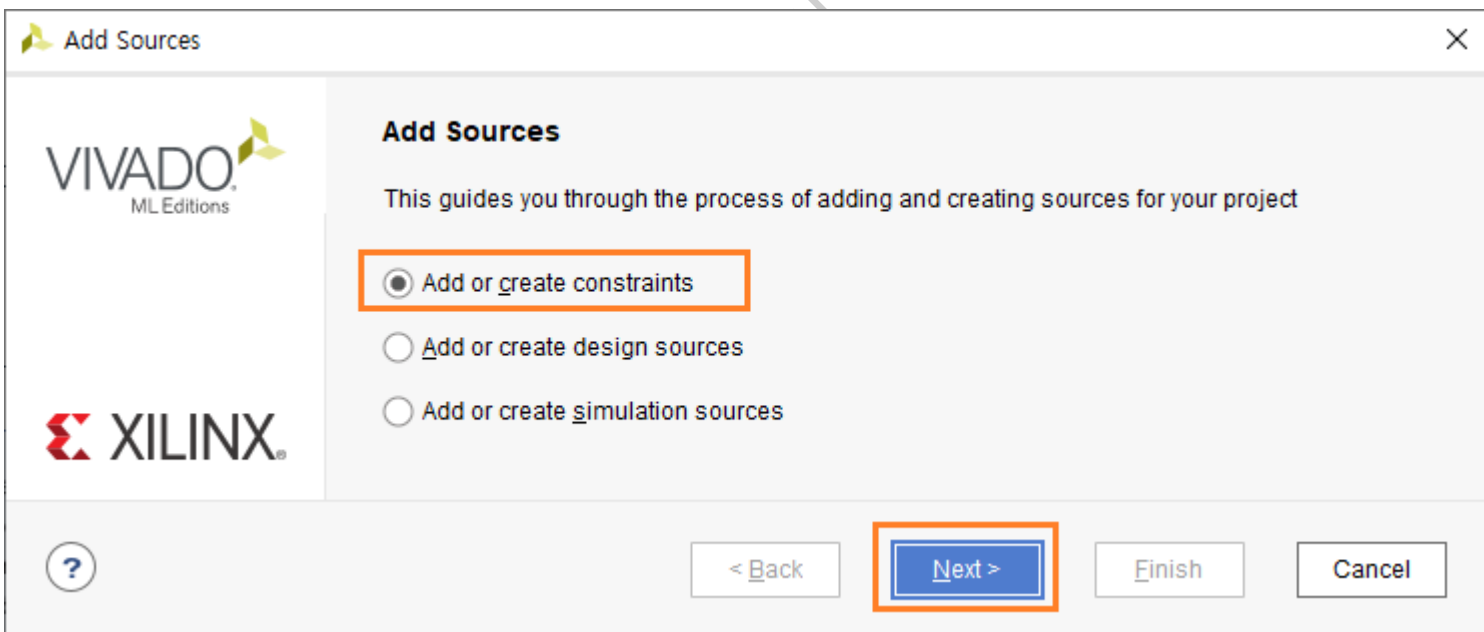
system_wrapper, led_counter 모듈을 포함하는 간단한 로직입니다.

4.1.5 xdc 구현

이번 장에서는 핀 할당을 구현합니다. Constrains - 우클릭 - Add Sources...를 클릭합니다.



“Add or Create constraints”을 선택하고 Next를 클릭합니다.



이후의 과정은 led_module을 추가할 때와 동일합니다. File name : BlazeTop.xdc로 입력해서 파일을 추가합니다.

BlazeTop.xdc 에 아래와 같이 코드를 추가합니다.

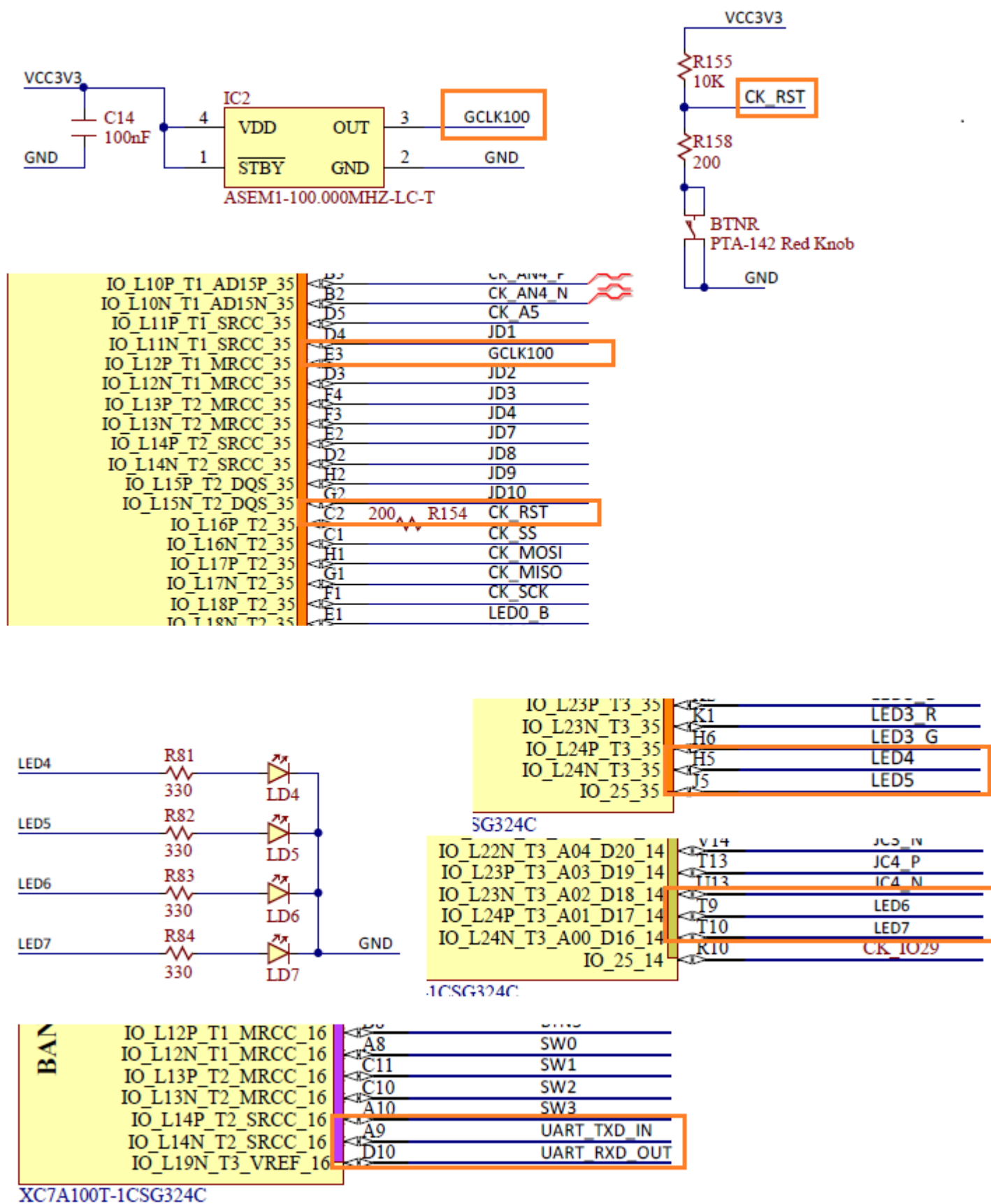
```

1 ## This file is a general .xdc for the Arty A7-35 Rev. D and Rev. E
2 ## To use it in a project:
3 ## - uncomment the lines corresponding to used pins
4 ## - rename the used ports (in each line, after get_ports) according to the top level signal name:
5
6 ## Clock signal
7 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clk }]; # gclk[100]
8 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clk }];
9
10 ## Misc. ChipKit Ports
11 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }]; # ck_rst
12
13 ## LEDs
14 set_property -dict { PACKAGE_PIN H5      IOSTANDARD LVCMOS33 } [get_ports { led[0] }]; # led[4]
15 set_property -dict { PACKAGE_PIN J5      IOSTANDARD LVCMOS33 } [get_ports { led[1] }]; # led[5]
16 set_property -dict { PACKAGE_PIN T9      IOSTANDARD LVCMOS33 } [get_ports { led[2] }]; # led[6]
17 set_property -dict { PACKAGE_PIN T10     IOSTANDARD LVCMOS33 } [get_ports { led[3] }]; # led[7]
18
19 ## USB-UART Interface
20 set_property -dict { PACKAGE_PIN A9      IOSTANDARD LVCMOS33 } [get_ports { uart_rxd }];
21 set_property -dict { PACKAGE_PIN D10     IOSTANDARD LVCMOS33 } [get_ports { uart_txd }];
22
23
24 #####
25 # MISC
26 #####
27 set_operating_conditions -ambient_temp 80.0
28
29 set_property CONFIG_MODE SPIx1 [current_design]
30
31 set_property CFGBVS VCCO [current_design]
32
33 set_property CONFIG_VOLTAGE 3.3 [current_design]
34
35 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
36
37 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
38 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
39
40 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

```

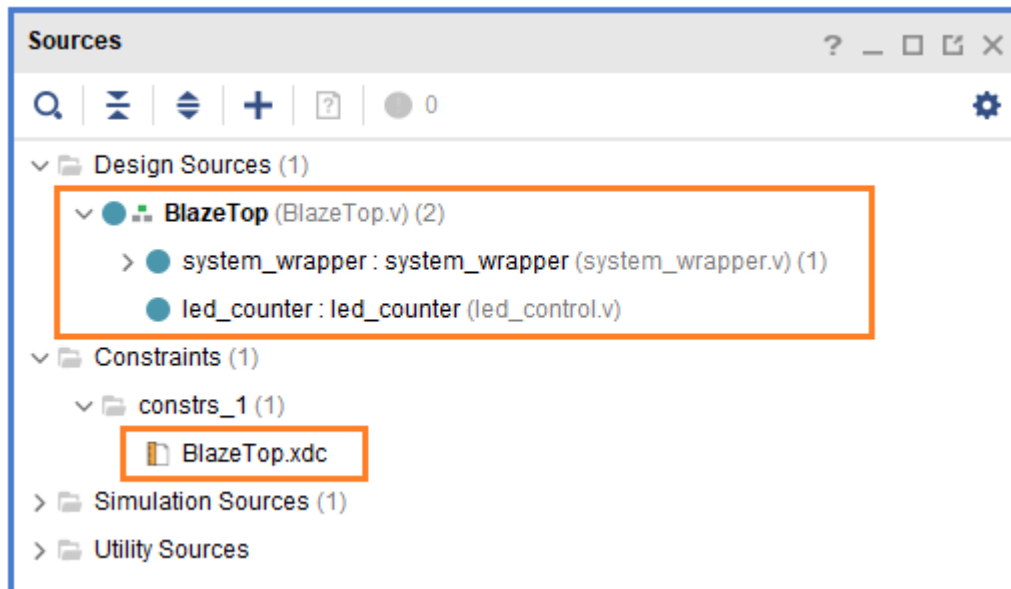
- ✓ 라인 7 - 8 : sys_clk 핀 설정, 100Mhz
- ✓ 라인 11 : reset 핀 설정
- ✓ 라인 14 - 17 : led4 ~ 7 핀 설정
- ✓ 라인 20 - 21 : usb to uart 핀 설정
- ✓ 라인 24 - 40 : configuration option 설정, 없어도 동작 합니다.

아래는 Arty A7 35T 보드 회로에서 할당한 핀들을 보여줍니다.



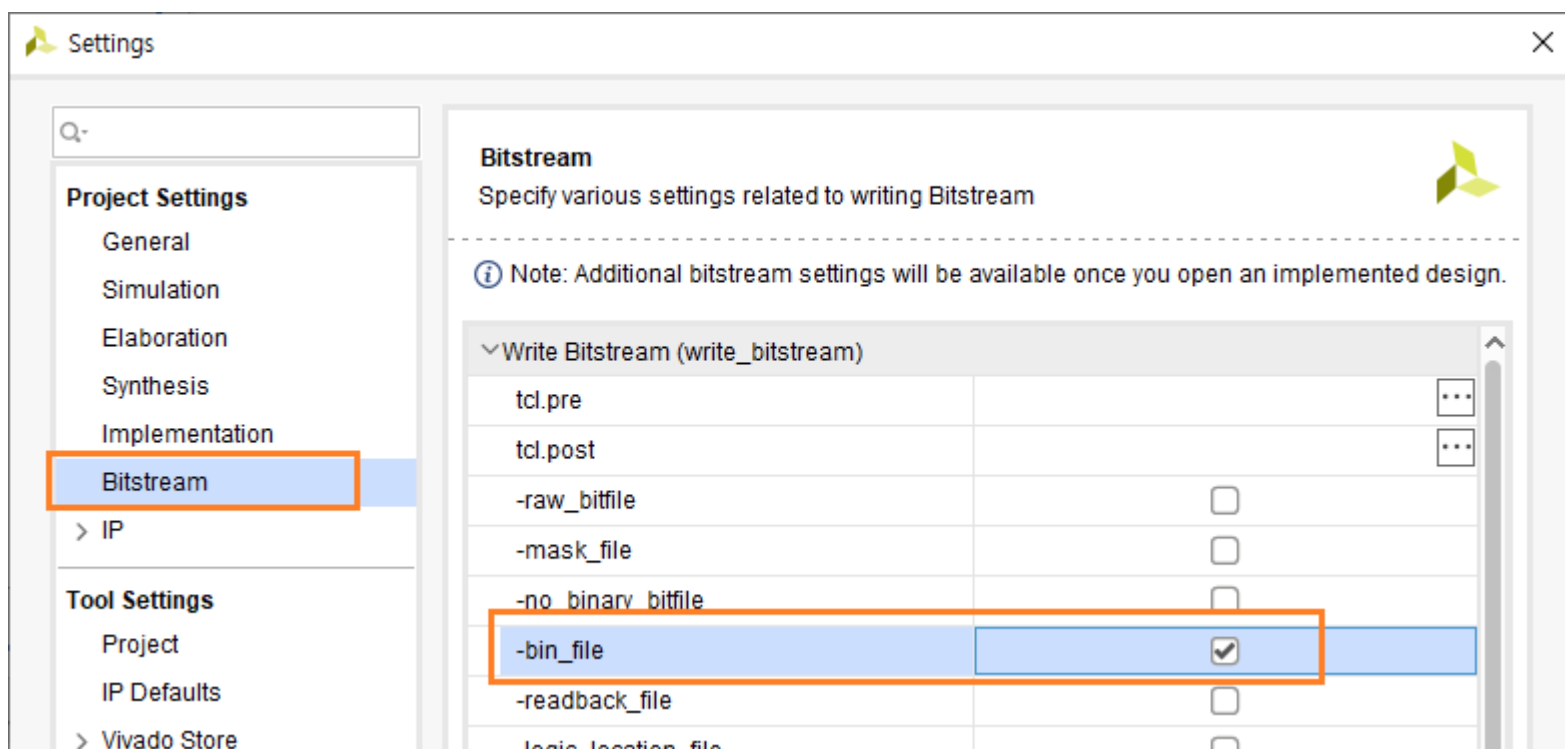
4.1.6 Generate Bitstream

아래는 지금까지 구현된 Source 구조를 보여줍니다. BlazeTop 모듈이 Top 모듈로 설정되어 있습니다.



Project Settings 에서 Bitstream 생성시 bin파일을 생성하도록 아래와 같이 설정합니다. PROJECT MANAGER - Settings 을 클릭합니다.

Settings 윈도우에서 Bitstream을 클릭하고 -bin_file 항목을 check 하고 OK를 클릭합니다.



PROMGRAM AND DEBUG - Generate Bitstream을 클릭해서 Bitstream을 생성합니다.

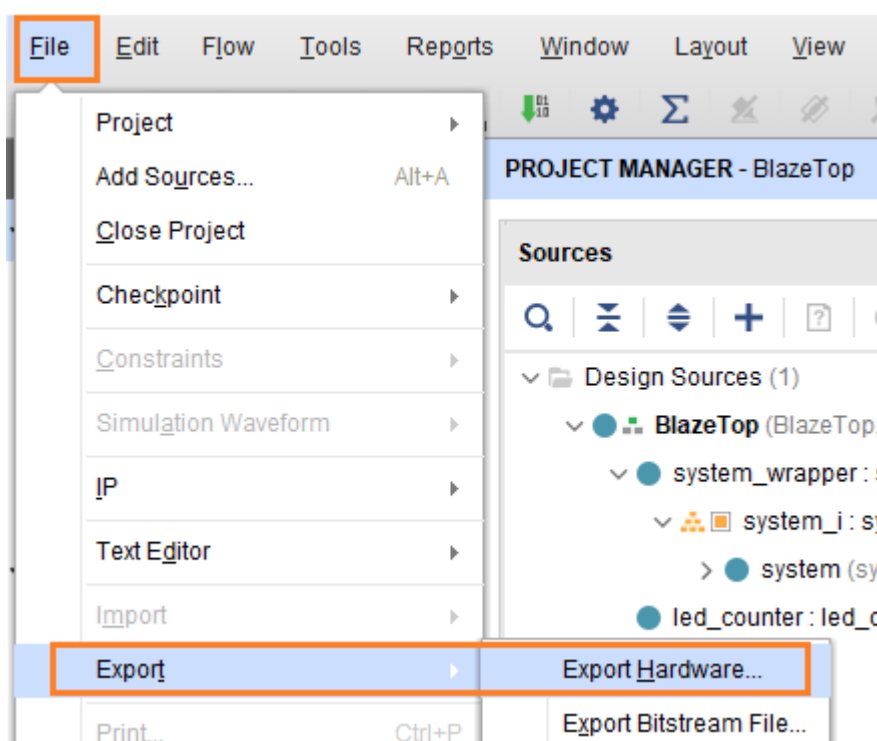
완료 되면 Cancel 버튼을 클릭합니다.

아래 그림은 Project summary를 보여줍니다. Implementation에 1개의 warning 메시지가 있습니다. 혹 여러개의 warning 메시지가 있거나 critical warning 메시지가 있는 경우에는 Block Design에 오류가 있는 경우입니다. Clock 이나 reset 핀들이 잘 연결되었는지 확인해 보시길 바랍니다.

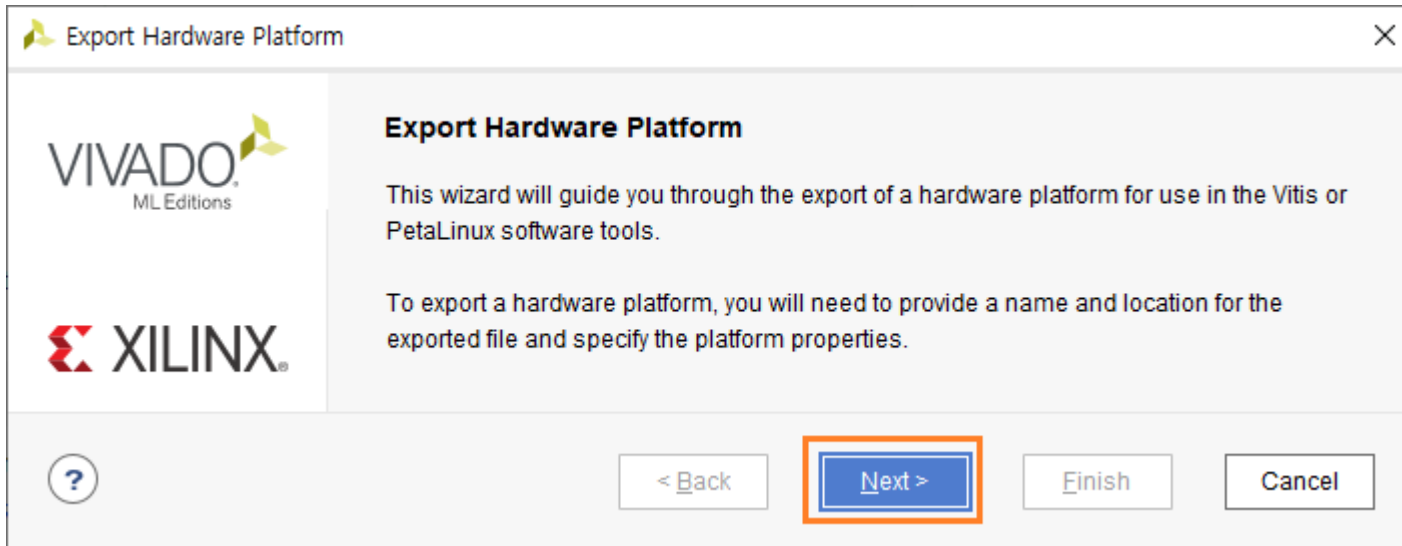
Synthesis	Implementation	Summary Route Status
Status: ✓ Complete	Status: ✓ Complete	
Messages: ! 913 warnings	Messages: ! 1 warning	
Active run: synth_1	Active run: impl_1	
Part: xc7a35tcs324-1	Part: xc7a35tcs324-1	
Strategy: Vivado Synthesis Defaults	Strategy: Vivado Implementation Defaults	
Report Strategy: Vivado Synthesis Default Reports	Report Strategy: Vivado Implementation Default Reports	
Incremental synthesis: Automatically selected checkpoint	Incremental implementation: None	

4.1.7 Export Hardware

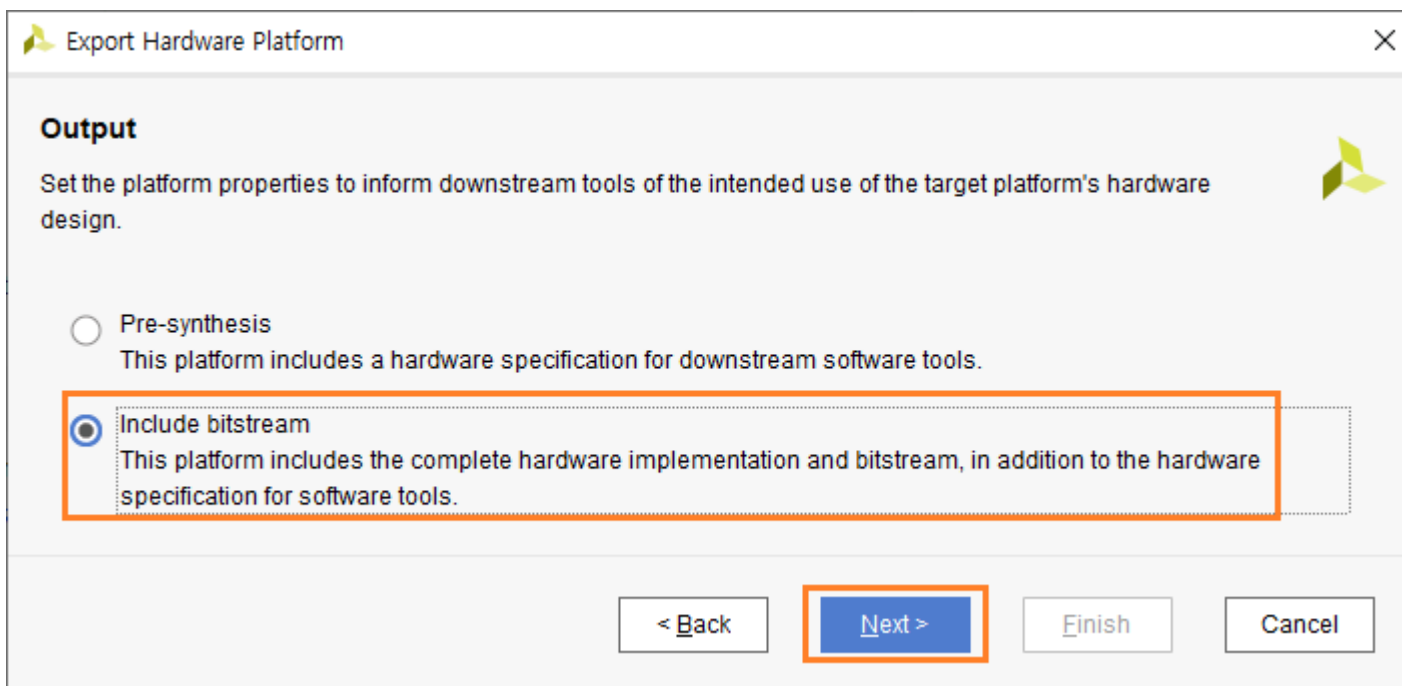
생성된 로직을 vitis (Embedded sw 개발 tool)에서 사용하기 위해서는 Export Hardware를 합니다. 메뉴에서 File - Export - Export Hardware를 클릭합니다.



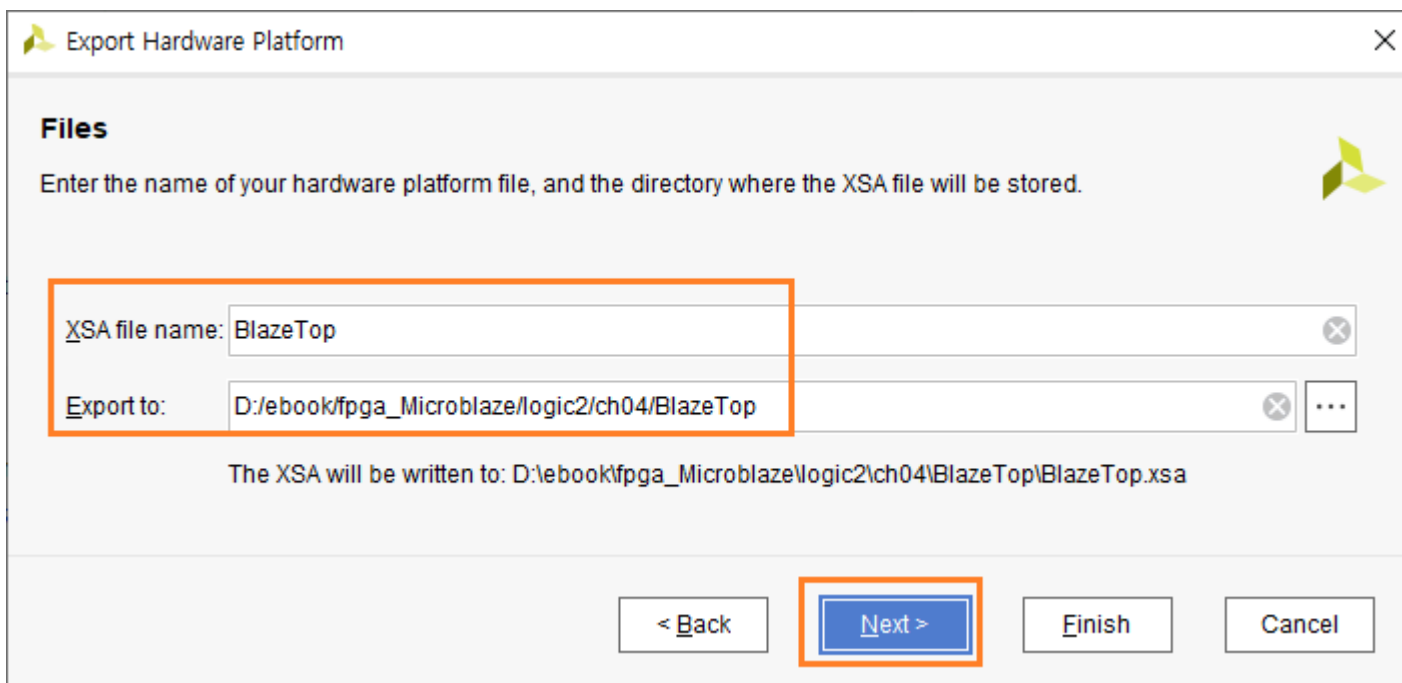
Next를 클릭합니다.



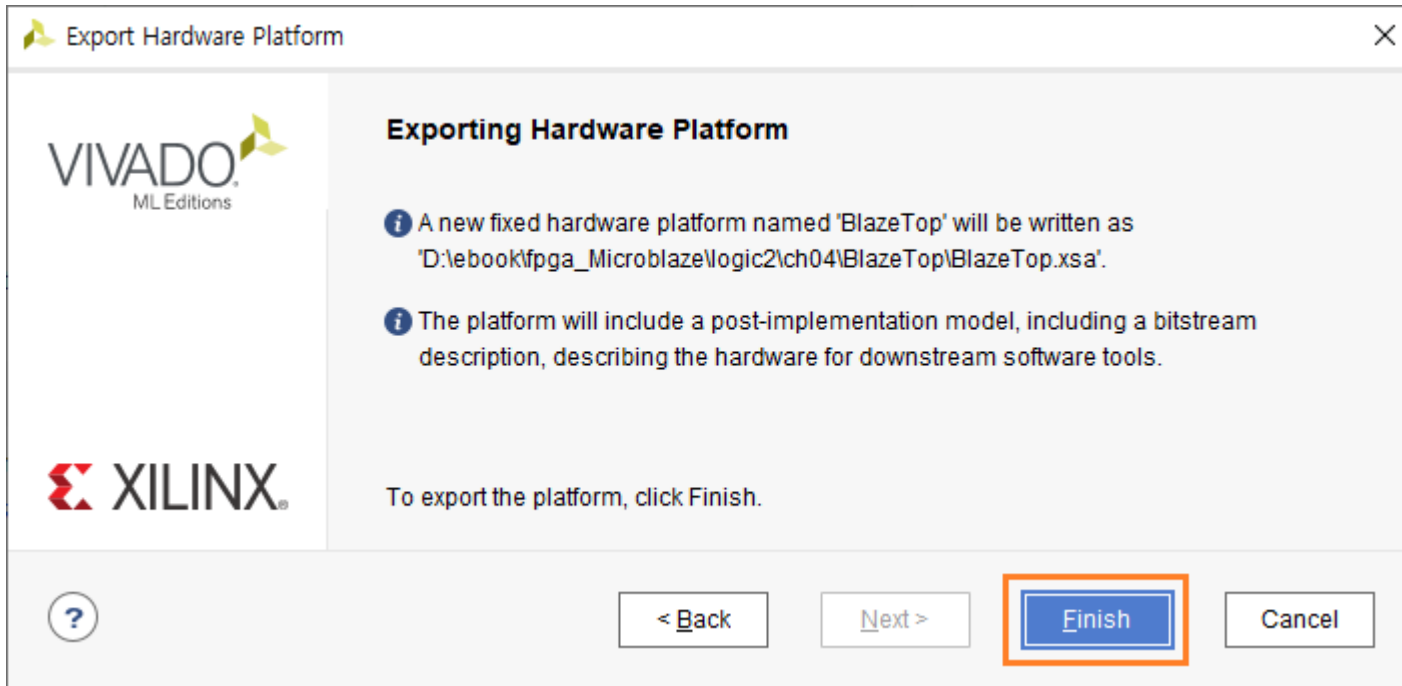
“Include bitstream”을 선택하고 Next를 클릭합니다.



생성되는 파일은 xsa 확장자를 가진 파일입니다.파일명과 경로를 확인하고 Next를 클릭합니다.



Finish를 클릭하면 XSA 파일이 생성됩니다.



기본 Template 이 완료되었습니다. 다음장에서는 기본 Template 폴더를 복사하여 이름을 변경하여 사용하도록 하겠습니다. 다음 장에서는 UART를 통하여 Hello world Message를 출력하고, LED를 On/Off 하는 것을 구현합니다.

참고사항) vivado는 폴더의 경로를 바꾸어서 사용해도 별 문제가 되지 않지만, vitis는 프로젝트를 생성하고 폴더(경로나 이름)를 변경하면 인식을 못하는 문제가 있습니다. 물론 프로젝트 환경 파일을 수동으로 수정해서 사용하면 되지만 번거로운 일입니다. 따라서 vitis를 진행하기 전에 기본 Template를 복사해서 사용합니다.

4.2 Embedded SW 구현

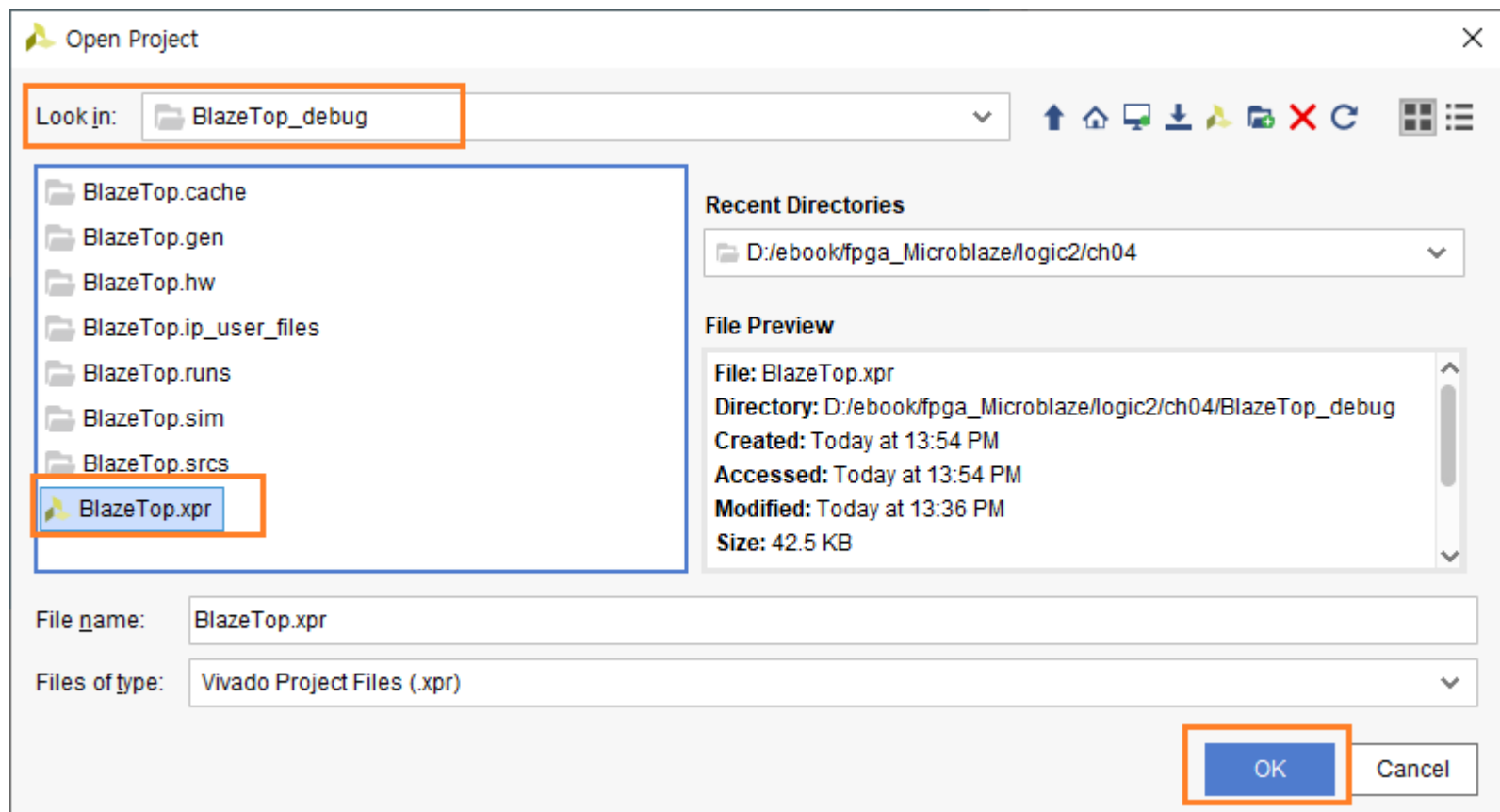
이번 장에서는 vitis를 이용하여 Embedded SW를 구현합니다. vitis는 많은 예제 프로그램을 제공합니다. 그 중에 Hello world를 구현하는 예제를 사용합니다.

먼저 앞장에서 생성했던 폴더를 복사하여 이름을 BlazeTop_debug로 변경합니다.

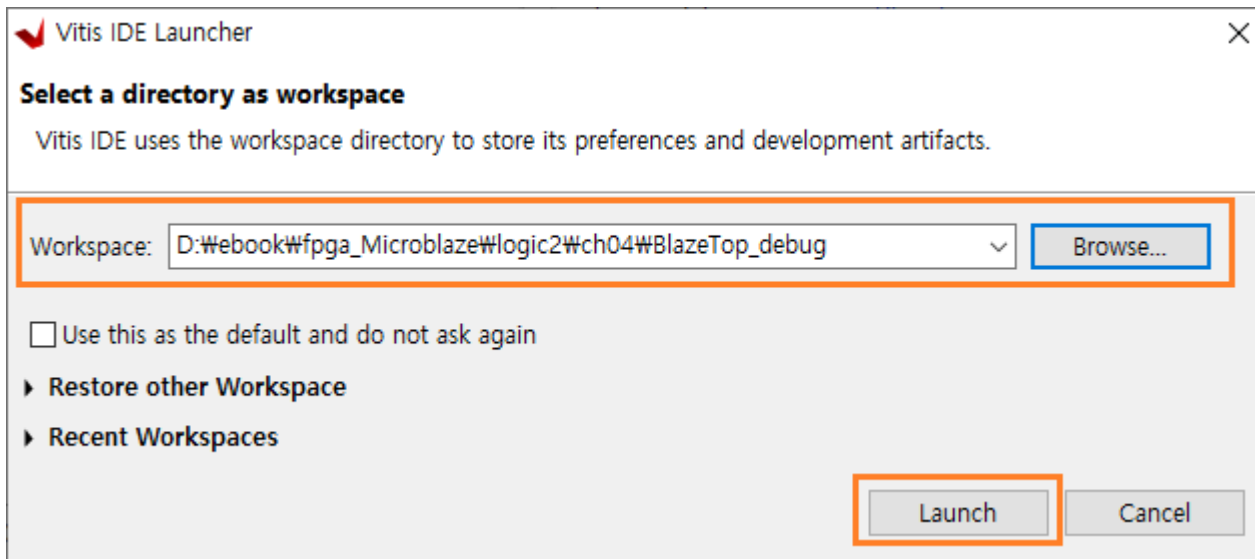
이름	수정한 날짜	유형	크기
BlazeTop	2023-08-10 오후 1:44	파일 폴더	
BlazeTop_debug	2023-08-10 오후 1:54	파일 폴더	

BlazeTop 폴더는 앞장에서 생성한 기본 Template 폴더입니다. BlazeTop_debug는 BlazeTop 폴더를 복사한 후 이름을 변경하였습니다.

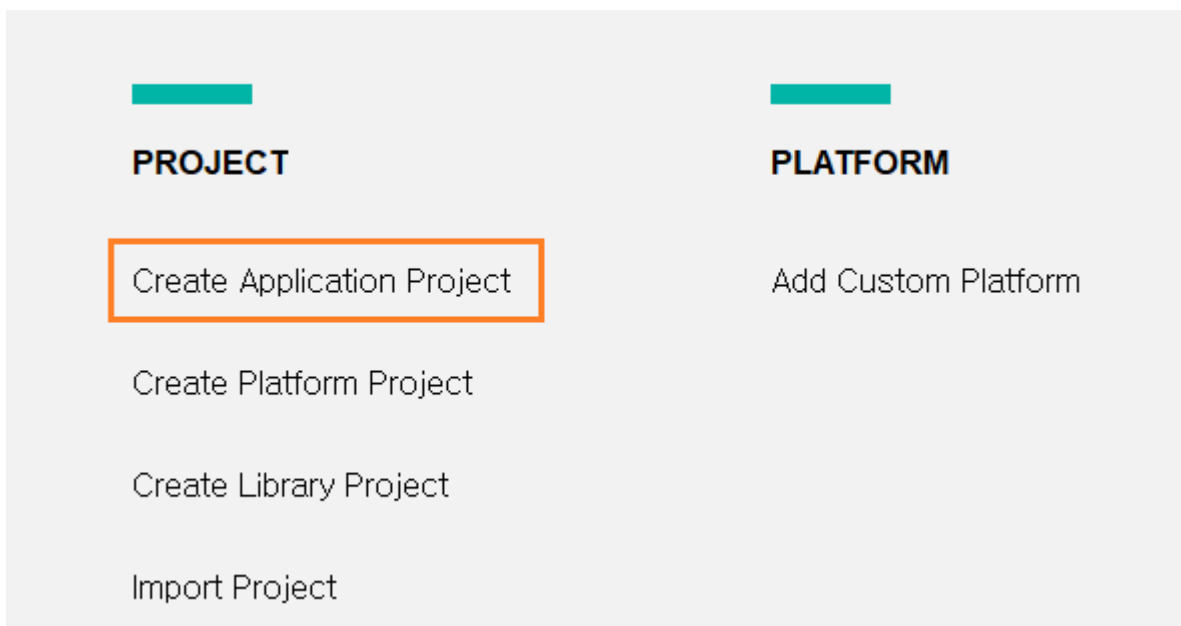
vivado에서 BlazeTop Project를 닫고(File - Close Project), Open Project를 클릭해서 BlazeTop_debug 폴더에 있는 BlazeTop.xpr 파일을 Open 합니다.



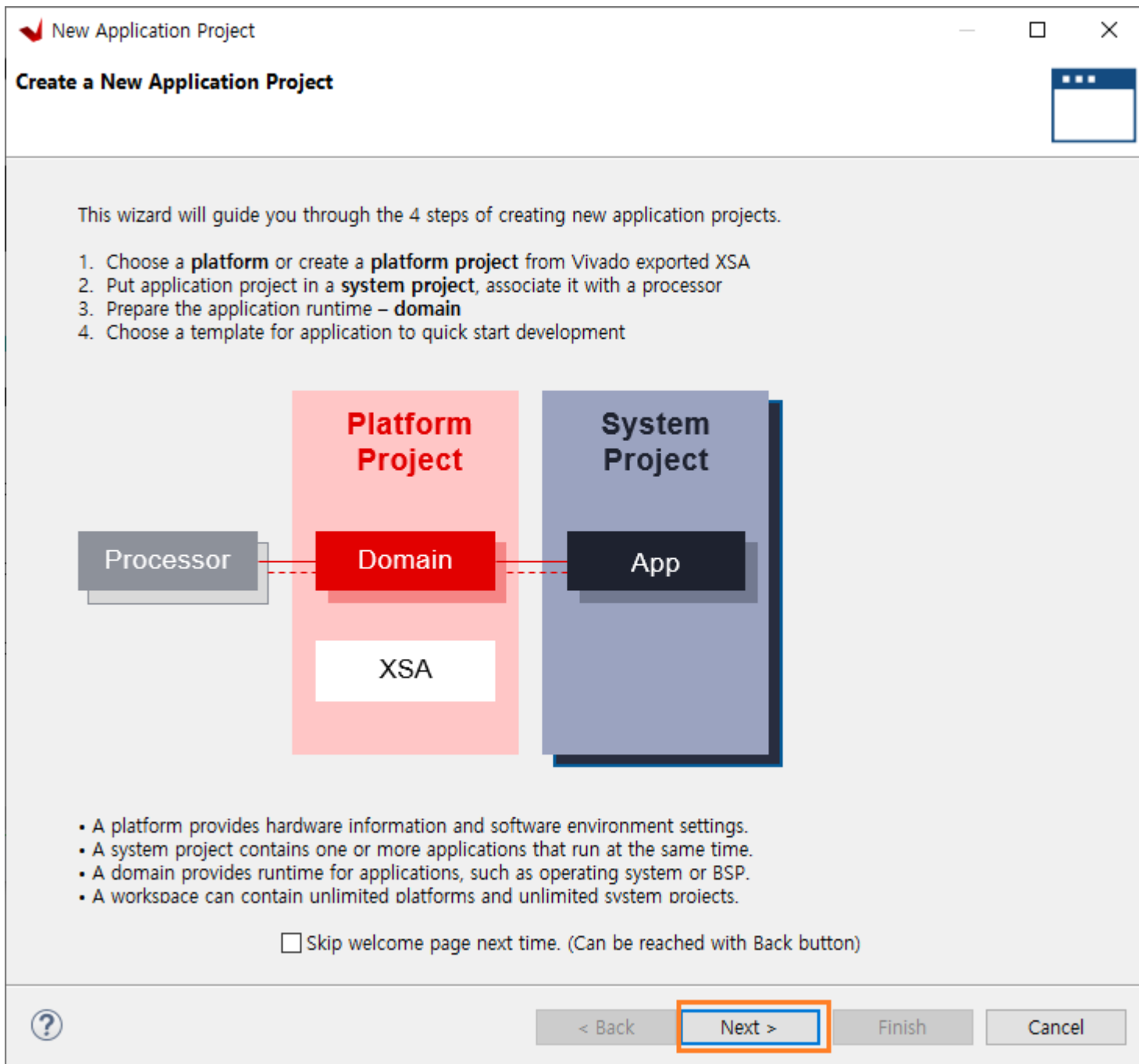
XSA 파일까지 생성하였기 때문에 바로 Vitis에서 진행할 수 있습니다. Tools - Launch Vitis IDE를 클릭하면 vitis가 실행됩니다. Workspace가 BlazeTop_debug 인지를 확인하고 Launch를 클릭합니다. (Workspace 위치가 다르면 Browse... 클릭해서 해당 위치를 변경합니다)



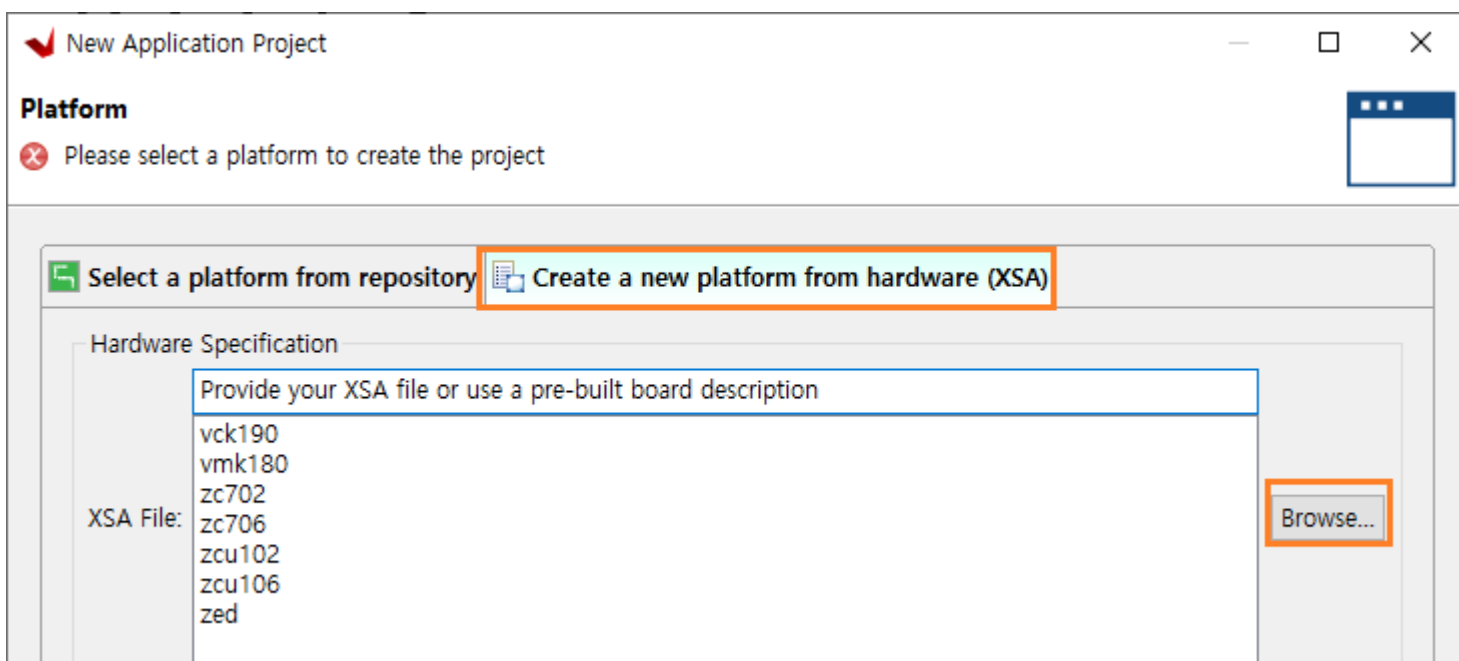
새로운 프로젝트를 생성하기 위하여 Create Application Project를 클릭합니다.

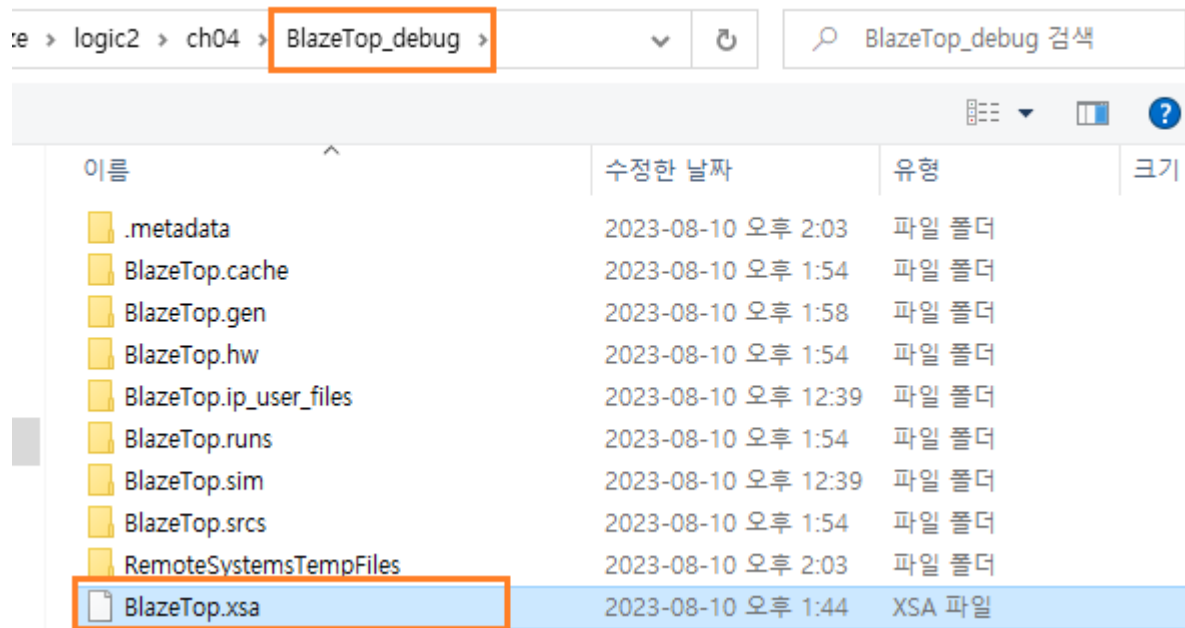


Next를 클릭합니다.

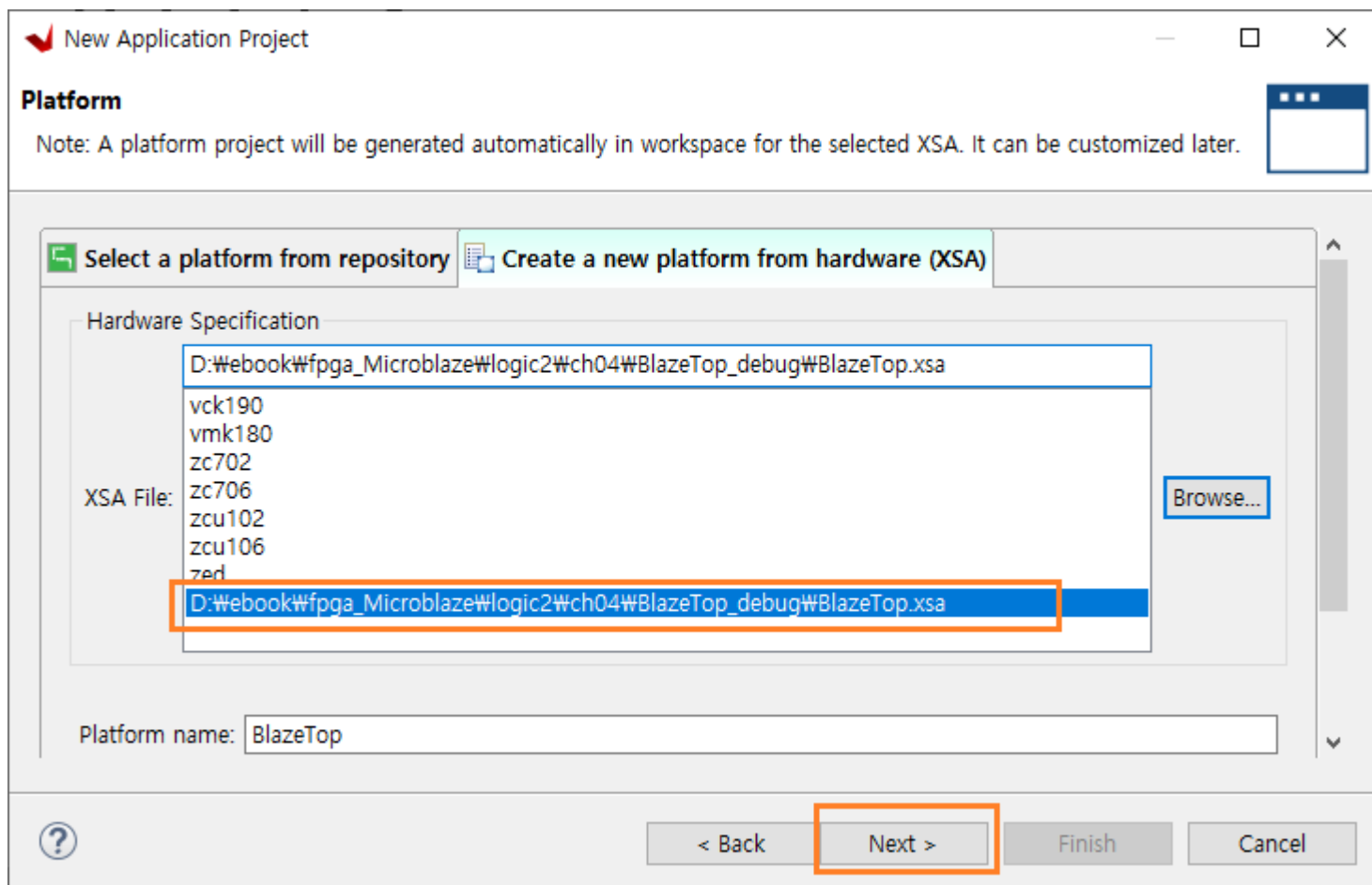


Create a new platform from hardware (XSA) 탭을 선택하고, Browe... 클릭해서 vivado에서 생성한 xsa 파일을 open 합니다.





xsa 파일을 설정하면 Next 버튼이 활성화 됩니다. Next를 클릭합니다.



Application project name (BlazeTopApp)을 설정합니다. Next를 클릭합니다.

New Application Project

Application Project Details
Specify the application project name and its system project properties

Application project name:

System Project
Create a new system project for the application or select an existing one from the workspace

Select a system project
+ Create new...

System project details

System project name:

Target processor

Select target processor for the Application project.

Processor	Associated applications
microblaze_0	BlazeTopApp

Show all processors in the hardware specification ☒

< Back **Next >** Finish Cancel

Next를 클릭합니다.

New Application Project

Domain
Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

Select a domain
+ Create new...

Domain details

Name:

Display Name:

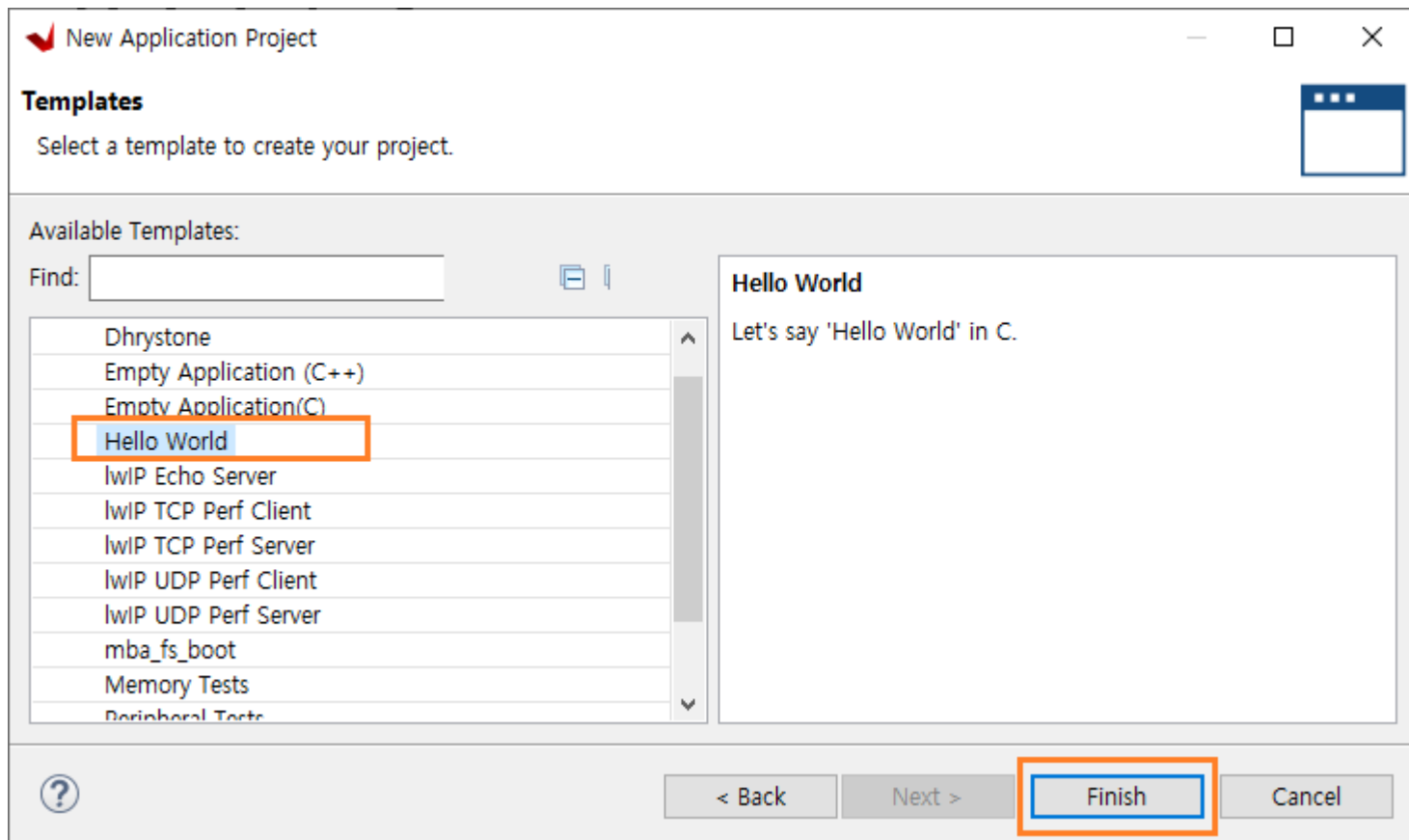
Operating System:

Processor:

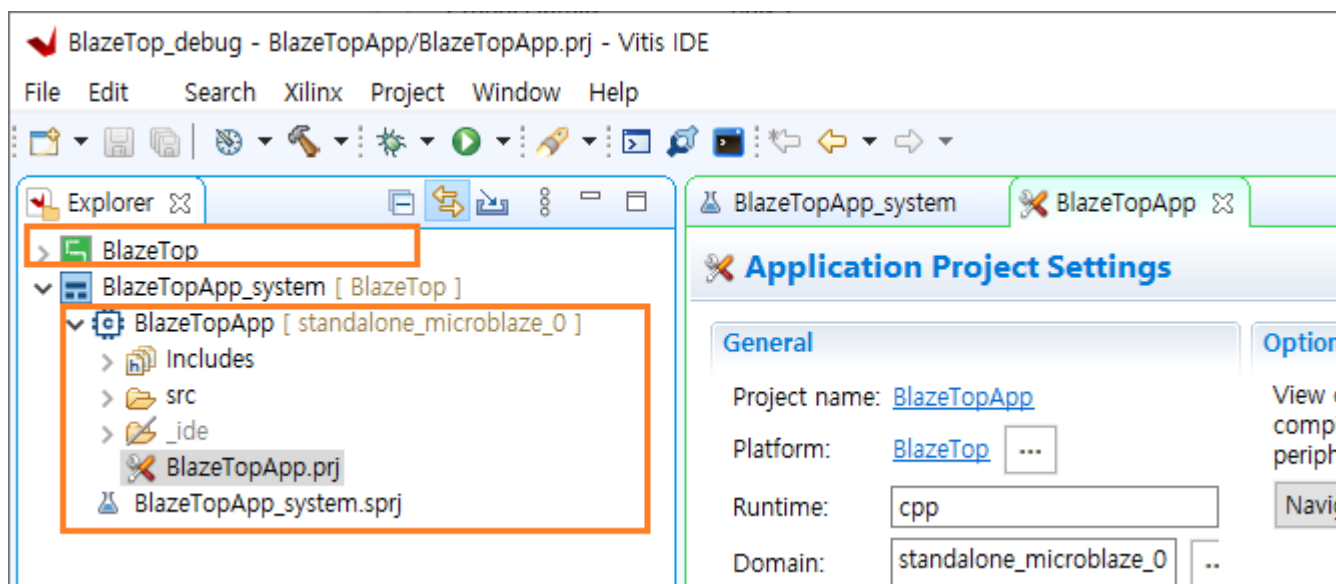
Architecture:

< Back **Next >** Finish Cancel

Templates 에서 “Hello World”를 선택하고 Finish를 클릭하면 프로젝트가 생성됩니다.



Vitis에는 2개의 Project가 있습니다. 하나는 Vivado에서 생성 후 XSA 파일로 변환한 프로젝트 파일이고, 나머지 하나는 Vitis에서 생성한 Embedded SW용 프로젝트 입니다. BlazeTop이 vivado에서 생성한 프로젝트, BlazeTopApp가 vitis에서 생성한 프로젝트입니다.



vivado에서 HW관련된 내용들이 수정되어 xsa 파일이 수정되면, BlazeTop을 Update - Rebuild를 해야 합니다. Embedded sw가 수정되면, BlazeTopApp를 Build 해야 합니다.

아래는 src - helloworld.c 의 내용을 보여줍니다.

```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56
57     print("Hello World\n\r");
58     print("Successfully ran Hello World application");
59     cleanup_platform();
60     return 0;
61 }

```

코드 내용을 아래와 같이 수정합니다.

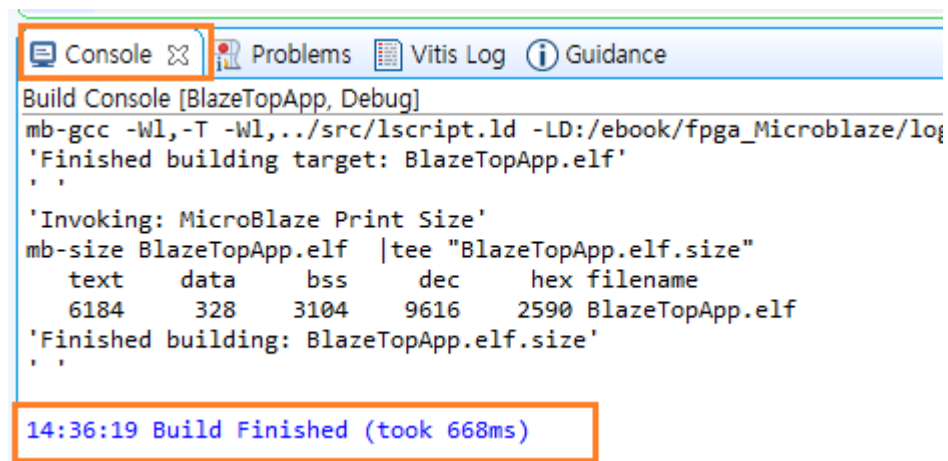
```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53 int main()
54 {
55     int cnt = 0;
56
57     init_platform();
58
59     print("Hello World ..\r\n");
60     while(1)
61     {
62         xil_printf("count : %d \r\n", cnt++);
63         sleep(1);
64     }
65     cleanup_platform();
66     return 0;
67 }

```

1초마다 메시지를 표시하는 간단한 프로그램입니다.

BlazeTopApp - 우클릭 - Build Project를 클릭합니다. 에러가 없이 Build 되었습니다.



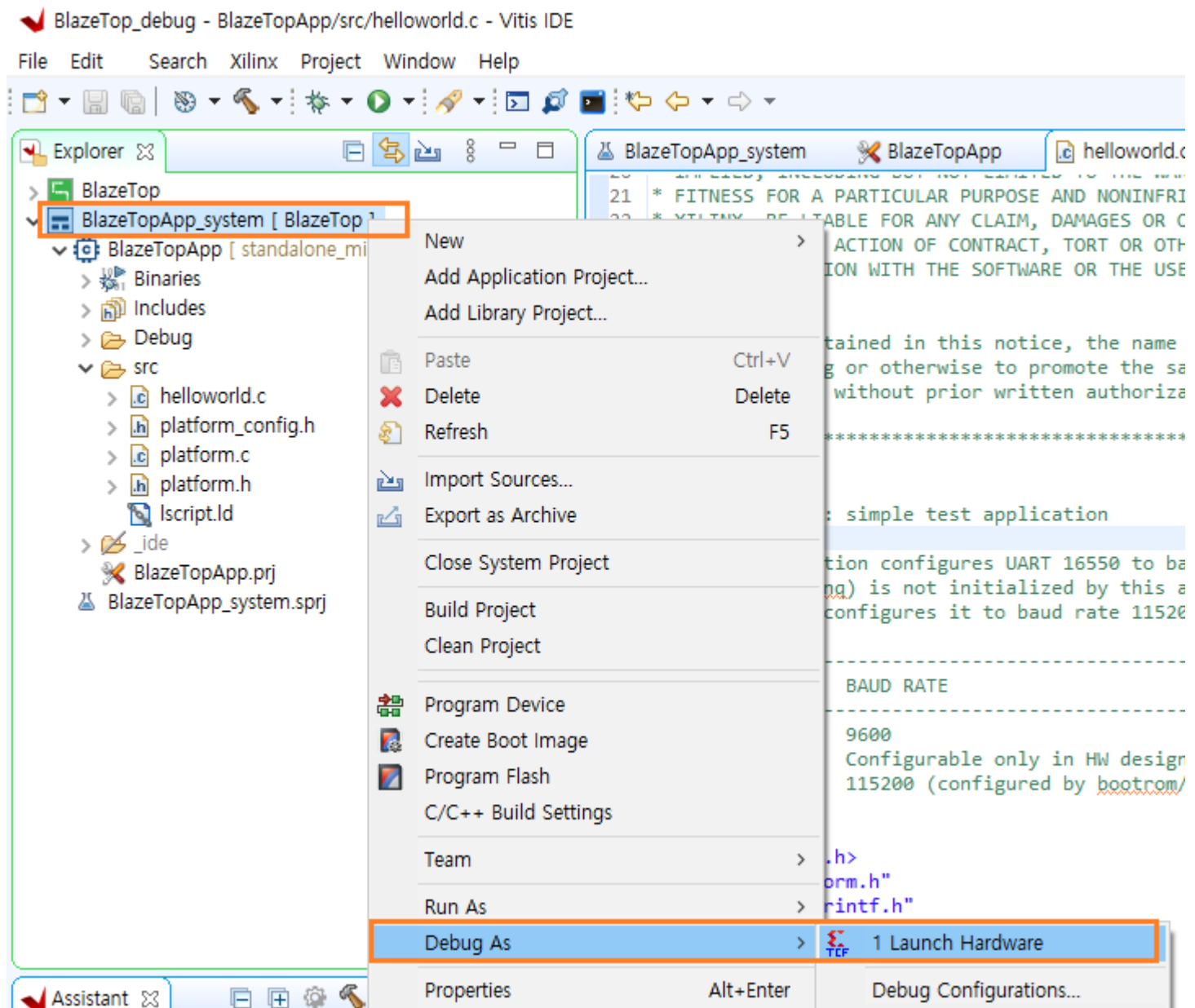
```

Console Problems Vitis Log Guidance
Build Console [BlazeTopApp, Debug]
mb-gcc -Wl,-T -Wl,../src/lscript.ld -LD:/ebook/fpga_Microblaze/lo
'Finished building target: BlazeTopApp.elf'
'Invoking: MicroBlaze Print Size'
mb-size BlazeTopApp.elf |tee "BlazeTopApp.elf.size"
  text  data  bss   dec   hex filename
  6184   328  3104  9616  2590 BlazeTopApp.elf
'Finished building: BlazeTopApp.elf.size'
14:36:19 Build Finished (took 668ms)

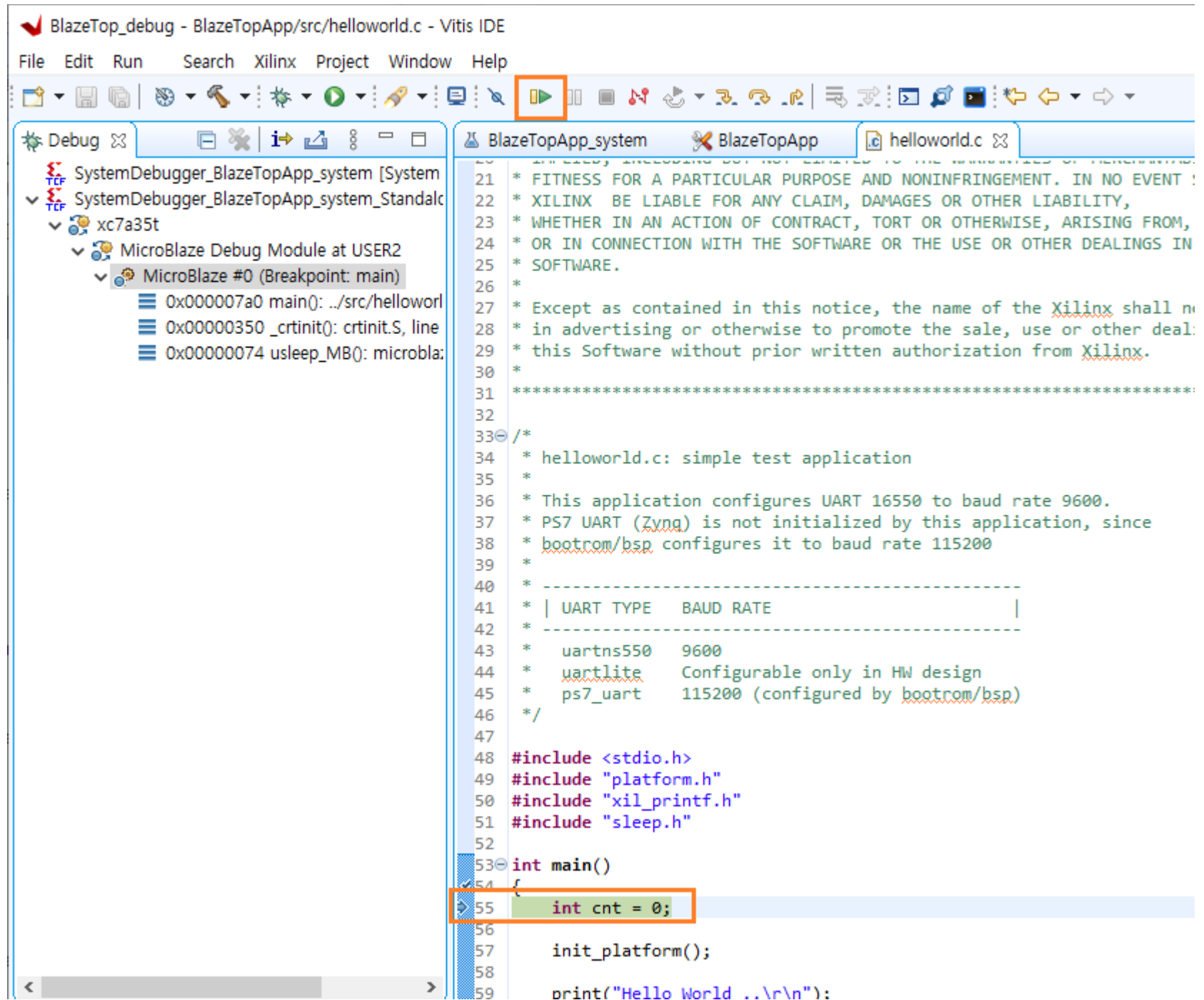
```

결과를 확인하기 위하여 보드에 다운로드를 합니다. 먼저 Arty A7 35T 보드와 PC를 USB로 연결합니다. 전원이 인가되고 보드가 인식됩니다.

BlazeTopApp_system - 우클릭 - Debug As - 1 Launch Hardware를 클릭합니다.



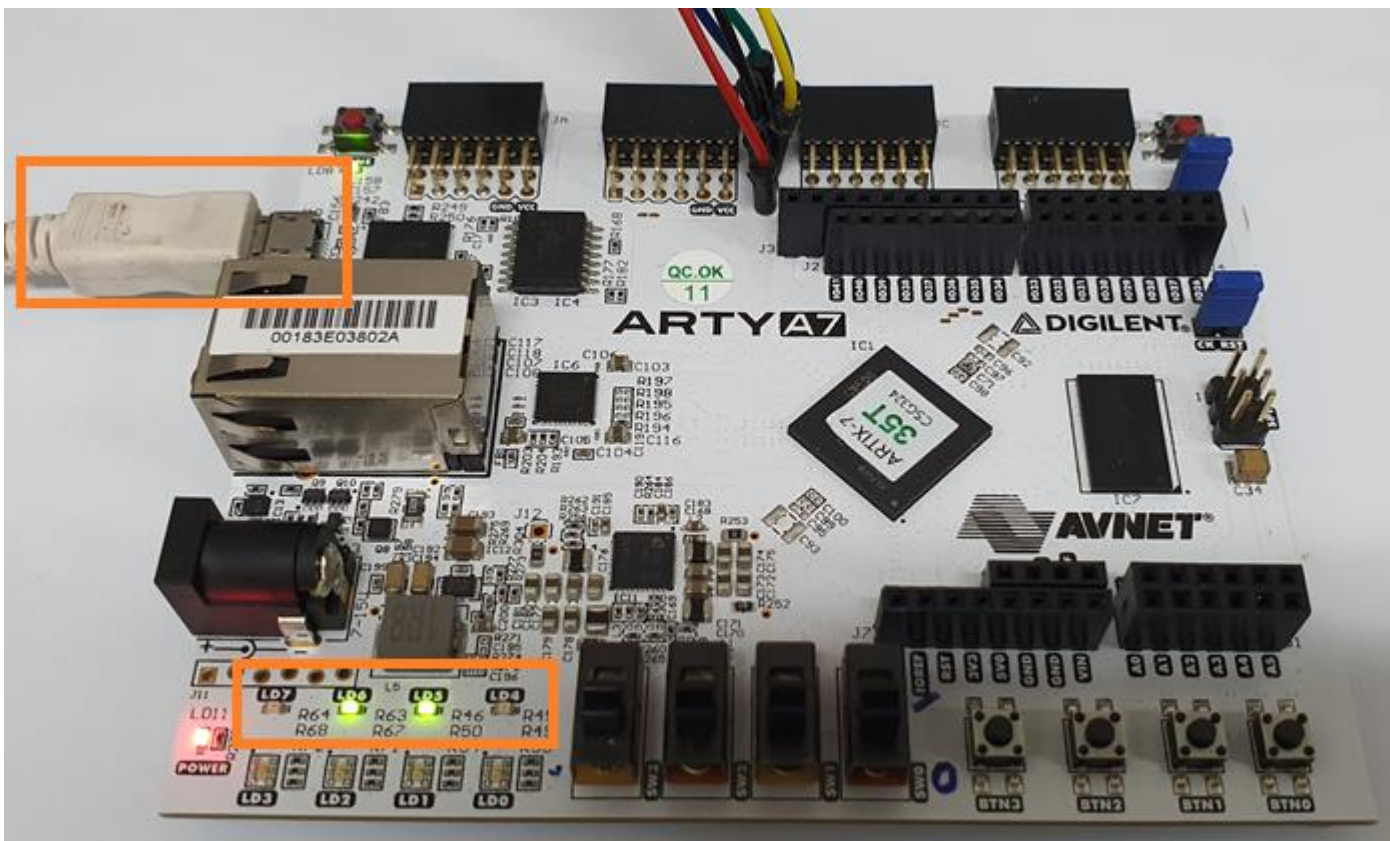
프로그램이 다운로드 됩니다. 우측 하단에 다운로드 과정이 %로 표시됩니다. 모두 다운로드가 되면 디버깅 모드가 되고 main()함수의 시작부분에 커서가 위치합니다.



Resume 아이콘()을 클릭하면 프로그램이 동작합니다. WinIDT 프로그램을 실행하고 Comport를 Open하면 메시지가 1초마다 출력됩니다.



또한 LD4 ~ LD7이 0.5초 간격으로 On/Off 됩니다.



기본적인 동작이 완료 되었습니다. led_counter 모듈에서 구현한 LED on/off 동작과 Embedded sw에서 구현한 1초마다 메시지를 표시하는 것이 완료 되었습니다.

이번 장에서는 결과를 디버그 모드로 확인하였습니다. FPGA 내부의 sram에 프로그램을 다운로드 하였기 때문에 전원을 Off 하면 데이터가 사라집니다. 다음장에서는 외부 Serial Flash에 프로그램을 다운로드 하는 2가지 방법을 설명합니다. Arty A7 35T 보드에는 128Mbits Serial Flash가 장착되어 있어서 이곳에 프로그램을 다운로드 할 수 있습니다.



4.3 Flash에 프로그램 다운로드

이번 장에서는 Flash에 프로그램을 다운로드하는 2가지 방법을 설명합니다. 첫번째 방법은 vitis에서 bit 파일을 생성해서 다운로드 하는 방법이고, 두번째 방법은 elf 파일(Embedded sw 결과 파일)을 vivado에서 추가해서 다운로드 하는 방법입니다.

4.3.1 vitis에서 다운로드

vitis를 종료합니다. vivado에서 File - Close Project를 클릭해서 Project를 Close 합니다. 기본 Template 폴더 (BlazeTop)를 복사한 후 폴더 이름을 BlazeTop_prog_flag 로 변경합니다.

이름	수정한 날짜	유형	크기
BlazeTop	2023-08-10 오후 1:44	파일 폴더	
BlazeTop_debug	2023-08-10 오후 2:24	파일 폴더	
BlazeTop_prog_flash	2023-08-10 오후 3:10	파일 폴더	

vivado에서 Open Project로 BlazeTop_prog_flash내의 프로젝트를 Open 합니다. Tools - Launch Vitis IDE를 클릭해서 vitis를 실행합니다. 이후 4.2장에서 진행했던 것과 동일하게 진행합니다. Project Build 까지 진행합니다.

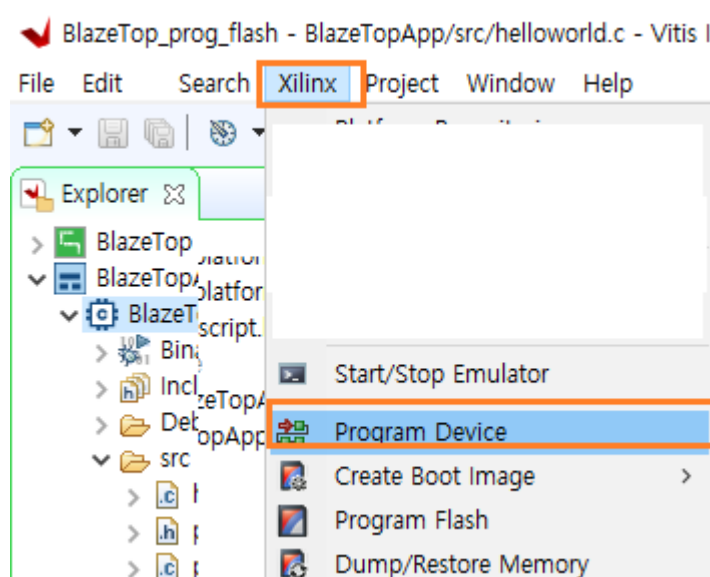
vitis 실행 시, workspace를 BlazeTop_prog_flash 로 하시길 바랍니다.

helloworld.c를 수정 후 Project를 Build 합니다.

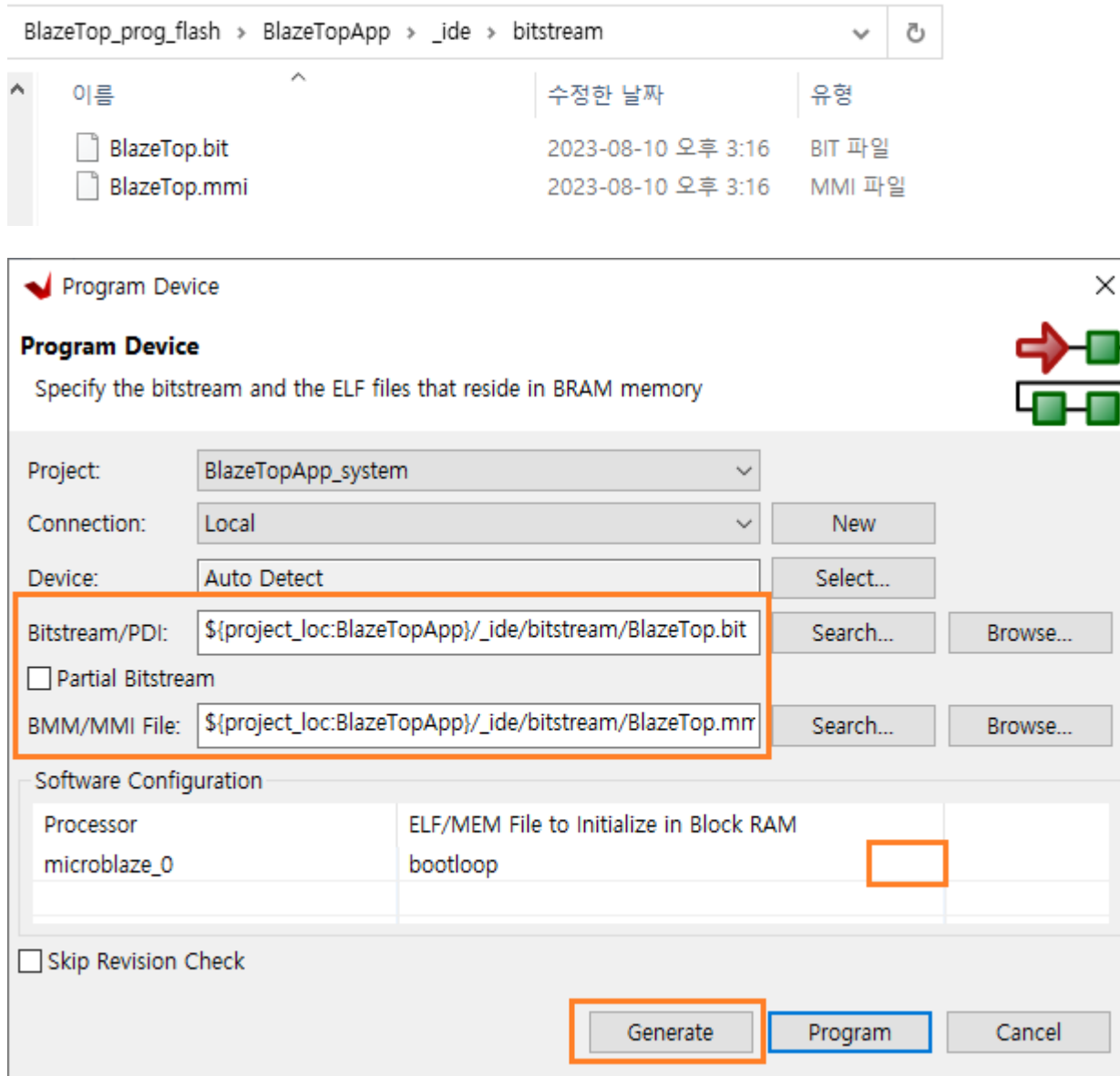
Flash에 다운로드할 bit 파일을 생성합니다. 이 파일은 3가지 파일이 포함됩니다.

- 1) BlazeTop.bit : vivado 에서 생성한 bit 파일
- 2) mmi : vivado에서 생성한 memory map information 파일
- 3) elf : vitis에서 생성한 embedded sw binary 파일

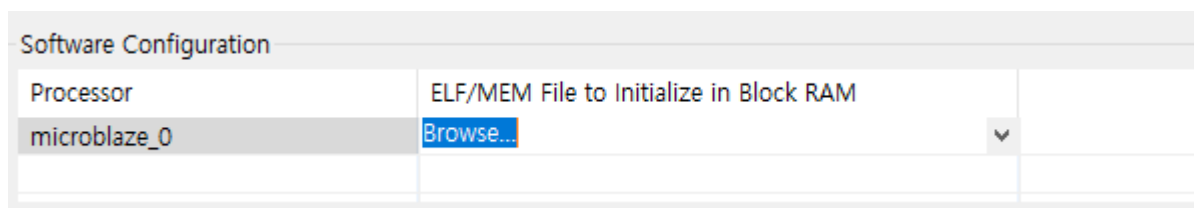
bit 파일을 생성하기 위하여 Xilinx - Program Device를 클릭합니다.



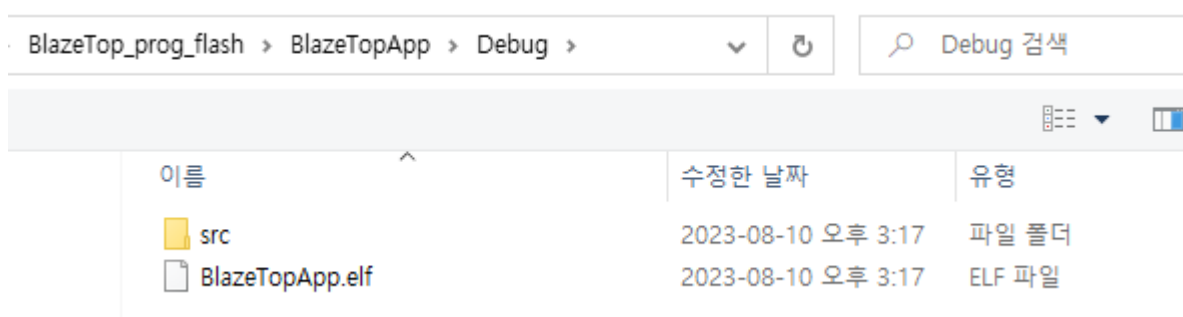
기본적으로 bit 파일과 mmi 파일은 선택되어 있습니다. 혹 파일이 보이지 않으면 Browse...를 클릭해서 해당 파일을 선택하면 됩니다. 파일 경로는 아래와 같습니다.



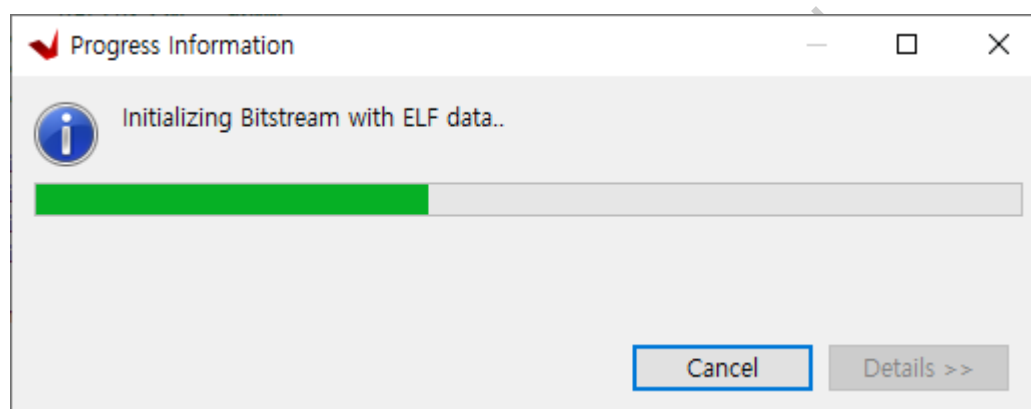
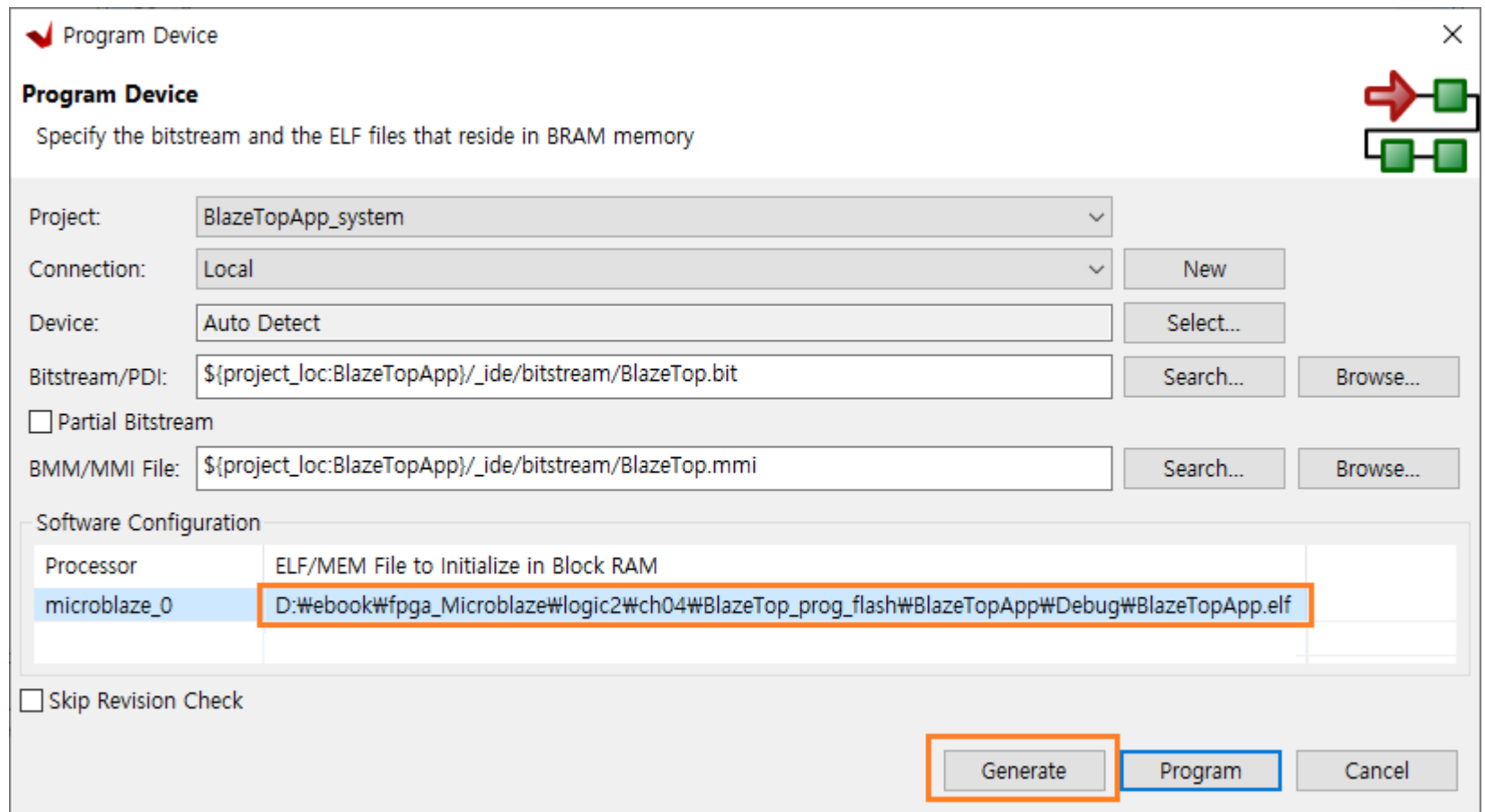
elf 파일을 선택하기 위하여 bootloop 우측을 클릭하면 리스트 박스가 나옵니다. 목록중에 Browse... 를 선택합니다.



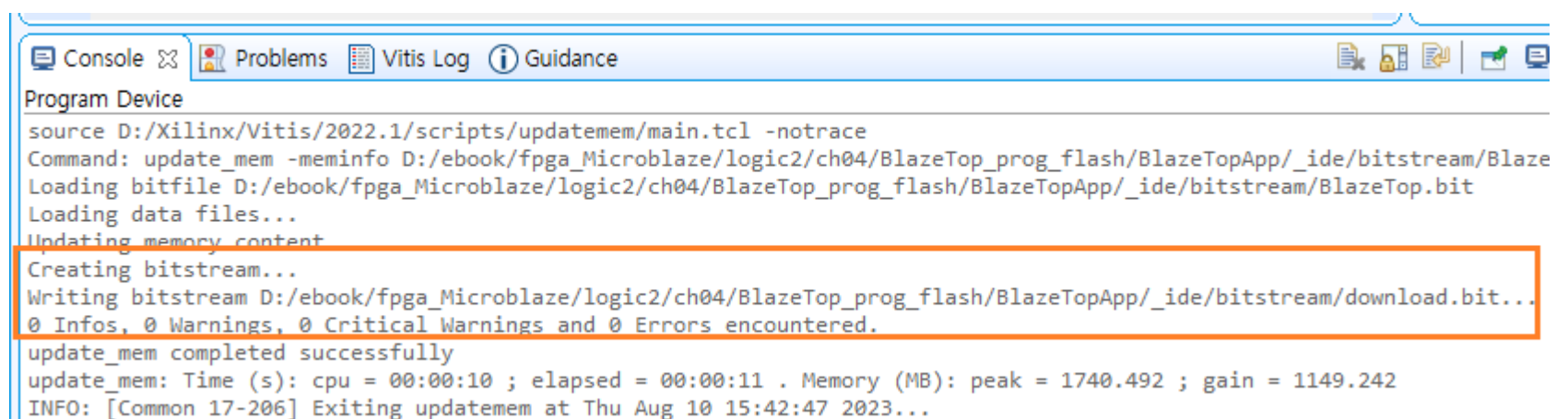
그리고 Generate를 클릭하면 elf 파일을 Open 하는 윈도우가 나타납니다. elf 파일을 선택해서 open 합니다. elf 파일은 아래 경로에 있습니다.



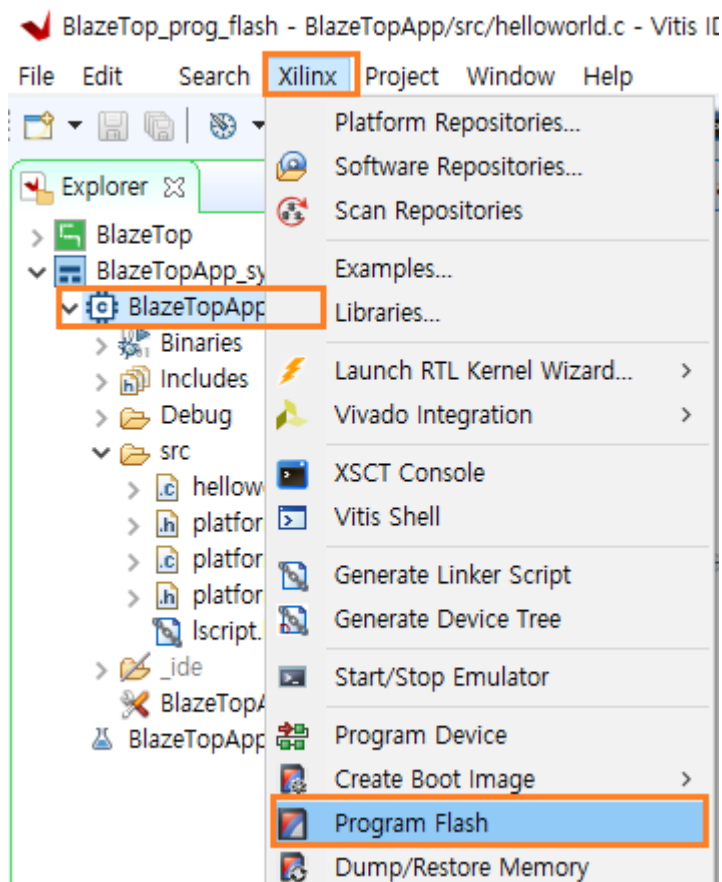
아래와 같이 elf 파일이 선택되었으면 다시 Generate를 클릭합니다. 그러면 download.bit 파일이 생성됩니다.



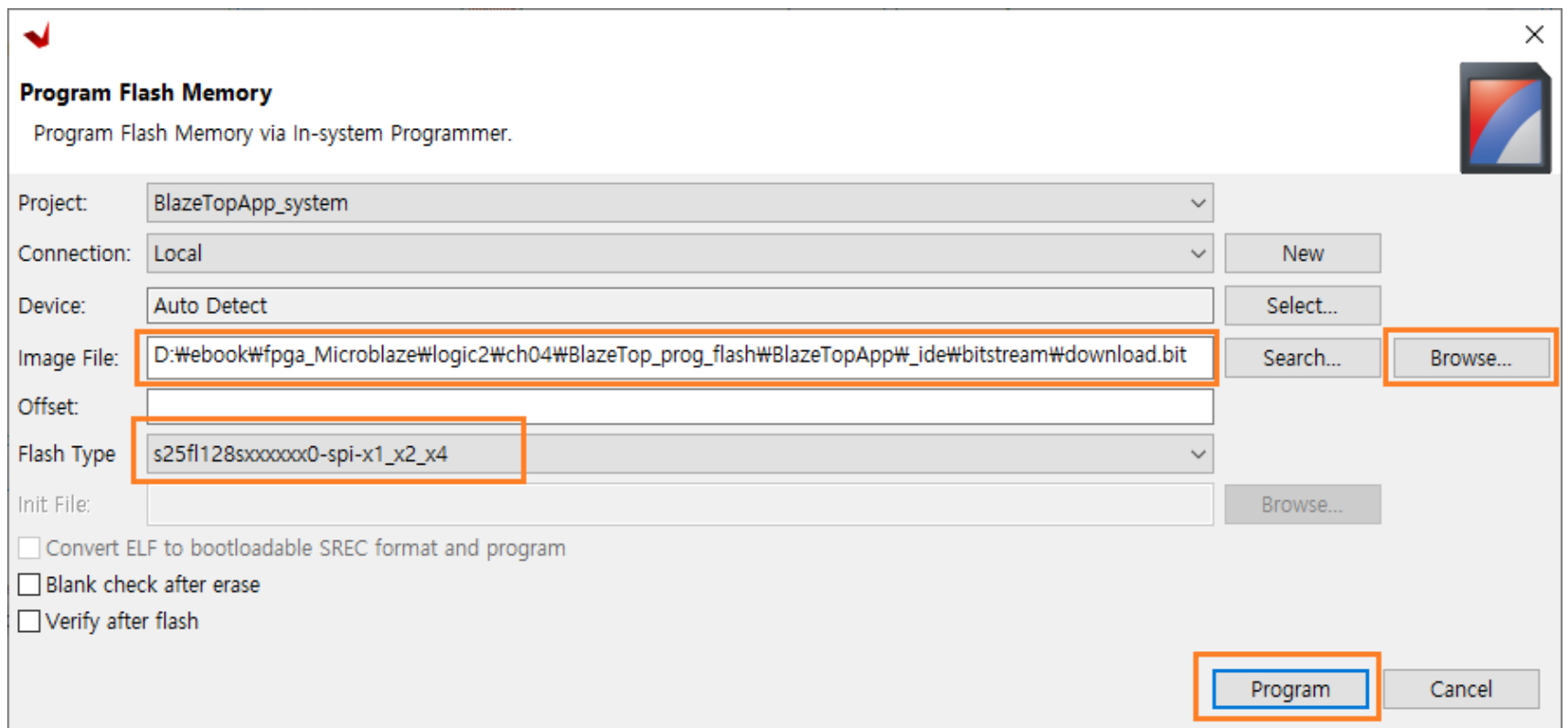
하단 Console 윈도우를 보면 download.bit 파일이 생성된 것을 알 수 있습니다.



이제 보드와 PC를 연결하고 Flash에 다운로드합니다. 보드와 PC를 연결한 후, Xilinx - Program Flash를 클릭합니다.

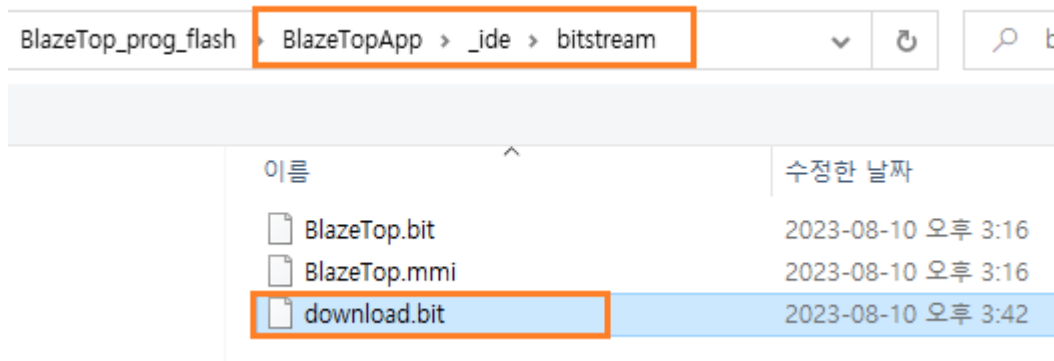


아래 윈도우에서 Image File 옆의 Browse...를 클릭해서 download.bit 파일을 Open 합니다. Flash Type은 s25fl128xxxxx0-spi-x1_x2_x4를 선택합니다. (보드를 연결하지 않으면 Flash Type : <none>으로 표시됩니다. 반드시 보드를 연결 후 실행해야 합니다) (보드 버전에 따라 mt25ql128 인 경우도 있습니다)

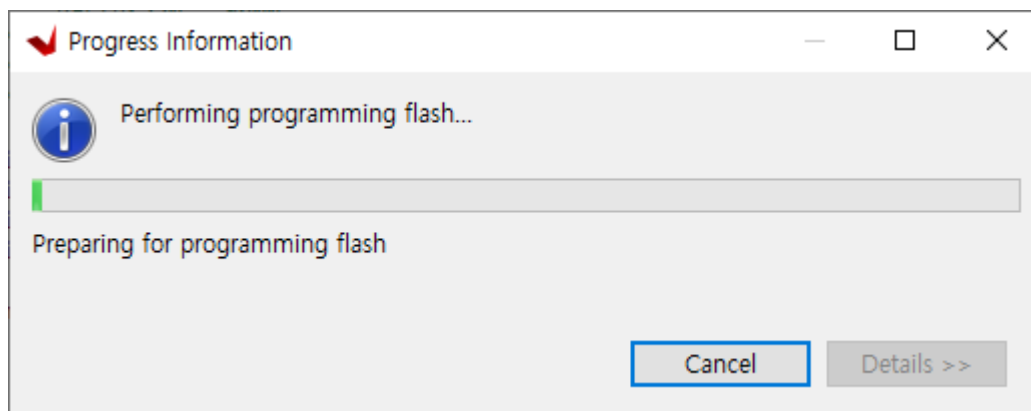


Program 버튼을 클릭하면 Flash에 다운로드가 됩니다.

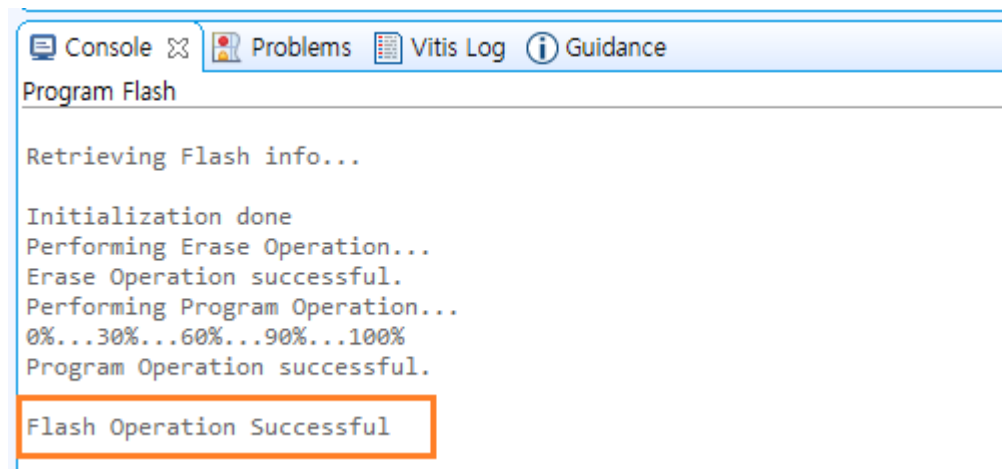
download.bit 파일은 아래 경로에 있습니다.



Erase - Program 과정을 거치며 Flash에 다운로드가 됩니다.



완료되면 Console에 메시지가 나타납니다.



보드의 전원을 Off (케이블 제거)후에 다시 전원을 인가하면, LD4 - LD7이 0.5초마다 on/off 되고, WinIDT에도 동일하게 메시지가 표시되는 것을 확인할 수 있습니다.

4.3.2 elf 파일을 vivado에서 추가 후 다운로드

이번 장에서는 vitis에서 생성한 elf 파일을 vivado의 block design에 추가하여 bitstream을 생성하고 다운로드 하는 방법을 설명합니다. 동일하게 vitis를 종료하고, vivado에서 project를 close 합니다. BlazeTop 폴더를 복사해서 이름을 BlazeTop_elf로 변경하고, vivado에서 다시 BlazeTop_elf 폴더의 Project를 Open 합니다.

vitis를 실행하고(workspace를 BlazeTop_elf 로 설정) 앞장과 동일하게 프로젝트를 생성하고 Build까지 진행합니다.

helloworld.c 를 아래와 같이 수정하겠습니다.

```

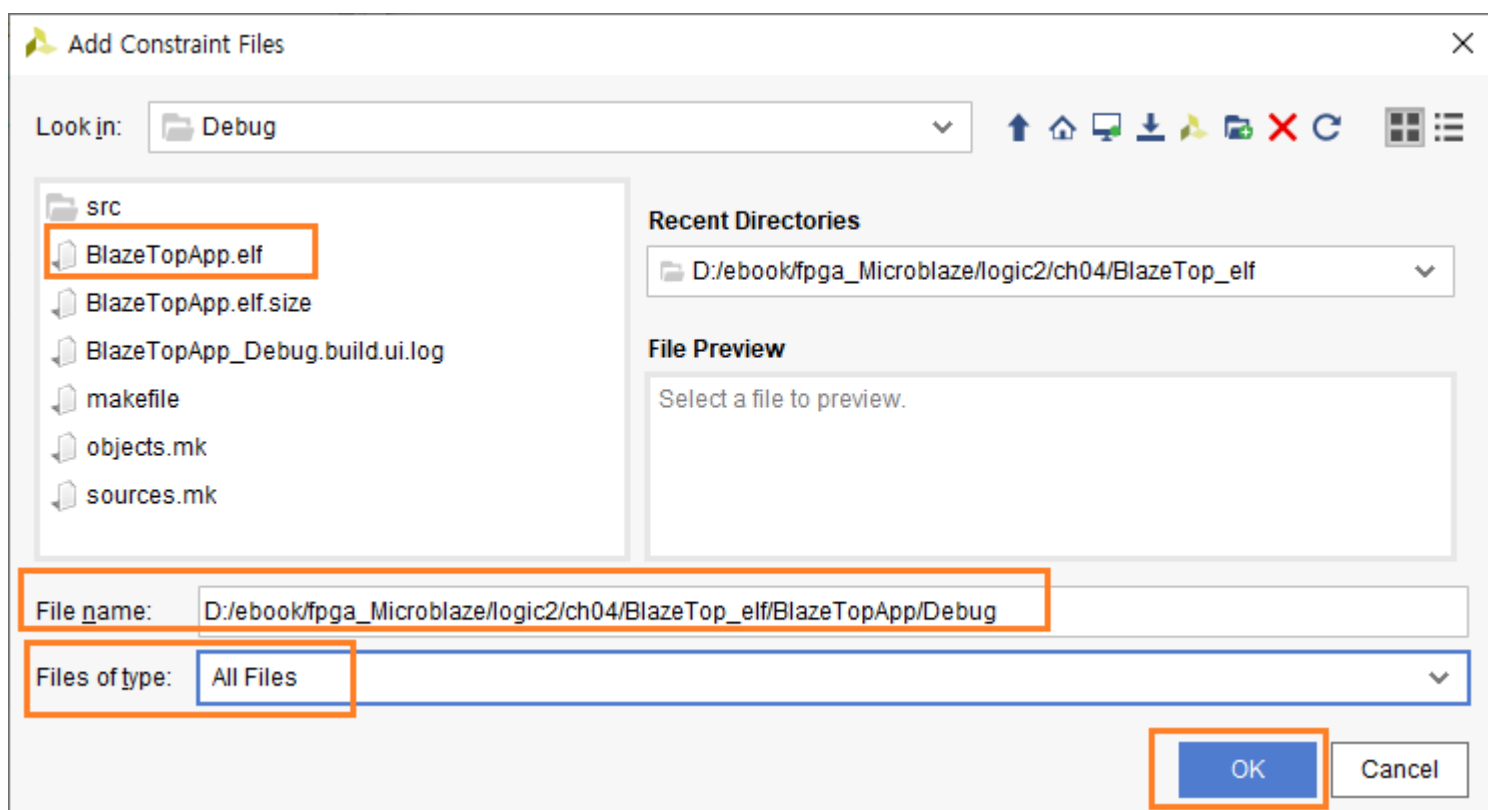
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53 int main()
54 {
55     int cnt = 0;
56
57     init_platform();
58
59     print("Hello World ..\r\n");
60     while(1)
61     {
62         xil_printf("ELF - count : %d \r\n", cnt++);
63         sleep(1);
64     }
65     cleanup_platform();
66     return 0;
67 }

```

62 라인의 메시지에 “ELF - “를 추가하였습니다. Project를 Build 해서 elf 파일을 생성합니다.

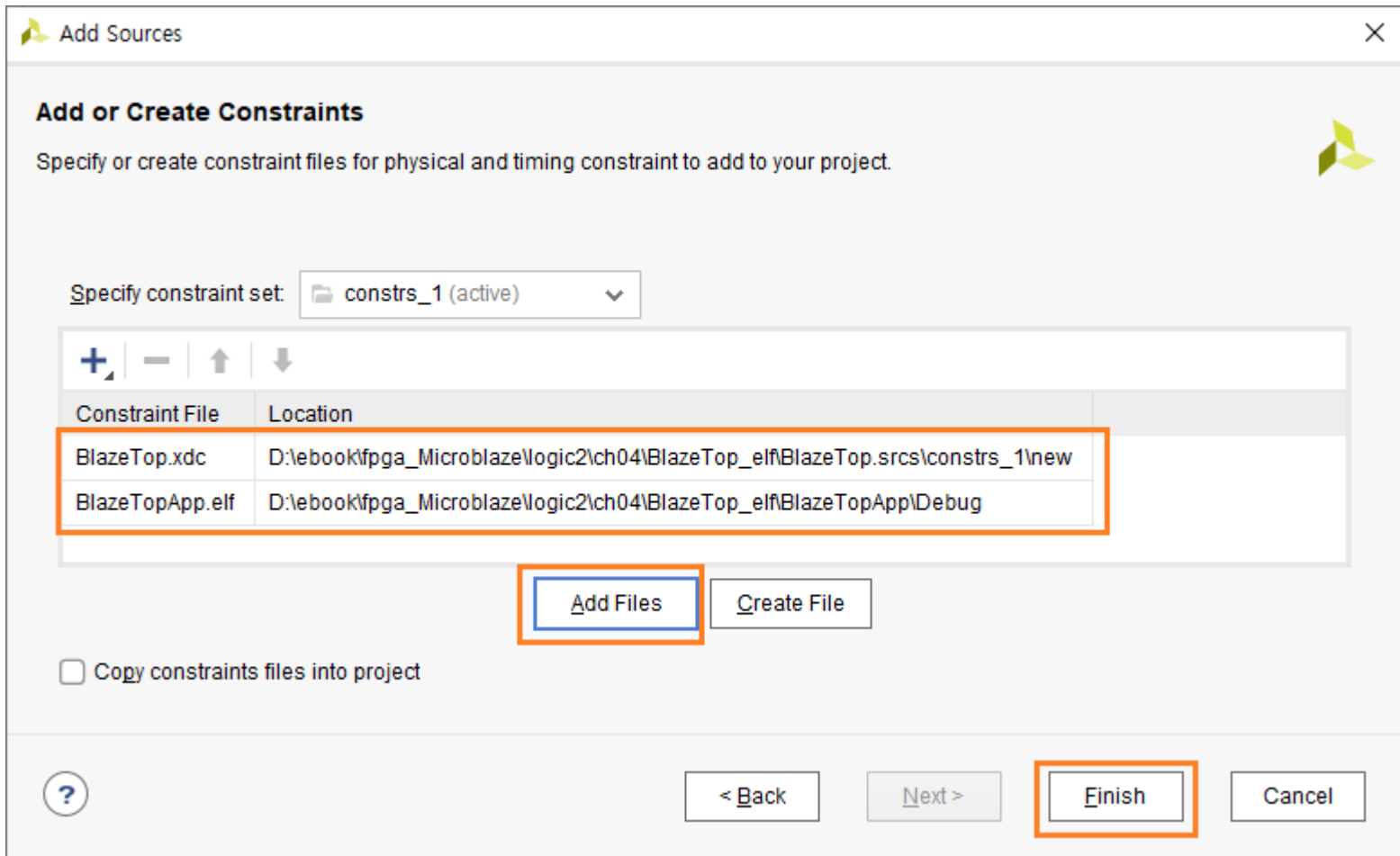
이제 생성된 elf 파일을 vivado에 추가하도록 하겠습니다.

vivado에서 Constraints - 우클릭 - Add Sources를 클릭해서 vitis에서 생성한 elf 파일을 추가합니다.

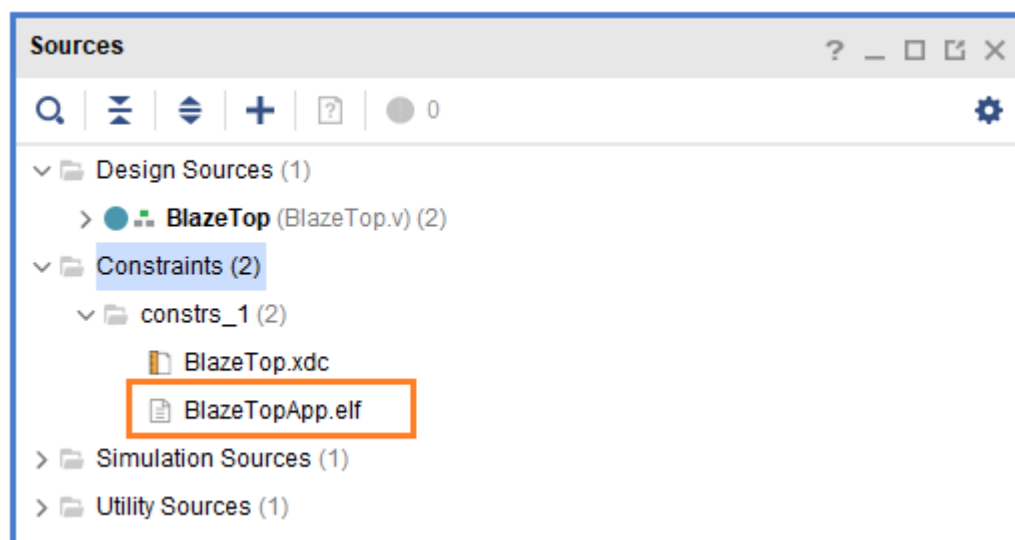


Files of type : 을 All Files 로 변경하고, elf 파일이 있는 위치로 이동해서 BlazeTopApp.elf 파일을 선택하고 OK를 클릭합니다.

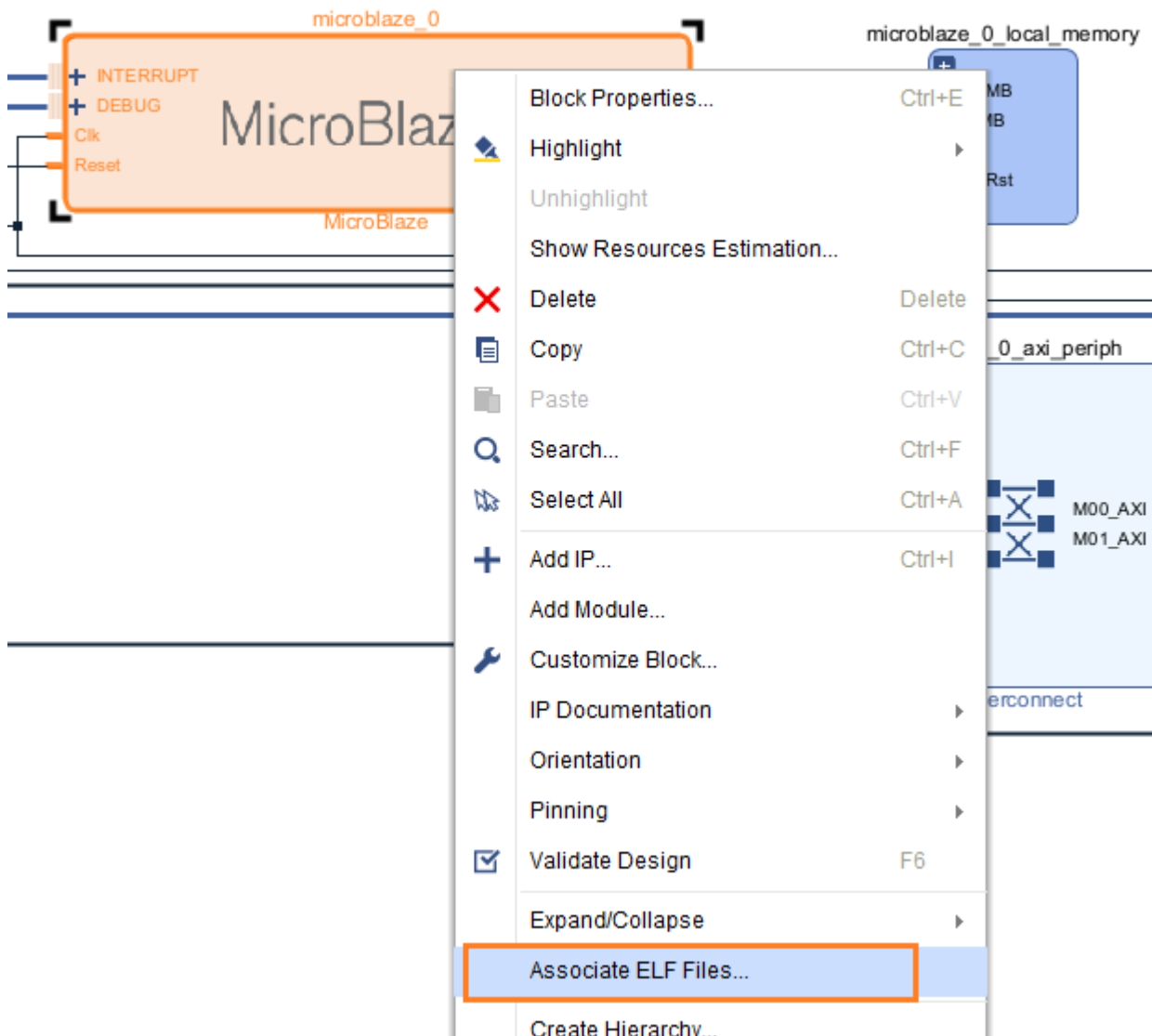
2개의 파일이 추가된 것을 알 수 있습니다. Finish를 클릭합니다.



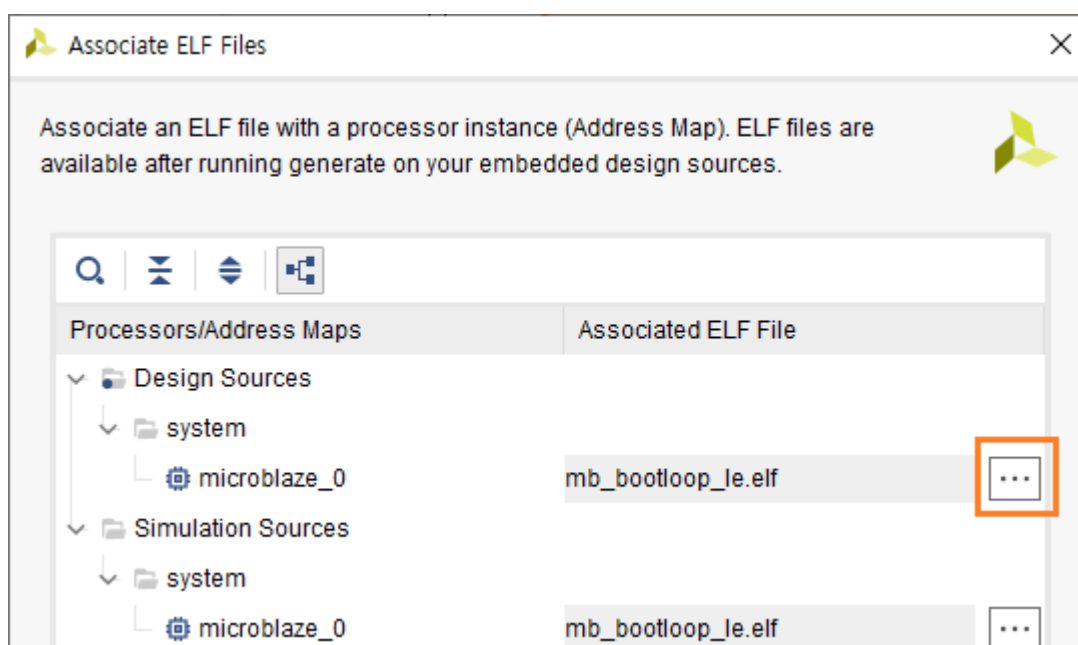
아래는 elf 파일이 추가된 모습을 보여줍니다.



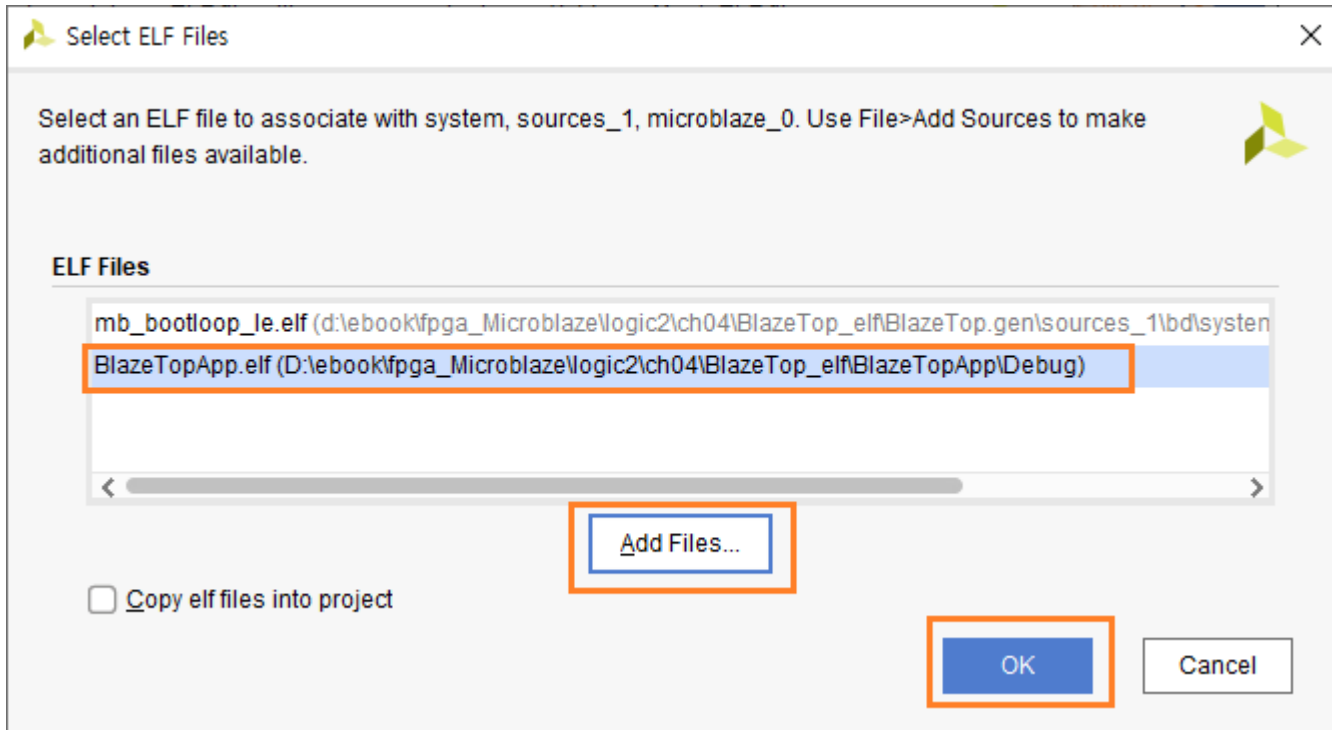
이제 MicroBlaze Processor 안에 elf 파일을 추가합니다. Open Block Design을 클릭하면 Diagram 윈도우가 나타납니다. microblaze_0를 선택하고 우클릭 하면 메뉴가 나타납니다. Associate ELF Files... 를 클릭합니다.



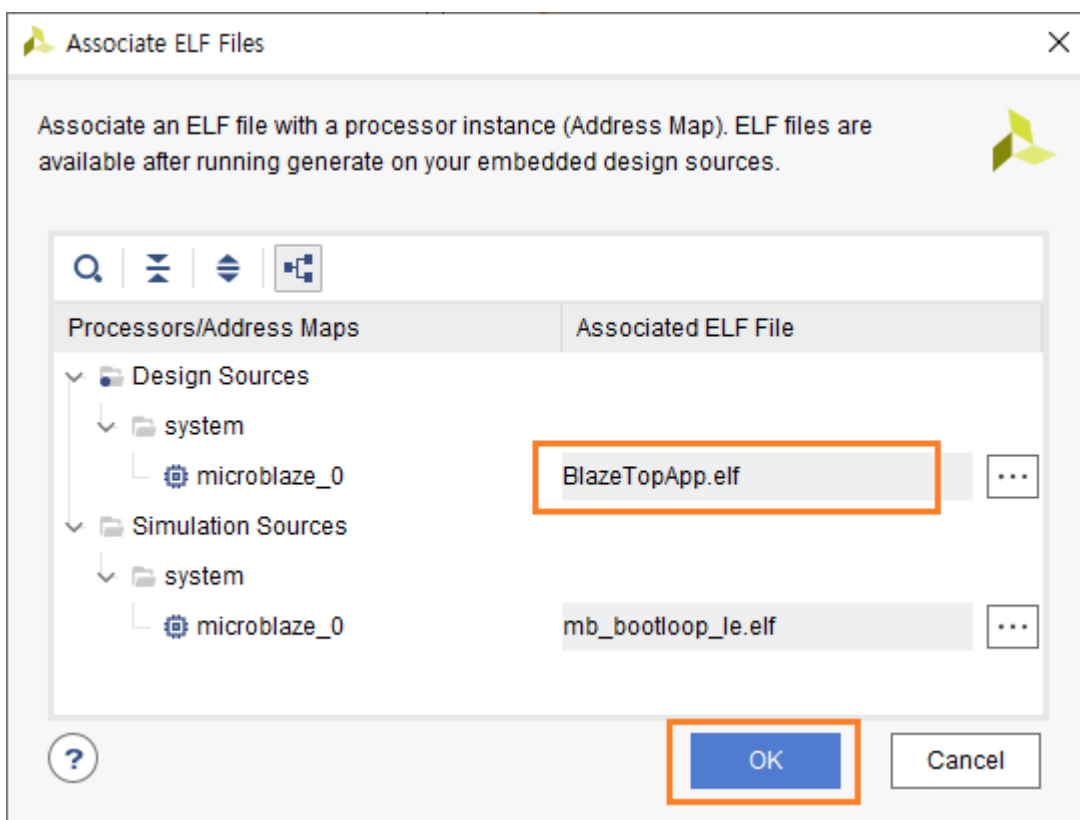
Design Sources - system - microblaze_0 옆의 “...”을 클릭해서 elf 파일을 설정합니다.



Add Files... 를 클릭해서 elf 파일을 선택하고, OK를 클릭합니다.

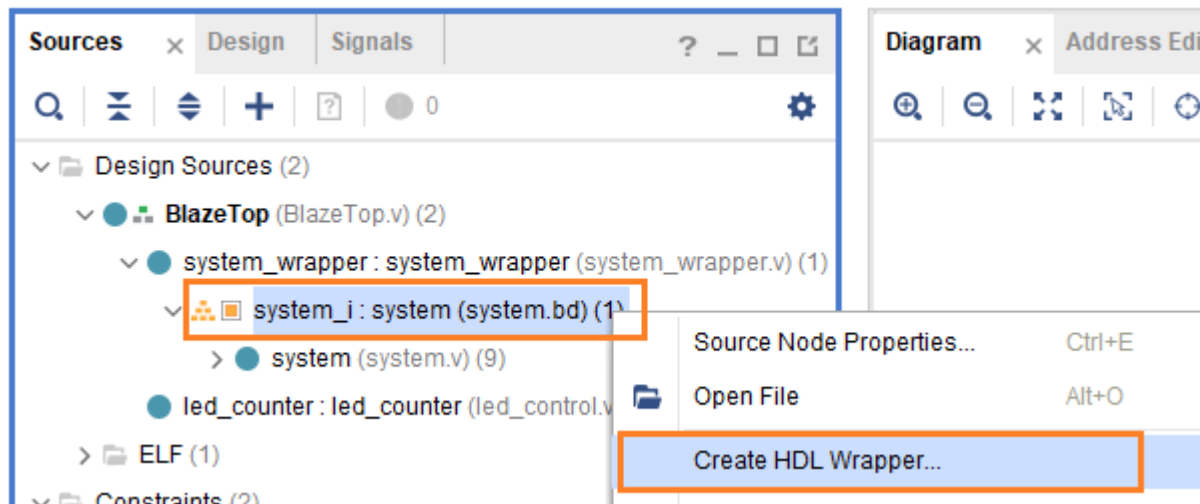


BlazeTopApp.elf 파일로 변경됩니다. OK를 클릭합니다.



Block Design이 변경되었습니다. 아래와 같은 순서로 Bitstream을 생성합니다.

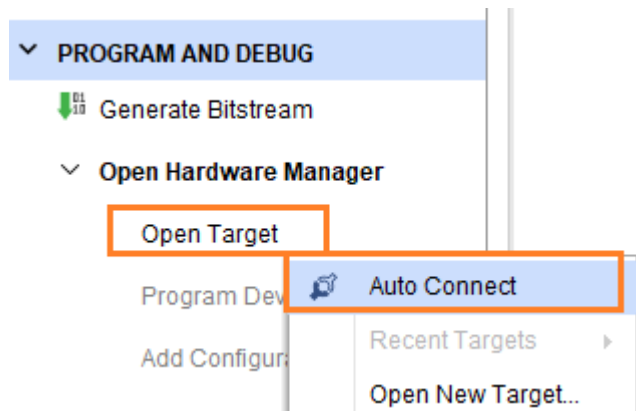
- 1) validate design을 클릭해서 에러를 확인합니다.
- 2) Create HDL Wrapper... 를 클릭해서 wrapper 파일을 생성합니다.



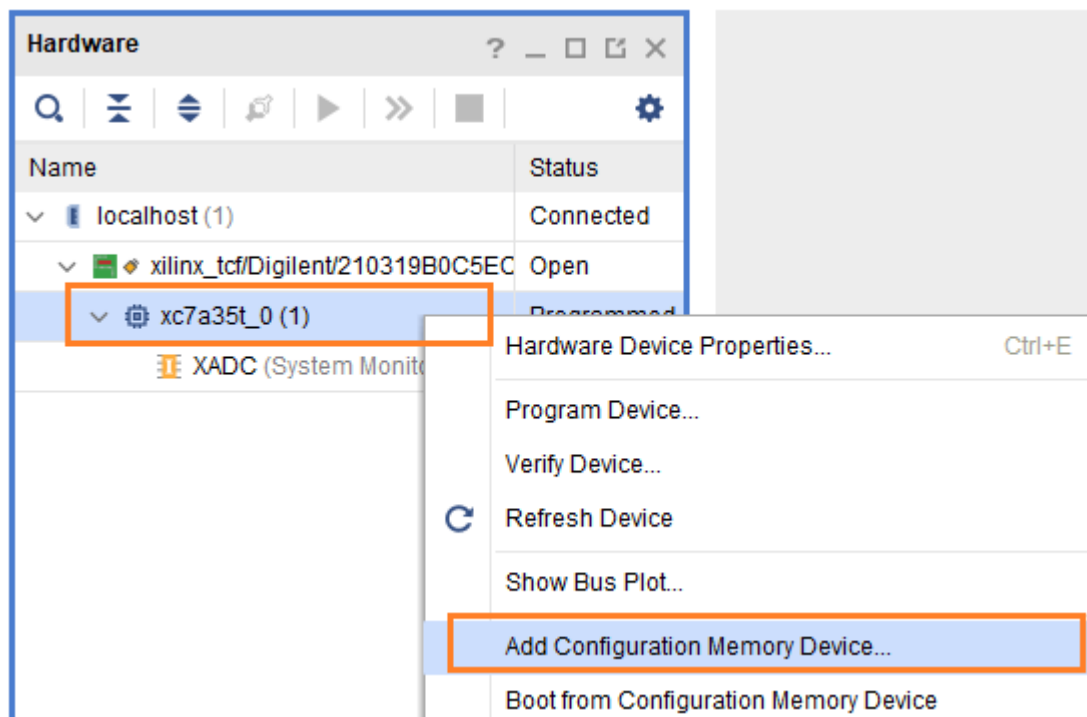
- 3) Generate Bitstream을 클릭해서 Bitstream을 생성합니다.

Bitstream이 생성되면 보드에 케이블을 연결하고 bitstream 파일을 다운로드합니다.

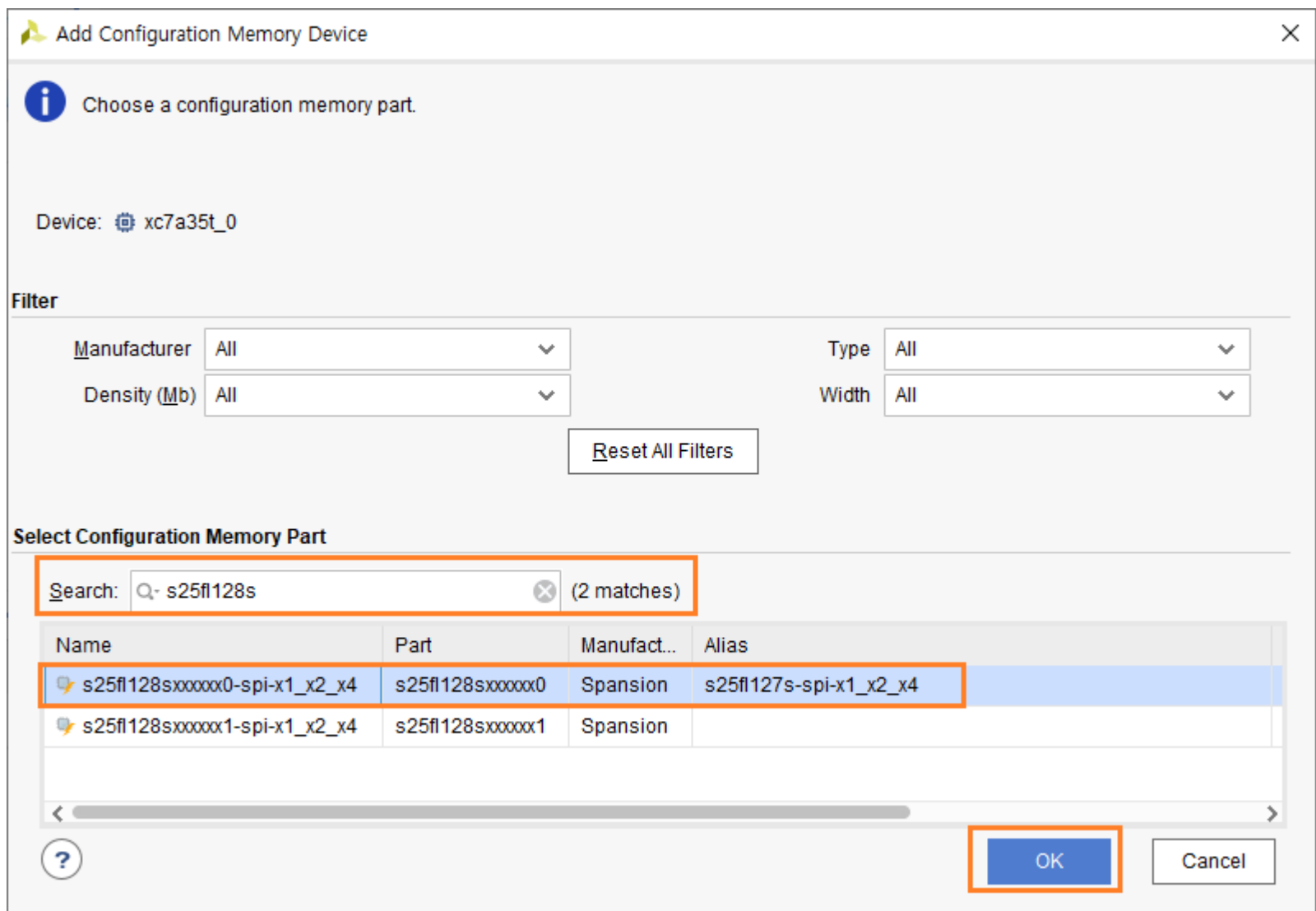
- 1) PROGRAM AND DEBUG - Open Hardware Manager를 클릭합니다.
- 2) open Target - Auto Connect를 클릭합니다.



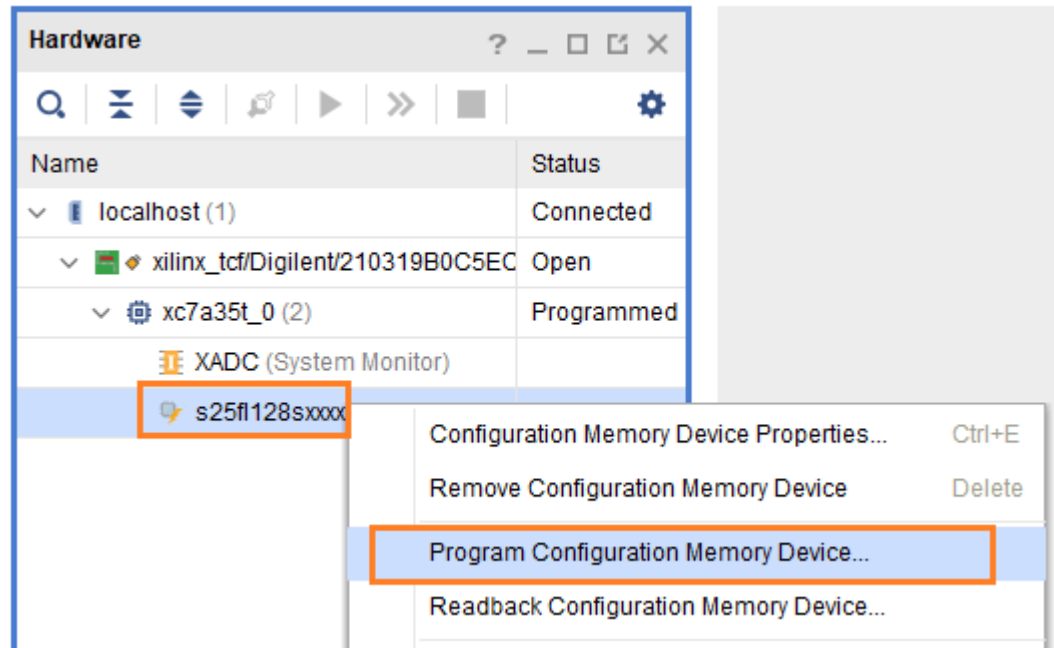
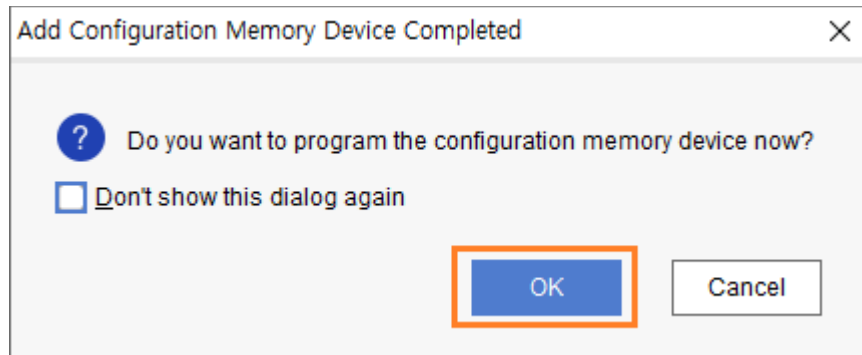
3) xc7a35t_0 - 우클릭 - Add Configuration Memory Device... 를 클릭합니다.



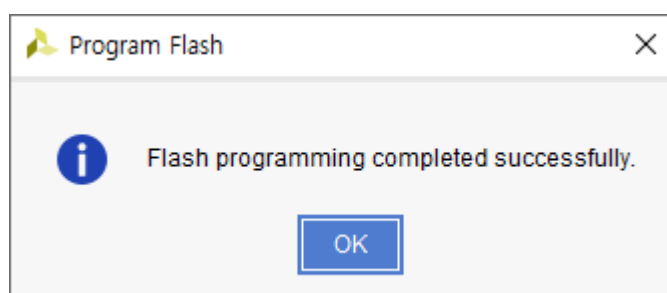
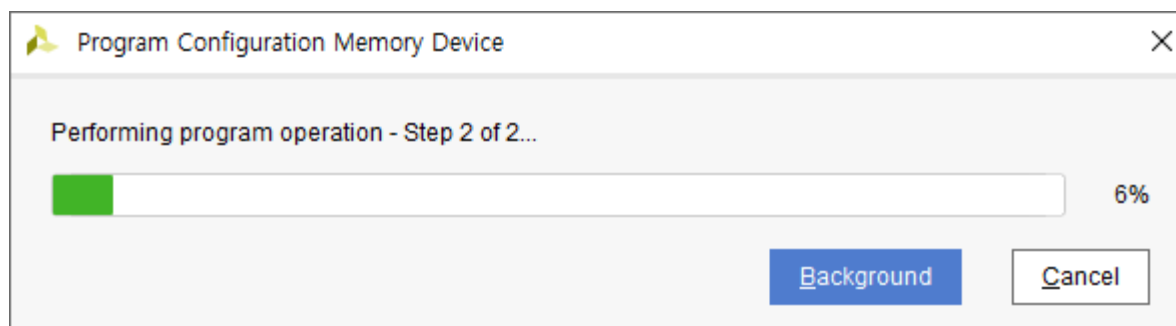
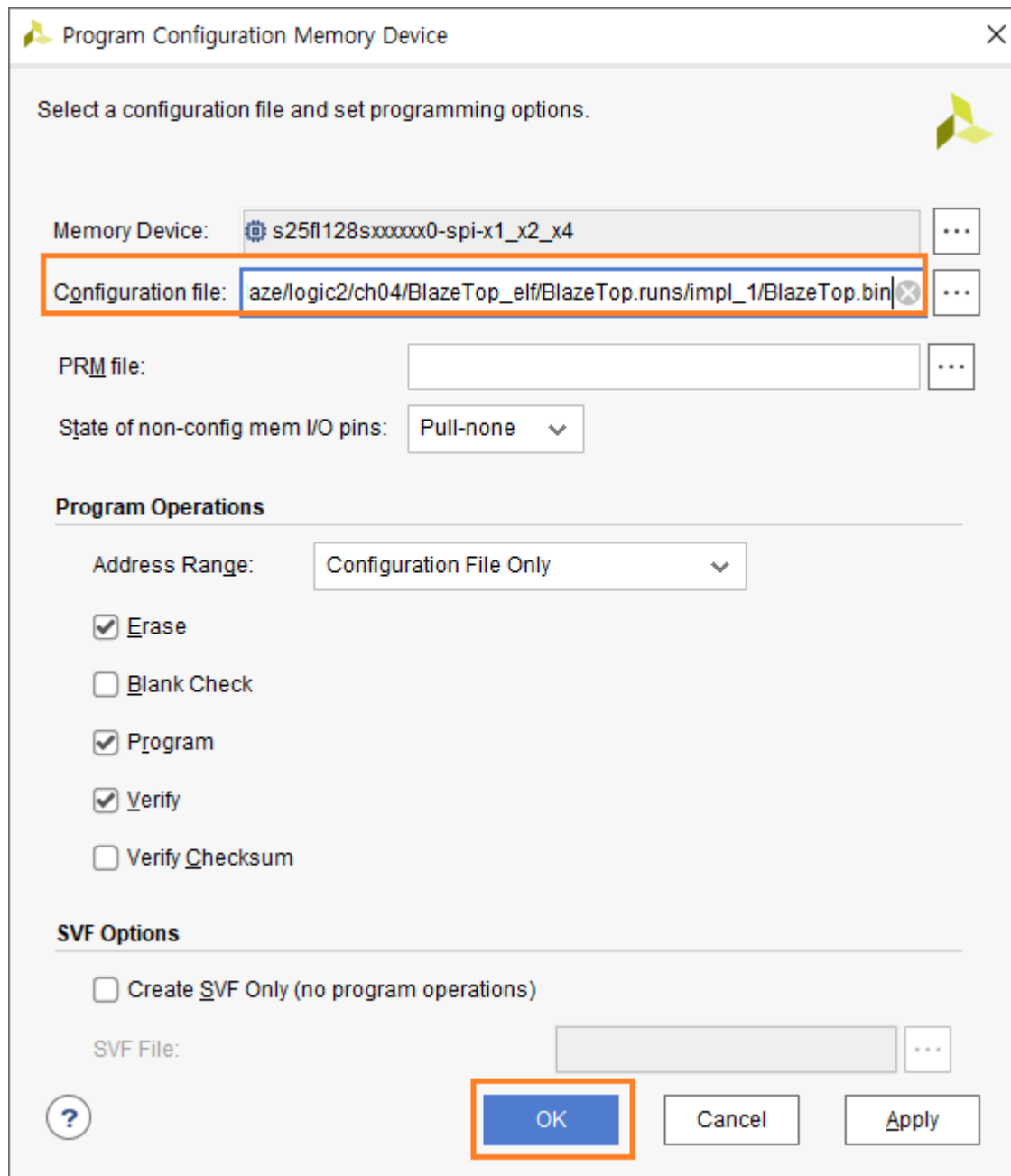
4) search : s25fl128s 을 입력하고, 첫번째 디바이스를 선택하고 OK를 클릭합니다. (보드 버전에 따라 mt25ql128 인 경우도 있습니다)



- 5) OK 버튼을 클릭하면 Program 윈도우가 나타납니다. (Cancel을 클릭하고 해당 Flash 선택 - 우클릭 - Program... 을 클릭 해도 됩니다)



6) Configuration 파일을 선택하고 (BlazeTop.bin) OK를 클릭하면 Flash에 Program을 진행합니다.



다운로드가 완료되면, Hardware Manager 윈도우를 닫고, 보드 전원을 Off 후 다시 On 하면 보드가 동작하는 것을 확인할 수 있습니다. 아래는 변경된 메시지가 나타나는 것을 보여줍니다.



```
COM10 open
ELF - count : 2
ELF - count : 3
ELF - count : 4
ELF - count : 5
ELF - count : 6
ELF - count : 7
r?jRELF - count : 1
```

이로써 Flash에 프로그램을 저장하는 2가지 방법을 알아보았습니다.

다음 장에서는 Processor의 Peripheral에 대해서 설명합니다.

다음장부터는 MicroBlaze 에 Peripheral을 추가하고, 궁극적으로는 사용자가 만든 Logic 파트와 MicroBlaze가 서로 인터페이스를 구성해서 MicroBlaze를 이용하여 사용자가 만든 Logic을 제어하는 것을 구현하도록 하겠습니다. FPGA에서 MicroBlaze를 사용하는 목적이 단순히 MCU를 만들어 사용하는 것이라면, 범용 MCU를 사용하는 것이 낫습니다. FPGA에서 MicroBlaze를 사용하는 목적은 사용자가 만든 Logic을 제어하고, Logic 으로 구현할 수 없는 부분을 MicroBlaze 로 구현하는 것입니다. FPGA에서 MCU를 포팅(or IP를 생성)해서 사용하는 궁극적인 목적은 HW로 구현하는 부분과 SW로 구현하는 부분을 나누어서 구현하고 서로 간에 인터페이스를 구현하여 외부에 추가적인 부품을 사용하지 않는 SOC (System On Chip)을 구현하는 것입니다. 다음장부터 이러한 기능들을 하나씩 구현해 가도록 하겠습니다. 내용이 좀 어려운 부분이 있을 수 있는데, 인내심을 가지고 몇 번에 걸쳐 내용을 자세히 보거나, 인터넷에서 추가적인 자료를 보시면 이해할 수 있을 것입니다.

5. MicroBlaze Peripheral 구현

이번 장에서는 MicroBlaze의 Peripheral 을 구현하도록 합니다. 모든 Peripheral을 다루지는 않고 필요한 부분 몇개를 다루도록 하겠습니다. 기본적인 몇 가지를 구현할 수 있으면 나머지는 내용들이 비슷해서 User Manual 등 다른 문서들을 참조하면 어렵지 않게 구현할 수 있습니다.

이번장에서 구현하는 Peripheral 들은 다음과 같습니다.

- ✓ GPIO - 1 : 4-bits output, 보드의 LD0(Blue) - LD3(Blue) On/Off
- ✓ GPIO - 2 : 1-bit input, External Interrupt - 1 구현, 보드의 BTN0 입력을 처리함.
- ✓ GPIO - 3 : 1-bit input, External Interrupt - 2 구현, 보드의 BTN1 입력을 처리함.
- ✓ GPIO - 4 : 2-bits input, BTN2,3 입력을 폴링 (Polling) 방식으로 처리함.
- ✓ Timer : sw timer, timer Interrupt 구현
- ✓ Uart : Rx Interrupt 처리

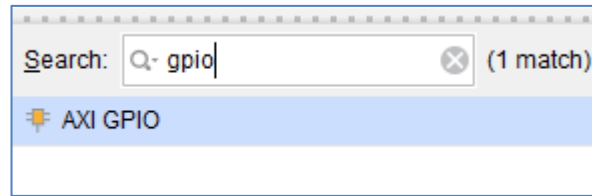
5.1 Block Design

4장에서 만든 기본 Template을 복사해서 사용합니다. ch05 폴더를 만들고 BlazeTop 폴더를 복사해서 폴더 이름을 BlazeTopPeri 로 합니다. vitis를 종료하고, vivado에 열린 project는 닫고, BlazeTopPeri 폴더내의 프로젝트를 Open 합니다. 이번장부터는 결과를 Debug Mode로 확인합니다.

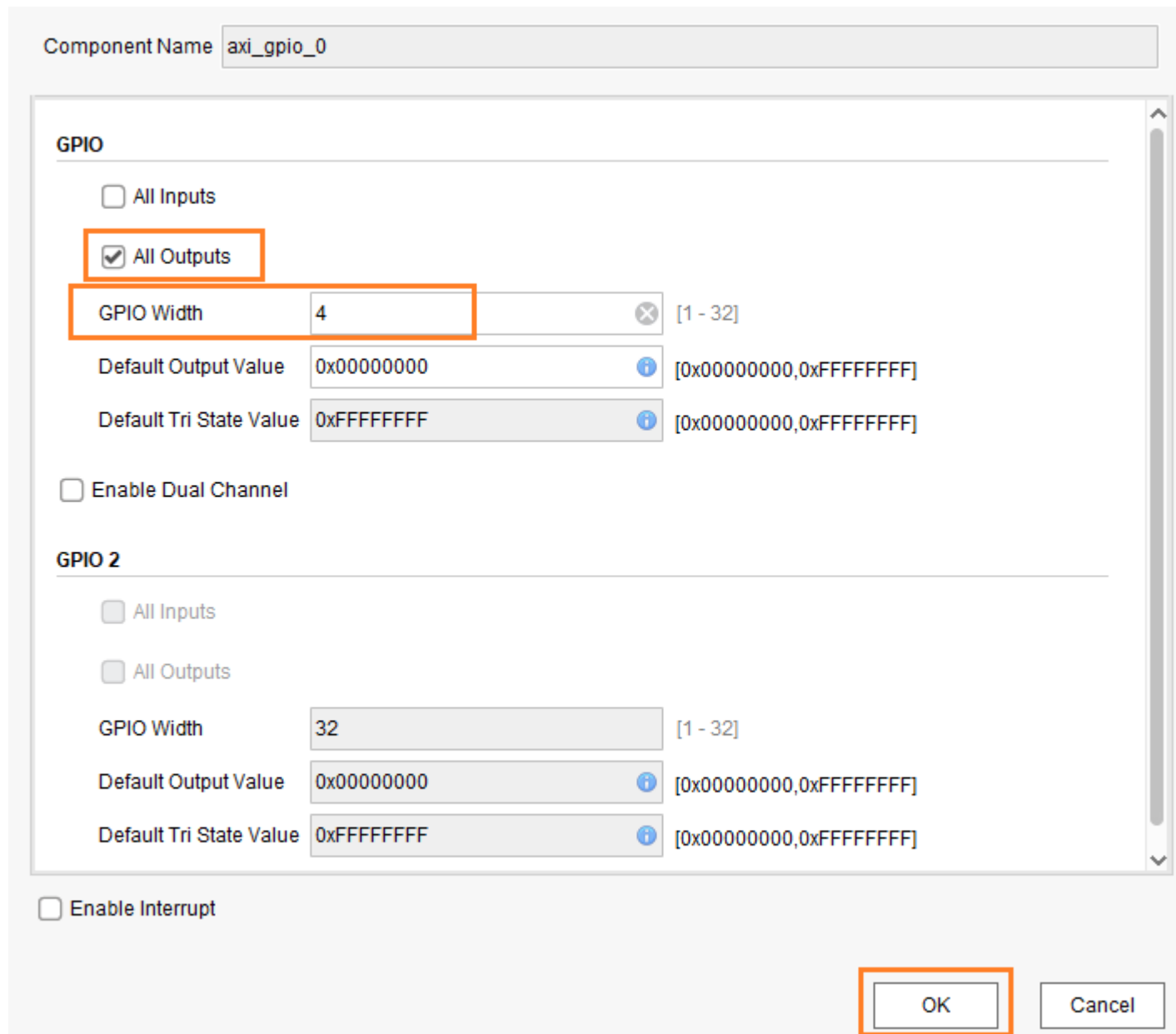
Open Block Design을 클릭하면 Diagram 윈도가 나타납니다. Processor와 Uart는 이미 구현이 되었습니다. GPIO, Timer를 추가합니다.

5.1.1 GPIO : LED_4bits

Add IP 클릭 후 나타나는 윈도우에서 gpio 를 입력하고 AXI GPIO를 Double Click 해서 GPIO를 추가합니다.



추가된 axi_gpio_0 를 Double Click 해서 아래와 같이 속성을 설정합니다. axi_gpio_0는 LD4(Blue) ~ LD7(Blue)의 On/Off를 제어합니다. Interrupt를 사용하지 않기 때문에 Enable Interrupt는 check 하지 않습니다. OK를 클릭합니다.



5.1.2 GPIO : btn0 (External Interrupt-1)

Add IP 클릭 후 나타나는 윈도우에서 gpio 를 입력하고 AXI GPIO를 Double Click 해서 GPIO를 추가합니다. 추가된 axi_gpio_1을 더블 클릭해서 속성을 설정합니다. BTN0의 입력을 인터럽트 방식으로 처리합니다. 인터럽트를 사용하기 위하여 Enable Interrupt를 Check 합니다. OK를 클릭합니다.

Component Name `axi_gpio_1`

GPIO

☒ All Inputs

☐ All Outputs

GPIO Width `1` [1 - 32]

Default Output Value `0x00000000` [0x00000000,0xFFFFFFFF]

Default Tri State Value `0xFFFFFFFF` [0x00000000,0xFFFFFFFF]

☐ Enable Dual Channel

GPIO 2

☐ All Inputs

☐ All Outputs

GPIO Width `32` [1 - 32]

Default Output Value `0x00000000` [0x00000000,0xFFFFFFFF]

Default Tri State Value `0xFFFFFFFF` [0x00000000,0xFFFFFFFF]

☒ Enable Interrupt

OK Cancel

5.1.3 GPIO : btn1 (External Interrupt-2)

Add IP 클릭 후 나타나는 윈도우에서 gpio 를 입력하고 AXI GPIO를 Double Click 해서 GPIO를 추가합니다. 추가된 axi_gpio_2을 더블 클릭해서 속성을 설정합니다. BTN0의 입력을 인터럽트 방식으로 처리합니다. 인터럽트를 사용하기 위하여 Enable Interrupt를 Check 합니다.

The screenshot displays the configuration window for the AXI GPIO component. The 'Component Name' is set to 'axi_gpio_2'. The window is divided into two sections: 'GPIO' and 'GPIO 2'.

GPIO Section:

- ☒ All Inputs
- ☐ All Outputs
- GPIO Width: 1 [1 - 32]
- Default Output Value: 0x00000000 [0x00000000, 0xFFFFFFFF]
- Default Tri State Value: 0xFFFFFFFF [0x00000000, 0xFFFFFFFF]
- ☐ Enable Dual Channel

GPIO 2 Section:

- ☐ All Inputs
- ☐ All Outputs
- GPIO Width: 32 [1 - 32]
- Default Output Value: 0x00000000 [0x00000000, 0xFFFFFFFF]
- Default Tri State Value: 0xFFFFFFFF [0x00000000, 0xFFFFFFFF]

At the bottom of the window, the ☒ Enable Interrupt checkbox is checked. The 'OK' button is highlighted with an orange box.

5.1.4 GPIO : btn3, 4 (General Input)

Add IP 클릭 후 나타나는 윈도우에서 gpio 를 입력하고 AXI GPIO를 Double Click 해서 GPIO를 추가합니다. 추가된 axi_gpio_3을 더블 클릭해서 속성을 설정합니다. 인터럽트를 사용하지 않고 Polling 방식으로 처리합니다.

Component Name: axi_gpio_3

GPIO

☒ All Inputs

☐ All Outputs

GPIO Width: 2 [1 - 32]

Default Output Value: 0x00000000 [0x00000000, 0xFFFFFFFF]

Default Tri State Value: 0xFFFFFFFF [0x00000000, 0xFFFFFFFF]

☐ Enable Dual Channel

GPIO 2

☐ All Inputs

☐ All Outputs

GPIO Width: 32 [1 - 32]

Default Output Value: 0x00000000 [0x00000000, 0xFFFFFFFF]

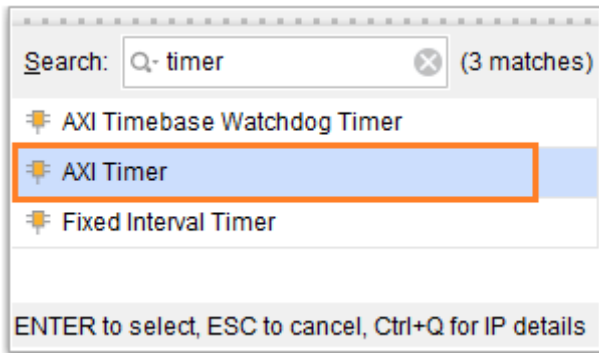
Default Tri State Value: 0xFFFFFFFF [0x00000000, 0xFFFFFFFF]

☐ Enable Interrupt

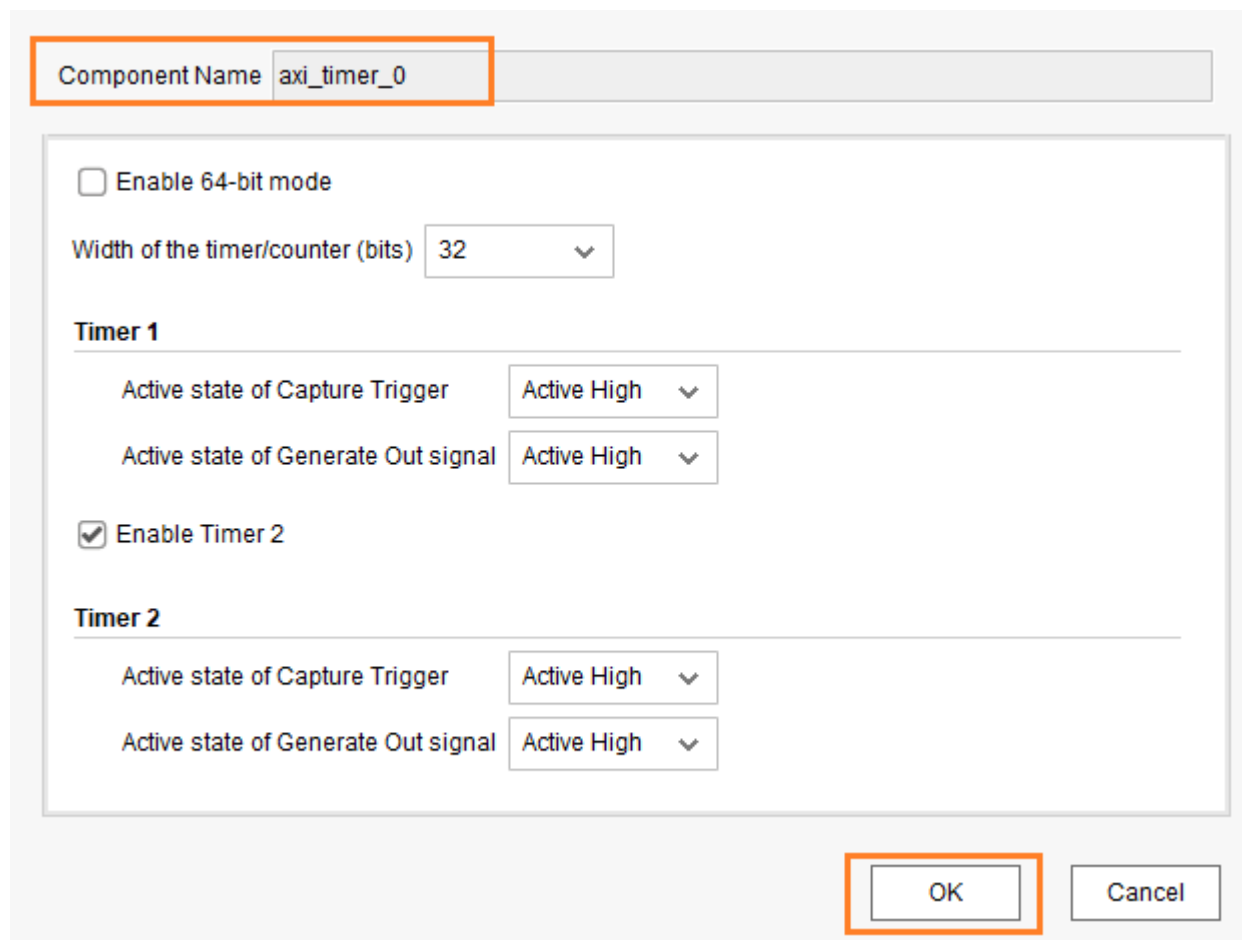
OK Cancel

5.1.5 Timer

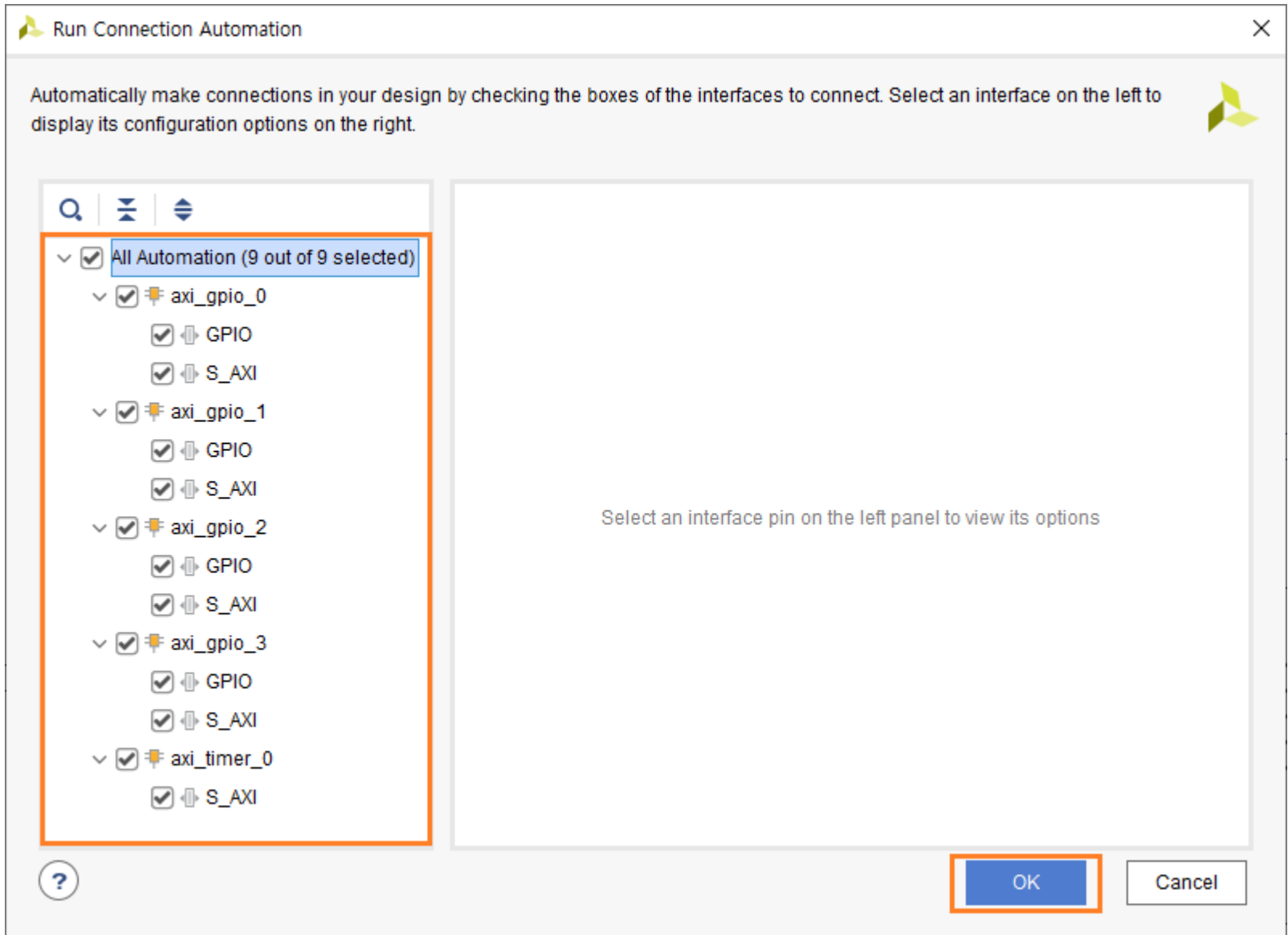
Add IP 클릭 후 나타나는 윈도우에서 timer 를 입력하고 AXI Timer를 Double Click 해서 Timer를 추가합니다.



추가된 axi_timer_0을 더블 클릭해서 속성을 설정합니다. Timer Block 은 기본설정을 사용합니다. 타이머 타임아웃은 Application SW 에서 설정합니다.

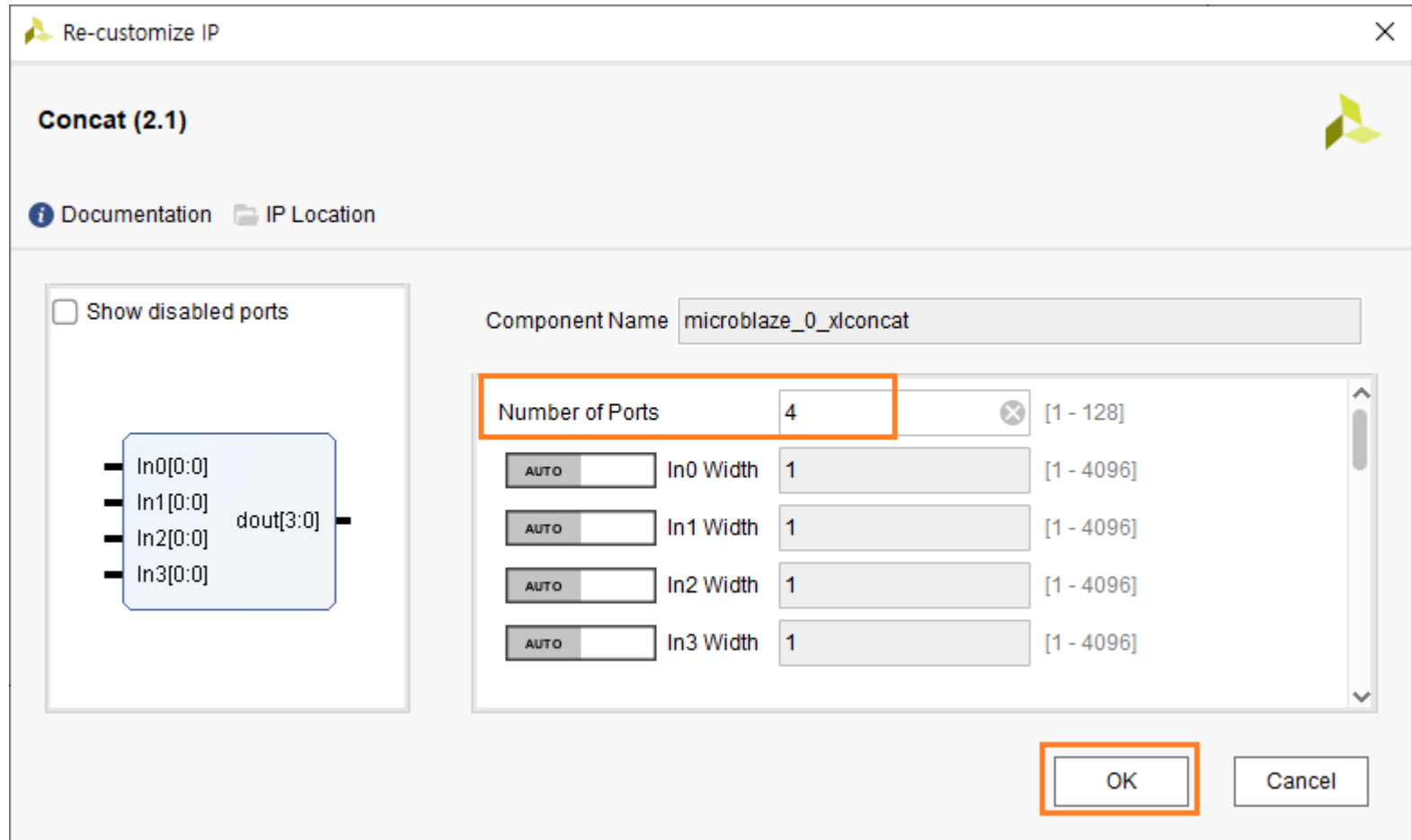


모든 필요한 Peripheral Block이 추가되었습니다. 상단의 “Run Connection Automation”을 클릭해서 연결을 완료합니다.
All Automation을 check 해서 모든 Block들을 선택합니다. OK를 클릭합니다.



인터럽트를 매뉴얼로 연결해야 합니다. 추가되어야 할 인터럽트는 3개 (axi_gpio_1, axi_gpio_2, axi_timer_0) 입니다. Uart 인터럽트는 4장에서 이미 연결하였습니다. Concat (microblaze_0_xlconcat) Block에 연결한 포트가 1개 밖에 여유가 없습니다. 2개를 더 추가해야 합니다. Concat 속성을 변경하기 위하여 더블 클릭합니다.

Number of Ports에 4를 입력하고 엔터를 입력하면 포트가 4개로 변경됩니다. OK를 클릭합니다.



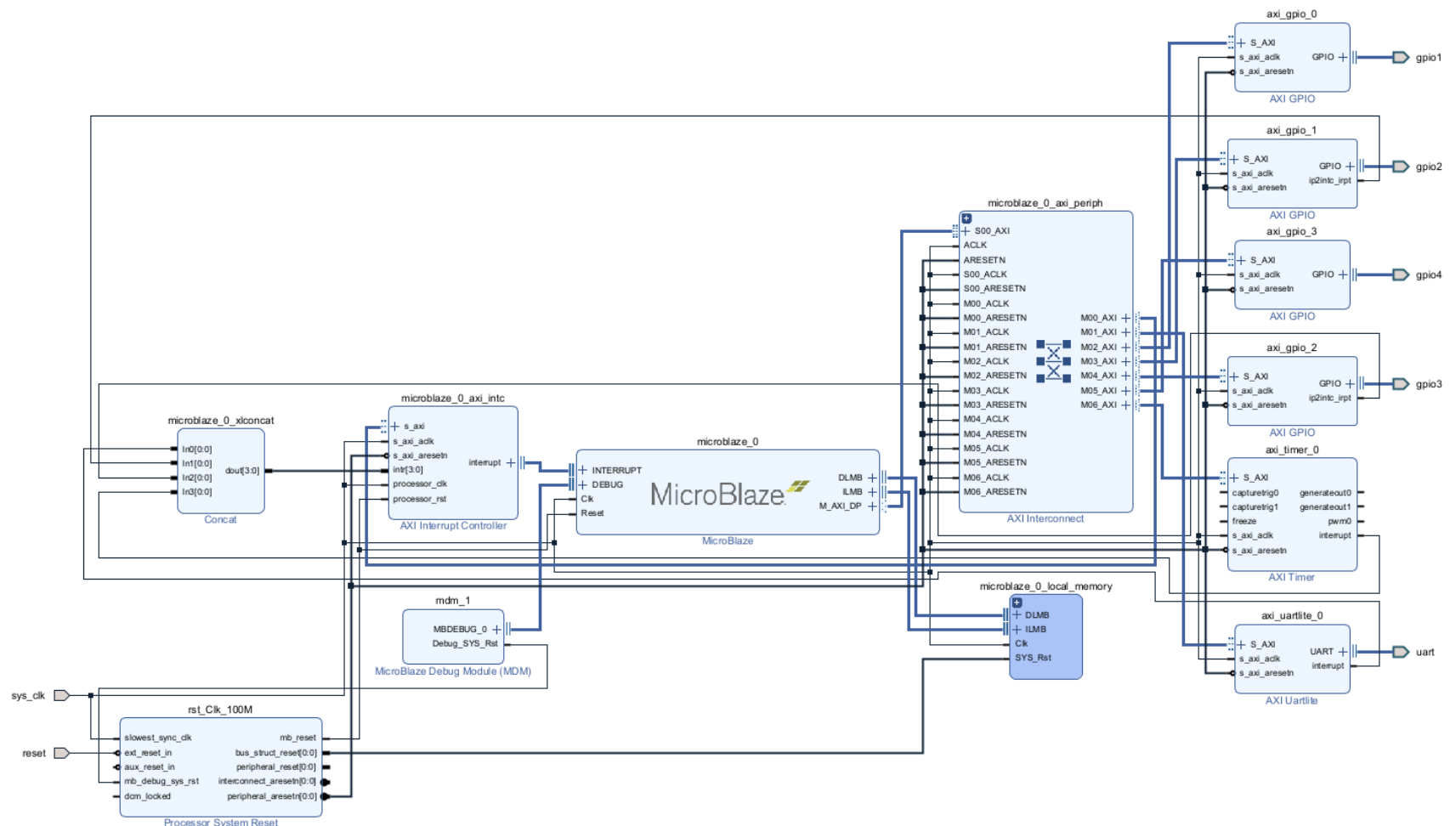
Concat 입력 포트와 각 Block의 인터럽트 출력을 매뉴얼로 연결합니다.

자동으로 생성된 포트의 이름을 변경합니다.

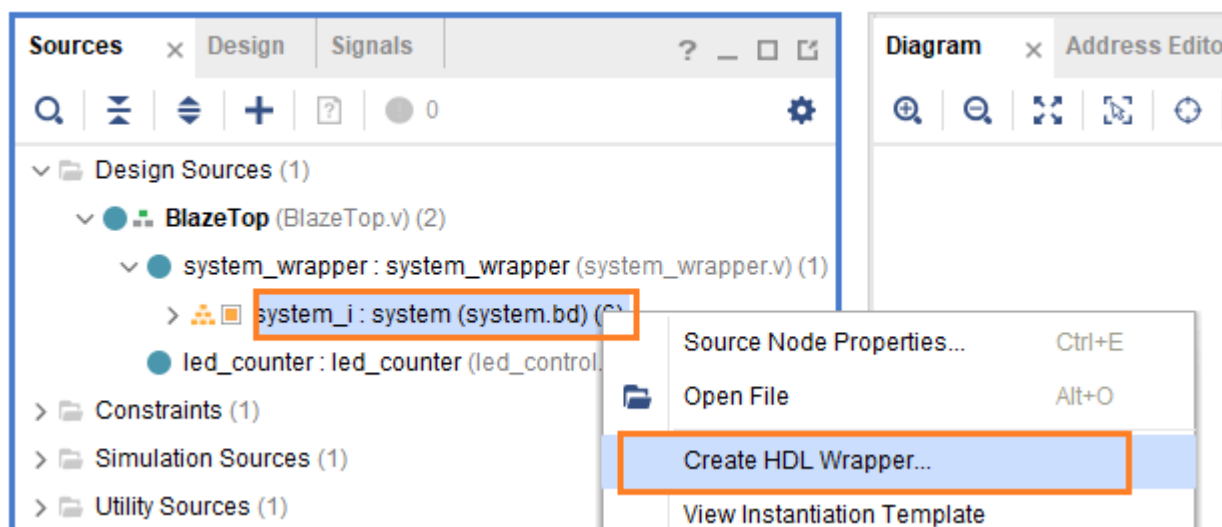
gpio_rtl_0 -> gpio1, gpio_rtl_1 -> gpio2, gpio_rtl_2 -> gpio_rtl_3, gpio_rtl_3 -> gpio4

Validate design 아이콘을 클릭해서 에러가 있는지 확인합니다. 에러가 없으면 Regenerate layout 아이콘을 클릭합니다.

아래는 지금까지 구현된 Block Design을 보여줍니다. (자료실에 diagram_ch5.png을 참조하세요)



Wrapper 파일을 생성합니다. system - 우클릭 - Create HDL Wrapper... 클릭합니다.



아래는 system_wrapper 모듈의 코드를 보여줍니다.

```

12 module system_wrapper
13     (gpio1_tri_o,
14      gpio2_tri_i,
15      gpio3_tri_i,
16      gpio4_tri_i,
17      reset,
18      sys_clk,
19      uart_rxd,
20      uart_txd);
21     output [3:0]gpio1_tri_o;
22     input  [0:0]gpio2_tri_i;
23     input  [0:0]gpio3_tri_i;
24     input  [1:0]gpio4_tri_i;
25     input reset;
26     input sys_clk;
27     input uart_rxd;
28     output uart_txd;
29
30     wire [3:0]gpio1_tri_o;
31     wire [0:0]gpio2_tri_i;
32     wire [0:0]gpio3_tri_i;
33     wire [1:0]gpio4_tri_i;
34     wire reset;
35     wire sys_clk;
36     wire uart_rxd;
37     wire uart_txd;
38
39     system system_i
40         (.gpio1_tri_o(gpio1_tri_o),
41          .gpio2_tri_i(gpio2_tri_i),
42          .gpio3_tri_i(gpio3_tri_i),
43          .gpio4_tri_i(gpio4_tri_i),
44          .reset(reset),
45          .sys_clk(sys_clk),
46          .uart_rxd(uart_rxd),
47          .uart_txd(uart_txd));
48 endmodule

```

추가한 포트들이 추가된 것을 알 수 있습니다. 포트가 추가되었기 때문에 BlazeTop.v 파일과 BlazeTop.xdc 파일을 수정해야 합니다.

system_wrapper.v 파일을 참조해서 BlazeTop.v 을 아래와 같이 수정합니다. 추가된 포트를 추가합니다.

```

23 module BlazeTop (
24     reset          ,
25     sys_clk        ,
26     gpio1_tri_o    ,
27     gpio2_tri_i    ,
28     gpio3_tri_i    ,
29     gpio4_tri_i    ,
30     uart_rxd       ,
31     uart_txd       ,
32     led            ,
33 );
34 input      reset      ;
35 input      sys_clk    ;
36 output [3:0] gpio1_tri_o ;
37 input      gpio2_tri_i ;
38 input      gpio3_tri_i ;
39 input [1:0] gpio4_tri_i ;
40 input      uart_rxd    ;
41 output     uart_txd    ;
42 output [3:0] led       ;
43
44 wire [3:0] gpio1_tri_o ;
45 system_wrapper system_wrapper (
46     .reset      (reset      ),
47     .sys_clk     (sys_clk    ),
48     .gpio1_tri_o (gpio1_tri_o),
49     .gpio2_tri_i (gpio2_tri_i),
50     .gpio3_tri_i (gpio3_tri_i),
51     .gpio4_tri_i (gpio4_tri_i),
52     .uart_rxd    (uart_rxd   ),
53     .uart_txd    (uart_txd   ),
54 );
55
56 wire [3:0] led ;
57 led_counter led_counter (
58     .reset      (reset      ),
59     .clock       (sys_clk    ),
60     .led         (led        )
61 );
62
63 endmodule

```

- ✓ 라인 26 - 29 : gpio1 ~ gpio4 포트가 추가되었습니다.
- ✓ 라인 44 : output은 wire로 선언해 주어야 합니다.
- ✓ 라인 48 - 51 : gpio1 ~ gpio4 포트가 추가되었습니다.

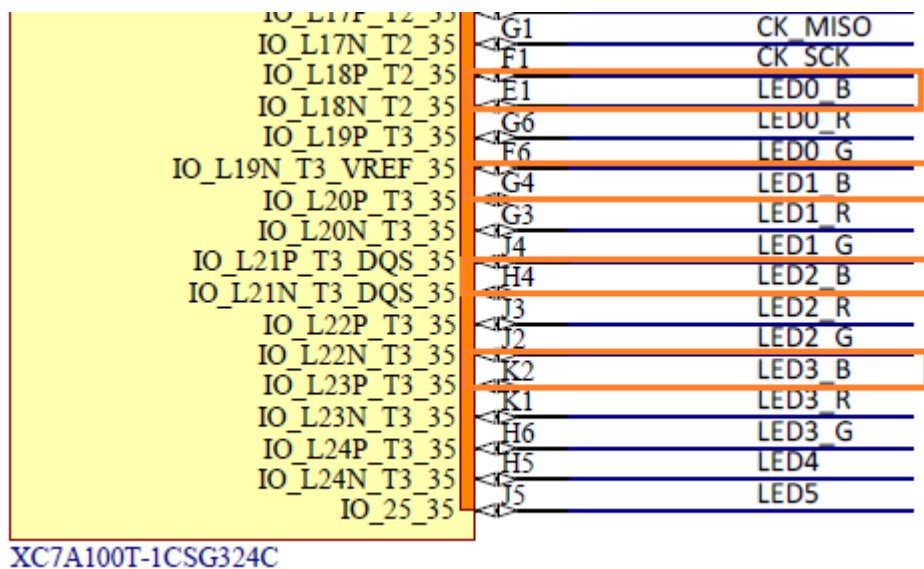
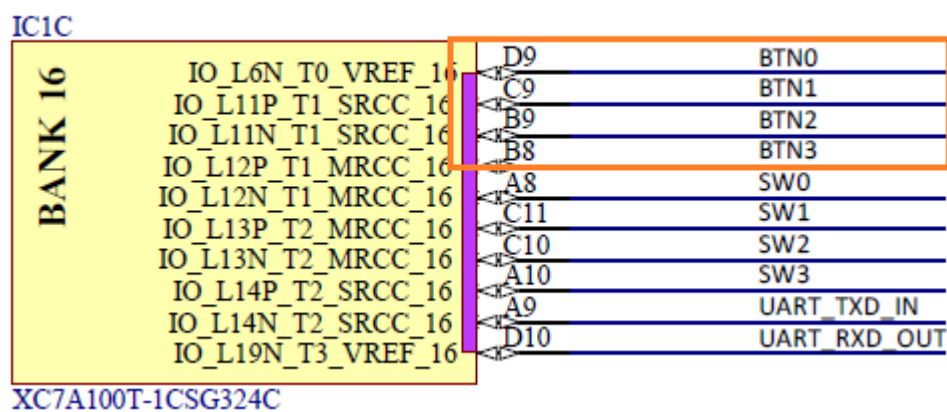
추가된 포트에 대해서 핀을 할당해야 합니다. BlazeTop.xdc 파일을 아래와 같이 수정합니다. 추가된 내용만 보여줍니다.

```
19 set_property -dict { PACKAGE_PIN E1 IOSTANDARD LVCMOS33 } [get_ports { gpio1_tri_o[0] }]; # led0_b
20 set_property -dict { PACKAGE_PIN G4 IOSTANDARD LVCMOS33 } [get_ports { gpio1_tri_o[1] }]; # led1_b
21 set_property -dict { PACKAGE_PIN H4 IOSTANDARD LVCMOS33 } [get_ports { gpio1_tri_o[2] }]; # led2_b
22 set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { gpio1_tri_o[3] }]; # led3_b
23
24 set_property -dict { PACKAGE_PIN D9 IOSTANDARD LVCMOS33 } [get_ports { gpio2_tri_i }]; # btn0
25 set_property -dict { PACKAGE_PIN C9 IOSTANDARD LVCMOS33 } [get_ports { gpio3_tri_i }]; # btn1
26 set_property -dict { PACKAGE_PIN B9 IOSTANDARD LVCMOS33 } [get_ports { gpio4_tri_i[0] }]; # btn2
27 set_property -dict { PACKAGE_PIN B8 IOSTANDARD LVCMOS33 } [get_ports { gpio4_tri_i[1] }]; # btn3
```

✓ 라인 19 - 22 : led0_blue ~ led3_blue 핀을 할당합니다.

✓ 라인 24 - 27 : btn0 ~ btn3을 할당합니다.

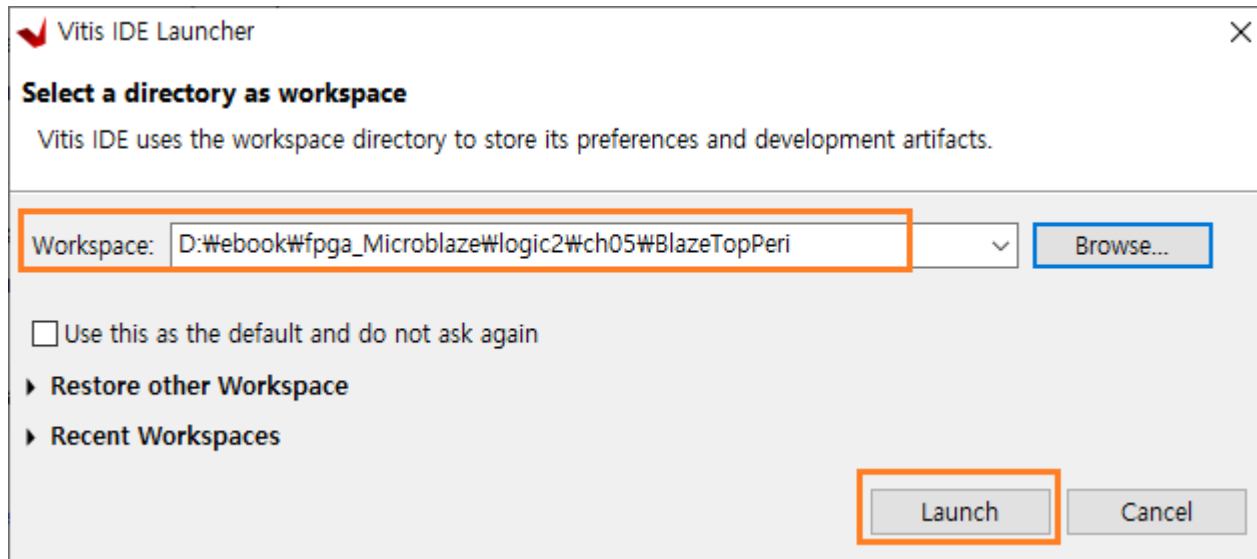
아래는 Arty a7 35T 회로도를 보여줍니다.



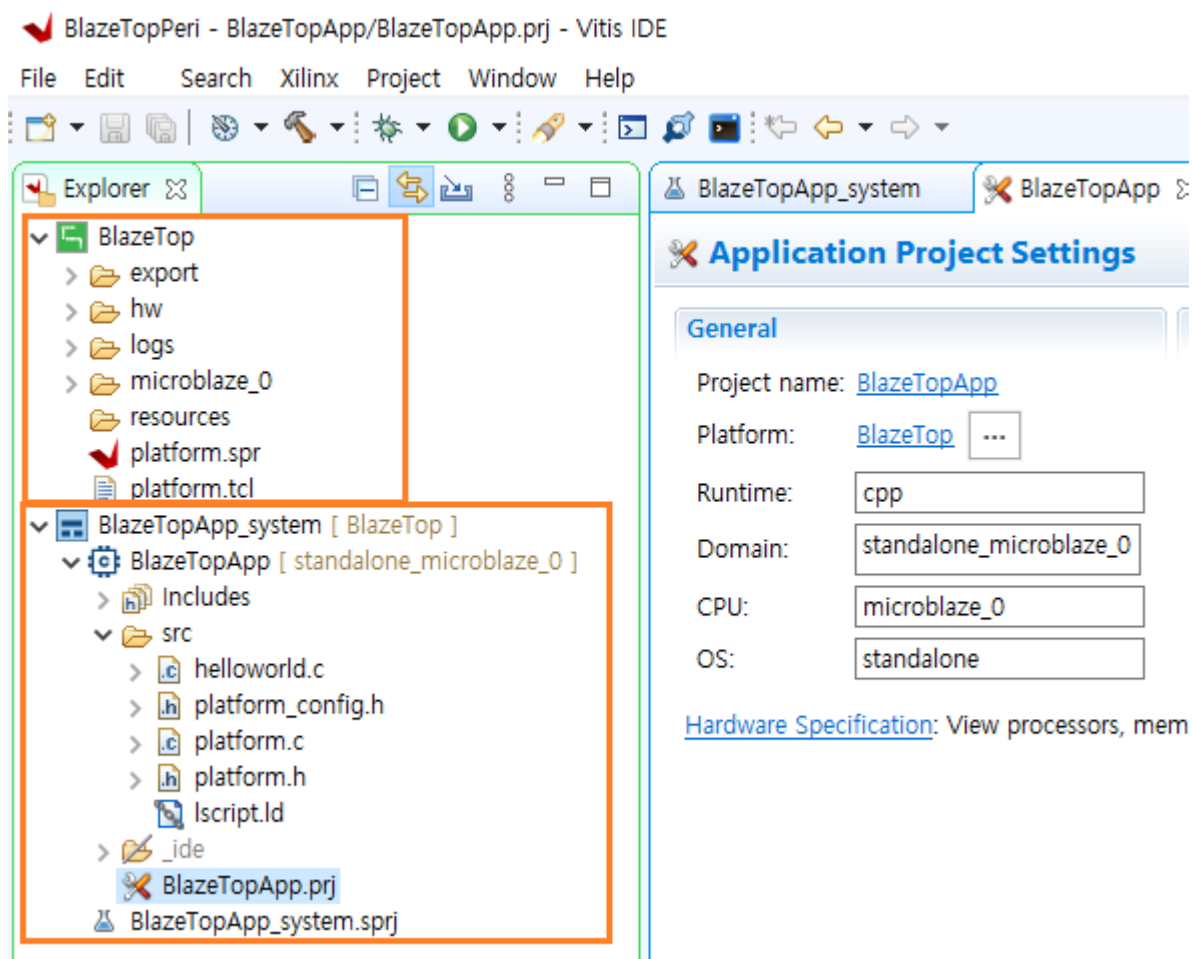
코드가 모두 수정되었습니다. Bitstream을 생성합니다. Bitstream이 생성되면 vitis에서 사용하기 위하여 XSA 파일을 생성합니다. (자세한 사항은 4.1을 참조하세요)

5.2 Application SW

Tools - Launch Vitis IDE 를 클릭해서 Vitis 를 실행합니다. Browse... 를 클릭하여 프로젝트 위치를 변경 후 Launch 를 클릭합니다.



이후의 과정은 4.1장과 동일합니다. Template 에서 Hello World를 선택하여 프로젝트를 생성합니다. 아래는 생성된 프로젝트의 파일 구조를 보여줍니다.



BlazeTop은 vivado에서 생성한 파일이고, BlazeTopApp는 vitis에서 구현하는 파일입니다.

5.2.1 xparameters.h file

xparameters.h 파일은 HW Design(xsa file) 에서 설정된 내용들을 정의한 파일입니다. sw를 구현할 때, xparameters.h 파일 안에 있는 define된 값들을 사용해야 합니다. 사실 vivado에서 block design을 구현할 때 각 모듈들의 Address 들이 자동으로 설정됩니다. 아래는 vivado의 diagram 에서 Address Editor 탭의 내용을 보여줍니다.

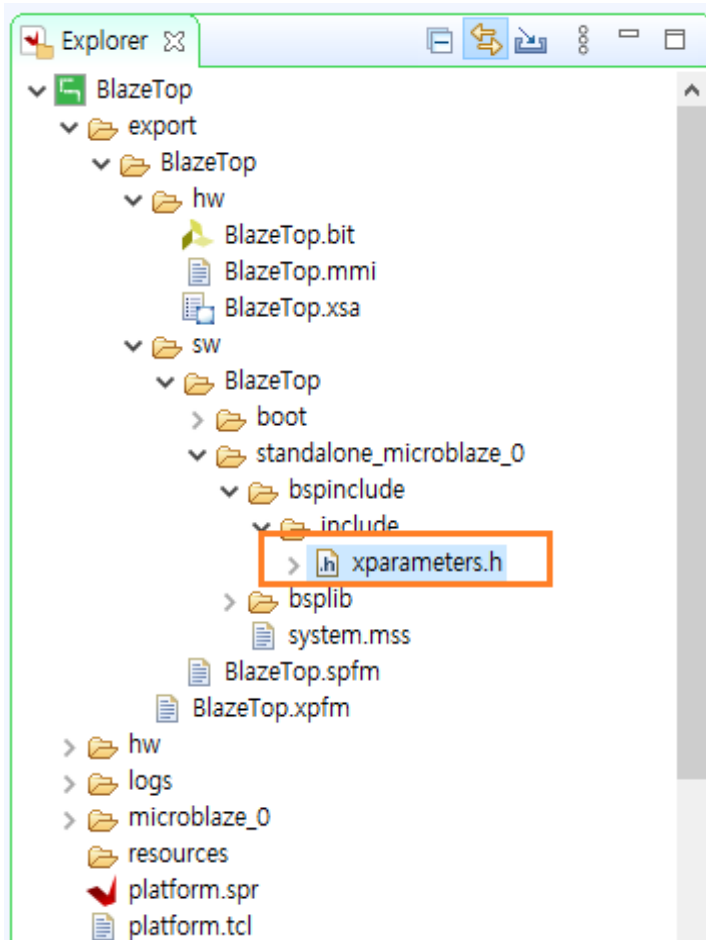
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/microblaze_0/DLMB					
/microblaze_0_local_memory/dlmb_b SLMB	Mem		0x0000_0000	32K	0x0000_7FFF
/microblaze_0/M_AXI_DP					
/axi_gpio_0/S_AXI	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF
/axi_gpio_1/S_AXI	S_AXI	Reg	0x4001_0000	64K	0x4001_FFFF
/axi_gpio_2/S_AXI	S_AXI	Reg	0x4002_0000	64K	0x4002_FFFF
/axi_gpio_3/S_AXI	S_AXI	Reg	0x4003_0000	64K	0x4003_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/microblaze_0_axi_intc/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
Network 1					
/microblaze_0					
/microblaze_0/Instruction (32 address bits : 4G)					
/microblaze_0/LMB					
/microblaze_0_local_memory/ilmb_br SLMB	Mem		0x0000_0000	32K	0x0000_7FFF

Microblaze는 0x0000_0000 에서 32KB 만큼 할당되어 있고, 각 Peripheral Block들은 0x4000_0000 부터 기본 64KB 씩 할당되어 있습니다. 이 값을 사용자가 block design 시에 변경할 수 있지만, 대부분은 자동으로 사용되는 값들을 사용합니다. microblaze가 32bits processor 여서, 사용가능한 address 영역이 32bits로 충분하기 때문에 특별히 사용자가 변경할 필요가 없습니다.

vivado에서 block design으로 생성한 파일을 xsa 파일로 변환하고, vitis에서 프로젝트 생성시 xsa 파일을 사용하기 때문에 block design에서 생성한 내용들을 그대로 가지고 와서 사용합니다. 이 값들이 BlazeTop 프로젝트 폴더안에 설정되어 있습니다.

vitis에서 embedded sw (or application sw)을 구현하기 위해서는 vivado에서 설정한 내용들을 그대로 사용해서 구현합니다. design block에서 설정한 값들 중에 대부분 중요한 값들은 주로 xparameters.h 파일에 정의되어 있습니다.

xparameters.h 파일을 대략적으로 살펴보겠습니다. xparameter.h 파일은 아래 위치에 있습니다.



아래는 Microblaze 관련된 값들을 보여줍니다. 많은 값들이 정의되어 있습니다. 일부를 보여줍니다.

```

17 /* Definitions for peripheral MICROBLAZE_0 */
18 #define XPAR_MICROBLAZE_0_ADDR_SIZE 32
19 #define XPAR_MICROBLAZE_0_ADDR_TAG_BITS 17
20 #define XPAR_MICROBLAZE_0_ALLOW_DCACHE_WR 1
21 #define XPAR_MICROBLAZE_0_ALLOW_ICACHE_WR 1
22 #define XPAR_MICROBLAZE_0_AREA_OPTIMIZED 0
23 #define XPAR_MICROBLAZE_0_ASYNC_INTERRUPT 1
24 #define XPAR_MICROBLAZE_0_ASYNC_WAKEUP 3
25 #define XPAR_MICROBLAZE_0_AVOID_PRIMITIVES 0
26 #define XPAR_MICROBLAZE_0_BASE_VECTORS 0x0000000000000000
27 #define XPAR_MICROBLAZE_0_BRANCH_TARGET_CACHE_SIZE 0
28 #define XPAR_MICROBLAZE_0_CACHE_BYTE_SIZE 8192
29 #define XPAR_MICROBLAZE_0_DADDR_SIZE 32
30 #define XPAR_MICROBLAZE_0_DATA_SIZE 32
31 #define XPAR_MICROBLAZE_0_DCACHE_ADDR_TAG 17
32 #define XPAR_MICROBLAZE_0_DCACHE_ALWAYS_USED 1
33 #define XPAR_MICROBLAZE_0_DCACHE_BASEADDR 0x00000000
34 #define XPAR_MICROBLAZE_0_DCACHE_BYTE_SIZE 8192
35 #define XPAR_MICROBLAZE_0_DCACHE_DATA_WIDTH 0
36 #define XPAR_MICROBLAZE_0_DCACHE_FORCE_TAG_LUTRAM 0
37 #define XPAR_MICROBLAZE_0_DCACHE_HIGHADDR 0x3FFFFFFF
38 #define XPAR_MICROBLAZE_0_DCACHE_LINE_LEN 4
  
```


아래는 GPIO 관련된 값들을 보여줍니다.

```

565 /* Definitions for driver GPIO */
566 #define XPAR_XGPIO_NUM_INSTANCES 4
567
568 /* Definitions for peripheral AXI_GPIO_0 */
569 #define XPAR_AXI_GPIO_0_BASEADDR 0x40000000
570 #define XPAR_AXI_GPIO_0_HIGHADDR 0x4000FFFF
571 #define XPAR_AXI_GPIO_0_DEVICE_ID 0
572 #define XPAR_AXI_GPIO_0_INTERRUPT_PRESENT 0
573 #define XPAR_AXI_GPIO_0_IS_DUAL 0
574
575
576 /* Definitions for peripheral AXI_GPIO_1 */
577 #define XPAR_AXI_GPIO_1_BASEADDR 0x40010000
578 #define XPAR_AXI_GPIO_1_HIGHADDR 0x4001FFFF
579 #define XPAR_AXI_GPIO_1_DEVICE_ID 1
580 #define XPAR_AXI_GPIO_1_INTERRUPT_PRESENT 1
581 #define XPAR_AXI_GPIO_1_IS_DUAL 0
582
583
584 /* Definitions for peripheral AXI_GPIO_2 */
585 #define XPAR_AXI_GPIO_2_BASEADDR 0x40020000
586 #define XPAR_AXI_GPIO_2_HIGHADDR 0x4002FFFF
587 #define XPAR_AXI_GPIO_2_DEVICE_ID 2
588 #define XPAR_AXI_GPIO_2_INTERRUPT_PRESENT 1
589 #define XPAR_AXI_GPIO_2_IS_DUAL 0
590
591
592 /* Definitions for peripheral AXI_GPIO_3 */
593 #define XPAR_AXI_GPIO_3_BASEADDR 0x40030000
594 #define XPAR_AXI_GPIO_3_HIGHADDR 0x4003FFFF
595 #define XPAR_AXI_GPIO_3_DEVICE_ID 3
596 #define XPAR_AXI_GPIO_3_INTERRUPT_PRESENT 0
597 #define XPAR_AXI_GPIO_3_IS_DUAL 0

```

GPIO Instances는 4개를 생성했기 때문에 4로 정의되어 있고, 각각의 GPIO Block의 속성을 정의하고 있습니다. 정의된 내용을 보면, Canonical definitions 이 동일한 내용으로 추가되어 있습니다. 이것은 아마 버전이 업데이트 되면서 이전 버전과의 호환성을 위해 있는 것으로 생각됩니다. 둘 중에 아무거나 사용해도 동일한 값이므로 무관합니다.

아래는 Interrupt controller에 관한 정의를 보여줍니다.

```

633 #define XPAR_INTC_MAX_NUM_INTR_INPUTS 4
634 #define XPAR_XINTC_HAS_IPR 1
635 #define XPAR_XINTC_HAS_SIE 1
636 #define XPAR_XINTC_HAS_CIE 1
637 #define XPAR_XINTC_HAS_IVR 1
638 /* Definitions for driver INTC */
639 #define XPAR_XINTC_NUM_INSTANCES 1
640
641 /* Definitions for peripheral MICROBLAZE_0_AXI_INTC */
642 #define XPAR_MICROBLAZE_0_AXI_INTC_DEVICE_ID 0
643 #define XPAR_MICROBLAZE_0_AXI_INTC_BASEADDR 0x41200000
644 #define XPAR_MICROBLAZE_0_AXI_INTC_HIGHADDR 0x4120FFFF
645 #define XPAR_MICROBLAZE_0_AXI_INTC_KIND_OF_INTR 0xFFFFFFFF1
646 #define XPAR_MICROBLAZE_0_AXI_INTC_HAS_FAST 1
647 #define XPAR_MICROBLAZE_0_AXI_INTC_IVAR_RESET_VALUE 0x0000000000000010
648 #define XPAR_MICROBLAZE_0_AXI_INTC_NUM_INTR_INPUTS 4
649 #define XPAR_MICROBLAZE_0_AXI_INTC_NUM_SW_INTR 0
650 #define XPAR_MICROBLAZE_0_AXI_INTC_ADDR_WIDTH 32

```

인터럽트 개수로 4로 정의되어 있습니다.

아래는 Timer, Uart 에 관련된 값들을 보여줍니다.

```

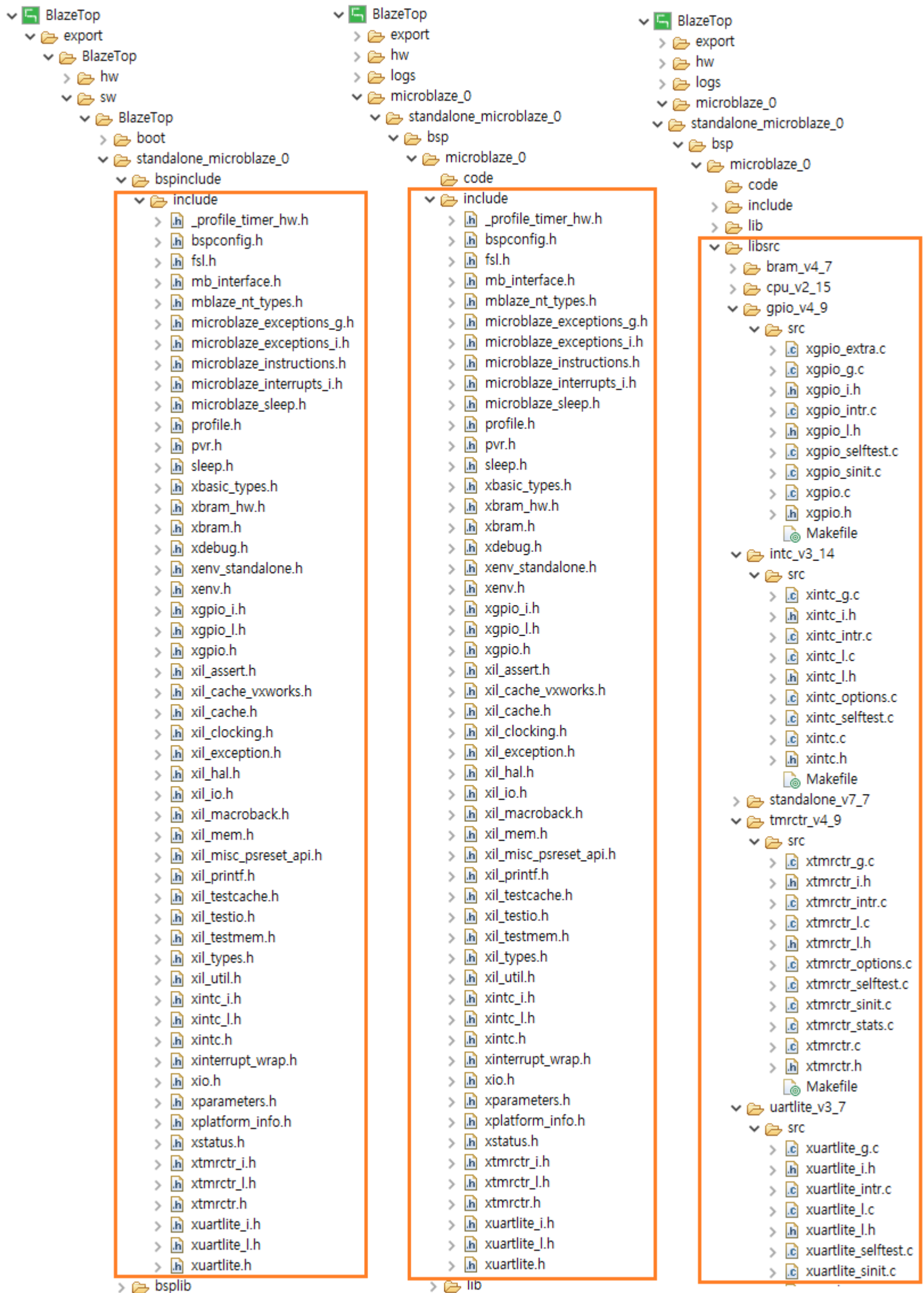
689 /* Definitions for driver TMRCTR */
690 #define XPAR_XTMRCTR_NUM_INSTANCES 1U
691
692 /* Definitions for peripheral AXI_TIMER_0 */
693 #define XPAR_AXI_TIMER_0_DEVICE_ID 0U
694 #define XPAR_AXI_TIMER_0_BASEADDR 0x41C00000U
695 #define XPAR_AXI_TIMER_0_HIGHADDR 0x41C0FFFFU
696 #define XPAR_AXI_TIMER_0_CLOCK_FREQ_HZ 100000000U
697
698
699 /*****
700
701 /* Canonical definitions for peripheral AXI_TIMER_0 */
702 #define XPAR_TMRCTR_0_DEVICE_ID 0U
703 #define XPAR_TMRCTR_0_BASEADDR 0x41C00000U
704 #define XPAR_TMRCTR_0_HIGHADDR 0x41C0FFFFU
705 #define XPAR_TMRCTR_0_CLOCK_FREQ_HZ XPAR_AXI_TIMER_0_CLOCK_FREQ_HZ
706
707 /*****
708
709 /* Definitions for driver UARTLITE */
710 #define XPAR_XUARTLITE_NUM_INSTANCES 1U
711
712 /* Definitions for peripheral AXI_UARTLITE_0 */
713 #define XPAR_AXI_UARTLITE_0_DEVICE_ID 0U
714 #define XPAR_AXI_UARTLITE_0_BASEADDR 0x40600000U
715 #define XPAR_AXI_UARTLITE_0_HIGHADDR 0x4060FFFFU
716 #define XPAR_AXI_UARTLITE_0_BAUDRATE 115200U
717 #define XPAR_AXI_UARTLITE_0_USE_PARITY 0U
718 #define XPAR_AXI_UARTLITE_0_ODD_PARITY 0U
719 #define XPAR_AXI_UARTLITE_0_DATA_BITS 8U
720
721 /* Canonical definitions for peripheral AXI_UARTLITE_0 */
722 #define XPAR_UARTLITE_0_DEVICE_ID 0U
723 #define XPAR_UARTLITE_0_BASEADDR 0x40600000U
724 #define XPAR_UARTLITE_0_HIGHADDR 0x4060FFFFU
725 #define XPAR_UARTLITE_0_BAUDRATE 115200U
726 #define XPAR_UARTLITE_0_USE_PARITY 0U
727 #define XPAR_UARTLITE_0_ODD_PARITY 0U
728 #define XPAR_UARTLITE_0_DATA_BITS 8U

```

Address나 기타 내용들이 Block Design에서 구현했던 내용 그대로 정의되어 있는 것을 알 수 있습니다.

5.2.2 소스 구조

다음 그림은 Processor, Peripheral 관련 header, source 파일들을 보여줍니다. 파일들이 여러 군데 중복되어 있습니다. embedded sw를 구현할 때 주로 참조하는 파일들입니다. 예를 들어 gpio 를 구현한다고 하면, xgpio.h, xgpio_l.h, xgpio.c 파일들을 참조해서 구현합니다. xgpio.h 파일이 3군데 모두 존재하는 것을 알 수 있습니다. 대부분 header 파일의 함수 정의 부분을 참조해서 구현합니다.



5.2.3 코드 구현

이제 코드를 구현합니다. helloworld.c 에 아래와 같이 코드를 추가합니다.

```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "xparameters.h"
52 #include "xgpio.h"
53 #include "xuartlite.h"
54 #include "xtmrctr.h"
55 #include "sleep.h"
56 #include "xil_exception.h"
57 #include "xintc.h"

```

✓ 라인 51 - 57 : include header file, GPIO, UART, TIMER, INTERRUPT 등

```

59 /* definitions for LED_4bits, AXI_GPIO_0 */
60 #define GPIO_LED4BITS_DEVICE_ID XPAR_GPIO_0_DEVICE_ID
61 #define LED4BITS_CHANNEL1      1
62
63 /* definitions for btn0, AXI_GPIO_1 */
64 #define GPIO_BTN0_DEVICE_ID    XPAR_GPIO_1_DEVICE_ID
65 #define BTN0_CHANNEL1         1
66 #define BTN0_INT_MSK          XGPIO_IR_CH1_MASK /* Channel 1 Interrupt Mask */
67
68 /* definitions for btn1, AXI_GPIO_2 */
69 #define GPIO_BTN1_DEVICE_ID    XPAR_GPIO_2_DEVICE_ID
70 #define BTN1_CHANNEL1         1
71 #define BTN1_INT_MSK          XGPIO_IR_CH1_MASK /* Channel 1 Interrupt Mask */
72
73 /* definitions for btn23, AXI_GPIO_3 */
74 #define GPIO_BTN23_DEVICE_ID   XPAR_GPIO_3_DEVICE_ID
75 #define BTN23_CHANNEL1        1
76
77 /* definitions for UART */
78 #define UARTLITE_DEVICE_ID     XPAR_UARTLITE_0_DEVICE_ID
79 #define UARTLITE_INT_IRQ_ID    XPAR_INTC_0_UARTLITE_0_VEC_ID
80
81 /* definitions for Timer */
82 #define TMRCTR_DEVICE_ID       XPAR_TMRCTR_0_DEVICE_ID
83 #define TMRCTR_INTERRUPT_ID    XPAR_INTC_0_TMRCTR_0_VEC_ID
84
85 /* definitions for Interrupt */
86 #define INTC_DEVICE_ID         XPAR_INTC_0_DEVICE_ID
87 #define INTC_BTN0_INT_VEC_ID   XPAR_INTC_0_GPIO_1_VEC_ID
88 #define INTC_BTN1_INT_VEC_ID   XPAR_INTC_0_GPIO_2_VEC_ID

```

✓ 라인 59 - 88 : xparameters.h 에 정의된 값들을 필요한 값만 재 정의해서 사용합니다. DEVICE_ID는 해당 Block이 생성된 순서대로 0, 1, 2 ~ 값을 갖습니다. GPIO는 총 4개를 생성하였으므로 0 ~ 3 까지의 값을 갖고, UART, TIMER, INTERRUPT는 1개씩 사용되므로 0의 값을 갖습니다. CHANNEL은 1, 2 의 값을 갖는데, CHANNEL2는 Disable로 생성했기 때문에 CHANNEL1만 사용합니다. Interrupt ID는 순서대로 UART(0), GPIO1(1), GPIO2(2), TIMER(3) 입니다.

```

91 /* definitions for LED bits */
92 #define LED0_B          0x01
93 #define LED1_B          0x02
94 #define LED2_B          0x04
95 #define LED3_B          0x08
96
97 #define UART_BUFFER_SIZE 100 /* UART RX Buffer Size */
98
99 #define TIMER_CNTR_0      0
100 #define TMR_RST_VALUE    (0xffffffff - 100000000) /* period : 1s */
101
102 void Gpio1Handler(void *CallbackRef);
103 void Gpio2Handler(void *CallbackRef);
104 void UartSendHandler(void *CallbackRef, unsigned int EventData);
105 void UartRecvHandler(void *CallbackRef, unsigned int EventData);
106 void TimerCounterHandler(void *CallbackRef, u8 TmrCtrNumber);
107
108 XGpio      Gpio_led;          /* The instance of the led */
109 XGpio      Gpio_btn_0;        /* The instance of the btn0 */
110 XGpio      Gpio_btn_1;        /* The instance of the btn1 */
111 XGpio      Gpio_btn_23;       /* The instance of the btn23 */
112 XUartLite  UartLite;          /* The instance of the UartLite Device */
113 XTmrCtr    TimerCounter;      /* The instance of the Timer Counter */
114 XIntc      Intc;              /* The instance of the Interrupt Controller */
115
116 static volatile int TotalSentCount; /* dummy */
117
118 int UartRxIndex1, UartRxIndex2;
119 u8 UartRxBuf[UART_BUFFER_SIZE];
120
121 u8 TimerFlag;

```

- ✓ 라인 91 - 95 : gpio1에 4bits led를 연결하였습니다. // BlazeTop.v, 36 line
- ✓ 라인 97 : Uart 수신 버퍼 사이즈를 정의합니다.
- ✓ 라인 99 - 100 : Timer 초기값, 주기를 설정합니다. 주기는 1초로 설정됩니다.
- ✓ 라인 102 - 106 : Interrupt Handler를 정의합니다.
- ✓ 라인 108 - 114 : Block Design 에서 생성한 각 IP들은 sw에서 instance를 정의하여 사용합니다. instance의 정의는 header 파일에 정의되어 있습니다. 예를 들어 XGpio 는 아래와 같이 정의되어 있습니다.

```

124 * This typedef contains configuration information for the device.
125 */
126 typedef struct {
127     u16 DeviceId;          /**< Unique ID of device */
128     UINTPTR BaseAddress;   /**< Device base address */
129     int InterruptPresent;  /**< Are interrupts supported in h/w */
130     int IsDual;           /**< Are 2 channels supported in h/w */
131 } XGpio_Config;
132
133 /**
134 * The XGpio driver instance data. The user is required to allocate a
135 * variable of this type for every GPIO device in the system. A pointer
136 * to a variable of this type is then passed to the driver API functions.
137 */
138 typedef struct {
139     UINTPTR BaseAddress;   /**< Device base address */
140     u32 IsReady;          /**< Device is initialized and ready */
141     int InterruptPresent;  /**< Are interrupts supported in h/w */
142     int IsDual;           /**< Are 2 channels supported in h/w */
143 } XGpio;

```

- ✓ 라인 116 : Uart Tx Interrupt Handler에서 사용하는 dummy 변수입니다.
- ✓ 라인 118 - 119 : Uart Rx Interrupt Handler에서 사용됩니다. 데이터가 수신되면 수신 버퍼에 저장합니다.
- ✓ 라인 121 : Timer Interrupt 발생시 1로 설정합니다. main() 함수에서는 이 값을 사용합니다.

```

123 void interrupt_init(void)
124 {
125     XIntc_Initialize(&Intc, INTC_DEVICE_ID);
126
127     XIntc_Connect(&Intc, INTC_BTN0_INT_VEC_ID, (XInterruptHandler)Gpio1Handler, &Gpio_btn_0);
128     XIntc_Connect(&Intc, INTC_BTN1_INT_VEC_ID, (XInterruptHandler)Gpio2Handler, &Gpio_btn_1);
129     XIntc_Connect(&Intc, UARLITE_INT_IRQ_ID, (XInterruptHandler)XUartLite_InterruptHandler,
130                 (void *)&UartLite);
131     XIntc_Connect(&Intc, TMRCTR_INTERRUPT_ID, (XInterruptHandler)XTmrCtr_InterruptHandler,
132                 (void *)&TimerCounter);
133
134     XIntc_Start(&Intc, XIN_REAL_MODE);
135     XIntc_Enable(&Intc, UARLITE_INT_IRQ_ID);
136     XIntc_Enable(&Intc, INTC_BTN0_INT_VEC_ID);
137     XIntc_Enable(&Intc, INTC_BTN1_INT_VEC_ID);
138     XIntc_Enable(&Intc, TMRCTR_INTERRUPT_ID);
139 }
140
141 void exception_init(void)
142 {
143     Xil_ExceptionInit();
144     Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT, (Xil_ExceptionHandler)XIntc_InterruptHandler,
145                               &Intc);
146     Xil_ExceptionEnable();
147 }
148
149 void uart_init(void)
150 {
151     XUartLite_Initialize(&UartLite, UARLITE_DEVICE_ID);
152     XUartLite_SelfTest(&UartLite);
153
154     XUartLite_SetSendHandler(&UartLite, UartSendHandler, &UartLite);
155     XUartLite_SetRecvHandler(&UartLite, UartRecvHandler, &UartLite);
156
157     XUartLite_EnableInterrupt(&UartLite);
158
159     UartRxIndex1 = 0;
160     UartRxIndex2 = 0;
161 }
162
163 void gpio_init(void)
164 {
165     XGpio_Initialize(&Gpio_led, GPIO_LED4BITS_DEVICE_ID);
166     XGpio_Initialize(&Gpio_btn_0, GPIO_BTN0_DEVICE_ID);
167     XGpio_Initialize(&Gpio_btn_1, GPIO_BTN1_DEVICE_ID);
168     XGpio_Initialize(&Gpio_btn_23, GPIO_BTN23_DEVICE_ID);
169
170     XGpio_InterruptEnable(&Gpio_btn_0, BTN0_CHANNEL1);
171     XGpio_InterruptEnable(&Gpio_btn_1, BTN1_CHANNEL1);
172
173     XGpio_InterruptGlobalEnable(&Gpio_btn_0);
174     XGpio_InterruptGlobalEnable(&Gpio_btn_1);
175 }
176
177 void timer_init(void)
178 {
179     XTmrCtr_Initialize(&TimerCounter, TMRCTR_DEVICE_ID);
180     XTmrCtr_SelfTest(&TimerCounter, TIMER_CNTR_0);
181
182     XTmrCtr_SetHandler(&TimerCounter, TimerCounterHandler, &TimerCounter);
183
184     XTmrCtr_SetOptions(&TimerCounter, TIMER_CNTR_0, XTC_INT_MODE_OPTION | XTC_AUTO_RELOAD_OPTION);
185
186     XTmrCtr_SetResetValue(&TimerCounter, TIMER_CNTR_0, TMR_RST_VALUE);
187     TimerFlag = 0;
188 }

```

- ✓ 라인 123 - 188 : 각 module를 초기화 합니다. 순서는 Initialize, Interrupt Handler 등록, Enable Interrupt, 기타입니다.


```

191 int main()
192 {
193     int cnt_1s = 0;
194
195     init_platform();
196
197     gpio_init();
198     uart_init();
199     timer_init();
200     interrupt_init();
201     exception_init();
202
203     xil_printf("\r\n App Start v1.0a.. \r\n");
204     XTmrCtr_Start(&TimerCounter, TIMER_CNTR_0); // Timer Start
205     while (1)
206     {
207         // Uart Rx Task
208         if(UartRxIndex1 != UartRxIndex2)
209         {
210             xil_printf("\r\n rcv : %c", UartRxBuf[UartRxIndex2]);
211             UartRxIndex2++;
212             if(UartRxIndex2 >= UART_BUFFER_SIZE)
213                 UartRxIndex2 = 0;
214         }
215
216         // Timer0 Task
217         if(TimerFlag==1)
218         {
219             TimerFlag = 0;
220             xil_printf("\r\n Timer0 isr %d ", cnt_1s++);
221         }
222
223         // BTN2 Task
224         if ((XGpio_DiscreteRead(&Gpio_btn_23, BTN23_CHANNEL1) & 0x0001))
225         {
226             XGpio_DiscreteWrite(&Gpio_led, LED4BITS_CHANNEL1, LED2_B);
227         }
228         else
229         {
230             XGpio_DiscreteClear(&Gpio_led, LED4BITS_CHANNEL1, LED2_B);
231         }
232
233         // BTN3 Task
234         if ((XGpio_DiscreteRead(&Gpio_btn_23, BTN23_CHANNEL1) & 0x0002))
235         {
236             XGpio_DiscreteWrite(&Gpio_led, LED4BITS_CHANNEL1, LED3_B);
237         }
238         else
239         {
240             XGpio_DiscreteClear(&Gpio_led, LED4BITS_CHANNEL1, LED3_B);
241         }
242     }
243
244     cleanup_platform();
245     return 0;
246 }
247

```

- ✓ 라인 191 - 247 : main 함수입니다.
- ✓ 라인 197 - 201 : 각 module 초기화 합니다.
- ✓ 라인 204 : timer 값을 초기화 합니다.
- ✓ 라인 208 - 214 : uart 수신 데이터가 있으면 출력합니다.
- ✓ 라인 216 - 221 : timer 인터럽트 발생시 메시지를 출력합니다. 1초마다 출력됩니다.
- ✓ 라인 223 - 231 : BTN2의 값을 읽어서 LED2_B를 On/Off 합니다.
- ✓ 라인 234 - 242 : BTN3의 값을 읽어서 LED3_B를 On/Off 합니다.

```

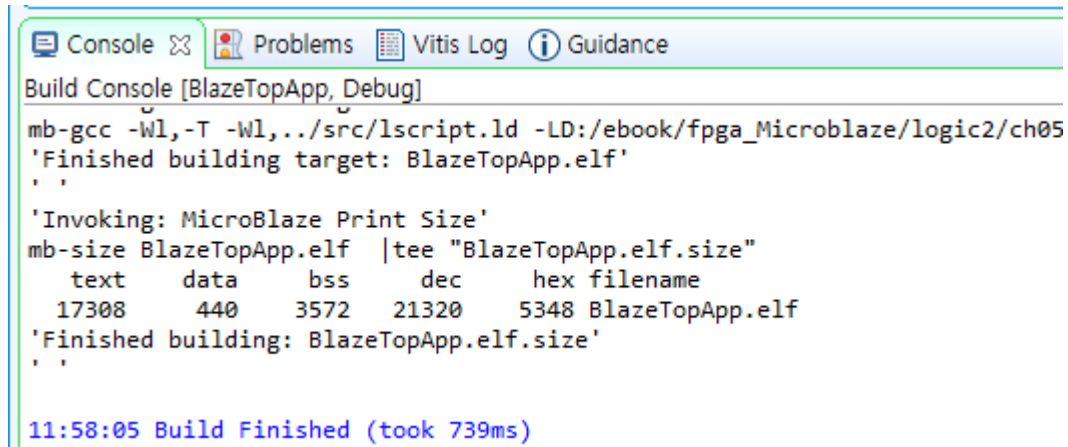
249 void UartSendHandler(void *CallbackRef, unsigned int EventData)
250 {
251     TotalSentCount = EventData;
252 }
253
254 void UartRecvHandler(void *CallbackRef, unsigned int EventData)
255 {
256     u8 rxData;
257     XUartLite_Recv(&UartLite, &rxData, 1);
258     UartRxBuf[UartRxIndex1] = rxData;
259
260     UartRxIndex1++;
261     if(UartRxIndex1 >= UART_BUFFER_SIZE)
262         UartRxIndex1 = 0;
263 }
264
265 void Gpio1Handler(void *CallbackRef)
266 {
267     static int intrCount = 0;
268     XGpio *GpioPtr = (XGpio *)CallbackRef;
269
270     if ((XGpio_DiscreteRead(&Gpio_btn_0, BTN0_CHANNEL1)&0x0001))
271         xil_printf("\r\n interrupt-0 occurs %d\r\n", intrCount++);
272
273     if(intrCount & 0x01)
274         XGpio_DiscreteWrite(&Gpio_led, LED4BITS_CHANNEL1, LED0_B);
275     else
276         XGpio_DiscreteClear(&Gpio_led, LED4BITS_CHANNEL1, LED0_B);
277
278     XGpio_InterruptClear(GpioPtr, BTN0_CHANNEL1); /* Clear the Interrupt */
279 }
280
281 void Gpio2Handler(void *CallbackRef)
282 {
283     static int intrCount = 0;
284     XGpio *GpioPtr = (XGpio *)CallbackRef;
285
286     if ((XGpio_DiscreteRead(&Gpio_btn_1, BTN1_CHANNEL1)&0x0001))
287         xil_printf("\r\n interrupt-1 occurs %d\r\n", intrCount++);
288
289     if(intrCount & 0x01)
290         XGpio_DiscreteWrite(&Gpio_led, LED4BITS_CHANNEL1, LED1_B);
291     else
292         XGpio_DiscreteClear(&Gpio_led, LED4BITS_CHANNEL1, LED1_B);
293
294     XGpio_InterruptClear(GpioPtr, BTN1_CHANNEL1); /* Clear the Interrupt */
295 }
296
297 void TimerCounterHandler(void *CallbackRef, u8 TmrCtrNumber)
298 {
299     TimerFlag = 1;
300 }

```

- ✓ 라인 249 - 300 : Interrupt Handler를 구현합니다.
- ✓ 라인 240 - 252 : Uart Tx Interrupt Handler입니다.
- ✓ 라인 254 - 263 : Uart Rx Interrupt Handler 입니다. 수신데이터를 수신 버퍼에 저장합니다.
- ✓ 라인 265 - 279 : GPIO-1 Interrupt Handler 입니다. BTN0가 눌러지면 인터럽트가 발생하고, LED0_B를 Toggling 합니다.
- ✓ 라인 281 - 195 : GPIO-2 Interrupt Handler 입니다. BTN1가 눌러지면 인터럽트가 발생하고, LED1_B를 Toggling 합니다.
- ✓ 라인 297 - 300 : Timer Interrupt Handler 입니다. Timer Interrupt가 발생하면 TimerFlag를 1로 설정합니다.

5.3 다운로드 및 결과 확인

프로젝트를 Build 합니다. BlazeApp - 우클릭 - Build Project를 클릭합니다. 에러가 없이 Build 됩니다.



```

Build Console [BlazeTopApp, Debug]
mb-gcc -Wl,-T -Wl,.../src/lscript.ld -LD:/ebook/fpga_Microblaze/logic2/ch05
'Finished building target: BlazeTopApp.elf'
'Invoking: MicroBlaze Print Size'
mb-size BlazeTopApp.elf |tee "BlazeTopApp.elf.size"
  text  data    bss    dec    hex filename
 17308   440   3572  21320   5348 BlazeTopApp.elf
'Finished building: BlazeTopApp.elf.size'

11:58:05 Build Finished (took 739ms)

```

보드를 연결하고 디버깅 모드로 다운로드 합니다.

BlazeTopApp_system - 우클릭 - Debug As - 1 Launch Hardware 를 클릭합니다.

프로그램이 다운로드 됩니다. 완료되면 main() 함수의 처음 위치에 멈춰 있습니다. Resume 버튼을 클릭하면 프로그램이 동작합니다.

아래 그림은 결과를 보여줍니다.

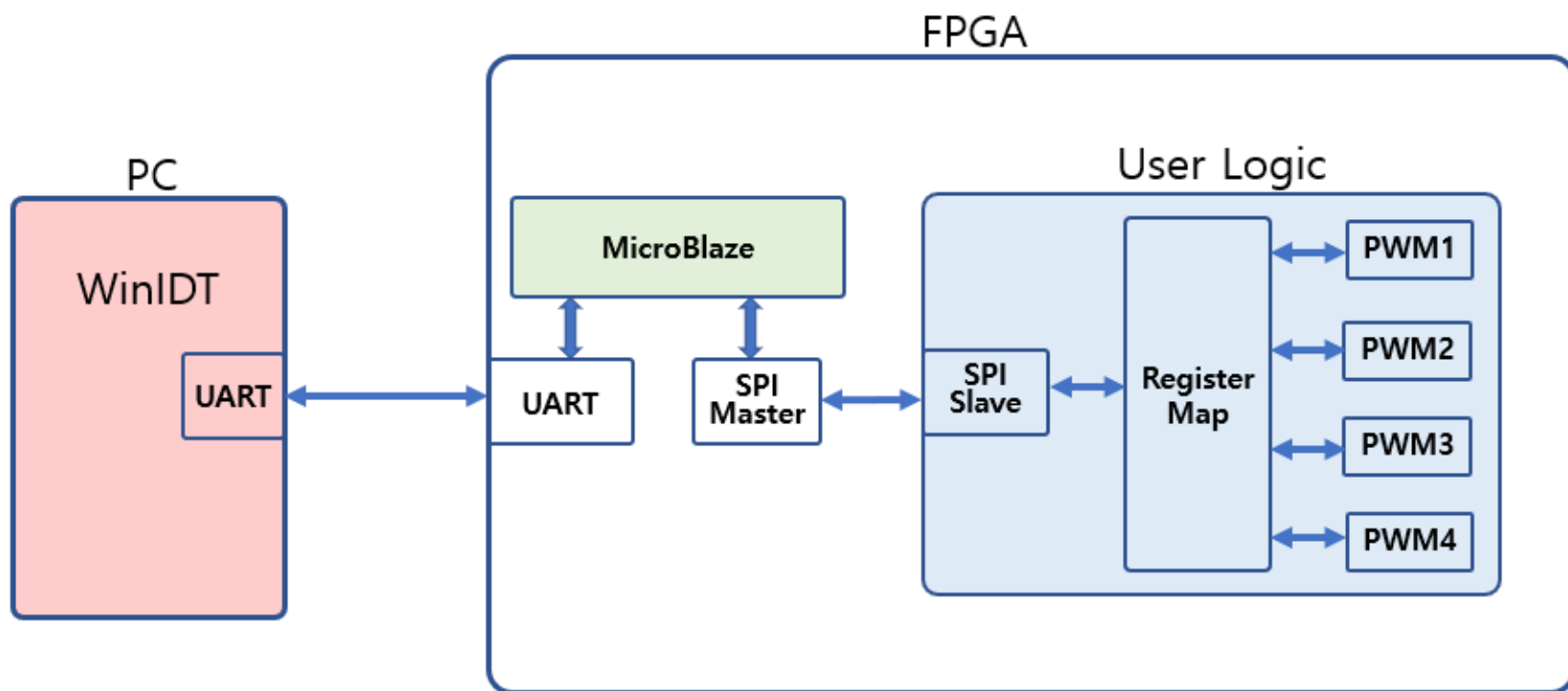
- ✓ Timer0 인터럽트가 1초 간격으로 발생합니다.
- ✓ BTN0를 누르면 “interrupt-0 occurs 0” 가 표시됩니다. LED0_Blue가 Toggle 됩니다.
- ✓ BTN1을 누르면 “interrupt-1 occurs 0, 1” 가 표시됩니다. LED1_Blue 가 Toggle 됩니다.
- ✓ BTN2을 누르면 LED2_Blue 이 On 되고, 누르지 않으면 Off 됩니다.
- ✓ BTN3을 누르면 LED3_Blue이 On 되고, 누르지 않으면 Off 됩니다.
- ✓ WinIDT 에서 Freq1 우측에 있는 “W” 버튼을 클릭하면 PC에서 전송한 데이터가 차례대로 표시됩니다.



6. User Logic 인터페이스 구현

이번 장에서는 사용자 Logic 을 추가하고, MicroBlaze와의 인터페이스를 구현합니다. 인터페이스를 구현하기 위한 많은 방법이 있습니다. UART, I2C, Parallel, SPI 등이 있을 수 있습니다. 그 중에서 SPI 인터페이스를 사용하도록 합니다. SPI를 사용하는 이유는 연결되는 선이 4개로 적고, I2C에 비해서 데이터를 주고받는 속도가 빠르고 구현하는데 어렵지 않다는 것입니다. 사용자 Logic은 간단하게 구성할 수 있는 PWM Controller를 구현합니다. [그림 6-1]은 System Block을 보여 줍니다.

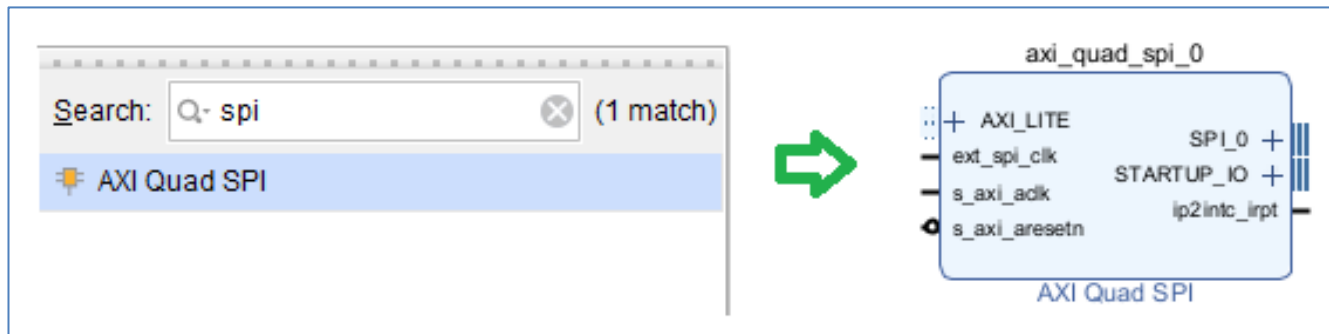
[그림 6-1] System Block



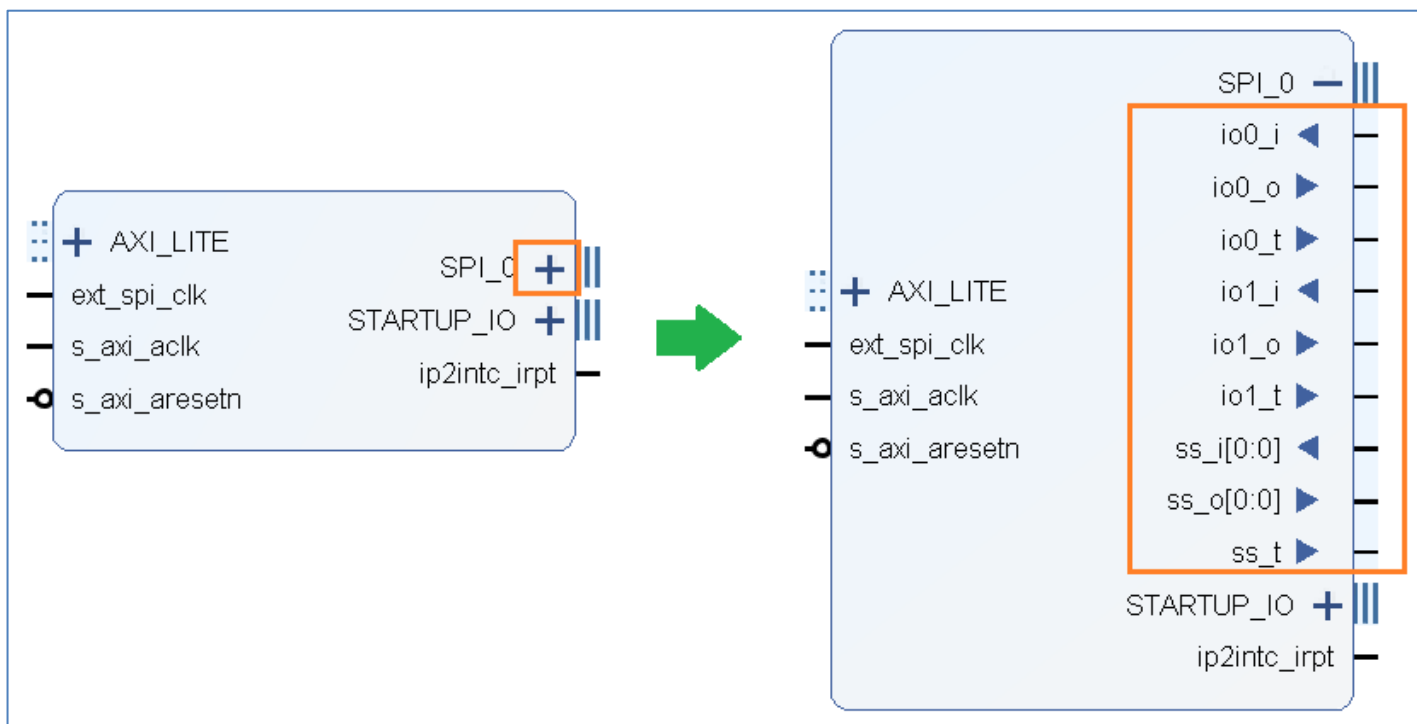
PC의 Application Program(WinIDT)에서 User Logic (PWM)을 제어하기 위하여 UART로 데이터를 전송합니다. MicroBlaze는 명령어를 수신해서 SPI Master로 Register Read/Wrtie 명령어를 전송합니다. User Logic의 SPI Slave에서 Register 값을 Read/Write 합니다. 이러한 기본 구조를 가지고 있으면 언제든지 User Logic에 필요한 부분들을 추가하여 사용할 수 있습니다. 다음 장부터 이러한 구조를 위한 Block 들을 하나씩 추가하도록 하겠습니다.

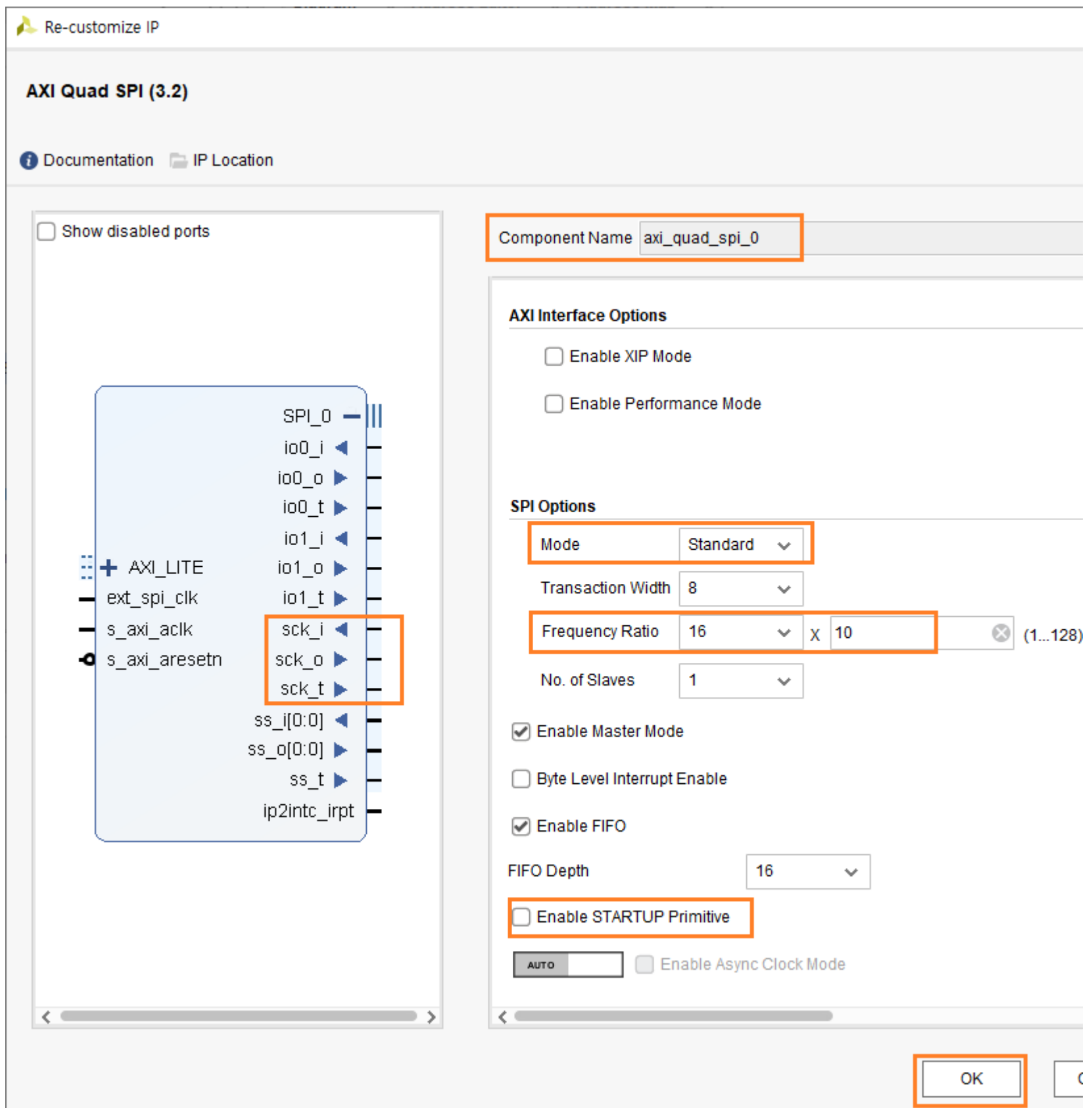
6.1 Block Design

4장에서 만든 기본 Template을 복사해서 사용합니다. ch06 폴더를 만들고 BlazeTop 폴더를 복사해서 폴더 이름을 BlazeTopSpi 로 합니다. vitis를 종료하고, vivado에 열린 project는 닫고, BlazeTopSpi 폴더내의 프로젝트를 Open 합니다. Open Block Design을 클릭하면 Diagram 윈도우가 나타납니다. Processor와 Uart는 이미 구현이 되었습니다. SPI Controller (Master) 를 추가합니다. Add IP 아이콘을 클릭 후, spi 로 검색하고, “AXI Quad SPI”를 더블 클릭해서 IP를 추가합니다.



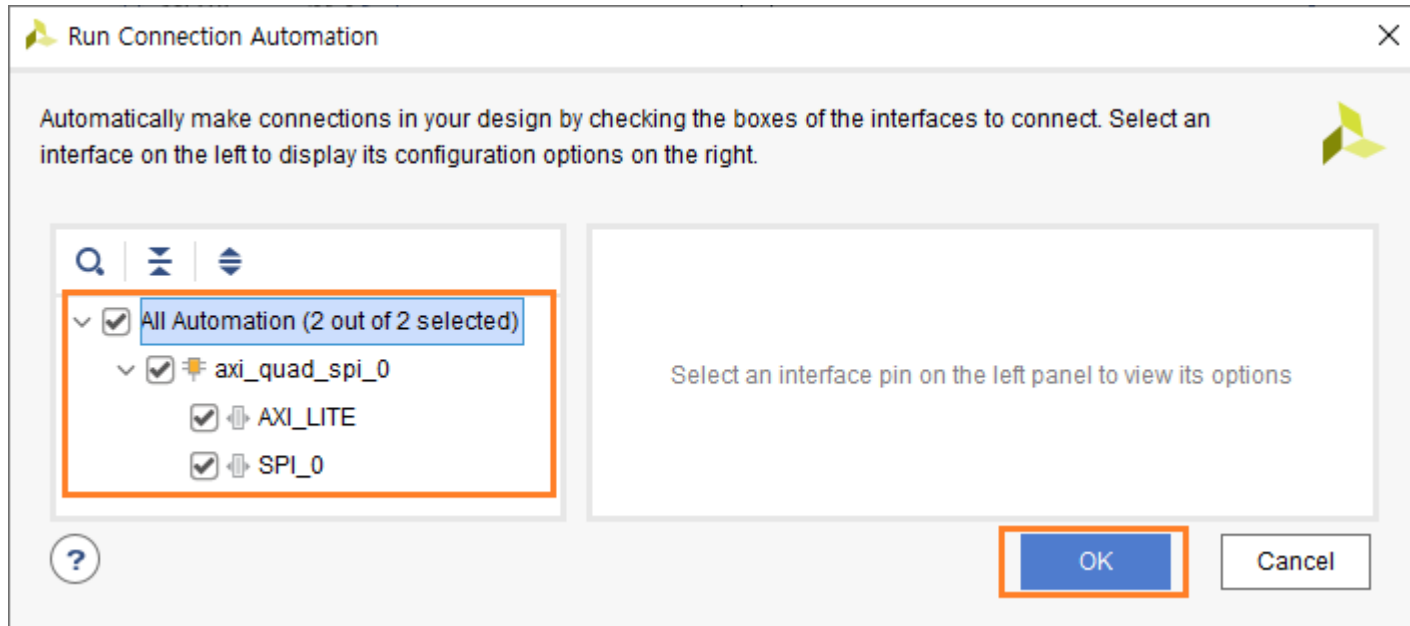
SPI_0 옆의 “+”를 클릭하면 세부 핀들을 확인할 수 있습니다. io0, io1, ss 3개의 핀들이 있고, 각각 input, output, tristate 핀으로 구성되어 있습니다. SPI Controller를 Master 혹은 Slave 로 사용할 수 있어서 input, output, tristate 3개의 핀으로 구성되어 있습니다. 그런데 SCK 핀이 보이지 않습니다. SPI Controller 를 Master로 사용하기 위해서는 SCK 핀이 필요합니다. 왜 SCK 핀이 기본적으로 생성되지 않았는지는 잘 모르겠습니다. SCK핀을 생성하기 위해서는 속성을 변경해 주어야 합니다. axi_quad_spi_0을 더블 클릭합니다. “Enable STARTUP Primitive”를 Uncheck 합니다. (이것을 알아내는데 많은 시간을 소요했습니다. πππ) 그리고 SPI 속도를 설정해 줍니다. 현재 사용하고 있는 System Clock 이 100Mhz 입니다. Frequency Ratio 를 16 x 10 정도로 설정하면, $100\text{Mhz}/160 = 0.625\text{Mhz}$ 가 됩니다.





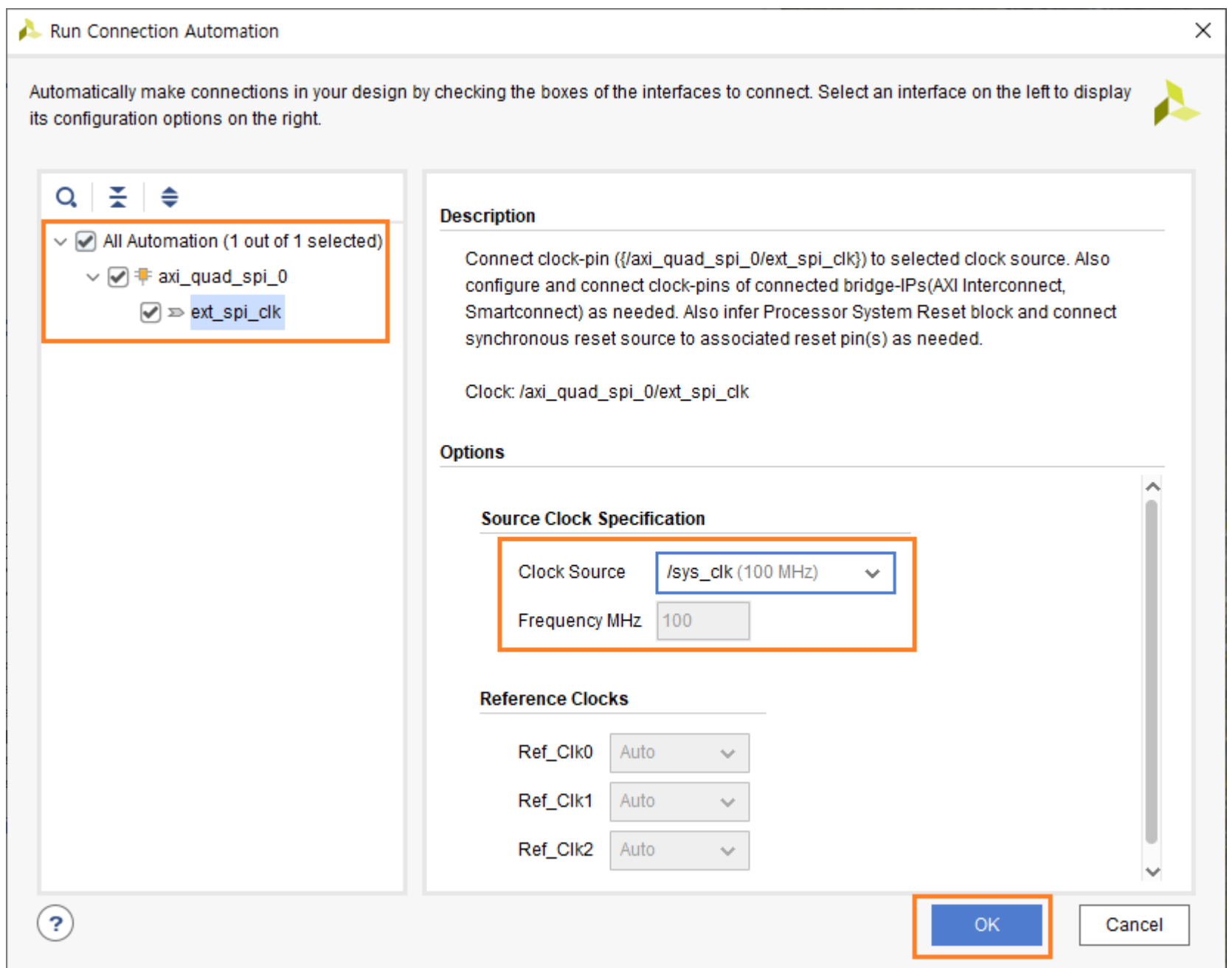
OK 버튼을 클릭합니다.

“Run Connection Automation”을 클릭해서 spi controller를 연결해 줍니다.

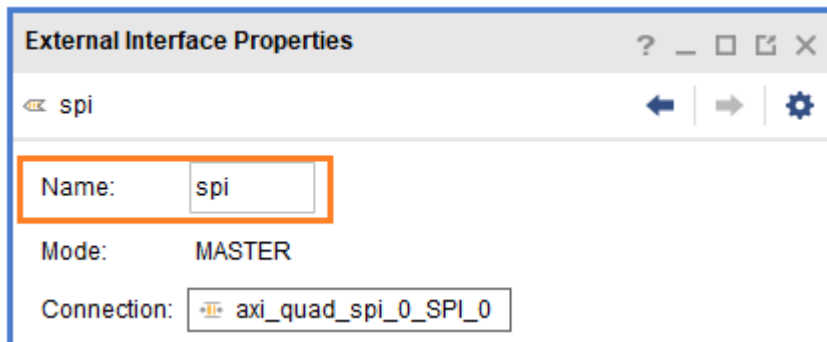


spi의 clk이 연결되지 않았습니다. 다시 한번 “Run Connection Automation”을 클릭합니다.

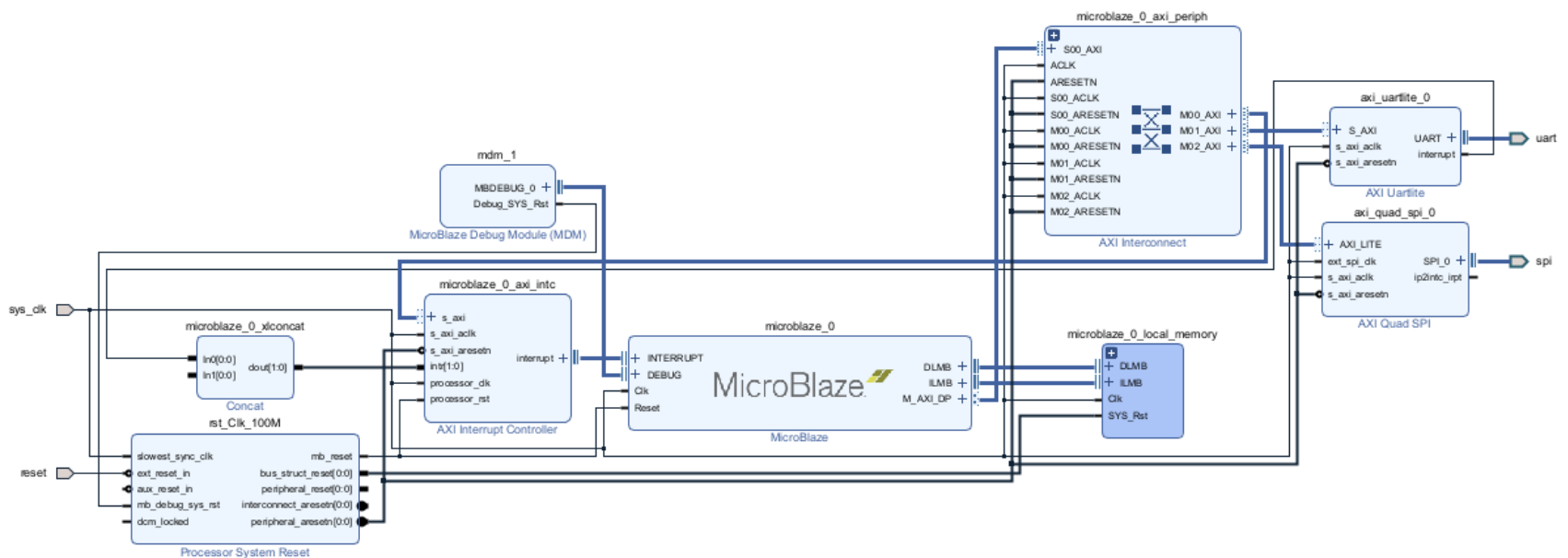
Clock Source : /sys_clk (100Mhz)를 확인후 OK 를 클릭합니다.



axi_quad_spi_0의 SPI_0 신호의 이름을 spi로 변경합니다. spi_rtl_0 포트를 클릭하면 포트 속성창이 나오는데 이름을 spi로 변경합니다.

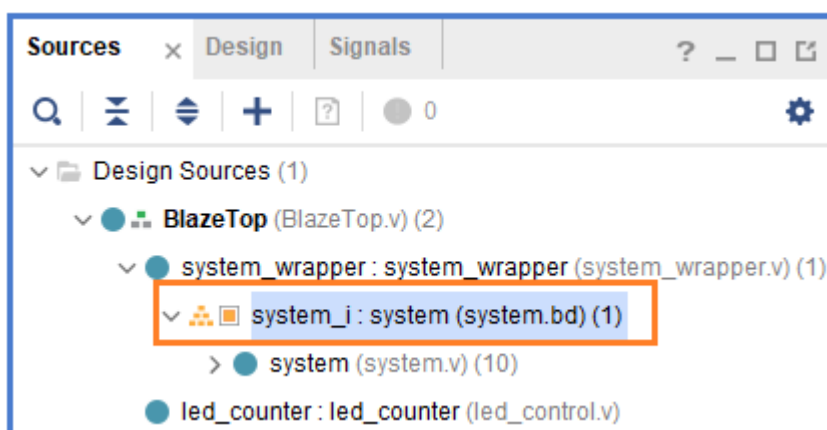


Validation Design 아이콘을 클릭해서 에러를 Check 합니다. 에러가 없음을 확인하고 Regenerate Layout을 클릭합니다. 최종적으로 생성된 Diagram 은 아래와 같습니다. (자료실에 diagram_ch6.png을 참조하세요)



Block Design이 완료되었습니다.

system_i - 우클릭 - Create HDL Wrapper... 클릭해서 Wrapper 파일을 생성합니다. (4.1.3 을 참조하세요)

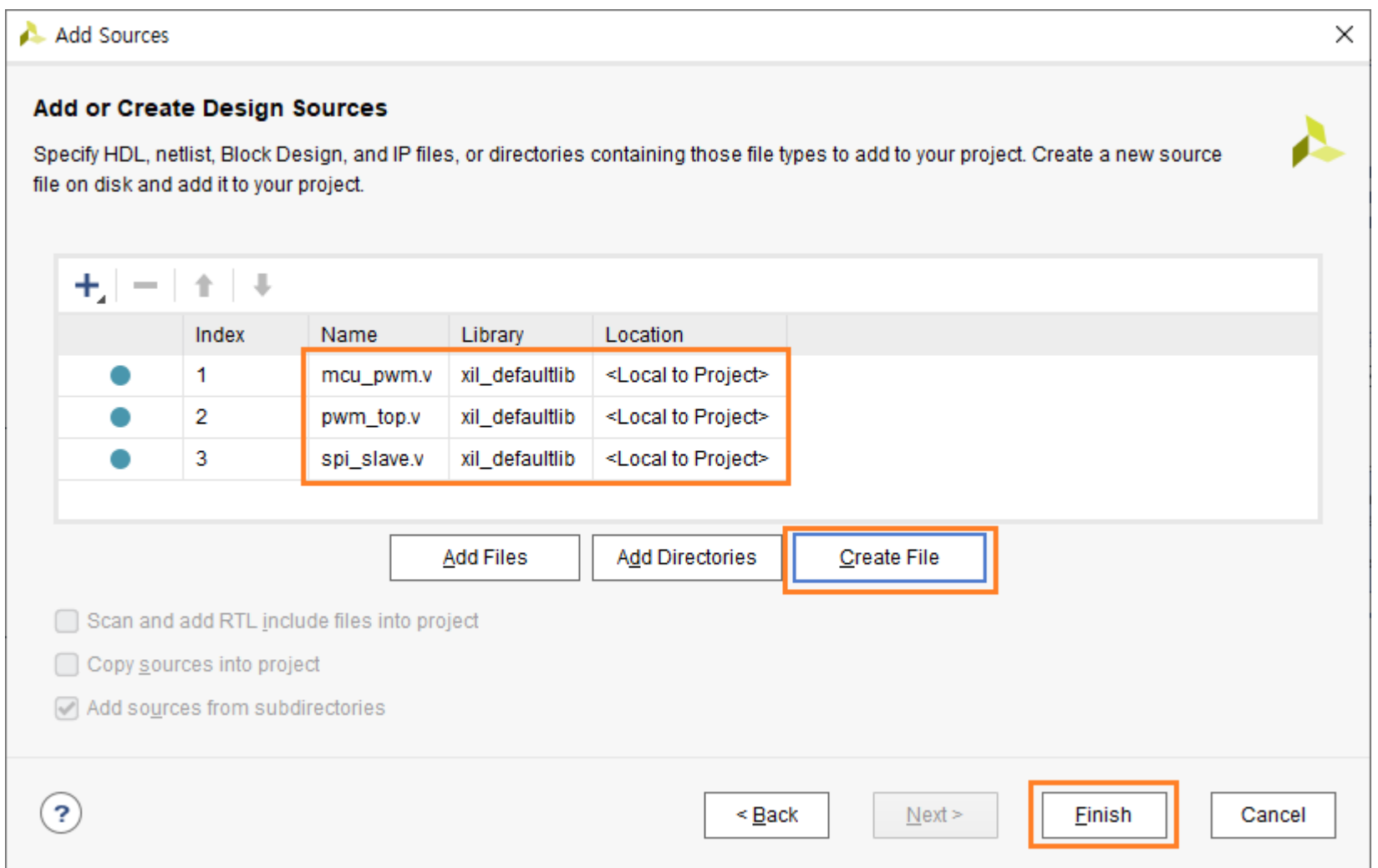


6.2 User Logic 구현

이번장에서는 User Logic을 구현합니다. 크게 3개의 모듈로 구성되어 있습니다.

- ✓ pwm_top : pwm 4개의 모듈로 구성
- ✓ spi_slave : MicroBlaze의 Spi Master와의 통신을 위한 모듈
- ✓ BlazeTop : Top 모듈

Design Sources 에 mcu_pwm.v, pwm_top.v, spi_slave.v 파일을 추가합니다.



코드를 구현합니다.

6.2.1 mcu_pwm.v

mcu_pwm 모듈은 입력으로 freq, duty를 받고 pwm 신호를 출력합니다.

◆ duty[6:0]

- ✓ 0 : low 출력
- ✓ 100 : High 출력
- ✓ 0 ~ 100 : duty를 갖는 pwm 출력

◆ freq[15:0]

- ✓ pwm frequency를 설정. frequency = (sys_clk) / ((freq + 1) * 101)

아래는 mcu_pwm.v 코드를 보여줍니다.

```

9 module mcu_pwm (
10     reset
11     ,
12     mclk
13     ,
14     freq
15     ,
16     duty
17     ,
18     out
19 );
20
21 input      reset ;
22 input      mclk  ;
23 input [15:0] freq ;
24 input [6:0]  duty ;
25 output      out  ;
26
27 reg [15:0] cnt_freq;
28 reg [6:0]  cnt_duty;
29 always @(posedge mclk or negedge reset)
30 begin
31     if(!reset) begin
32         cnt_freq <= 16'd0;
33         cnt_duty <= 7'd0;
34     end
35     else begin
36         cnt_freq <= (cnt_freq==freq) ? 16'd0 : cnt_freq + 1'b1;
37         cnt_duty <= (cnt_freq==16'd0) ? ((cnt_duty==7'd100) ? 7'd0 : (cnt_duty+1'b1)) : cnt_duty;
38     end
39 end
40
41 reg out;
42 always @(posedge mclk or negedge reset)
43 begin
44     if(!reset) out <= 1'b0;
45     else out <= (duty == 7'd0) ? 1'b0 :
46                 (duty == 7'd100) ? 1'b1 :
47                 (cnt_duty == 7'd0) ? 1'b1 :
48                 (cnt_duty == duty) ? 1'b0 : out;
49 end
50 endmodule

```

6.2.2 pwm_top.v

pwm_top 모듈은 4개의 mcu_pwm 모듈로 구성되어 있습니다. 4개의 pwm 출력을 구현합니다.

아래는 코드를 보여줍니다.

```

23 module pwm_top (
24     reset,
25     mclk ,
26
27     pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4,
28     pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4,
29
30     pwm_out1,  pwm_out2,  pwm_out3,  pwm_out4
31 );
32
33 input      reset;
34 input      mclk;
35
36 input  [15:0] pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4 ;
37 input  [6:0]  pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4 ;
38 output      pwm_out1,  pwm_out2,  pwm_out3,  pwm_out4 ;
39
40
41 wire        pwm_out1;
42 mcu_pwm     mcu_pwm1 (
43     .reset (reset      ),
44     .mclk  (mclk       ),
45     .freq  (pwm_freq1  ),
46     .duty  (pwm_duty1  ),
47     .out   (pwm_out1   )
48 );
49
50 wire        pwm_out2;
51 mcu_pwm     mcu_pwm2 (
52     .reset (reset      ),
53     .mclk  (mclk       ),
54     .freq  (pwm_freq2  ),
55     .duty  (pwm_duty2  ),
56     .out   (pwm_out2   )
57 );
58
59 wire        pwm_out3;
60 mcu_pwm     mcu_pwm3 (
61     .reset (reset      ),
62     .mclk  (mclk       ),
63     .freq  (pwm_freq3  ),
64     .duty  (pwm_duty3  ),
65     .out   (pwm_out3   )
66 );
67
68 wire        pwm_out4;
69 mcu_pwm     mcu_pwm4 (
70     .reset (reset      ),
71     .mclk  (mclk       ),
72     .freq  (pwm_freq4  ),
73     .duty  (pwm_duty4  ),
74     .out   (pwm_out4   )
75 );
76
77
78 endmodule

```

6.2.3 spi_slave.v

spi_slave 모듈은 MicroBlaze 의 Spi Master 와 통신을 하고, User Logic의 Register 를 Write/Read 합니다.

6.2.3.1 통신 프로토콜

통신 프로토콜은 slave_id, address, data 로 구성되어 있습니다.

slave_id (8bits)	address (8bits)	data (16bits)
write : 0x64 read : 0x65	0x00 ~ 0xff	0x0000 ~ 0xffff

Slave_id 는 data write 시 0x64, data read 시 0x65 로 구성되어 있습니다. 사용자가 원하는 값으로 수정할 수 있습니다.

6.2.3.2 레지스터 맵

레지스터 맵은 아래와 같습니다.

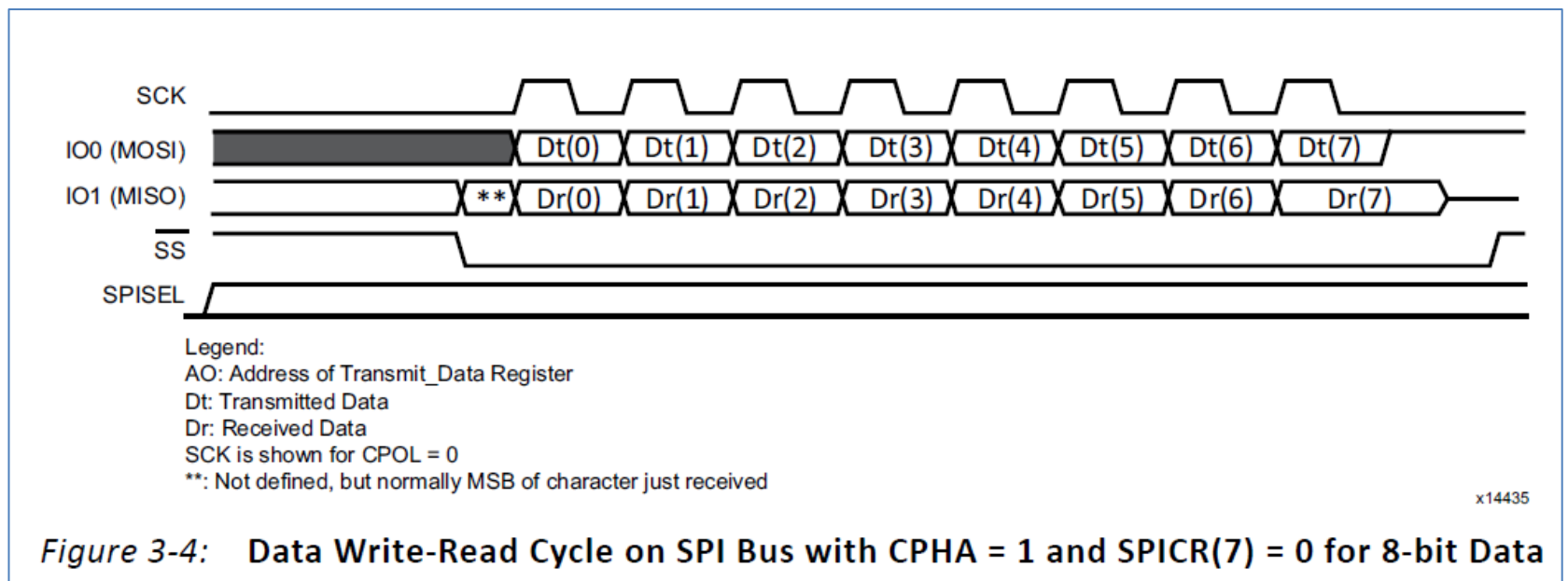
address	data	r/w	description
0x00 ~ 0x0F	-	-	reserved
0x10	pwm_freq1 [15:0]	r/w	pwm1 frequency, 0 ~ 65535
0x11	pwm_freq2 [15:0]	r/w	pwm1 frequency, 0 ~ 65535
0x12	pwm_freq3 [15:0]	r/w	pwm1 frequency, 0 ~ 65535
0x13	pwm_freq4 [15:0]	r/w	pwm1 frequency, 0 ~ 65535
0x14 ~ 0x1F	-	-	reserved
0x20	pwm_duty1 [6:0]	r/w	pwm1 duty, 0 ~ 100
0x21	pwm_duty2 [6:0]	r/w	pwm1 duty, 0 ~ 100
0x22	pwm_duty3 [6:0]	r/w	pwm1 duty, 0 ~ 100
0x23	pwm_duty4 [6:0]	r/w	pwm1 duty, 0 ~ 100
0x24 ~ 0xFF	-	-	reserved

6.2.3.3 파형 분석

SPI Slave를 구현하기 위해서는 MicroBlaze의 SPI Master Controller 의 파형을 알아야 합니다. (SPI 통신 인터페이스가 정해져 있지만, 그래도 파형을 분석하고 확인하는 것이 좋습니다. 그냥 “이렇게 되겠지 ~” 생각하고 진행하면 나중에 문제가 발생할 수 있습니다. 반드시 SPI Master 파형을 분석한 후, SPI Slave를 구현해야 합니다)

[그림 6-2]는 “AXI Quad SPI v3.2” User Manual 에 있는 파형입니다.

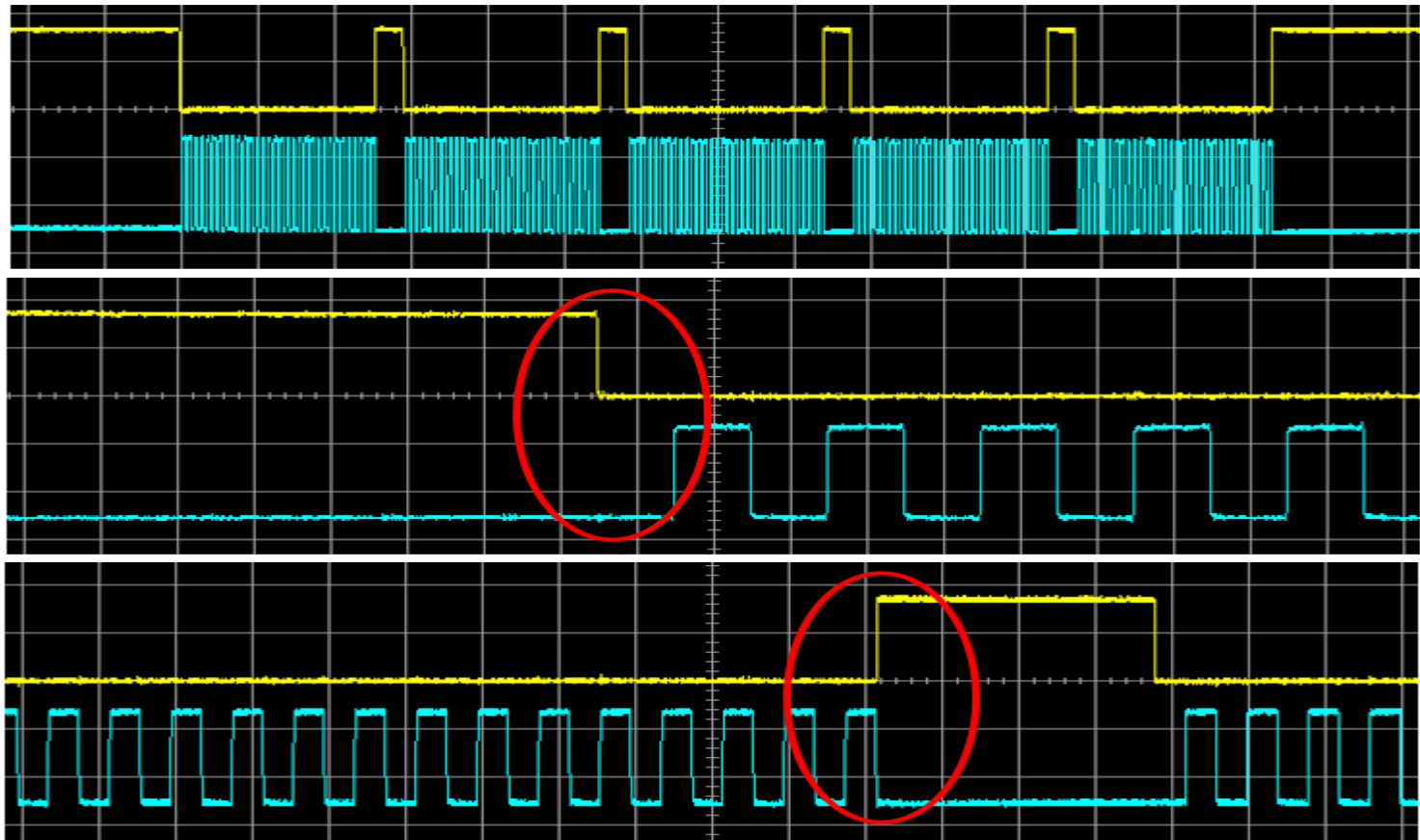
[그림 6-2] SPI 파형



SS가 Active(Low) 가 되고 SCK 에 따라서 데이터가 전송됩니다. SCK 가 Rising Edge 에서 데이터가 변하기 때문에 수신 측(Slave)에서는 Falling Edge에서 데이터를 받으면 됩니다. 그런데 실제 파형을 측정한 것과 약간의 차이가 발생하였습니다. [그림 6-2]에 나와 있는 대로 SPI Slave를 구현하고 보드에 올려서 테스트를 진행했는데 동작에 문제가 발생하였습니다. 어떤 때에는 동작하는데 어떤 때에는 동작하지 않았습니다. 이것으로 인해 SPI Slave를 구현하는데 많은 시간이 소모되었습니다. 급기야 MicroBlaze의 SPI Master에서 출력되는 파형을 스코프로 확인한 후에 문제를 파악할 수 있었습니다.

[그림 6-3]은 MicroBlaze의 SPI Master 에서 4바이트씩 5번 전송하도록 프로그램을 하고, SS 와 SCK의 파형을 스코프로 측정한 파형입니다. 노란색(CH1)이 SS 신호이고, 초록색(CH2)이 SCK 신호입니다.

[그림 6-3] MicroBlaze SPI 파형



첫번째 그림은 4바이트씩, 총 5번 전송하는 파형입니다. 예상대로 잘 동작하는 것 같습니다. SS가 Active(Low)일 때 SCK 파형이 발생합니다. 두번째 파형은 전송 시작시 SS와 SCK 를 보여줍니다. SS가 Active(Low)가 되고 얼마만큼의 delay후에 SCK가 발생합니다. 예상했던 파형입니다. 세번째 파형은 전송 완료시 SS와 SCK를 나타냅니다. 문제는 여기에 있었습니다. 통상적인 SPI 파형은 SCK가 Low가 되고 일정 delay 후에 SS신호가 “H”가 됩니다. 그러나 파형에서 보여지듯이 SS가 High가 되는 시점하고, SCK가 Low가 되는 시점이 같은 시간에 발생하였습니다. SPI Slave을 구현할 때, SCL가 Low가 된 후에 SS 신호가 High가 되면 전송을 완료하도록 프로그램 했었는데, 이 두 사건이 (SCL : H -> L, SS : L -> H) 동시에 발생하기 때문에 전송 완료가 되지 못하였습니다. (저자의 생각에 이는 MicroBlaze SPI Master Controller의 버그라 생각합니다)

SPI Slave Controller를 구현할 때 이 부분을 반드시 확인해야 합니다. Test bench를 만들어서 Simulation할 때 이 부분을 확인하도록 하겠습니다.

6.2.3.4 코드 구현

이번 장에서는 SPI Slave Controller를 구현합니다. 모듈 이름은 spi_slave(spi_slave.v) 입니다.

Port를 정의합니다.

signal	size	in/out	description
reset	[0]	in	reset, active low
clock	[0]	in	clock, 100 Mhz
ss	[0]	in	spi ss
sck	[0]	in	spi sck
mosi	[0]	in	spi mosi (master out slave in)
miso	[0]	out	spi miso (master in slave out)
pwm_freq1	[15:0]	out	pwm1 frequency
pwm_freq2	[15:0]	out	pwm2 frequency
pwm_freq3	[15:0]	out	pwm3 frequency
pwm_freq4	[15:0]	out	pwm4 frequency
pwm_duty1	[6:0]	out	pwm1 duty
pwm_duty2	[6:0]	out	pwm2 duty
pwm_duty3	[6:0]	out	pwm3 duty
pwm_duty4	[6:0]	out	pwm4 duty

```

22  `define      rPWM_FREQ1      8'h10
23  `define      rPWM_FREQ2      8'h11
24  `define      rPWM_FREQ3      8'h12
25  `define      rPWM_FREQ4      8'h13
26
27  `define      rPWM_DUTY1       8'h20
28  `define      rPWM_DUTY2       8'h21
29  `define      rPWM_DUTY3       8'h22
30  `define      rPWM_DUTY4       8'h23

```

✓ 라인 22 - 30 : define of register address

```

33 module spi_slave(
34     reset      ,
35     clock      ,
36     ss         ,
37     sck        ,
38     mosi       ,
39     miso       ,
40
41     pwm_freq1  ,
42     pwm_freq2  ,
43     pwm_freq3  ,
44     pwm_freq4  ,
45     pwm_duty1  ,
46     pwm_duty2  ,
47     pwm_duty3  ,
48     pwm_duty4  ,
49 );
50
51 input      reset ;
52 input      clock ;
53 input      ss    ;
54 input      sck   ;
55 input      mosi  ;
56 output     miso  ;
57 output [15:0] pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4 ;
58 output [6:0]  pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4 ;

```

✓ 라인 33 - 58 : port 선언

```

60 parameter      CHIP_IDW  = 8'h64 ;
61 parameter      CHIP_IDR  = 8'h65 ;
62
63 // -----
64 // 1) define state
65 parameter      IDLE      = 3'd0 ;
66 parameter      SLAVEID   = 3'd1 ;
67 parameter      WADDR     = 3'd2 ;
68 parameter      WDATA     = 3'd3 ;
69 parameter      RADDR     = 3'd4 ;
70 parameter      RDATA     = 3'd5 ;
71 parameter      DONE      = 3'd6 ;
72
73 // -----
74 // 2) state flag
75 reg [2:0]      s_state;
76 wire          s_idle    = (s_state==IDLE   ) ? 1'b1 : 1'b0;
77 wire          s_slaveid = (s_state==SLAVEID ) ? 1'b1 : 1'b0;
78 wire          s_waddr   = (s_state==WADDR   ) ? 1'b1 : 1'b0;
79 wire          s_wdata   = (s_state==WDATA   ) ? 1'b1 : 1'b0;
80 wire          s_raddr   = (s_state==RADDR   ) ? 1'b1 : 1'b0;
81 wire          s_rdata   = (s_state==RDATA   ) ? 1'b1 : 1'b0;
82 wire          s_done    = (s_state==DONE    ) ? 1'b1 : 1'b0;

```

✓ 라인 60 - 61 : address for write, read

✓ 라인 65 - 71 : state를 정의합니다. IDLE, SLAVEID, WADDR, WDATA, RADDR, RDATA, DONE 7개의 state를 정의합니다. SLAVEID state에서 들어오는 bit[0]의 값에 따라 Write state (WADDR, WDATA), Read State (RADDR, RDATA)로 이동합니다.

✓ 라인 75 - 82 : state flag를 정의합니다.

```

84 // -----
85 // 3) code implementation
86 reg          ss_ld, ss_2d ;
87 wire         ss_pedge = ss_ld & ~ss_2d;
88 wire         ss_nedge = ~ss_ld & ss_2d;
89 always @(posedge clock or negedge reset)
90 begin
91     if(!reset)      begin
92         ss_ld <= 1'b0;
93         ss_2d <= 1'b0;
94     end
95     else begin
96         ss_ld <= ss;
97         ss_2d <= ss_ld ;
98     end
99 end
100
101 reg          sck_ld, sck_2d ;
102 wire         sck_pedge = sck_ld & ~sck_2d;
103 wire         sck_nedge = ~sck_ld & sck_2d;
104 always @(posedge clock or negedge reset)
105 begin
106     if(!reset)      begin
107         sck_ld <= 1'b0;
108         sck_2d <= 1'b0;
109     end
110     else begin
111         sck_ld <= sck;
112         sck_2d <= sck_ld ;
113     end
114 end
115
116 reg          mosi_ld, mosi_2d;
117 always @(posedge clock or negedge reset)
118 begin
119     if(!reset)      begin
120         mosi_ld <= 1'b0;
121         mosi_2d <= 1'b0;
122     end
123     else begin
124         mosi_ld <= mosi;
125         mosi_2d <= mosi_ld ;
126     end
127 end

```

- ✓ 라인 86 - 99 : ss 신호의 positive edge, negative edge를 생성합니다.
- ✓ 라인 101 - 114 : sck 신호의 positive edge, negative edge를 생성합니다.
- ✓ 라인 116 - 127 : mosi 신호 입력은 2개의 flip/flop을 거친 신호를 사용합니다. 통상적으로 다른 clock domain에서 입력되는 신호를 사용할 때에는 이처럼 2~4개의 flip/flop을 거친 신호를 사용합니다.

```

129 reg    [3:0]    sid_sckn_cnt;
130 always @(posedge clock or negedge reset)
131 begin
132     if(!reset)    sid_sckn_cnt <= 4'b0;
133     else          sid_sckn_cnt <= ~s_slaveid ? 4'b0 :
134                     sck_nedge ? (sid_sckn_cnt+1'b1) : sid_sckn_cnt ;
135 end
136
137 reg    [7:0]    slave_id;
138 always @(posedge clock or negedge reset)
139 begin
140     if(!reset)    slave_id <= 8'b0;
141     else begin
142         slave_id[7] <= s_idle ? 1'b0 :
143             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd0)) ? mosi_2d : slave_id[7] ;
144         slave_id[6] <= s_idle ? 1'b0 :
145             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd1)) ? mosi_2d : slave_id[6] ;
146         slave_id[5] <= s_idle ? 1'b0 :
147             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd2)) ? mosi_2d : slave_id[5] ;
148         slave_id[4] <= s_idle ? 1'b0 :
149             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd3)) ? mosi_2d : slave_id[4] ;
150         slave_id[3] <= s_idle ? 1'b0 :
151             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd4)) ? mosi_2d : slave_id[3] ;
152         slave_id[2] <= s_idle ? 1'b0 :
153             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd5)) ? mosi_2d : slave_id[2] ;
154         slave_id[1] <= s_idle ? 1'b0 :
155             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd6)) ? mosi_2d : slave_id[1] ;
156         slave_id[0] <= s_idle ? 1'b0 :
157             (s_slaveid & sck_pedge & (sid_sckn_cnt==4'd7)) ? mosi_2d : slave_id[0] ;
158     end
159 end

```

✓ 라인 129 - 135 : SLAVEID 상태에서 sck negative edge를 count 하는 counter를 생성합니다.

✓ 라인 137 - 159 : sid_sckn_cnt 값에 따라서 slave_id 값을 생성합니다.

```

162 reg    [3:0]    wa_sckn_cnt;
163 always @(posedge clock or negedge reset)
164 begin
165     if(!reset)    wa_sckn_cnt <= 4'b0;
166     else          wa_sckn_cnt <= ~s_waddr ? 4'b0 :
167                     sck_nedge ? (wa_sckn_cnt+1'b1) : wa_sckn_cnt ;
168 end
169
170 reg    [7:0]    waddr;
171 always @(posedge clock or negedge reset)
172 begin
173     if(!reset)    waddr <= 8'b0;
174     else begin
175         waddr[7] <= s_idle ? 1'b0 :
176             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd0)) ? mosi_2d : waddr[7] ;
177         waddr[6] <= s_idle ? 1'b0 :
178             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd1)) ? mosi_2d : waddr[6] ;
179         waddr[5] <= s_idle ? 1'b0 :
180             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd2)) ? mosi_2d : waddr[5] ;
181         waddr[4] <= s_idle ? 1'b0 :
182             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd3)) ? mosi_2d : waddr[4] ;
183         waddr[3] <= s_idle ? 1'b0 :
184             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd4)) ? mosi_2d : waddr[3] ;
185         waddr[2] <= s_idle ? 1'b0 :
186             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd5)) ? mosi_2d : waddr[2] ;
187         waddr[1] <= s_idle ? 1'b0 :
188             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd6)) ? mosi_2d : waddr[1] ;
189         waddr[0] <= s_idle ? 1'b0 :
190             (s_waddr & sck_pedge & (wa_sckn_cnt==4'd7)) ? mosi_2d : waddr[0] ;
191     end
192 end

```

✓ 라인 162 - 168 : WADDR 상태에서 sck negative edge를 count 하는 counter를 생성합니다.

✓ 라인 170 - 192 : wa_sckn_cnt 값에 따라서 waddr 값을 생성합니다.


```

195 reg      [4:0]   wd_sckn_cnt;
196 always @(posedge clock or negedge reset)
197 begin
198     if(!reset)      wd_sckn_cnt <= 5'b0;
199     else             wd_sckn_cnt <= ~s_wdata ? 5'b0 :
200                               sck_nedge ? (wd_sckn_cnt+1'b1) : wd_sckn_cnt ;
201 end
202
203 reg      [15:0]   wdata;
204 always @(posedge clock or negedge reset)
205 begin
206     if(!reset)      wdata <= 16'b0;
207     else begin
208         wdata[15] <= s_idle ? 1'b0 :
209                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd0)) ? mosi_2d : wdata[15] ;
210         wdata[14] <= s_idle ? 1'b0 :
211                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd1)) ? mosi_2d : wdata[14] ;
212         wdata[13] <= s_idle ? 1'b0 :
213                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd2)) ? mosi_2d : wdata[13] ;
214         wdata[12] <= s_idle ? 1'b0 :
215                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd3)) ? mosi_2d : wdata[12] ;
216         wdata[11] <= s_idle ? 1'b0 :
217                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd4)) ? mosi_2d : wdata[11] ;
218         wdata[10] <= s_idle ? 1'b0 :
219                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd5)) ? mosi_2d : wdata[10] ;
220         wdata[9]  <= s_idle ? 1'b0 :
221                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd6)) ? mosi_2d : wdata[9] ;
222         wdata[8]  <= s_idle ? 1'b0 :
223                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd7)) ? mosi_2d : wdata[8] ;
224         wdata[7]  <= s_idle ? 1'b0 :
225                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd8)) ? mosi_2d : wdata[7] ;
226         wdata[6]  <= s_idle ? 1'b0 :
227                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd9)) ? mosi_2d : wdata[6] ;
228         wdata[5]  <= s_idle ? 1'b0 :
229                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd10)) ? mosi_2d : wdata[5] ;
230         wdata[4]  <= s_idle ? 1'b0 :
231                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd11)) ? mosi_2d : wdata[4] ;
232         wdata[3]  <= s_idle ? 1'b0 :
233                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd12)) ? mosi_2d : wdata[3] ;
234         wdata[2]  <= s_idle ? 1'b0 :
235                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd13)) ? mosi_2d : wdata[2] ;
236         wdata[1]  <= s_idle ? 1'b0 :
237                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd14)) ? mosi_2d : wdata[1] ;
238         wdata[0]  <= s_idle ? 1'b0 :
239                       (s_wdata & sck_pedge & (wd_sckn_cnt==5'd15)) ? mosi_2d : wdata[0] ;
240     end
241 end

```

- ✓ 라인 195 - 201 : WDATA 상태에서 sck negative edge를 count하는 counter를 생성합니다.
- ✓ 라인 203 - 241 : wd_sckn_cnt 값에 따라서 wdata 값을 생성합니다.

```

244 reg      [3:0]   ra_sckn_cnt;
245 always @(posedge clock or negedge reset)
246 begin
247     if(!reset)      ra_sckn_cnt <= 4'b0;
248     else              ra_sckn_cnt <= ~s_raddr ? 4'b0 :
249                             sck_nedge ? (ra_sckn_cnt+1'b1) : ra_sckn_cnt ;
250 end
251
252 reg      [7:0]   raddr;
253 always @(posedge clock or negedge reset)
254 begin
255     if(!reset)      raddr <= 8'b0;
256     else begin
257         raddr[7] <= s_idle ? 1'b0 :
258                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd0)) ? mosi_2d : raddr[7] ;
259         raddr[6] <= s_idle ? 1'b0 :
260                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd1)) ? mosi_2d : raddr[6] ;
261         raddr[5] <= s_idle ? 1'b0 :
262                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd2)) ? mosi_2d : raddr[5] ;
263         raddr[4] <= s_idle ? 1'b0 :
264                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd3)) ? mosi_2d : raddr[4] ;
265         raddr[3] <= s_idle ? 1'b0 :
266                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd4)) ? mosi_2d : raddr[3] ;
267         raddr[2] <= s_idle ? 1'b0 :
268                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd5)) ? mosi_2d : raddr[2] ;
269         raddr[1] <= s_idle ? 1'b0 :
270                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd6)) ? mosi_2d : raddr[1] ;
271         raddr[0] <= s_idle ? 1'b0 :
272                     (s_raddr & sck_pedge & (ra_sckn_cnt==4'd7)) ? mosi_2d : raddr[0] ;
273     end
274 end

```

- ✓ 라인 244 - 250 : RADDR 상태에서 sck negative edge를 count하는 counter를 생성합니다.
- ✓ 라인 252 - 274 : ra_sckn_cnt 값에 따라서 raddr 값을 생성합니다.


```

277 reg      [4:0]   rd_sckn_cnt;
278 always @(posedge clock or negedge reset)
279 begin
280     if(!reset) rd_sckn_cnt <= 5'b0;
281     else       rd_sckn_cnt <= ~s_rdata ? 5'b0 : sck_nedge ? (rd_sckn_cnt+1'b1) : rd_sckn_cnt ;
282 end
283
284 reg      sck_nedge_ld;
285 always @(posedge clock or negedge reset)
286 begin
287     if(!reset) sck_nedge_ld <= 1'b0;
288     else       sck_nedge_ld <= sck_nedge;
289 end
290
291 reg      sck_pedge_ld;
292 always @(posedge clock or negedge reset)
293 begin
294     if(!reset) sck_pedge_ld <= 1'b0;
295     else       sck_pedge_ld <= sck_pedge;
296 end
297
298
299 reg      [15:0]   rdata;
300 reg      miso;
301 always @(posedge clock or negedge reset)
302 begin
303     if(!reset) miso <= 1'b0;
304     else       miso <= s_idle ? 1'b0 :
305         (sck_nedge_ld & (rd_sckn_cnt==5'd0)) ? rdata[15] :
306         (sck_nedge_ld & (rd_sckn_cnt==5'd1)) ? rdata[14] :
307         (sck_nedge_ld & (rd_sckn_cnt==5'd2)) ? rdata[13] :
308         (sck_nedge_ld & (rd_sckn_cnt==5'd3)) ? rdata[12] :
309         (sck_nedge_ld & (rd_sckn_cnt==5'd4)) ? rdata[11] :
310         (sck_nedge_ld & (rd_sckn_cnt==5'd5)) ? rdata[10] :
311         (sck_nedge_ld & (rd_sckn_cnt==5'd6)) ? rdata[9] :
312         (sck_nedge_ld & (rd_sckn_cnt==5'd7)) ? rdata[8] :
313         (sck_nedge_ld & (rd_sckn_cnt==5'd8)) ? rdata[7] :
314         (sck_nedge_ld & (rd_sckn_cnt==5'd9)) ? rdata[6] :
315         (sck_nedge_ld & (rd_sckn_cnt==5'd10)) ? rdata[5] :
316         (sck_nedge_ld & (rd_sckn_cnt==5'd11)) ? rdata[4] :
317         (sck_nedge_ld & (rd_sckn_cnt==5'd12)) ? rdata[3] :
318         (sck_nedge_ld & (rd_sckn_cnt==5'd13)) ? rdata[2] :
319         (sck_nedge_ld & (rd_sckn_cnt==5'd14)) ? rdata[1] :
320         (sck_nedge_ld & (rd_sckn_cnt==5'd15)) ? rdata[0] : miso ;
321 end
322
323
324 reg      [1:0]   done_cnt;
325 always @(posedge clock or negedge reset)
326 begin
327     if(!reset) done_cnt <= 2'b0;
328     else       done_cnt <= ~s_done ? 2'b0 : done_cnt+1'b1 ;
329 end

```

- ✓ 라인 277 - 282 : RDATA 상태에서 sck negative edge를 count하는 counter를 생성합니다.
- ✓ 라인 284 - 296 : 타이밍을 맞추기 위해서 sck_nedge_ld, sck_pedge_ld를 생성합니다. 코딩할 때에는 알아내기 어렵습니다. simulation을 진행하면서 타이밍을 맞추는 과정에서 추가되는 신호들입니다. 저도 spi_slave.v를 구현하기 위해서, simulation을 진행하고 타이밍 맞추고 하는 과정을 수없이 많이 진행하였습니다.
- ✓ 라인 299 - 321 : rd_sckn_cnt 값에 따라서 rdata 값을 생성합니다.
- ✓ 라인 324 - 329 : DONE 상태의 counter를 생성합니다.

```

331 // -----
332 // 4) state transition
333 always @(posedge clock or negedge reset)
334 begin
335     if(!reset) begin
336         s_state <= 3'h0;
337     end
338     else begin
339         s_state <= (s_idle & ss_nedge) ? SLAVEID :
340             (s_slaveid & (sid_sckn_cnt==4'd8)) ? ((slave_id==CHIP_IDW) ? WADDR :
341                 (slave_id==CHIP_IDR) ? RADDR : IDLE) :
342             (s_waddr & (wa_sckn_cnt==4'd8)) ? WDATA :
343             (s_wdata & ss_pedge) ? DONE :
344             (s_raddr & (ra_sckn_cnt==4'd8)) ? RDATA :
345             (s_rdata & ss_pedge) ? DONE :
346             (s_done & (done_cnt==2'd3)) ? IDLE : s_state ;
347     end
348 end

```

- ✓ 라인 331 - 348 : 상태 변이를 구현합니다. 340-341 라인의 수신된 slave_id 값에 따라서 WADDR, RADDR 상태로 각각 이동함에 주의하시길 바랍니다.

```

351 reg [15:0] pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4 ;
352 reg [6:0]  pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4 ;
353 always @(posedge clock or negedge reset)
354 begin
355     if(!reset) begin
356         pwm_freq1 <= 16'd100;
357         pwm_freq2 <= 16'd100;
358         pwm_freq3 <= 16'd100;
359         pwm_freq4 <= 16'd100;
360         pwm_duty1 <= 7'd50;
361         pwm_duty2 <= 7'd50;
362         pwm_duty3 <= 7'd50;
363         pwm_duty4 <= 7'd50;
364     end
365     else begin
366         pwm_freq1 <= (s_done & (waddr==`rPWM_FREQ1)) ? wdata : pwm_freq1 ;
367         pwm_freq2 <= (s_done & (waddr==`rPWM_FREQ2)) ? wdata : pwm_freq2 ;
368         pwm_freq3 <= (s_done & (waddr==`rPWM_FREQ3)) ? wdata : pwm_freq3 ;
369         pwm_freq4 <= (s_done & (waddr==`rPWM_FREQ4)) ? wdata : pwm_freq4 ;
370         pwm_duty1 <= (s_done & (waddr==`rPWM_DUTY1)) ? wdata[6:0] : pwm_duty1 ;
371         pwm_duty2 <= (s_done & (waddr==`rPWM_DUTY2)) ? wdata[6:0] : pwm_duty2 ;
372         pwm_duty3 <= (s_done & (waddr==`rPWM_DUTY3)) ? wdata[6:0] : pwm_duty3 ;
373         pwm_duty4 <= (s_done & (waddr==`rPWM_DUTY4)) ? wdata[6:0] : pwm_duty4 ;
374     end
375 end

```

- ✓ 라인 351 - 375 : SPI Write 동작시에 각각의 register 값을 wdata 값으로 변경(write)합니다.

```

378 always @(posedge clock or negedge reset)
379 begin
380     if(!reset) rdata <= 16'b0;
381     else rdata <= s_idle ? 16'b0 :
382         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ1)) ? pwm_freq1 :
383         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ2)) ? pwm_freq2 :
384         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ3)) ? pwm_freq3 :
385         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ4)) ? pwm_freq4 :
386         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY1)) ? {9'b0, pwm_duty1} :
387         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY2)) ? {9'b0, pwm_duty2} :
388         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY3)) ? {9'b0, pwm_duty3} :
389         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY4)) ? {9'b0, pwm_duty4} :
390         rdata ;
391 end
392
393
394 endmodule

```

- ✓ 라인 378 - 394 : SPI Read 동작시에 각각의 register 값을 rdata에 넘겨주어, rdata 값이 miso 신호로 출력되도록 합니다. rdata 값은 miso 신호로 출력되기 전에 생성되어야 합니다.

6.2.3.5 simulation

이번장에서는 spi_slave.v 를 simulation을 통하여 결과를 확인합니다. Test bench는 tb_spi_slave.v 입니다.

Sources - Simulation Sources - 우클릭 - Add Sources... 클릭해서 “tp_spi_slave.v” 파일을 추가합니다. 소스의 양이 많지만, 대부분 ss, sck, sda 신호를 타이밍에 맞추어 생성하는 코드입니다. 세부적인 내용은 생략합니다. 다음은 전체적인 시퀀스를 보여줍니다.

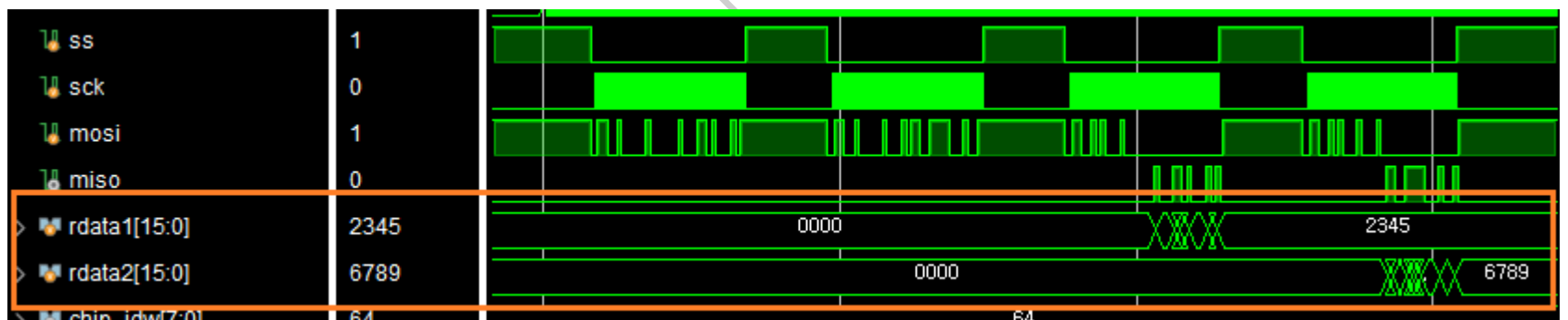
tb_spi_slave.v 는 아래와 같은 동작을 합니다.

- ✓ spi write 1 : 0x10 (pwm_freq1) 번지에 0x2345 값을 write 합니다.
- ✓ spi_write 2 : 0x11 (pwm_freq2) 번지에 0x6789 값을 write 합니다.
- ✓ spi_read 1 : 0x10 번지의 값을 read 합니다. (0x2345 가 read 됩니다)
- ✓ spi_read 2 : 0x11 번지의 값을 read 합니다. (0x6789 가 read 됩니다)

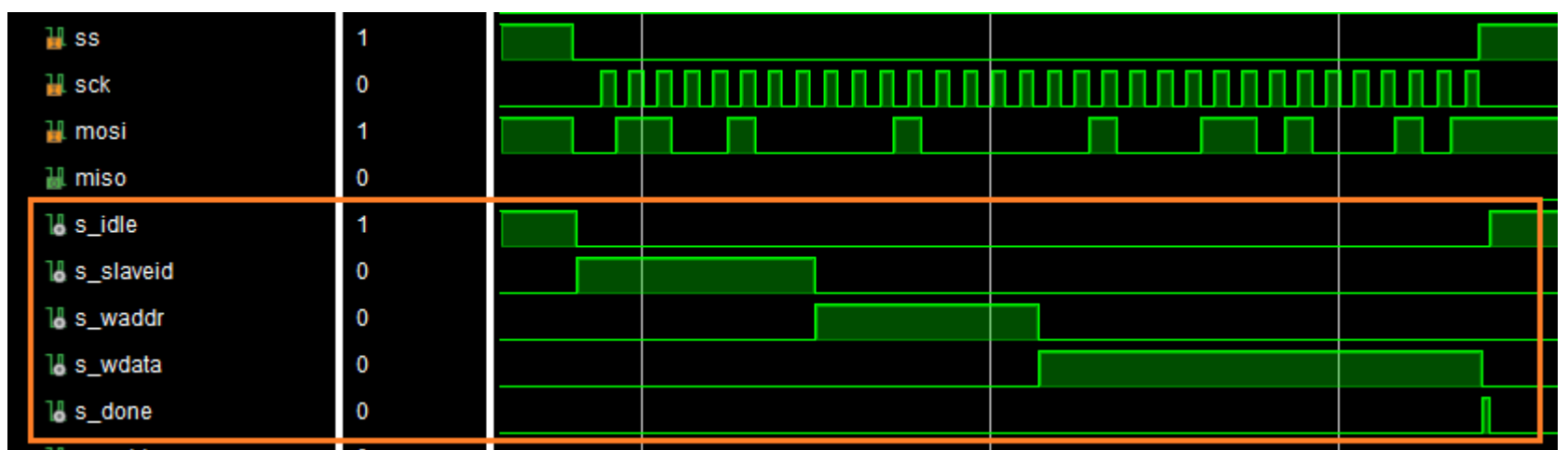
자료실에 있는 코드를 복사해서 tb_spi_slave.v 에 추가합니다. simulation을 위해서 tb_spi_slave - 우클릭 - Set as Top 을 클릭해서 tb_spi_slave를 Top Module로 설정하고 simulation을 진행합니다.

spi_slave 신호들을 wave 윈도우에 추가하고 “0.05ms” 동안 simulation을 진행합니다.

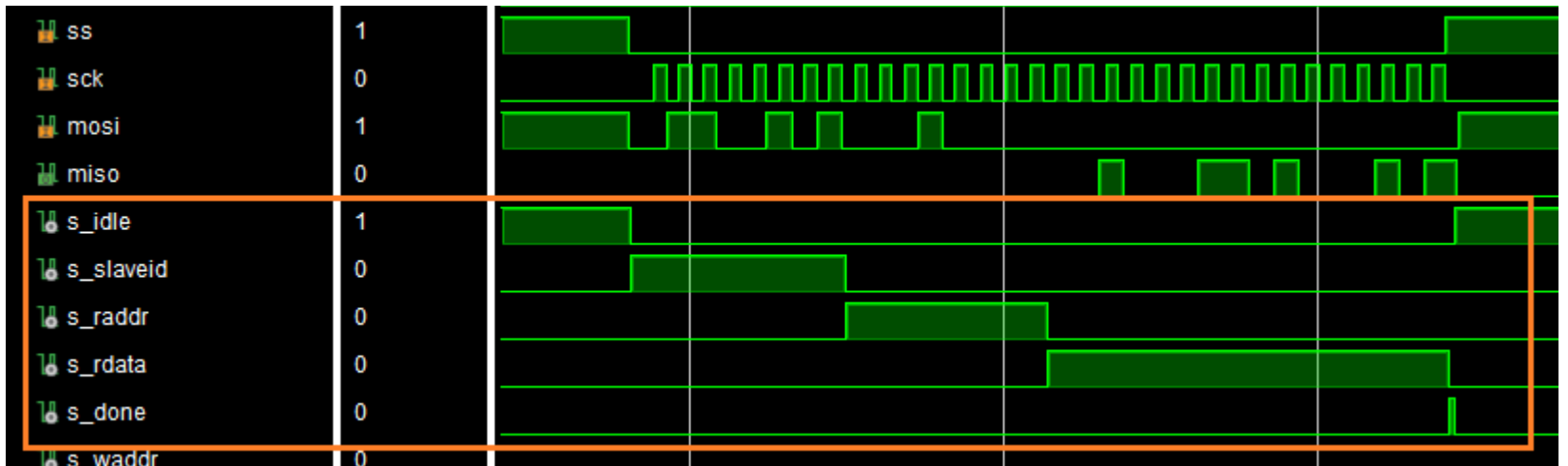
아래 그림은 전체적인 파형을 보여줍니다. rdata1은 0x10 번지의 read 값이고, rdata2는 0x11 번지의 read 값입니다. read 동작후에 각각의 값들이 0x2345, 0x6789로 업데이트 되었습니다. 이는 write, read 동작이 정상적으로 동작함을 알려줍니다.



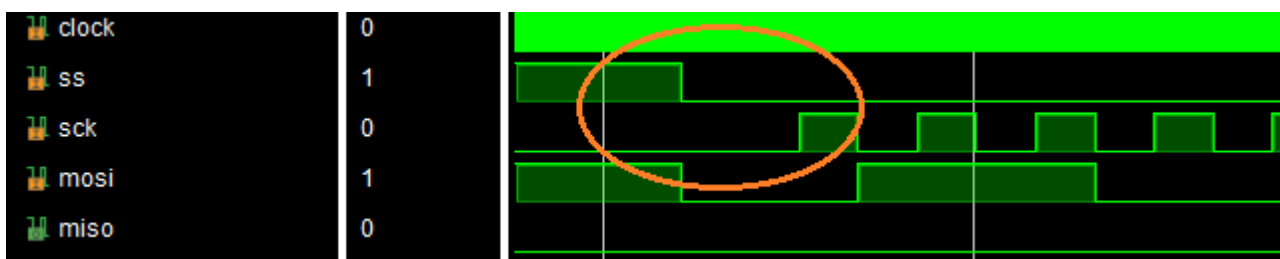
아래 그림은 write 동작에서의 각 state를 보여줍니다. s_idle, s_slaveid, s_waddr, s_wdata, s_done, s_idle 순서로 상태가 변하는 것을 알 수 있습니다.



아래 그림은 read 동작에서의 각 state를 보여줍니다. s_idle, s_slaveid, s_raddr, s_rdata, s_done, s_idle 순서로 상태가 변하는 것을 알 수 있습니다.



아래 그림은 write, read를 시작하는 부분의 ss, sck 를 보여줍니다. ss 신호가 Active 되고 일정 시간후에 sck 가 H 로 변하는 것을 알 수 있습니다. tb_spi_slave.v 에서 ss, sck 신호를 생성할 때, 데이터 시트에 맞게 생성함을 알 수 있습니다.



아래 그림은 write, read가 끝나는 부분의 ss, sck 를 보여줍니다. ss 신호가 Not Active (L -> H) 되는 시점과 sck 신호가 H->L 로 변하는 시점을 동일한 시간으로 만들었습니다. 앞장에서 살펴보았듯이 MicroBlaze의 SPI Master 파형을 참고하여 그대로 생성 하였습니다. (75page, [그림 6-3] 참고) 이처럼 test bench를 구현할 때에는 최대한 실제상황과 동일하게 만들어 주는 것이 중요합니다.



spi_slave.v 구현이 완료되었습니다. 다음장에서는 BlazeTop 모듈을 구현합니다.

6.2.4 BlazeTop.v

이번장에서는 BlazeTop 모듈을 구현하도록 하겠습니다. BlazeTop 모듈에 pwm_top, spi_slave 모듈을 추가하고, MicroBlaze 모듈의 변경된 포트를 추가합니다.

아래는 system_wrapper 모듈을 보여줍니다. spi 관련 신호들이 추가되었습니다.

```

12 module system_wrapper
13     (reset,
14      spi_io0_io,
15      spi_iol_io,
16      spi_sck_io,
17      spi_ss_io,
18      sys_clk,
19      uart_rxd,
20      uart_txd);
21 input reset;
22 inout spi_io0_io;
23 inout spi_iol_io;
24 inout spi_sck_io;
25 inout [0:0]spi_ss_io;
26 input sys_clk;
27 input uart_rxd;
28 output uart_txd;
29
30 wire reset;
31 wire spi_io0_i;
32 wire spi_io0_o;
33 wire spi_io0_t;
34 wire spi_iol_i;
35 wire spi_iol_o;
36 wire spi_iol_t;
37 wire spi_sck_i;
38 wire spi_sck_o;
39 wire spi_sck_t;
40 wire [0:0]spi_ss_i_0;
41 wire [0:0]spi_ss_o_0;
42 wire spi_ss_t;
43 wire sys_clk;
44 wire uart_rxd;
45 wire uart_txd;
46
47 IOBUF spi_io0_iobuf
48     (.I(spi_io0_o),
49      .IO(spi_io0_io),
50      .O(spi_io0_i),
51      .T(spi_io0_t));
52 IOBUF spi_iol_iobuf
53     (.I(spi_iol_o),
54      .IO(spi_iol_io),
55      .O(spi_iol_i),
56      .T(spi_iol_t));
57 IOBUF spi_sck_iobuf
58     (.I(spi_sck_o),
59      .IO(spi_sck_io),
60      .O(spi_sck_i),
61      .T(spi_sck_t));
62 IOBUF spi_ss_iobuf_0
63     (.I(spi_ss_o_0),
64      .IO(spi_ss_io[0]),
65      .O(spi_ss_i_0),
66      .T(spi_ss_t));

```

in/out port 중에 spi 관련 신호들이 inout 으로 선언되고, 아래에 IOBUF 를 사용하였습니다. 우리가 구현한 spi_slave 모듈은 inout port로 구현하지 않았기 때문에 바로 연결하면 에러가 발생합니다. 따라서 system_wrapper 모듈을 사용하는 대신에 system 모듈을 사용하도록 하겠습니다.

아래는 코드를 보여줍니다.

```

23 module BlazeTop(
24     reset      ,
25     sys_clk    ,
26     uart_rxd   ,
27     uart_txd   ,
28     led        ,
29     pwm_out    ,
30     tp_ja1     ,
31     tp_ja2     ,
32     tp_ja3     ,
33     tp_ja4     ,
34 );
35 input      reset      ;
36 input      sys_clk    ;
37 input      uart_rxd   ;
38 output     uart_txd   ;
39 output [3:0] pwm_out   ;
40 output [3:0] led       ;
41 output     tp_ja1     ;
42 output     tp_ja2     ;
43 output     tp_ja3     ;
44 output     tp_ja4     ;

```

- ✓ 라인 29 : 4개의 pwm_out 이 추가되었습니다.
- ✓ 라인 30 - 33 : spi 신호들을 스코프로 확인하기 위하여 tp 신호를 통해 출력합니다.

```

46 wire      spi_ss      ;
47 wire      spi_sck     ;
48 wire      spi_mosi    ;
49 wire      spi_miso    ;
50 system    system_i (
51     .reset      (reset      ),
52     .sys_clk    (sys_clk    ),
53     .spi_io0_i  (1'b0      ),
54     .spi_io0_o  (spi_mosi   ),
55     .spi_io0_t  (           ),
56     .spi_iol_i  (spi_miso   ),
57     .spi_iol_o  (           ),
58     .spi_iol_t  (           ),
59     .spi_sck_i  (1'b0      ),
60     .spi_sck_o  (spi_sck    ),
61     .spi_sck_t  (           ),
62     .spi_ss_i   (1'b0      ),
63     .spi_ss_o   (spi_ss     ),
64     .spi_ss_t   (           ),
65     .uart_rxd   (uart_rxd   ),
66     .uart_txd   (uart_txd   )
67 );

```

- ✓ 라인 46 - 49 : spi_slave.v 모듈의 port가 system 모듈과 연결됩니다.
- ✓ 라인 52 - 63 : input port를 사용하지 않으면 0을 입력합니다. spi_slave 모듈과 연결합니다.


```

69 wire    [3:0]    led ;
70 led_counter    led_counter(
71     .reset      (reset      ),
72     .clock       (sys_clk    ),
73     .led         (led        )
74 );
75
76 // -----
77 // pwm_top
78 wire    [15:0]    pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4 ;
79 wire    [6:0]     pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4 ;
80 wire                                pwm_out1, pwm_out2, pwm_out3, pwm_out4 ;
81 wire    [3:0]     pwm_out ;
82 pwm_top          u_pwm_top (
83     .reset        (reset      ),
84     .mclk         (sys_clk    ),
85
86     .pwm_freq1    (pwm_freq1  ),
87     .pwm_freq2    (pwm_freq2  ),
88     .pwm_freq3    (pwm_freq3  ),
89     .pwm_freq4    (pwm_freq4  ),
90     .pwm_duty1    (pwm_duty1  ),
91     .pwm_duty2    (pwm_duty2  ),
92     .pwm_duty3    (pwm_duty3  ),
93     .pwm_duty4    (pwm_duty4  ),
94
95     .pwm_out1     (pwm_out[0] ),
96     .pwm_out2     (pwm_out[1] ),
97     .pwm_out3     (pwm_out[2] ),
98     .pwm_out4     (pwm_out[3] )
99 );
100
101 // -----
102 // spi_slave
103 spi_slave        u_spi_slave(
104     .reset        (reset      ),
105     .clock        (sys_clk    ),
106     .ss           (spi_ss     ),
107     .sck          (spi_sck    ),
108     .mosi         (spi_mosi   ),
109     .miso         (spi_miso   ),
110
111     .pwm_freq1    (pwm_freq1  ),
112     .pwm_freq2    (pwm_freq2  ),
113     .pwm_freq3    (pwm_freq3  ),
114     .pwm_freq4    (pwm_freq4  ),
115     .pwm_duty1    (pwm_duty1  ),
116     .pwm_duty2    (pwm_duty2  ),
117     .pwm_duty3    (pwm_duty3  ),
118     .pwm_duty4    (pwm_duty4  )
119 );
120
121 wire            tp_ja1 = spi_ss ;
122 wire            tp_ja2 = spi_sck ;
123 wire            tp_ja3 = spi_mosi ;
124 wire            tp_ja4 = spi_miso ;
125
126
127
128 endmodule

```

- ✓ 라인 69 - 74 : led_counter 모듈을 추가합니다.
- ✓ 라인 76 - 99 : pwm_top 모듈을 추가합니다.
- ✓ 라인 102 - 120 : spi_slave 모듈을 추가합니다.
- ✓ 라인 122 - 125 : spi 신호들을 tp 신호로 출력합니다. 스코프로 신호들을 측저합니다.

6.2.5 xdc 생성

BlazeTop.xdc 파일을 아래아 같이 수정합니다. 아래는 추가된 부분을 보여줍니다.

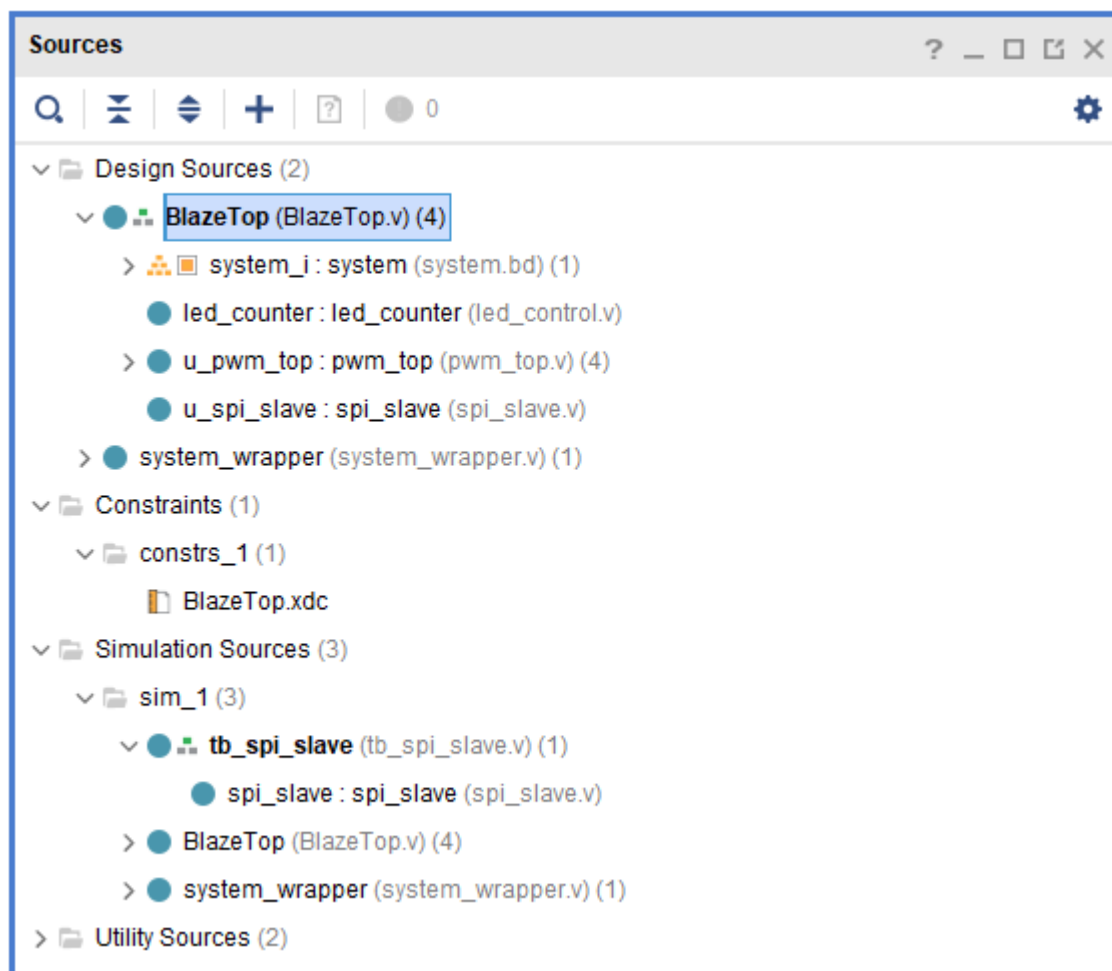
```

23 set_property -dict { PACKAGE_PIN D4   IOSTANDARD LVCMOS33 } [get_ports {pwm_out[0]}}; #JD1
24 set_property -dict { PACKAGE_PIN D3   IOSTANDARD LVCMOS33 } [get_ports {pwm_out[1]}}; #JD2
25 set_property -dict { PACKAGE_PIN F4   IOSTANDARD LVCMOS33 } [get_ports {pwm_out[2]}}; #JD3
26 set_property -dict { PACKAGE_PIN F3   IOSTANDARD LVCMOS33 } [get_ports {pwm_out[3]}}; #JD4
27
28 set_property -dict { PACKAGE_PIN G13  IOSTANDARD LVCMOS33 } [get_ports {tp_ja1}};   #JA1
29 set_property -dict { PACKAGE_PIN B11  IOSTANDARD LVCMOS33 } [get_ports {tp_ja2}};   #JA2
30 set_property -dict { PACKAGE_PIN A11  IOSTANDARD LVCMOS33 } [get_ports {tp_ja3}};   #JA3
31 set_property -dict { PACKAGE_PIN D12  IOSTANDARD LVCMOS33 } [get_ports {tp_ja4}};   #JA4

```

- ✓ 라인 23 - 26 : pwm 출력 4개가 추가되었습니다.
- ✓ 라인 28 - 31 : spi 신호를 확인하기 위한 tp 신호가 추가되었습니다.

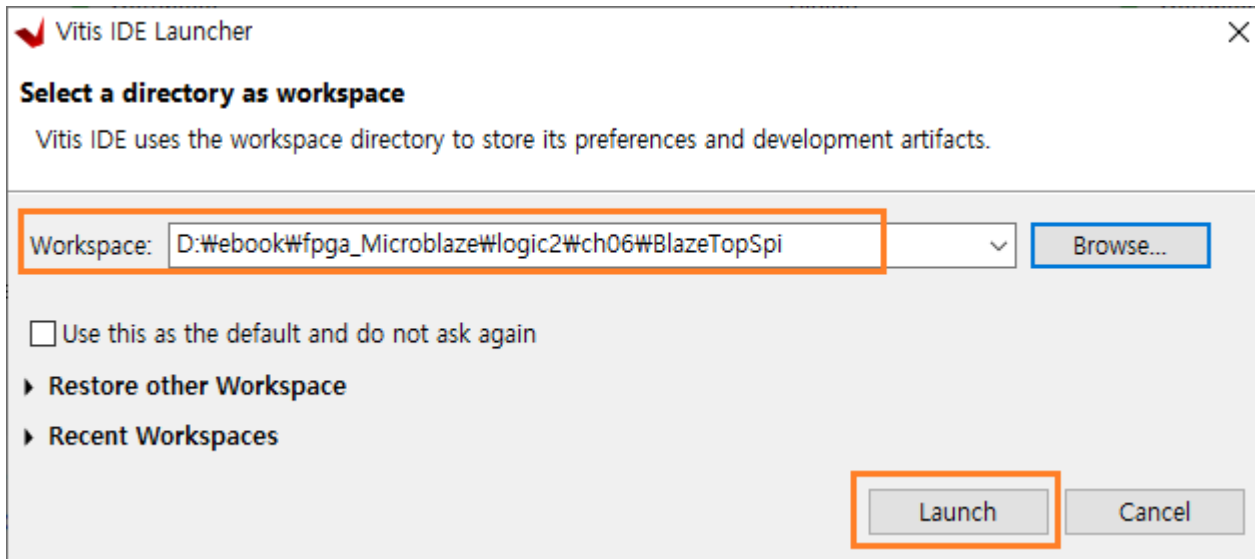
아래는 지금까지 구현된 파일 구조를 보여줍니다.



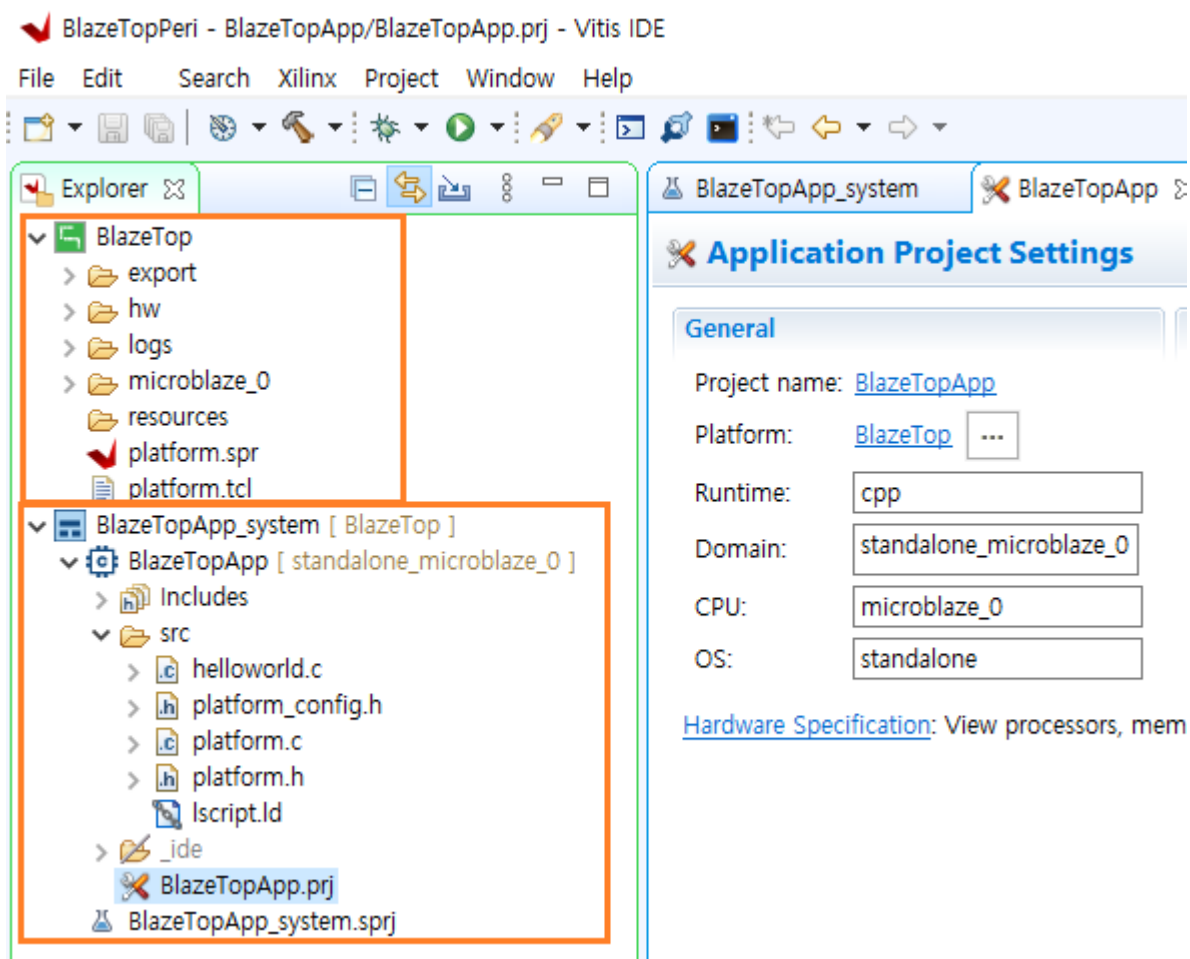
코드가 모두 완료 되었습니다. Bitstream을 생성합니다. Bitstream이 생성되면 vitis에서 사용하기 위하여 XSA 파일을 생성합니다. (자세한 사항은 4.1을 참조하세요)

6.3 Application SW

Tools - Launch Vitis IDE 를 클릭해서 Vitis 를 실행합니다. Browse... 를 클릭하여 프로젝트 위치를 변경 후 Launch 를 클릭합니다.



이후의 과정은 4.1장과 동일합니다. Template 에서 Hello World를 선택하여 프로젝트를 생성합니다. 아래는 생성된 프로젝트의 파일 구조를 보여줍니다.



BlazeTop은 vivado에서 생성한 파일이고, BlazeTopApp는 vitis에서 구현하는 파일입니다. 다음장부터 코드를 구현합니다.

6.3.1 comm_task 복사

제공받은 소스에서 comm_task.c, comm_task.h 파일을 “BlazeTopSpi\blazeTopApp\src\” 폴더에 복사합니다.

6.3.2 helloworld.c

helloworld.c 를 아래와 같이 수정합니다.

```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "xuartlite.h"
52 #include "xspi.h"
53 #include "comm_task.h"
54 #include "sleep.h"
55 #include "xil_exception.h"
56 #include "xintc.h"
57
58 /* definitions for UART */
59 #define UARTLITE_DEVICE_ID      XPAR_UARTLITE_0_DEVICE_ID
60 #define UARTLITE_INT_IRQ_ID     XPAR_MICROBLAZE_0_AXI_INTC_AXI_UARTLITE_0_INTERRUPT_INTR
61
62 /* definitions for Interrupt */
63 #define INTC_DEVICE_ID          XPAR_INTC_0_DEVICE_ID
64
65
66 /* definitions for SPI */
67 #define SPI_DEVICE_ID           XPAR_SPI_0_DEVICE_ID
68
69 #define UART_BUFFER_SIZE      100 /* UART RX Buffer Size */
70
71 void UartSendHandler(void *CallBackRef, unsigned int EventData);
72 void UartRecvHandler(void *CallBackRef, unsigned int EventData);
73
74 XUartLite UartLite;           /* The instance of the UartLite Device */
75 XIntc      Intc;               /* The instance of the Interrupt Controller */
76 XSpi       SpiInstance;       /* The instance of the SPI device */
77
78 static volatile int TotalSentCount; /* dummy */
79
80 int UartRxIndex1, UartRxIndex2;
81 u8 UartRxBuf[UART_BUFFER_SIZE];

```

- ✓ 라인 52 - 56 : xspi.h, comm_task.h, xintc.h 파일을 include 합니다.
- ✓ 라인 58 - 67 : uart, interrupt, spi 의 ID를 선언합니다.
- ✓ 라인 69 : uart 수신 버퍼 사이즈입니다.
- ✓ 라인 71 - 72 : uart rx, tx interrupt handler를 선언합니다.
- ✓ 라인 74 - 76 : uart, interrupt, spi Instance를 선언합니다.
- ✓ 라인 80 - 81 : uart rx 버퍼입니다.

```

83 void interrupt_init(void)
84 {
85     XIntc_Initialize(&Intc, INTC_DEVICE_ID);
86
87     XIntc_Connect(&Intc, UARLITE_INT_IRQ_ID,
88                 (XInterruptHandler)XUartLite_InterruptHandler, (void *)&UartLite);
89
90     XIntc_Start(&Intc, XIN_REAL_MODE);
91     XIntc_Enable(&Intc, UARLITE_INT_IRQ_ID);
92 }
93
94 void exception_init(void)
95 {
96     Xil_ExceptionInit();
97     Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
98                               (Xil_ExceptionHandler)XIntc_InterruptHandler, &Intc);
99     Xil_ExceptionEnable();
100 }
101
102 void uart_init(void)
103 {
104     XUartLite_Initialize(&UartLite, UARLITE_DEVICE_ID);
105     XUartLite_SelfTest(&UartLite);
106
107     XUartLite_SetSendHandler(&UartLite, UartSendHandler, &UartLite);
108     XUartLite_SetRecvHandler(&UartLite, UartRecvHandler, &UartLite);
109
110     XUartLite_EnableInterrupt(&UartLite);
111
112     UartRxIndex1 = 0;
113     UartRxIndex2 = 0;
114 }
115
116 void spi_init(void)
117 {
118     XSpi_Config *ConfigPtr; /* Pointer to Configuration data */
119
120     ConfigPtr = XSpi_LookupConfig(SPI_DEVICE_ID);
121     XSpi_CfgInitialize(&SpiInstance, ConfigPtr, ConfigPtr->BaseAddress);
122     XSpi_SelfTest(&SpiInstance);
123
124     XSpi_SetOptions(&SpiInstance, XSP_MASTER_OPTION );
125     XSpi_Start(&SpiInstance); /* Start SPI */
126     XSpi_IntrGlobalDisable(&SpiInstance); /* Disable interrupt */
127     XSpi_SetSlaveSelect(&SpiInstance, 0x35); /* 이 문이 빠지면 동작하지 않음 */
128 }

```

- ✓ 라인 83 - 92 : interrupt controller를 초기화 합니다.
- ✓ 라인 94 - 100 : exception 모듈을 초기화 합니다.
- ✓ 라인 102 - 114 : uart controller를 초기화 합니다.
- ✓ 라인 116 - 128 : spi controller를 초기화 합니다.

```

130 int main()
131 {
132     //int cnt_1s = 0;
133
134     init_platform();
135
136     uart_init();
137     spi_init();
138     interrupt_init();
139     exception_init();
140
141     xil_printf("\r\n App Start v1.6a.. \r\n");
142     while (1)
143     {
144         // Uart Rx Task
145         if(UartRxIndex1 != UartRxIndex2)
146         {
147             // xil_printf("\r\n rcv : %c", UartRxBuf[UartRxIndex2]);
148             comm_decode(UartRxBuf[UartRxIndex2]);
149             UartRxIndex2++;
150             if(UartRxIndex2 >= UART_BUFFER_SIZE)
151                 UartRxIndex2 = 0;
152         }
153     }
154
155     cleanup_platform();
156     return 0;
157 }
158
159
160 void UartSendHandler(void *CallBackRef, unsigned int EventData)
161 {
162     TotalSentCount = EventData;
163 }
164
165 void UartRecvHandler(void *CallBackRef, unsigned int EventData)
166 {
167     u8 rxData;
168     XUartLite_Recv(&UartLite, &rxData, 1);
169     UartRxBuf[UartRxIndex1] = rxData;
170
171     UartRxIndex1++;
172     if(UartRxIndex1 >= UART_BUFFER_SIZE)
173         UartRxIndex1 = 0;
174 }

```

- ✓ 라인 130 - 157 : main 함수입니다. 각 모듈을 초기화 하고, uart 수신 메시지를 처리합니다. comm_decode 함수는 uart 수신 메시지를 처리합니다. 통신 프로토콜을 다음장을 참조하세요.
- ✓ 라인 160 - 163 : uart tx interrupt handler 입니다.
- ✓ 라인 165 - 174 : uart rx interrupt handler 입니다.

6.3.3 통신 프로토콜

PC의 Application 프로그램 (WinIDT) 와 FPGA 는 UART 로 명령어를 주고 받습니다. 아래 표는 통신 프로토콜을 나타냅니다. 데이터는 Hexa 값을 String 으로 변환 후 총 4바이트를 전송합니다. read 시에는 데이터 영역에는 데이터를 보내지 않습니다.

stx (1)	command (4)	data (4)	etx (1)	description
@	"pfw1"	"0000" ~ "ffff"	*	pwm-1 frequency write
@	"pfw2"	"0000" ~ "ffff"	*	pwm-2 frequency write
@	"pfw3"	"0000" ~ "ffff"	*	pwm-3 frequency write
@	"pfw4"	"0000" ~ "ffff"	*	pwm-4 frequency write
@	"pdw1"	"0000" ~ "0064"	*	pwm-1 duty write
@	"pdw2"	"0000" ~ "0064"	*	pwm-2 duty write
@	"pdw3"	"0000" ~ "0064"	*	pwm-3 duty write
@	"pdw4"	"0000" ~ "0064"	*	pwm-4 duty write
@	"pfr1"	-	*	pwm-1 frequency read
@	"pfr2"	-	*	pwm-2 frequency read
@	"pfr3"	-	*	pwm-3 frequency read
@	"pfr4"	-	*	pwm-4 frequency read
@	"pdr1"	-	*	pwm-1 duty read
@	"pdr2"	-	*	pwm-2 duty read
@	"pdr3"	-	*	pwm-3 duty read
@	"pdr4"	-	*	pwm-4 duty read

6.3.4 comm_task

comm_task 는 2개의 함수가 있습니다.

- ✓ comm_decode() : UART 로 수신한 데이터에서 stx, etx 를 제외한 command, data 을 comm_task() 함수로 전달
- ✓ comm_task() : 명령어에 따라 SPI Master 로 데이터 전달

아래는 spi write 를 구현한 코드입니다. 전송할 데이터를 4바이트 설정하고 Xspi_Transfer() 함수로 데이터를 전달하였습니다. 세번째 인자는 read 할 버퍼의 주소를 할당하는데, Write 시에는 NULL 로 설정하면 됩니다.

```

61 // pwm1 frequency write
62 if( (comm_buf[0]=='p') && (comm_buf[1]=='f') && (comm_buf[2]=='w') && (comm_buf[3]=='1') )
63 {
64     data16 = atoi(comm_buf+4);
65     wbuf[0] = 0x64;
66     wbuf[1] = 0x10;
67     wbuf[2] = (uint8_t)(data16>>8);
68     wbuf[3] = (uint8_t)(data16&0xff);
69     XSpi_Transfer(&SpiInstance, wbuf, NULL, 4);
70     xil_printf("pwm1 freq. write : %d \r\n", data16);
71 }

```

아래는 spi read 를 구현한 코드입니다. 전송할 데이터를 4바이트 설정하고, read 할 버퍼의 주소를 전달하면 read 된 값을 버퍼를 통해 전달 받을 수 있습니다.

```

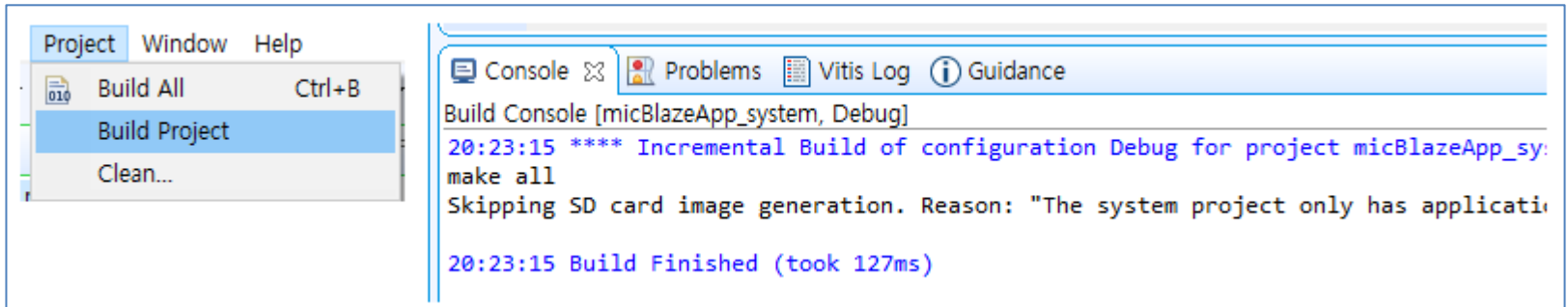
151 // pwm1 frequency read
152 else if( (comm_buf[0]=='p') && (comm_buf[1]=='f') && (comm_buf[2]=='r') && (comm_buf[3]=='1') )
153 {
154     wbuf[0] = 0x65;
155     wbuf[1] = 0x10;
156     wbuf[2] = 0;
157     wbuf[3] = 0;
158     XSpi_Transfer(&SpiInstance, wbuf, rbuf, 4);
159     xil_printf("pwm1 freq. read : %d \r\n", (uint16_t)(rbuf[2]<<8|rbuf[3]));
160 }

```

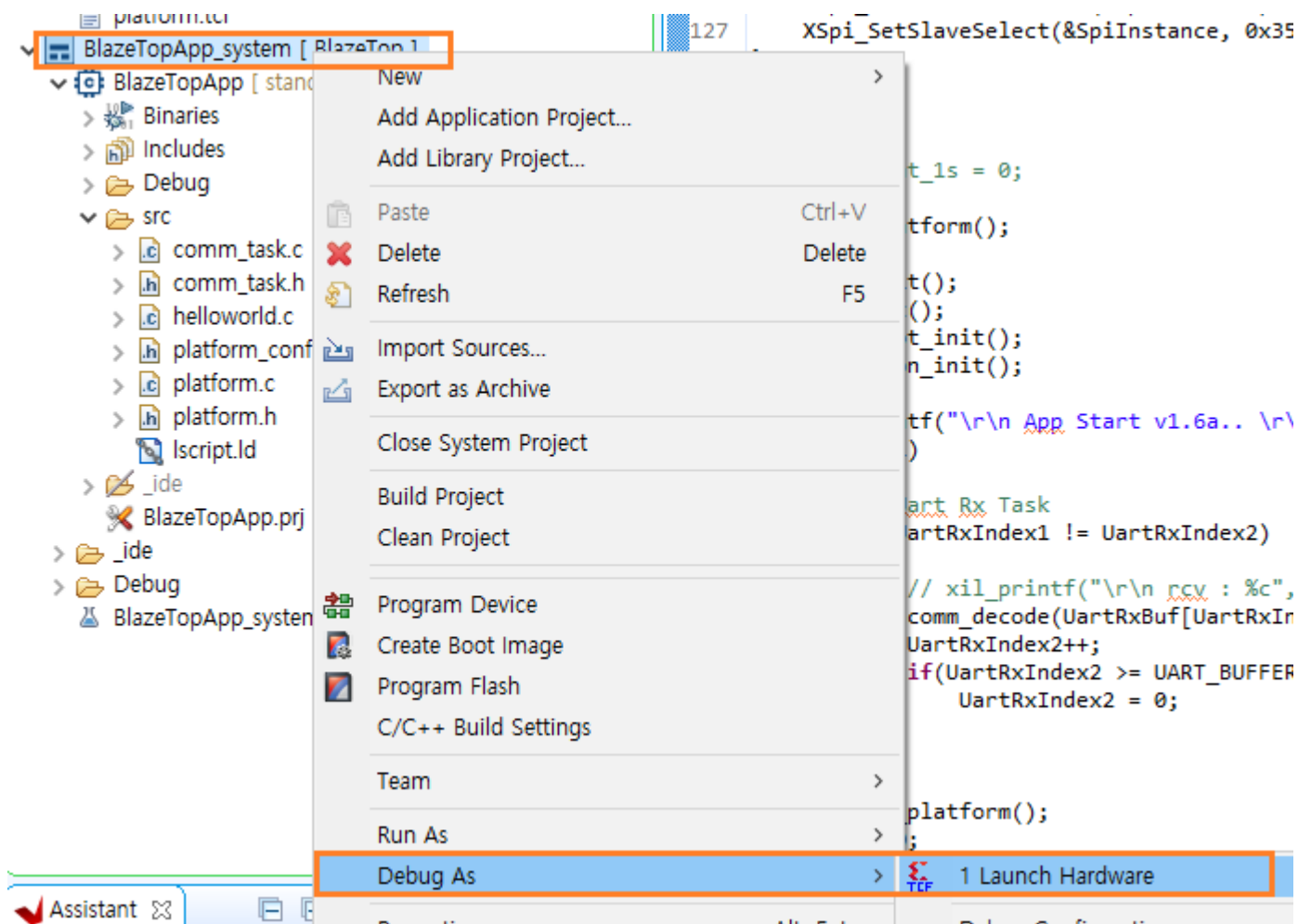
Application SW 구현이 완료되었습니다. 다음장에서는 Application SW를 Build 하고 다운로드 해서 결과를 확인하도록 하겠습니다.

6.4 다운로드 및 결과 확인

Project를 Build 합니다. 에러가 없는 것을 확인합니다.

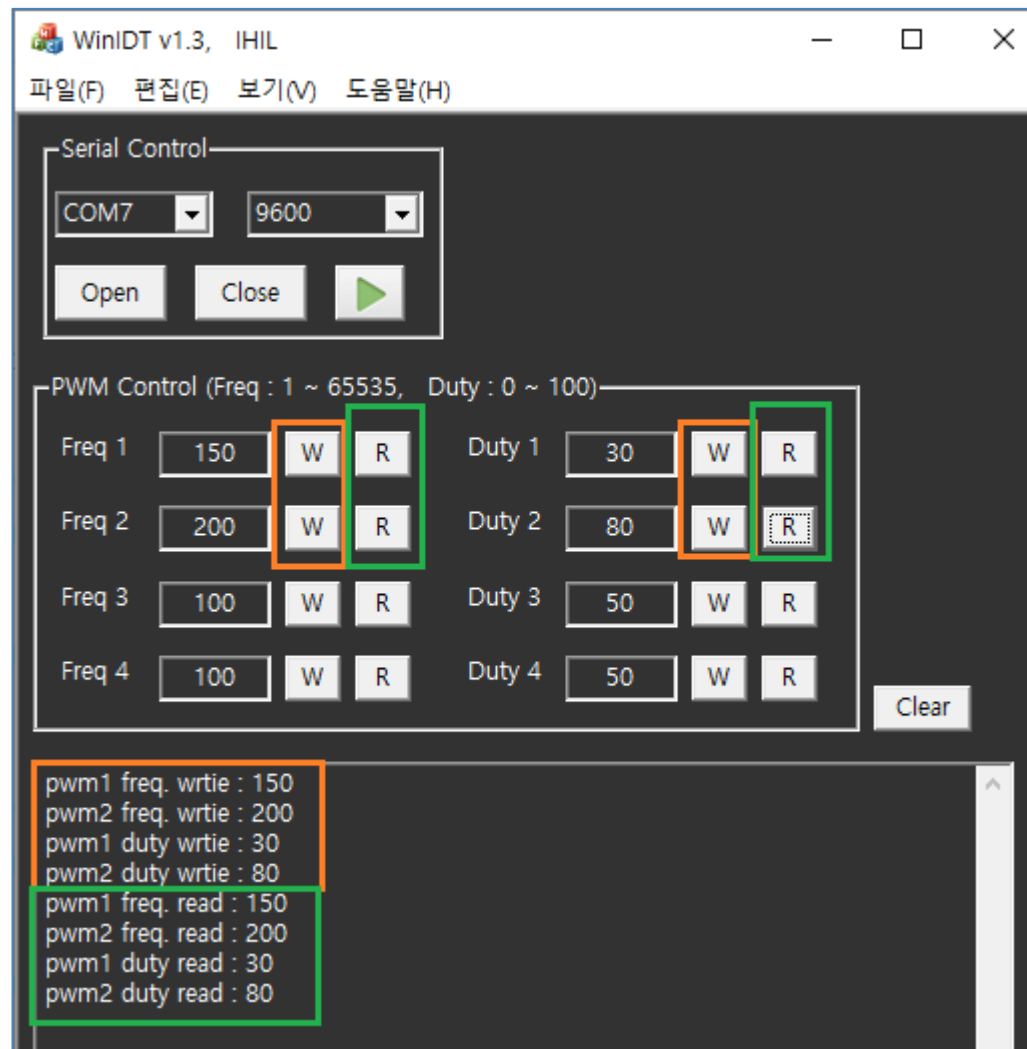


디버그 모드로 다운로드를 합니다. 보드에 전원을 인가하고 (커넥터를 연결), BlazeTpApp_system - 우클릭 - Debug As - 1 Launch Hardware 를 클릭합니다.



다운로드가 완료되면 main() 함수의 처음 위치에 커서가 위치합니다. Resume 아이콘을 클릭하면 프로그램이 동작합니다.

아래 그림은 WinIDT 프로그램 실행 결과입니다.

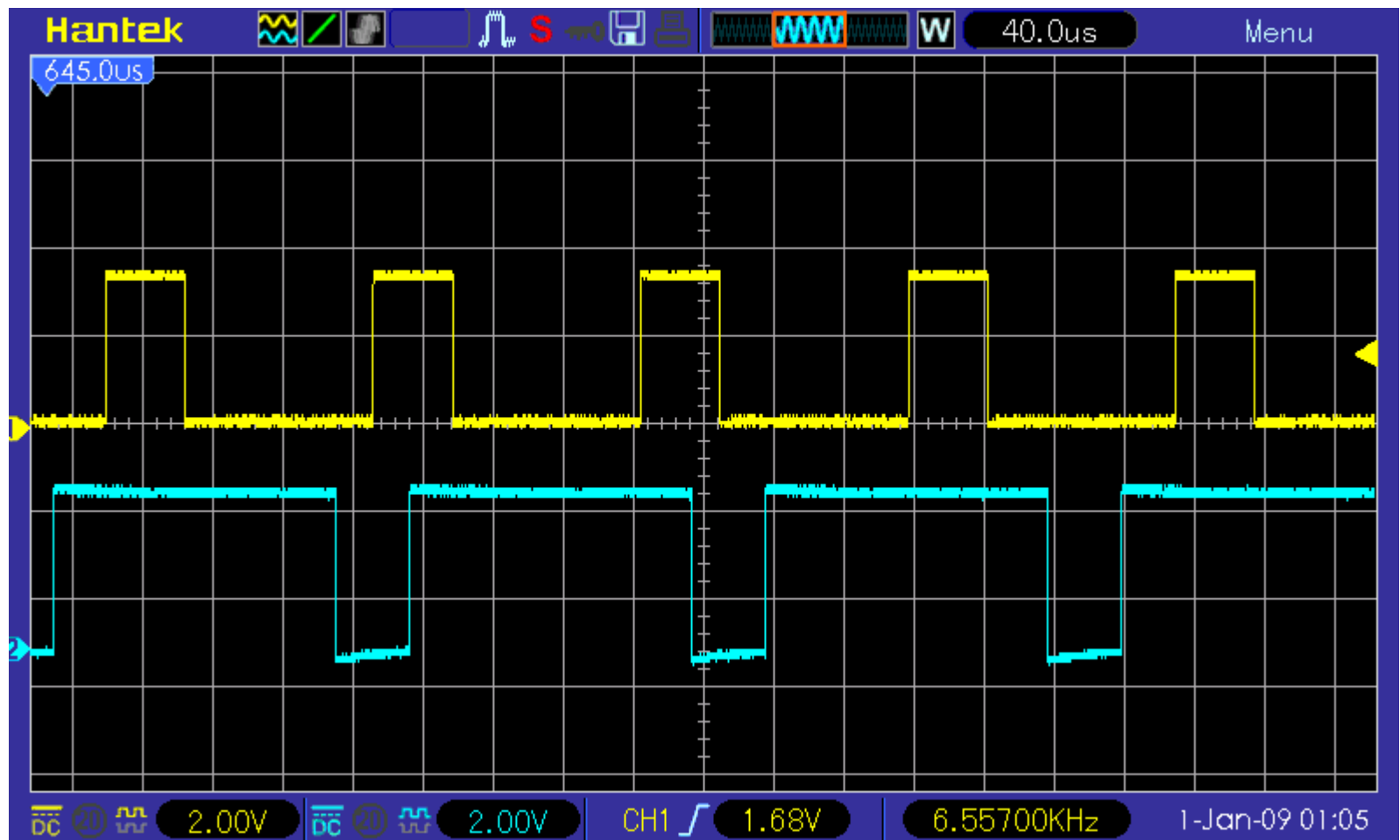


- ✓ Freq1 : 150 을 Write 합니다.
- ✓ Freq2 : 200 을 Write 합니다.
- ✓ Duty1 : 30 을 Write 합니다.
- ✓ Duty2 : 80 을 Write 합니다.
- ✓ Freq1 을 Read 합니다. -> write 한 150 값이 read 되었습니다.
- ✓ Freq2 을 Read 합니다. -> write 한 200 값이 read 되었습니다.
- ✓ Duty1 을 Read 합니다. -> write 한 30 값이 read 되었습니다.
- ✓ Duty2 을 Read 합니다. -> write 한 80 값이 read 되었습니다.

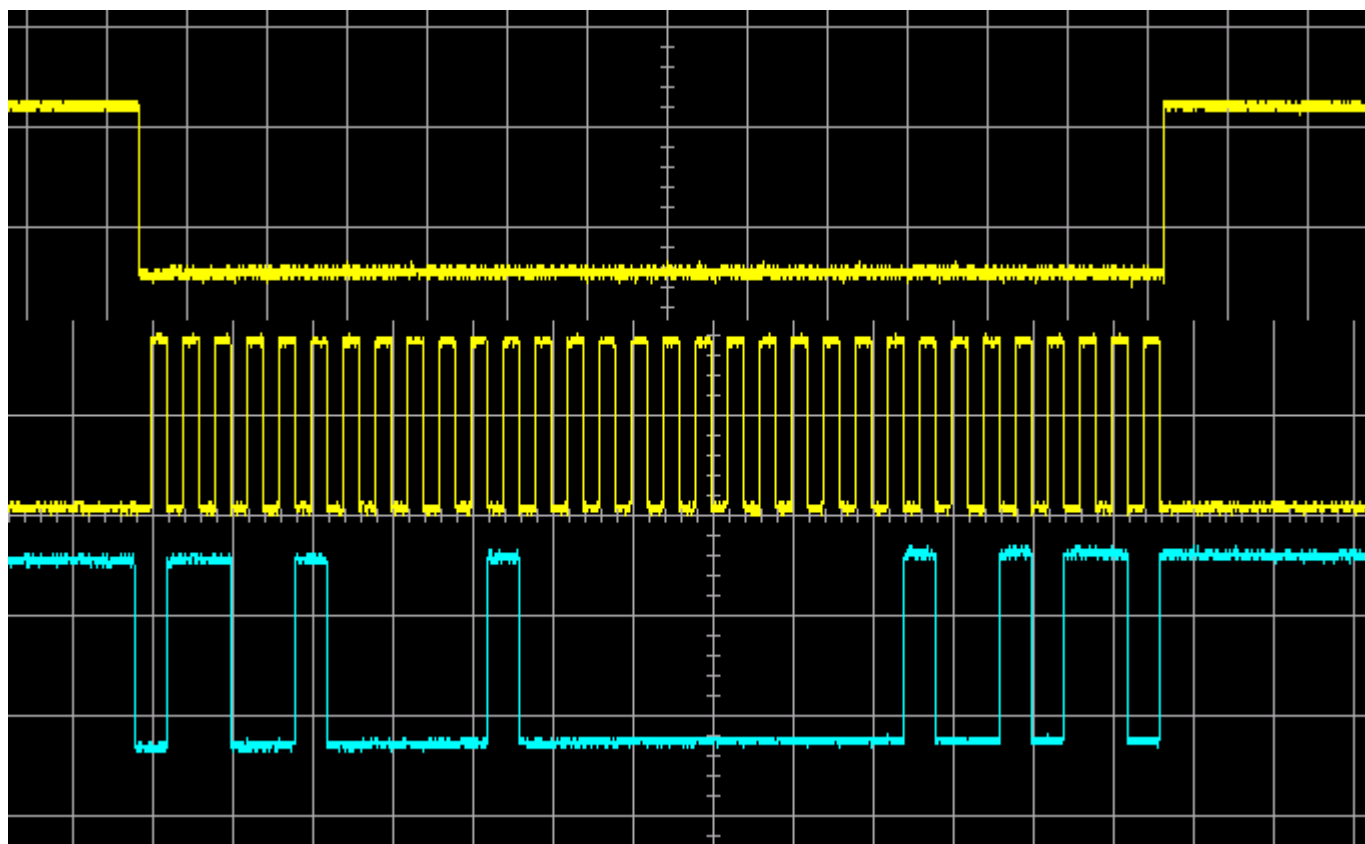
아래 그림은 Freq1, Freq2 의 파형을 스코프로 측정한 그림입니다. 보드의 JD 커넥터에서 1번핀(pwm1), 2번핀(pwm2)을 측정하였습니다. 노란색 파형은 pwm1, 초록색 파형은 pwm2 를 나타냅니다. 주파수와 Duty 가 원하는 대로 설정되었습니다.

pwm1 주파수 : $(100\text{MHz}) / ((150+1) * 101) = 6.557 \text{ khz}$

pwm2 주파수 : $(100\text{MHz}) / ((200+1) * 101) = 4.926 \text{ khz}$



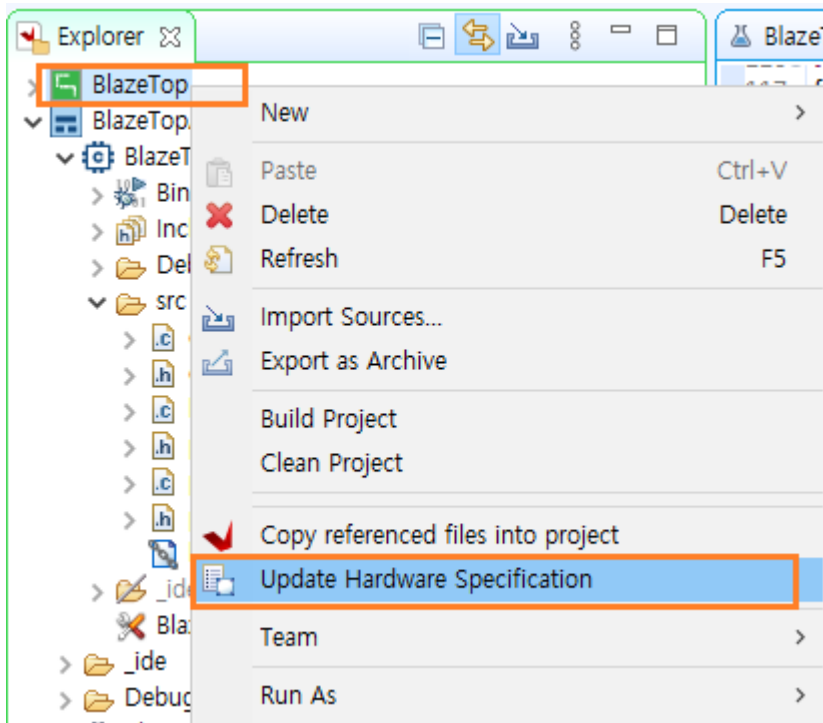
아래 그림은 spi write 구간에 ss, sck, somi 신호를 보여줍니다.



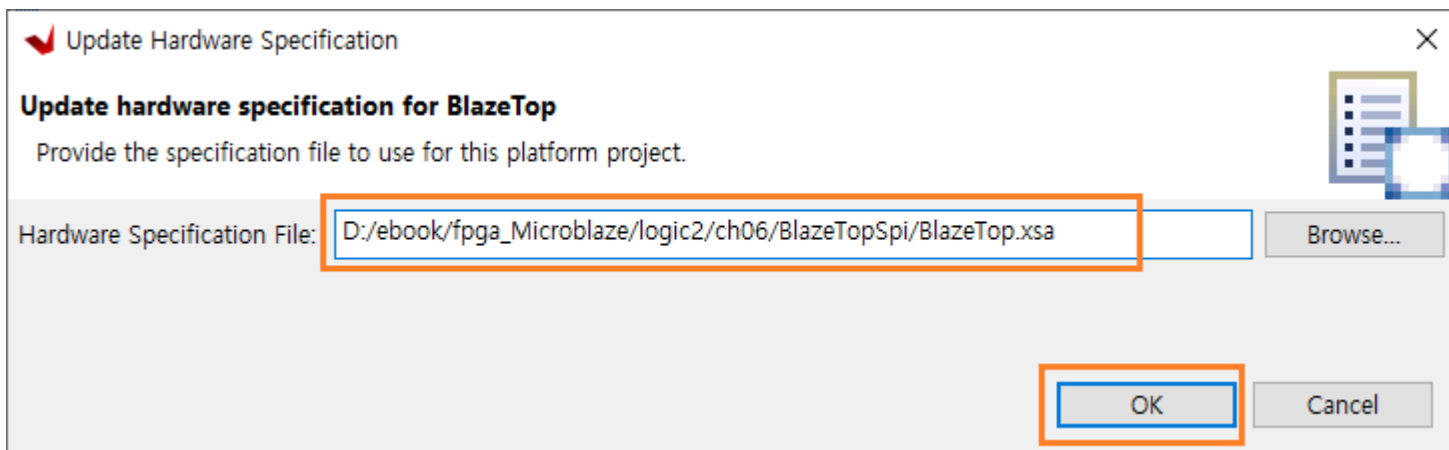
6.5 HW update

프로젝트를 진행하다 보면, vivado에서 hw block, 코드를 변경하는 경우가 있습니다. Block Design이 변경되었을 때에는 Create HDL Wrapper를 다시 생성합니다. 그리고 Bitstream을 재생성하고, Export Hardware를 통해 XSA 파일을 생성합니다.

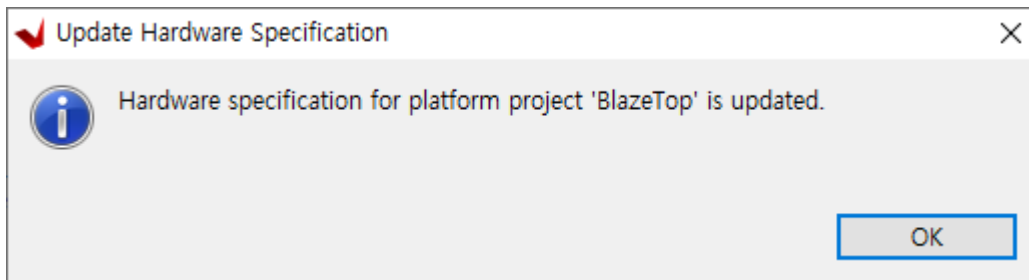
vitis에 이를 적용하는 방법은 HW Project에 업데이트된 내용을 적용합니다. 이를 위해서는 BlazeTop - 우클릭 - Update Hardware Specification 을 클릭합니다.



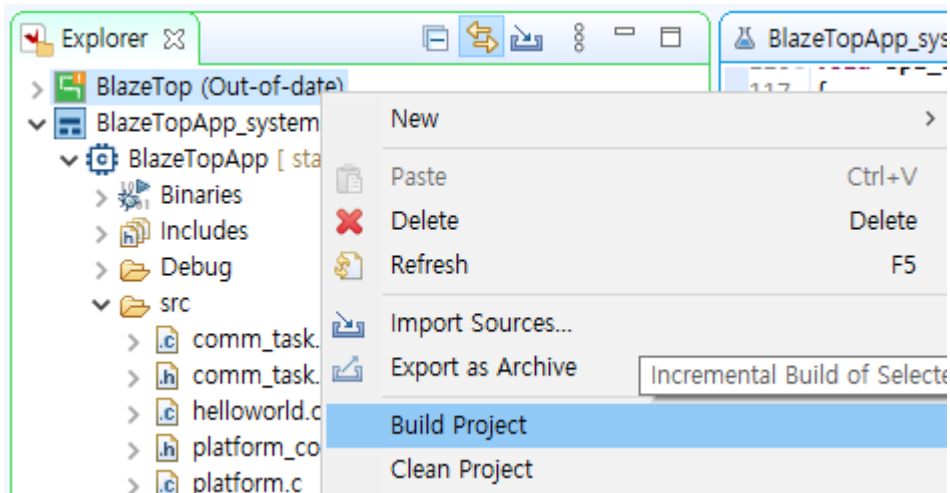
변경된 XSA 파일을 설정하고 OK를 클릭합니다.



OK를 클릭합니다.

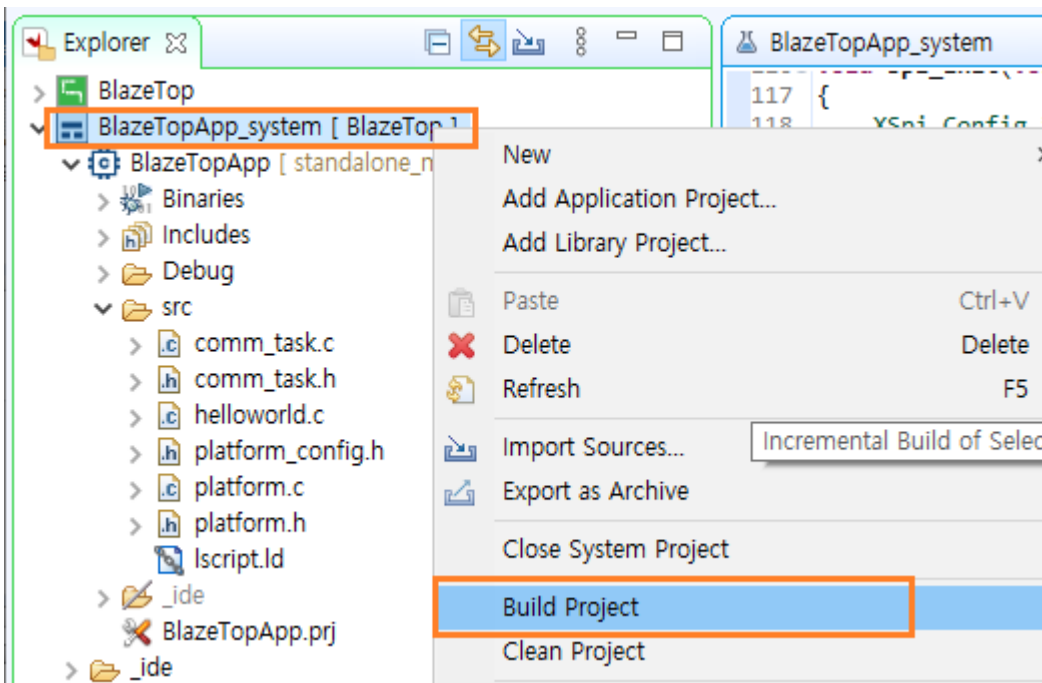


HW Project (BlazeTop) 옆에 (Out-of-date)라고 표시됩니다. 다시 Build를 해야 합니다. BlazeTop - 우클릭 - Build Project 를 클릭합니다.



Build가 완료되면, (Out-of-date)가 사라집니다.

이제 Application Project를 Build 합니다. (Clean Project 후에 Build Project를 해 주면 더 좋습니다)

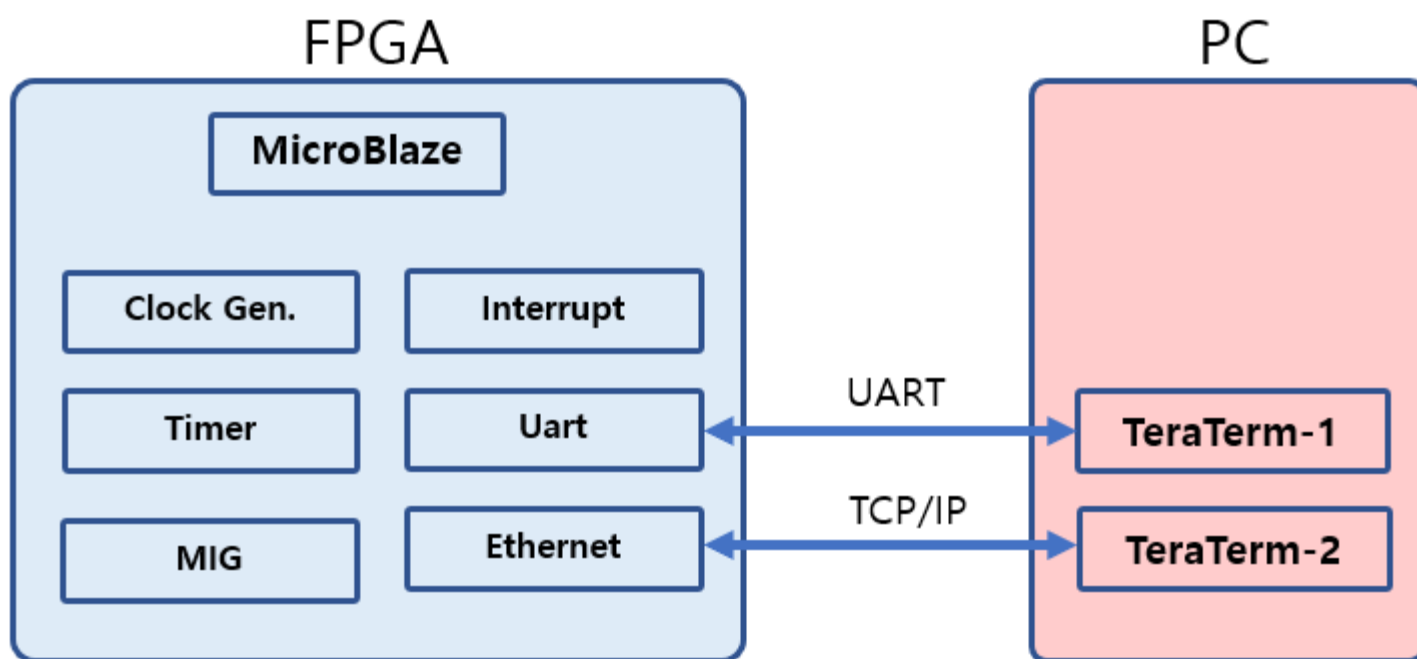


7. LightWeight IP (lwIP) Echo Server 구현

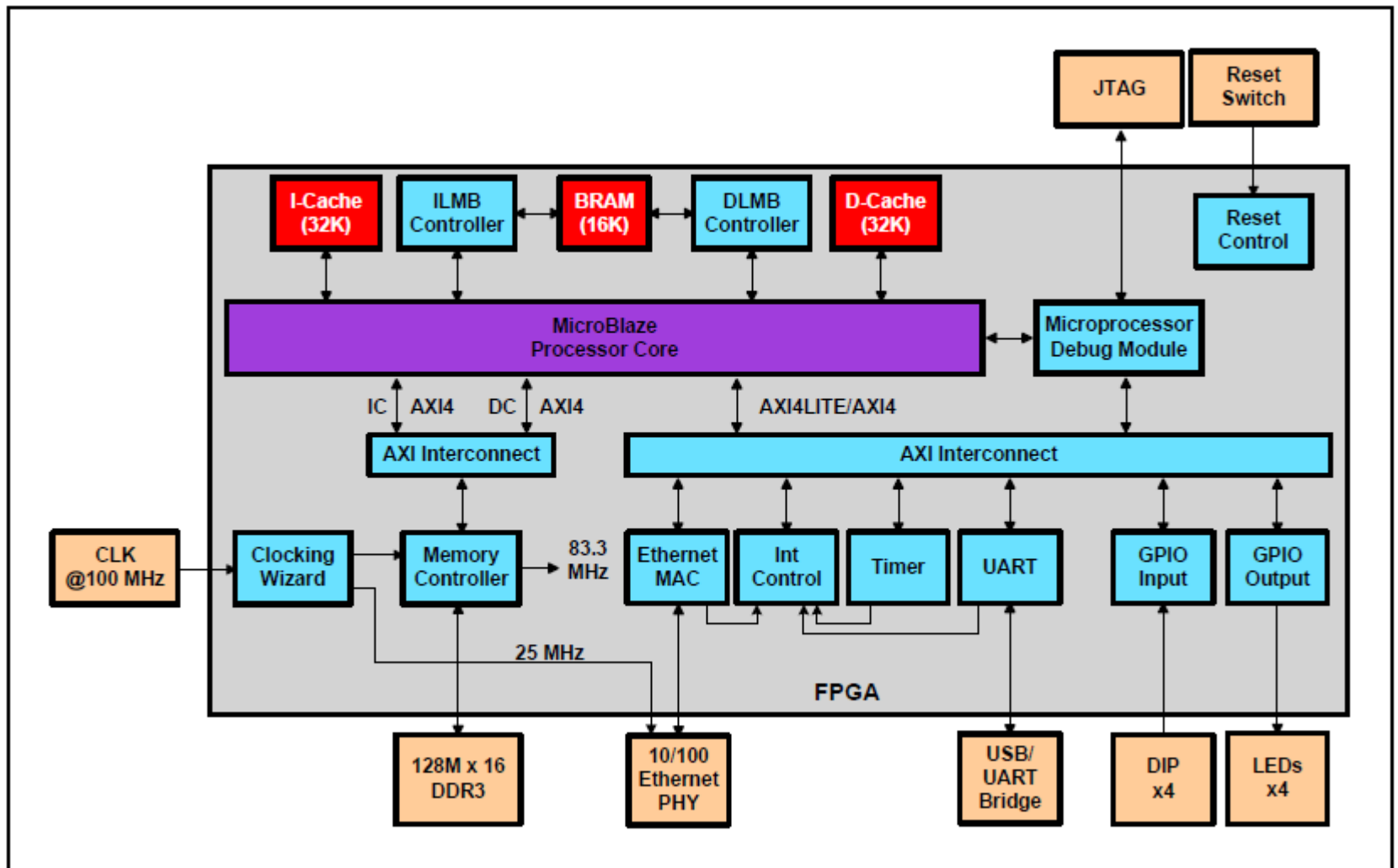
LwIP (LightWeight IP)는 임베디드 시스템에서 사용되는 오픈 소스 TCP/IP 스택입니다. 스웨덴 컴퓨터 과학 연구소의 Adam Dunkels에 의해 처음 개발되었으며 현재는 전 세계 개발자 네트워크에 의해 개발 및 유지 관리되고 있다고 합니다. 이번 장에서는 “AXI EthernetLite” IP를 이용하여 lwIP Echo Server를 구현합니다. Xilinx는 MicroBlaze에서 사용할 수 있는 lwIP SDK(Software Development Kit)를 제공합니다. 이번 장에서는 ddr3 memory를 사용하여 Microblaze의 Cache (Instruction Cache, Data Cache)를 구현하는 방법과, Xilinx SDK에서 제공하는 Template 코드를 사용하여 Echo Server를 구현하도록 하겠습니다.

7.1 시스템 구성

전체적인 시스템 구성은 아래와 같습니다. FPGA 보드와 PC는 Tera Term을 이용하여 연결합니다. TeraTerm1은 시리얼 통신으로 연결하고, TeraTerm2는 TCP/IP로 연결합니다. TeraTerm1은 디버깅 메시지를 표시하는 용도로 사용되고, TeraTerm2는 Echo Server 송수신용으로 사용됩니다. TeraTerm2에서 임의의 데이터를 전송하면 FPGA보드는 데이터를 수신 후 동일한 데이터를 전송합니다.



아래 그림은 본장에서 구현하는 HW Design Block을 자세히 보여줍니다. (출처 : LwIP Applications For the Arty Evaluation Board)

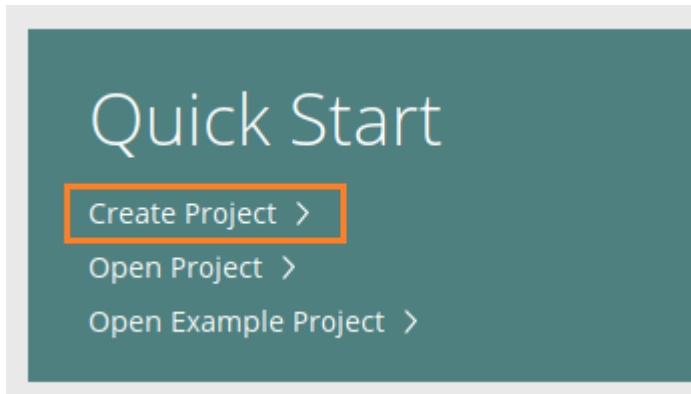


그림에 나와 있는 대로 다음장에서 IP를 하나씩 구현하도록 하겠습니다.

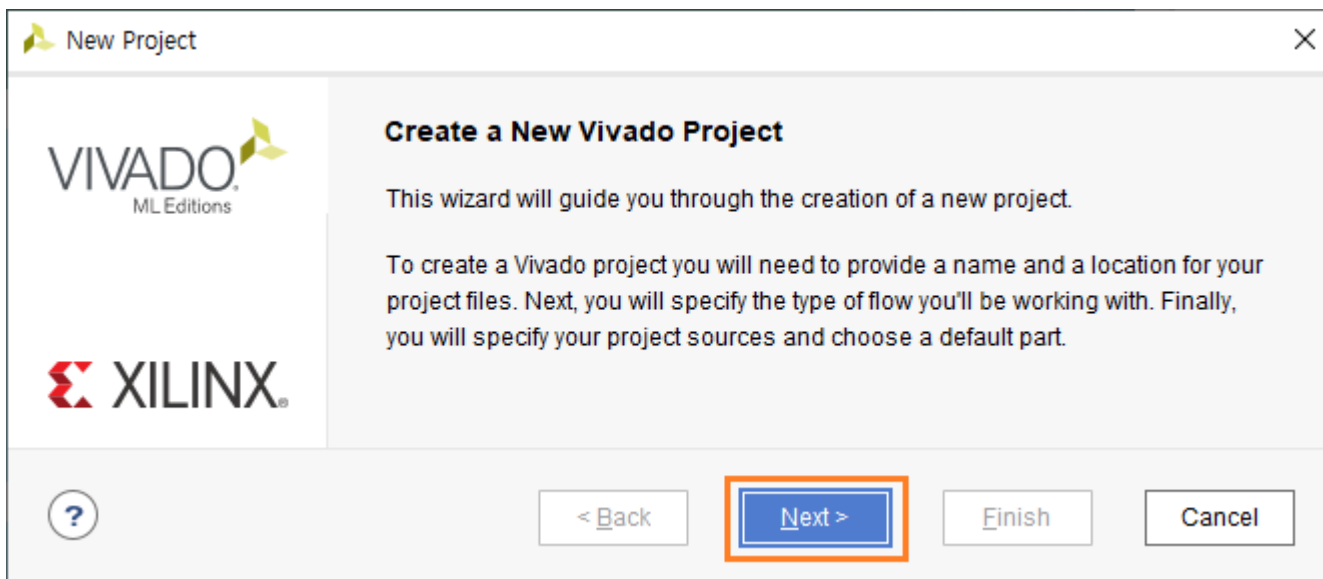
7.2 lwIP HW 구현

7.2.1 프로젝트 생성

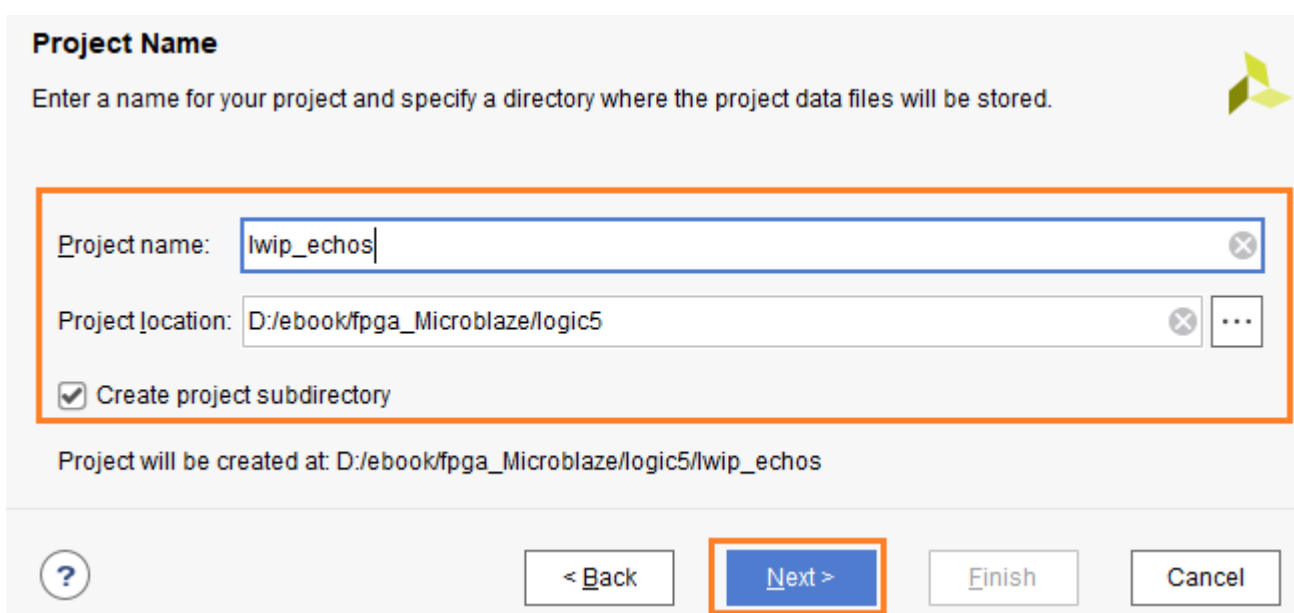
vivado2022.1 을 실행합니다. Create Project를 클릭합니다.



Next를 클릭합니다.



프로젝트 위치를 설정하고 프로젝트 이름을 입력하고 Next를 클릭합니다. (Create project subdirectory를 check 합니다)



Project Type : RTL Project를 선택하고 Next를 클릭합니다.

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☐ Do not specify sources at this time

☐ Project is an extensible Yitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.

☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

< Back **Next >** Finish Cancel

Add Source, Add Constrains 에서는 나중에 필요할 때 파일을 추가하기 때문에 Next를 클릭합니다.

Device Part를 선택합니다. Search 창에 “xc7a35tcsg” 입력 후, Part 리스트 중에 xc7a35tcsg324-1을 선택하고 Next를 클릭합니다.

Search: (8 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs
xc7a35tcsg324-2L	324	210	20800	41600	50	0
xc7a35tcsg324-1	324	210	20800	41600	50	0
xc7a35tcsg325-3	325	150	20800	41600	50	0
xc7a35tcsg325-2	325	150	20800	41600	50	0

< Back **Next >** Finish

Finish 버튼을 클릭하면 프로젝트가 생성됩니다.

7.2.2 HW Block 생성

PROJECT MANAGER에서 IP INTEGRATOR - Create Block Design을 클릭합니다.

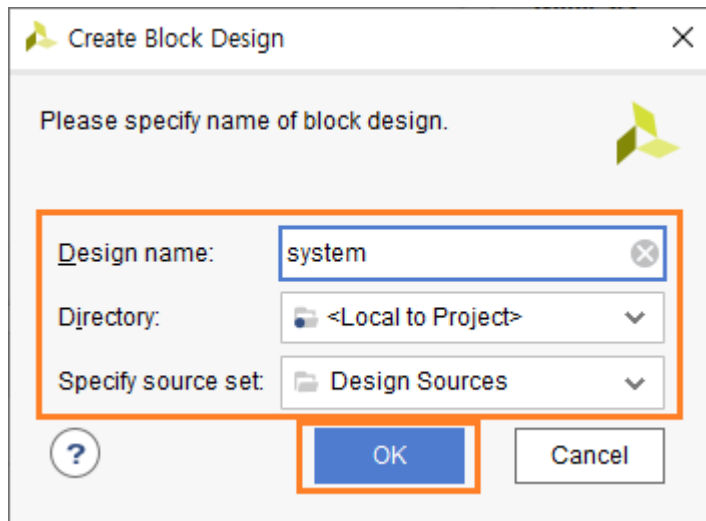
▼ IP INTEGRATOR

Create Block Design

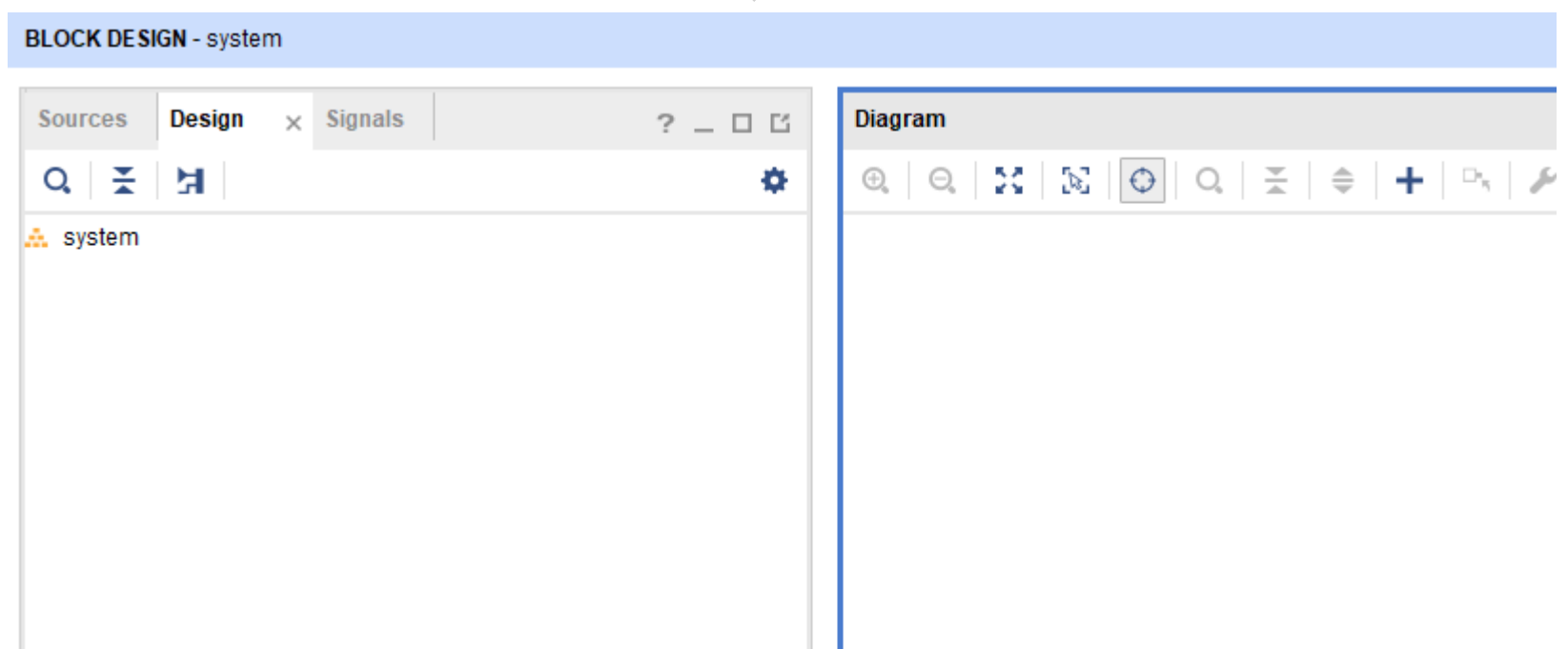
Open Block Design

Generate Block Design

Design name : system 으로 입력하고 OK를 클릭합니다. Directory, Specify source set 값은 Default 값을 사용합니다.



Block Design, Diagram 윈도우가 생성되었습니다.



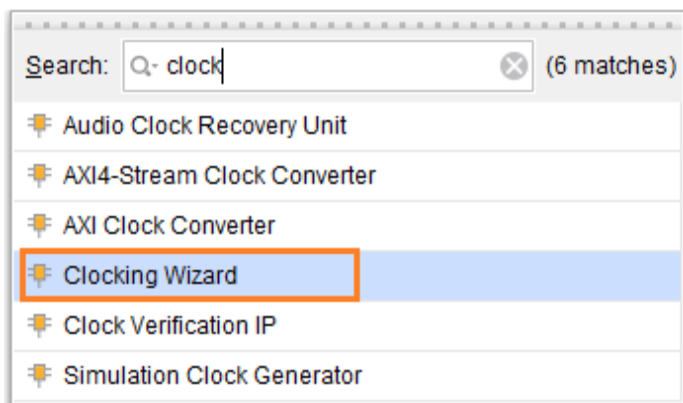
이제 각 IP들을 생성/추가해 주고, 속성을 지정하고, 각 Block 들을 신호들을 알맞게 연결해 주어야 합니다. 여기에서는 순서가 중요합니다. 순서를 임의대로 하면 각 Block들을 연결할 때 많은 문제들이 발생할 수 있습니다.

아래는 IP를 생성하는 순서입니다. 이 순서에 맞게 IP를 생성하고, 속성을 지정하고, 각 신호들을 연결해 주어야 합니다.

- 1) Clock Generator
- 2) Memory Interface Generator
- 3) Microblaze
- 4) Uart
- 5) Ethernet
- 6) Timer

1) - 3)까지는 순서가 중요합니다. 4) - 6)은 1) - 3)까지를 먼저 생성하고 적당히 추가하면 됩니다.

Add IP (+ 아이콘)을 클릭합니다. “clock”을 입력하고 리스트 중에 “Clocking Wizard”를 더블 클릭해서 IP를 추가합니다.



IP (clk_wiz_0)가 추가되었습니다.



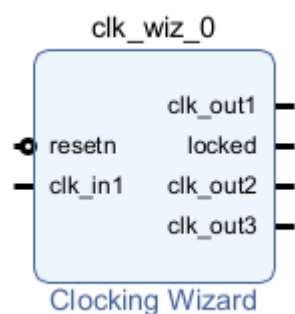
clk_wiz_0를 더블 클릭해서 속성을 아래와 같이 설정합니다. clk_out1 : 166.66667, clk_out2 : 200, clk_out3 : 25 를 입력하고, Reset Type : Active Low로 설정합니다.

Clocking Options					
Output Clocks					
The phase is calculated relative to the active input clock.					
Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)	
		Requested	Actual	Requested	Actual
☒ clk_out1	clk_out1	166.66667	166.66667	0.000	0.000
☒ clk_out2	clk_out2	200	200.00000	0.000	0.000
☒ clk_out3	clk_out3	25	25.00000	0.000	0.000
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A

Reset Type

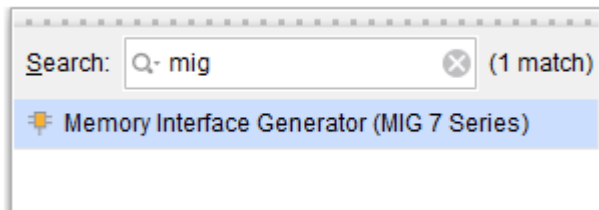
☐ Active High ☒ Active Low

clk_out1 (166.67 Mhz)은 ddr controller의 system clock으로 사용되고, clk_out2 (200Mhz)는 ddr controller의 reference clock으로 사용되고, clk_out3(25 Mhz)은 Ethernet Transceiver의 main clock으로 사용됩니다. OK 버튼을 클릭하면 아래와 같이 IP가 변경됩니다.

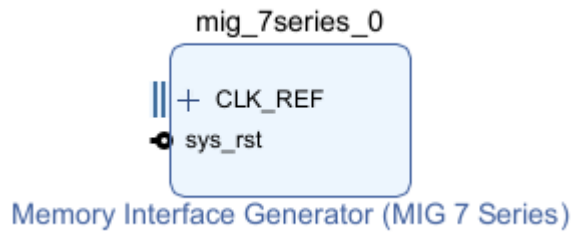


DDR3를 사용하는 이유는 Xilinx에서 제공하는 lwip Echo Server Templates 코드에서 Microblaze Processor가 I-Cache, D-Cache를 사용하도록 설정되었기 때문입니다. (105페이지의 HW Design Block을 참고하세요)

ddr3 memory controller를 추가합니다. Add IP (+ 아이콘)을 클릭합니다. “mig”을 입력하고 리스트 중에 “Memory Interface Generator (MIG7 Series)”를 더블 클릭해서 IP를 추가합니다

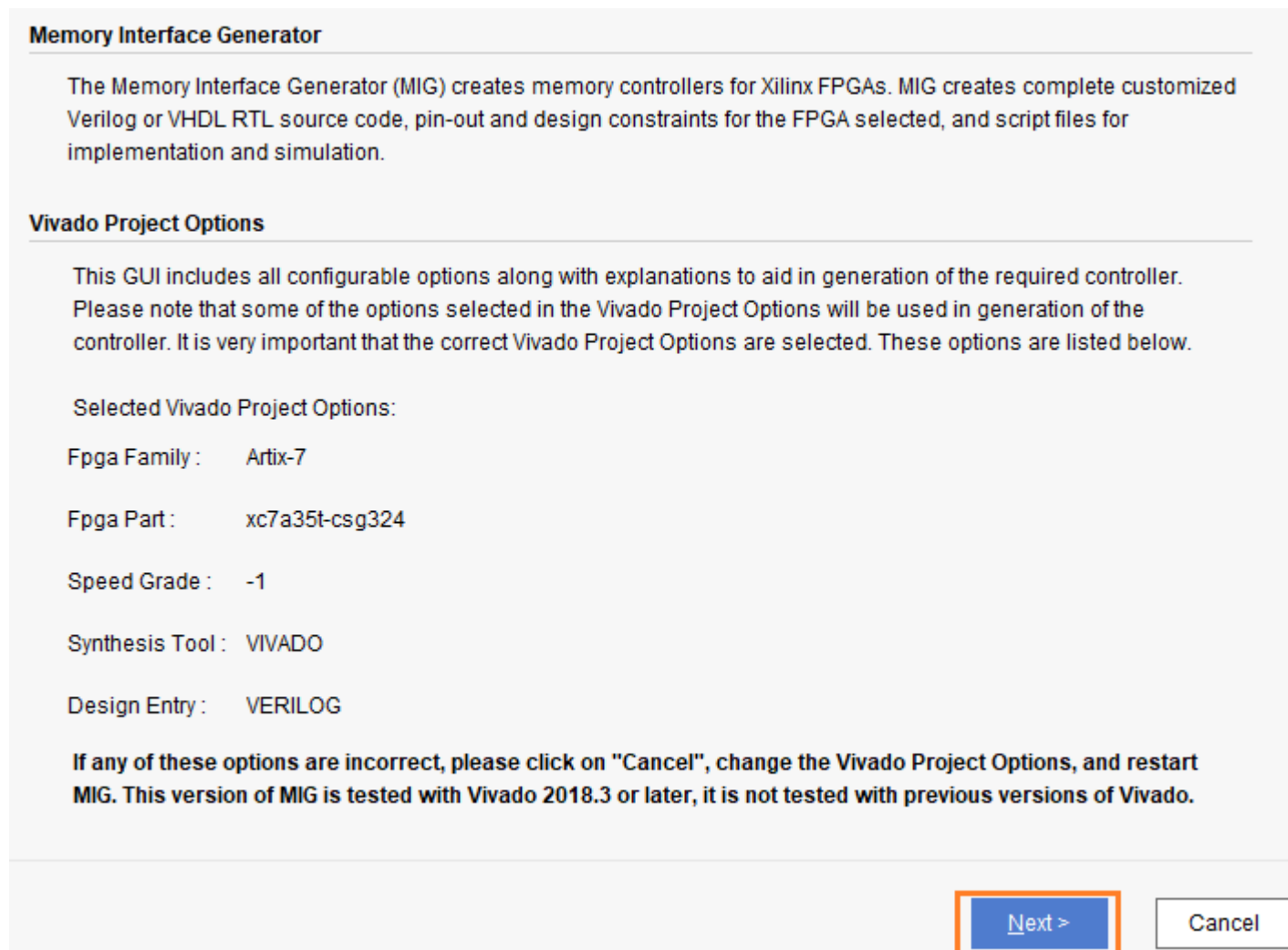


IP (mig_7series_0)가 추가되었습니다.



mig_7series_0을 더블 클릭해서 속성을 변경합니다. (Memory Interface Generator에 관한 자세한 내용은 전자문서 “Verilog를 이용한 FPGA 활용2 - DDR Controller”에 자세히 나와 있으니 참고하시길 바랍니다. 본서에서는 대략적인 내용을 설명합니다)

Next를 클릭합니다.



Default로 설정되어 있는 내용을 확인하고 Next를 클릭합니다.

MIG Output Options

☒ **Create Design**
 Select this option to generate a memory controller. Generating a memory controller will create RTL, XDC, implementation and simulation files.

☐ **Verify Pin Changes and Update Design**
 Selecting this feature verifies the modified XDC for a design already generated through MIG. This option will allow you to change the pin out and validate it instantly. It updates the input XDC file to be compatible with the current version of MIG. While updating the XDC it preserves the pin outs of the input XDC. This option will also generate the new design with the Component Name you selected in this page.

Component Name

 Please specify the component name for the memory interface. The design directories will be generated under a directory with this name. Three directories will be created "example_design", "user_design" and "docs". The user_design will contain the generated memory interface. The example_design adds a simple example application connected to the generated memory interface.

 Component Name

Multi-Controller

 Up to maximum of 8 controllers with a combination of DDR3 SDRAM, QDRII+ SRAM or RLDRAM II can be generated. The number of controllers that can be accommodated may be limited by the data width and the number of banks available in device. Refer user guide for more information

 Number of controllers

AXI4 Interface

 Enables the AXI4 interface. AXI4 interface is supported only for DDR3 SDRAM and DDR2 SDRAM controllers with Verilog design entry.

☒ AXI4 Interface

Pin Compatible FPGAs 윈도우에서도 변경없이 Next를 클릭합니다.

DDR3 SDRAM을 선택하고 Next를 클릭합니다.

Memory Selection

 Select the type of memory interface. Please refer to the User Guide for a detailed list of supported controllers for each FPGA family. The list below shows currently available interface(s) for the specific FPGA, speed grade and design entry chosen.

 Select the controller type:

☒ **DDR3 SDRAM**

☐ DDR2 SDRAM

DDR3 Memory의 HW 관련 내용을 설정합니다. 회로도를 참조해서 설정합니다. 자세한 설명은 전자문서 “Verilog를 이용한 FPGA 활용2 - DDR Controller” 을 참고하시길 바랍니다.

Options for Controller 0 - DDR3 SDRAM

Clock Period: Choose the clock period for the desired frequency. The allowed period range(2500 - 3300) is a function of the selected FPGA part and FPGA speed grade. Refer to the User Guide for more information. 3,000 ps 333.33 MHz

PHY to Controller Clock Ratio: Select the PHY to Memory Controller clock ratio. The PHY operates at the Memory Clock Period chosen above. The controller operates at either 1/4 or 1/2 of the PHY rate. The selected Memory Clock Period will limit the choices. 4:1

Memory Type: Select the memory type. Type(s) marked with a warning symbol are not compatible with the frequency selection above. Components

Memory Part: Select the memory part. Part(s) marked with a warning symbol are not compatible with the frequency selection above. Find an equivalent part or create a part using the "Create Custom Part" button if the part needed is not listed here. The "Create Custom Part" feature is not supported for RLDRAM II. MT41K128M16XX-15E Create Custom Part

Memory Voltage: Select the Voltage of the Memory part selected. 1.35V

Data Width: Select the Data Width. Parts marked with a warning symbol are not compatible with the frequency and memory part selected above. 16

ECC: MIG supports ECC for 72 bit data width configuration. To be able to select ECC, select a data width that has ECC supported. Disabled

Data Mask: Enable or disable the generation of Data Mask (DM) pins using this check box. This option can be selectable only if the memory part selected has DM pins. Uncheck this box to not use data masks and save FPGA I/Os that are used for DM signals. ECC designs (DDR3 SDRAM, DDR2 SDRAM) will not use Data Mask. ☒

Number of Bank Machines: This parameter defines the number of bank machines. A given bank machine manages a single DRAM bank at any given time. 4

ORDERING: Normal mode allows the memory controller to reorder commands to the memory to obtain the highest possible efficiency. Strict mode forces the controller to execute commands in the exact order received. Normal

Memory Details: 2Gb, x16, row:14, col:10, bank:3, data bits per strobe:8, with data mask, single rank, 1.35V,1.5V

< Back Next > Cancel

AXI 버스 관련 내용입니다. 아래와 같이 설정(기본값 사용)하고 Next를 클릭합니다.

Axi Parameter Options C0 - DDR3 SDRAM

Data Width

AXI DATA WIDTH: Data width of AXI read & write channels. The data width is less than or equal to user interface data width with the possible values 32, 64, 128, 256 & 512.

128

Arbitration Scheme

Select the arbitration scheme between the read and write address channels

RD_PRI_REG

Narrow Burst Support

Enables logic to support narrow bursts on the AXI4 slave interface. Can be set to zero if no masters in the system issue narrow bursts and all the data widths are equal. (1-Enable, 0-Disable)

0

Address Width

AXI4 address width of read and write address channels.

28

ID Width

AXI4 ID width for read and write channels. AXI4 ID is used as the identification tag for write or read address group of signals

4

< Back **Next >** Cancel

아래와 같이 설정하고 Next를 클릭합니다.

Memory Options C0 - DDR3 SDRAM

Input Clock Period: Select the period for the PLL input clock (CLKIN). MIG determines the allowable input clock periods based on the Memory Clock Period entered above and the clocking guidelines listed in the User Guide. The generated design will use the selected Input Clock and Memory Clock Periods to generate the required PLL parameters. If the required input clock period is not available, the Memory Clock Period must be modified.

6000 ps (166.667 MHz) ▼

☐ **Select Additional Clocks (if required)**

MIG can generate up to 5 additional clocks to be used in Fabric logic. This will be generated from the same MMCM which is used for generation of UI_CLK. The first clock(Clock 0) has a wider range of choices. All the values in the additional clocks drop downs are calculated considering the MMCM VCO frequency as 1501.50146 ps (666 MHz) Mhz. For complete details on clocking of MIG, refer to MIG User Guide.

Clock	Selection	D
Clock 0	NONE ▼	D = 1
Clock 1	NONE ▼	D = 1
Clock 2	NONE ▼	D = 1
Clock 3	NONE ▼	D = 1
Clock 4	NONE ▼	D = 1

Choose the Memory Options for the memory device. Memory Option selections are restricted to those supported by the controller. Consult the memory vendor data sheet for more information.

Read Burst Type and Length

The burst type determines the data ordering within a burst. Consult the memory datasheet for more information. Burst length 8 is the only supported value.

Sequential ▼

Output Driver Impedance Control

Programmable impedance for the output buffer.

RZQ/6 ▼

RTT (nominal) - On Die Termination (ODT)

Select the nominal value of ODT for the DQ, DQS/DQS# and DM signals on the component or DIMM interface. This must be set to RZQ/6 (40 ohms) for data rates at 1333 Mbps and above. In 2 slot DIMM configurations this value will be used for the unwritten slot during a write and will also be used for the unselected slot during a read. Use board level simulation to choose the optimum value.

RZQ/6 ▼

Controller Chip Select Pin

The Chip Select (CS#) pin can be tied low externally to save one pin in the address/command group when this selection is set to 'Disable'. Disable is only valid for single rank configurations.

Enable ▼

Memory Address Mapping Selection

User Address

☐ ROW | BANK | COLUMN

☒ BANK | ROW | COLUMN

아래와 같이 설정하고 Next를 클릭합니다.

System Clock

Choose the desired input clock configuration. Design clock can be Differential or Single-Ended.

System Clock No Buffer

Reference Clock

Choose the desired reference clock configuration. Reference clock can be Differential or Single-Ended.

Reference Clock No Buffer

System Reset Polarity

Choose the desired System Reset Polarity.

System Reset Polarity ACTIVE LOW

Internal Vref

Internal Vref can be used to allow the use of the Vref pins as normal IO pins. This option can only be used at 800 Mbps and lower data rates. This can free 2 pins per bank where inputs are used. This setting has no effect on banks with only outputs.

Internal Vref ☒

IO Power Reduction

Significantly reduces average IO power by automatically disabling DQ/DQS IBUFs and internal terminations during WRITES and periods of inactivity

IO Power Reduction ON

XADC Instantiation

The memory interface uses the temperature reading from the XADC block to perform temperature compensation and keep the read DQS centered in the data window. There is one XADC block per device. If the XADC is not currently used anywhere in the design, enable this option to have the block instantiated. If the XADC is already used, disable this MIG option. The user is then required to provide the temperature value to the top level 12-bit device_temp_i input port. Refer to Answer Record 51687 or the UG586 for detailed information.

XADC Instantiation Enabled

아래와 같이 설정하고 Next를 클릭합니다.

Internal Termination for High Range Banks

Select the internal termination (IN_TERM) impedance for the High Range (HR) banks. This setting applies **only** to the HR banks used in the interface.

Internal Termination Impedance 50 Ohms

Fixed Pin Out: 을 선택하고 Next를 클릭합니다.

Pin/Bank Selection Mode

☐ New Design: Pick the optimum banks for a new design

☒ Fixed Pin Out: Pre-existing pin out is known and fixed

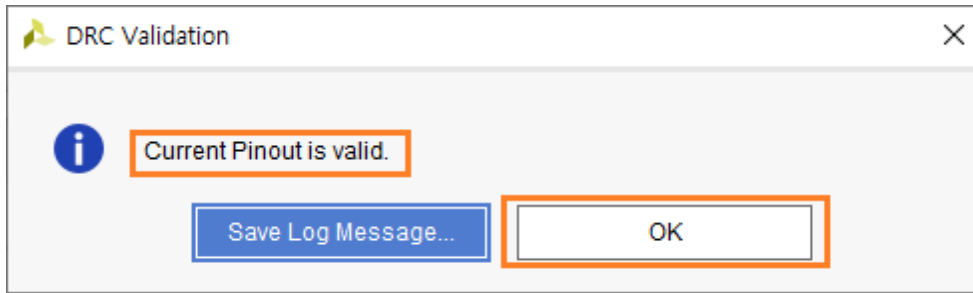
핀을 할당합니다. 회로도를 참조해서 각 핀들을 설정할 수도 있고, 미리 설정된 파일을 불러올 수도 있습니다. “Read/XDC/UCF” 버튼을 클릭해서 “ddr3_pin.ucf” 파일을 불러옵니다. (ddr3_pin.ucf 파일은 다운로드 자료에 포함되어 있습니다) 각 핀들이 할당되었습니다. Validate 를 클릭하여 유효성을 확인합니다. 에러가 발생하지 않아야 Next 버튼이 활성화 됩니다.

Pin Selection For Controller 0 - DDR3 SDRAM

	Signal Name	Bank Number	Byte Number	Pin Number	IO Standard
1	ddr3_dq[0]	34	T0	K5	SSTL135
2	ddr3_dq[1]	34	T0	L3	SSTL135
3	ddr3_dq[2]	34	T0	K3	SSTL135
4	ddr3_dq[3]	34	T0	L6	SSTL135
5	ddr3_dq[4]	34	T0	M3	SSTL135
6	ddr3_dq[5]	34	T0	M1	SSTL135
7	ddr3_dq[6]	34	T0	L4	SSTL135
8	ddr3_dq[7]	34	T0	M2	SSTL135
9	ddr3_dq[8]	34	T1	V4	SSTL135
10	ddr3_dq[9]	34	T1	T5	SSTL135
11	ddr3_dq[10]	34	T1	U4	SSTL135
12	ddr3_dq[11]	34	T1	V5	SSTL135
13	ddr3_dq[12]	34	T1	V1	SSTL135
14	ddr3_dq[13]	34	T1	T3	SSTL135
15	ddr3_dq[14]	34	T1	U3	SSTL135
16	ddr3_dq[15]	34	T1	R3	SSTL135
17	ddr3_dm[0]	34	T0	L1	SSTL135

INFO: Press **Validate** to proceed.

OK 버튼을 클릭하면 Next 버튼이 활성화 됩니다. Next를 클릭합니다.



아래와 같이 설정(기본값 사용)하고 Next를 클릭합니다.

System Signals Selection

Select the system pins below appropriately for the interface. Customization of these pins can also be made in the XDC after the design is generated. For more information see [UG586 Bank and Pin rules](#).

System Clock and Reference Clock pin selections will not be visible if the 'No Buffer' option was selected in the FPGA Options page.

System Signals

These signals may be connected internally to other logic or brought out to a pin.

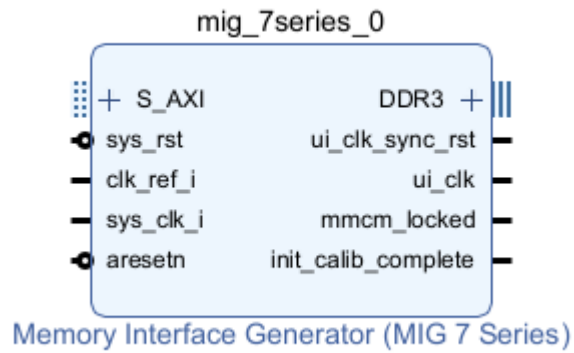
- **sys_rst**: This input signal is used to reset the interface.
- **init_calib_complete**: This signal indicates that the interface has completed calibration and memory initialization and is ready for commands. LOC constraint will be generated in XDC for Example design only based on "Pin Number" selection below.
- **error**: This output signal indicates that the traffic generator in the Example Design has detected a data mismatch. This signal does not exist in the User Design.

Signal Name	Bank Number	Pin Number
sys_rst	Select Bank ▼	No connect ▼
init_calib_complete	Select Bank ▼	No connect ▼
tg_compare_error	Select Bank ▼	No connect ▼

All pins must be constrained to specific locations in order to generate a bit file in the implementation phase (this is not required for simulation).

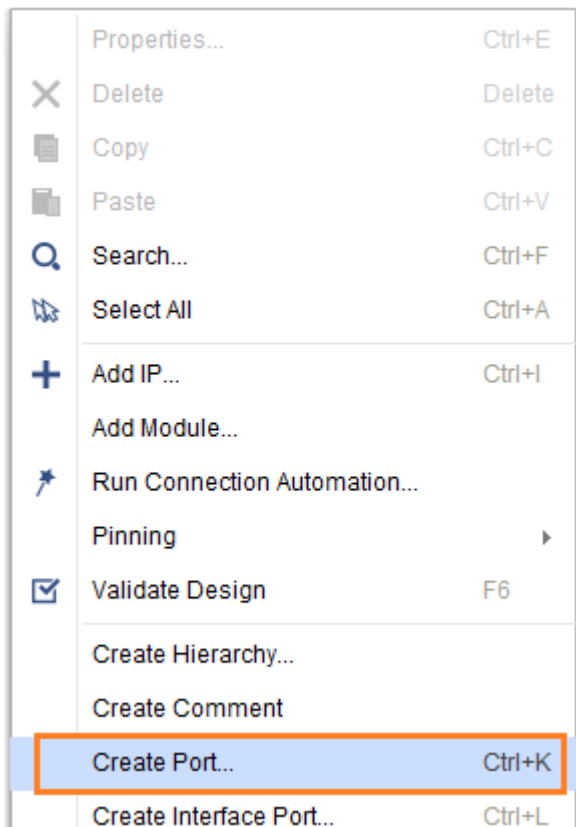
< Back Next > Cancel

이후 Summay, License 등 내용을 확인하고 몇번의 Next 버튼과 Generate을 클릭하면 Memory Controller가 생성됩니다.
생성된 Memory Controller 입니다.

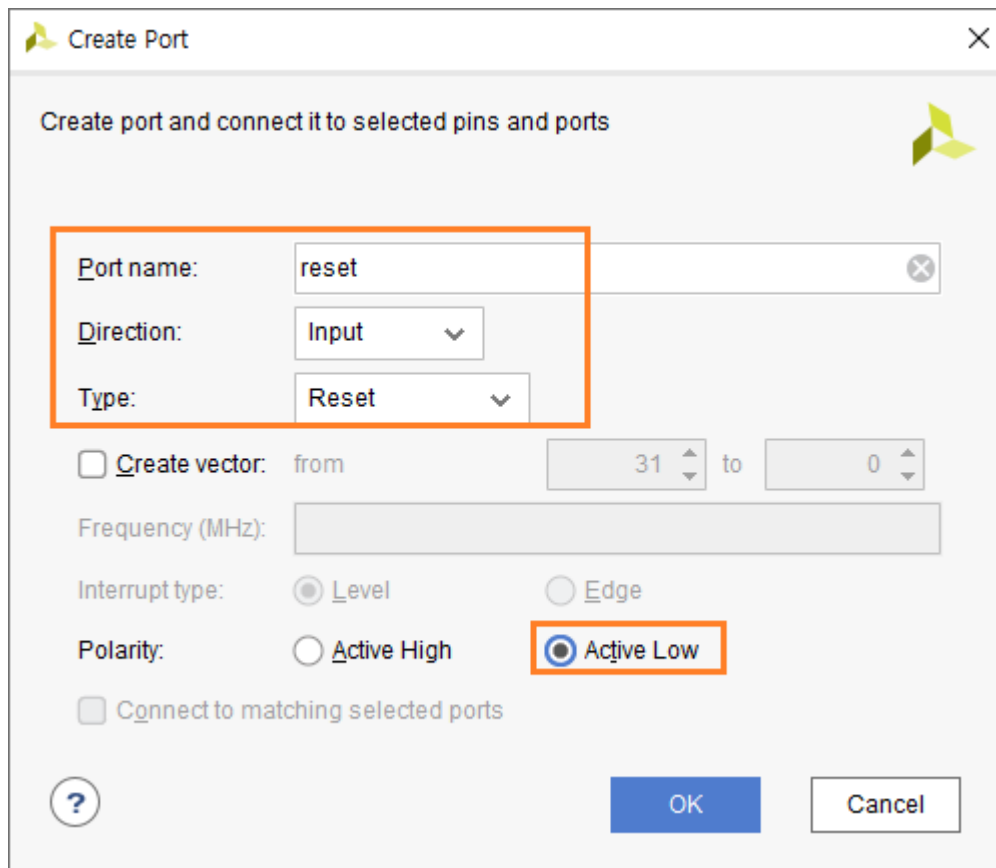


다음 IP를 추가하기 전에 Port를 생성하고, 신호들을 연결하도록 하겠습니다.

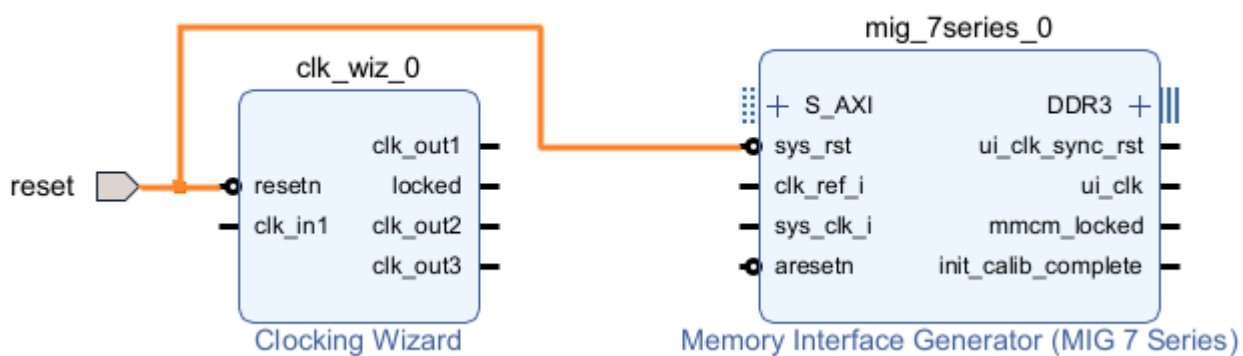
reset port 생성 : Diagram의 빈 곳을 클릭후, 마우스 우클릭을 하면 메뉴가 나타납니다. Create Port ... 를 클릭합니다.



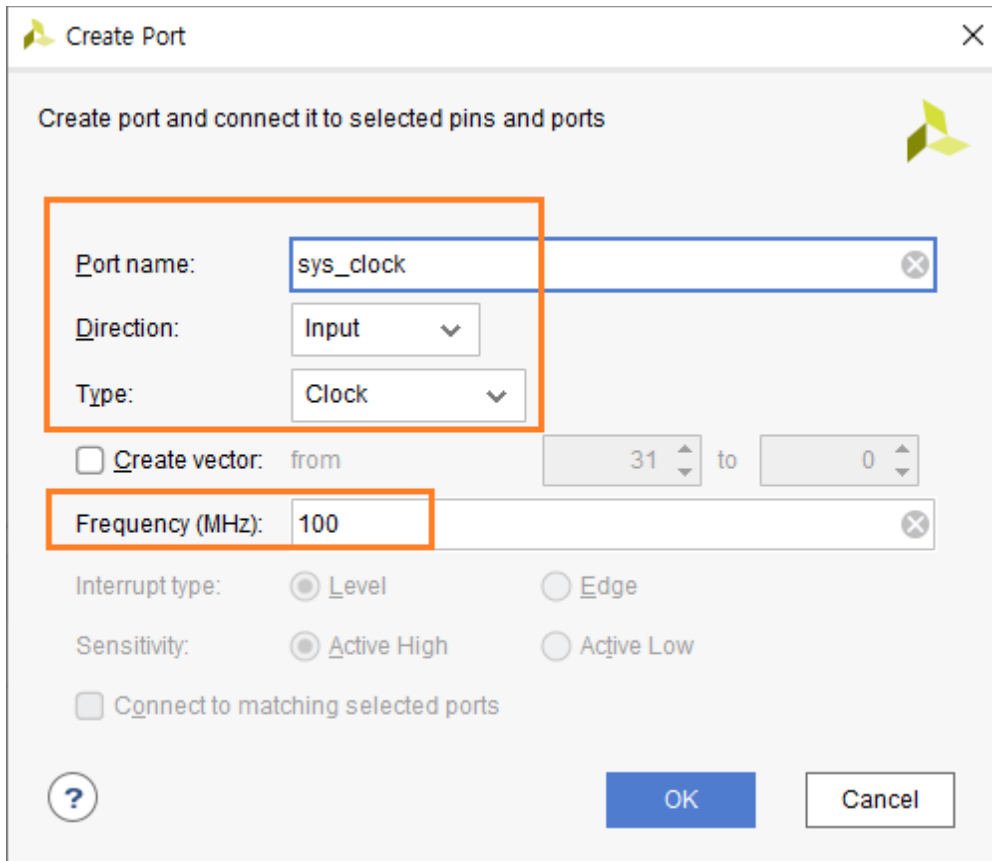
Port Name : reset, Direction : input, Type : Reset, Polarity : Active Low 를 입력/선택하고 OK 를 클릭합니다.



생성된 reset port를 clk_wiz_0의 resetn 신호와 mig_7series_0의 sys_rst 신호와 연결합니다. 연결하는 방법은 해당 핀으로 마우스를 이동하면 연필 모양의 아이콘이 나타나고 이때 클릭해서 반대편 신호까지 드래그하면 됩니다. 아래는 연결된 모습을 보여줍니다.

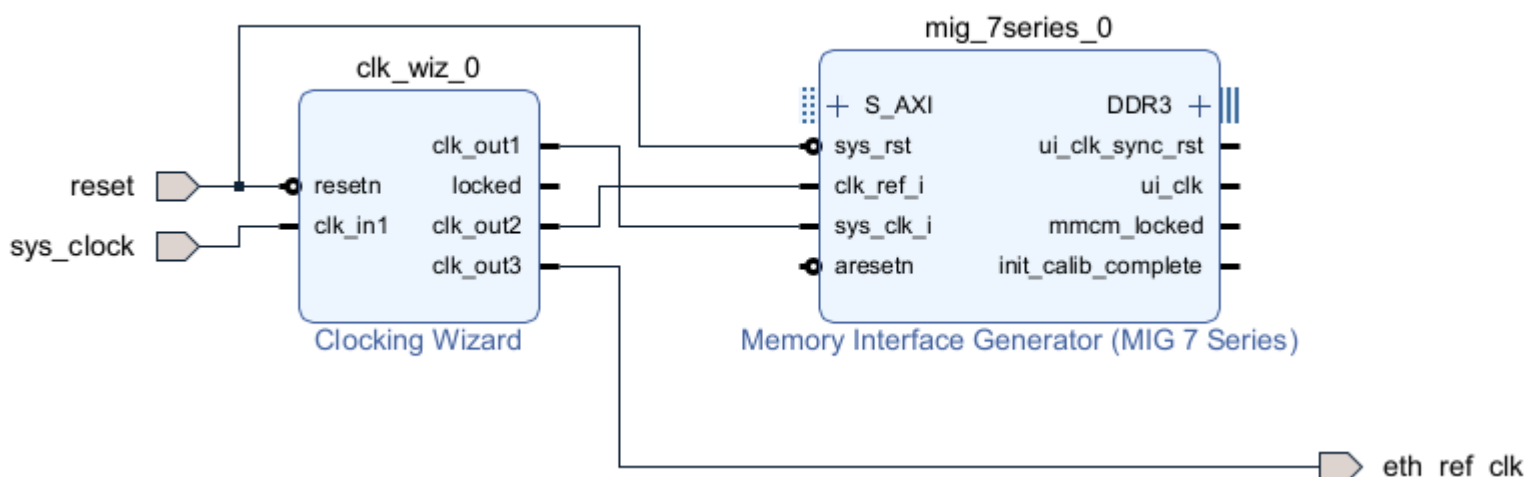


동일하게 sys_clock 포트를 생성(Port name : sys_clock, Direction : Input, Type : Clock, Frequency : 100(Mhz))하고, clk_wiz_0의 clk_in1 신호와 연결합니다.

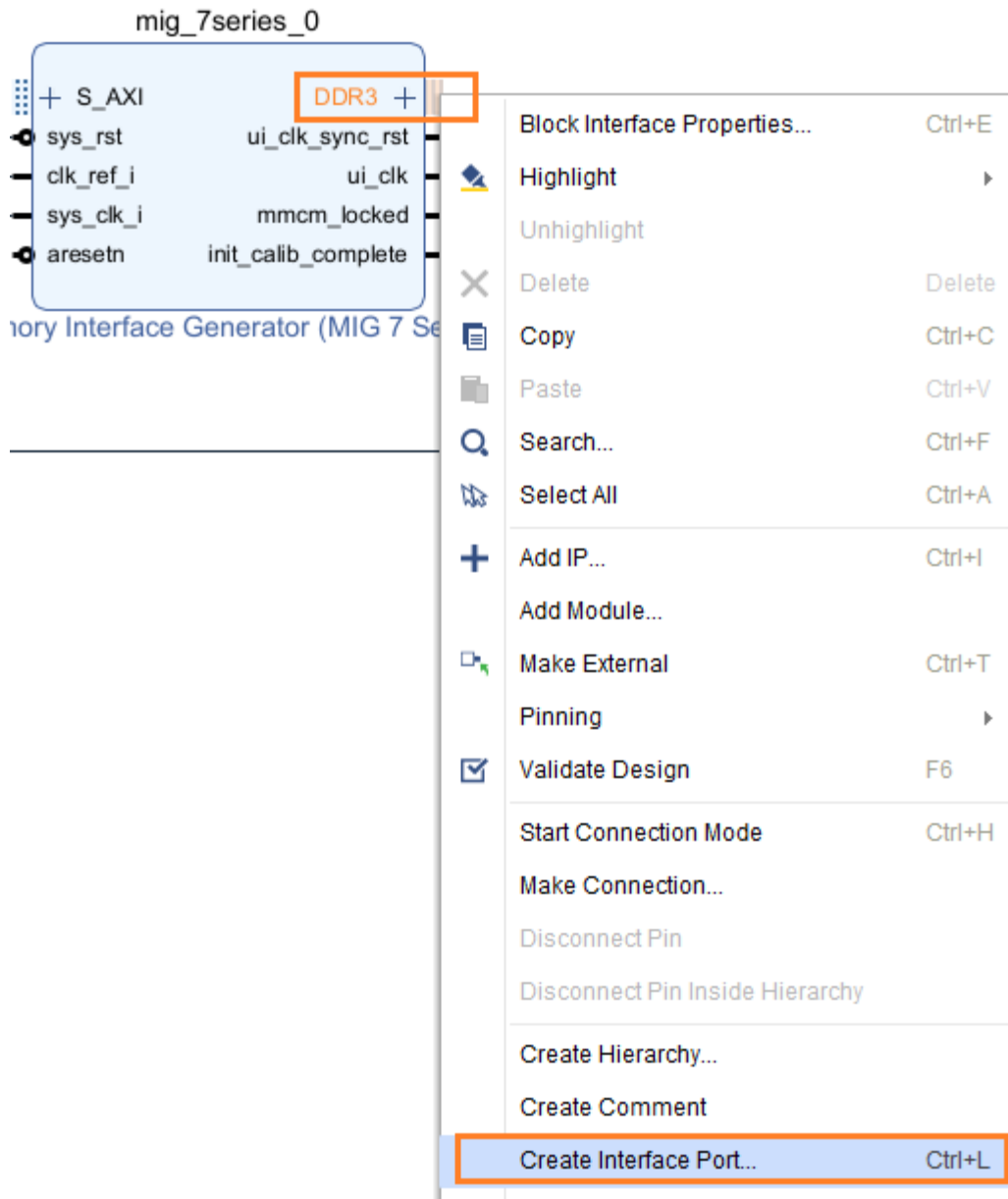


eth_ref_clk 포트(Ethernet Transceiver 의 main clock)를 생성하고, clk_wiz_0의 clk_out3신호와 연결합니다.(port name : eth_ref_clk, direction : output, type : clock)

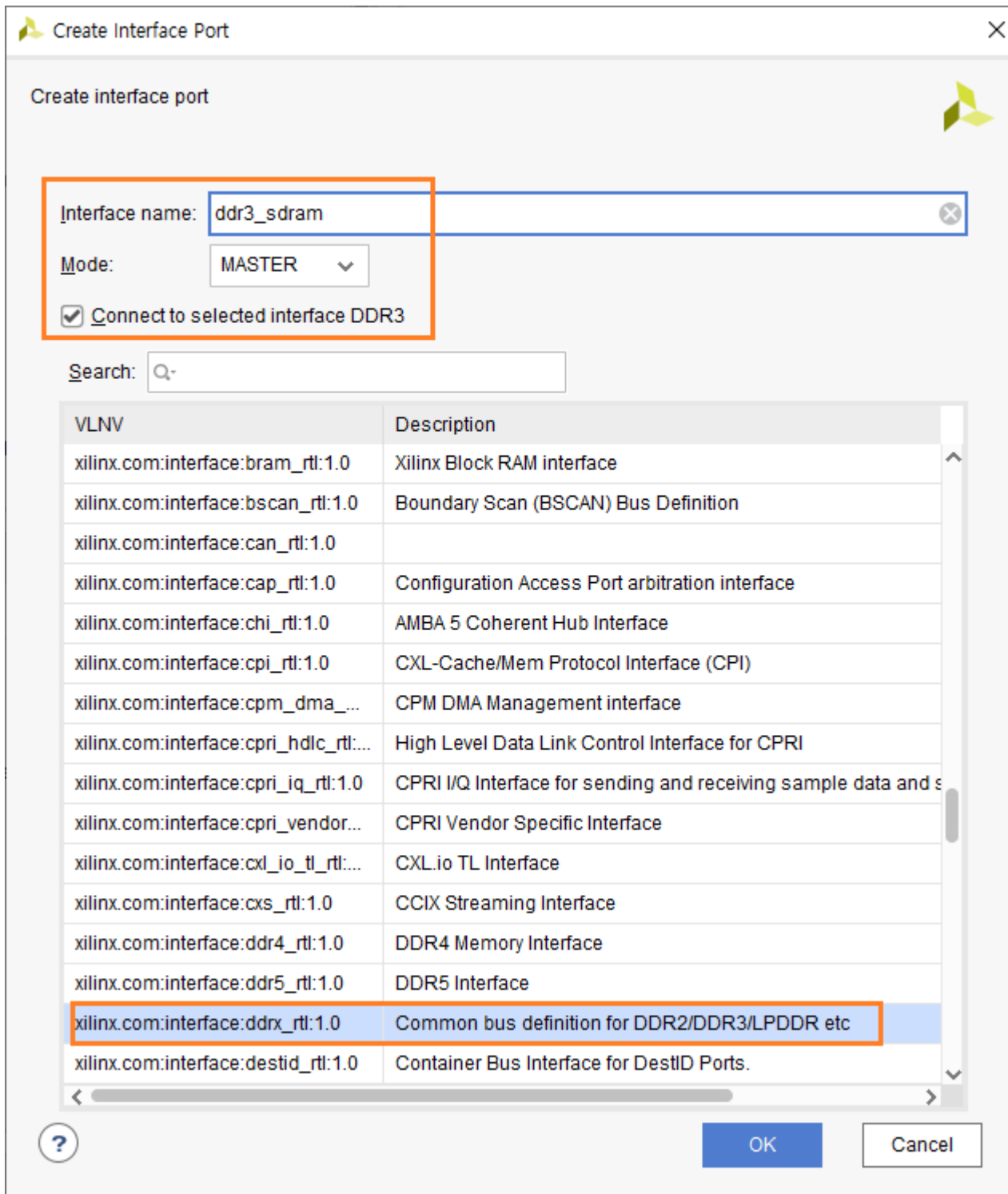
clk_wiz_0의 clk_out1 신호와 mig_7series_0의 sys_clk_i, clk_wiz_0의 clk_out2와 mig_7series_0의 clk_ref_i 를 각각 연결합니다. 지금까지의 내용이 적용된 Diagram을 보여줍니다.



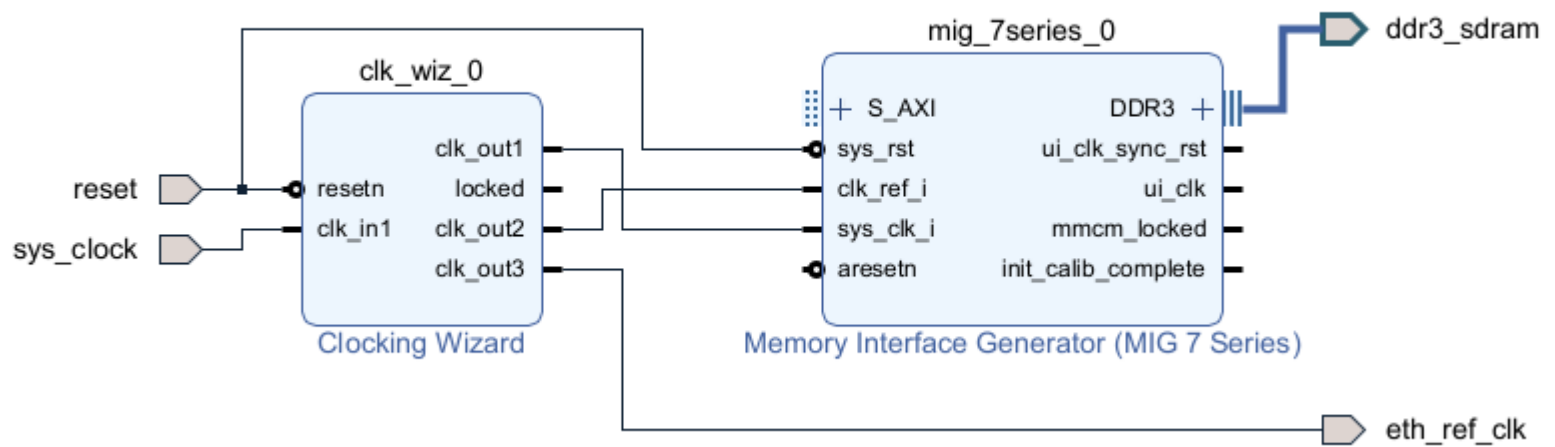
dd3 memory 포트를 추가합니다. 지금까지는 단일핀을 추가했는데, 이번에는 Interface 핀을 추가합니다. mig_7series_0 의 DDR3 핀을 선택하고 마우스 우클릭을 하면 메뉴가 나타납니다. Create Interface Port... 를 클릭합니다.



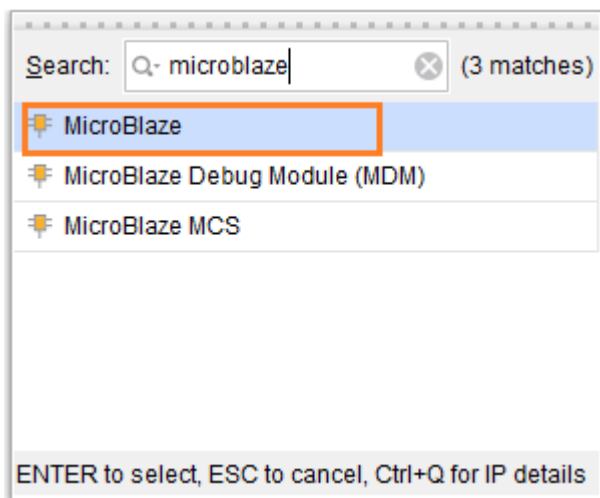
Create Interface Port 윈도우에서 Interface Name : ddr3_sdram, Mode : Master, Connect to selected interface DDR3 : check 을 입력/선택하고 OK 버튼을 클릭합니다. (Common bus definition for DDR2/DDR3/LPDDR etc 가 자동으로 선택되어 있습니다)



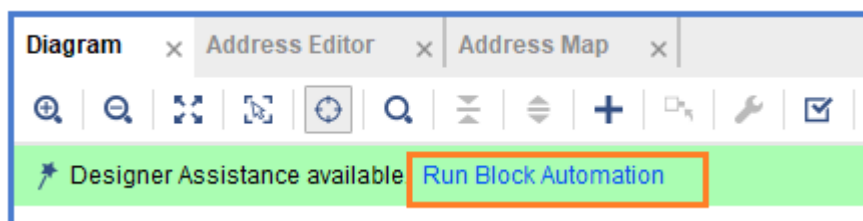
아래는 지금까지 설정된 내용을 보여줍니다.



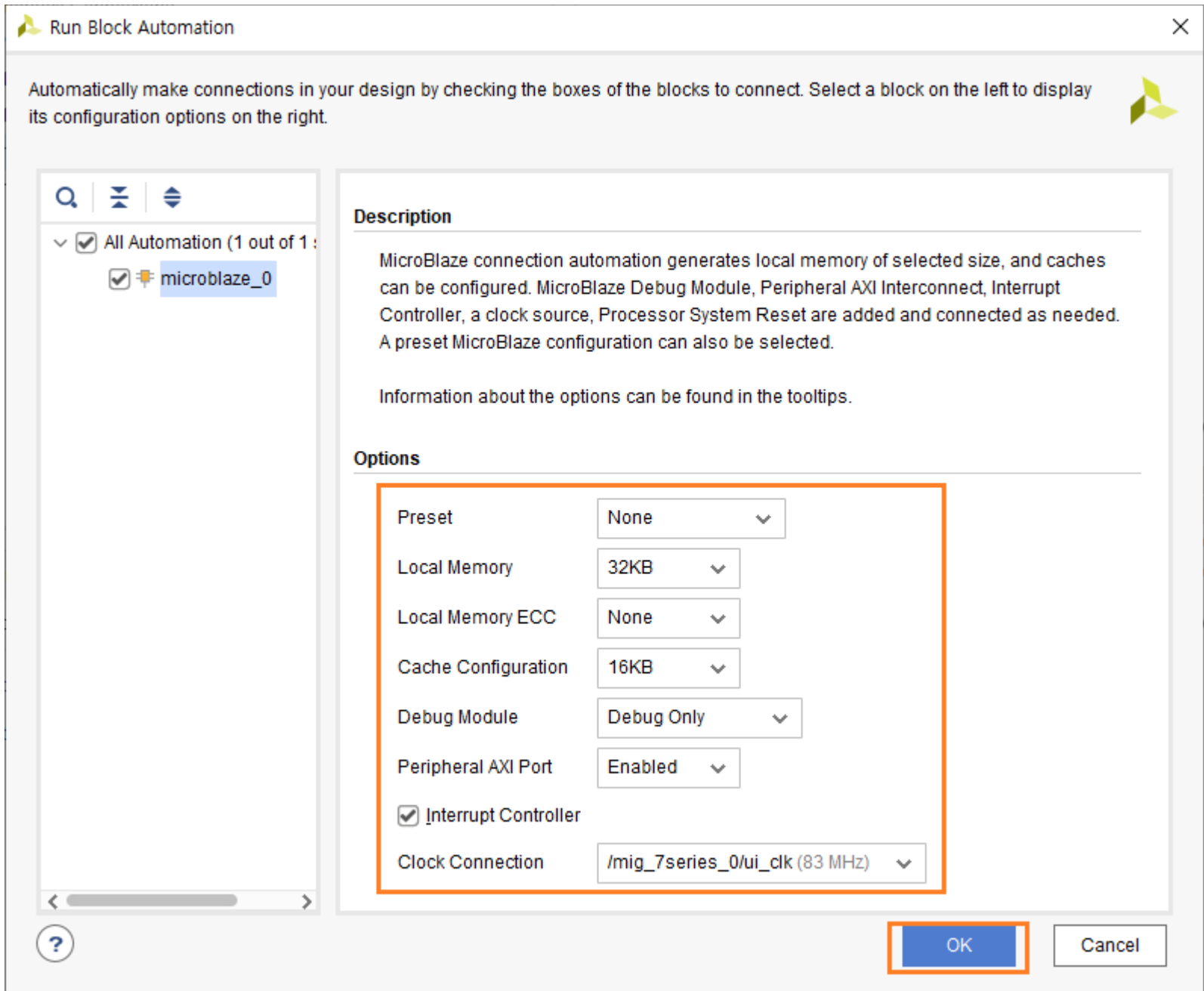
Microblaze를 추가합니다. Add IP (+ 아이콘)을 클릭하고 “microblaze”을 입력하고 리스트 중에 “MicroBlaze”를 더블 클릭해서 IP를 추가합니다



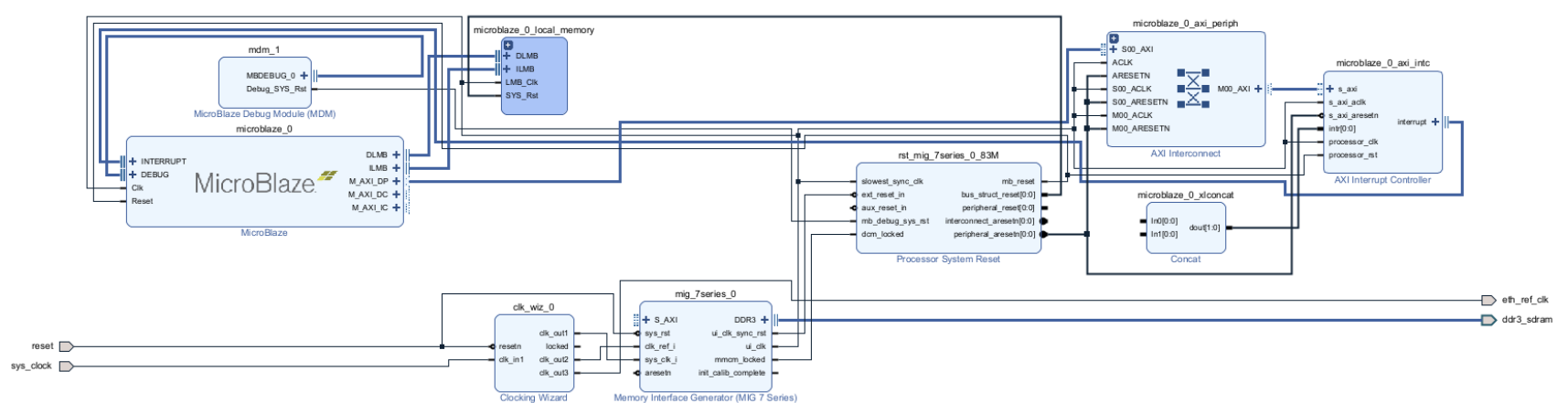
Run Block Automation 을 클릭합니다.



아래와 같이 설정을 변경하고 OK 를 클릭합니다. 특히 Interrupt Controller : check, Clock connection : /mig_7series_0/ui_clk(83 Mhz) 는 반드시 확인해야 합니다.

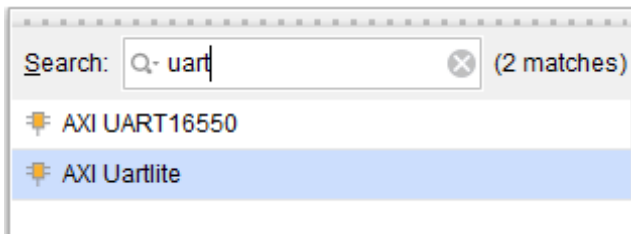


지금까지 설정된 내용입니다.

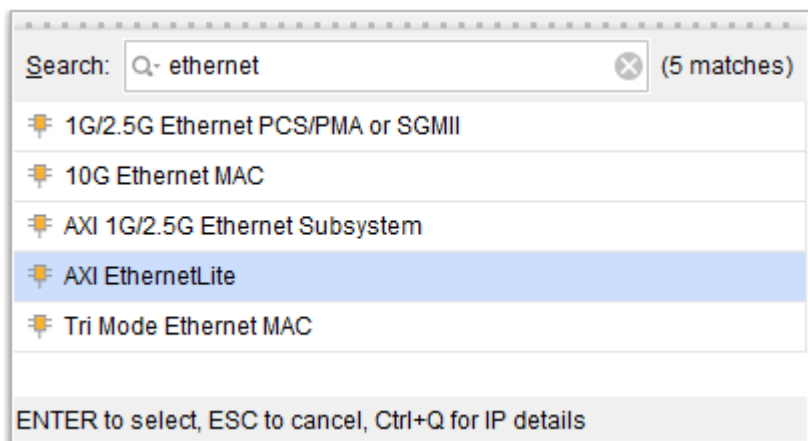


이제 나머지 3개의 IP (Uart, Ethernet, Timer)를 추가합니다.

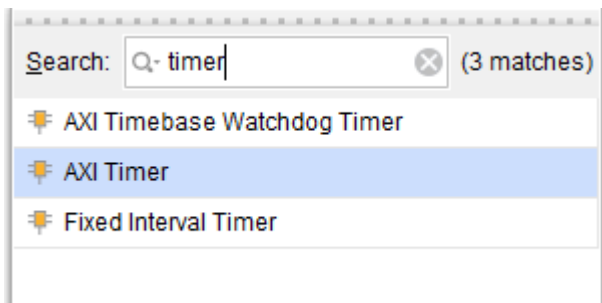
AXI Uartlite 를 더블 클릭합니다.



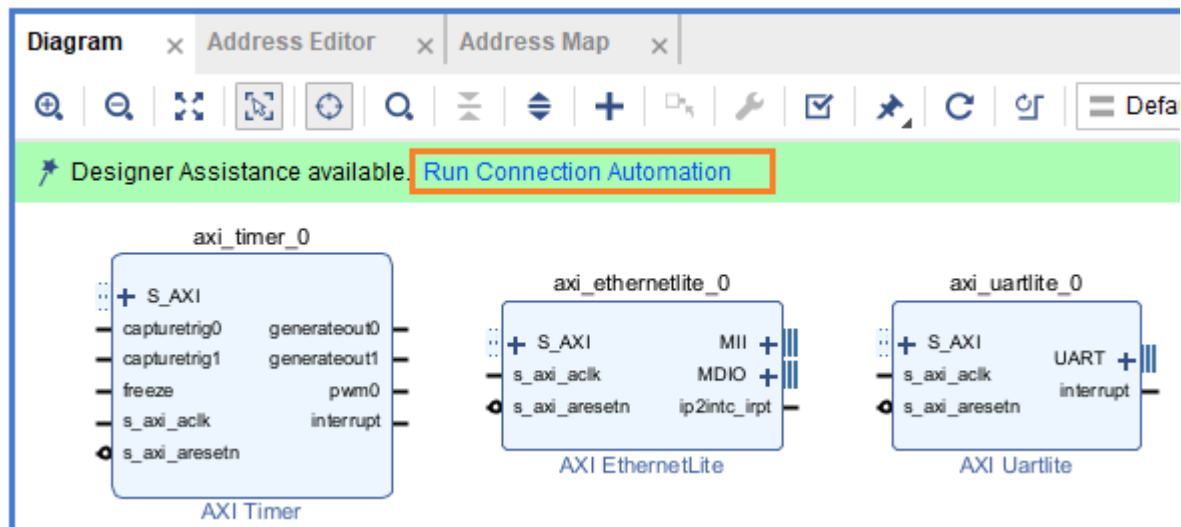
AXI EthernetLite 를 더블 클릭합니다.



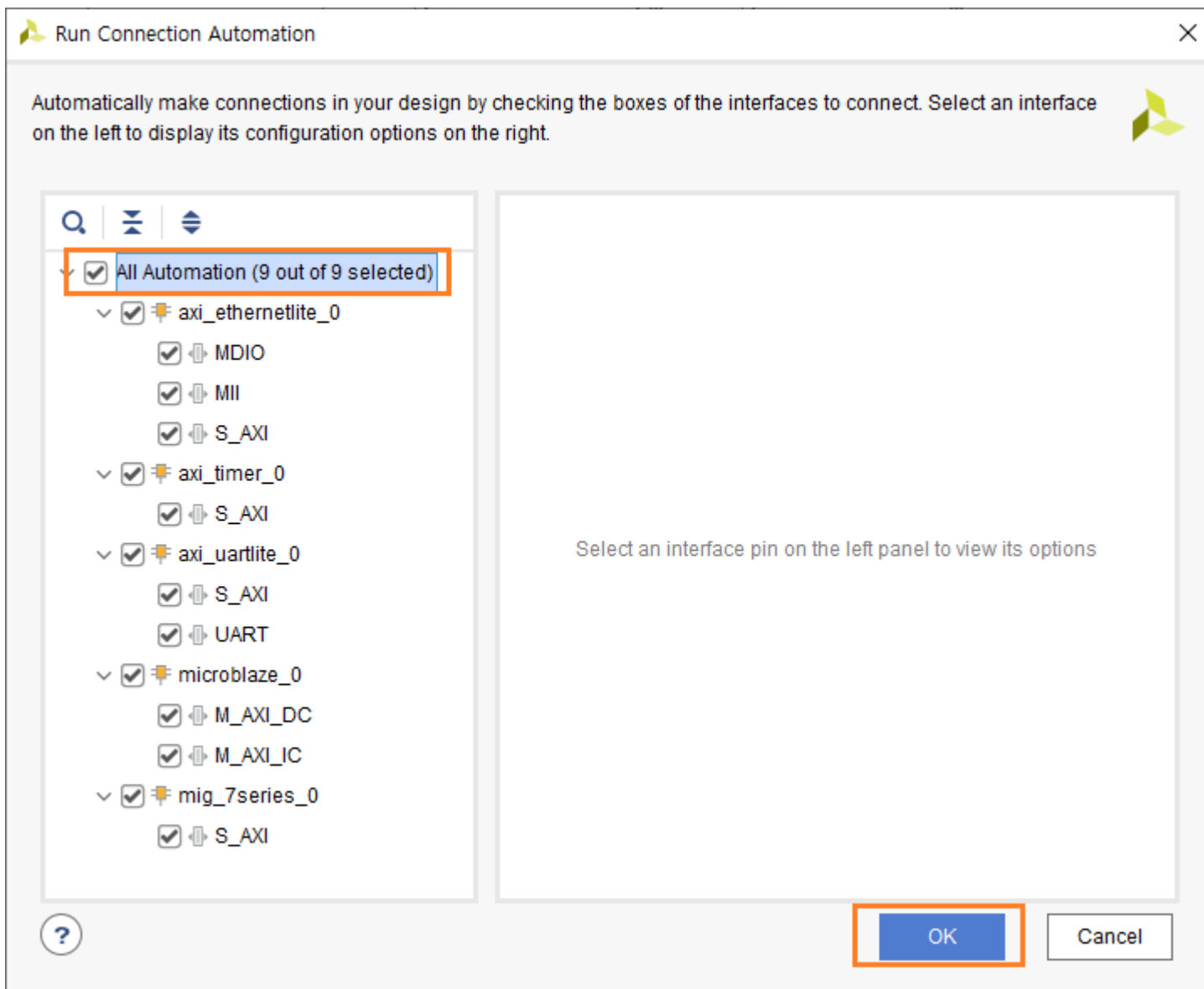
AXI Timer를 더블 클릭합니다.



추가된 3개의 Block에 대해서 “Run Connection Automation”을 클릭합니다.

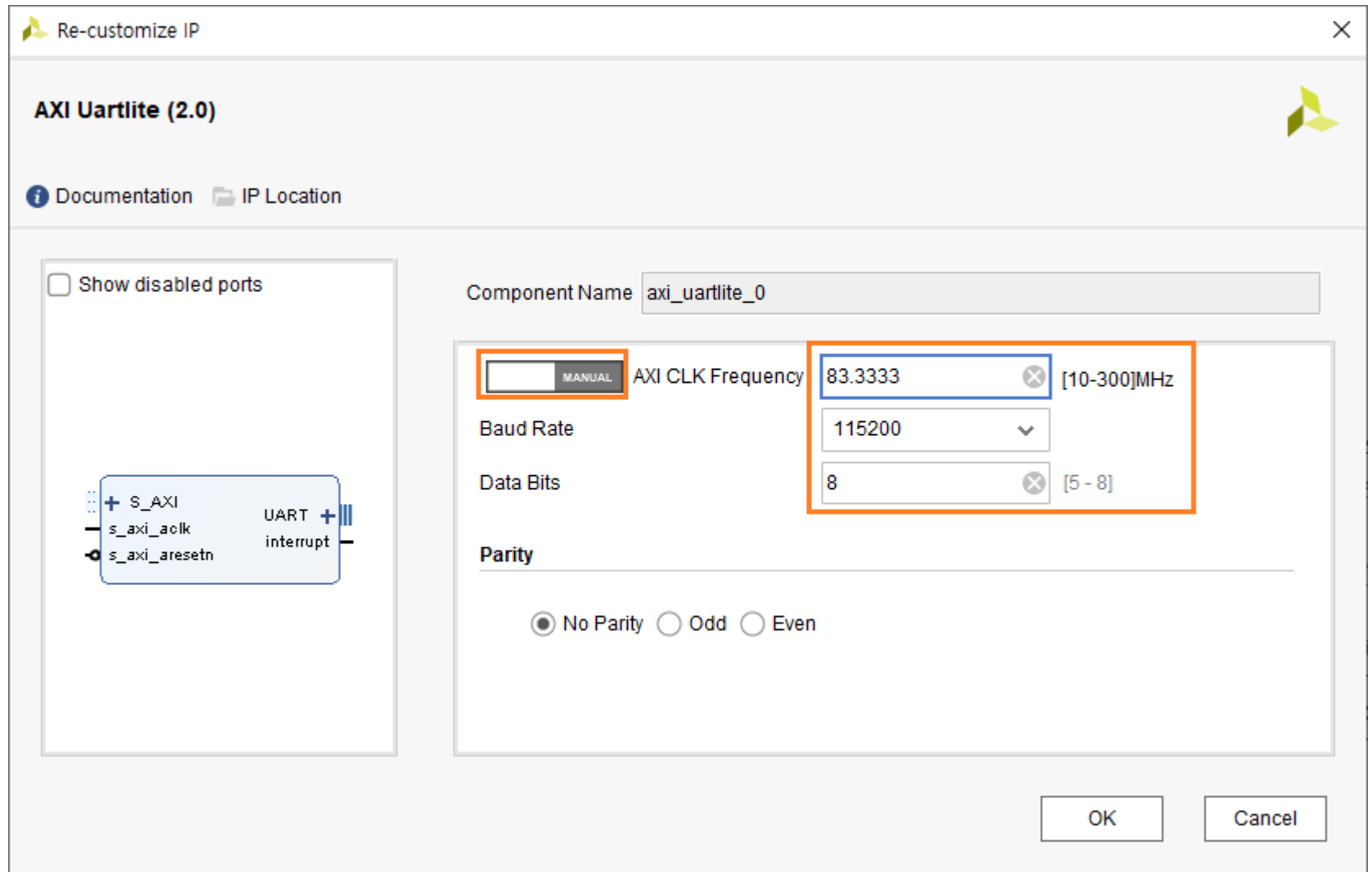


All Automation을 Check 해서 전체를 선택 후 OK를 클릭합니다.



axi_uartlite_0을 더블 클릭해서 속성을 변경합니다.

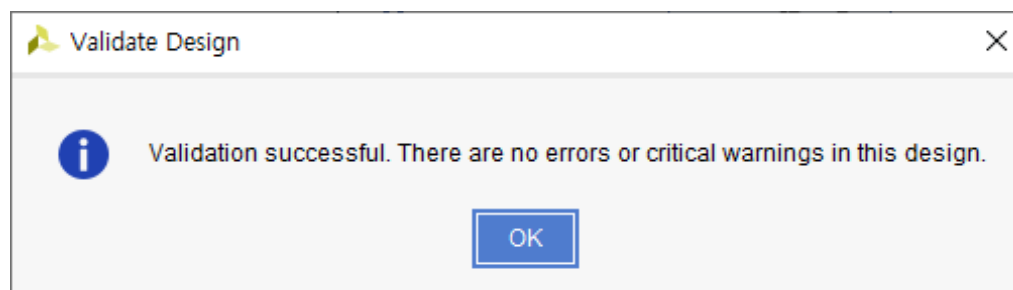
AXI CLK Frequency 를 변경합니다. 왼쪽의 AUTO를 MANUAL로 변경후 83.3333 (Mhz)로 수정합니다. Baud Rate : 115200으로 변경합니다. OK 를 클릭합니다.



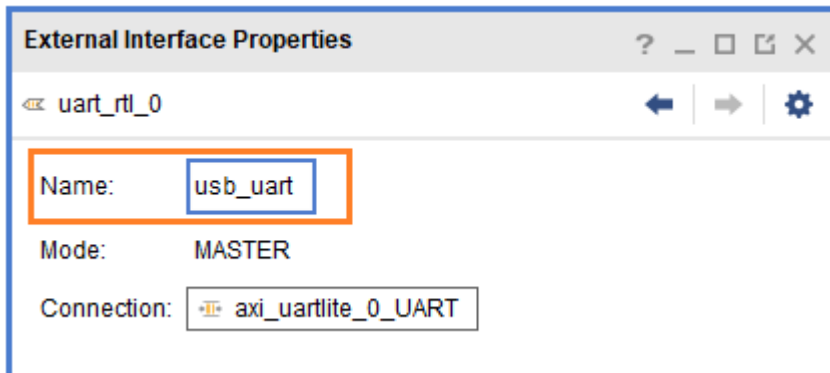
axi_ethernetlite_0, axi_timer_0는 기본 설정값을 사용합니다.

마지막으로 interrupt 신호를 연결합니다. axi_timer_0의 interrupt를 microblaze_0_xlconcat의 In0[0:0]에, axi_ethernetlite_0의 ip2intc_irpt를 microblaze_0_xlconcat의 In1[0:0]에 연결합니다.

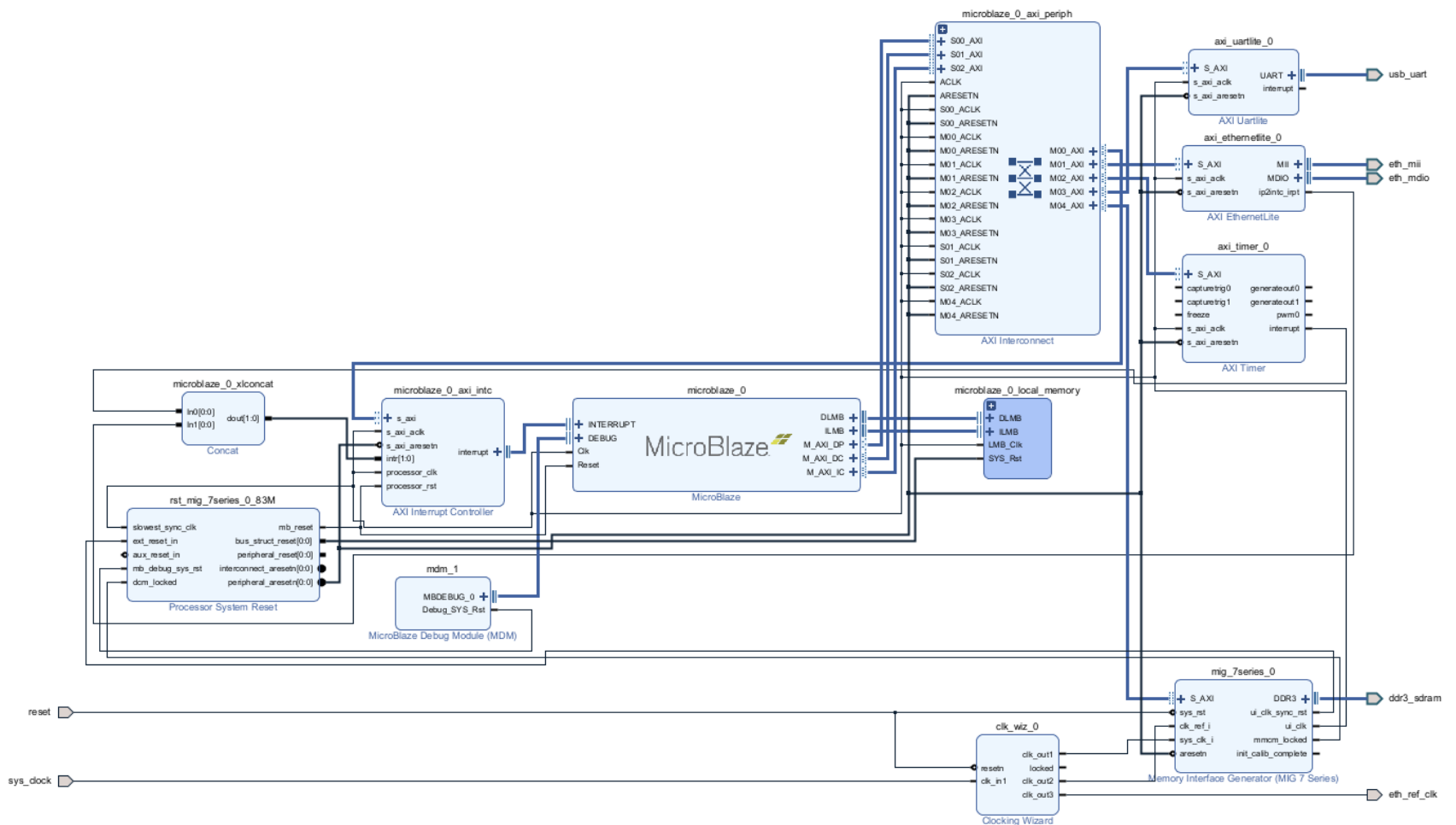
Validate Design () 아이콘을 클릭해서 지금까지 설계된 내용에 오류가 없는지 확인합니다. 오류가 없으면 아래와 같이 “Validation successful” 메시지가 나타납니다. OK를 클릭합니다.



자동으로 생성된 Port 이름을 변경합니다. uart_rtl_0 -> usb_uart, mii_rtl_0 -> eth_mii, mdio_rtl_0 -> eth_mdio 로 변경합니다. 해당 포트를 클릭하면 Diagram 왼쪽 하단부에 이름을 변경할 수 있는 윈도가 나타납니다.

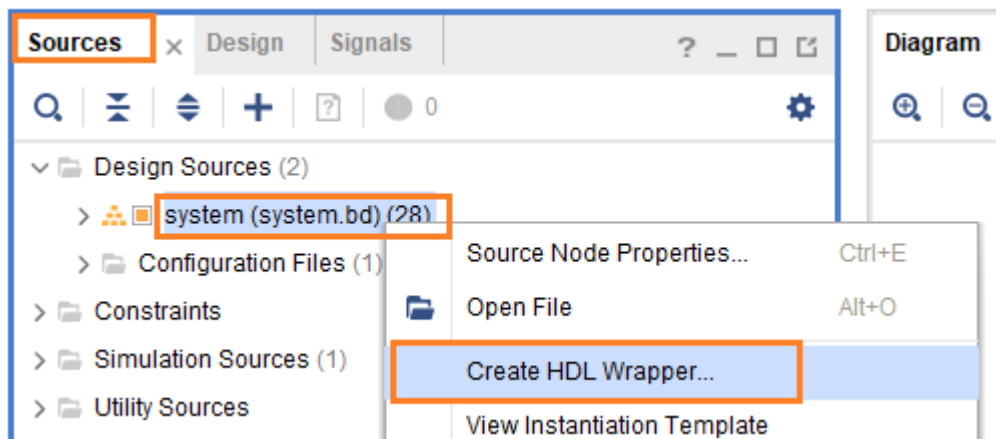


Validate Design 아이콘을 클릭해서 오류가 없는지 확인합니다. Design이 수정되면 반드시 Validate Design 아이콘을 클릭해서 오류를 확인해야 합니다. 오류가 없으면 Regenerate layout () 아이콘을 클릭해서 보기 좋은 Layout으로 변경합니다. 아래 그림은 최종 Diagram을 보여줍니다.

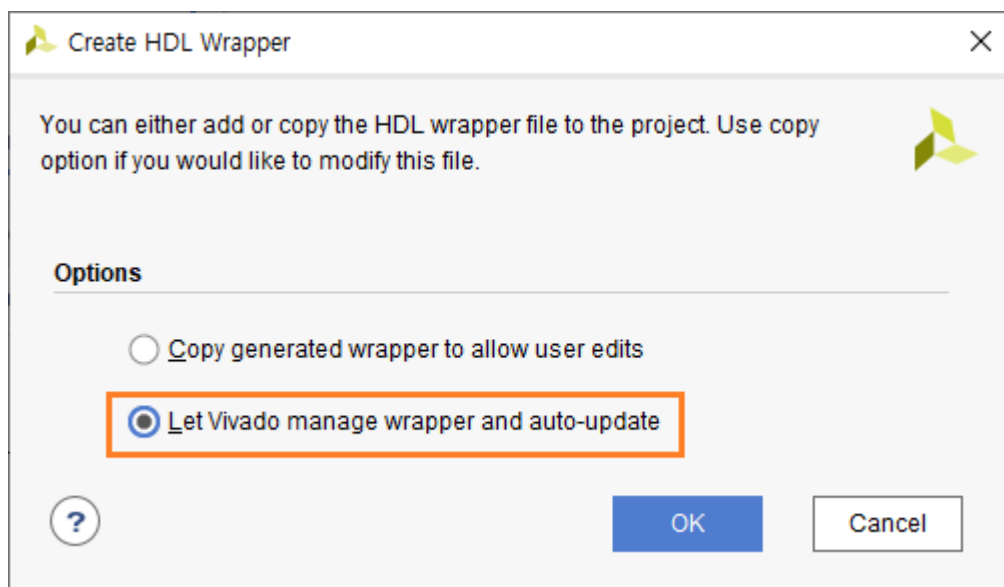


7.2.3 Create HDL Wrapper

지금까지 생성한 HW Block을 HDL Wrapper로 변환합니다. Sources 탭에서 Design Sources - system - 우클릭 - Create HDL Wrapper... 를 클릭합니다.



“Let Vivado manage wrapper and auto-update”를 선택(Default)하고 OK 를 클릭합니다.



Wrapper 파일이 생성되었습니다. 파일 생성 위치를 확인합니다. xdc 파일 생성시 참조해야 합니다.



생성된 wrapper 파일을 기준으로 핀 할당을 하기 위해서 xdc 파일을 생성합니다. 먼저 생성된 wrapper 파일을 확인합니다.

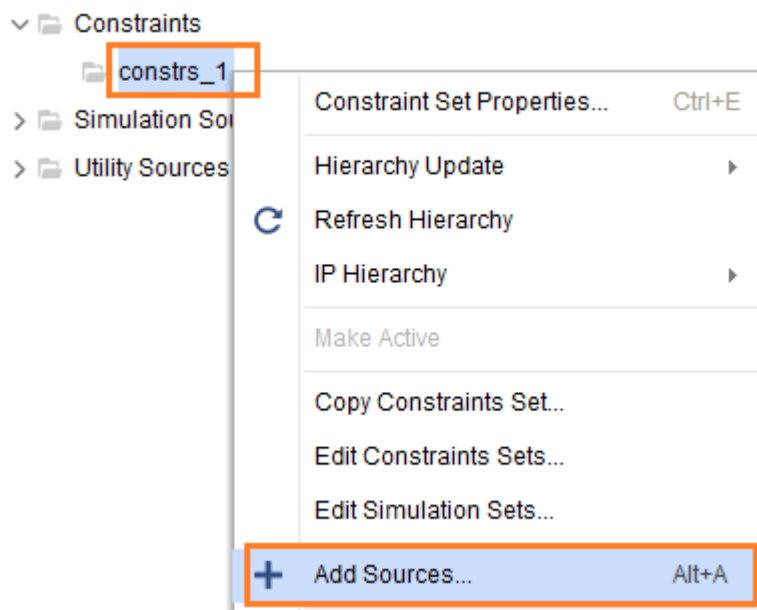
```

12 module system_wrapper
13     (ddr3_sdram_addr,
14      ddr3_sdram_ba,
15      ddr3_sdram_cas_n,
16      ddr3_sdram_ck_n,
17      ddr3_sdram_ck_p,
18      ddr3_sdram_cke,
19      ddr3_sdram_cs_n,
20      ddr3_sdram_dm,
21      ddr3_sdram_dq,
22      ddr3_sdram_dqs_n,
23      ddr3_sdram_dqs_p,
24      ddr3_sdram_odt,
25      ddr3_sdram_ras_n,
26      ddr3_sdram_reset_n,
27      ddr3_sdram_we_n,
28      eth_mdio_mdc,
29      eth_mdio_mdio_io,
30      eth_mii_col,
31      eth_mii_crs,
32      eth_mii_rst_n,
33      eth_mii_rx_clk,
34      eth_mii_rx_dv,
35      eth_mii_rx_er,
36      eth_mii_rxd,
37      eth_mii_tx_clk,
38      eth_mii_tx_en,
39      eth_mii_txd,
40      eth_ref_clk,
41      reset,
42      sys_clock,
43      usb_uart_rxd,
44      usb_uart_txd);
45 output [13:0] ddr3_sdram_addr;
46 output [2:0] ddr3_sdram_ba;
47 output ddr3_sdram_cas_n;
48 output [0:0] ddr3_sdram_ck_n;
49 output [0:0] ddr3_sdram_ck_p;
50 output [0:0] ddr3_sdram_cke;
51 output [0:0] ddr3_sdram_cs_n;
52 output [1:0] ddr3_sdram_dm;
53 inout [15:0] ddr3_sdram_dq;
54 inout [1:0] ddr3_sdram_dqs_n;
55 inout [1:0] ddr3_sdram_dqs_p;
56 output [0:0] ddr3_sdram_odt;
57 output ddr3_sdram_ras_n;
58 output ddr3_sdram_reset_n;
59 output ddr3_sdram_we_n;
60 output eth_mdio_mdc;
61 inout eth_mdio_mdio_io;
62 input eth_mii_col;
63 input eth_mii_crs;
64 output eth_mii_rst_n;
65 input eth_mii_rx_clk;
66 input eth_mii_rx_dv;
67 input eth_mii_rx_er;
68 input [3:0] eth_mii_rxd;
69 input eth_mii_tx_clk;
70 output eth_mii_tx_en;
71 output [3:0] eth_mii_txd;
72 output eth_ref_clk;
73 input reset;
74 input sys_clock;
75 input usb_uart_rxd;
76 output usb_uart_txd;
77

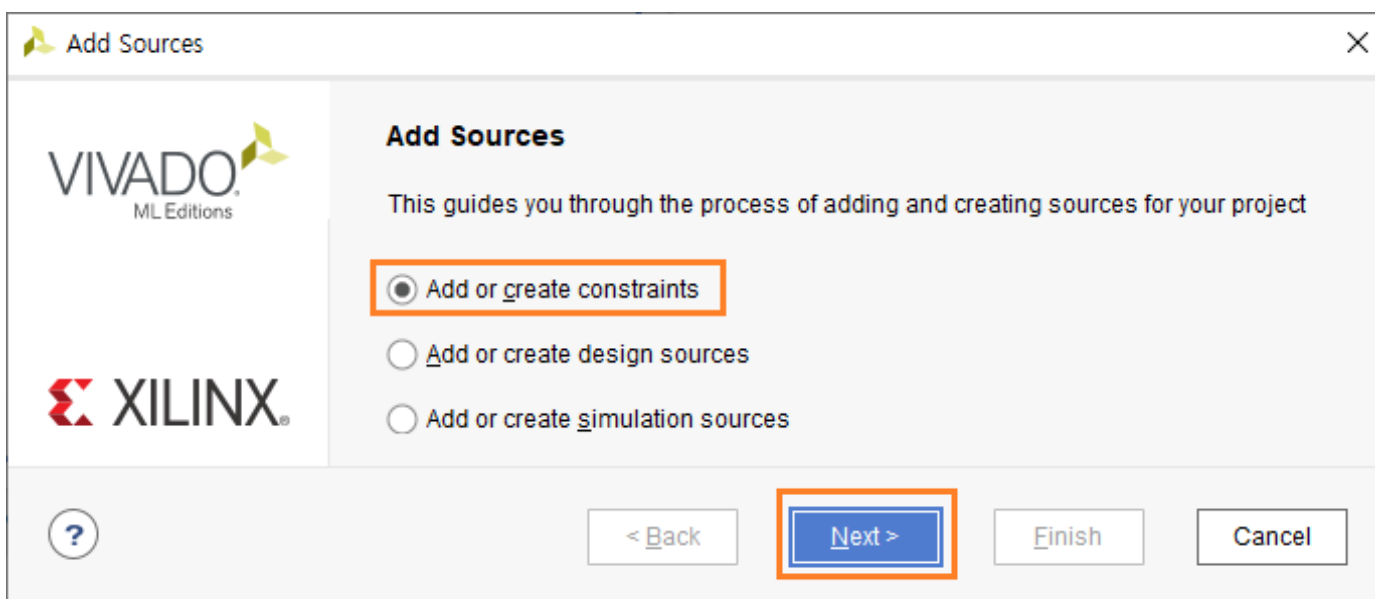
```

ddr3_sdram 관련 핀들은 IP 생성할 때 핀할당을 했기 때문에 할 필요가 없고 나머지 포트들의 핀할당을 합니다. 포트 이름을 참고하여 xdc 파일을 생성하고 내용을 추가합니다.

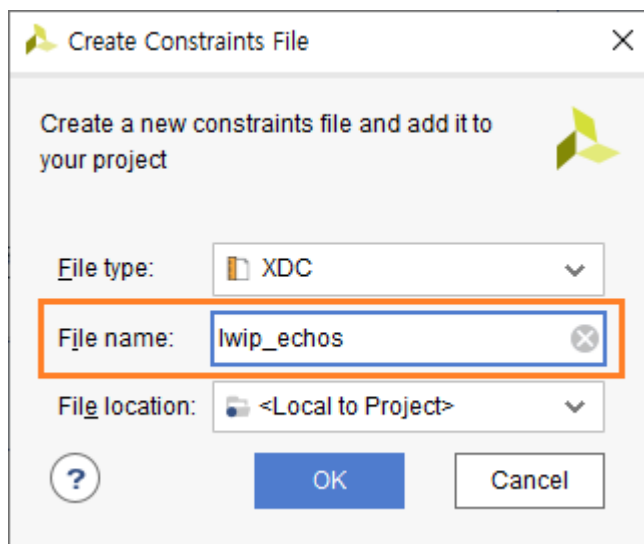
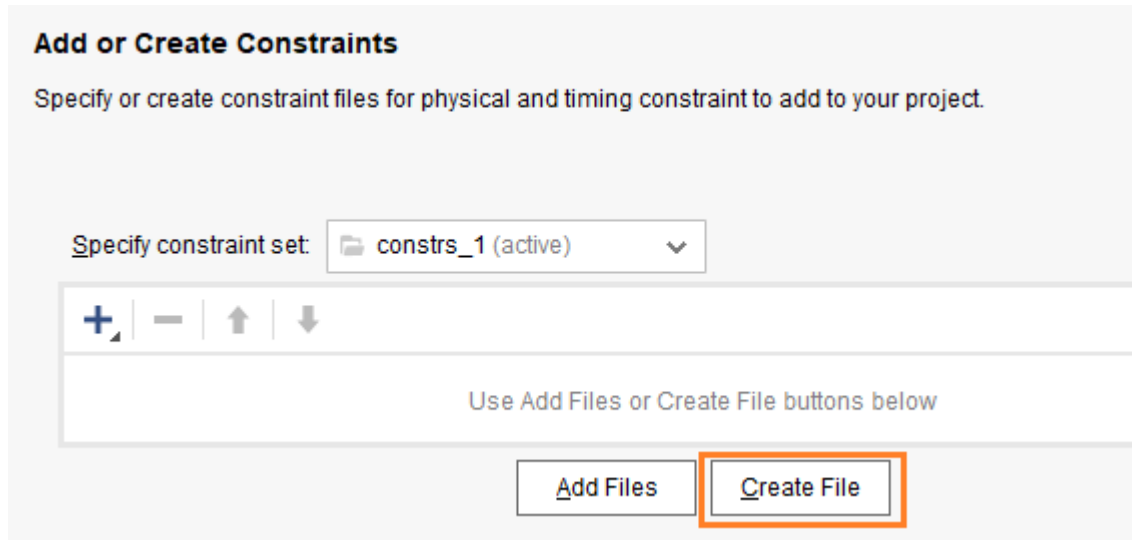
Sources 탭에서 Constraints - constrs_1 - 우클릭 - Add Sources... 를 클릭합니다.



“Add or create constraints”를 선택하고 Next를 클릭합니다.



Create File 버튼을 클릭하고, 파일 이름을 “lwip_echoes”로 입력하고 OK 를 클릭합니다.



Finish 버튼을 클릭하면, “lwip_echoes.xdc” 파일이 생성됩니다.



lwip_echoes.xdc 파일에 핀 할당 정보를 추가합니다. 회로도를 참조하여 핀 할당을 하거나, Digilent 사에서 제공하는 파일을 참조하여 내용을 추가합니다. 다운로드 자료중에 “lwip_echoes.xdc” 파일의 내용을 복사해서 사용하면 됩니다.

lwip_echos.xdc 파일 내용입니다. wrapper 파일과 포트 이름이 동일해야 합니다.

```

1 ## Clock signal
2 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clock }];
3 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clock }];
4
5 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
6
7 set_property -dict { PACKAGE_PIN G18     IOSTANDARD LVCMOS33 } [get_ports { eth_ref_clk }];
8
9 set_property -dict { PACKAGE_PIN F16     IOSTANDARD LVCMOS33 } [get_ports { eth_mdio_mdc }];
10 set_property -dict { PACKAGE_PIN K13     IOSTANDARD LVCMOS33 } [get_ports { eth_mdio_mdio_io }];
11 set_property -dict { PACKAGE_PIN D17     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_col }];
12 set_property -dict { PACKAGE_PIN G14     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_crs }];
13 set_property -dict { PACKAGE_PIN C16     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rst_n }];
14 set_property -dict { PACKAGE_PIN F15     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rx_clk }];
15 set_property -dict { PACKAGE_PIN G16     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rx_dv }];
16 set_property -dict { PACKAGE_PIN C17     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rx_er }];
17 set_property -dict { PACKAGE_PIN D18     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rxd[0] }];
18 set_property -dict { PACKAGE_PIN E17     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rxd[1] }];
19 set_property -dict { PACKAGE_PIN E18     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rxd[2] }];
20 set_property -dict { PACKAGE_PIN G17     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_rxd[3] }];
21 set_property -dict { PACKAGE_PIN H16     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_tx_clk }];
22 set_property -dict { PACKAGE_PIN H15     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_tx_en }];
23 set_property -dict { PACKAGE_PIN H14     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_txd[0] }];
24 set_property -dict { PACKAGE_PIN J14     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_txd[1] }];
25 set_property -dict { PACKAGE_PIN J13     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_txd[2] }];
26 set_property -dict { PACKAGE_PIN H17     IOSTANDARD LVCMOS33 } [get_ports { eth_mii_txd[3] }];
27
28 set_property -dict { PACKAGE_PIN A9      IOSTANDARD LVCMOS33 } [get_ports { usb_uart_rxd }];
29 set_property -dict { PACKAGE_PIN D10     IOSTANDARD LVCMOS33 } [get_ports { usb_uart_txd }];

```

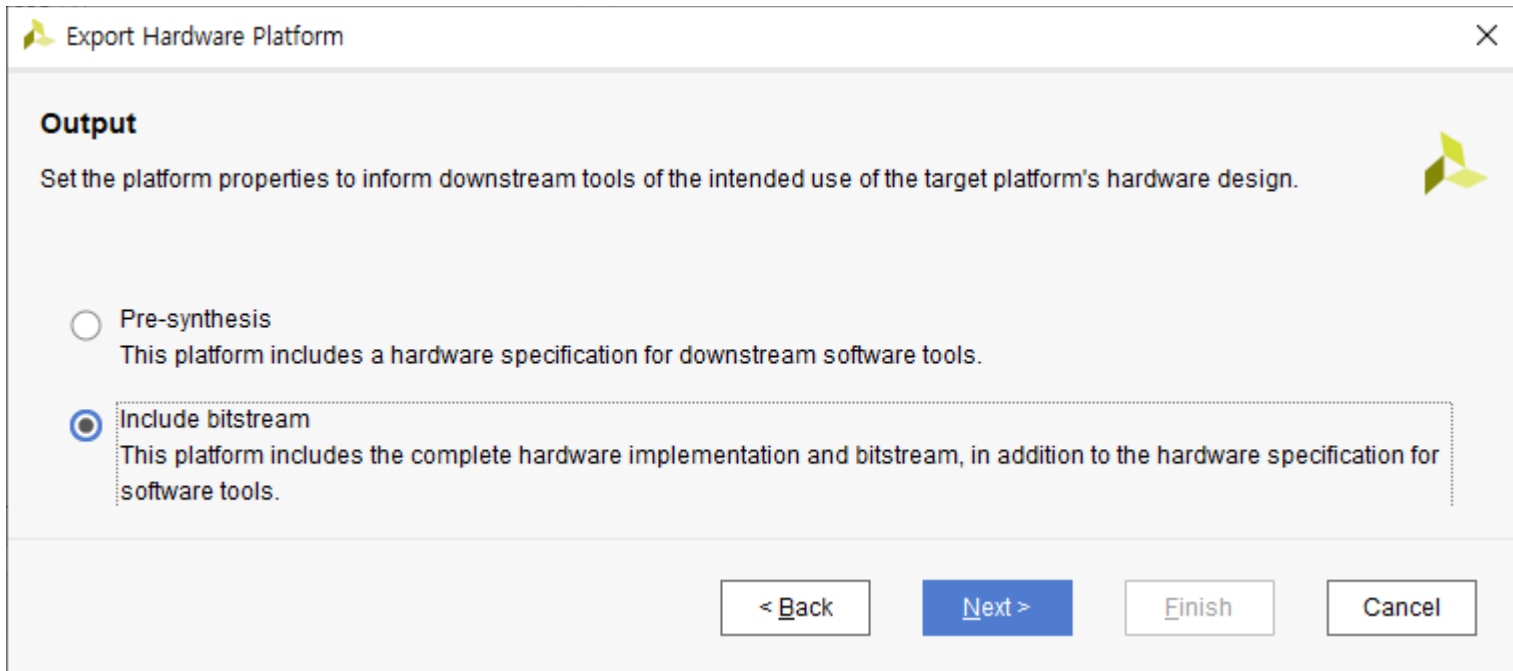
이제 HW Block Design은 완료되었습니다. Bitstream을 생성합니다.

PROGRAM AND DEBUG - Generate Bitstream을 클릭합니다. 이 과정은 수분 ~ 수십분의 시간이 필요합니다. ddr3 memory controller가 있어서 많은 시간이 소요됩니다.

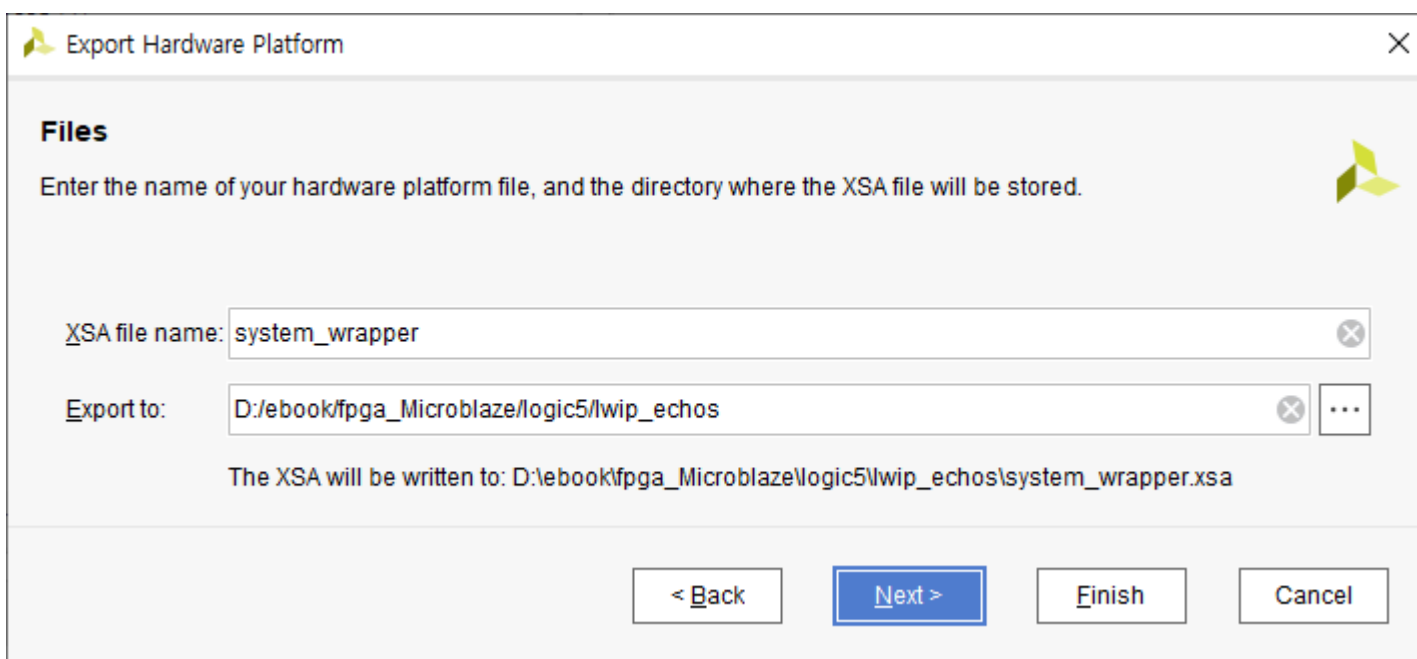
완료가 되면 Cancel 버튼을 클릭할 수도 있고, “Open Implemented Design”를 선택하고 OK를 클릭해서 핀할당이 잘 되었는지 확인할 수 있습니다.

Cancel 버튼을 클릭합니다.

File - Export - Export Hardware... 를 실행하여 Vivado에서 생성한 파일을 Application SW (Vitis)에서 사용하기 위하여 Export 합니다. Include bitstream 을 선택하고 Next 를 클릭합니다.



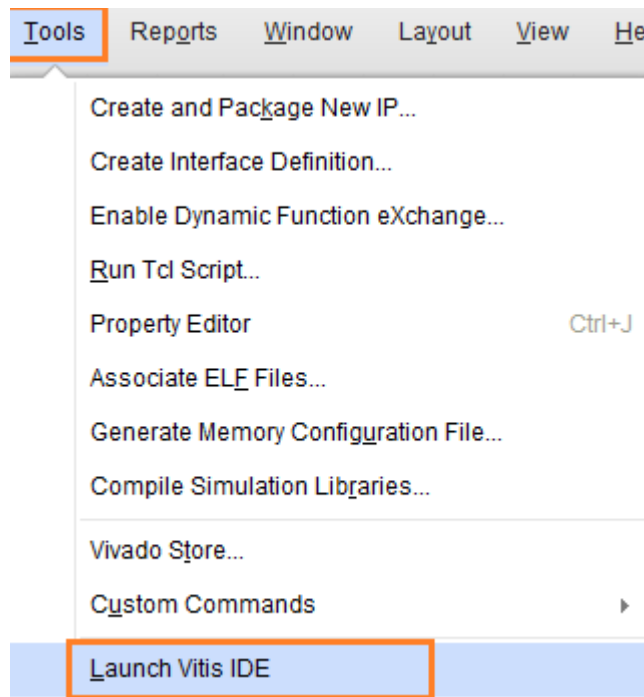
xsa 파일의 생성 위치와 파일명을 확인하고 Next를 클릭합니다.



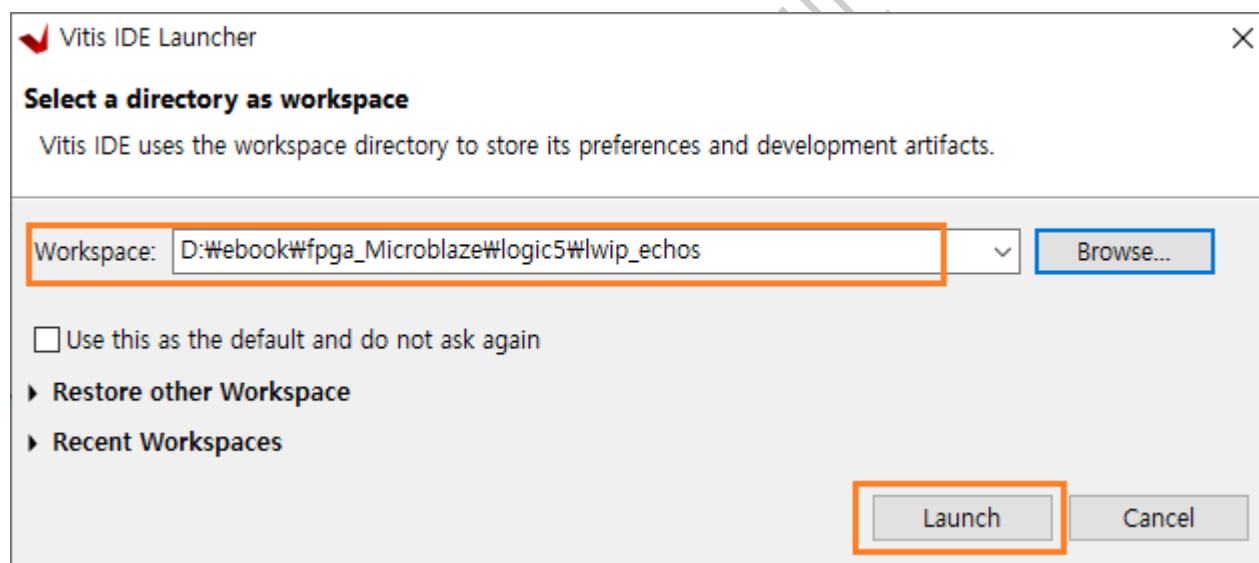
Finish 버튼을 클릭하면 xsa 파일이 생성됩니다.

7.2.4 Application SW

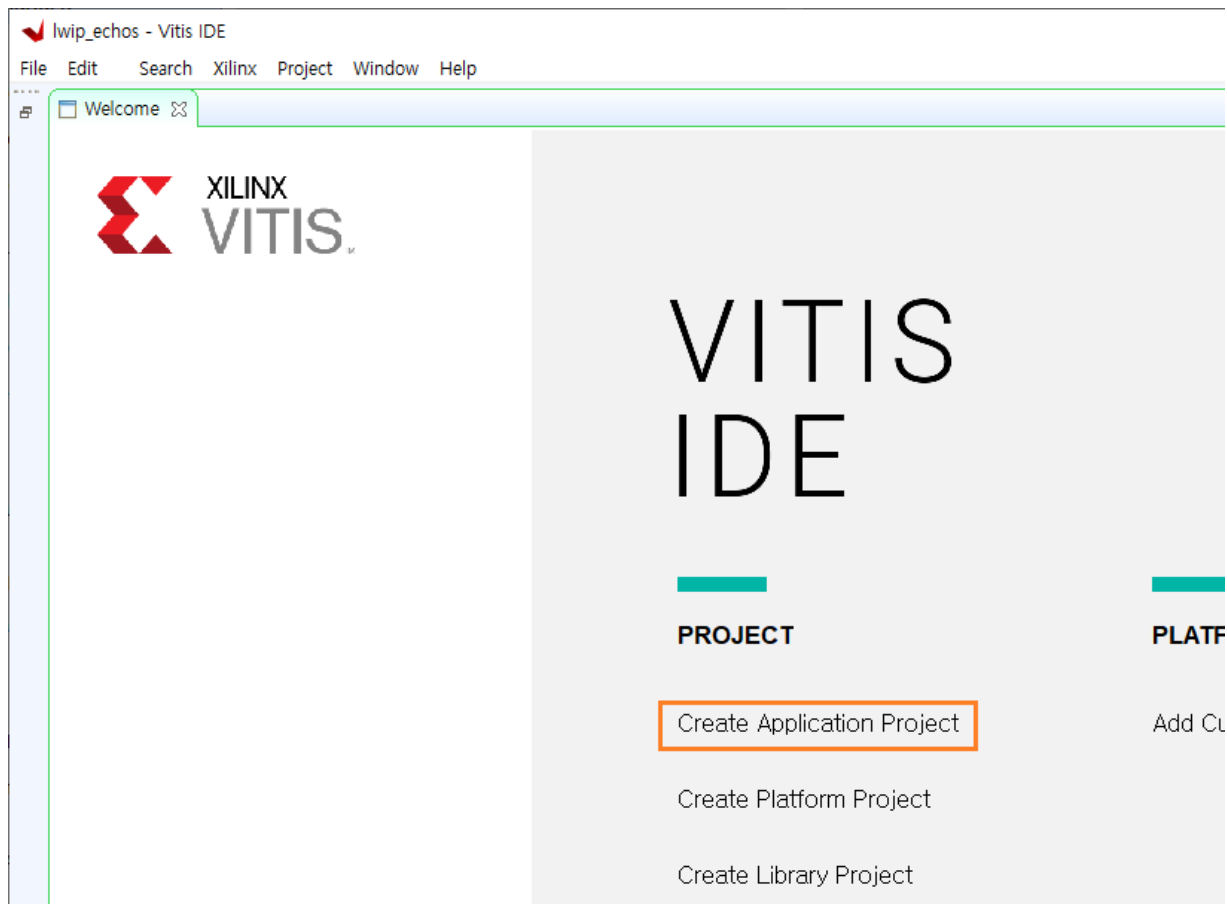
Tools - Launch Vitis IDE 를 클릭해서 Vitis 를 실행합니다.



Workspace 위치가 HW Design 위치와 같은 지 확인하고 (다르면 Browse... 버튼을 클릭해서 경로를 변경합니다) Launch 버튼을 클릭합니다.

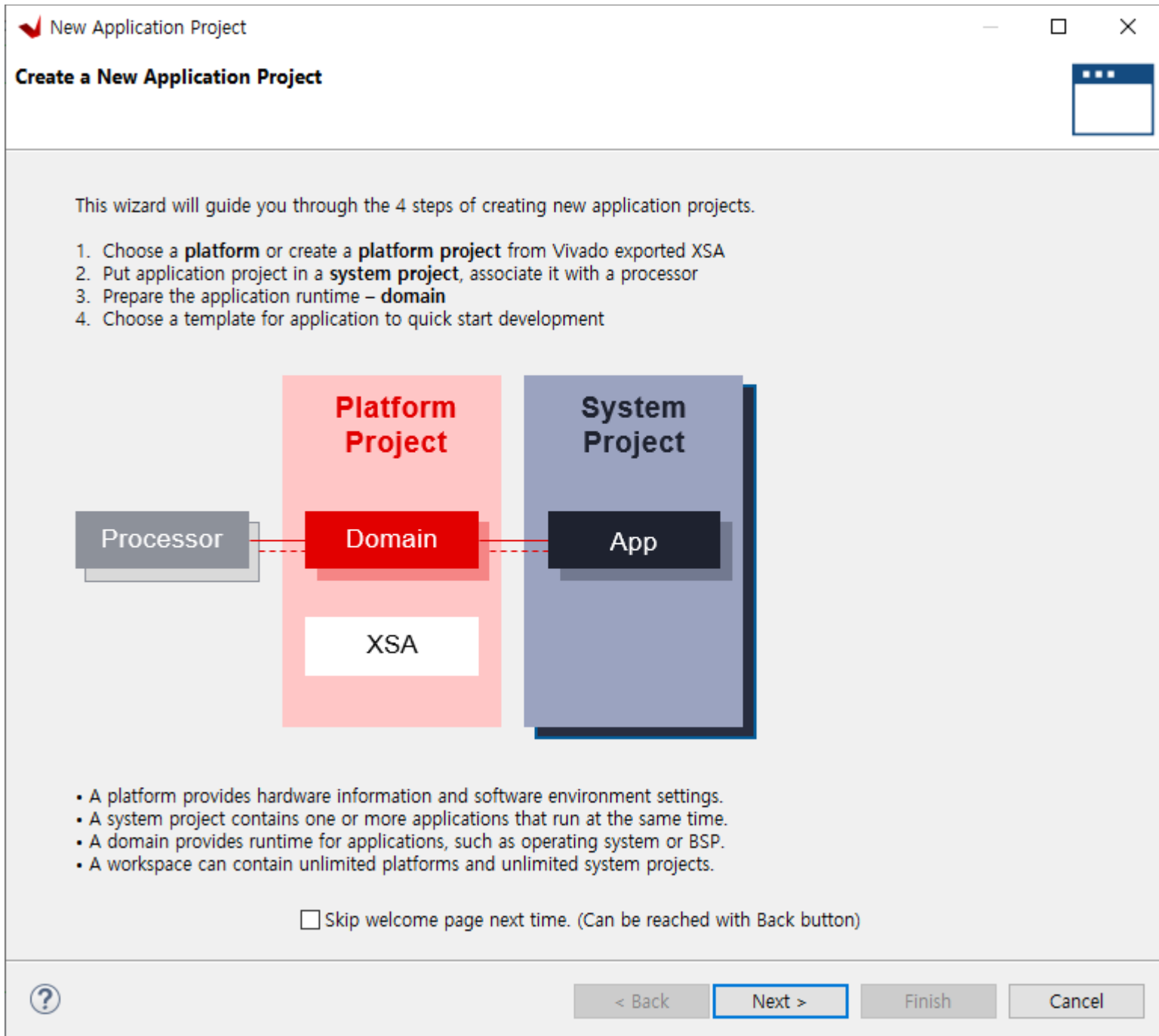


Vitis가 실행되었습니다. Create Application Project를 클릭해서 새로운 프로젝트를 생성합니다.

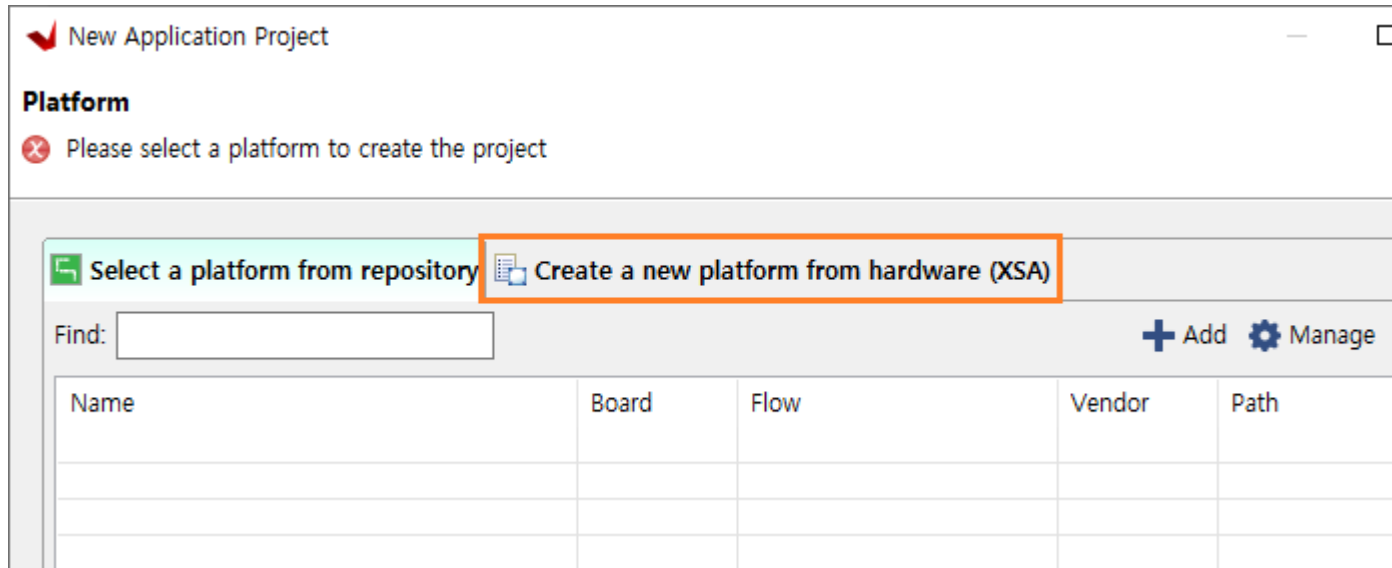


아이힐

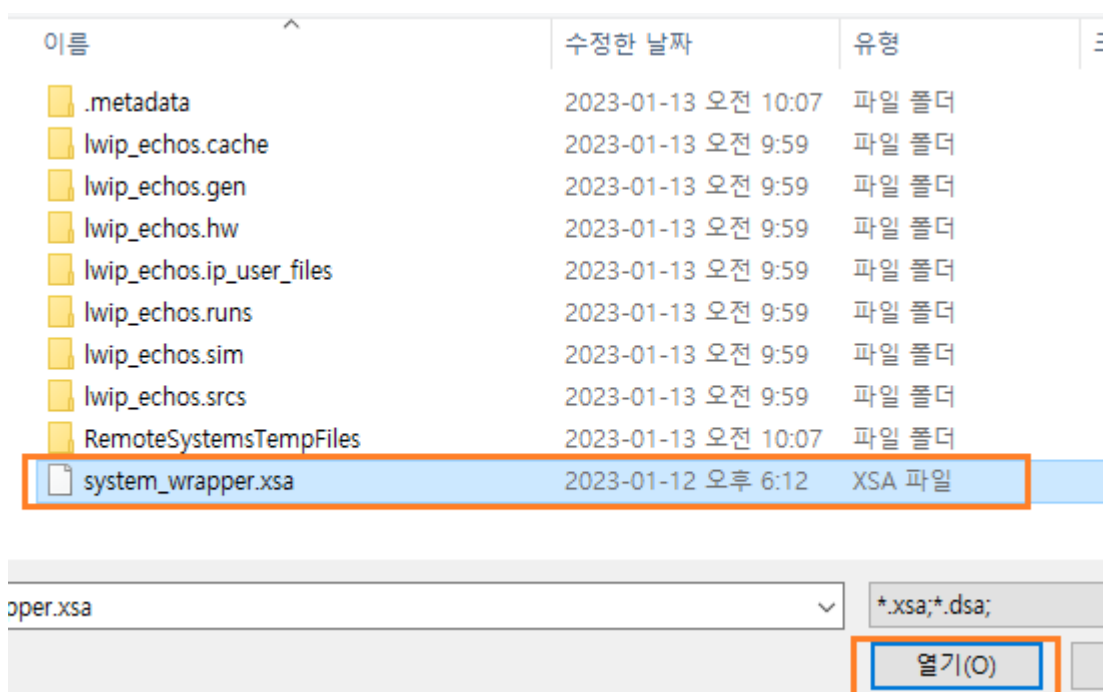
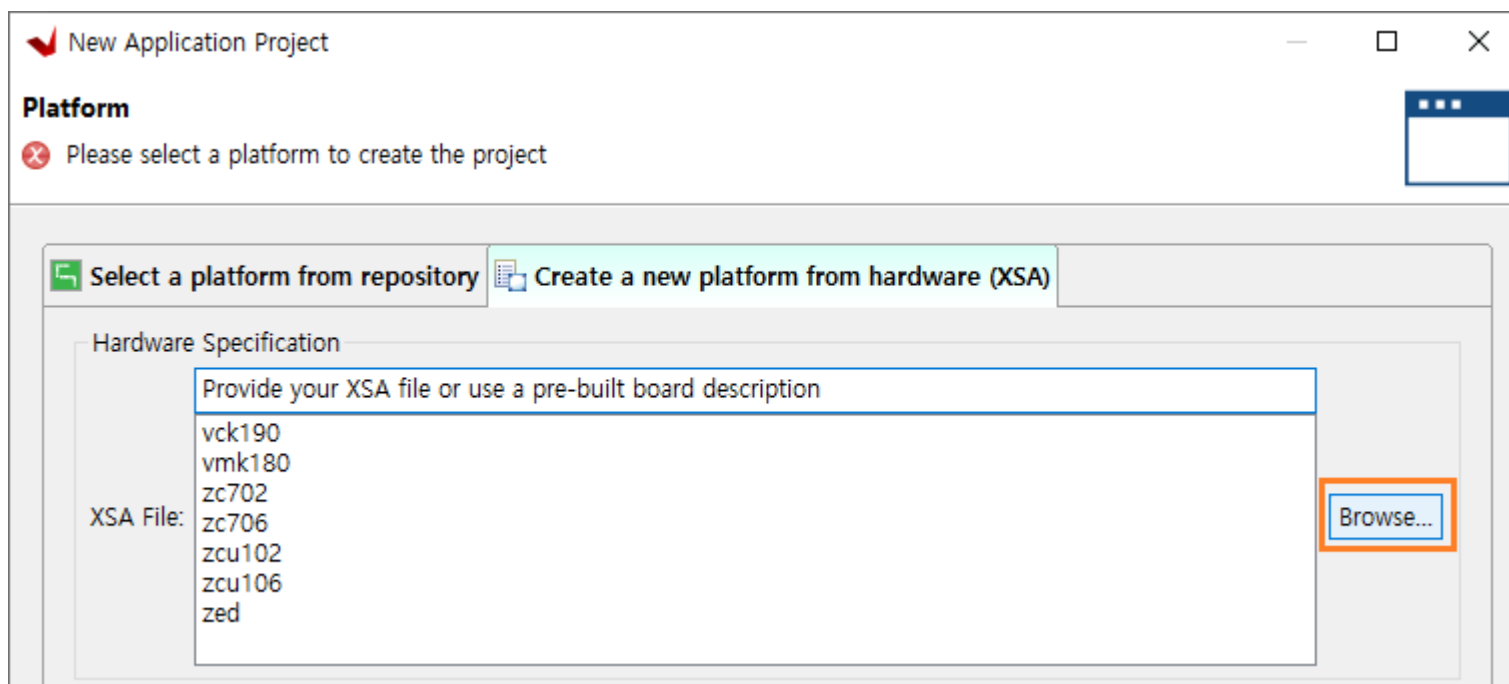
프로젝트 생성과정을 보여줍니다. 첫번째 vivado에서 생성한 xsa 파일을 포함하는 “platform project”를 생성합니다. 다음에 “application project”를 생성합니다. Next를 클릭합니다.



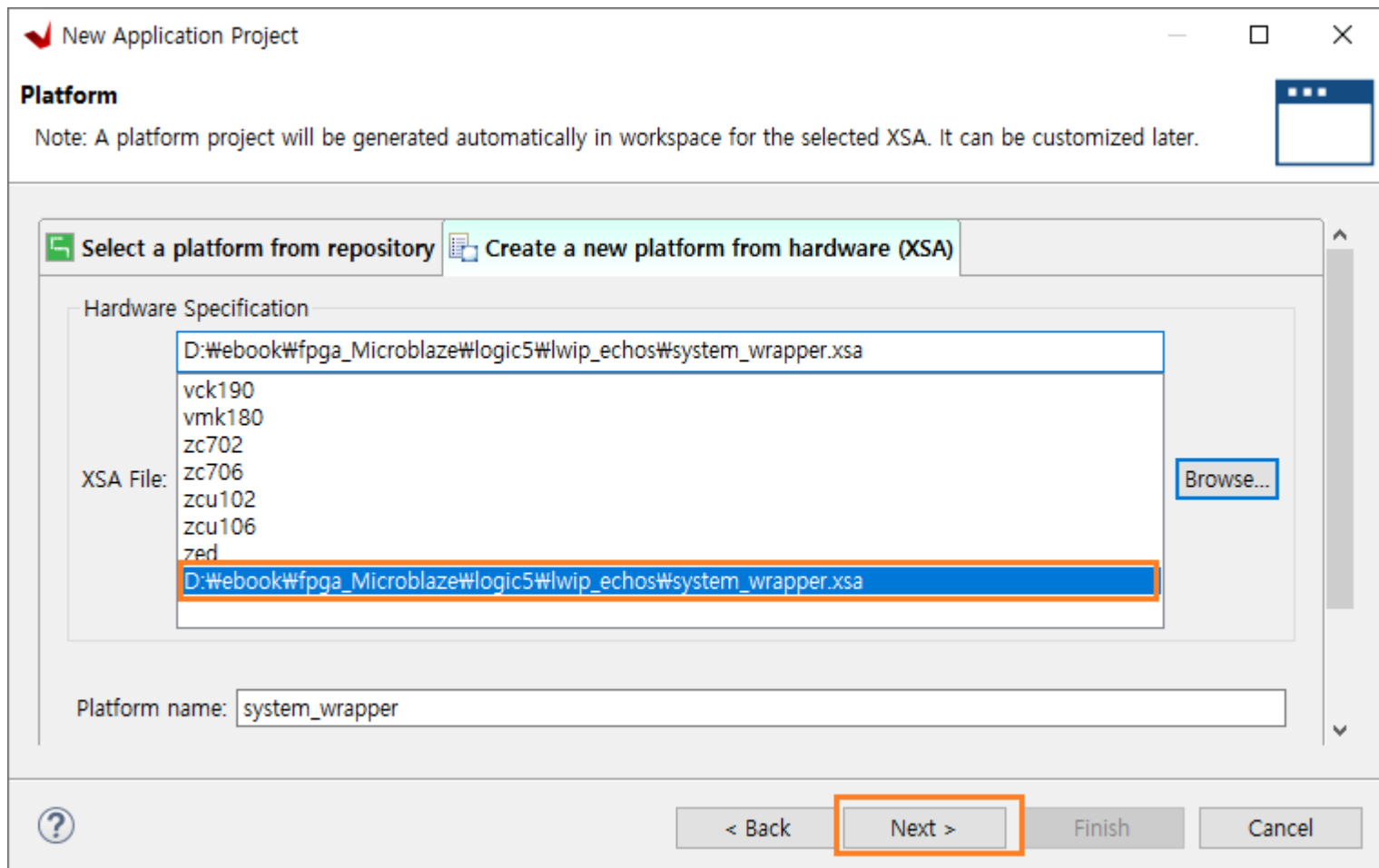
“Create a new platform from hardware (XSA)”를 클릭해서 platform project를 생성합니다.



Browse... 버튼을 클릭해서 vivado에서 생성했던 xsa 파일을 선택합니다.



Next 버튼을 클릭합니다.



Application Project를 생성합니다. 프로젝트 이름을 설정하고 Next 버튼은 클릭합니다. (lwip_echos_app 라고 합니다)

New Application Project

Application Project Details
Specify the application project name and its system project properties

Application project name:

System Project
Create a new system project for the application or select an existing one from the workspace

Select a system project
+ Create new...

System project details

System project name:

Target processor

Select target processor for the Application project.

Processor	Associated applications
microblaze_0	lwip_echos_app2

Show all processors in the hardware specification ☒

< Back **Next >** Finish Cancel

아래 내용을 확인하고 Next 버튼을 클릭합니다.

New Application Project

Domain
Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

Select a domain
+ Create new...

Domain details

Name:

Display Name:

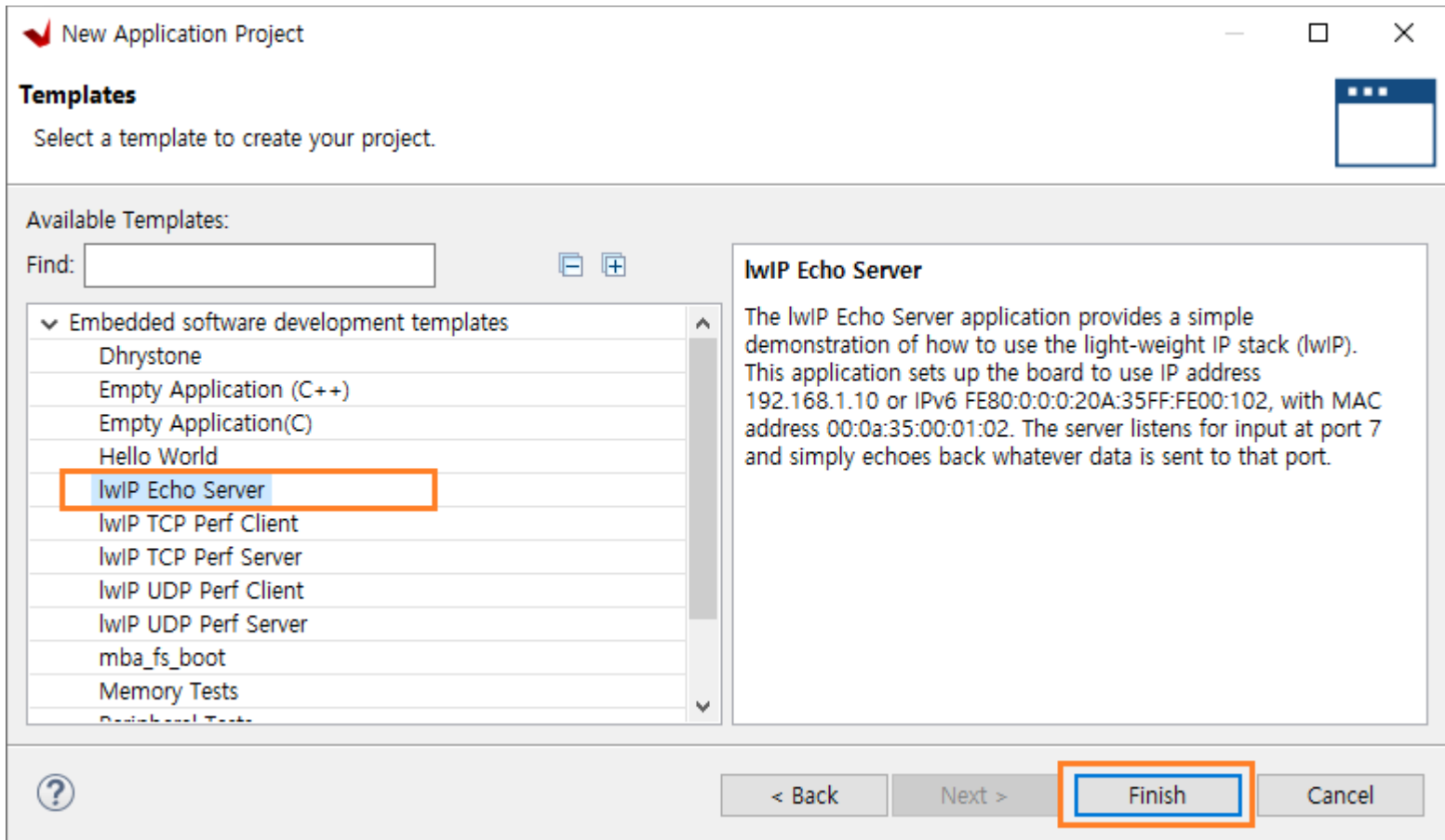
Operating System: standalone

Processor: microblaze_0

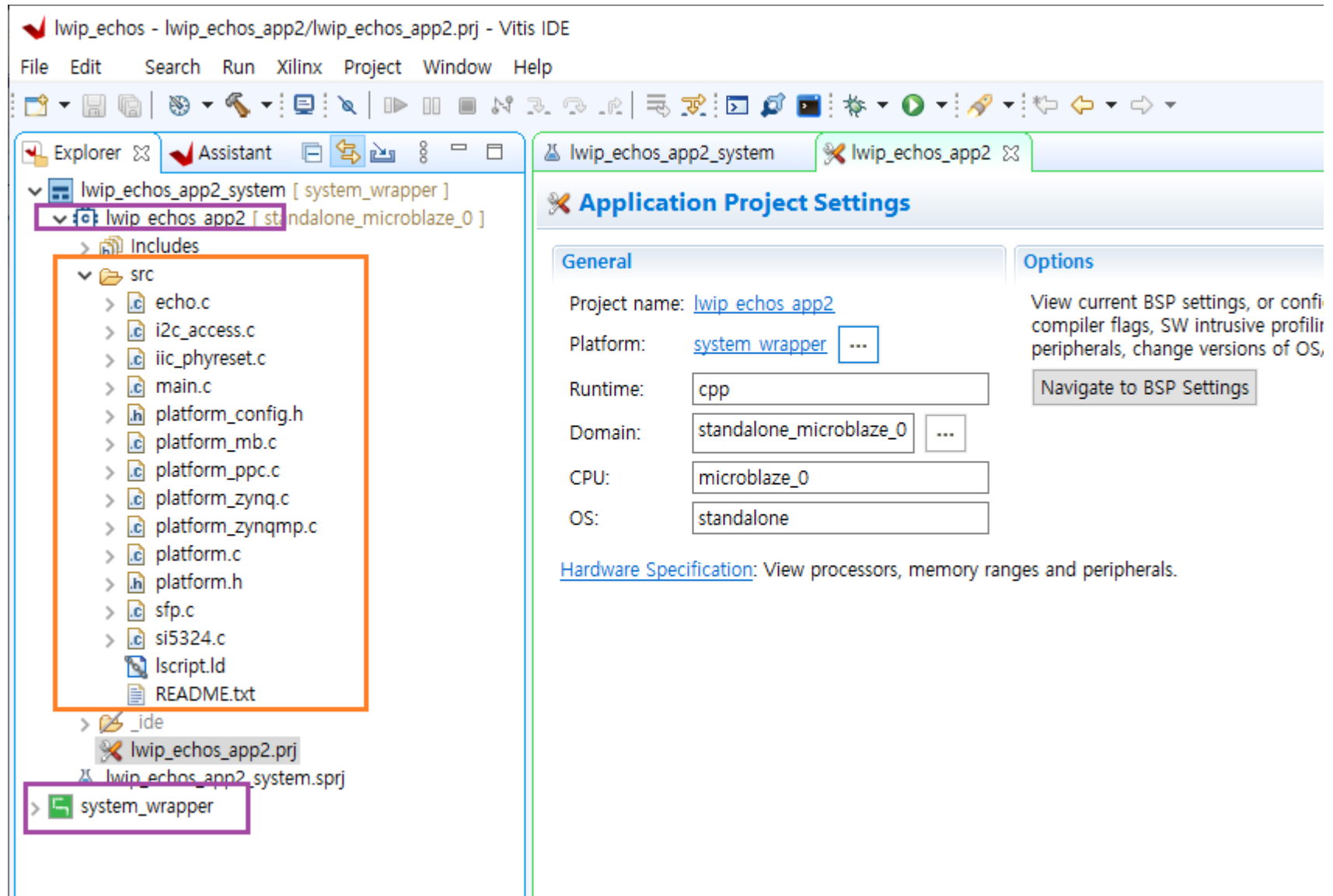
Architecture: 32-bit

< Back **Next >** Finish Cancel

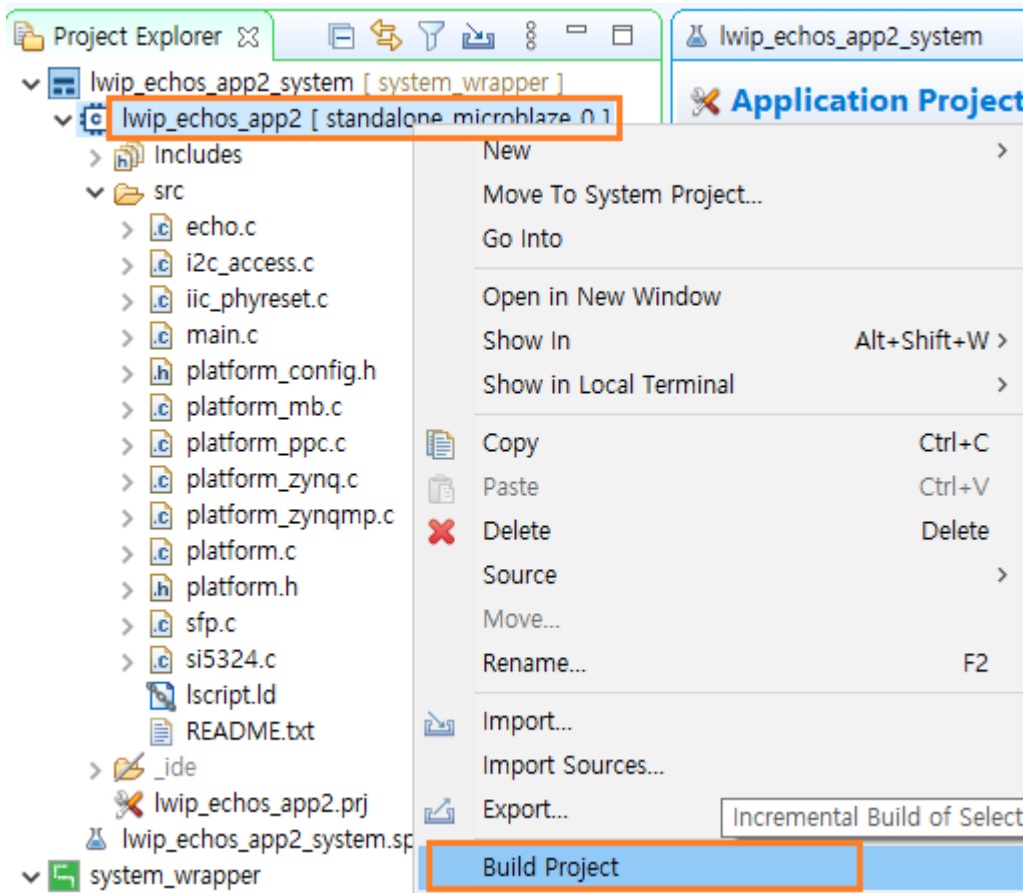
Template 에서 “lwIP Echo Server”를 선택합니다. Echo Server Template 외에 lwIP 관련된 Template들이 많이 있습니다. 기회가 되면 구현해 보시길 바랍니다. Finish 버튼을 클릭하면 프로젝트가 생성됩니다.



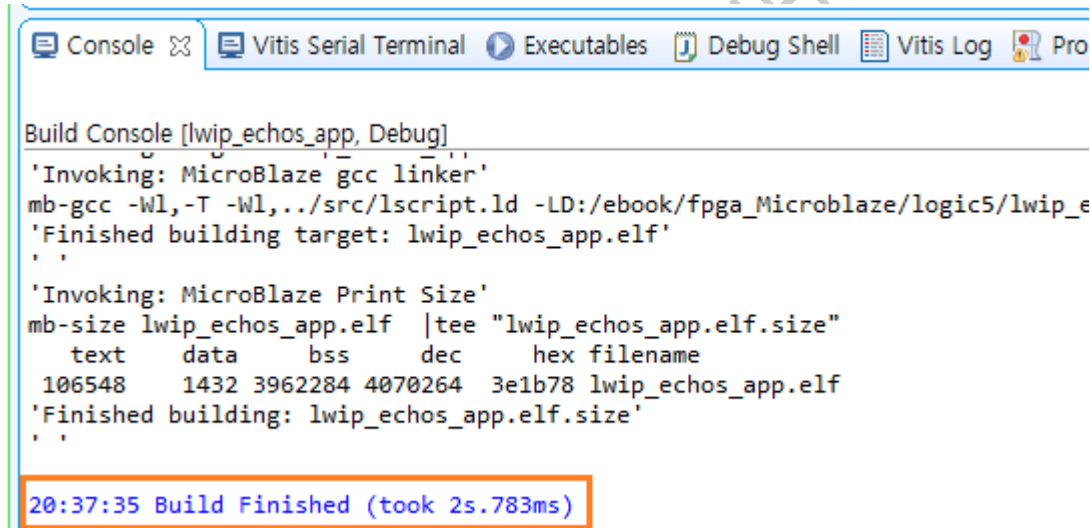
프로젝트가 생성되었습니다. src 폴더에는 프로그램 소스들이 있습니다. 프로그램 소스에 대한 내용은 이장 마지막에서 설명하고 동작확인을 위한 과정을 진행하도록 하겠습니다.



프로젝트를 Build 합니다. Explorer - lwip_echos_app_system - 우클릭 - Build Project 를 클릭합니다.



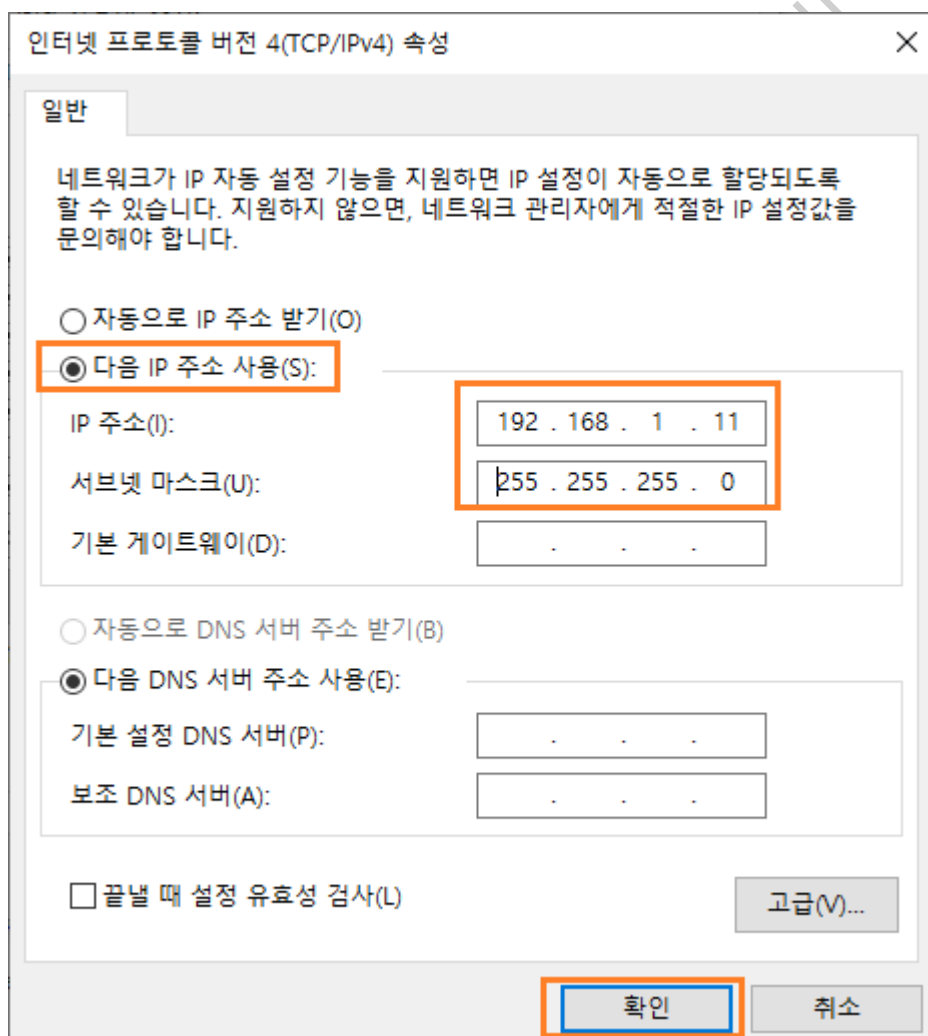
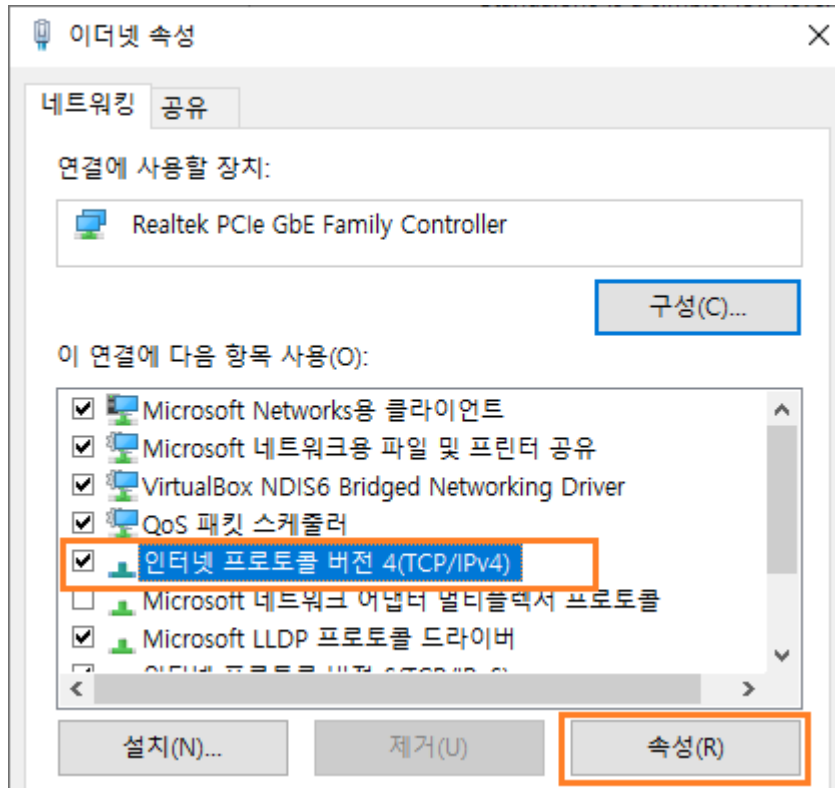
Console 윈도우에서 오류 없이 Build 가 완료되었음을 확인합니다.



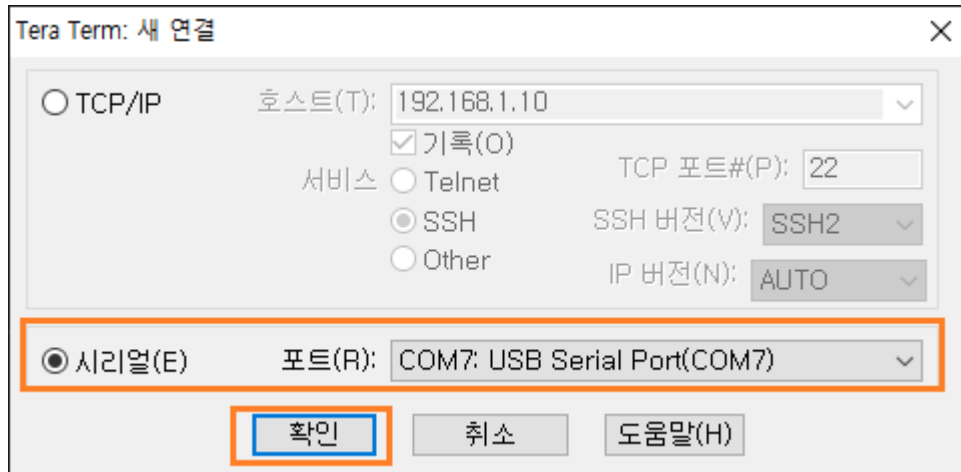
다음 장에서는 결과를 보드에 다운로드 하고, Tera Term을 이용하여 결과를 확인합니다.

7.2.5 결과 확인

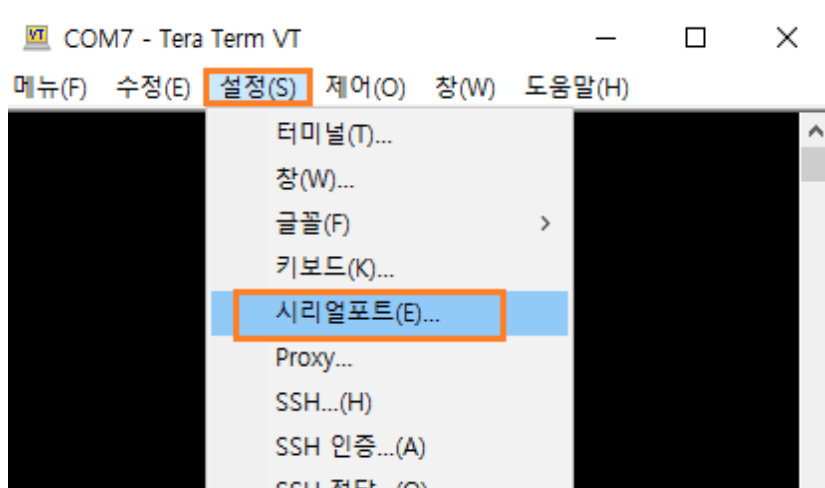
결과를 확인하기 위하여 PC와 Arty A7 보드를 직접 Ethernet Cable로 연결합니다. 또한 PC의 네트워크 정보를 아래와 같이 수정합니다. (네트워크 및 인터넷 설정 - 어댑터 옵션 변경 - 이더넷 더블 클릭 - 속성 - 인터넷 프로토콜 버전 4(TCP/IPv4) - 속성 클릭합니다) IP의 끝자리는 11 ~ 250 까지의 값을 사용합니다.



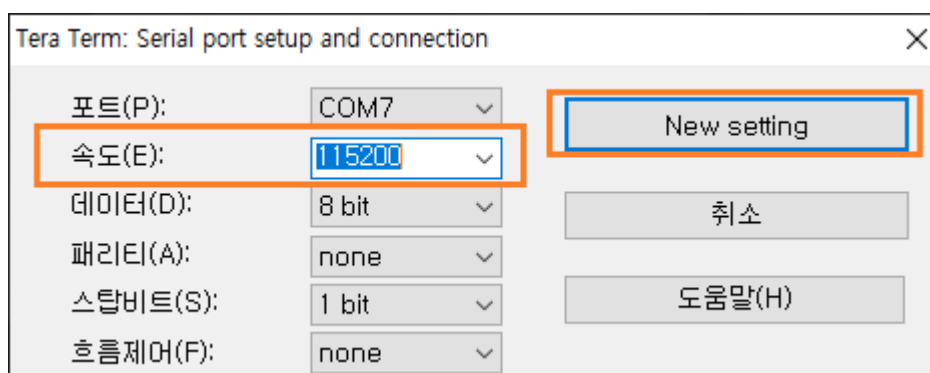
PC에서 TeraTerm을 2개를 실행합니다. TeraTerm1은 보드와 Serial로 연결합니다. 속도는 115200 bps 입니다.



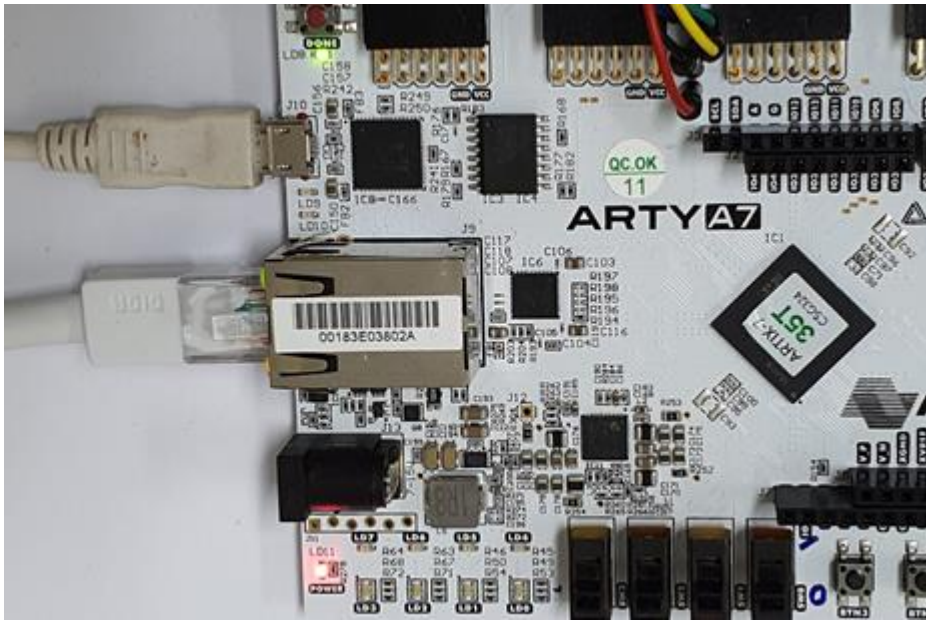
시리얼의 속도가 기본으로 9600으로 설정되어 있습니다. 115200으로 변경합니다. 설정 - 시리얼포트... 를 클릭합니다.



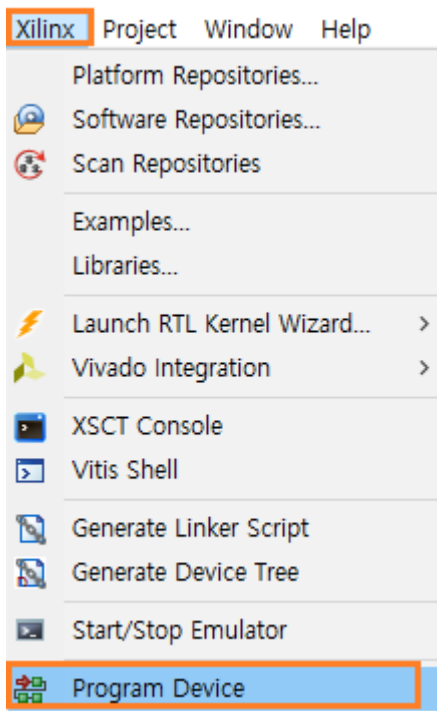
속도를 115200 으로 변경하고 “New setting”을 클릭합니다.



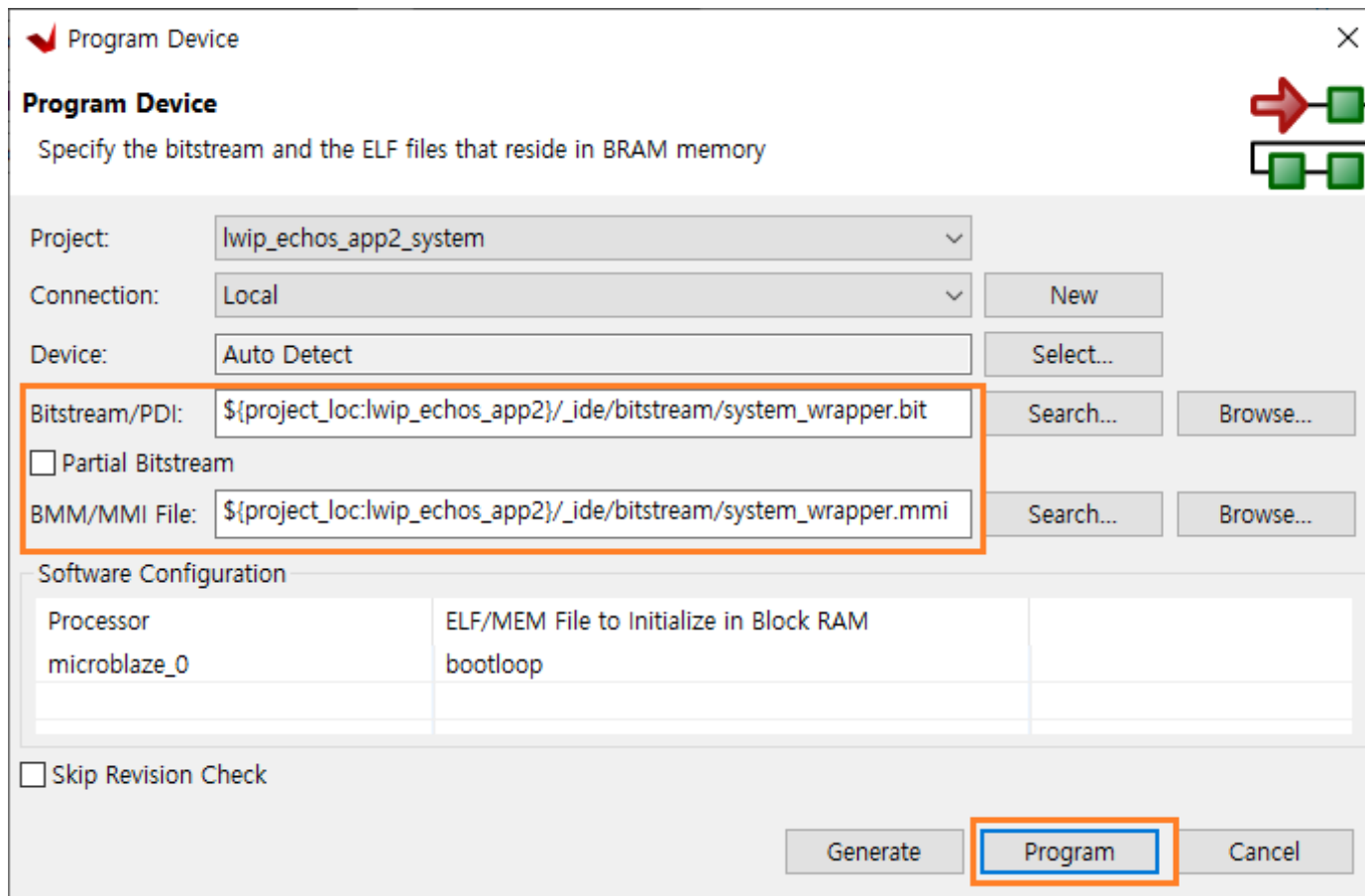
두번째 TeraTerm을 실행하기 전에 보드에 결과를 다운로드 합니다. 보드가 서버가 되고 PC가 Client로 동작하기 때문에 먼저 보드의 서버를 동작시켜야 합니다. 보드에 다운로드 하기 전에 보드에 USB 케이블을 연결해서 전원을 인가합니다.



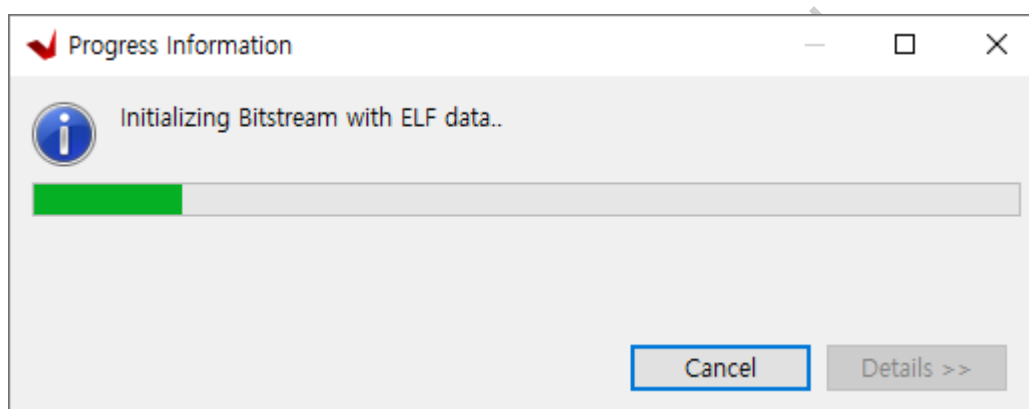
Vitis 에서 Xilinx - Program Device 를 클릭합니다.



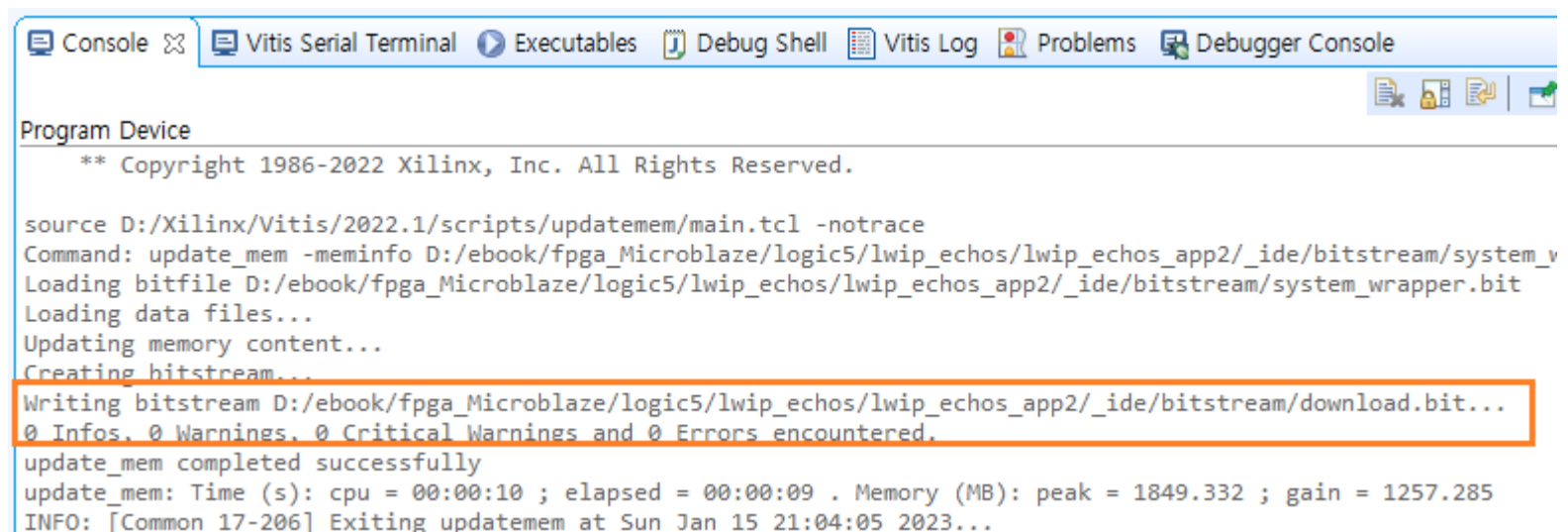
Program Device 윈도우에서 bit, mmi 파일위치를 확인하고 Program 버튼을 클릭합니다.



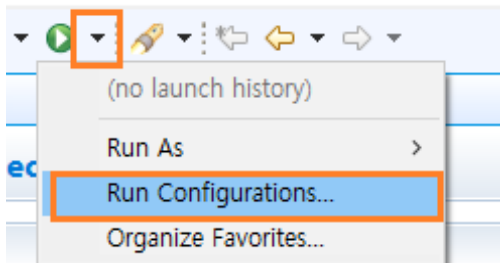
프로그램이 진행됩니다.



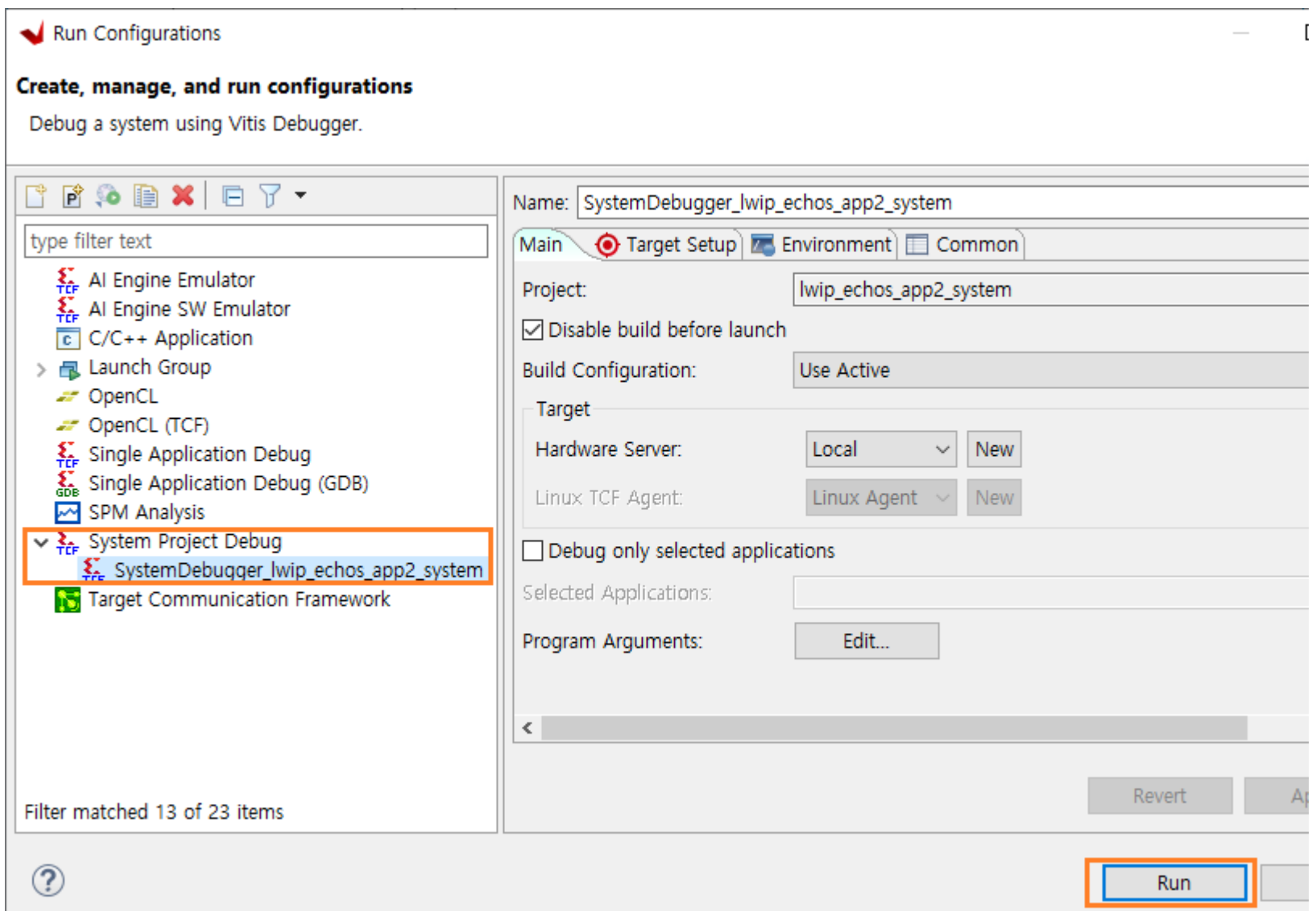
Console 창을 보면 download.bit 파일이 프로그램 되었음을 알 수 있습니다.



프로그램을 동작 시키기 위하여 Explorer 에서 lwip_echos_app 를 선택하고, Run () 아이콘 우측을 클릭합니다. Run Configurations...를 클릭합니다.

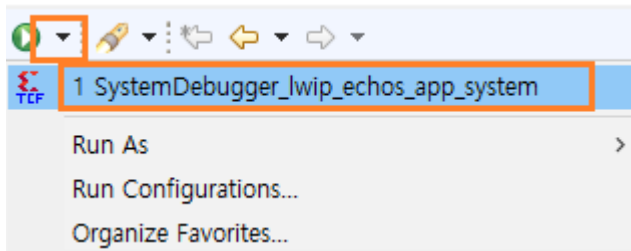


System Project Debug를 더블 클릭하면 SystemDebugger_lwip_echos_app_system 이 나타납니다. Run 버튼을 클릭합니다.

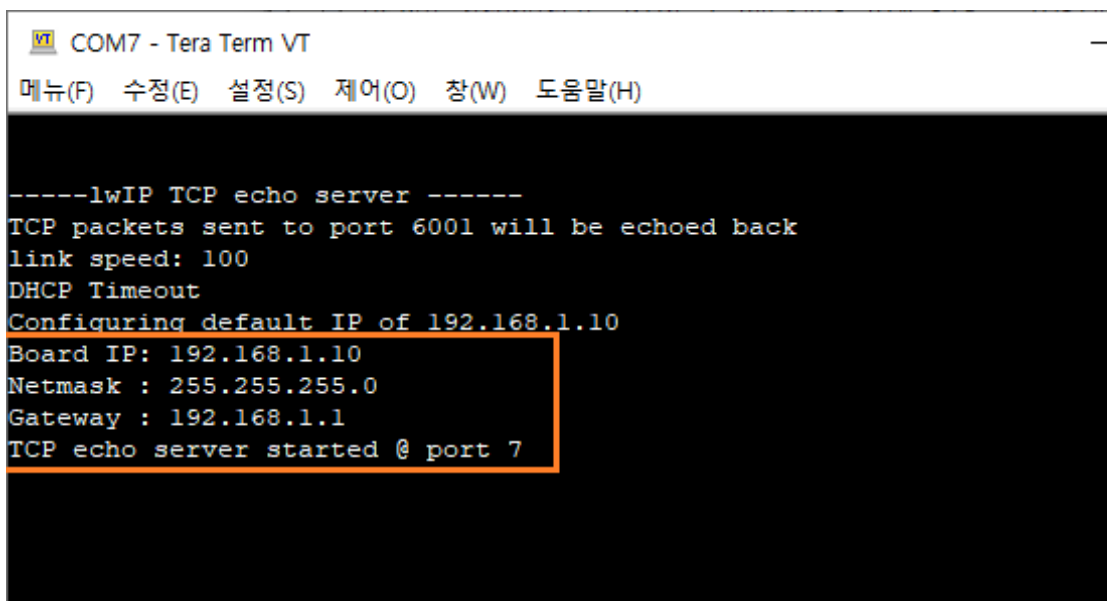


프로그램이 다운로드 되고 TeraTerm 창에 메시지가 나타납니다.

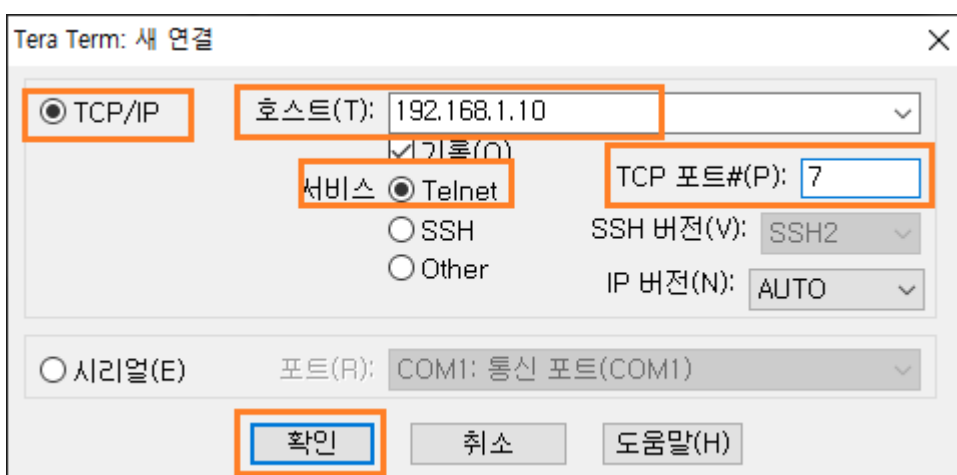
(다음부터는 Run Configuration...을 클릭할 필요없이, Xilinx - Program Device 후에 Run 아이콘 오른쪽을 클릭하여 나오는 “SystemDebugger_lwip_echos_app_system”을 클릭하면 바로 프로그램이 동작합니다)



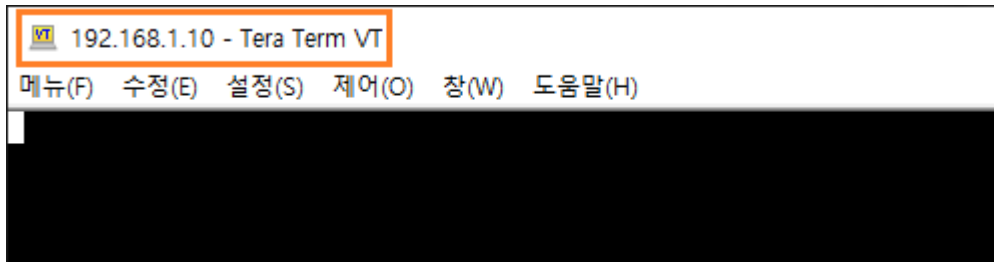
아래는 프로그램이 실행된 메시지를 보여줍니다. 보드가 TCP echo server (port : 7) 로 동작함을 나타냅니다.



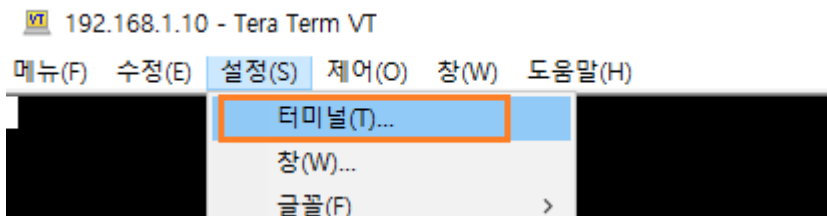
이제 Tera Term2을 실행해서 TCP echo server에 연결합니다. 아래와 같이 설정하고 확인 버튼을 클릭합니다.



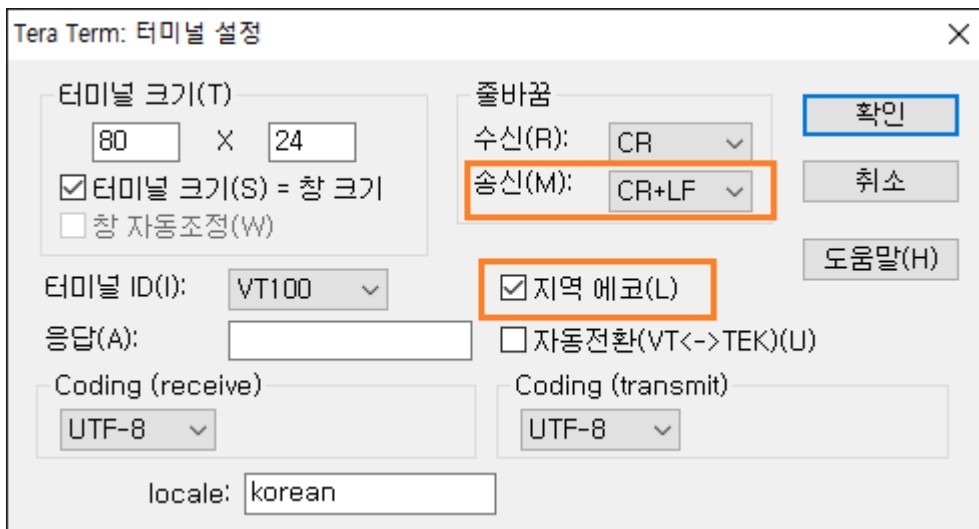
서버에 연결이 되었습니다.



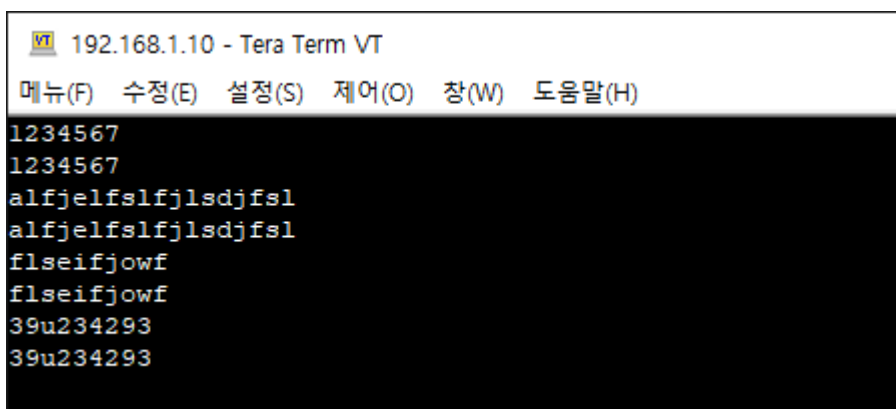
데이터를 전송하기 전에 설정을 바꾸겠습니다. 설정 - 터미널을 클릭합니다.



송신 : CR+LF, 지역 에코 : check 로 변경합니다. 데이터를 전송시 입력한 데이터를 표시하고 엔터키를 누르면 지금까지 입력한 데이터를 한꺼번에 전송한다는 의미입니다.



터미널에 임의의 문자를 입력하고 엔터키를 누르면 해당 문자가 다시 출력됩니다. 이는 보드에서 수신한 데이터를 다시 전송하기 때문입니다. 첫번째 라인은 송신한 데이터를 나타내고 두번째 라인은 보드(echo server)에서 보낸 데이터를 나타냅니다.



7.2.6 소스 분석

이번 장에서는 간단하게 소스를 분석합니다. TCP/IP 소스는 분량도 많고 본 강의의 범위를 벗어나기 때문에 전체적인 내용을 설명하지는 않고 대략적인 흐름을 이해하는 관점에서 설명하도록 하겠습니다. Xilinx에서 제공하는 문서도 많지가 않습니다. 자료실에 있는 “LightWeight IP Application Examples” (XAPP1026) 문서를 보시면 대략적인 내용이 있으니 참조하시길 바랍니다.

소스는 크게 아래의 3가지를 보면 됩니다.

- ✓ main.c
- ✓ platform.c
- ✓ echo.c

```

121 int main()
122 {
123     #if LWIP_IPV6==0
124         ip_addr_t ipaddr, netmask, gw;
125     #endif
126     /* the mac address of the board. this should be unique per board */
127     unsigned char mac_ethernet_address[] =
128     { 0x00, 0x0a, 0x35, 0x00, 0x01, 0x02 };
129
130     echo_netif = &server_netif;
131     #if defined (__arm__) && !defined (ARMR5)
132     #if XPAR_GIGE_PCS_PMA_SGMII_CORE_PRESENT == 1 || XPAR_GIGE_PCS_PMA_1000BASEX_CORE_PRESENT == 1
133         ProgramSi5324();
134         ProgramSfpPhy();
135     #endif
136     #endif
137
138     /* Define this board specific macro in order perform PHY reset on ZCU102 */
139     #ifdef XPS_BOARD_ZCU102
140         if(IicPhyReset()) {
141             xil_printf("Error performing PHY reset \n\r");
142             return -1;
143         }
144     #endif
145
146     init_platform();
147

```

- ✓ 라인 123 : IPV6은 사용하지 않습니다. LWIP_IPV6 은 opt.h 파일의 2377 라인에 0로 define 되어 있습니다.
- ✓ 라인 128 : mac address를 임의의 값으로 할당합니다. 만약 같은 여러 개의 보드를 사용할 때에는 각각 다른 address를 지정해야 합니다.
- ✓ 라인 131 : 서버로 동작하기 때문에 server network information (ip, netmask, gw)을 echo_netif 에 할당합니다.
- ✓ 라인 147 : platform() 을 초기화 합니다. 주로 HW 적인 부분입니다. 아래 int_platform() 함수를 참조하세요.

```

174 void init_platform()
175 {
176     enable_caches();
177
178     #ifdef STDOUT_IS_16550
179         XUartNs550_SetBaud(STDOUT_BASEADDR, XPAR_XUARTNS550_CLOCK_HZ, 9600);
180         XUartNs550_SetLineControlReg(STDOUT_BASEADDR, XUN_LCR_8_DATA_BITS);
181     #endif
182
183     platform_setup_interrupts();
184 }

149 #if LWIP_IPV6==0
150 #if LWIP_DHCP==1
151     ipaddr.addr = 0;
152     gw.addr = 0;
153     netmask.addr = 0;
154 #else
155     /* initialize IP addresses to be used */
156     IP4_ADDR(&ipaddr, 192, 168, 1, 10);
157     IP4_ADDR(&netmask, 255, 255, 255, 0);
158     IP4_ADDR(&gw, 192, 168, 1, 1);
159 #endif
160 #endif
161     print_app_header();
162
163     lwip_init();
164
165 #if (LWIP_IPV6 == 0)
166     /* Add network interface to the netif_list, and set it as default */
167     if (!xemac_add(echo_netif, &ipaddr, &netmask,
168                  &gw, mac_ethernet_address,
169                  PLATFORM_EMAC_BASEADDR)) {
170         xil_printf("Error adding N/W interface\n\r");
171         return -1;
172     }
173 #else
174     /* Add network interface to the netif_list, and set it as default */
175     if (!xemac_add(echo_netif, NULL, NULL, NULL, mac_ethernet_address,
176                  PLATFORM_EMAC_BASEADDR)) {
177         xil_printf("Error adding N/W interface\n\r");
178         return -1;
179     }
180     echo_netif->ip6_autoconfig_enabled = 1;
181
182     netif_create_ip6_linklocal_address(echo_netif, 1);
183     netif_ip6_addr_set_state(echo_netif, 0, IP6_ADDR_VALID);
184
185     print_ip6("\n\rBoard IPv6 address ", &echo_netif->ip6_addr[0].u_addr.ip6);
186
187 #endif
188     netif_set_default(echo_netif);
189
190     /* now enable interrupts */
191     platform_enable_interrups();
192
193     /* specify that the network if is up */
194     netif_set_up(echo_netif);

```

- ✓ 라인 150 - 160 : DHCP를 사용할 때, ip, gw, netmask 을 0로 초기화합니다. DHCP를 사용하지 않을 때에는 ip : 192.168.1.10, netmask : 255.255.255.0, gw : 192.168.1.1 로 초기화 합니다.
- ✓ 라인 161 : “-----lwIP TCP echo server -----”을 화면에 표시합니다.
- ✓ 라인 163 : lwip를 초기화 합니다.
- ✓ 라인 165 - 187 : network interface를 추가합니다.
- ✓ 라인 191 : interrupt를 enable 합니다.

```

196 #if (LWIP_IPV6 == 0)
197 #if (LWIP_DHCP==1)
198     /* Create a new DHCP client for this interface.
199     * Note: you must call dhcp_fine_tmr() and dhcp_coarse_tmr() at
200     * the predefined regular intervals after starting the client.
201     */
202     dhcp_start(echo_netif);
203     dhcp_timeoutcnt = 24;
204
205     while(((echo_netif->ip_addr.addr) == 0) && (dhcp_timeoutcnt > 0))
206         xemacif_input(echo_netif);
207
208     if (dhcp_timeoutcnt <= 0) {
209         if ((echo_netif->ip_addr.addr) == 0) {
210             xil_printf("DHCP Timeout\r\n");
211             xil_printf("Configuring default IP of 192.168.1.10\r\n");
212             IP4_ADDR(&(echo_netif->ip_addr), 192, 168, 1, 10);
213             IP4_ADDR(&(echo_netif->netmask), 255, 255, 255, 0);
214             IP4_ADDR(&(echo_netif->gw), 192, 168, 1, 1);
215         }
216     }
217
218     ipaddr.addr = echo_netif->ip_addr.addr;
219     gw.addr = echo_netif->gw.addr;
220     netmask.addr = echo_netif->netmask.addr;
221 #endif
222
223     print_ip_settings(&ipaddr, &netmask, &gw);
224
225 #endif
226     /* start the application (web server, rxttest, txttest, etc..) */
227     start_application();

```

- ✓ 라인 196 - 221 : dhcp 서버로 부터 IP를 할당받습니다. 만약 6초 (250ms * 24) 동안 IP를 할당 받지 못하면 "192.168.1.10"으로 고정합니다.
- ✓ 라인 223 : ip, netmask, gw 값을 출력합니다.
- ✓ 라인 227 : 설정된 network 정보를 이용하여 echo server를 start 합니다.

아래 코드를 참조하세요.

```

96 int start_application()
97 {
98     struct tcp_pcb *pcb;
99     err_t err;
100     unsigned port = 7;
101
102     /* create new TCP PCB structure */
103     pcb = tcp_new_ip_type(IPADDR_TYPE_ANY);
104     if (!pcb) {
105         xil_printf("Error creating PCB. Out of Memory\n\r");
106         return -1;
107     }
108
109     /* bind to specified @port */
110     err = tcp_bind(pcb, IP_ANY_TYPE, port);
111     if (err != ERR_OK) {
112         xil_printf("Unable to bind to port %d: err = %d\n\r", port, err);
113         return -2;
114     }
115
116     /* we do not need any arguments to callback functions */
117     tcp_arg(pcb, NULL);
118
119     /* listen for connections */
120     pcb = tcp_listen(pcb);
121     if (!pcb) {
122         xil_printf("Out of memory while tcp_listen\n\r");
123         return -3;
124     }
125
126     /* specify callback to use for incoming connections */
127     tcp_accept(pcb, accept_callback);
128
129     xil_printf("TCP echo server started @ port %d\n\r", port);
130
131     return 0;
132 }

```

- ✓ 라인 103 : tcp pcb를 생성합니다.
- ✓ 라인 110 : 생성된 pcb를 port 7 에 bind 합니다.
- ✓ 라인 120 - 124 : client의 접속을 기다립니다.
- ✓ 라인 127 : client의 접속을 accept 할 때 호출하는 call back 함수 accept_callback 를 지정합니다.

아래는 accept_callback() 함수입니다.

```

78 err_t accept_callback(void *arg, struct tcp_pcb *newpcb, err_t err)
79 {
80     static int connection = 1;
81
82     /* set the receive callback for this connection */
83     tcp_recv(newpcb, recv_callback);
84
85     /* just use an integer number indicating the connection id as the
86        callback argument */
87     tcp_arg(newpcb, (void*)(UINTPTR)connection);
88
89     /* increment for subsequent accepted connections */
90     connection++;
91
92     return ERR_OK;
93 }

```

- ✓ 라인 83 : 수신 데이터가 있을 때 recv_callback 함수가 호출되도록 설정합니다.

아래는 recved_callback() 함수입니다.

```

52 err_t recv_callback(void *arg, struct tcp_pcb *tpcb,
53                    struct pbuf *p, err_t err)
54 {
55     /* do not read the packet if we are not in ESTABLISHED state */
56     if (!p) {
57         tcp_close(tpcb);
58         tcp_recv(tpcb, NULL);
59         return ERR_OK;
60     }
61
62     /* indicate that the packet has been received */
63     tcp_recved(tpcb, p->len);
64
65     /* echo back the payload */
66     /* in this case, we assume that the payload is < TCP_SND_BUF */
67     if (tcp_sndbuf(tpcb) > p->len) {
68         err = tcp_write(tpcb, p->payload, p->len, 1);
69     } else
70         xil_printf("no space in tcp_sndbuf\n\r");
71
72     /* free the received pbuf */
73     pbuf_free(p);
74
75     return ERR_OK;
76 }

```

- ✓ 라인 67 - 70 : 수신한 데이터를 다시 전송(echo back) 합니다.
- ✓ lwip를 이용한 Application을 개발할 때, 이 부분을 사용하면 됩니다. PC(Client)로 부터 수신되는 데이터를 받아서 처리하고, PC(Client)로 전송할 일이 있으면, tcp_write() 함수를 사용하면 됩니다 !!


```

229  /* receive and process packets */
230  while (1) {
231      if (TcpFastTmrFlag) {
232          tcp_fasttmr();
233          TcpFastTmrFlag = 0;
234      }
235      if (TcpSlowTmrFlag) {
236          tcp_slowtmr();
237          TcpSlowTmrFlag = 0;
238      }
239      xemacif_input(echo_netif);
240      transfer_data();
241  }
242
243  /* never reached */
244  cleanup_platform();
245
246  return 0;
247 }

```

✓ 라인 231 - 234 : 250ms 마다 TcpFastTmrFlag가 1이 되어, tcp_fasttmr() 함수가 실행됩니다.

✓ 라인 235 - 238 : 500ms 마다 TcpSlowTmrFlag가 1이 되어, tcp_slowtmr() 함수가 실행됩니다.

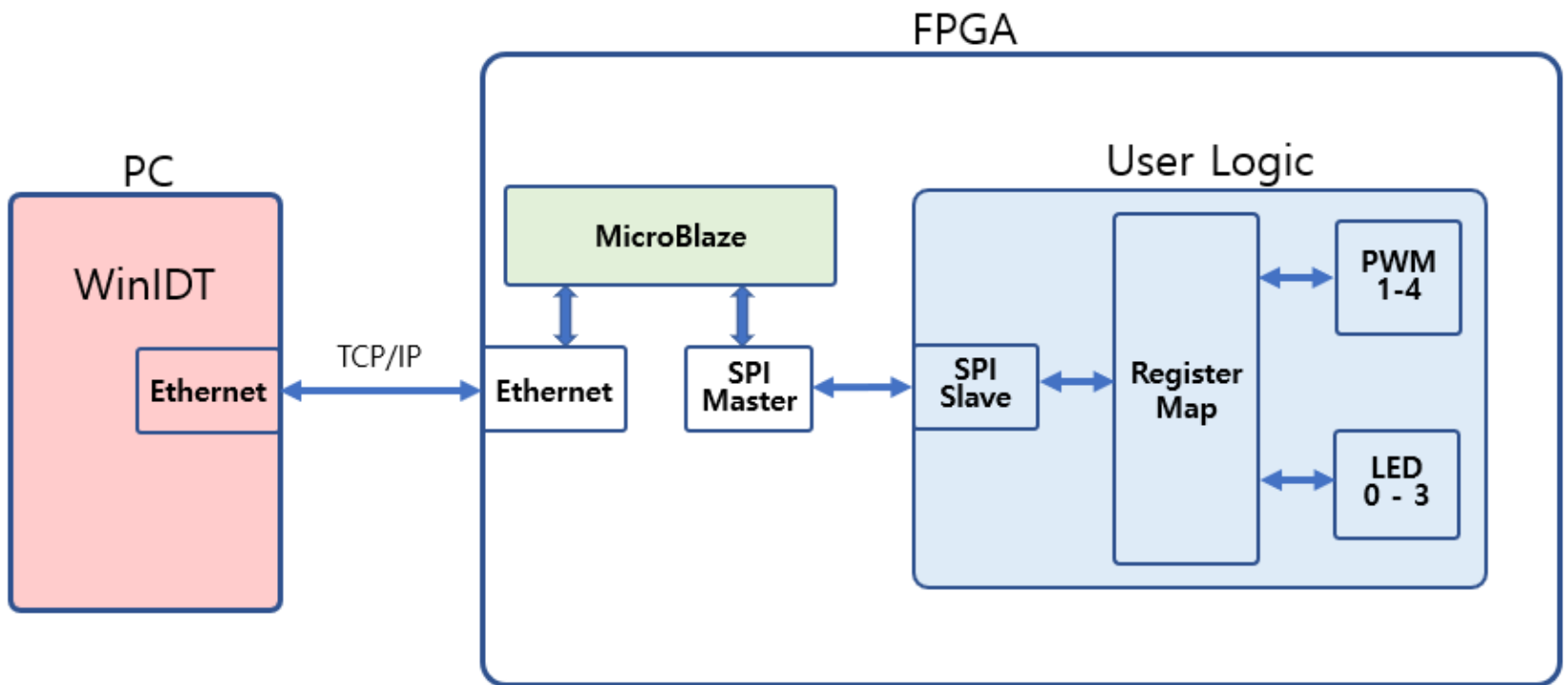
lwip 프로그램 소스에 대한 내용은 아래 링크에 좀 더 자세히 설명되어 있습니다. 아래 링크는 STM32 HAL 드라이버에서 제공하는 LwIP TCH Echo Server에 대한 소스를 설명하고 있으니 참조하시길 바랍니다.

참조 링크 : <https://blog.naver.com/eziya76/221854875861>

다음 장에서는 PC와 TCP/IP 통신을 통하여서 보드의 LED를 제어하는 것을 구현하도록 하겠습니다.

8. lwIP 활용

이번장에서는 앞에서 구현한 lwIP를 이용하여 PC와 TCP/IP 통신을 통하여 보드의 LED를 제어하는 것을 구현하도록 하겠습니다. 6장에서는 UART 통신을 통하여서 PC에서 PWM을 제어하는 것을 구현하였는데, 이번장에서는 TCP/IP 통신을 통하여 PC에서 보드의 LED를 제어하는 것을 구현합니다. 아래는 System Block을 보여줍니다.



8.1 HW Design

vivado 2022.1에서 IP INTEGRATOR - Open Block Design을 클릭해서 7장에서 구현했던 Block 을 open 합니다.

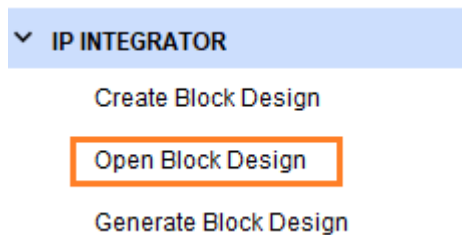
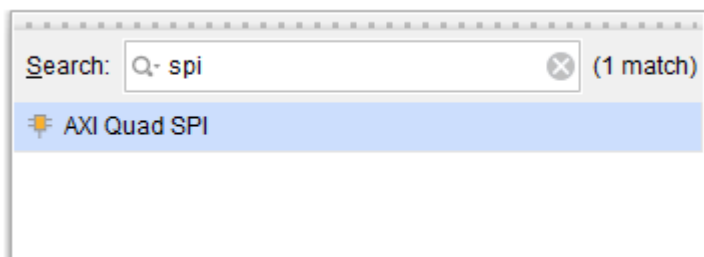
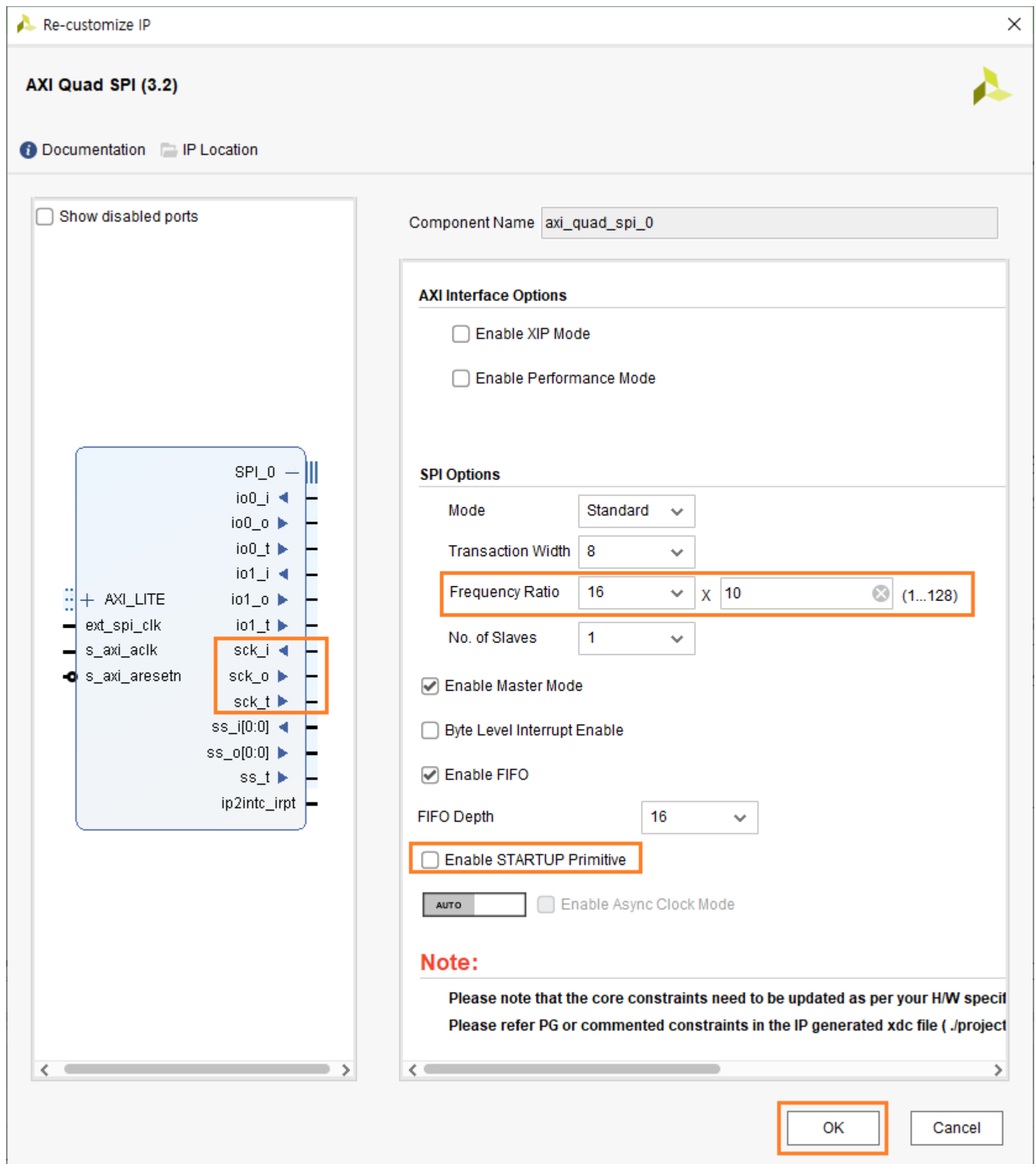


Diagram 에서 Add IP (+) 를 클릭하고, Search : spi 를 입력하고 AXI Quad SPI 를 더블클릭해서 SPI를 추가합니다.

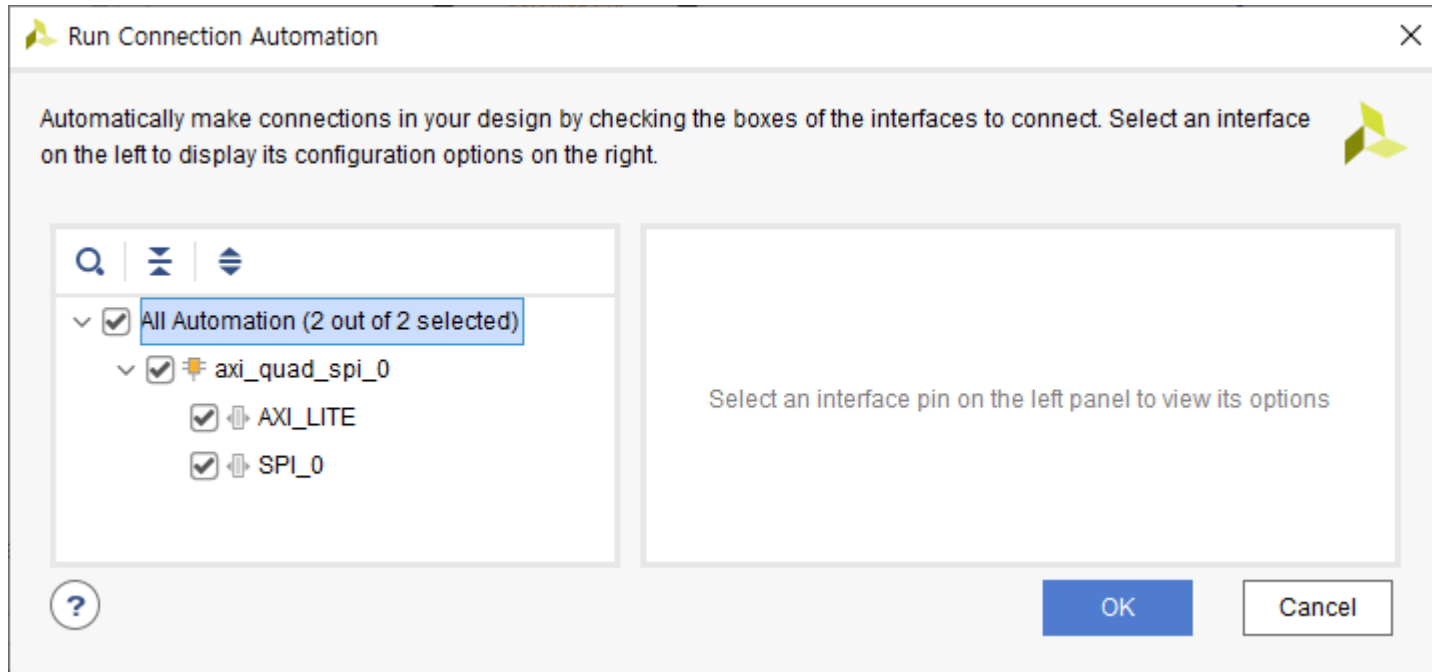


추가된 axi_quad_spi_0를 더블 클릭해서 속성을 아래와 같이 설정합니다. Frequency Ratio는 16×10 ($83\text{M} / 160 = 0.52\text{M}$)로 설정하고 Enable STARTUP Primitive를 uncheck 해야 sck 신호가 나옵니다. OK를 클릭합니다.

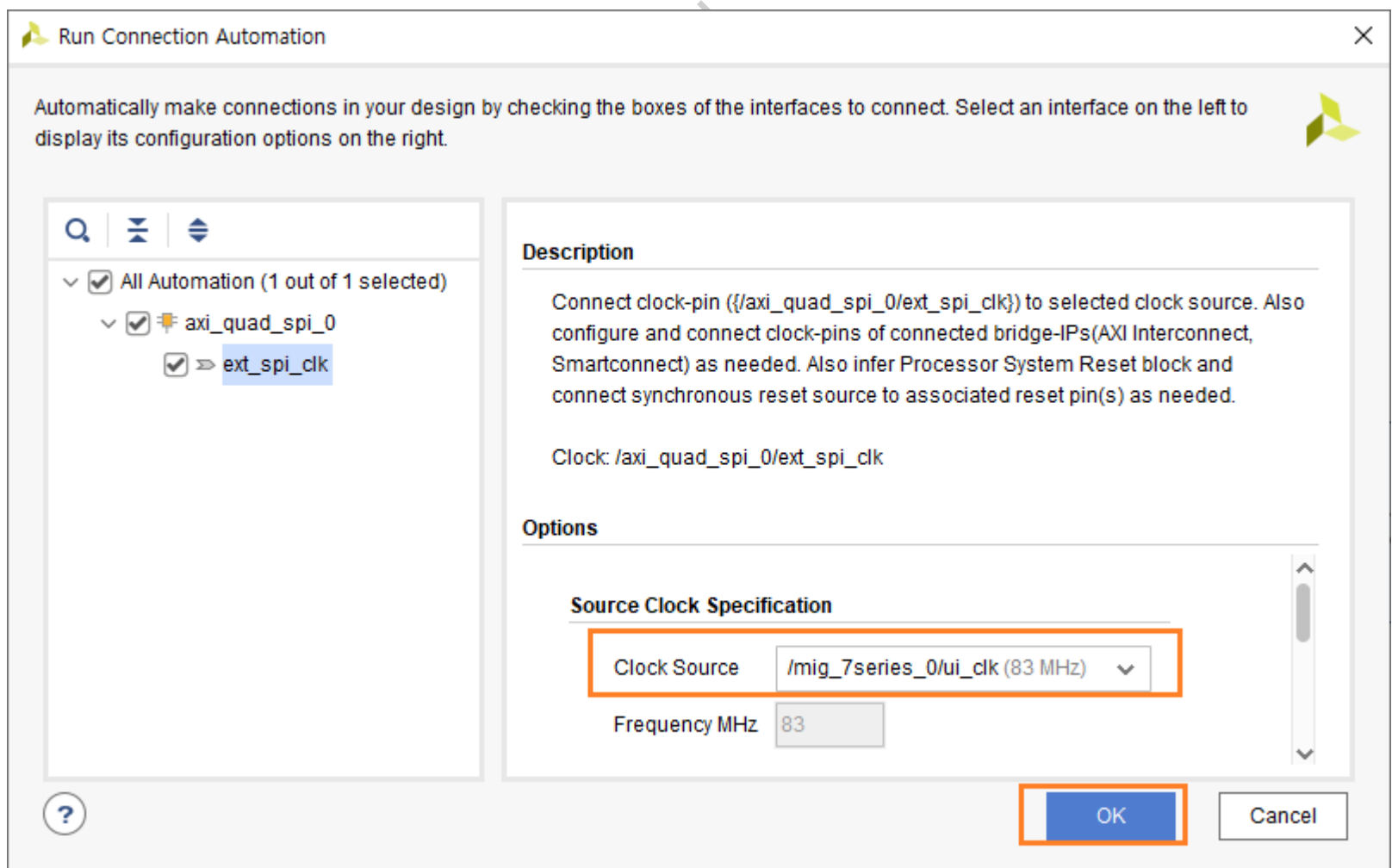


Run Connection Automation을 클릭해서 각 신호들을 연결해 줍니다.

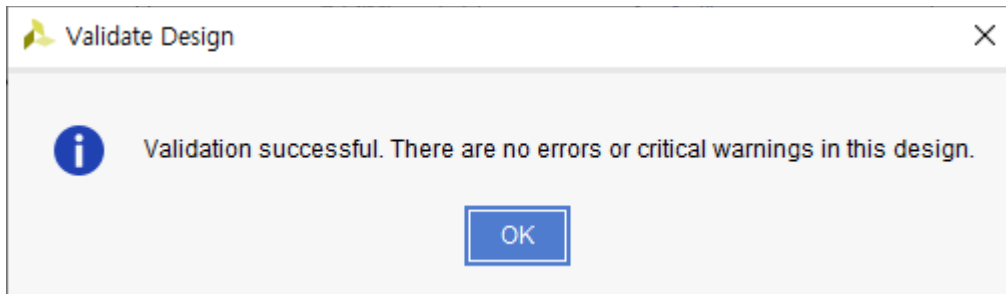
“All Automation은 Check 한 후 OK를 클릭합니다.



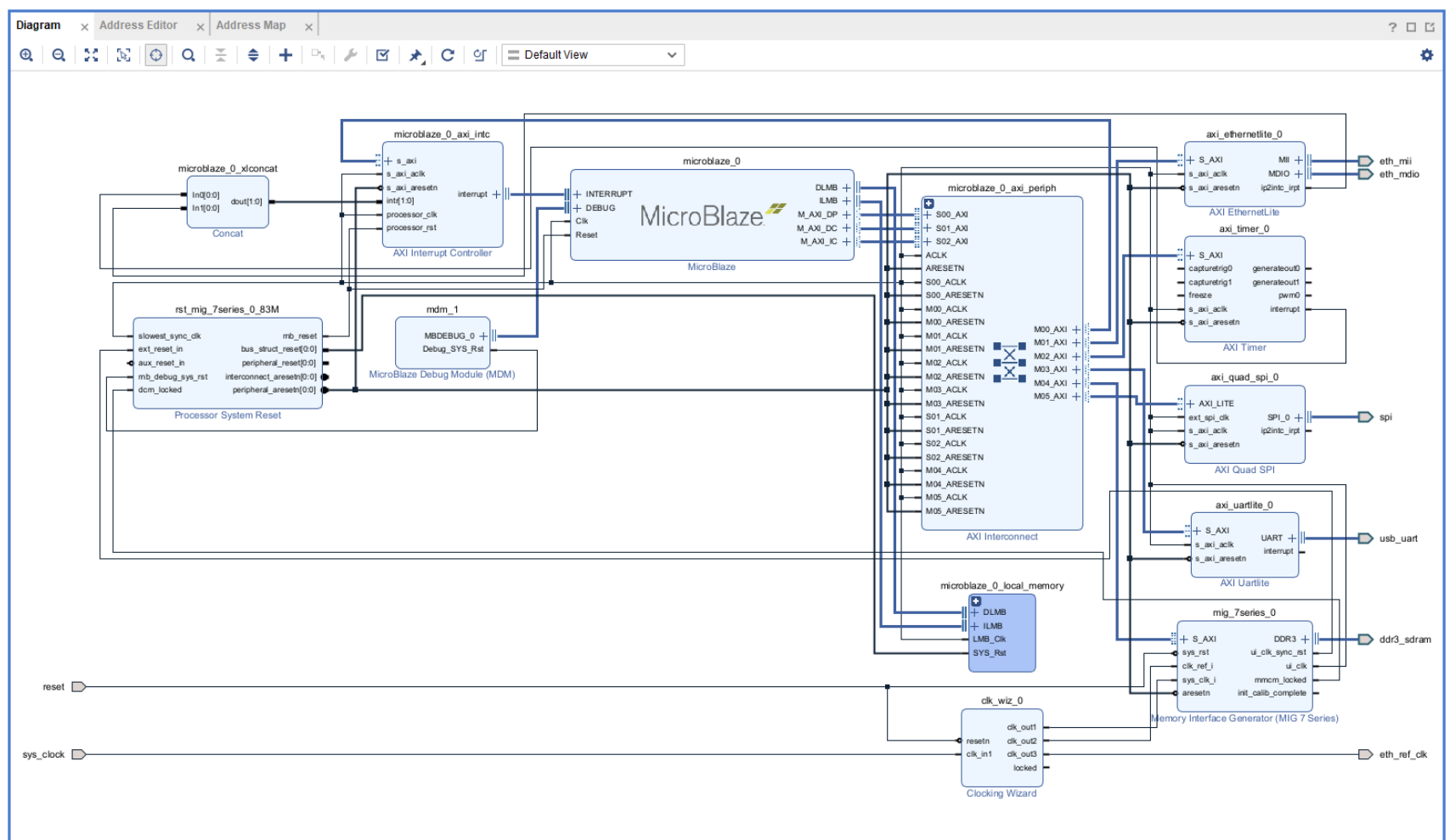
Run Connection Automation을 다시한번 클릭합니다. Clock Source : /mig_7series_0/ui_clk(83 Mhz)로 선택하고 OK를 클릭합니다.



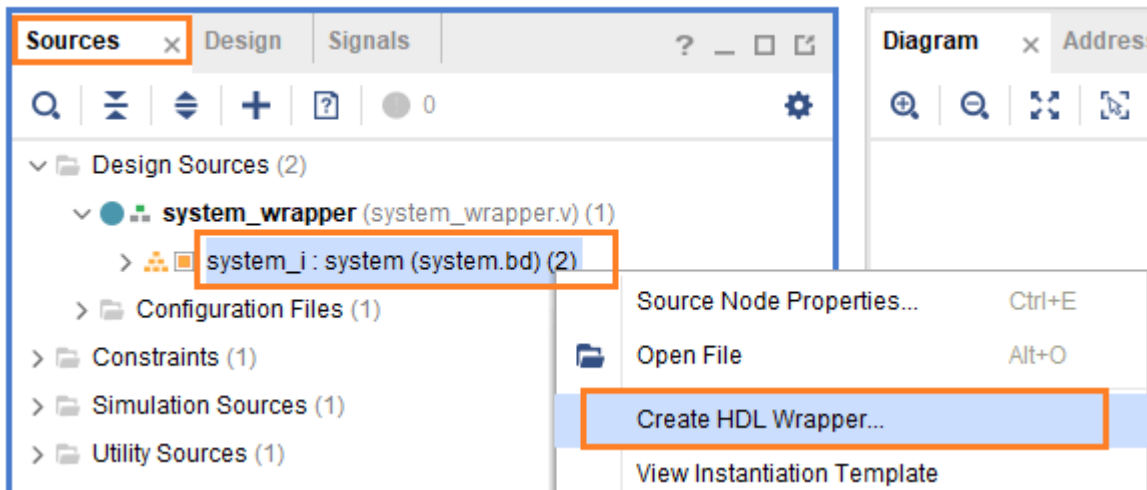
spi_rtl_o 핀 이름은 spi 으로 변경 후, Validate Design 아이콘을 클릭해서 오류를 확인합니다. 오류가 없음을 확인하고 OK 를 클릭합니다.



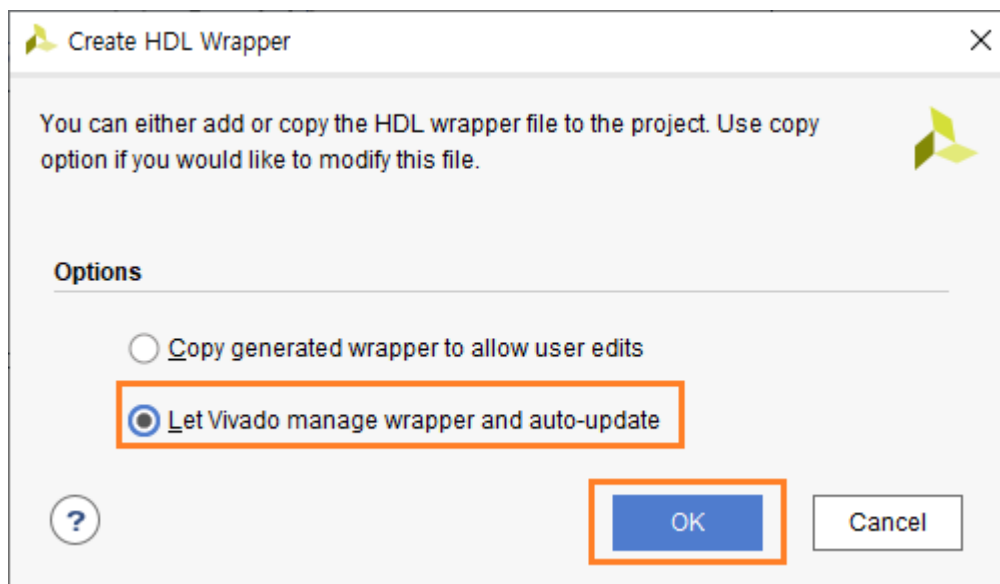
Regenerate Layout 아이콘을 클릭해서 layout을 보기좋게 정리합니다. 아래 그림은 최종적인 Diagram을 보여줍니다. (Diagram 은 다운받은 자료에 png 파일로 확인하실 수 있습니다.)



Sources 탭에서 Design Source - system_wrapper - system_i 을 선택하고 우클릭해서 “Create HDL Wrapper...”를 클릭합니다.



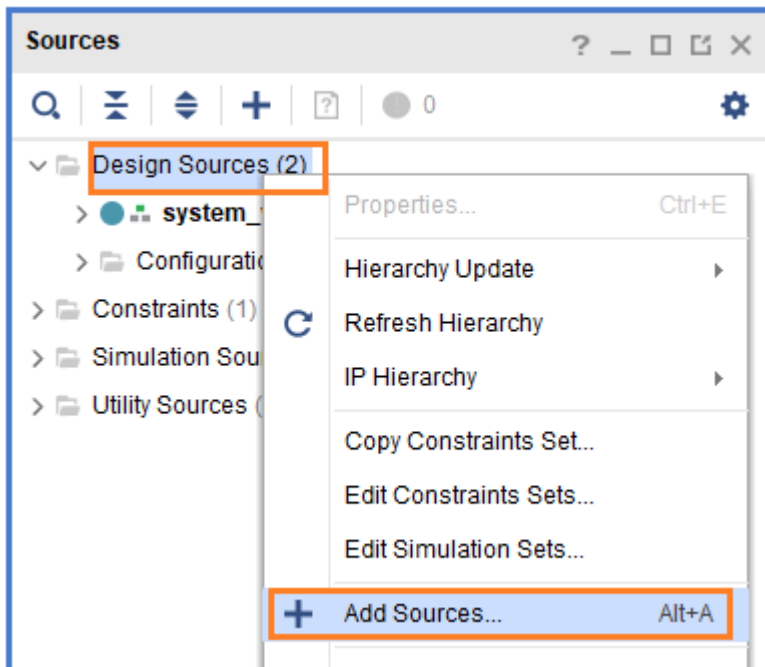
“Let Vivado manage wrapper and auto-update”를 선택하고 OK를 클릭합니다.



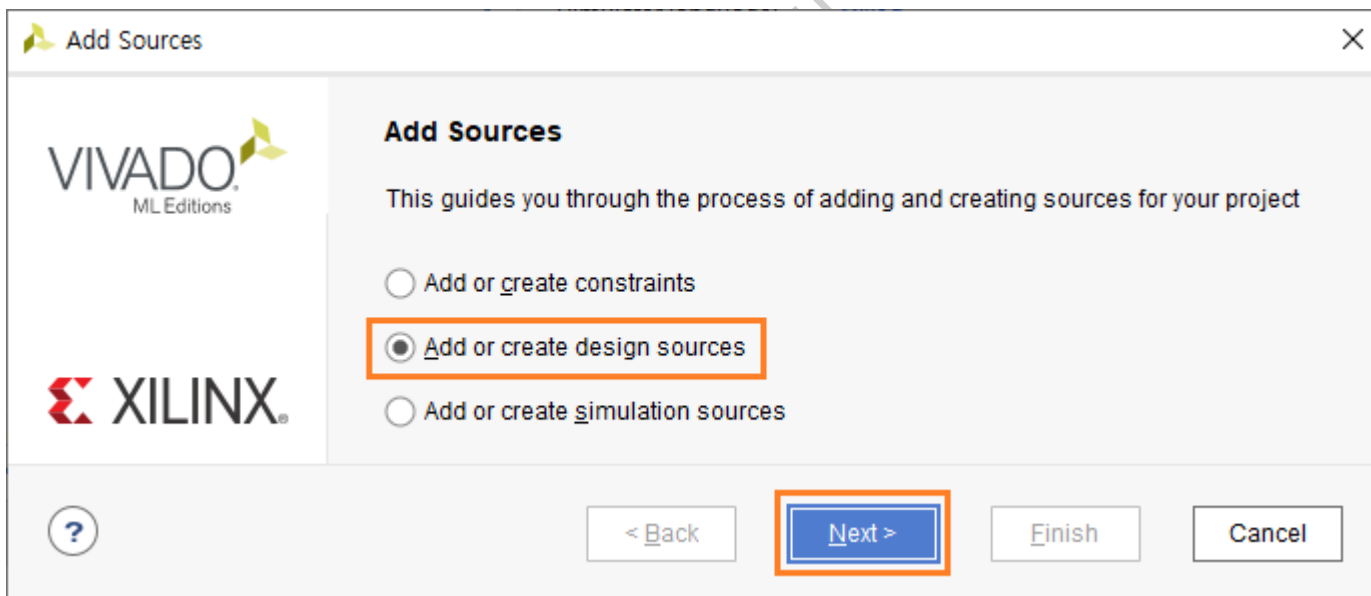
이제 system_wrapper.v 파일을 참조하여 User Logic을 추가하도록 하겠습니다.

8.2 User Logic 구현

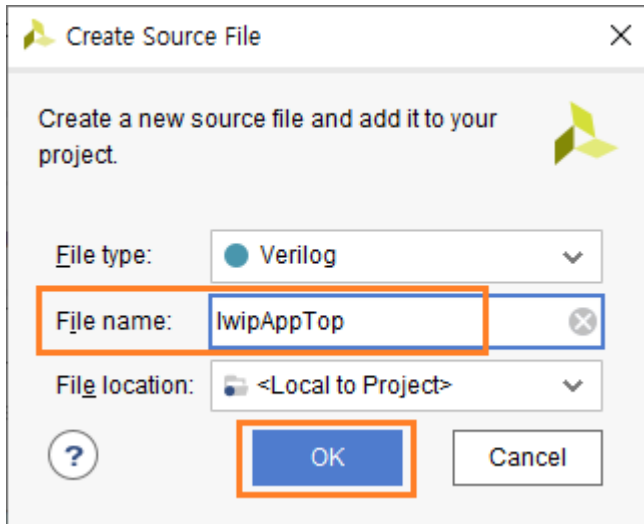
이번장에서는 User Logic을 추가합니다. 6장에서 진행했던 내용과 거의 동일합니다. Top 모듈을 추가합니다. Design Sources - 우클릭 - Add Sources... 클릭합니다.



“Add or create design sources”를 선택하고 Next를 클릭합니다.



Create File 클릭해서 이름을 “lwipAppTop”으로 지정하도록 하겠습니다. OK를 클릭합니다.



추가된 lwipAppTop 모듈을 “Set as Top” 메뉴를 선택해서 Top Module로 지정합니다. 6장에서 사용했던 3개의 파일을 복사해서 “lwip_echos.srcs\srcs_1\new” 폴더에 복사합니다. 6장에서 사용했던 구조를 그대로 사용합니다. Design Sources - 우클릭 - Add Sources... 클릭해서 3개의 파일을 추가합니다. 기본적인 환경은 설정되었습니다. 이제 소스를 수정하도록 하겠습니다.

먼저 spi_slave 모듈에 LED를 제어하는 코드를 추가합니다. 수정되는 부분만 설명하도록 하겠습니다. 자세한 내용은 6장을 참조하시길 바랍니다.

```

32  `define      rLED_FLAG0      8'h30
33  `define      rLED_FLAG1      8'h31
34  `define      rLED_FLAG2      8'h32
35  `define      rLED_FLAG3      8'h33

```

- ✓ 라인 32 - 35 : 보드에는 RGB LED0 ~ 3이 장착되어 있습니다. 각각의 LED를 제어하기 위한 SPI register address 입니다.

```

54      led_flag0 ,
55      led_flag1 ,
56      led_flag2 ,
57      led_flag3

69  output [2:0]  led_flag0, led_flag1, led_flag2, led_flag3 ;

```

- ✓ 라인 54 - 57, 69 : 포트 추가, [2] : R, [1] : G, [0] : B


```

317 reg    [15:0]  pwm_freq1, pwm_freq2, pwm_freq3, pwm_freq4 ;
318 reg    [6:0]   pwm_duty1, pwm_duty2, pwm_duty3, pwm_duty4 ;
319 reg    [2:0]   led_flag0, led_flag1, led_flag2, led_flag3 ;
320 always @(posedge clock or negedge reset)
321 begin
322     if(!reset) begin
323         pwm_freq1    <= 16'd100;
324         pwm_freq2    <= 16'd100;
325         pwm_freq3    <= 16'd100;
326         pwm_freq4    <= 16'd100;
327         pwm_duty1    <= 7'd50;
328         pwm_duty2    <= 7'd50;
329         pwm_duty3    <= 7'd50;
330         pwm_duty4    <= 7'd50;
331         led_flag0    <= 3'b0;
332         led_flag1    <= 3'b0;
333         led_flag2    <= 3'b0;
334         led_flag3    <= 3'b0;
335     end
336     else begin
337         pwm_freq1    <= (s_done & (waddr==`rPWM_FREQ1 )) ? wdata : pwm_freq1 ;
338         pwm_freq2    <= (s_done & (waddr==`rPWM_FREQ2 )) ? wdata : pwm_freq2 ;
339         pwm_freq3    <= (s_done & (waddr==`rPWM_FREQ3 )) ? wdata : pwm_freq3 ;
340         pwm_freq4    <= (s_done & (waddr==`rPWM_FREQ4 )) ? wdata : pwm_freq4 ;
341         pwm_duty1    <= (s_done & (waddr==`rPWM_DUTY1 )) ? wdata[6:0] : pwm_duty1 ;
342         pwm_duty2    <= (s_done & (waddr==`rPWM_DUTY2 )) ? wdata[6:0] : pwm_duty2 ;
343         pwm_duty3    <= (s_done & (waddr==`rPWM_DUTY3 )) ? wdata[6:0] : pwm_duty3 ;
344         pwm_duty4    <= (s_done & (waddr==`rPWM_DUTY4 )) ? wdata[6:0] : pwm_duty4 ;
345         led_flag0    <= (s_done & (waddr==`rLED_FLAG0 )) ? wdata[2:0] : led_flag0 ;
346         led_flag1    <= (s_done & (waddr==`rLED_FLAG1 )) ? wdata[2:0] : led_flag1 ;
347         led_flag2    <= (s_done & (waddr==`rLED_FLAG2 )) ? wdata[2:0] : led_flag2 ;
348         led_flag3    <= (s_done & (waddr==`rLED_FLAG3 )) ? wdata[2:0] : led_flag3 ;
349     end
350 end

```

✓ 라인 319 : register 선언

✓ 라인 345 - 348 : spi write 동작 시, register address에 따라 led_flag0 ~ 3 값 Update

```

353 always @(posedge clock or negedge reset)
354 begin
355     if(!reset) rdata <= 16'b0;
356     else      rdata <= s_idle ? 16'b0 :
357         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ1)) ? pwm_freq1 :
358         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ2)) ? pwm_freq2 :
359         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ3)) ? pwm_freq3 :
360         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_FREQ4)) ? pwm_freq4 :
361         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY1)) ? {9'b0, pwm_duty1} :
362         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY2)) ? {9'b0, pwm_duty2} :
363         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY3)) ? {9'b0, pwm_duty3} :
364         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rPWM_DUTY4)) ? {9'b0, pwm_duty4} :
365         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rLED_FLAG0)) ? {13'b0, led_flag0} :
366         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rLED_FLAG1)) ? {13'b0, led_flag1} :
367         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rLED_FLAG2)) ? {13'b0, led_flag2} :
368         (s_raddr & sck_pedge_ld & (ra_sckn_cnt==4'd7) & (raddr == `rLED_FLAG3)) ? {13'b0, led_flag3} :
369         rdata ;
370 end

```

✓ 라인 365 - 368 : spi read 동작 시, register address에 따라 led_flag0 ~ 3 값 전달함.

pwm_top 모듈과 mcu_pwm 모듈은 그대로 사용합니다.

lwipAppTop 모듈을 수정합니다. system_wrapper.v 파일과 6장에서 진행했던 micBlazeAppTop 모듈을 참조해서 수정합니다.

```

23 module lwipAppTop (
24     reset                      ,
25     sys_clock                  ,
26     ddr3_sdram_addr           ,
27     ddr3_sdram_ba             ,
28     ddr3_sdram_cas_n          ,
29     ddr3_sdram_ck_n           ,
30     ddr3_sdram_ck_p           ,
31     ddr3_sdram_cke            ,
32     ddr3_sdram_cs_n           ,
33     ddr3_sdram_dm             ,
34     ddr3_sdram_dq             ,
35     ddr3_sdram_dqs_n          ,
36     ddr3_sdram_dqs_p          ,
37     ddr3_sdram_odt            ,
38     ddr3_sdram_ras_n          ,
39     ddr3_sdram_reset_n        ,
40     ddr3_sdram_we_n           ,
41     eth_mdio_mdc              ,
42     eth_mdio_mdio_io          ,
43     eth_mii_col               ,
44     eth_mii_crs               ,
45     eth_mii_rst_n             ,
46     eth_mii_rx_clk            ,
47     eth_mii_rx_dv             ,
48     eth_mii_rx_er             ,
49     eth_mii_rxd               ,
50     eth_mii_tx_clk            ,
51     eth_mii_tx_en             ,
52     eth_mii_txd               ,
53     eth_ref_clk               ,
54     usb_uart_rxd              ,
55     usb_uart_txd              ,
56     oPWM                      ,
57     led0                      ,
58     led1                      ,
59     led2                      ,
60     led3                      ,
61 );

```

- ✓ 라인 23 - 61 : 포트 설정, system_wrapper 모듈의 포트를 그대로 사용합니다. spi 관련 포트는 내부에서 사용하기 때문에 빠집니다. pwm 출력과 led 출력이 추가되었습니다.

```

127 wire      spi_ss      ;
128 wire      spi_sck     ;
129 wire      spi_mosi    ;
130 wire      spi_miso    ;
131
132 IOBUF eth_mdio_mdio_iobuf (
133     .I (eth_mdio_mdio_o ),
134     .IO(eth_mdio_mdio_io),
135     .O (eth_mdio_mdio_i ),
136     .T (eth_mdio_mdio_t )
137 );

```

- ✓ 라인 127 - 130 : spi 관련 신호
- ✓ 라인 132 - 137 : IOBUF를 통하여 eth_mdio_mdio inout 포트 구현.

```

245 wire [2:0] led0 = led_flag0 ;
246 wire [2:0] led1 = led_flag1 ;
247 wire [2:0] led2 = led_flag2 ;
248 wire [2:0] led3 = led_flag3 ;

```

✓ 라인 245 - 248 : led 출력 구현

xdc 파일을 수정합니다. 아래와 같이 PWM Output, RGB LED 를 추가합니다.

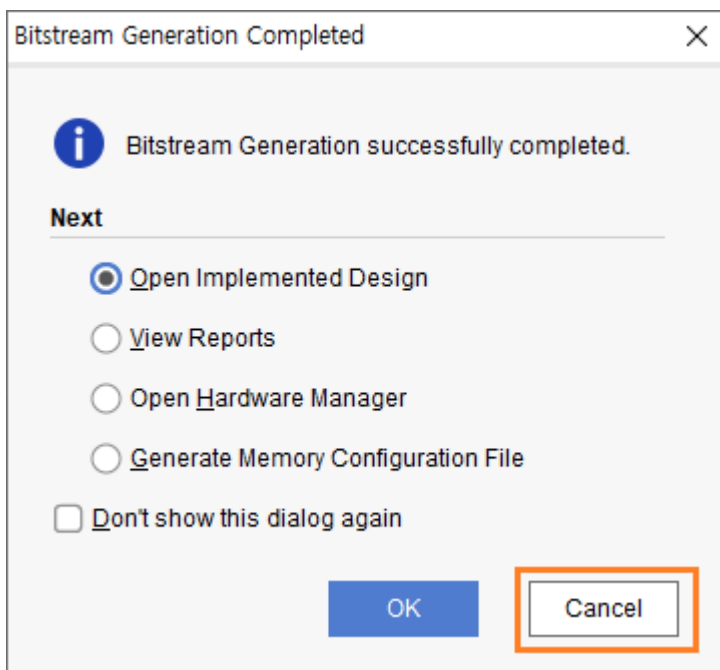
```

31 # PWM Output
32 set_property -dict { PACKAGE_PIN D4 IOSTANDARD LVCMOS33 } [get_ports {oPWM[0]}};
33 set_property -dict { PACKAGE_PIN D3 IOSTANDARD LVCMOS33 } [get_ports {oPWM[1]}};
34 set_property -dict { PACKAGE_PIN F4 IOSTANDARD LVCMOS33 } [get_ports {oPWM[2]}};
35 set_property -dict { PACKAGE_PIN F3 IOSTANDARD LVCMOS33 } [get_ports {oPWM[3]}};
36
37 ## RGB LEDs
38 set_property -dict { PACKAGE_PIN E1 IOSTANDARD LVCMOS33 } [get_ports { led0[0] }];
39 set_property -dict { PACKAGE_PIN F6 IOSTANDARD LVCMOS33 } [get_ports { led0[1] }];
40 set_property -dict { PACKAGE_PIN G6 IOSTANDARD LVCMOS33 } [get_ports { led0[2] }];
41 set_property -dict { PACKAGE_PIN G4 IOSTANDARD LVCMOS33 } [get_ports { led1[0] }];
42 set_property -dict { PACKAGE_PIN J4 IOSTANDARD LVCMOS33 } [get_ports { led1[1] }];
43 set_property -dict { PACKAGE_PIN G3 IOSTANDARD LVCMOS33 } [get_ports { led1[2] }];
44 set_property -dict { PACKAGE_PIN H4 IOSTANDARD LVCMOS33 } [get_ports { led2[0] }];
45 set_property -dict { PACKAGE_PIN J2 IOSTANDARD LVCMOS33 } [get_ports { led2[1] }];
46 set_property -dict { PACKAGE_PIN J3 IOSTANDARD LVCMOS33 } [get_ports { led2[2] }];
47 set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { led3[0] }];
48 set_property -dict { PACKAGE_PIN H6 IOSTANDARD LVCMOS33 } [get_ports { led3[1] }];
49 set_property -dict { PACKAGE_PIN K1 IOSTANDARD LVCMOS33 } [get_ports { led3[2] }];

```

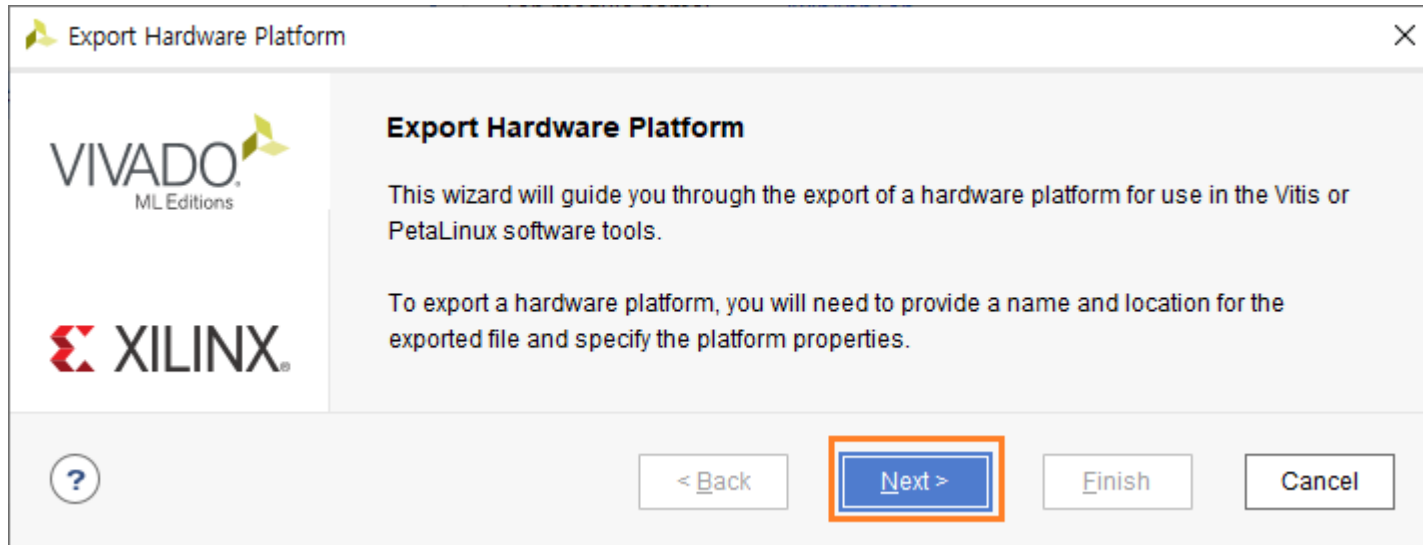
코드 구현이 완료되었습니다.

PROGRAM AND DEBUG - Generate Bitstream 을 클릭해서 Bitstream을 생성합니다. Bitstream이 생성되면 Cancel을 클릭합니다.

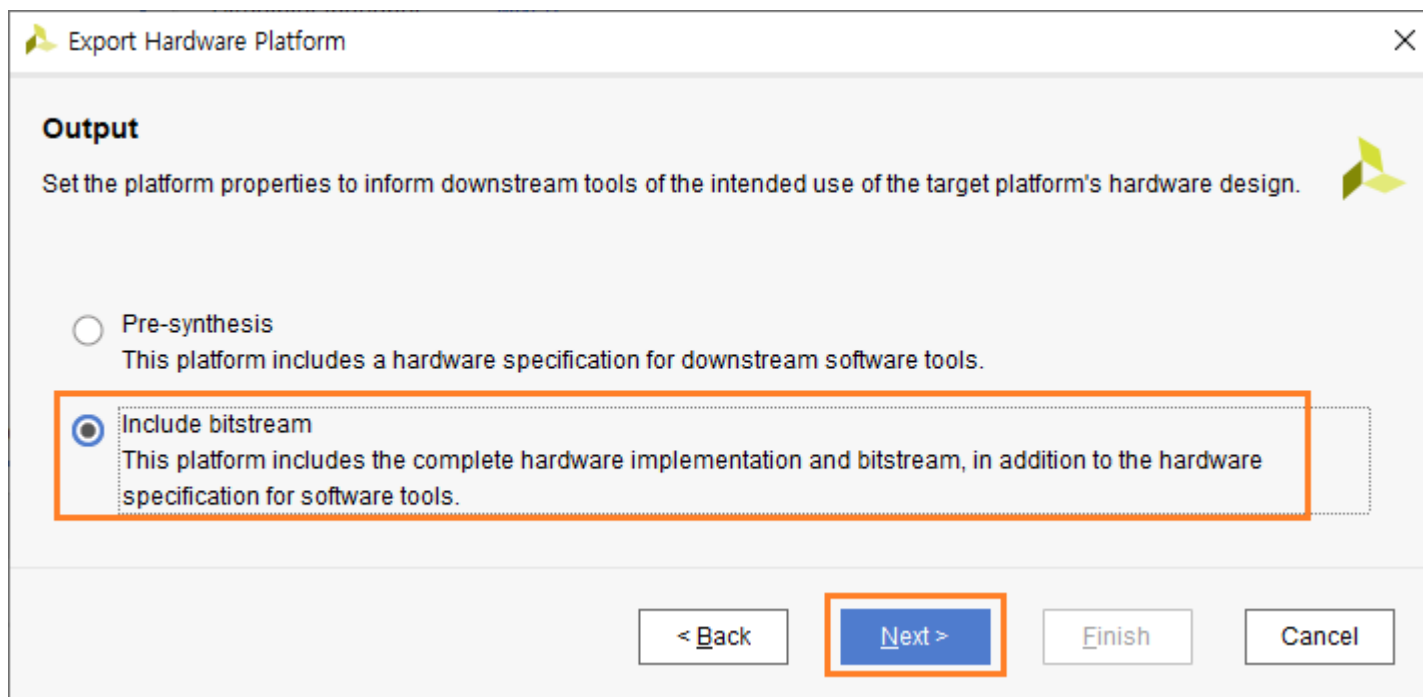


생성된 HW Design을 vitis sw application에서 사용하기 위하여 File - Export - Export Hardware 를 클릭합니다.

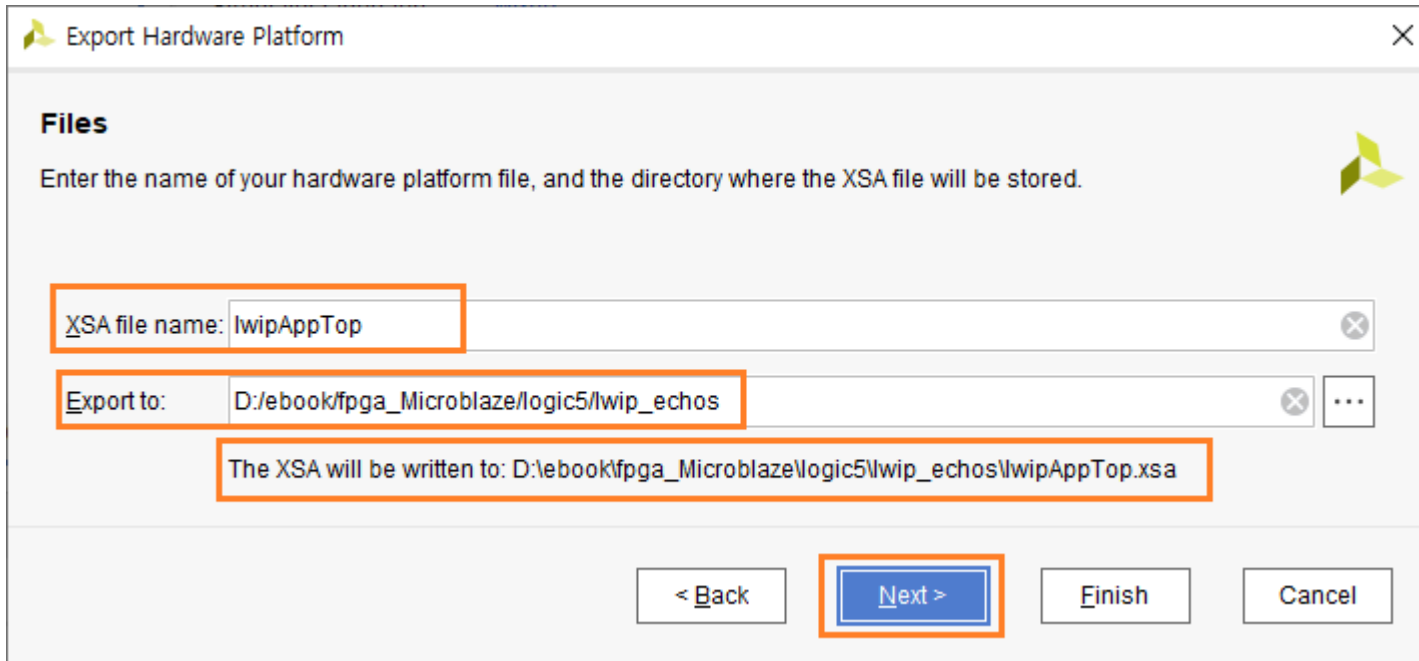
Next를 클릭합니다.



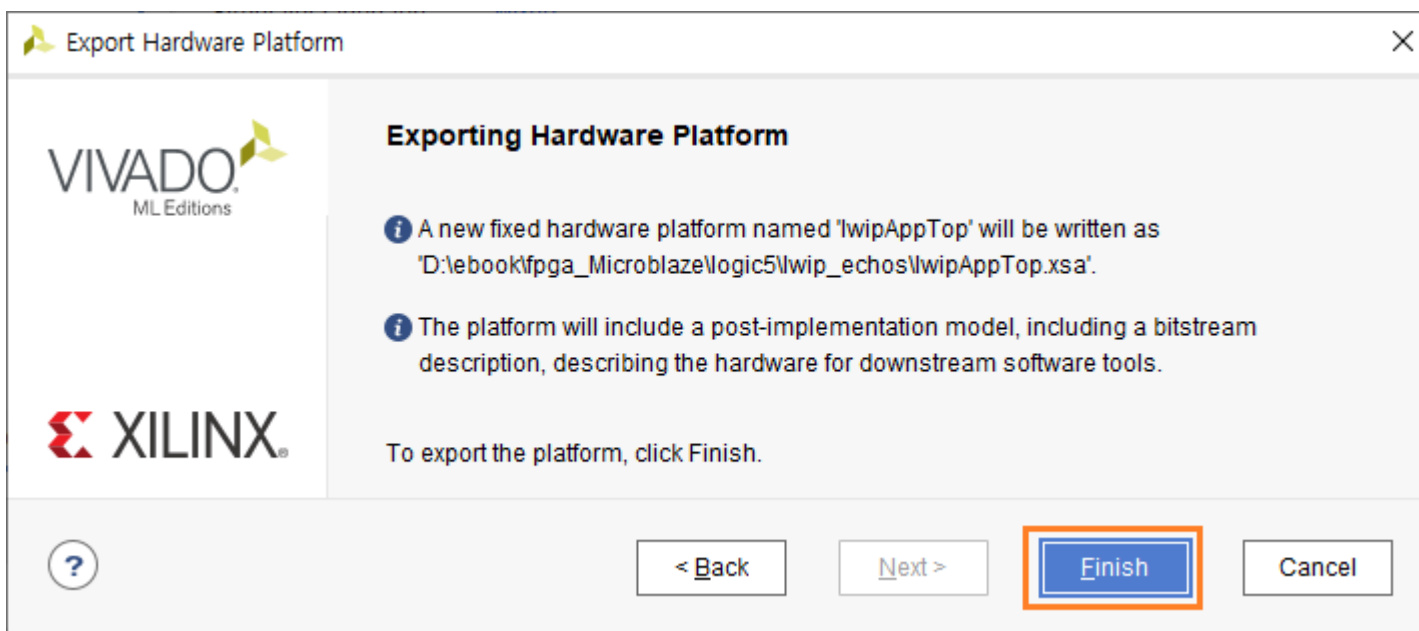
“Include bitstream”을 선택하고 Next를 클릭합니다.



xsa 파일의 생성위치와 이름을 확인하고 Next를 클릭합니다.



Finish를 클릭합니다.



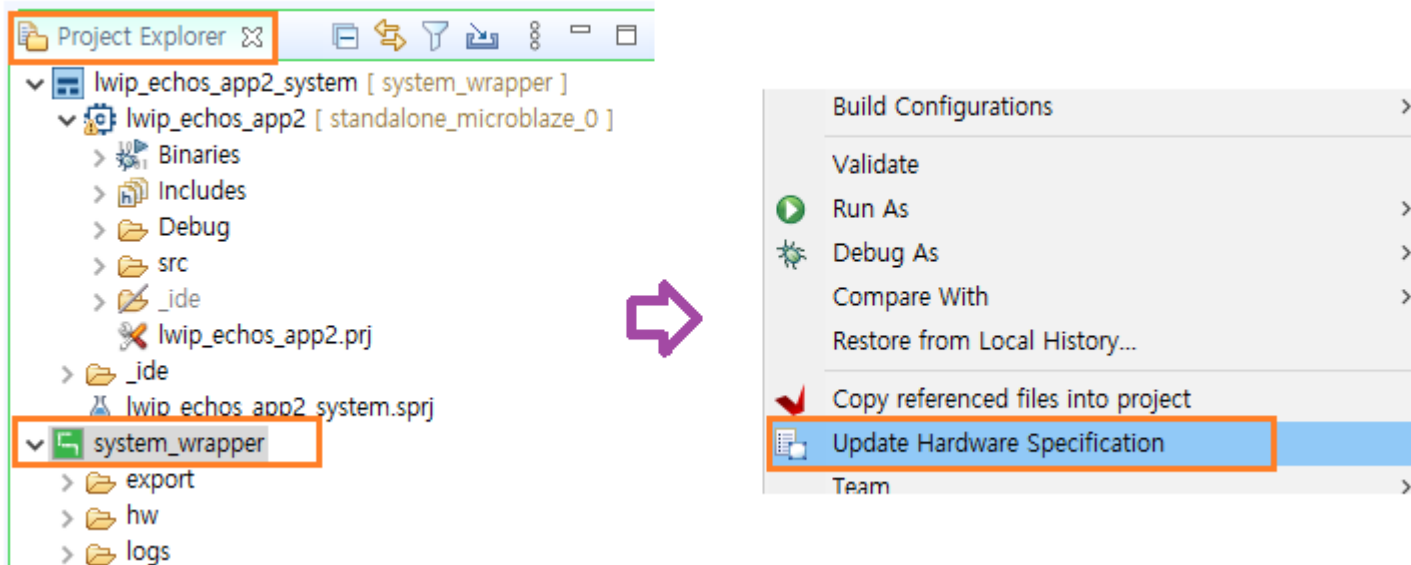
HW Design이 완료되었습니다.

Tools - Launch Vitis IDE를 클릭해서 Vitis를 실행합니다.

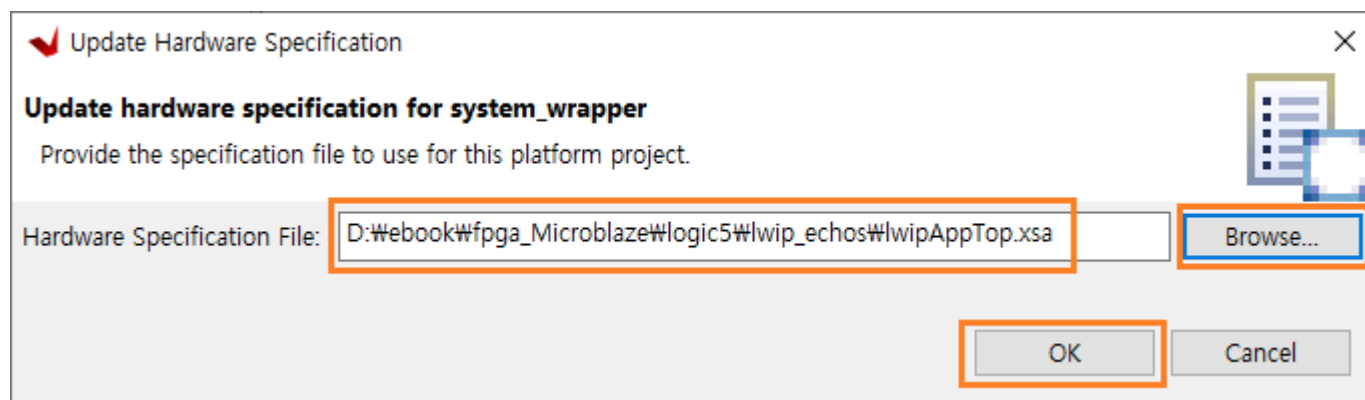
다음 장에서는 Application SW를 구현합니다.

8.3 Application SW 구현

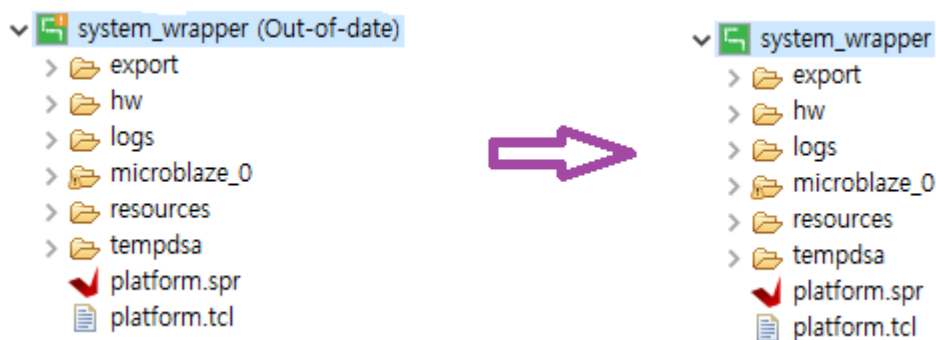
HW Design이 변경되었기 때문에 이를 적용해 주어야 합니다. Vitis의 Project Explorer 에서 system_wrapper를 선택하고 우클릭하면 나오는 메뉴에서 “Update Hardware Specification”을 선택합니다.



Browse...를 클릭해서 반드시 8.2에서 생성한 “lwipAppTop.xsa” 파일을 선택 후 OK 버튼을 클릭합니다.



적용된 xsa 파일로 Build를 합니다. (Build 전에는 system_wrapper(Out-of-date) 로 표시됩니다) system_wrapper를 선택하고 우클릭 - Build Project 를 클릭합니다. Build가 완료되면 오른쪽과 같이 “Out-of-date” 가 사라집니다.



이제 프로그램 소스를 수정하도록 하겠습니다.

main.c 를 아래와 같이 수정합니다.

```

43 #include "xspi.h"
44
45 #if LWIP_IPV6==1
46 #include "lwip/ip.h"
47 #else
48 #if LWIP_DHCP==1
49 #include "lwip/dhcp.h"
50 #endif
51 #endif
52
53 /* definitions for SPI */
54 #define SPI_DEVICE_ID          XPAR_SPI_0_DEVICE_ID
55

```

✓ 라인 43 : xspi.h include

✓ 라인 54 : SPI_DEVICE_ID define

```

77
78 XSpi      SpiInstance;          /* The instance of the SPI device */
79

```

✓ 라인 78 : Xspi 변수 생성

```

127 void spi_init(void)
128 {
129     XSpi_Config *ConfigPtr; /* Pointer to Configuration data */
130
131     ConfigPtr = XSpi_LookupConfig(SPI_DEVICE_ID);
132     XSpi_CfgInitialize(&SpiInstance, ConfigPtr, ConfigPtr->BaseAddress);
133     XSpi_SelfTest(&SpiInstance);
134
135     XSpi_SetOptions(&SpiInstance, XSP_MASTER_OPTION );
136     XSpi_Start(&SpiInstance);          /* Start SPI */
137     XSpi_IntrGlobalDisable(&SpiInstance); /* Disable interrupt */
138     XSpi_SetSlaveSelect(&SpiInstance, 0x35); /* 이 문이 빠지면 동작하지 않음 */
139 }

```

✓ 라인 127 - 139 : spi 초기화 함수

```

167     init_platform();
168     spi_init();
169

```

✓ 라인 168 : spi_init() 함수로 spi 초기화

echo.c 를 아래와 같이 수정합니다.

```

37
38 #include "xspi.h"
39 #include "comm_task.h"
40
41 extern XSpi      SpiInstance;
42
43 int transfer_data() {
44     return 0;
45 }
46
47 void print_app_header()
48 {
49     #if (LWIP_IPV6==0)
50         xil_printf("\n\r\n\r-----lwIP TCP echo server ----- \n\r");
51     #else
52         xil_printf("\n\r\n\r-----lwIPv6 TCP echo server ----- \n\r");
53     #endif
54     xil_printf("TCP packets sent to port 6001 will be echoed back\n\r");
55 }
56
57 err_t recv_callback(void *arg, struct tcp_pcb *tpcb,
58                     struct pbuf *p, err_t err)
59 {
60     uint8_t *rbuf;
61     int i;
62
63     /* do not read the packet if we are not in ESTABLISHED state */
64     if (!p) {
65         tcp_close(tpcb);
66         tcp_recv(tpcb, NULL);
67         return ERR_OK;
68     }
69
70     /* indicate that the packet has been received */
71     tcp_recved(tpcb, p->len);
72
73     /* echo back the payload */
74     /* in this case, we assume that the payload is < TCP_SND_BUF */
75     if (tcp_sndbuf(tpcb) > p->len) {
76         rbuf = (uint8_t *)malloc(p->len);
77         memcpy(rbuf, p->payload, p->len);
78         for(i=0; i<p->len; i++) {
79             comm_decode(rbuf[i]);
80         }
81
82         err = tcp_write(tpcb, p->payload, p->len, 1);
83     } else
84         xil_printf("no space in tcp_sndbuf\n\r");
85
86     /* free the received pbuf */
87     pbuf_free(p);
88
89     return ERR_OK;
90 }

```

- ✓ 라인 38 - 39 : include xspi.h, comm_task.h
- ✓ 라인 41 : SpiInstance를 사용하기 위하여 extern 으로 선언
- ✓ 라인 60 - 61 : 수신된 데이터를 저장하기 위한 변수 선언
- ✓ 라인 76 - 80 : 수신된 데이터는 pbuf 의 payload에 저장되고, 수신된 데이터 사이즈는 pbuf의 len에 저장됩니다.
수신된 데이터를 rbuf에 저장하고 comm_decode 함수를 호출하여 수신된 데이터를 처리합니다.
- ✓ 라인 82 : 수신된 데이터를 다시 전송합니다. (Echo server)

comm_task.c를 아래와 같이 수정합니다. 6장에서 설명된 내용과 거의 비슷합니다. 추가/변경된 부분만 설명합니다.

```

17
18 #define rLED_FLAG0 0x30
19 #define rLED_FLAG1 0x31
20 #define rLED_FLAG2 0x32
21 #define rLED_FLAG3 0x33
22
23
24 u8 comm_buf[COMM_BUF_MAX];
25 u8 comm_etx_flag = 0;
26 u16 comm_index=0;
27
28 extern XSpi SpiInstance;
29
30 void comm_task(void);
31
32 uint8_t led_flag0 = 0;
33 uint8_t led_flag1 = 0;
34 uint8_t led_flag2 = 0;
35 uint8_t led_flag3 = 0;
36

```

- ✓ 라인 18 - 21 : LED 제어를 위한 SPI Register Address
- ✓ 라인 32 - 35 : led 제어를 위한 변수 생성

```

66 // led_r0 on
67 if( (comm_buf[0]=='1') && (comm_buf[1]=='r') && (comm_buf[2]=='0') && (comm_buf[3]=='1') )
68 {
69     led_flag0 = led_flag0 | 0x04;
70     wbuf[0] = 0x64;
71     wbuf[1] = rLED_FLAG0;
72     wbuf[2] = 0;
73     wbuf[3] = led_flag0;
74     XSpi_Transfer(&SpiInstance, wbuf, NULL, 4);
75     xil_printf("led_r0 on \r\n");
76 }
77 // led_r0 off
78 else if( (comm_buf[0]=='1') && (comm_buf[1]=='r') && (comm_buf[2]=='0') && (comm_buf[3]=='0') )
79 {
80     led_flag0 = led_flag0 & ~0x04;
81     wbuf[0] = 0x64;
82     wbuf[1] = rLED_FLAG0;
83     wbuf[2] = 0;
84     wbuf[3] = led_flag0;
85     XSpi_Transfer(&SpiInstance, wbuf, NULL, 4);
86     xil_printf("led_r0 off \r\n");
87 }

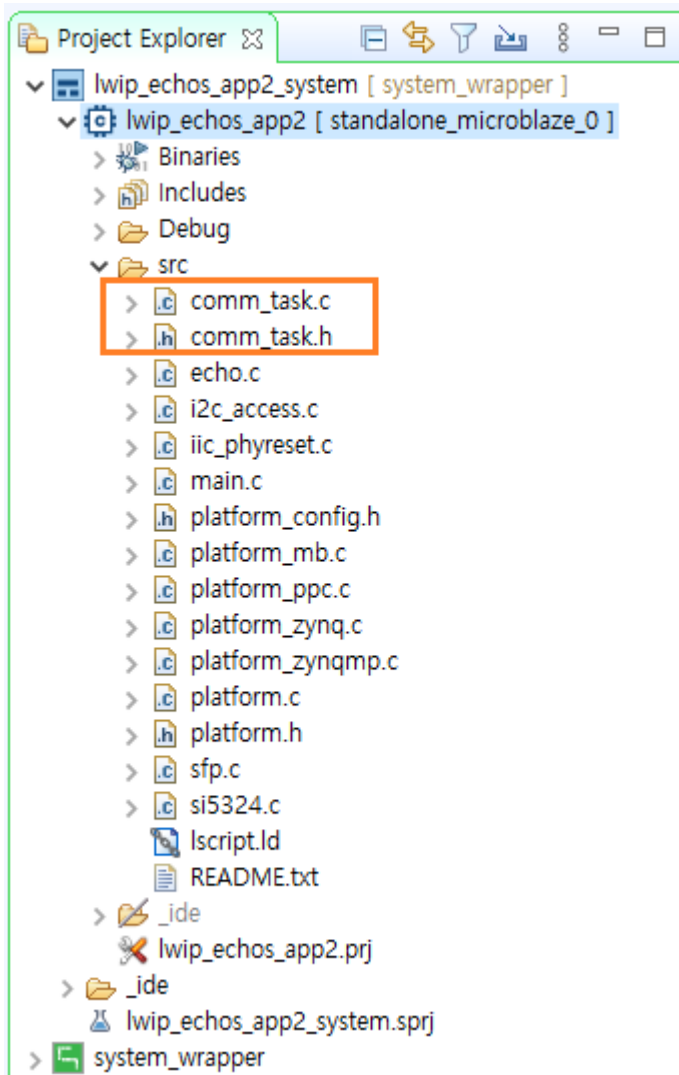
```

- ✓ 라인 67 - 76 : “led_r0 on” command를 수신하여 SPI Master로 Register Address(rLED_FLAG0)와 Register Data (0x00, led_flag0) 를 전송합니다. SPI Master에서 전송한 데이터는 SPI Slave에서 수신해서 해당 레지스터에 데이터를 Write 합니다. 결과적으로 LED On/Off 제어가 됩니다. com_task 내부의 코드가 동일한 코드를 수행합니다. 시리얼로 명령어에 대한 내용을 전송합니다.

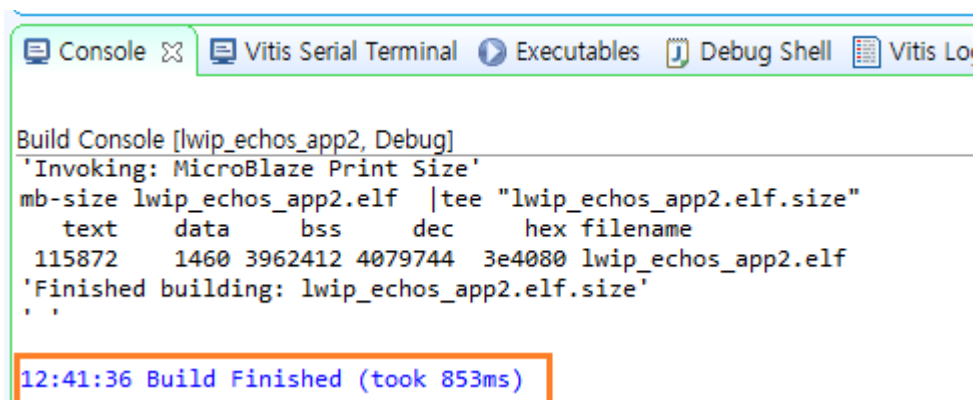
아래는 command 를 보여줍니다.

command	description
@lr01*	led_r0 on
@lr00*	led_r0 off
@lg01*	led_g0 on
@lg00*	led_g0 off
@lb01*	led_b0 on
@lb00*	led_b0 off
@lr11*	led_r1 on
@lr10*	led_r1 off
@lg11*	led_g1 on
@lg10*	led_g1 off
@lb11*	led_b1 on
@lb10*	led_b1 off
@lr21*	led_r2 on
@lr20*	led_r2 off
@lg21*	led_g2 on
@lg20*	led_g2 off
@lb21*	led_b2 on
@lb20*	led_b2 off
@lr31*	led_r3 on
@lr30*	led_r3 off
@lg31*	led_g3 on
@lg30*	led_g3 off
@lb31*	led_b3 on
@lb30*	led_b3 off

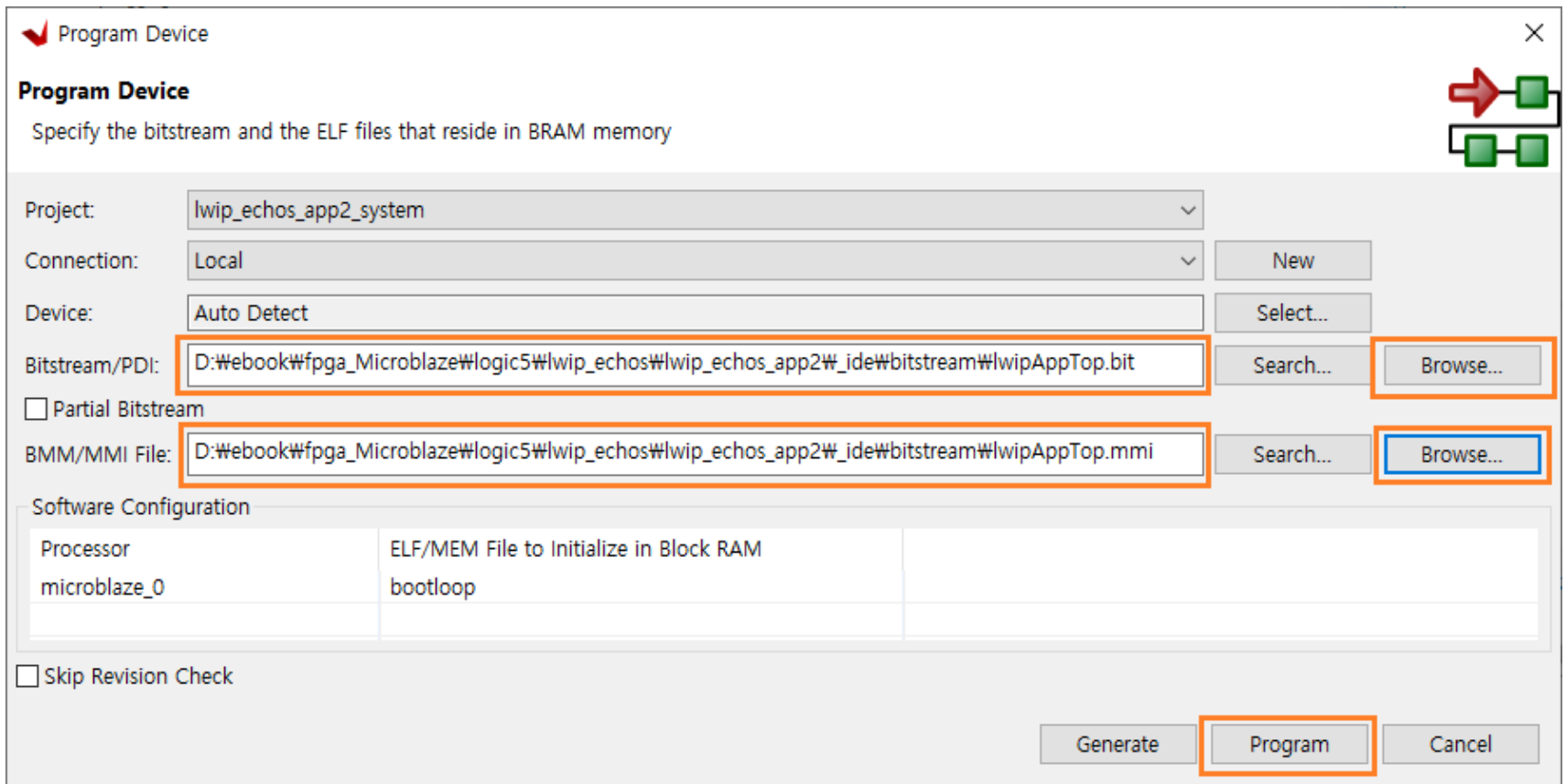
아래는 전체 소스 파일을 보여줍니다. comm_task.c, comm_task.h 파일이 추가되었습니다.



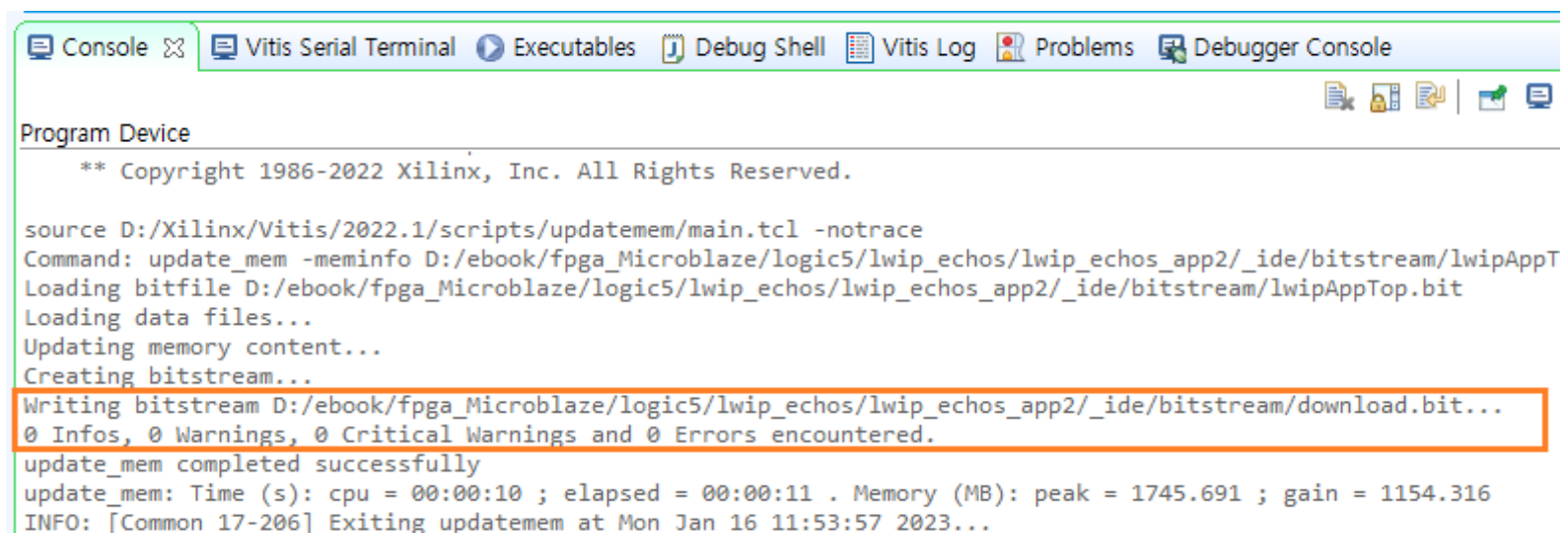
lwip_echos_app2를 선택 후 우클릭해서 “Build Project”를 클릭해서 Project를 Build 합니다. 에러 없이 Build 된 것을 확인합니다.



보드에 다운로드 합니다. Xilinx - Program Device 를 클릭합니다. Program 버튼이 활성화 되지 않았다면 Bitstream, MMI 파일경로가 잘못되었거나 파일명이 잘못된 경우입니다. Browse.. 클릭하여 lwipAppTop.bit 파일과 lwipAppTop.mmi 파일을 찾아서 열기를 합니다. Program 을 클릭하여 보드에 다운로드 합니다.

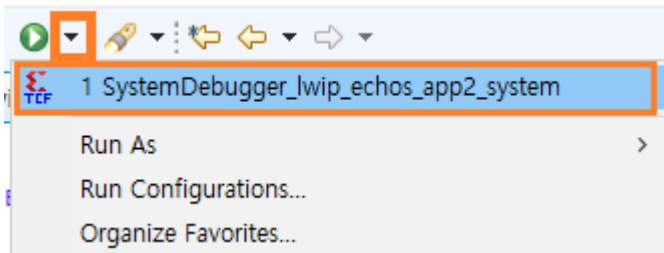


다운로드가 완료되었습니다.

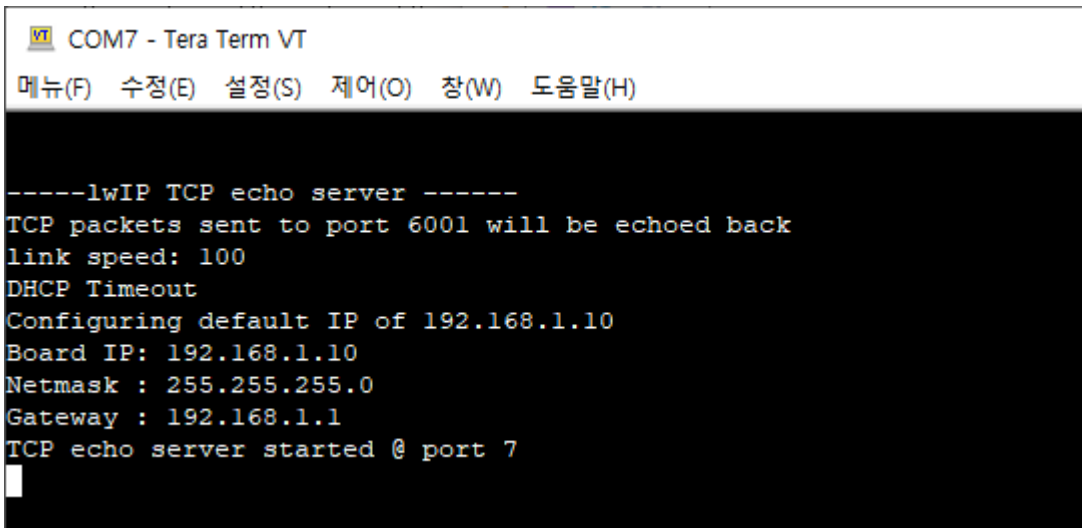


보드에 전원을 인가하고 Tera Term으로 Serial Port를 Open 합니다. 속도를 115200으로 변경합니다.

Run 오른쪽을 클릭해서 “1 SystemDebugger_lwip_echos_app2_system”을 클릭하면 프로그램이 실행됩니다. 만일 “1 SystemDebugger_lwip_echos_app2_system”이 보이지 않으면, Run Configuration에서 추가합니다.



프로그램이 다운로드 되고 Tera Term에 메시지가 나타납니다.



Tera Term을 1개 더 실행해서 Tcp/Ip로 Open 합니다. 호스트 : 192.168.1.10, 서비스 : Telnet, 포트 : 7 입니다.

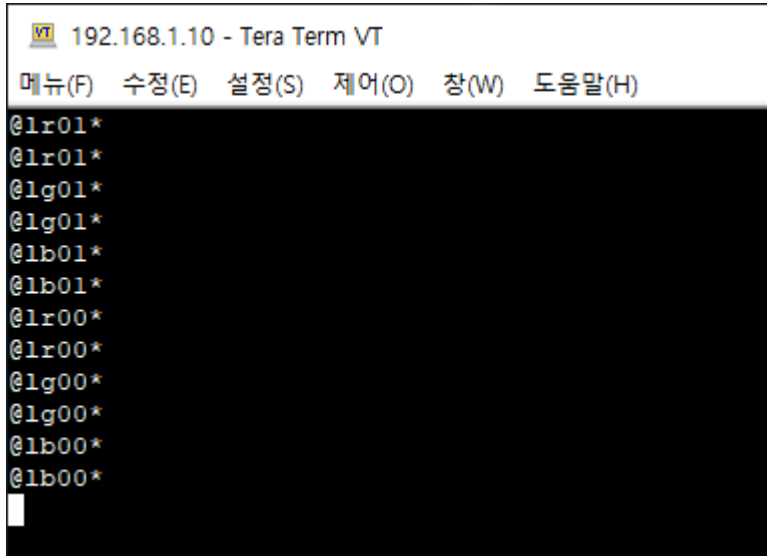
설정 - 터미널에서 줄바꿈 송신 : CR+LF, 지역 에코 : check 로 변경합니다.

TCP/IP 로 Open한 Tera Term에 아래와 같이 순서대로 입력합니다. (각 명령어 입력후 엔터키를 눌러야 합니다.)

- ✓ @lr01*
- ✓ @lg01*
- ✓ @lb01*
- ✓ @lr00*
- ✓ @lg00*
- ✓ @lb00*

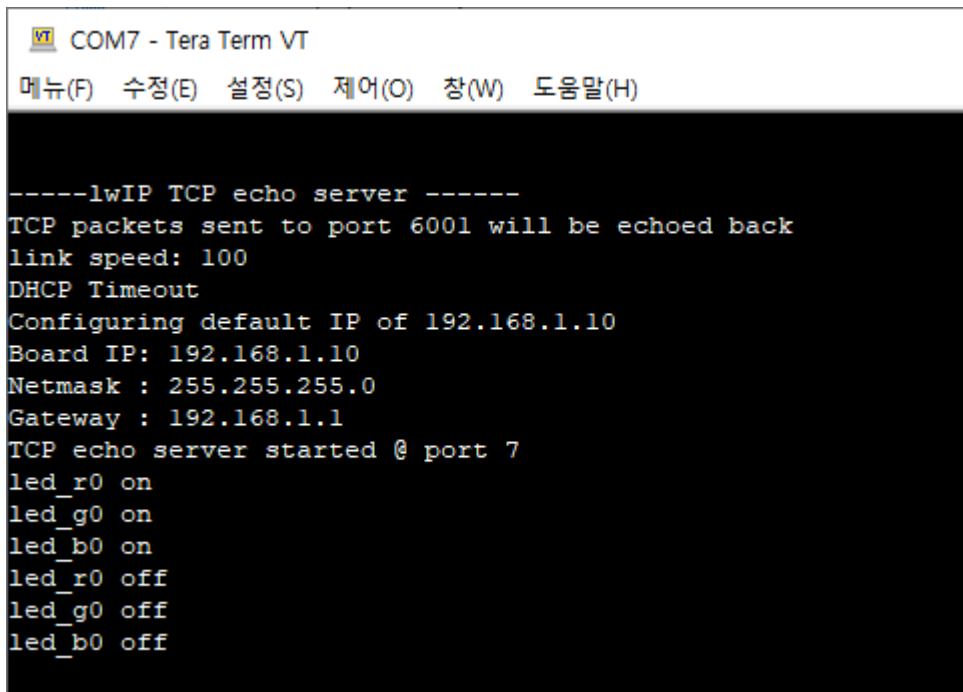
LED0 의 R, G, B 를 차례대로 On 한 후에, 다시 차례대로 Off 하는 명령어를 전송합니다. 아래는 그 결과를 보여줍니다.

TCP/IP는 입력된 명령어가 그대로 Echo Back 되어 출력됩니다.



```
VT 192.168.1.10 - Tera Term VT
메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)
@lr01*
@lr01*
@lg01*
@lg01*
@lb01*
@lb01*
@lr00*
@lr00*
@lg00*
@lg00*
@lb00*
@lb00*
```

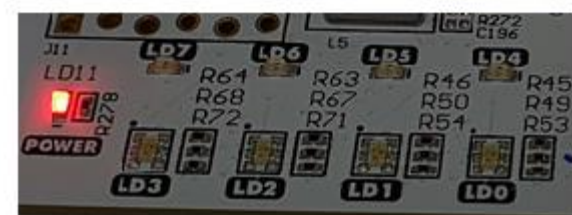
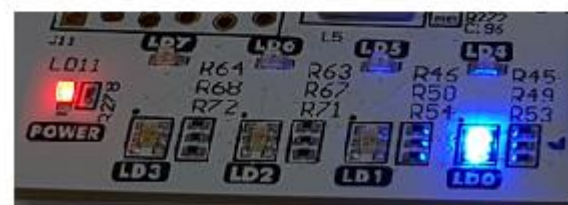
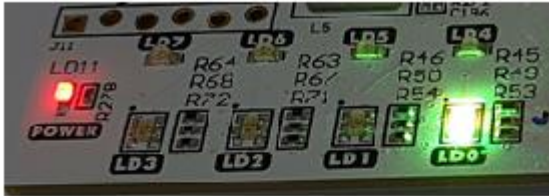
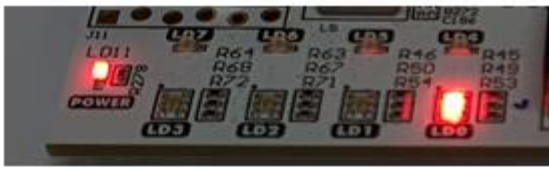
Serial 은 해당 명령어 수신후, 각 명령어에 대한 내용을 출력합니다.



```
VT COM7 - Tera Term VT
메뉴(F) 수정(E) 설정(S) 제어(O) 창(W) 도움말(H)

-----lwIP TCP echo server -----
TCP packets sent to port 6001 will be echoed back
link speed: 100
DHCP Timeout
Configuring default IP of 192.168.1.10
Board IP: 192.168.1.10
Netmask : 255.255.255.0
Gateway : 192.168.1.1
TCP echo server started @ port 7
led_r0 on
led_g0 on
led_b0 on
led_r0 off
led_g0 off
led_b0 off
```

마지막으로 보드의 LED의 상태를 보여줍니다. 순서대로 R(r on), R+G(g on), R+G+B(b on), G+B(r off), B(g off), Off (b off) 를 나타냅니다. TCP/IP 명령을 통하여 보드의 LED가 제어됨을 확인 할 수 있습니다.

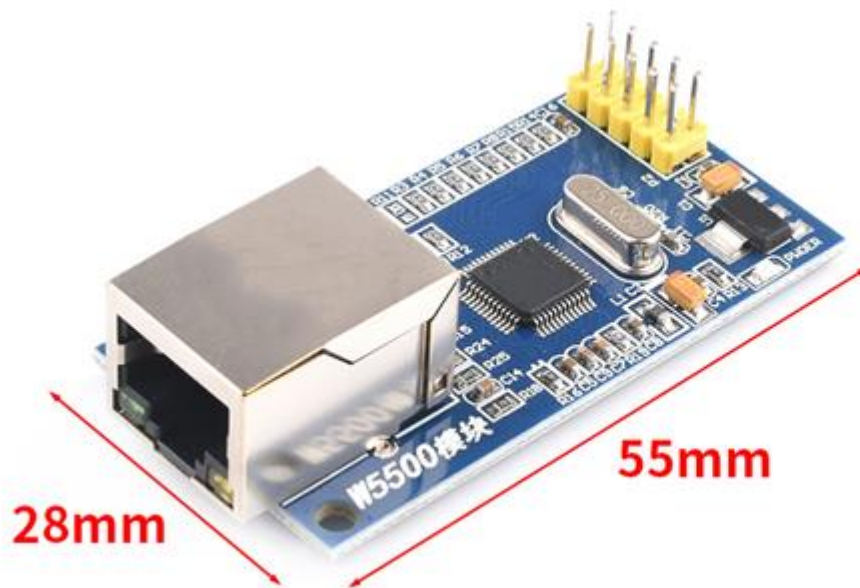


9. W5500을 이용한 TCP/IP 구현

이번 장에서는 wiznet사의 w5500 모듈을 이용한 tcp/ip를 구현하도록 합니다. w5500은 wiznet의 Hardware TCP/IP 기술을 이용한 임베디드용 인터넷 솔루션으로 하나의 칩에 TCP/IP 프로토콜 처리부터 10/100 Ethernet PHY와 MAC을 모두 내장하고 있습니다 (w5500 datasheet 내용)

w5500은 spi 인터페이스를 지원합니다. microblaze 와 spi를 이용하여 w5500을 제어하도록 하겠습니다. PC와 Network 으로 연결해서 데이터를 주고 받는 테스트를 진행합니다. sw 구현은 wiznet에서 제공하는 API를 사용하여 필요한 부분만 포팅해서 사용합니다.

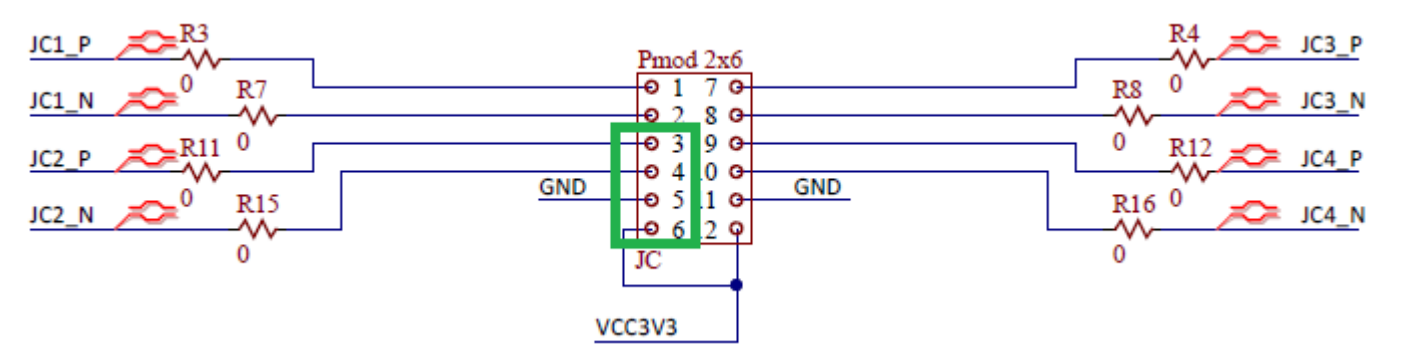
아래 그림은 실습에 사용된 모듈을 보여줍니다. (JK 전자 W5500 TCP-IP 이더넷 모듈 SPI)



위 사진 기준 핀 배열	
3.3V	5V
MISO	GND
MOSI	RST
SCS	INT
SCL	NC

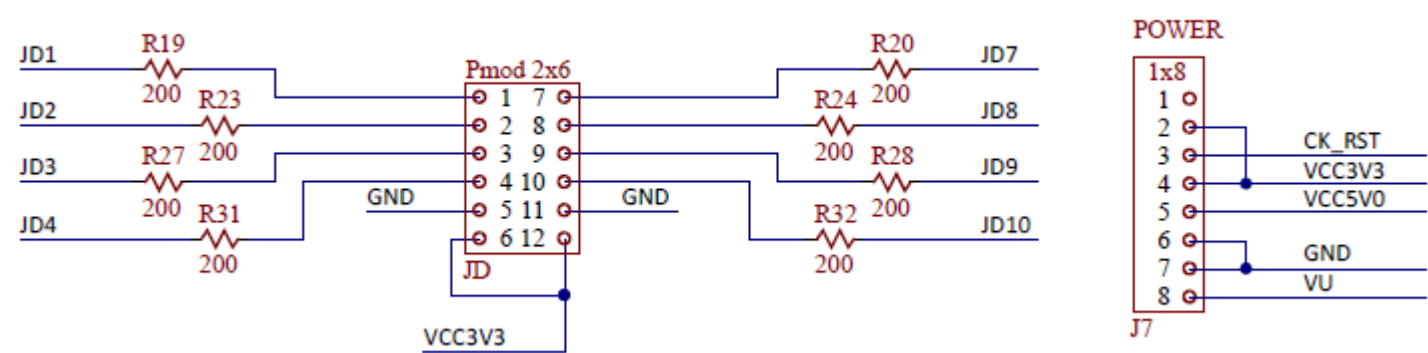
Arty A7 보드와의 핀맵은 아래와 같습니다.

- Max3232 Module 핀맵 : 디버깅 메시지를 표시하기 위한 용도입니다.



Arty A7 JC	Max3232 Module
3 (JC2_P)	2 (RXD)
4 (JC2_N)	3 (TXD)
5 (GND)	4 (GND)
6 (VCC3V3)	5 (VCC)

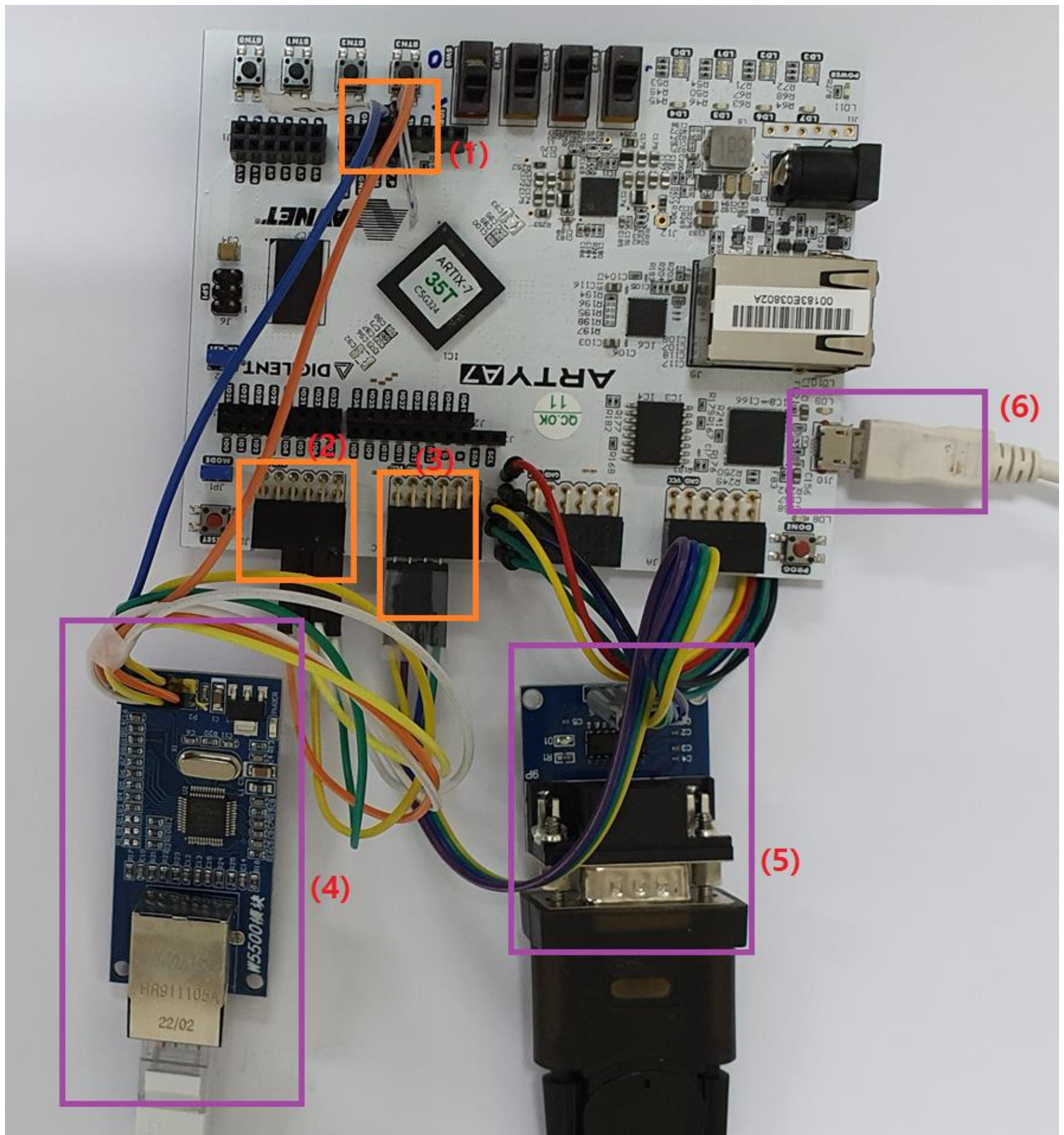
- w5500 Module 핀맵 : 전원은 5V만 인가하면 됩니다.



Arty A7 JD	Arty A7 J7	w5500 Module
1 (JD1, ETH_MISO)		8 (MISO)
2 (JD2, EHT_MOSI)		6 (MOSI)
3 (JD3, ETH_SCS)		4 (SCS)
4 (JD4, ETH_SCK)		2 (SCK)
7 (JD7, ETH_RST)		5 (RST)
8 (JD8, ETH_INT)		3 (INT)
9 (JD9, TP_1)		Test Pin1 for debug
10 (JD10, TP_2)		Test pin2 for debug

	5 (VCC5V0)	9 (5V)
	6 (GND)	7 (GND)

아래 그림은 연결된 모습을 보여줍니다.



(1) w5500 모듈에 5V 전원 공급

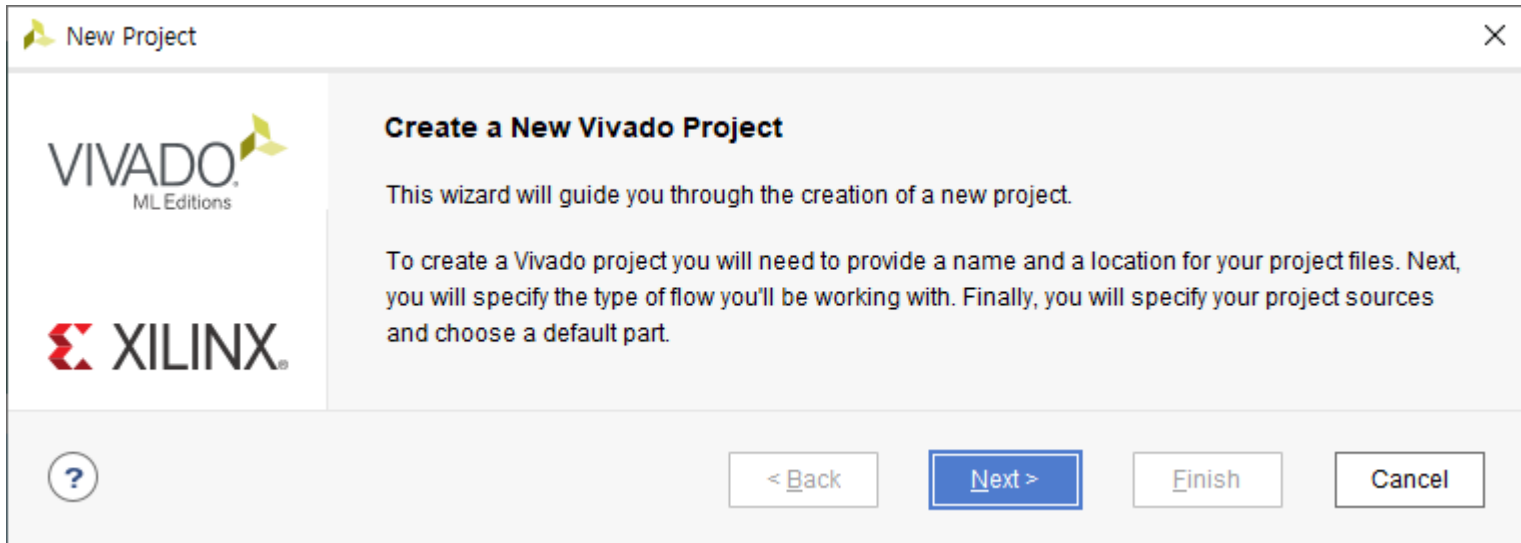
(2) w5500 모듈과 연결되는 JD connector

- (3) max3232 모듈과 연결되는 JC connector
- (4) w5500 모듈
- (5) max3232, usb to rs232 cable
- (6) 보드 전원 공급, 프로그램 다운로드를 위한 usb cable

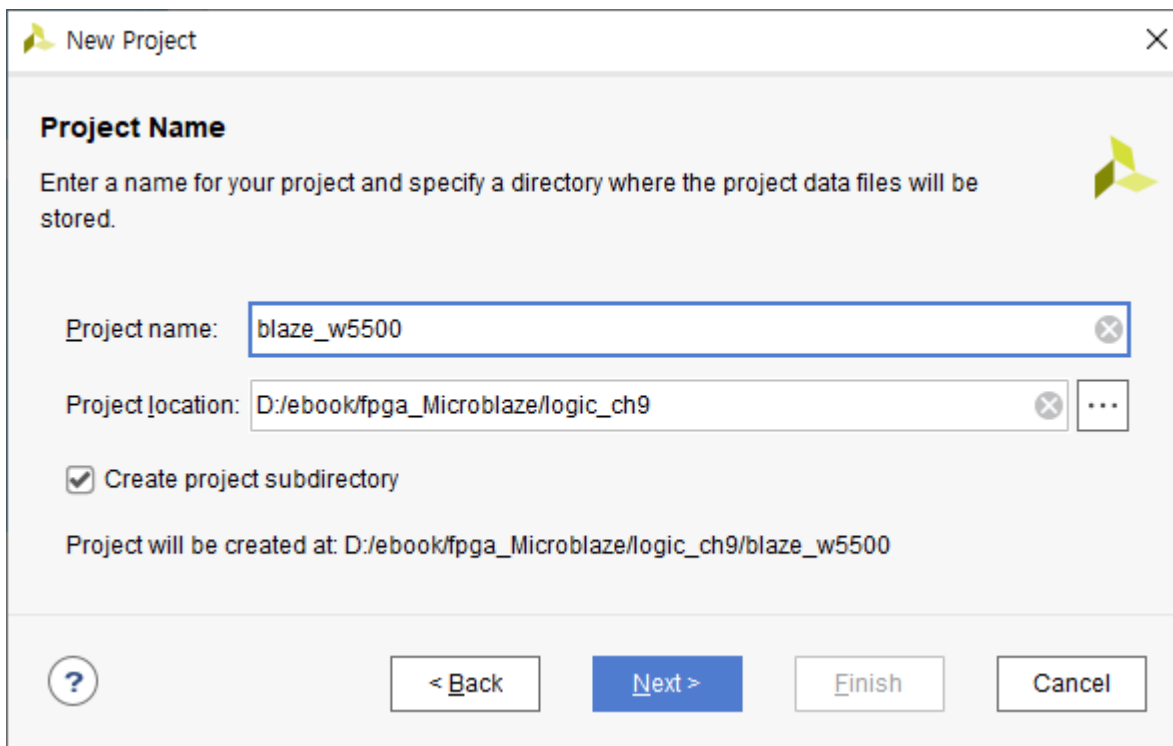


9.1 프로젝트 생성

vivado 2022.1 을 실행하고 Create Project를 클릭해서 Project를 생성합니다. Next를 클릭합니다.



프로젝트 이름은 “blaze_w5500”로 하겠습니다.



Project Type : RTL Proejct를 선택하고 Next를 클릭합니다.

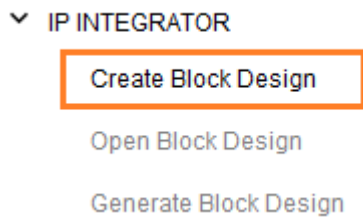
Add Sources, Add Constraints 윈도우에서 Next를 클릭합니다. 나중에 소스와 xdc 파일을 추가합니다.

Part : xc7a35tcsg324-1 을 선택하고 Next를 클릭합니다.

Finish를 클릭해서 프로젝트를 생성합니다.

9.2 Block Design

새로운 Block을 생성하기 위하여 PROJECT MANAGER에서 IP INTEGRATOR - Create Block Design을 클릭합니다.



Design Name : system 으로 하고 OK를 클릭합니다.

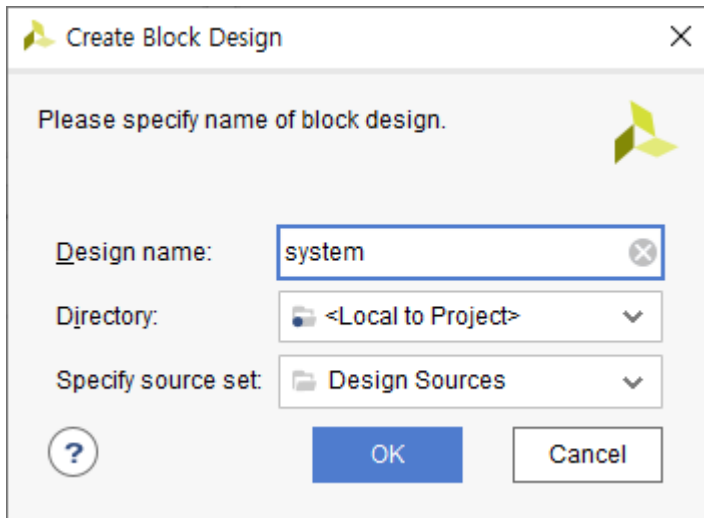


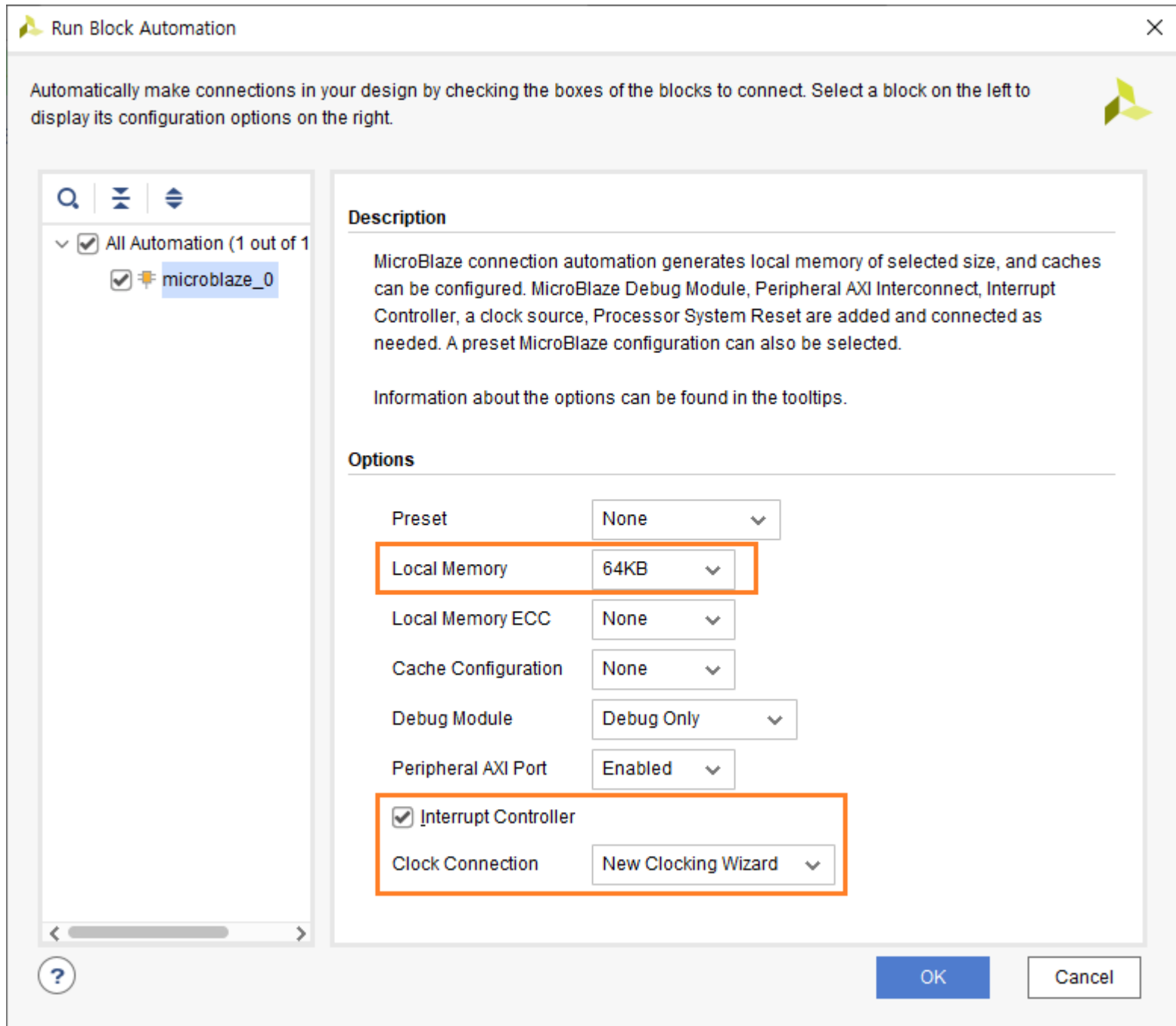
Diagram 윈도가 생성되었습니다. Add IP (+ 아이콘)을 클릭하여 4개의 IP를 추가합니다. Run Block Automation, Run Connection Automation은 4개를 모두 추가한 후에 진행합니다.

- ✓ MicroBlaze
- ✓ AXI Quad SPI
- ✓ AXI Uartlite
- ✓ AXI GPIO

4개의 IP를 추가한 후에 Run Block Automation을 클릭합니다.

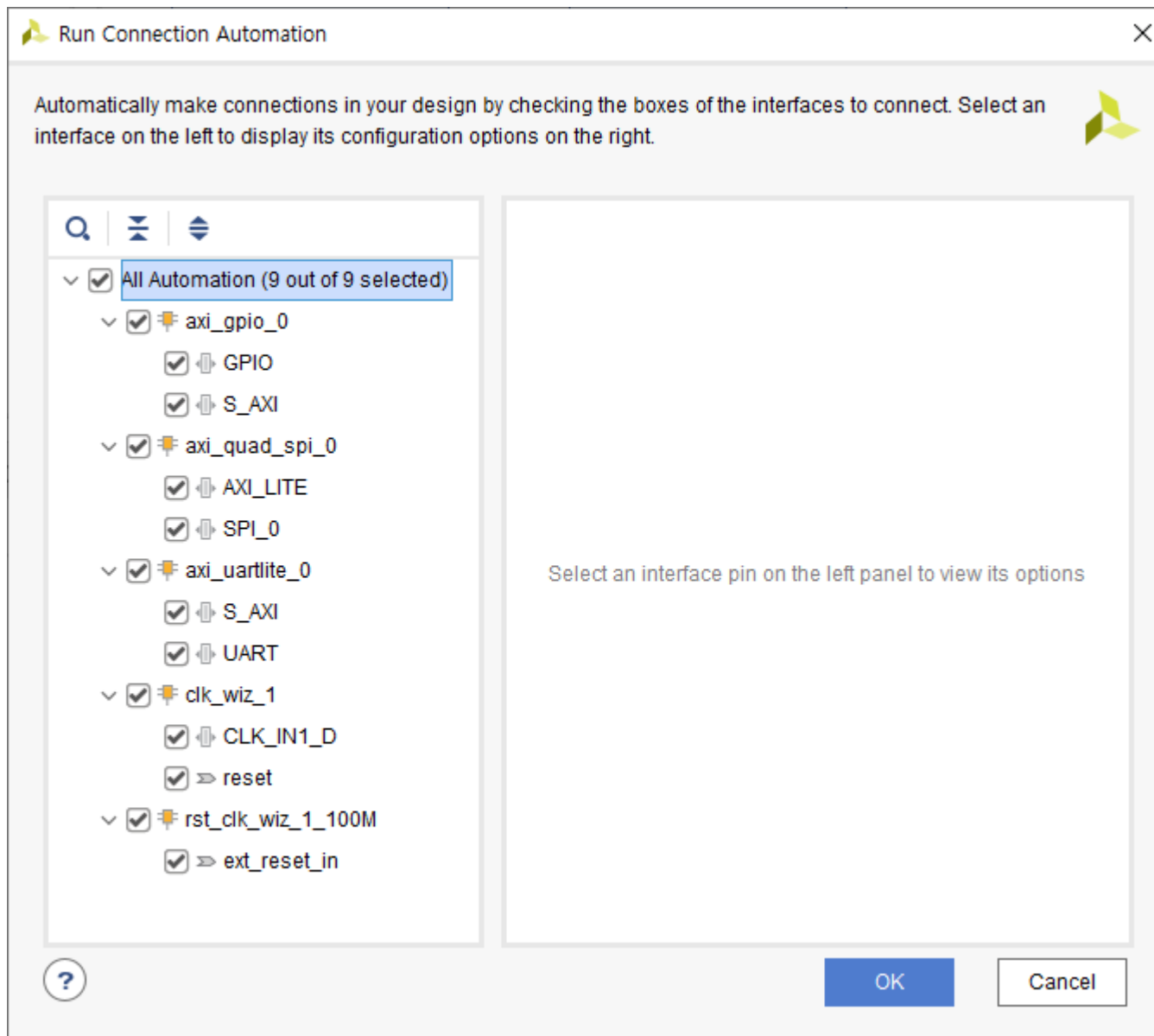
첫번째 Microblaze의 속성을 설정합니다.

- ✓ Local Memory : 64KB (32KB도 가능하지만, 64KB로 넉넉히 설정하겠습니다)
- ✓ Interrupt Controller : check
- ✓ Clock Connection : New Clocking Wizard

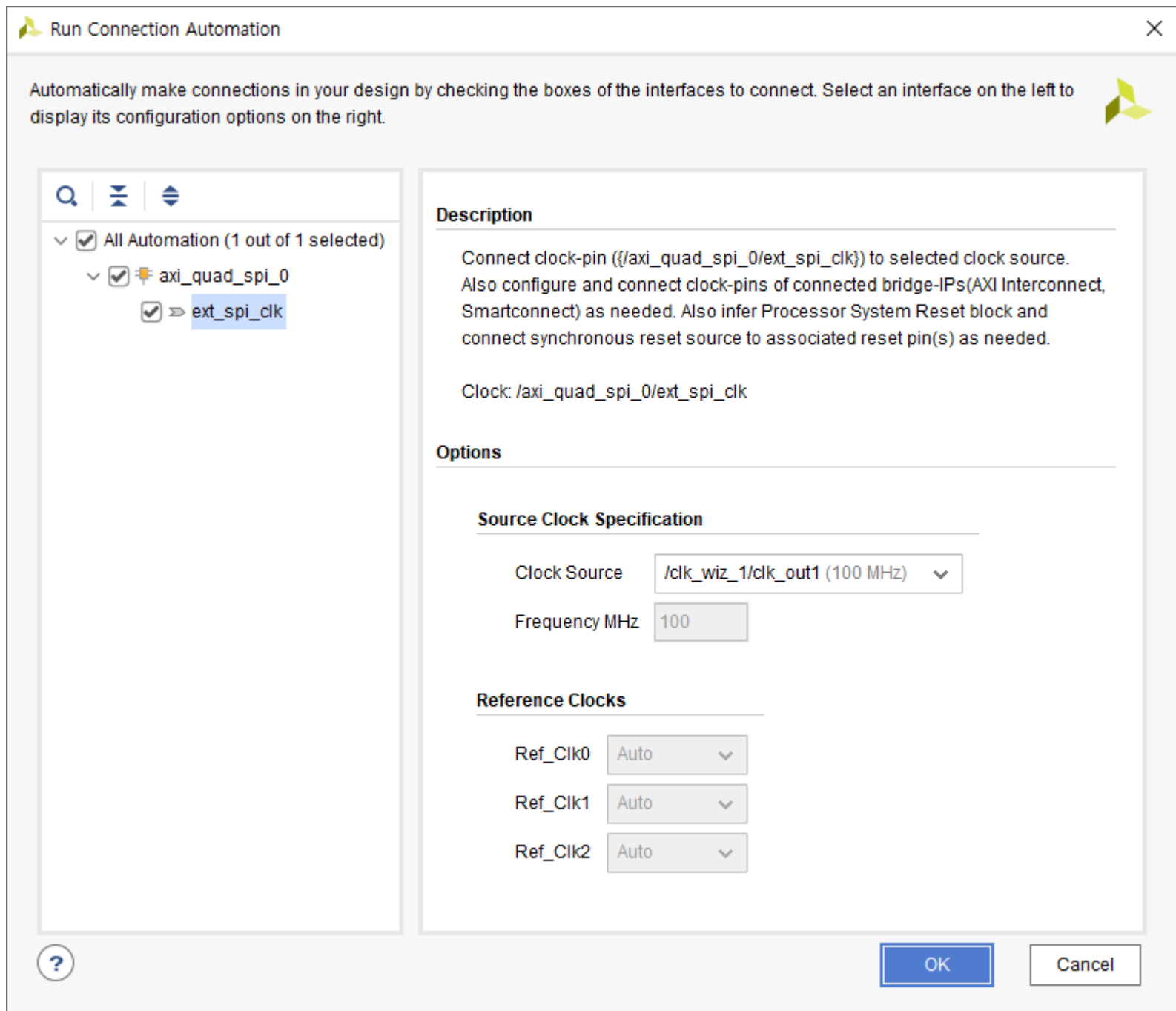


OK를 클릭합니다.

Run Connection Automation을 클릭합니다. All Automation을 클릭해서 전체 모듈을 선택하고 OK를 클릭합니다.



Run Connection Automation이 활성화되어 있으면 다시 클릭해 줍니다. All Automation 선택하고 OK를 클릭합니다.



Validate Design, Regenerate Layout을 차례로 클릭합니다. Error가 없음을 확인합니다.

몇가지 IP의 속성을 설정하겠습니다.

Clocking Wizard (clk_wiz_1)을 더블 클릭해서 속성을 변경합니다.

1) Clocking Options 탭

A. Input Clock (clk_in1) : Source를 Single ended clock capable pin

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter ^{^1}	Input Jitter	Source
☒	Primary	clk_in1	AUTO 100.000	10.000 - 200.000	UI	0.010	Single ended clock capable pin
☐	Secondary	clk_in2	AUTO 100.000	60.000 - 120.000		0.010	Single ended clock capable pin

2) Output Clocks 탭

A. Reset Type : Active Low

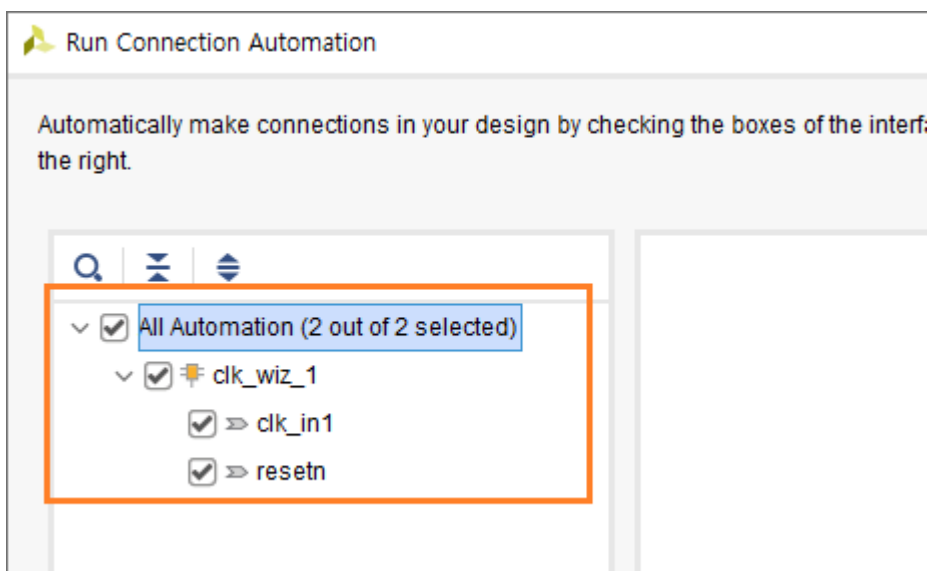
Reset Type

☐ Active High ☒ Active Low

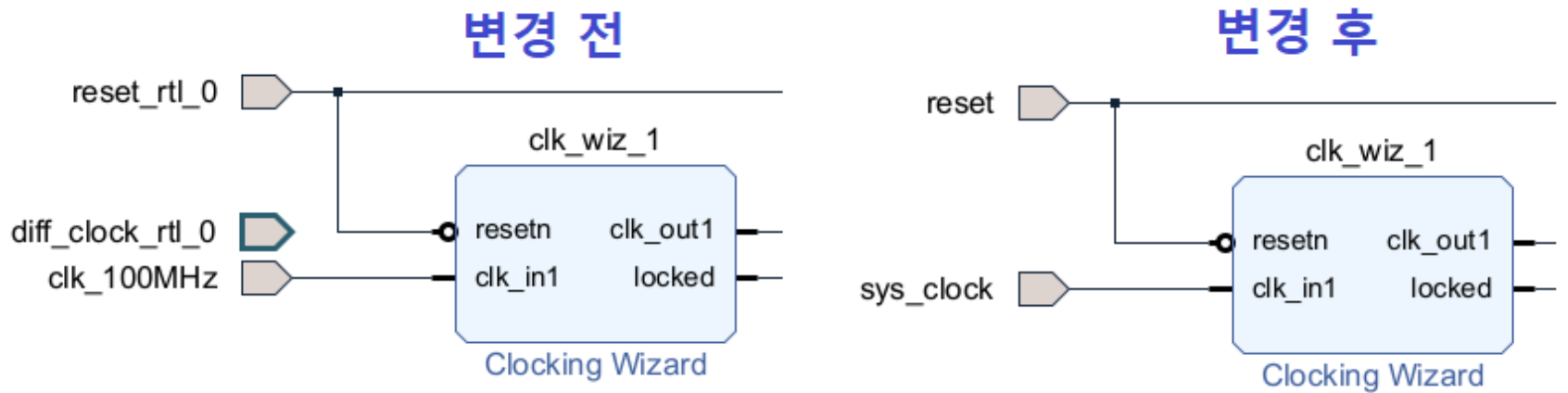
OK를 클릭합니다.

Run Connection Automation이 다시 활성화 되었습니다. reset_rtl_0 핀과 diff_clock_rtl_0 핀과 clk_wiz_1 을 서로 연결하기 위하여 Run Connection Automation을 클릭합니다. (수동으로 연결해도 됩니다)

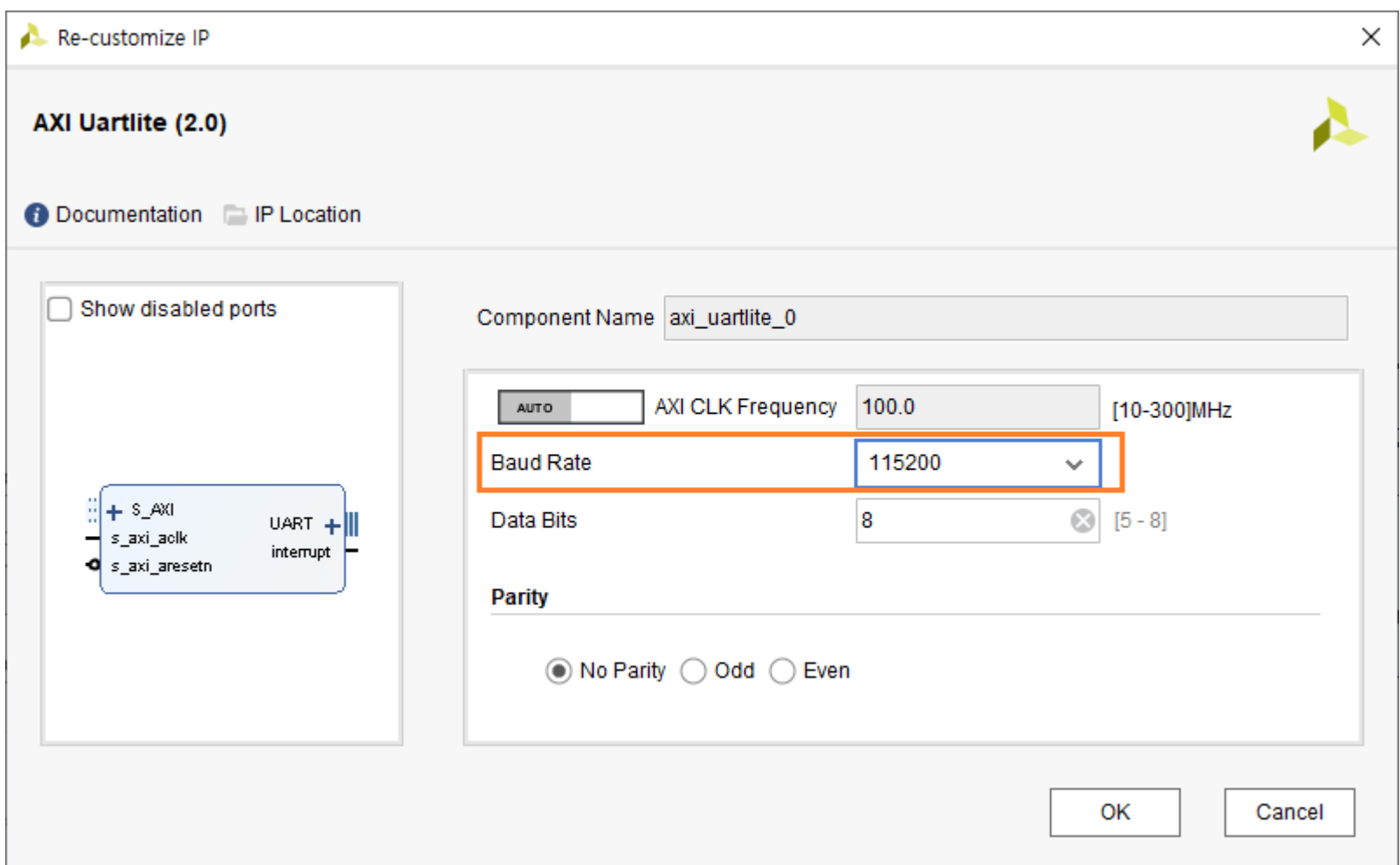
All Automation을 클릭하고 OK를 클릭합니다.



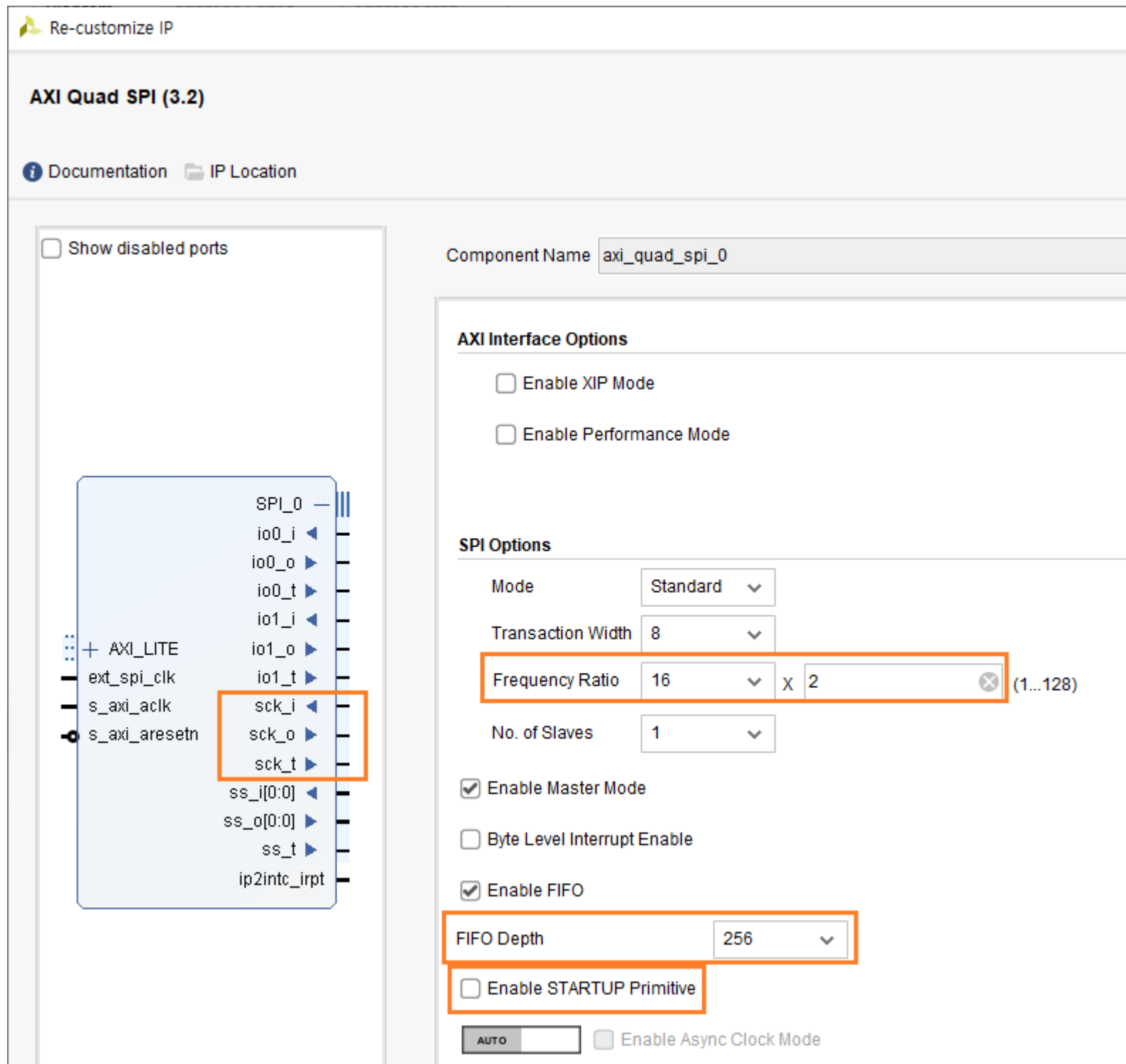
diff_clock_rtl_0를 삭제하고, reset_rtl_0 를 reset 으로, clk_100MHz를 sys_clock 으로 변경합니다. 포트 이름 변경은 포트를 선택하면 왼쪽에 Port Properties 윈도우가 나타납니다. Name을 변경하면 됩니다.



axi_uartlite_0 을 더블 클릭해서 속성을 변경합니다. Baud Rate : 115200 으로 변경하고 OK를 클릭합니다.



axi_quad_spi_0를 더블 클릭해서 속성을 변경합니다. spi 속성은 매우 중요합니다.



- ✓ Frequency Ratio : 16×2 , $100\text{MHz} / 32 = 3.125\text{MHz}$ (보드상에 점퍼로 연결해서 더 높이 설정하면 통신 에러 발생함)
- ✓ FIFO Depth : 16 or 256 지원함, 256 선택 (한번에 최대 전송할 수 있는 사이즈)
- ✓ Enable STARTUP Primitive : 반드시 uncheck 해야 우측의 sck 핀들이 나타남.

OK를 클릭합니다.

axi_gpio_0를 더블 클릭해서 속성을 변경합니다. gpio 핀은 총 5핀을 사용합니다. ([0] : w5500_reset, [1] ~ [4] : led)

- ✓ All Outputs : check
- ✓ GPIO Width : 5

Component Name

GPIO

☐ All Inputs

☒ All Outputs

GPIO Width [1 - 32]

Default Output Value [0x00000000,0xFFFFFFFF]

Default Tri State Value [0x00000000,0xFFFFFFFF]

☐ Enable Dual Channel

GPIO 2

☐ All Inputs

☐ All Outputs

GPIO Width [1 - 32]

Default Output Value [0x00000000,0xFFFFFFFF]

Default Tri State Value [0x00000000,0xFFFFFFFF]

☐ Enable Interrupt

OK를 클릭합니다.

포트 이름을 변경합니다.

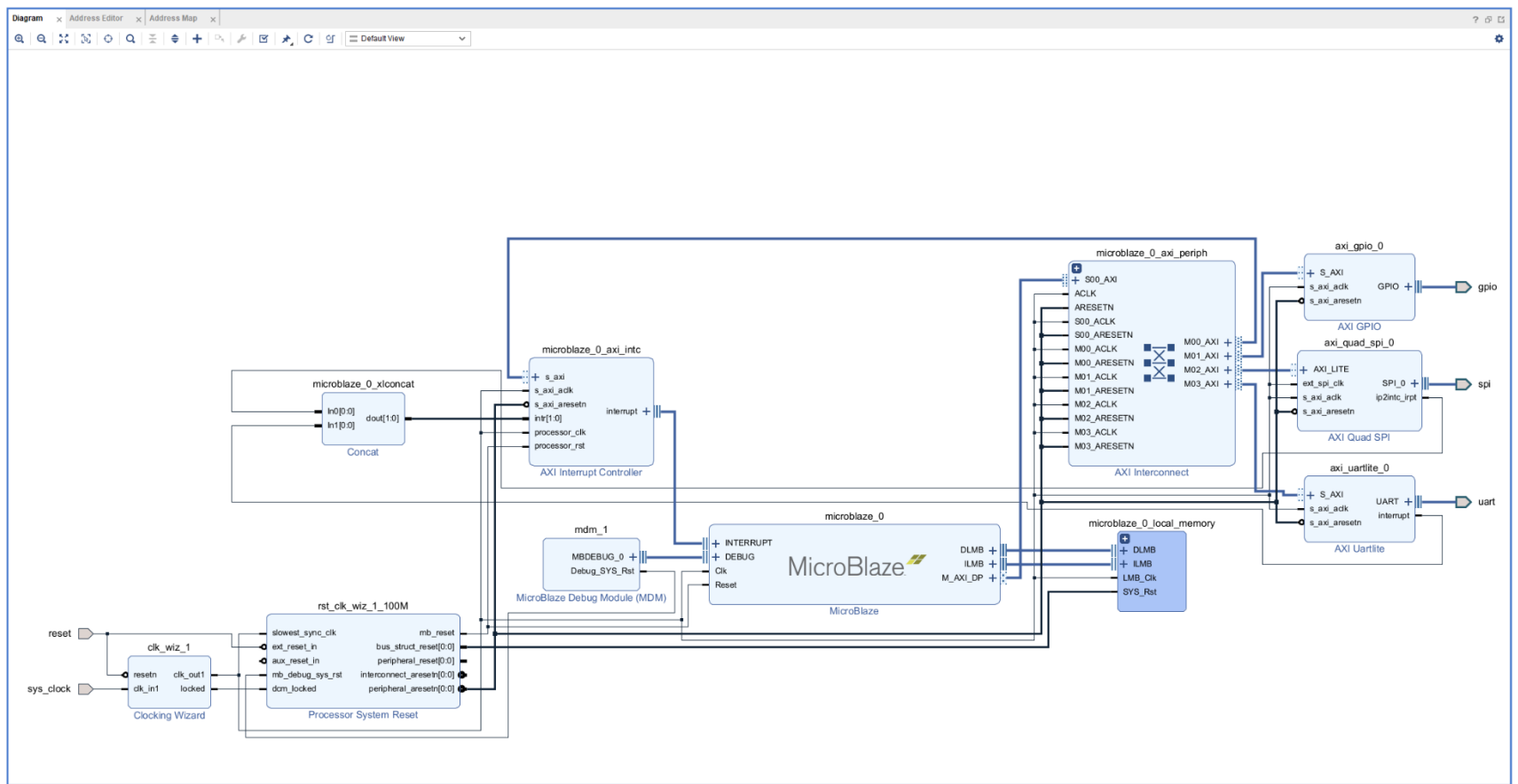
- ✓ gpio_rtl_0 -> gpio
- ✓ spi_rtl_0 -> spi
- ✓ uart_rtl_0 -> uart

인터럽트 핀들을 연결합니다.

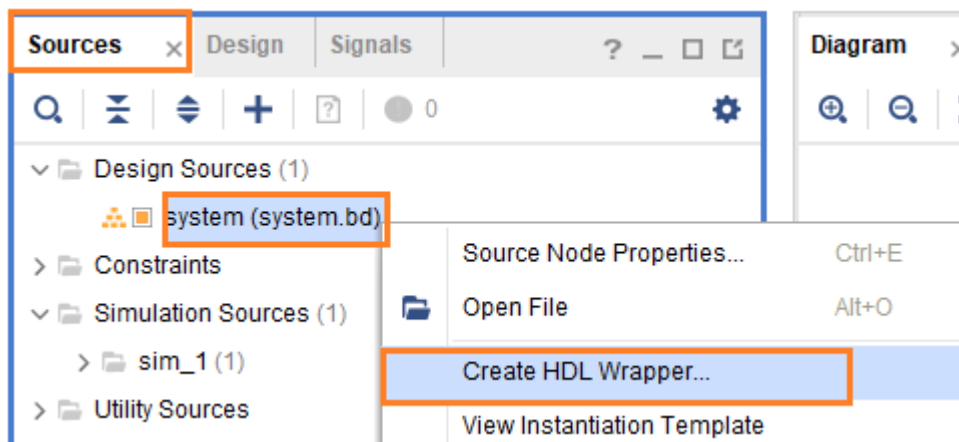
- ✓ microBlaze_0_xlconcat의 In0[0:0] 핀과 axi_quad_spi_0의 ip2intc_irpt 핀
- ✓ microBlaze_0_xlconcat의 In1[0:0] 핀과 axi_uartlite_0의 interrupt 핀

Validate Design, Regenerate layout 아이콘을 차례대로 클릭합니다. Validation successful 메시지 창을 확인합니다.

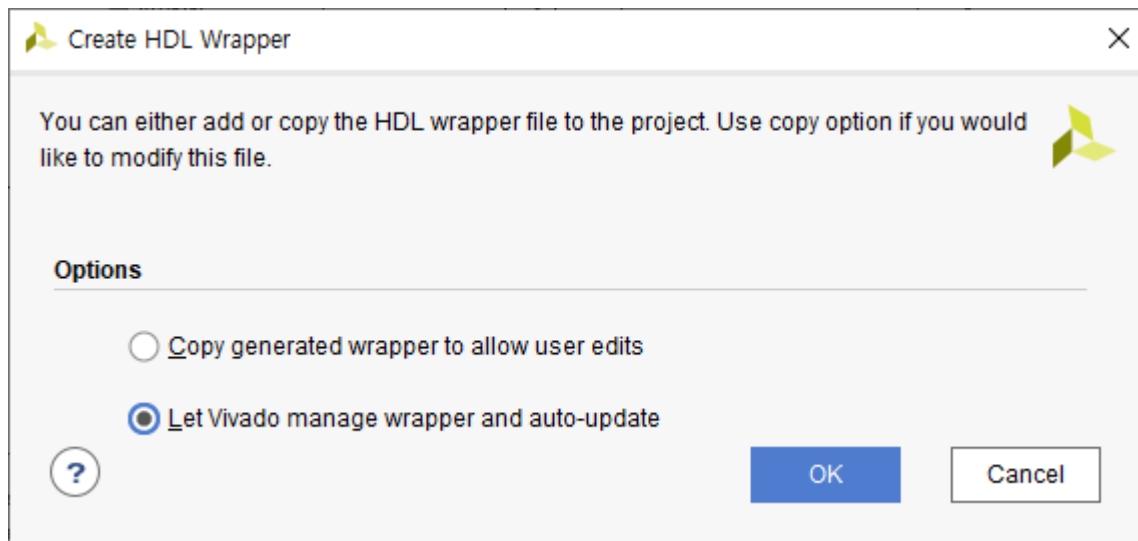
아래 그림은 지금까지 구현된 Diagram을 보여줍니다. (자료실에 Diagram 파일이 있으니 참조하시길 바랍니다)



Sources 탭에서 Design Sources - system - 우클릭 - Create HDL Wrapper... 를 클릭합니다.



OK를 클릭합니다.



HDL Wrapper 파일이 생성되었습니다. (system_wrapper.v)

다음 장에서는 Top Module을 구현하도록 하겠습니다.

IHIL

9.3 Top Module 구현

이번장에서는 앞장에서 생성한 system_wrapper.v 를 이용하여 Top Module을 생성하고 xdc 파일을 만들도록 하겠습니다.

Design Sources - 우클릭 - Add Sources... 클릭해서 Top Module을 추가합니다. 모듈 이름은 “blaze_w5500Top” 으로 하겠습니다. Add Sources 윈도우에서 Create File 버튼을 클릭해서 “blaze_w5500Top” 입력하고 OK를 클릭합니다. 모듈이 추가되면 Finish 버튼을 클릭합니다.

아래 코드를 blaze_w5500Top.v 에 추가합니다. (자료실에 코드가 있습니다.)

```

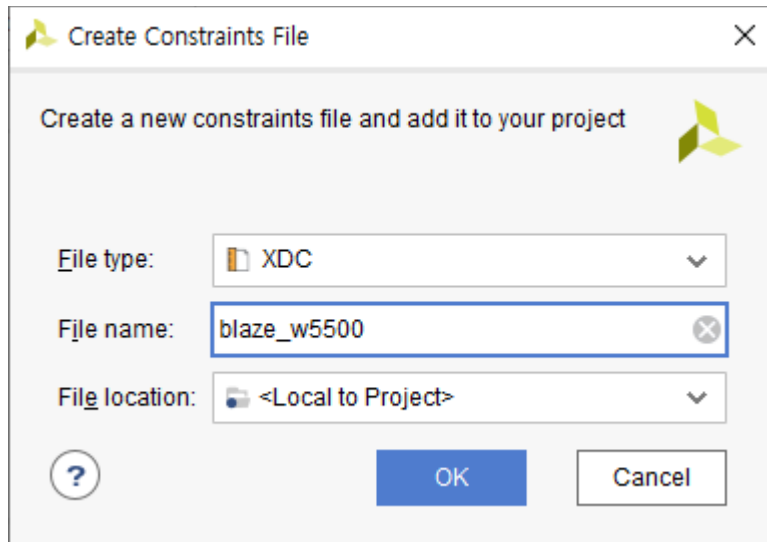
23 module blaze_w5500Top (
24     reset      ,
25     sys_clock  ,
26     uart_rxd   ,
27     uart_txd   ,
28     gpio_out   ,
29     spi_scs    ,
30     spi_sck    ,
31     spi_mosi   ,
32     spi_miso   ,
33 );
34 input      reset      ;
35 input      sys_clock  ;
36 input      uart_rxd   ;
37 output     uart_txd   ;
38 output [4:0] gpio_out ;
39 output     spi_scs    ;
40 output     spi_sck    ;
41 output     spi_mosi   ;
42 input      spi_miso   ;
43
44 wire       uart_txd   ;
45 wire [4:0] gpio_out   ;
46 wire       spi_scs    ;
47 wire       spi_sck    ;
48 wire       spi_mosi   ;
49
50 system system_i (
51     .reset      (reset      ),
52     .sys_clock  (sys_clock  ),
53     .uart_rxd   (uart_rxd   ),
54     .uart_txd   (uart_txd   ),
55     .gpio_tri_o (gpio_out   ),
56     .spi_io0_i  (1'b0      ),
57     .spi_io0_o  (spi_mosi   ),
58     .spi_io0_t  (           ),
59     .spi_iol_i  (spi_miso   ),
60     .spi_iol_o  (           ),
61     .spi_iol_t  (           ),
62     .spi_sck_i  (1'b0      ),
63     .spi_sck_o  (spi_sck    ),
64     .spi_sck_t  (           ),
65     .spi_ss_i   (1'b0      ),
66     .spi_ss_o   (spi_scs    ),
67     .spi_ss_t   (           )
68 );
69 endmodule

```

라인 23 - 42 : 모듈 선언, in/out port 선언 (uart, spi, gpio 가 추가되었습니다)

라인 44 - 67 : system 모듈 추가

xdc 파일을 생성합니다. Constraints - 우클릭 - Add Sources... 를 클릭합니다. Add Sources 윈도우에서 Create File을 클릭해서 “blaze_w5500”을 입력하고 OK를 클릭합니다.



Finish를 클릭하면 파일이 생성됩니다.

아래 코드를 blaze_w5500.xdc 파일에 추가합니다 (자료실에 파일을 참고하세요)

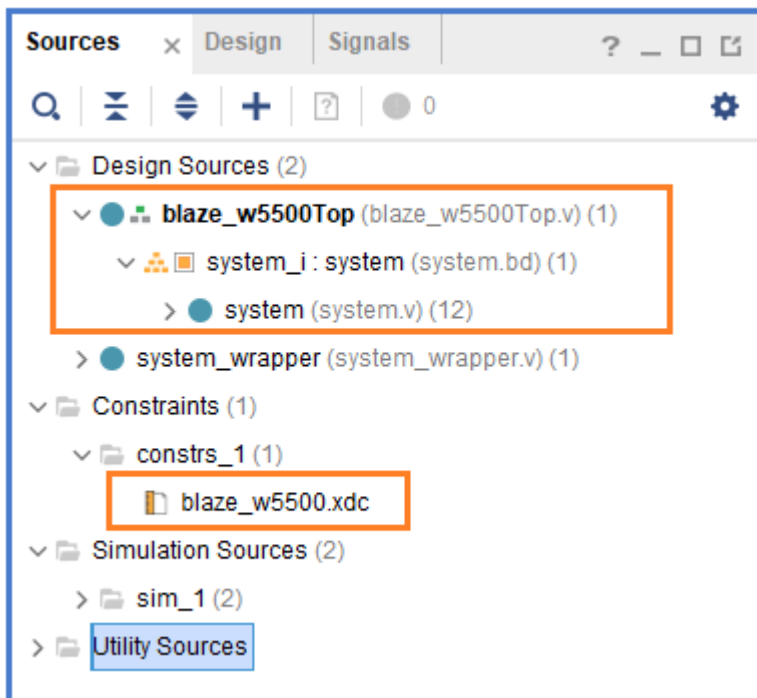
```

2 ## Clock signal
3 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clock }];
4 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clock }];
5
6 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
7
8 ## LEDs
9 set_property -dict { PACKAGE_PIN H5      IOSTANDARD LVCMOS33 } [get_ports { gpio_out[1] }]; #LED4
10 set_property -dict { PACKAGE_PIN J5      IOSTANDARD LVCMOS33 } [get_ports { gpio_out[2] }]; #LED5
11 set_property -dict { PACKAGE_PIN T9      IOSTANDARD LVCMOS33 } [get_ports { gpio_out[3] }]; #LED6
12 set_property -dict { PACKAGE_PIN T10     IOSTANDARD LVCMOS33 } [get_ports { gpio_out[4] }]; #LED7
13
14 ## Pmod Header JD
15 set_property -dict { PACKAGE_PIN D4      IOSTANDARD LVCMOS33 } [get_ports { spi_miso }]; # w5500_miso
16 set_property -dict { PACKAGE_PIN D3      IOSTANDARD LVCMOS33 } [get_ports { spi_mosi }]; # w5500_mosi
17 set_property -dict { PACKAGE_PIN F4      IOSTANDARD LVCMOS33 } [get_ports { spi_scs }]; # w5500_scs
18 set_property -dict { PACKAGE_PIN F3      IOSTANDARD LVCMOS33 } [get_ports { spi_sck }]; # w5500_sck
19 set_property -dict { PACKAGE_PIN E2      IOSTANDARD LVCMOS33 } [get_ports { gpio_out[0] }]; # w5500_rst
20
21 ## Pmod Header JC
22 set_property -dict { PACKAGE_PIN V10     IOSTANDARD LVCMOS33 } [get_ports { uart_rxd }];
23 set_property -dict { PACKAGE_PIN V11     IOSTANDARD LVCMOS33 } [get_ports { uart_txd }];
24
25
26 #####
27 # MISC
28 #####
29 set_operating_conditions -ambient_temp 80.0
30
31 set_property CONFIG_MODE SPIx1 [current_design]
32 set_property CFGBVS VCC0 [current_design]
33 set_property CONFIG_VOLTAGE 3.3 [current_design]
34
35 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
36 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
37 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
38 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

```

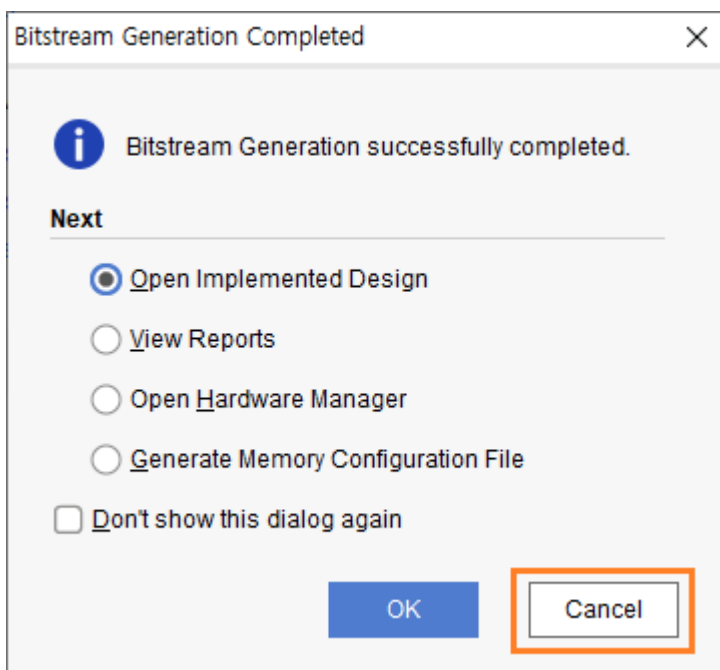
gpio_out[0]를 w5500_rst 핀으로 사용한 것에 유의하시길 바랍니다.

blaze_w5500Top 모듈을 Top 모듈로 지정합니다. 아래는 최종 파일 구조를 보여줍니다.

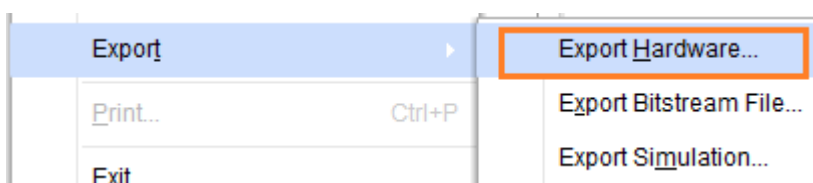


모든 파일이 준비되었습니다. PROGRAM AND DEBUG - Generate Bitstream을 클릭해서 Bitstream을 생성합니다. 각 IP들을 생성하고, Synthesis, Implementation 까지 진행해서 Bitstream을 생성합니다.

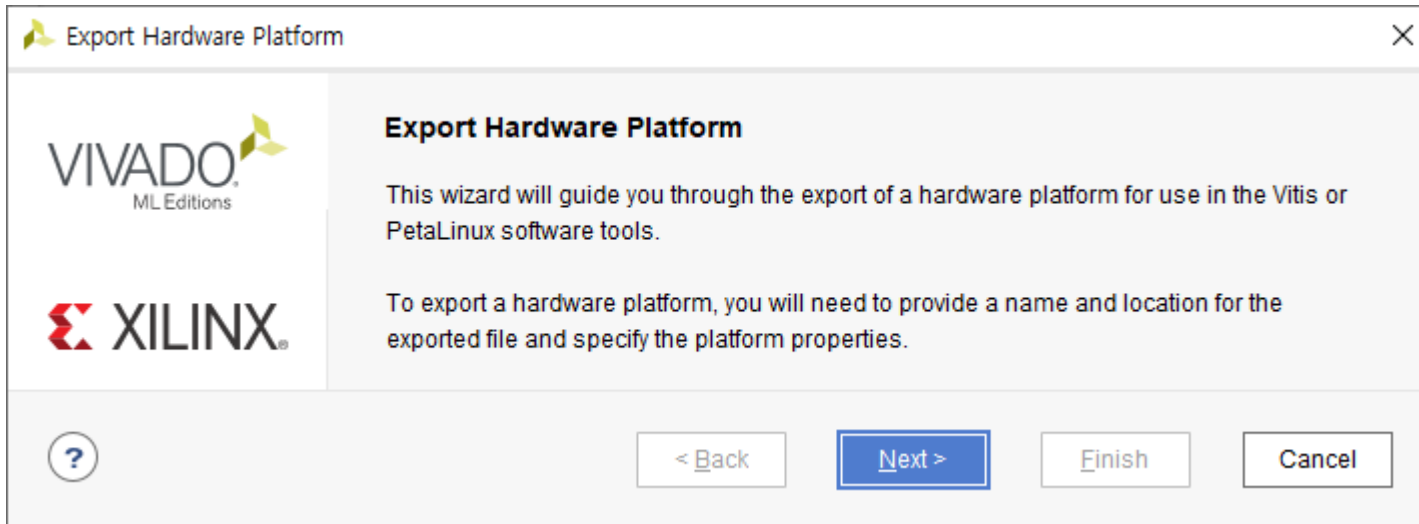
Bitstream이 생성되면, 완료 윈도우가 나타납니다. Cancel 을 클릭합니다.



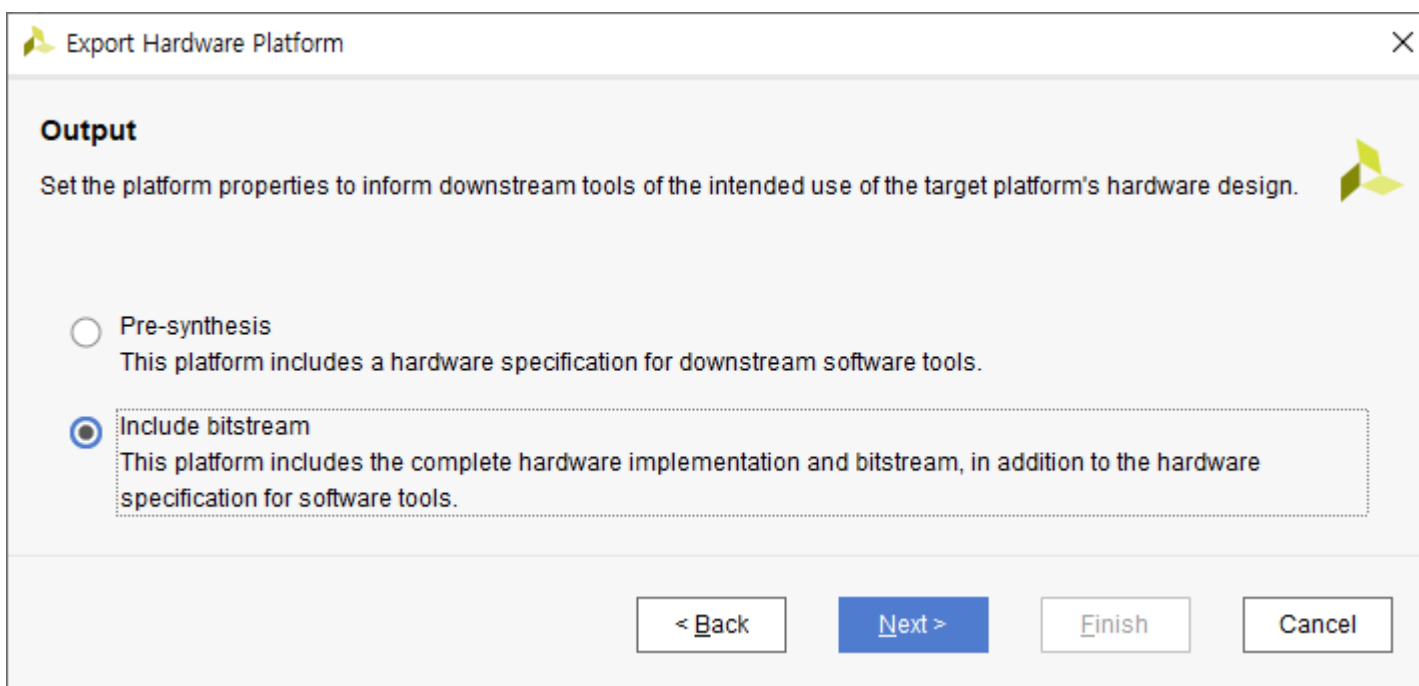
xsa 파일을 생성하기 위하여 File - Export - Export Hardware... 를 클릭합니다.



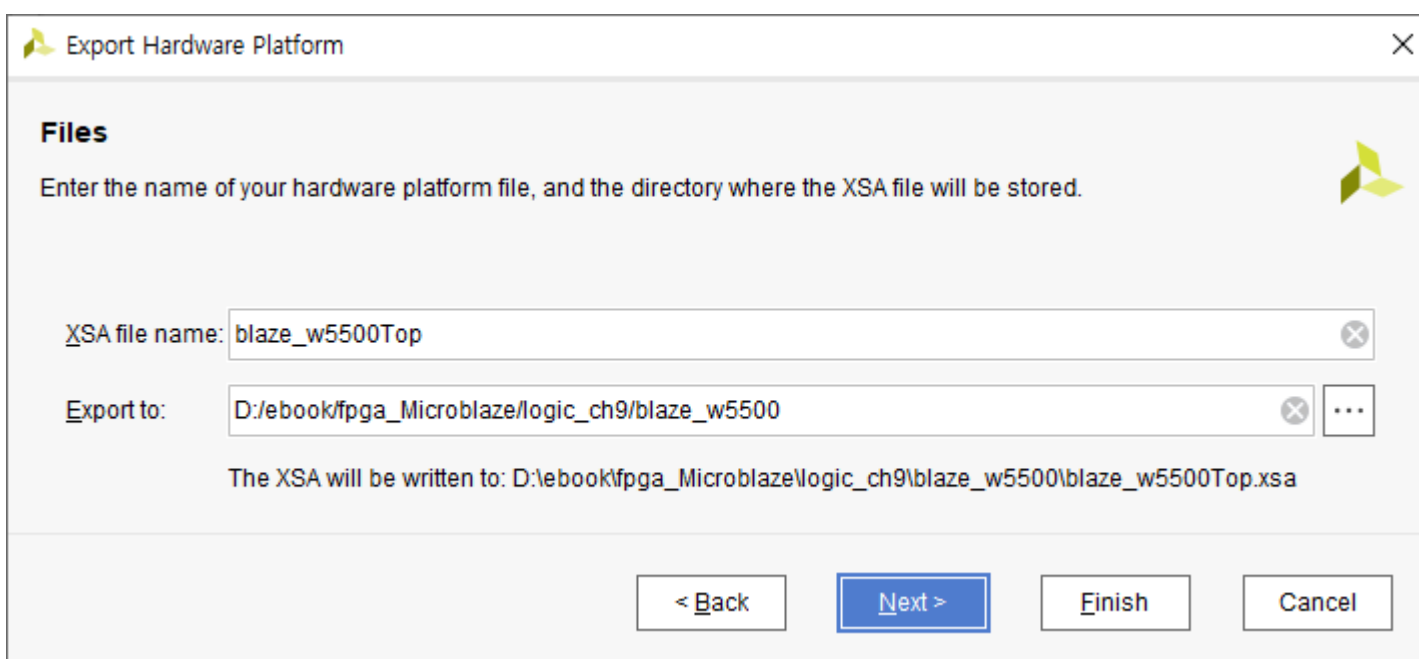
Next를 클릭합니다.



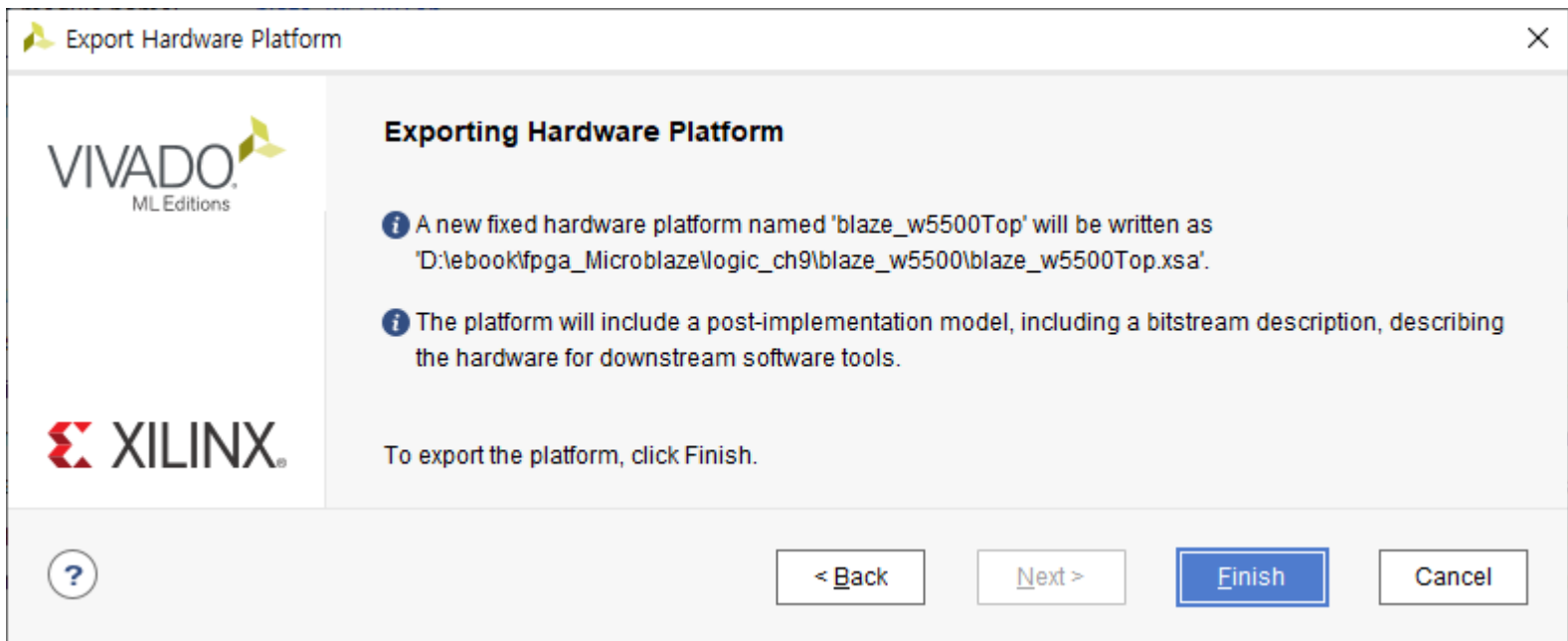
Include bitstream 을 선택하고 Next를 클릭합니다.



파일명, 위치를 확인하고 Next를 클릭합니다.



마지막으로 Finish를 클릭하면 xsa 파일이 생성됩니다.



다음장에서는 vitis 에서 sw를 구현합니다.

IIHIL

9.4 응용 SW 구현

이번 장에서는 vitis을 이용하여 응용 SW를 구현합니다. 먼저 w5500의 spi 타이밍을 간단하게 살펴보겠습니다.

w5500에서 지원하는 SPI Mode는 0, 3 입니다. Mode 0는 Idle 상태에서 SCLK 은 Low 이고, SCLK Falling Edge 에서 데이터(MISO/MOSI)가 변합니다. 데이터를 읽을 때에는 SCLK Rising Edge에서 읽습니다.

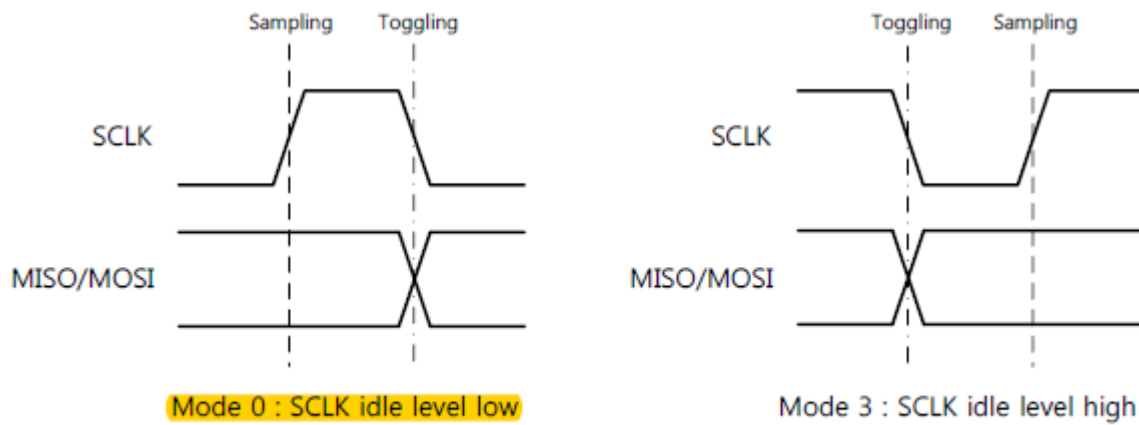


Figure 6. SPI Mode 0 & 3

아래 그림은 SPI Frame Format을 나타냅니다. Address Phase(16bits), Control Phase(8bits), Data Phase (8bits x N)으로 구성되고 MSB 부터 전송합니다.

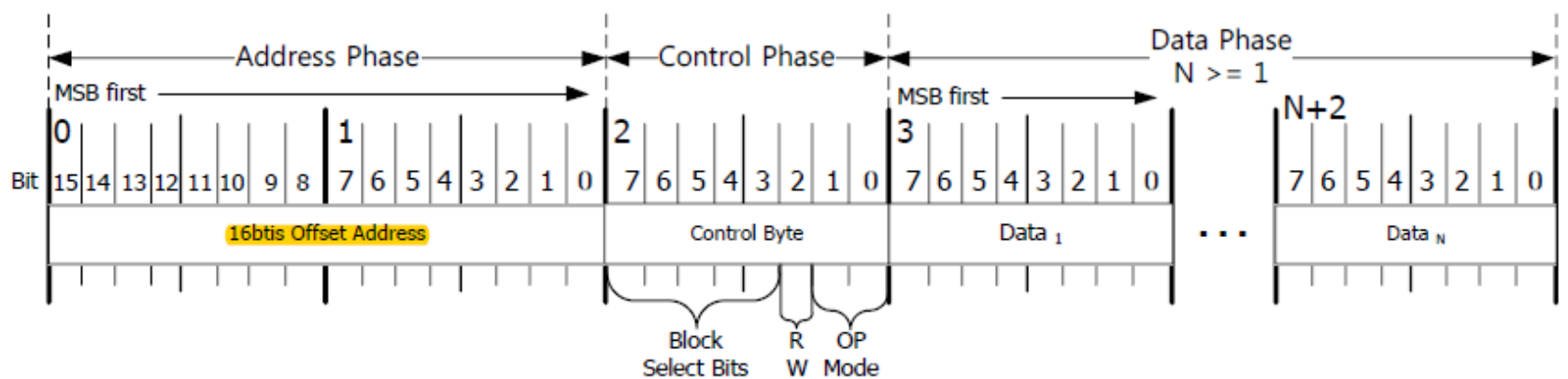
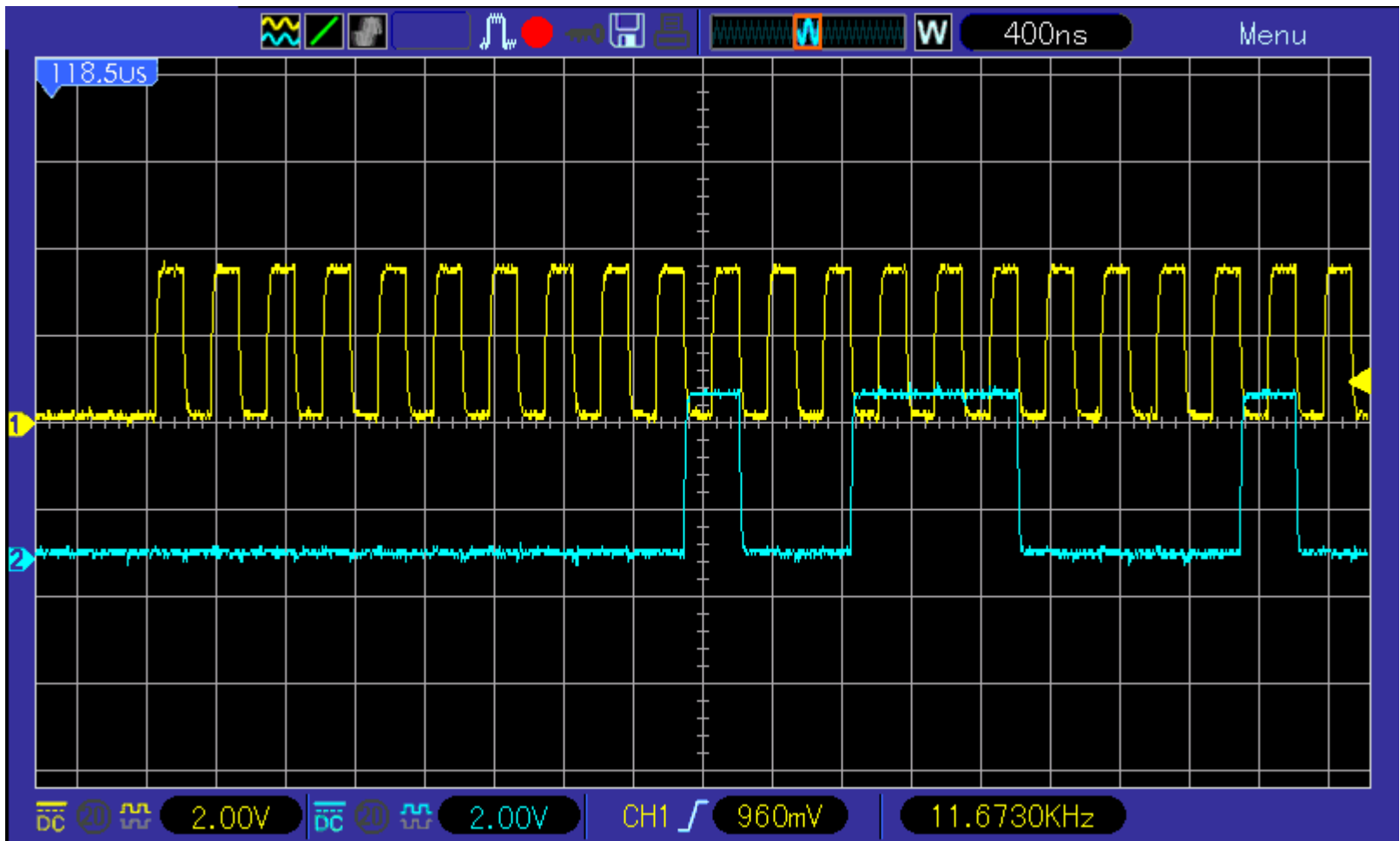
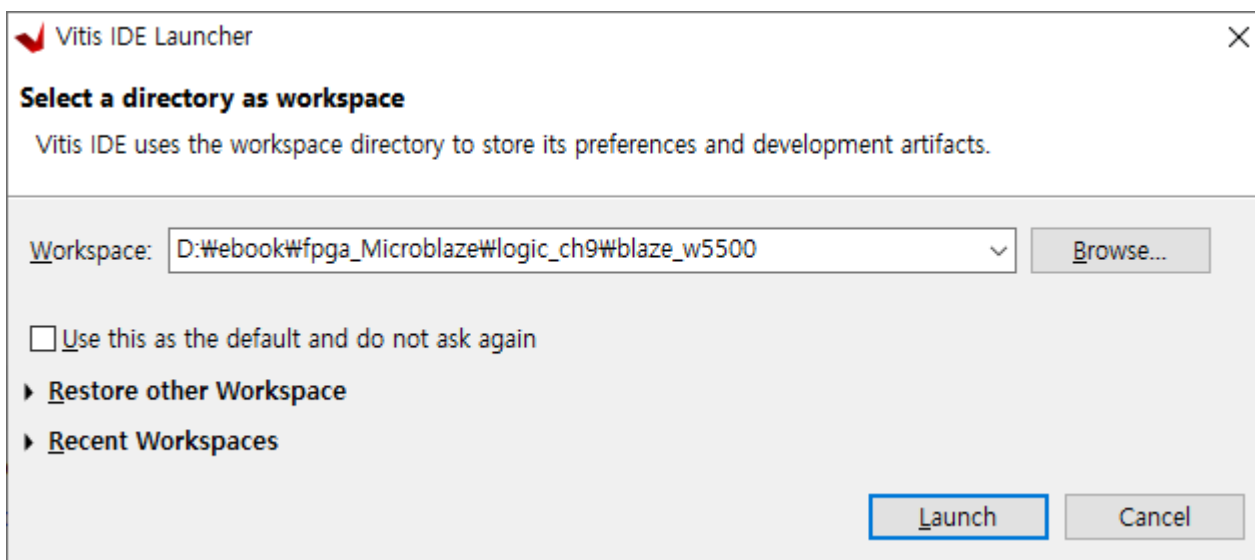


Figure 7. SPI Frame Format

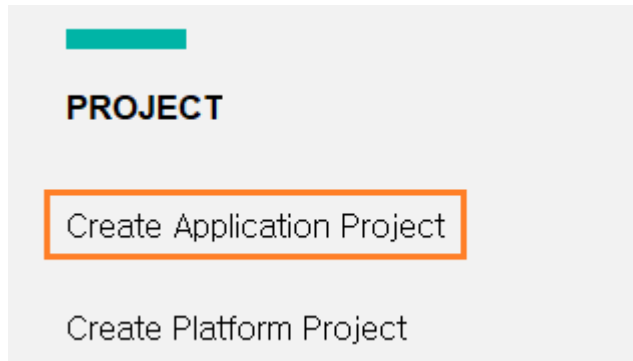
아래 그림은 AXI SPI 파형을 보여줍니다. xilinx 사에서 제공하는 user manual (AXI Quad SPI v3.2) 과 차이가 있습니다. 아래 파형은 sw 구현후에 PC와의 데이터 통신을 하는 파형을 스코프로 측정하였습니다. Falling Edge 에서 데이터가 변하고, Rising Edge 에서 데이터를 읽으면 됩니다. (채널 1 : SCLK, 채널 2 : MOSI 입니다)



vitis를 실행하기 위하여 Tools - Launch Vitis IDE 를 클릭합니다. 프로젝트 위치를 확인하고 Launch 버튼을 클릭하면 vitis가 실행됩니다.

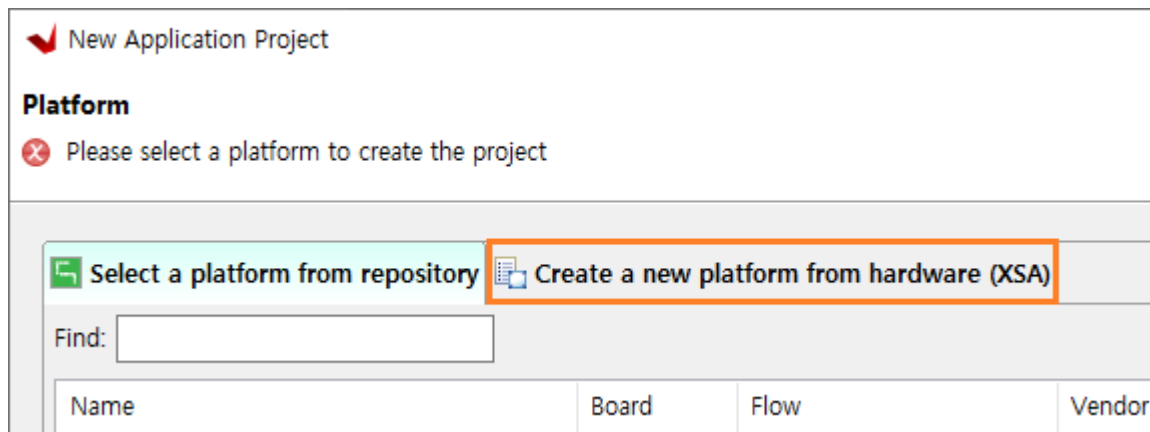


PROJECT - Create Application Project를 클릭합니다.

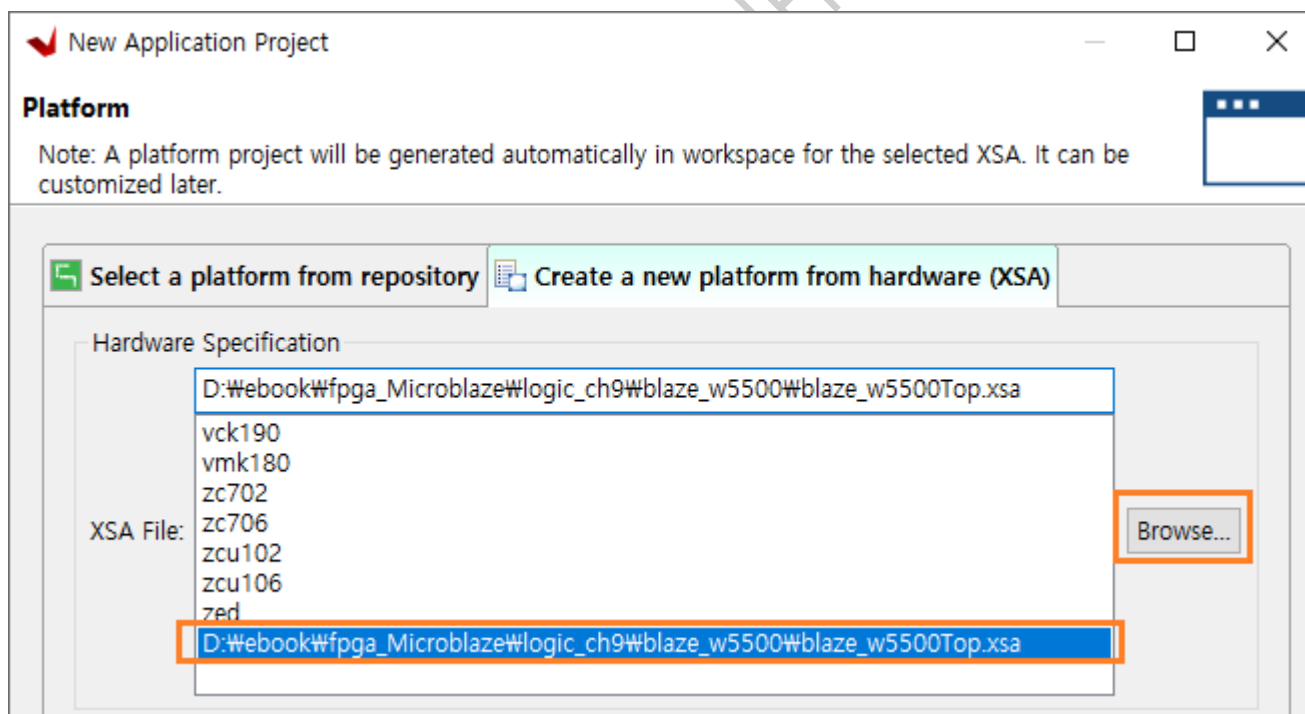


Create a New Application Project 윈도우에서 Next를 클릭합니다.

Platform 윈도우에서 Create a new platform from hardware (XSA)를 클릭합니다.



XSA File : 우측의 Browse... 를 클릭하여 생성한 blaze_w5500.xsa 파일을 선택합니다. Next 버튼을 클릭합니다.



Application project name 에 “blaze_w5500App”을 입력하고 Next를 클릭합니다.

New Application Project

Application Project Details
Specify the application project name and its system project properties

Application project name: blaze_w5500App

System Project
Create a new system project for the application or select an existing one from the workspace

Select a system project
+ Create new...

System project details

System project name: blaze_w5500App_system

Target processor

Select target processor for the Application project.

Processor	Associated applications
microblaze_0	blaze_w5500App

Show all processors in the hardware specification ☒

< Back Next > Finish Cancel

Next를 클릭합니다.

New Application Project

Domain
Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

Select a domain
+ Create new...

Domain details

Name: standalone_microblaze_0

Display Name: standalone_microblaze_0

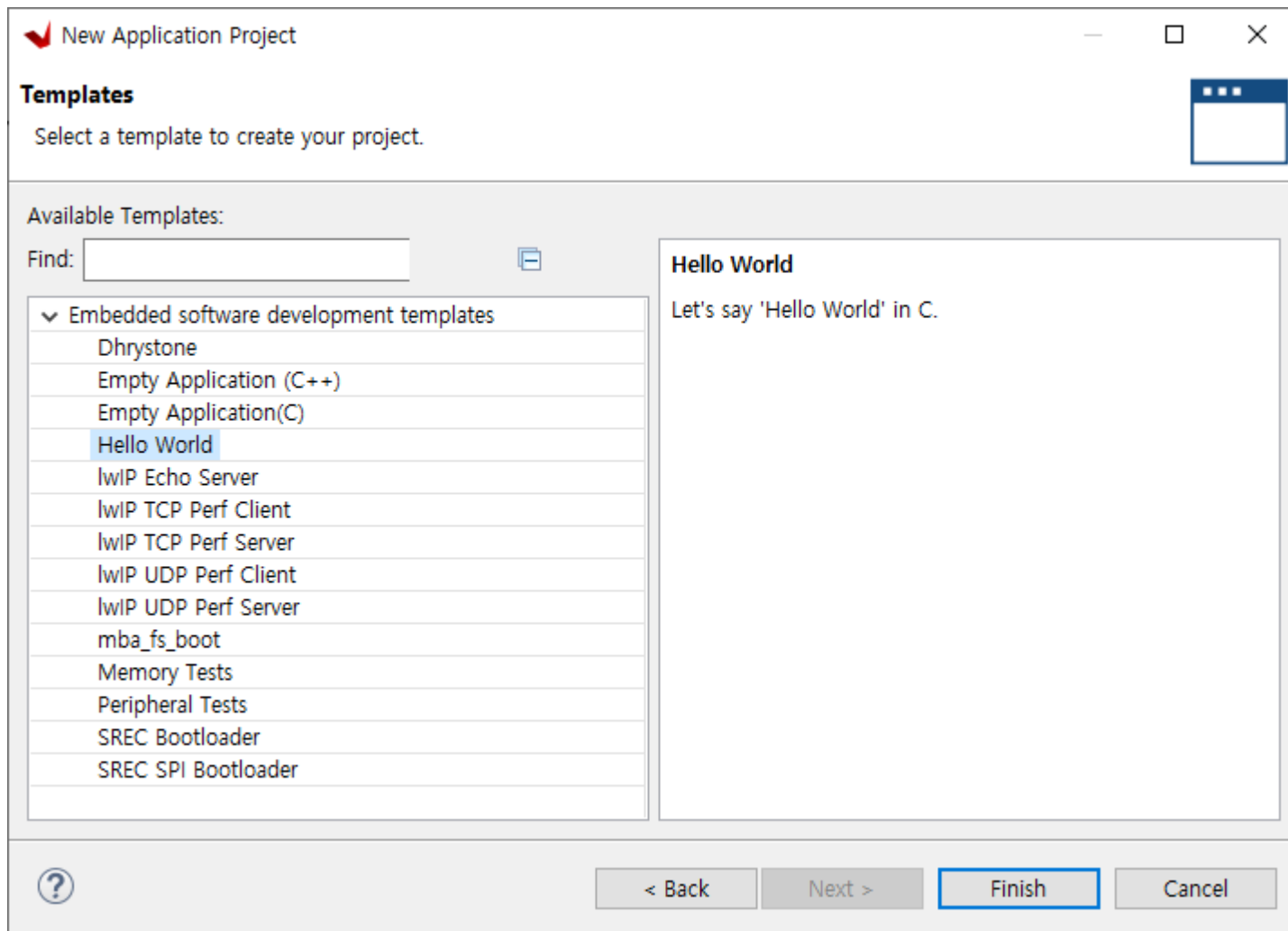
Operating System: standalone

Processor: microblaze_0

Architecture: 32-bit

< Back Next > Finish Cancel

Template 에서 Hello World를 선택하고 Finish를 클릭합니다.



프로젝트가 생성되었습니다. 이제 wiznet 사에서 제공하는 API를 이용하여 w5500 tcp/ip를 구현하도록 하겠습니다. API 는 자료실에서 “W5500_EVB-master.zip” 파일을 다운로드 받으면 됩니다. 필요한 파일은 아래와 같습니다.

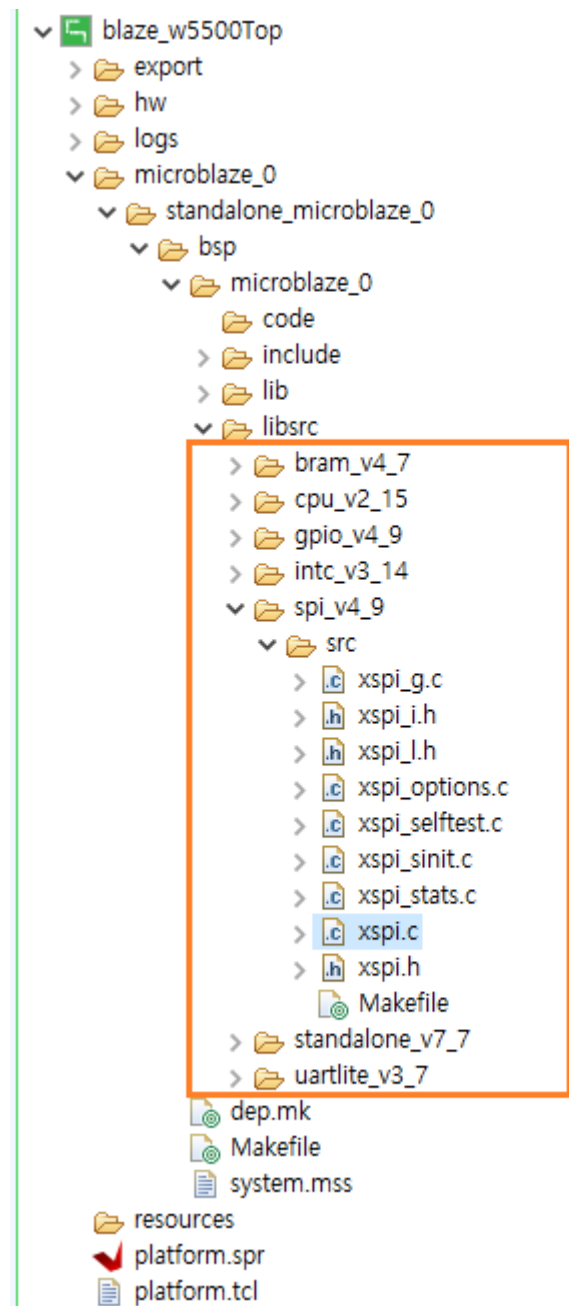
- ✓ loopback.c, loopback.h : W5500_EVB-master\WioLibrary\WAppmod\Loopback\ 폴더
- ✓ socket.c, socket.h : W5500_EVB-master\WioLibrary\WEthernet\ 폴더
- ✓ wizchip_conf.c, wizchip_conf.h : W5500_EVB-master\WioLibrary\WEthernet\ 폴더
- ✓ w5500.c, w5500.h : W5500_EVB-master\WioLibrary\WEthernet\W5500\ 폴더

w5500 API를 포팅하는 것은 인터넷에서 자료를 찾아서 참조하시길 바랍니다. 본 강의에서는 대략적인 내용만 설명하도록 하겠습니다.

w5500의 API를 포팅하기 위해서는 상위 API들은 그대로 활용하고, 하위 API를 사용자의 시스템에 맞게 변경해 주어야 합니다. 특별히 SPI 인터페이스 API를 사용자의 시스템에 맞게 변경해 주면 됩니다. SPI 관련 API 들은 다음의 4가지가 있습니다. (w5500.c 파일안에 있습니다)

- ✓ uint8_t WIZCHIP_READ(uint32_t AddrSel) : 1-바이트 read
- ✓ void WIZCHIP_WRITE(uint32_t AddrSel, uint8_t wb) : 1-바이트 write
- ✓ void WIZCHIP_READ_BUF(uint32_t AddrSel, uint8_t* pBuf, uint16_t len) : n-바이트 read
- ✓ void WIZCHIP_WRITE_BUF(uint32_t AddrSel, uint8_t* pBuf, uint16_t len) : n-바이트 write

위 4개의 함수들을 우리가 사용하는 “Xspi_Transfer” 함수로 변환해 주면 됩니다. Xspi_Transfer 함수는 xspi.c 안에 정의되어 있습니다. 파일 위치는 아래와 같습니다.



아래는 Xspi_Transfer 함수를 보여줍니다. (xspi.c)

```
525 int Xspi_Transfer(XSpi *InstancePtr, u8 *SendBufPtr,
526                  u8 *RecvBufPtr, unsigned int ByteCount)
527 {
```

- ✓ InstancePtr : Xspi instance pointer
- ✓ SendBufPtr : 전송할 데이터의 시작 주소
- ✓ RecvBufPtr : 수신한 데이터를 저장할 변수의 시작 주소, Write시에는 NULL로 지정
- ✓ ByteCount : 전송할 데이터의 바이트 수

아래 그림은 w5500 spi frame 구조를 보여줍니다.

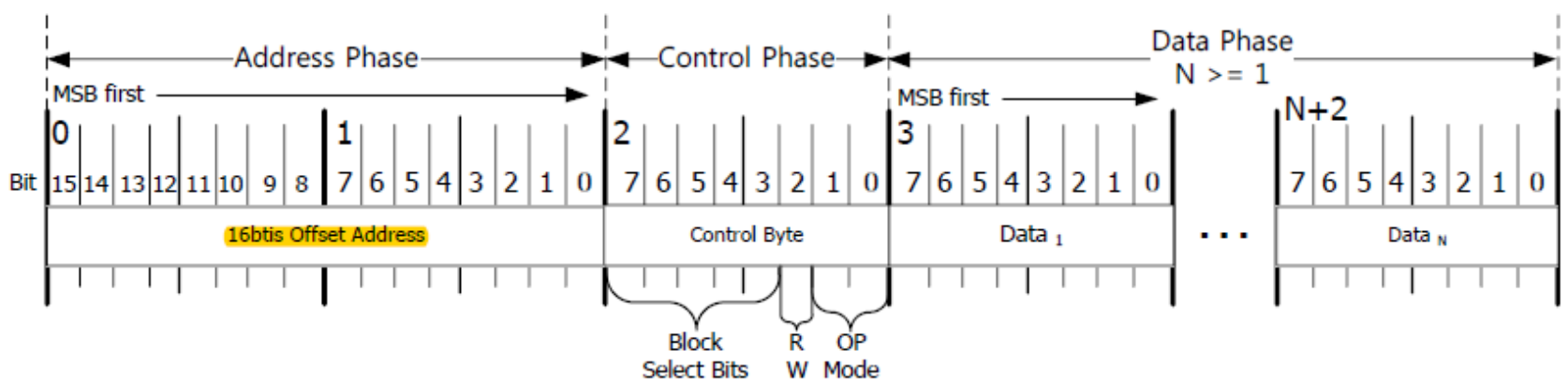


Figure 7. SPI Frame Format

w5500에 1-바이트 Data를 전송하기 위해서는 총 4바이트(Address Phase : 2바이트 + Control Phase : 1바이트 + Data Phase : 1바이트)를 전송해야 합니다. w5500에 N-바이트 데이터를 전송하기 위해서는 총 N+3 바이트를 전송해야 합니다. 따라서 write시에 ByteCount 값은 “전송할 데이터 사이즈 + 3”을 해 주어야 합니다.

w5500에서 read 하는 경우는 더욱 복잡합니다. Xspi_Transfer 함수는 데이터를 전송할 때, miso 핀으로 들어오는 데이터를 처음부터 읽게 됩니다. w5500의 spi read 는 Address Phase, Control Phase를 지나고 Data Phase 에 miso 핀으로 들어오는 데이터를 read data로 인식하는데, Xspi_Transfer 함수는 Address Phase부터 miso 핀으로 들어오는 데이터를 read data로 인식합니다. 따라서 Xspi_Transfer 함수의 read 부분을 수정할 필요가 있습니다. 즉 Address Phase, Control Phase의 read data는 버리고, Data Phase에서 수신되는 데이터만 RecvBufPtr에 저장하도록 해야합니다.

Xspi_Transfer 함수를 아래와 같이 수정합니다.

```

525 int Xspi_Transfer(Xspi *InstancePtr, u8 *SendBufPtr,
526                  u8 *RecvBufPtr, unsigned int ByteCount)
527 {
528     u32 ControlReg;
529     u32 GlobalIntrReg;
530     u32 StatusReg;
531     u32 Data = 0;
532     u8 DataWidth;
533     u32 DataLen;
534     u32 Index;
535     u32 rcv_index=0;
536

```

✓ 라인 535 : 수신된 데이터를 count 하기 위한 변수 생성.

```

716         while ((StatusReg & XSP_SR_RX_EMPTY_MASK) == 0) {
717
718             Data = Xspi_ReadReg(InstancePtr->BaseAddr,
719                               XSP_DRR_OFFSET);
720             if (DataWidth == XSP_DATAWIDTH_BYTE) {
721                 /*
722                  * Data Transfer Width is Byte (8 bit).
723                  */
724                 if(InstancePtr->RecvBufferPtr != NULL) {
725                     rcv_index++;
726                     if(rcv_index>3)
727                     {
728                         *InstancePtr->RecvBufferPtr++ = (u8)Data;
729                     }
730                 }

```

✓ 라인 725 - 729 : 수신된 데이터의 처음 3-바이트는 버리고 4번째 바이트부터 저장함.

```

599 /*
600  * Set up buffer pointers.
601  */
602 InstancePtr->SendBufferPtr = SendBufPtr;
603 InstancePtr->RecvBufferPtr = RecvBufPtr;
604
605 InstancePtr->RequestedBytes = ByteCount;
606 InstancePtr->RemainingBytes = ByteCount;

```

✓ 라인 603 : InstancePtr->RecvBufferPtr = RecvBufPtr, 두 포인터 변수가 동일함

여기서 한가지 주의할 점이 있습니다. xspi.c 는 bsp 폴더 안에 있습니다. 즉 vivado 에서 IP를 생성할 때 자동으로 생성되는 파일입니다. 따라서 vivado에서 Design을 수정후에, vitis에서 업데이트된 내용을 적용하기 위하여 “Update Hardware Specification”을 클릭하면 수정했던 코드(xspi.c)가 날라갑니다. 그렇기 때문에 xspi.c 파일을 수정후에는 반드시 다른 폴더에 저장해 둔 후에 다시 복사해 주어야 합니다. 이 과정이 매끄럽지 않아서 시퀀스를 정리합니다. 참조하시길 바랍니다.

- 1) vivado 에서 design 수정 (bitstream, xsa 파일 생성)
- 2) blaze_w5500Top - 우클릭 - Update Hardware Specification 클릭 - 업데이트 된 xsa 파일 지정
- 3) blaze_w5500Top - 우클릭 - Clean Project
- 4) blaze_w5500Top - 우클릭 - Build Project
- 5) 백업해둔 xspi.c 파일 해당 폴더(bsp\microblaze_0\libsrc\spi_v4_9\src)로 복사
- 6) 다시 blaze_w5500Top - 우클릭 - Build Project

w5500_write / w5500_read 함수를 구현합니다. (w5500.c)

```

28 uint8_t  tcpRxBuffer[SPI_FIFO_SIZE];
29 uint8_t  tcpTxBuffer[SPI_FIFO_SIZE];
30
31 void w5500_write(uint16_t addr16, uint8_t bsb, uint16_t data_len, uint8_t *dat8)
32 {
33     uint16_t i;
34
35     tcpTxBuffer[0] = (uint8_t)(addr16>>8);
36     tcpTxBuffer[1] = (uint8_t)(addr16&0xff);
37     tcpTxBuffer[2] = (bsb<<3) | (1<<2);
38     for(i=0; i<data_len; i++) tcpTxBuffer[i+3] = dat8[i];
39
40     XSpi_Transfer(&SpiInstance, tcpTxBuffer, NULL, data_len+3);
41     //xil_printf("spi wrtie ..\r\n");
42 }
43
44 void w5500_read(uint16_t addr16, uint8_t bsb, uint16_t data_len, uint8_t *rbuf)
45 {
46     tcpTxBuffer[0] = (uint8_t)(addr16>>8);
47     tcpTxBuffer[1] = (uint8_t)(addr16&0xff);
48     tcpTxBuffer[2] = (bsb<<3);
49
50     XSpi_Transfer(&SpiInstance, tcpTxBuffer, rbuf, data_len+3);
51     //xil_printf("spi read ..\r\n");
52 }

```

- ✓ 라인 28 - 29 : tx, rx에 사용할 버퍼를 정의합니다. SPI_FIFO_SIZE는 256 입니다. vivado 에서 AXI SPI를 설정시 FIFO를 256으로 설정하였습니다. 한번에 최대 256바이트까지 전송할 수 있습니다.
- ✓ 라인 31 - 42 : w5500_wrtie 함수를 구현합니다. addr16은 Address, bsp는 control, data_len은 전송할 메시지의 바이트 수입니다. Xspi_Transfer에서 전송할 바이트 수는 “data_len + 3” 이 됩니다. write 시에는 Xspi_Transfer 함수의 3번째 인자는 NULL로 설정합니다.
- ✓ 라인 44 - 52 : w5500_read 함수를 구현합니다. addr16은 Address, bsp는 control, data_len은 read할 데이터의 바이트 수입니다. Xspi_Transfer에서 전송할 바이트 수는 “data_len + 3” 이 됩니다.

이제 이 함수를 이용하여 WIZCHIP_READ ~ WIZCHIP_WRITE_BUF 함수를 구현하도록 하겠습니다.

```

54 void WIZCHIP_WRITE(uint32_t AddrSel, uint8_t wb )
55 {
56     w5500_write( (uint16_t)((AddrSel>>8)&0xffff), (uint8_t)((AddrSel>>3)&0x1f), 1, &wb);
57 }
58
59 void WIZCHIP_WRITE_BUF(uint32_t AddrSel, uint8_t* pBuf, uint16_t len)
60 {
61     w5500_write( (uint16_t)((AddrSel>>8)&0xffff), (uint8_t)((AddrSel>>3)&0x1f), len, pBuf);
62 }
63
64 uint8_t WIZCHIP_READ(uint32_t AddrSel)
65 {
66     uint8_t rbuf[6];
67
68     w5500_read( (uint16_t)((AddrSel>>8)&0xffff), (uint8_t)((AddrSel>>3)&0x1f), 1, rbuf);
69
70     return(rbuf[0]);
71 }
72
73 void WIZCHIP_READ_BUF (uint32_t AddrSel, uint8_t* pBuf, uint16_t len)
74 {
75     w5500_read( (uint16_t)((AddrSel>>8)&0xffff), (uint8_t)((AddrSel>>3)&0x1f), len, pBuf);
76 }

```

- ✓ 라인 54 - 57 : WIZCHIP_WRITE 함수를 구현합니다. 1-바이트를 write 합니다.
- ✓ 라인 59 - 62 : WIZCHIP_WRITE_BUF 함수를 구현합니다. N-바이트를 write 합니다.
- ✓ 라인 64 - 71 : WIZCHIP_READ 함수를 구현합니다. 1-바이트를 read 합니다.
- ✓ 라인 73 - 76 : WIZCHIP_READ_BUF 함수를 구현합니다. N-바이트를 read 합니다.

w5500 API들은 모두 WIZCHIP_WRITE ~ WIZCHIP_READ_BUF를 사용해서 구현되었기 때문에 w5500 API를 그대로 사용할 수 있습니다.

9.4.1 Memory Size

Linker Script (lscript.ld) 파일을 클릭하면 Stack, Heap Size를 확인할 수 있습니다. 현재 각각 0x400 (1KB), 0x800 (2KB)로 설정되어 있습니다.

Linker Script: lscript.ld

A linker script is used to control where different sections of an executable are placed in memory. In this page, you can define new memory regions, and change the assignment of sections to memory regions.

Available Memory Regions

Name	Base Address	Size
microblaze_0_local_memory_ilmb_bram_if_cntlr_Mem_microb...	0x50	0xFFB0

Stack and Heap Sizes

Stack Size	0x400
Heap Size	0x800

전역변수로 사용가능한 Memory는 2KB 입니다. 또한 AXI SPI 는 최대 256 바이트의 FIFO를 가지고 있습니다. AXI SPI로 한 번에 전송할 수 있는 바이트는 256 바이트입니다. 그러나 w5500은 data를 전송하기 전에 3바이트(Address 2바이트, Control 1바이트)을 전송하기 때문에 253바이트까지 전송할 수 있습니다. 본 강의에서는 한 번에 최대 240바이트를 전송하도록 하겠습니다. PC와 Network 으로 연결 후, TCP/IP 통신에서는 1200 바이트(240바이트 x 5)를 송수신하는 것을 구현합니다. 즉 PC에서 보드로 1200바이트를 전송하면, AXI SPI는 240바이트씩 5번 수신합니다. 보드에서 PC로 1200바이트를 전송할 때에도, AXI SPI는 240바이트씩 5번 전송합니다.

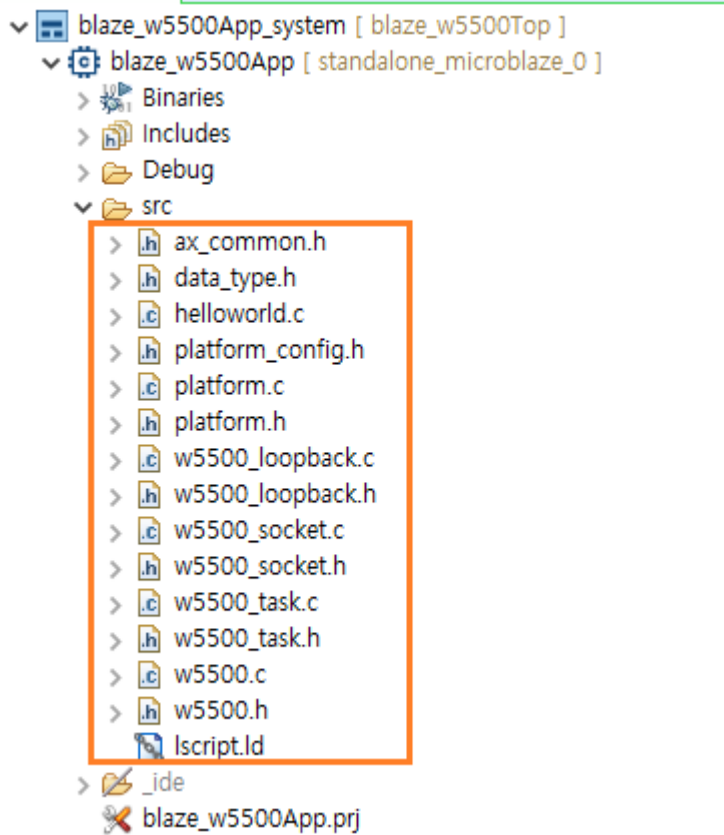
따라서 전역변수를 아래와 같이 3개를 설정하도록 합니다.

- ✓ uint8_t tcpRxBuffer[SPI_FIFO_SIZE]; // AXI SPI 송신에 사용되는 버퍼, SPI_FIFO_SIZE = 256
- ✓ uint8_t tcpTxBuffer[SPI_FIFO_SIZE]; // AXI SPI 수신에 사용되는 버퍼, SPI_FIFO_SIZE = 256
- ✓ uint8_t tcpMsgBuffer[TCP_MSG_MAX]; // 수신 메시지를 저장하는 버퍼, TCP_MSG_MAX = 1200

TCP/IP 송수신에 사용되는 전역변수는 총 1712 바이트입니다. Heap Size 가 2048 사이즈이기 때문에 충분합니다.

9.4.2 파일 구조

아래 그림은 파일 구조를 보여줍니다.



platform_config.h, platform.c, platform.h 파일은 기본적으로 생성되는 파일입니다. helloworld.c 안에 main() 함수가 있습니다.

9.4.3 data_type.h

변수 타입이 정의되어 있습니다. 즐겨 사용하는 uint8_t, uint16_t, uint32_t 를 정의합니다.

```

5 typedef    signed short    int16;
6 typedef    signed long     long32;
7 typedef    unsigned char   uchar8;
8 typedef    unsigned short   uint16;
9 typedef    unsigned long    ulong32;
10
11 #define     uint8_t         uchar8
12 #define     uint16_t        uint16
13 #define     uint32_t        ulong32

```

9.4.4 ax_common.h

각 IP들의 ID, TCP/IP 통신에 사용되는 버퍼 사이즈가 정의되어 있습니다.

```

4  /* definitions for UART */
5  #define GPIO_5BIT_DEVICE_ID    XPAR_GPIO_0_DEVICE_ID
6  #define GPIO_5BIT_CHANNEL1    1
7
8  /* definitions for UART */
9  #define UARTLITE_DEVICE_ID     XPAR_UARTLITE_0_DEVICE_ID
10 #define UARTLITE_INT_IRQ_ID    XPAR_INTC_0_UARTLITE_0_VEC_ID
11
12 /* definitions for SPI */
13 #define SPI_DEVICE_ID          XPAR_SPI_0_DEVICE_ID
14
15 #define SPI_FIFO_SIZE          256
16 #define TCP_MSG_SIZE           240
17 #define TCP_MSG_MAX            1200
--

```

9.4.5 w5500.c, w5500.h

wiznet에서 제공하는 w5500.c, w5500.h 파일을 본 강의에 맞게 수정하였습니다. 앞에서 설명한 w5500 read/write 함수들이 Xspi_Transfer() 함수로 구현되었습니다.

9.4.6 w5500_task.c, w5500_task.h

wiznet에서 제공하는 wizchip_conf.c, wizchip_conf.h 파일에서 필요한 부분만 가져와서 수정하였습니다. 주로 w5500의 초기화와 관련된 내용들이 포함되어 있습니다.

- ✓ w5500_reset_HW() : w5500 RESET 핀을 이용한 reset, gpio[0] 를 사용함
- ✓ w5500_sw_reset() : w5500 register들을 초기화 하기 위한 sw reset.
- ✓ w5500_init() : w5500에는 TX, RX 각각 8개의 Socket을 지원합니다. 각각의 Socket은 내부 메모리를 최대 32KB까지 설정할 수 있습니다. 각각의 Socket의 메모리를 설정합니다.
- ✓ w5500_getphylink() : Network Link를 Check 합니다.

9.4.7 w5500_socket.c, w5500_socket.h

wiznet에서 제공하는 socket.c, socket.h 파일을 본 강의에 맞게 수정하였습니다. 주로 socket에 관련된 API가 들어 있습니다.

- ✓ socket() : socket을 open 합니다. 본 강의에서는 w5500 보드가 server가 되고, PC의 응용 프로그램이 client가 됩니다. 전원이 인가되면, w5500은 network link를 확인해서 network link가 존재하면 socket을 open 해서 client의 접속을 기다립니다.
- ✓ close() : socket을 close 합니다.
- ✓ listen () : client의 접속을 기다립니다.
- ✓ send() : client로 데이터를 전송합니다.
- ✓ recv() : client에서 전송한 데이터를 수신합니다.

9.4.8 w5500_loopback.c, w5500_loopback.h

본 강의에서는 wiznet에서 제공하는 loopback Test를 이용하여 TCP/IP 통신을 진행합니다. 먼저 PC에서 1200 바이트를 전송합니다. microblaze는 w5500의 수신 상태 레지스터를 계속해서 확인해서 수신한 데이터가 있으면 수신한 데이터를 읽습니다. 1200 바이트를 수신하면, 수신한 데이터를 UART로 뿌려서 전송한 데이터가 맞는지 확인합니다. 그리고 PC로 다시 1200바이트를 전송합니다.

```

19 // TCP & UDP Loopback Test
20 int32_t loopback_tcps(uint8_t sn, uint8_t* buf, uint16_t port)
21 {
22     int32_t ret;
23     uint16_t size = 0;
24     uint8_t destip[4];
25     uint16_t destport;
26     int i, j;
27
28     switch(getSn_SR(sn))
29     {
30         // -----
31         // SOCK_ESTABLISHED
32         case SOCK_ESTABLISHED :
33             if(getSn_IR(sn) & Sn_IR_CON)
34             {
35                 getSn_DIPR(sn, destip);
36                 destport = getSn_DPORT(sn);
37                 xil_printf("\r\n Connected - %d.%d.%d.%d %d \r\n",
38                     destip[0], destip[1], destip[2], destip[3], destport);
39                 setSn_IR(sn, Sn_IR_CON);
40             }

```

- ✓ 라인 32 - 40 : socket이 연결되었을 때의 상태입니다. 연결된 client의 정보를 보여줍니다.

```

41         if((size = getSn_RX_RSR(sn)) > 0)
42         {
43             xil_printf(" W5500 Received size : %d \r\n", size);
44             tcpRxTotal += size;
45             while(1)
46             {
47                 if(size > TCP_MSG_SIZE)
48                 {
49                     memset(buf, 0, SPI_FIFO_SIZE);
50                     ret = recv(sn, buf, TCP_MSG_SIZE);
51                     if(ret <= 0) return ret;
52                     size -= TCP_MSG_SIZE;
53                     xil_printf(" BD Received size : %d \r\n", TCP_MSG_SIZE);
54                     memcpy(tcpMsgBuffer+tcpMsgIndex, buf, TCP_MSG_SIZE);
55                     tcpMsgIndex += TCP_MSG_SIZE;
56                 }
57                 else
58                 {
59                     memset(buf, 0, SPI_FIFO_SIZE);
60                     ret = recv(sn, buf, size);
61                     if(ret <= 0) return ret;
62                     xil_printf(" BD Received size : %d \r\n", size);
63                     memcpy(tcpMsgBuffer+tcpMsgIndex, buf, TCP_MSG_SIZE);
64                     tcpMsgIndex += size;
65
66                     break;
67                 }
68             }
69
70             if(tcpRxTotal >= TCP_MSG_MAX)
71             {
72                 tcpRxTotal = 0;
73                 tcpMsgIndex = 0;
74
75                 for(i=0; i<(TCP_MSG_MAX/48); i++)
76                 {
77                     xil_printf("B : %d ", i);
78                     for(j=0; j<48; j++) xil_printf("%c", tcpMsgBuffer[i*48+j]);
79                     xil_printf("\r\n");
80                 }
81
82                 send(SOCK_TCPS, tcpMsgBuffer, TCP_MSG_SIZE);
83                 send(SOCK_TCPS, tcpMsgBuffer+TCP_MSG_SIZE, TCP_MSG_SIZE);
84                 send(SOCK_TCPS, tcpMsgBuffer+2*TCP_MSG_SIZE, TCP_MSG_SIZE);
85                 send(SOCK_TCPS, tcpMsgBuffer+3*TCP_MSG_SIZE, TCP_MSG_SIZE);
86                 send(SOCK_TCPS, tcpMsgBuffer+4*TCP_MSG_SIZE, TCP_MSG_SIZE);
87             }
88         }
89         break;

```

- ✓ 라인 41 : 수신된 데이터를 확인하여 수신한 데이터가 있을 때 사이즈를 리턴합니다.
- ✓ 라인 43 - 68 : AXI SPI가 한번에 읽을 수 있는 데이터가 최대 253 (256-3) 입니다. 본 강의에서는 240바이트씩 읽도록 구현하였습니다. TCP_MSG_SIZE = 240
- ✓ 라인 70 - 89 : PC로 부터 1200 바이트를 수신하면, 수신한 데이터를 UART를 통해 화면에 보여줍니다. PC에서 송신한 데이터는 "A~X", "a~x"의 데이터가 반복해서 구성됩니다. (48바이트씩 25번, 48 x 25 = 1200) PC에서는 1200바이트를 전송합니다. 또한 수신한 데이터를 다시 PC로 전송합니다. 240바이트씩 5번 전송합니다.

```

90 // -----
91 // SOCK_CLOSE_WAIT
92 case SOCK_CLOSE_WAIT :
93     if((ret=disconnect(sn)) != SOCK_OK) return ret;
94     xil_printf("Socket closed .. \r\n");
95     break;
96 // -----
97 // SOCK_INIT
98 case SOCK_INIT :
99     xil_printf("Listen, TCP server loopback, port : %d \r\n", port);
100     if( (ret = listen(sn)) != SOCK_OK) return ret;
101     break;
102 // -----
103 // SOCK_CLOSED
104 case SOCK_CLOSED:
105     xil_printf("TCP server loopback start .. \r\n");
106     if((ret=socket(sn, Sn_MR_TCP, port, 0x00)) != sn)
107         return ret;
108     break;
109
110 default:
111     break;

```

- ✓ 라인 92 - 95 : socket이 close 될 때, SOCK_CLOSE_WAIT, SOCK_CLOSED 상태가 발생합니다.
- ✓ 라인 98 - 101 : socket 이 생성될 때 발생합니다.

Socket Status Register에 대한 자세한 사항은 w5500 데이터 시트의 48 페이지를 참조하시길 바랍니다.

IHL

9.4.9 helloworld.c

```

62 XGpio      Gpio_5bits;          /* The instance of the gpio */
63 XUartLite  UartLite;           /* The instance of the UartLite Device */
64 XSpi       SpiInstance;        /* The instance of the SPI device */
65
66 void Display_Net_Conf(void);
67 void Net_Conf(void);
68
69 void uart_init(void)
70 {
71     XUartLite_Initialize(&UartLite, UARLITE_DEVICE_ID);
72     XUartLite_SelfTest(&UartLite);
73 }
74
75 void gpio_init(void)
76 {
77     XGpio_Initialize(&Gpio_5bits, GPIO_5BIT_DEVICE_ID);
78 }
79
80 void spi_init(void)
81 {
82     XSpi_Config *ConfigPtr; /* Pointer to Configuration data */
83
84     ConfigPtr = XSpi_LookupConfig(SPI_DEVICE_ID);
85     XSpi_CfgInitialize(&SpiInstance, ConfigPtr, ConfigPtr->BaseAddress);
86     XSpi_SelfTest(&SpiInstance);
87
88     XSpi_SetOptions(&SpiInstance, XSP_MASTER_OPTION );
89     XSpi_Start(&SpiInstance);          /* Start SPI */
90     XSpi_IntrGlobalDisable(&SpiInstance); /* Disable interrupt */
91     XSpi_SetSlaveSelect(&SpiInstance, 0x35); /* 이 문이 빠지면 동작하지 않음 */
92 }

```

✓ 라인 62 - 64 : IP Instance 정의

✓ 라인 69 - 92 : uart, gpio, spi 초기화

```

95 int main()
96 {
97     uint8_t phy_link=0;
98     int32_t ret;
99
100     init_platform();
101     gpio_init();
102     uart_init();
103     spi_init();
104     XGpio_DiscreteWrite(&Gpio_5bits, GPIO_5BIT_CHANNEL1, 0xf);
105
106     xil_printf("\r\n Microblaze w5500 start .. \r\n");
107
108     // W5500_Init()
109     w5500_reset_HW();          // 1) HW reset
110     w5500_init();              // 2) tx, rx buffer size set..
111     Net_Conf();
112     xil_printf("Static Mode .. \r\n");
113     Display_Net_Conf();
114
115     // -----
116     // W5500 : Link Check..
117     phy_link = w5500_getphylink();
118     while(1)
119     {
120         if(phy_link==WIZ_YES)
121         {
122             // TCP server loopback test
123             if ((ret = loopback_tcps(SOCK_TCPS, tcpRxBuffer, PORT_TCPS)) < 0)
124             {
125                 xil_printf("SOCKET_TCP ERROR !! \r\n");
126             }
127         }
128     }
129     cleanup_platform();
130     return 0;
131 }

```

- ✓ 라인 109 - 113 : w5500 초기화, 아래의 정보로 w5500이 설정됩니다.

```
16 wiz_NetInfo gWIZNETINFO = {  
17     {0x0A, 0x15, 0x23, 0x07, 0x21, 0x01},  
18     {192, 168, 1, 11},  
19     {255, 255, 255, 0},  
20     {192, 168, 1, 1},  
21     {0, 0, 0, 0},  
22     NETINFO_STATIC  
23 };
```

- ✓ 라인 117 : network link를 확인합니다.
- ✓ 라인 120 - 127 : network link가 존재하면, w5500의 상태를 계속해서 확인합니다.

Display_Net_conf() : w5500에 설정된 Network 정보를 보여줍니다.

Net_Conf(); w5500에 Network 정보를 설정합니다.

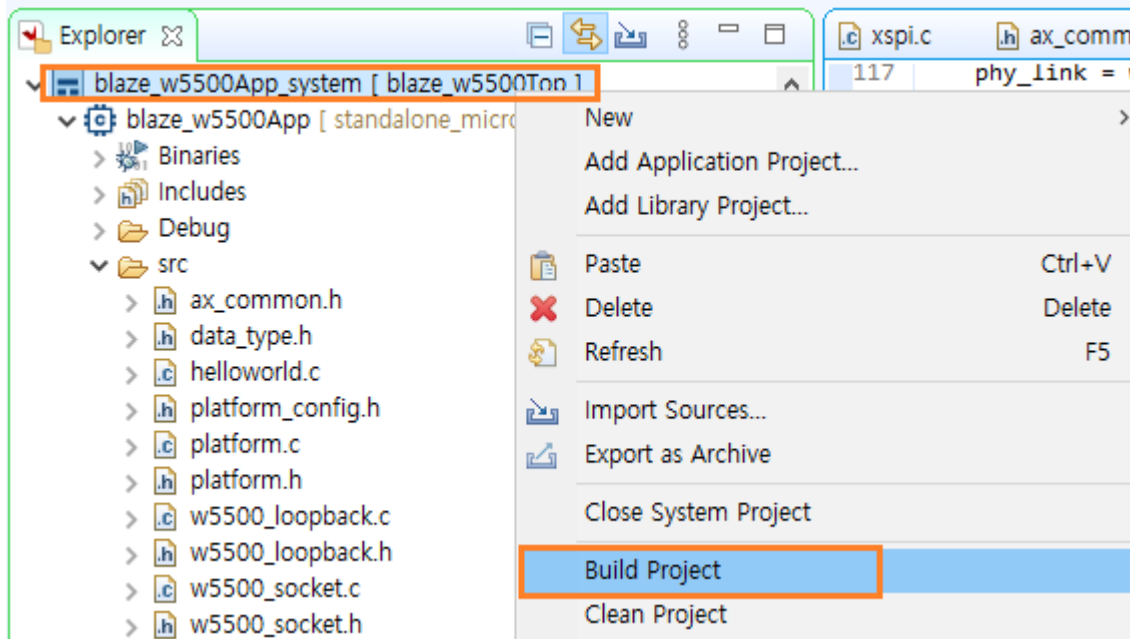
IIHIL

9.5 결과 확인

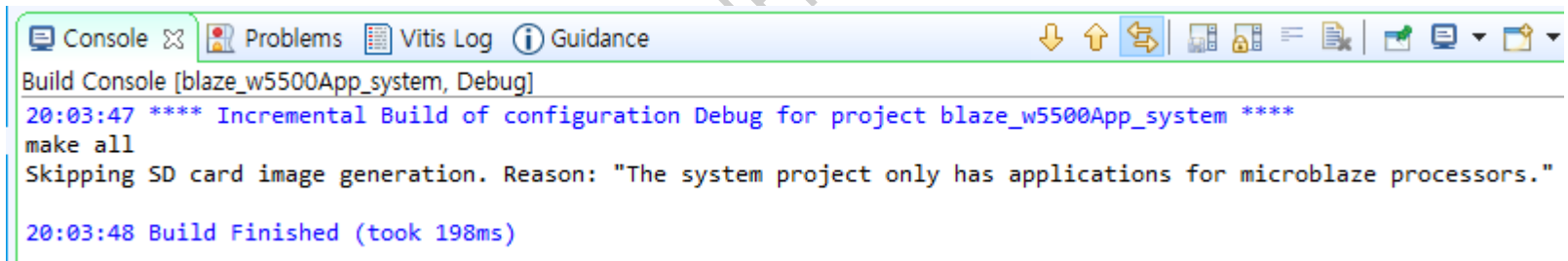
이번 장에서는 프로그램을 Build하고 보드에 다운로드 해서 결과를 확인합니다.

9.5.1 Build Project

vitis 에서 blaze_w5500App.system - 우클릭 - Build Project를 클릭합니다.



하단의 Console 창에 에러가 없이 Build가 완료되었음을 확인합니다.



9.5.2 PC Network 설정

보드와 데이터 송수신을 하기 위하여 PC의 Network 정보를 설정합니다. PC와 보드를 Network Cable로 바로 연결해도 되고 공유기를 거쳐서 연결해도 됩니다. PC 설정에서 네트워크 및 인터넷 - 고급 네트워크 설정 - 어댑터 옵션 변경 - 이더넷 선택 - 우클릭 - 속성 - 인터넷 프로토콜 버전 4 (TCP/IPv4) 선택 - 속성 클릭합니다.

아래와 같이 설정을 변경한 후 확인을 클릭합니다.

인터넷 프로토콜 버전 4(TCP/IPv4) 속성

일반

네트워크가 IP 자동 설정 기능을 지원하면 IP 설정이 자동으로 할당되도록 할 수 있습니다. 지원하지 않으면, 네트워크 관리자에게 적절한 IP 설정값을 문의해야 합니다.

☐ 자동으로 IP 주소 받기(O)

☒ 다음 IP 주소 사용(S):

IP 주소(I): 192 . 168 . 1 . 10

서브넷 마스크(U): 255 . 255 . 255 . 0

기본 게이트웨이(D): . . .

☐ 자동으로 DNS 서버 주소 받기(B)

☒ 다음 DNS 서버 주소 사용(E):

기본 설정 DNS 서버(P): . . .

보조 DNS 서버(A): . . .

☐ 끝낼 때 설정 유효성 검사(L)

고급(V)...

확인 취소

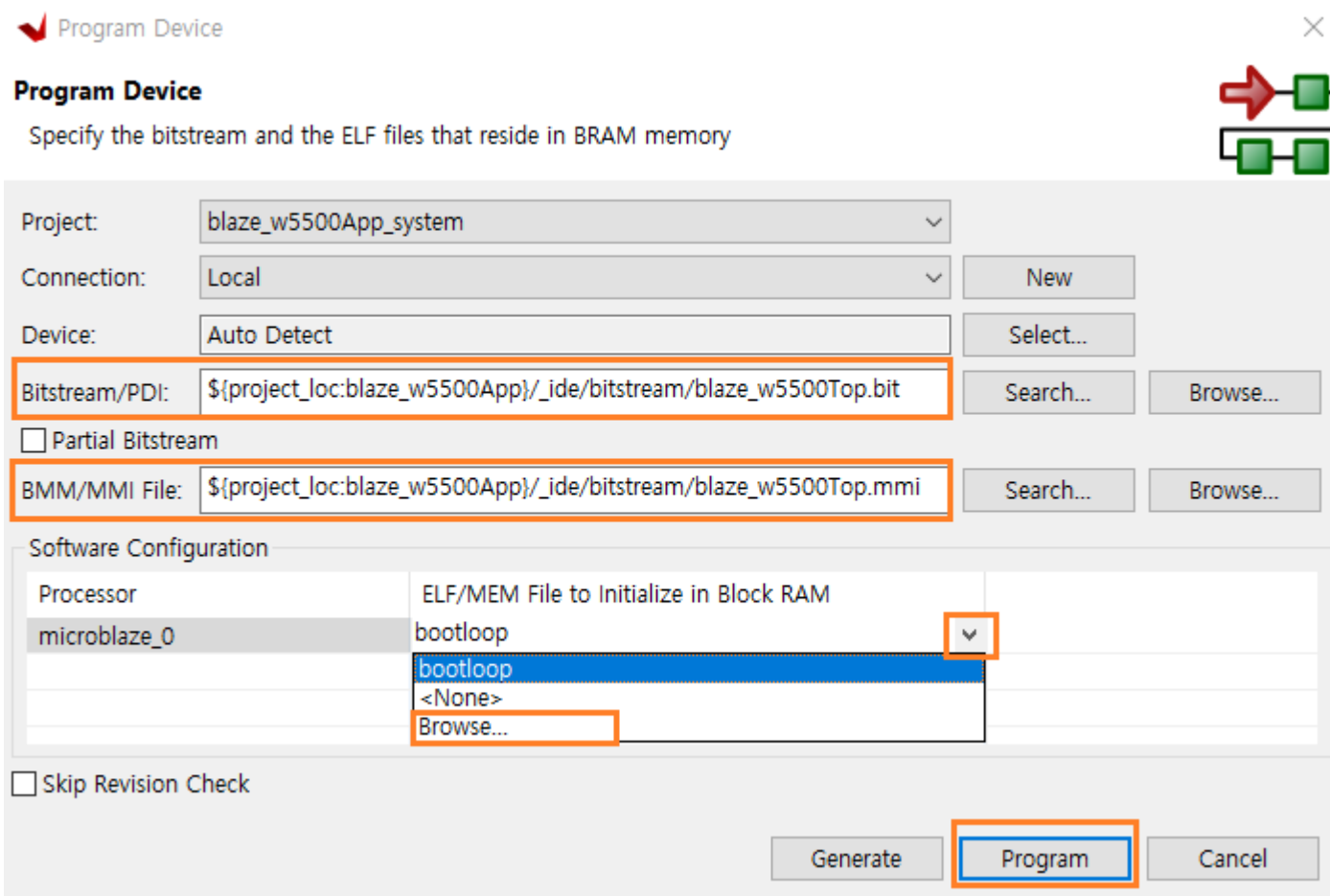
보드의 IP가 11번으로 설정되어 있습니다. PC는 10번으로 설정하면 됩니다.

9.5.3 프로그램 다운로드 및 결과 확인

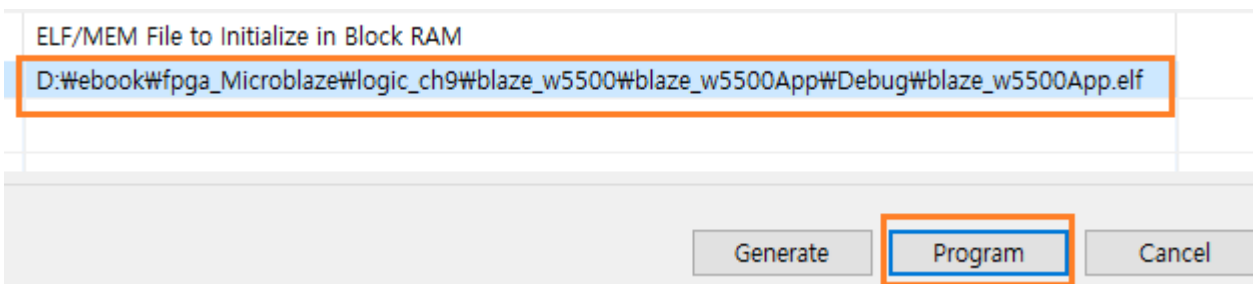
자료실에서 다운로드 받은 WinIDT_v17.zip 파일을 압축을 풀고, Release 폴더안에 있는 “WinIDT.exe”를 실행합니다. PC에 연결된 COM port를 확인하고, 속도는 115200으로 설정하고, Open 버튼을 클릭하여 Com Port를 Open합니다.



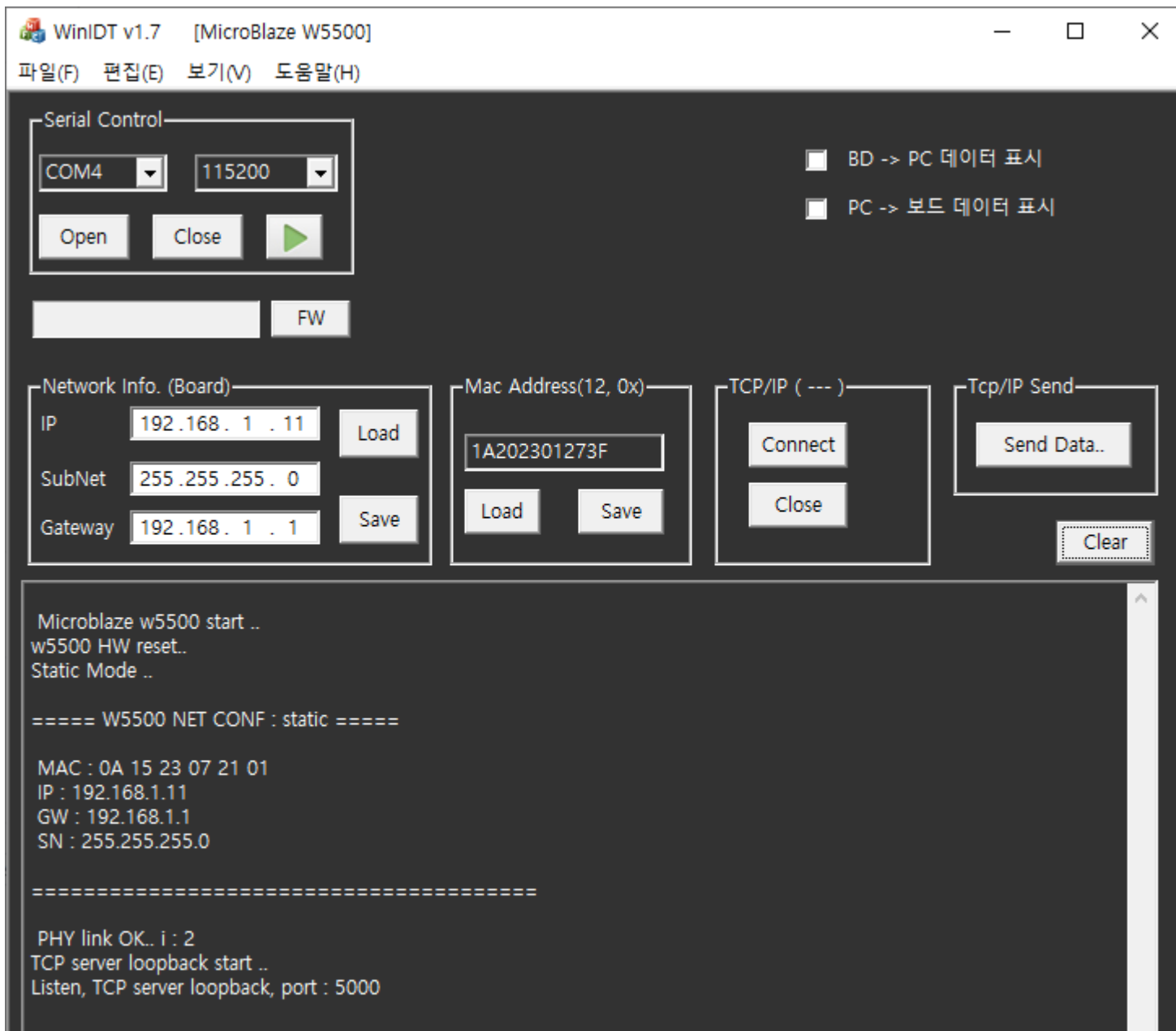
보드의 USB 케이블을 연결하여 전원을 인가하고 프로그램을 다운로드 합니다. vitis 에서 Xilinx - Program Device 를 클릭 합니다. Program Device 윈도우에서 Bitstream, MMI file 의 위치가 현재 프로젝트 폴더인지 확인하고, Software Configuration 의 microblaze_0 우측의 하단 화살표를 클릭하여 Brows... 를 선택하고 Program 버튼을 클릭합니다.



“blaze_w5500\blaze_w5500App\Debug” 폴더에 있는 blaze_w5500App.elf 를 더블 클릭합니다. 파일이 선택되었습니다. 다시 Program 버튼을 클릭하면 프로그램이 다운로드 됩니다.



아래 그림은 프로그램이 다운로드 되고 동작된 모습을 보여줍니다. PC의 Network 설정이 제대로 이루어지고, PC와 Network Cable이 연결되었다면, 아래와 같은 메시지가 나타납니다. w5500을 초기화 하고, w5500에 network 정보를 설정하고, w5500에 설정된 network 정보를 보여줍니다. PC와 Network 이 연결되었다면, PHY link Ok.. 가 되면서 w5500 은 TCP Server 로 동작하고, Client의 접속을 기다립니다.



Network Info. 가 제대로 설정되었는지 확인하고 (보드의 IP : 192.168.1.11) TCP/IP의 Connect 버튼을 클릭하면 연결됩니다. 첫번째 라인은 보드의 IP 이고, 두번째 라인은 PC의 IP 입니다.

```
connect 192.168.1.11.. OK.. connected..
Connected - 192.168.1.10 1064
```

Tcp/IP Send 의 Send Data.. 버튼을 클릭하면, PC에서 1200바이트의 데이터를 전송합니다. 보드에서는 1200 바이트를 수신하고, 디버깅을 위해 화면에 보여주고, 수신한 데이터를 다시 PC로 전송합니다.



- ✓ send ok ... : PC에서 1200 바이트 전송
- ✓ W5500 Received size ~ : w5500에서 1200 바이트 수신함, AXI SPI 를 통해 240바이트씩 5회 수신함, 수신한 데이터를 화면에 출력함(48바이트 x 25 라인)
- ✓ PC Received size ~ : 보드에서 수신한 데이터를 다시 PC로 전송, PC에서 수신한 데이터를 나타냄.

Send Data ... 버튼을 반복해서 클릭해도 1200 바이트를 정상적으로 송수신 하는 것을 확인할 수 있습니다.

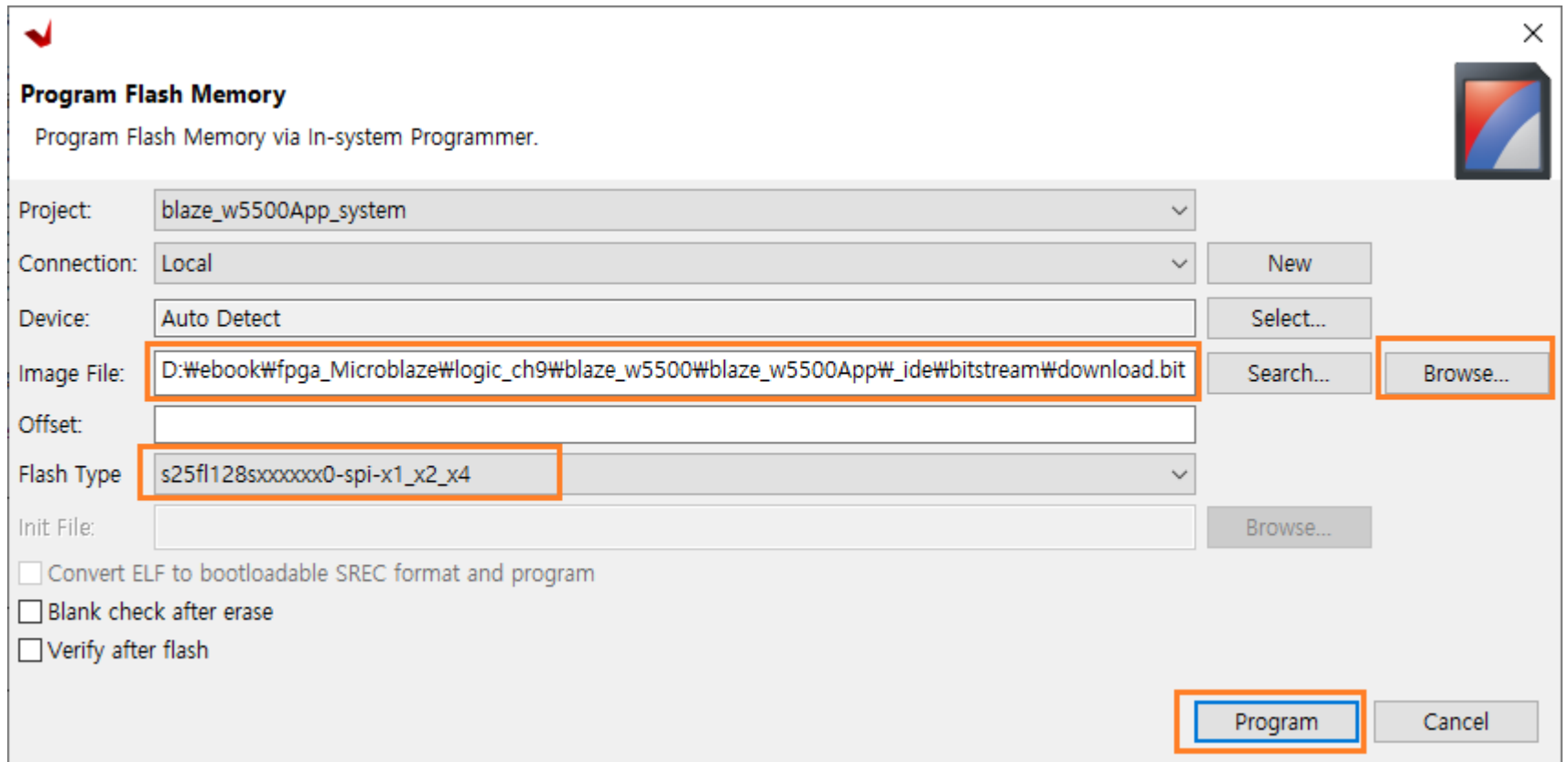
BD -> PC 데이터 표시를 check 하고 Send Data.. 버튼을 클릭하면, PC쪽 수신 데이터를 확인할 수 있습니다.



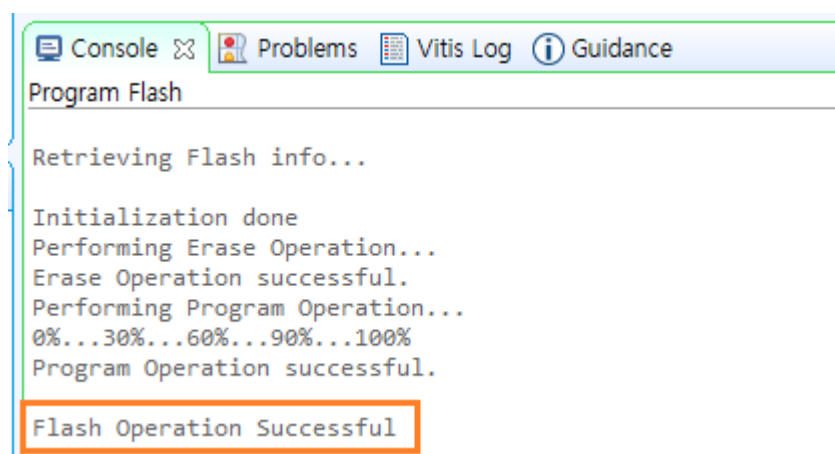
9.5.4 외부 Flash에 프로그램 다운로드

지금까지는 내부 SRAM에 프로그램을 다운로드 하였습니다. 전원을 Off 하면 데이터가 날라가 버려서 프로그램을 다시 시작할 수 없습니다. Arty A7 보드는 외부에 Flash를 가지고 있습니다. 외부 Flash에 프로그램을 저장하면 전원을 Off 해도 데이터가 사라지지 않아서 프로그램을 다시 동작할 수 있습니다.

vitis 에서 Xilinx - Program Flash 를 클릭합니다. Image File 우측 Browse... 를 클릭해서 download.bit 파일을 선택합니다. download.bit 파일은 “blaze_w5500\blaze_w5500App\ide\bitstream” 폴더 안에 있습니다. 그리고 Flash Type을 “s25fl128sxxxxx0-spi-x1_x2_x4”로 선택하고 Program 버튼을 클릭합니다.



만일 해당 폴더에 download.bit 파일이 보이지 않으면, Xilinx - Program Device를 클릭 후, elf 파일을 불러와서 Generate 버튼을 클릭하면 download.bit 파일이 생성됩니다. 정상적으로 다운로드 되면, 하단의 Console 창에 “Flash Operation Successful” 메시지가 나타납니다.



보드의 전원을 끄고, 다시 인가하면 프로그램이 동작하는 것을 확인할 수 있습니다.

9.6 결론

이번 장에서는 w5500 모듈을 이용하여 TCP/IP 를 구현하는 것을 알아보았습니다. microblaze 와 AXI_QUAD_SPI IP를 사용하여 구현하였습니다. AXI_QUAD_SPI IP의 FIFO 최대값이 256 바이트여서, 용량이 큰 데이터는 240바이트씩 나누어서 송수신해야 하는 단점도 있지만, 그래도 데이터 송수신이 잘 되고 있음을 확인할 수 있습니다. 이미지 데이터와 같은 대량의 데이터를 전송하지 않고, 간단한 제어 목적이나 텍스트 데이터를 위한 송수신에는 적용할 수 있을 것으로 기대합니다.

속도를 높이기 위해서는 spi clock 속도를 높일 수 있습니다. 본 강의에서는 w5500 모듈을 점퍼를 이용하여 연결했기 때문에 속도를 높이는데 한계가 있었지만, 저자의 경험에 의하면 24Mhz까지 사용해도 동작에 문제가 없었습니다.

이번장에서 구현된 내용은 FPGA MCU 포팅에서도 구현되었습니다. FPGA MCU 포팅 강의에서는 spi controller를 직접 코드로 구현해서 tcp/ip 송수신을 구현하였습니다. 관심 있으신 분들은 카페에 강의 문서가 무료로 제공되오니 다운 받아서 확인해 보시길 바랍니다. (프로그램은 강의를 구매한 분들에게만 제공됩니다. 먼저 문서를 보시고 필요시 구매하시면 됩니다)

지금까지 어려운 내용도 많이 있었을 텐데 끝까지 와 주셔서 감사드립니다. 부디 본 강의를 통하여 개발자 분들에게 도움이 많이 되시길 바랍니다. 감사합니다.



10. Block Memory Interface - 1

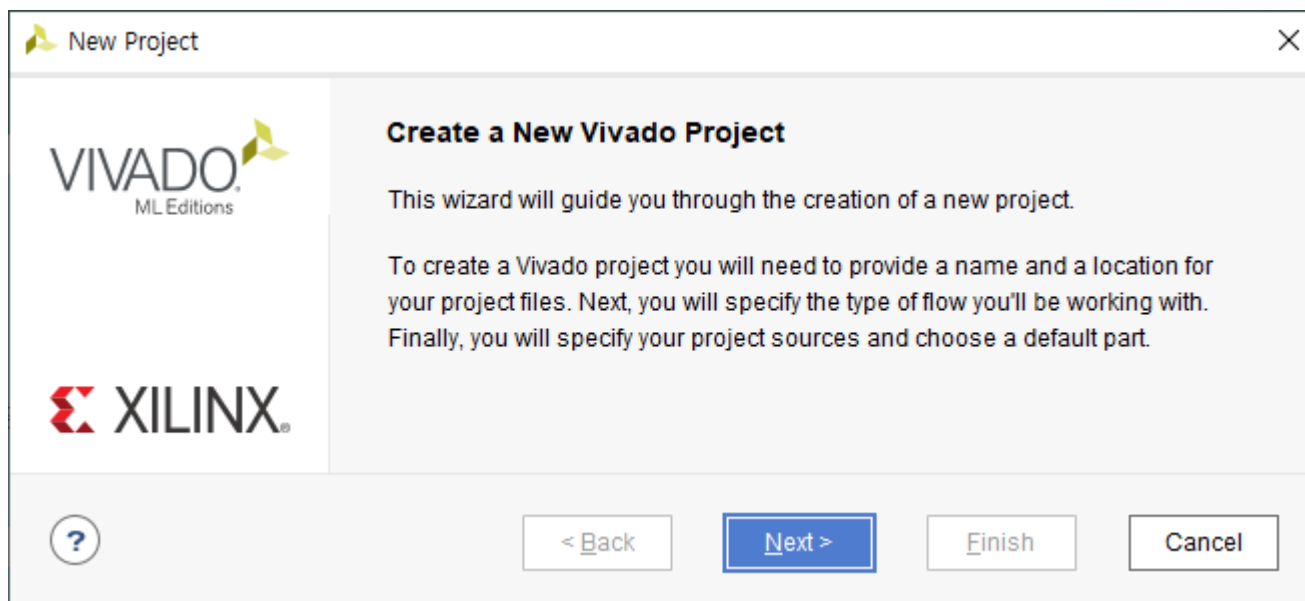
이번 장에서는 Block Memory Interface를 구현합니다. Interface Block과 Memory Block을 Block Design에서 지원하는 방법을 사용하여 구현합니다. 응용 sw에서는 구현된 메모리에 데이터를 read/write 하는 것을 구현합니다.

10.1 프로젝트 생성

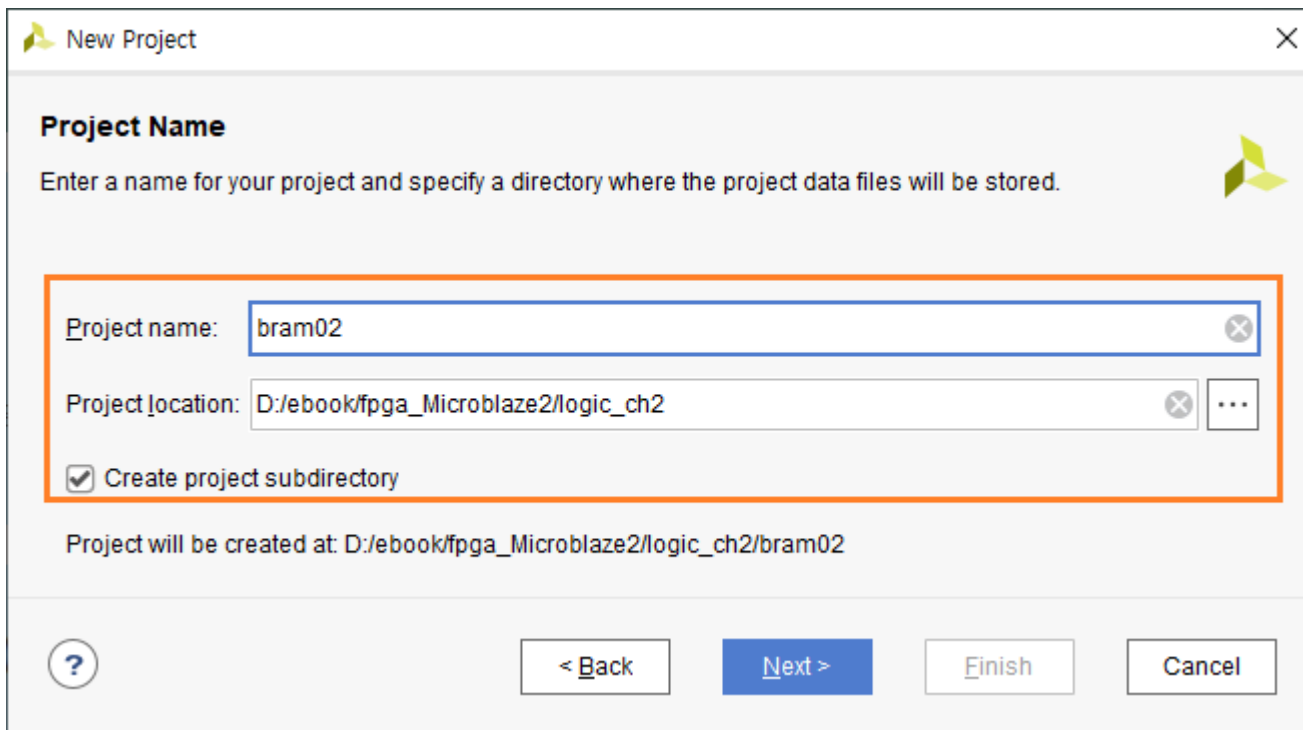
vivado 2022.1을 실행하고, Create Project를 클릭합니다.



Next를 클릭합니다.



프로젝트 이름을 “bram02”로 하고 프로젝트 위치를 설정하고 Next를 클릭합니다.

The image shows the 'New Project' dialog box in a software IDE. The dialog has a title bar with a green logo and a close button. Below the title bar, it says 'Project Name' and 'Enter a name for your project and specify a directory where the project data files will be stored.' There are two input fields: 'Project name:' with the text 'bram02' and 'Project location:' with the text 'D:/ebook/fpga_Microblaze2/logic_ch2'. Both fields have a small 'x' icon to clear the text. Below the input fields, there is a checkbox labeled 'Create project subdirectory' which is checked. At the bottom, it says 'Project will be created at: D:/ebook/fpga_Microblaze2/logic_ch2/bram02'. There are four buttons at the bottom: a help button with a question mark, a '< Back' button, a 'Next >' button (which is highlighted in blue), and a 'Cancel' button.

Project Type : RTL Project 를 선택하고 Next를 클릭합니다.

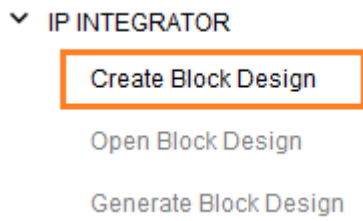
Add Sources, Add Constraints : Next를 클릭합니다. 파일은 나중에 추가합니다.

Part : xc7a35tcsq324-1을 선택하고 Next를 클릭합니다.

Finish를 클릭해서 프로젝트를 생성합니다.

10.2 Block Design

새로운 Block을 생성하기 위하여 PROJECT MANAGER에서 IP INTEGRATOR - Create Block Design을 클릭합니다.



Design Name : system 으로 하고 OK를 클릭합니다.

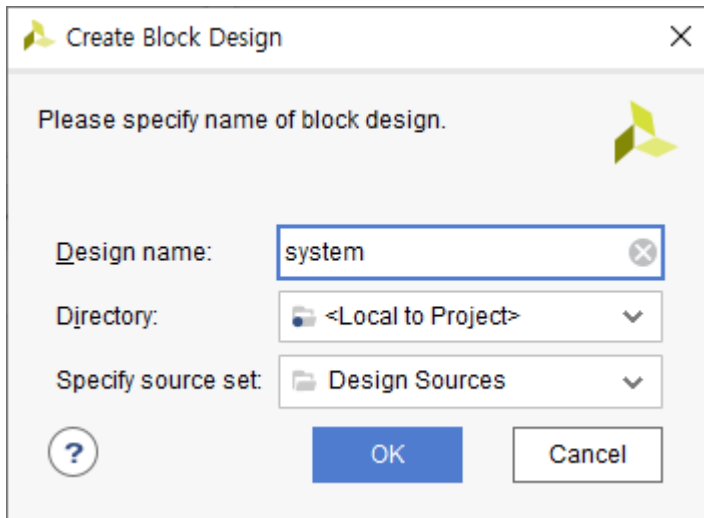


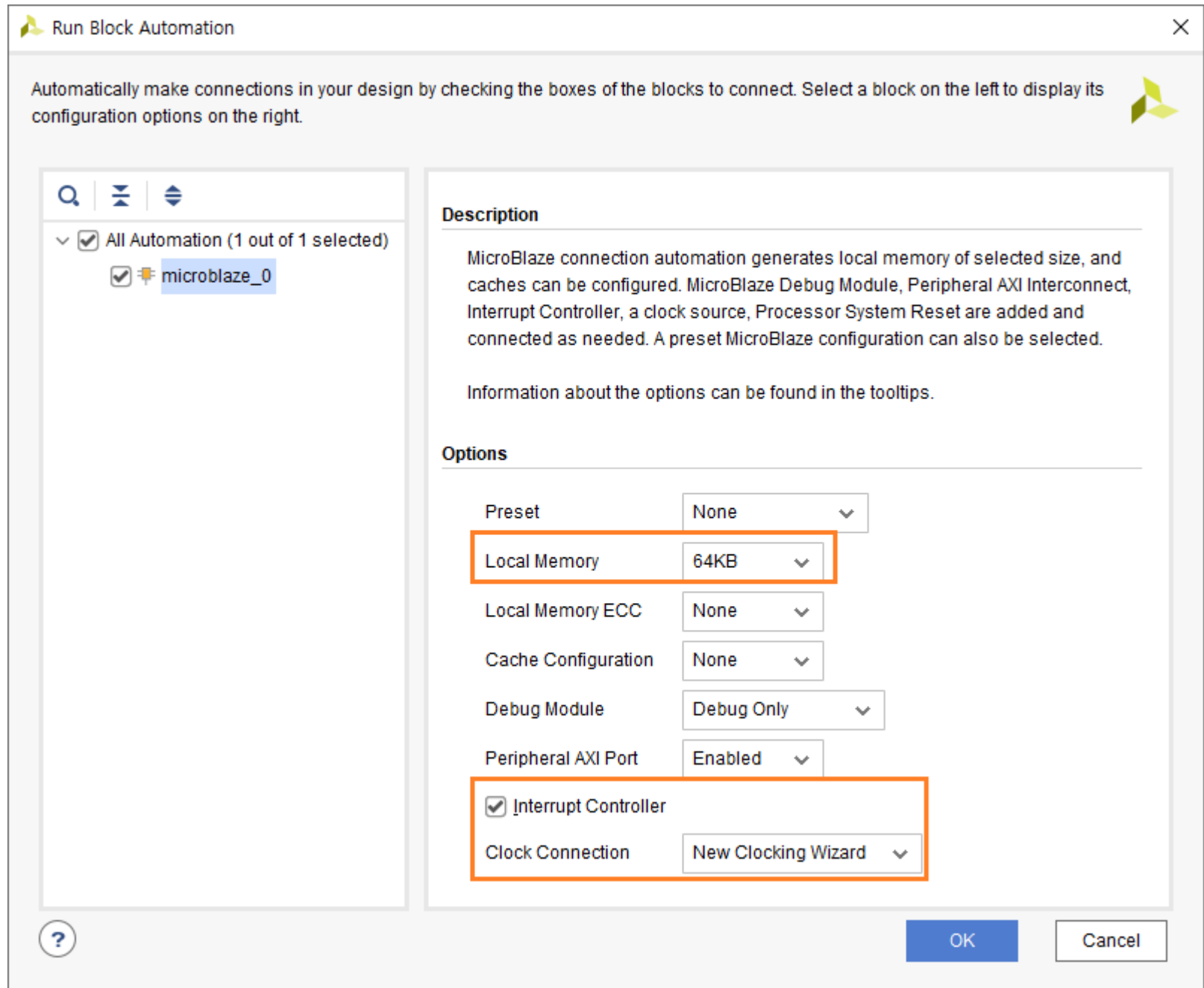
Diagram 윈도가 생성되었습니다. Add IP (+ 아이콘)을 클릭하여 5개의 IP를 추가합니다. Run Block Automation, Run Connection Automation은 5개를 모두 추가한 후에 진행합니다.

- ✓ MicroBlaze
- ✓ AXI Uartlite
- ✓ AXI GPIO
- ✓ AXI BRAM Controller
- ✓ Block Memory Generator

5개의 IP를 추가한 후에 Run Block Automation을 클릭합니다

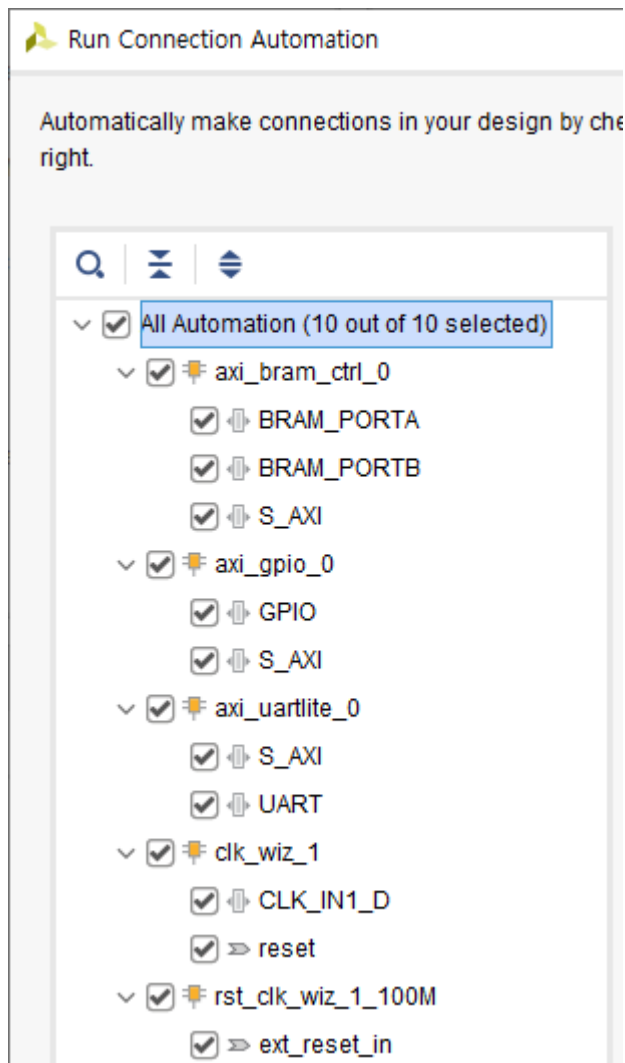
Microblaze의 속성을 설정합니다.

- ✓ Local Memory : 64KB (32KB도 가능하지만, 64KB로 넉넉히 설정하겠습니다)
- ✓ Interrupt Controller : check
- ✓ Clock Connection : New Clocking Wizard



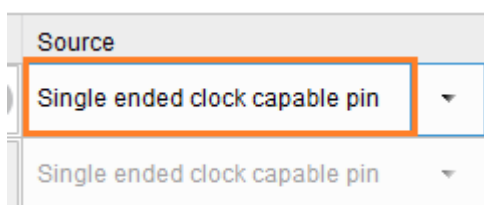
OK를 클릭합니다.

Run Connection Automation을 클릭합니다. All Automation을 클릭해서 전체 모듈을 선택하고 OK를 클릭합니다.



Clocking Wizard(clk_wiz_1)의 속성을 변경합니다. 해당 IP를 더블 클릭하면 속성창이 나타납니다.

- ✓ Input Clock Source : Single ended Clock capable pin
- ✓ Reset Type : Active Low



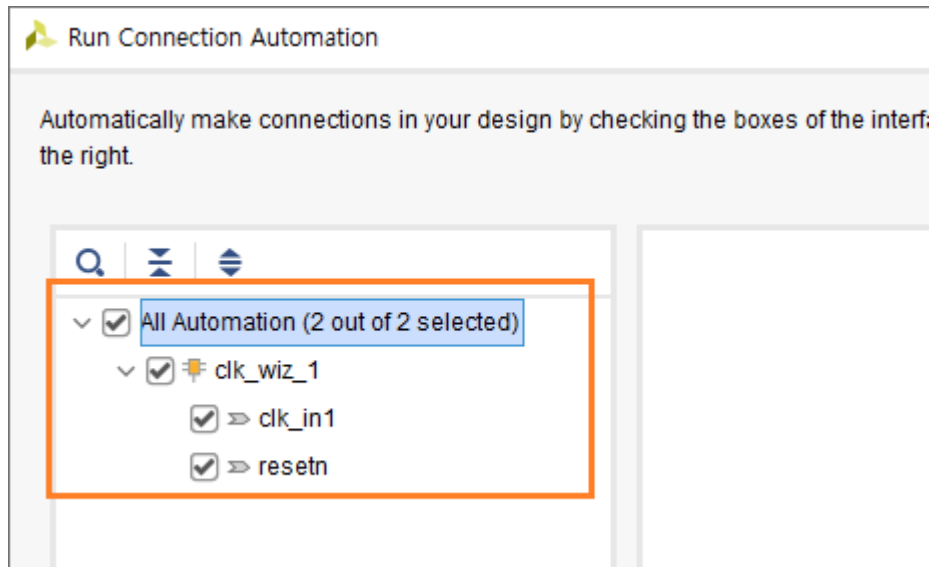
Reset Type

☐ Active High ☒ Active Low

속성 변경 후 OK를 클릭합니다.

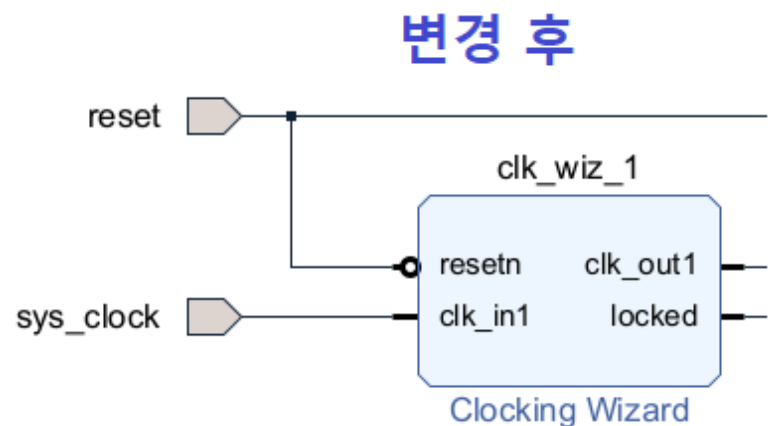
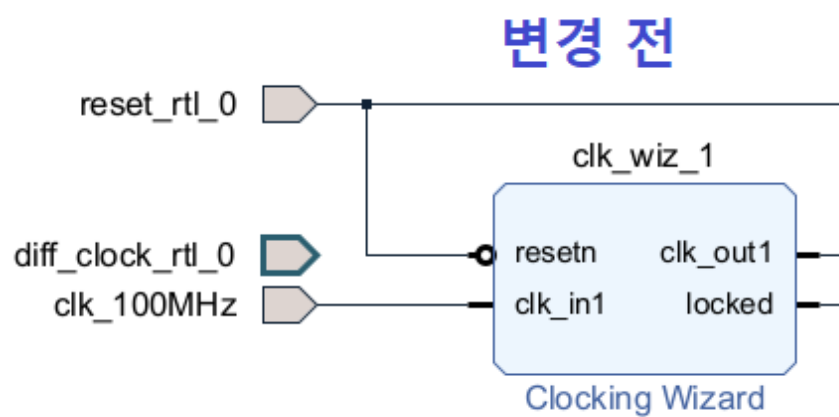
Run Connection Automation이 다시 활성화 되었습니다. reset_rtl_0 핀과 diff_clock_rtl_0 핀과 clk_wiz_1 을 서로 연결하기 위하여 Run Connection Automation을 클릭합니다. (수동으로 연결해도 됩니다)

All Automation을 클릭하고 OK를 클릭합니다.

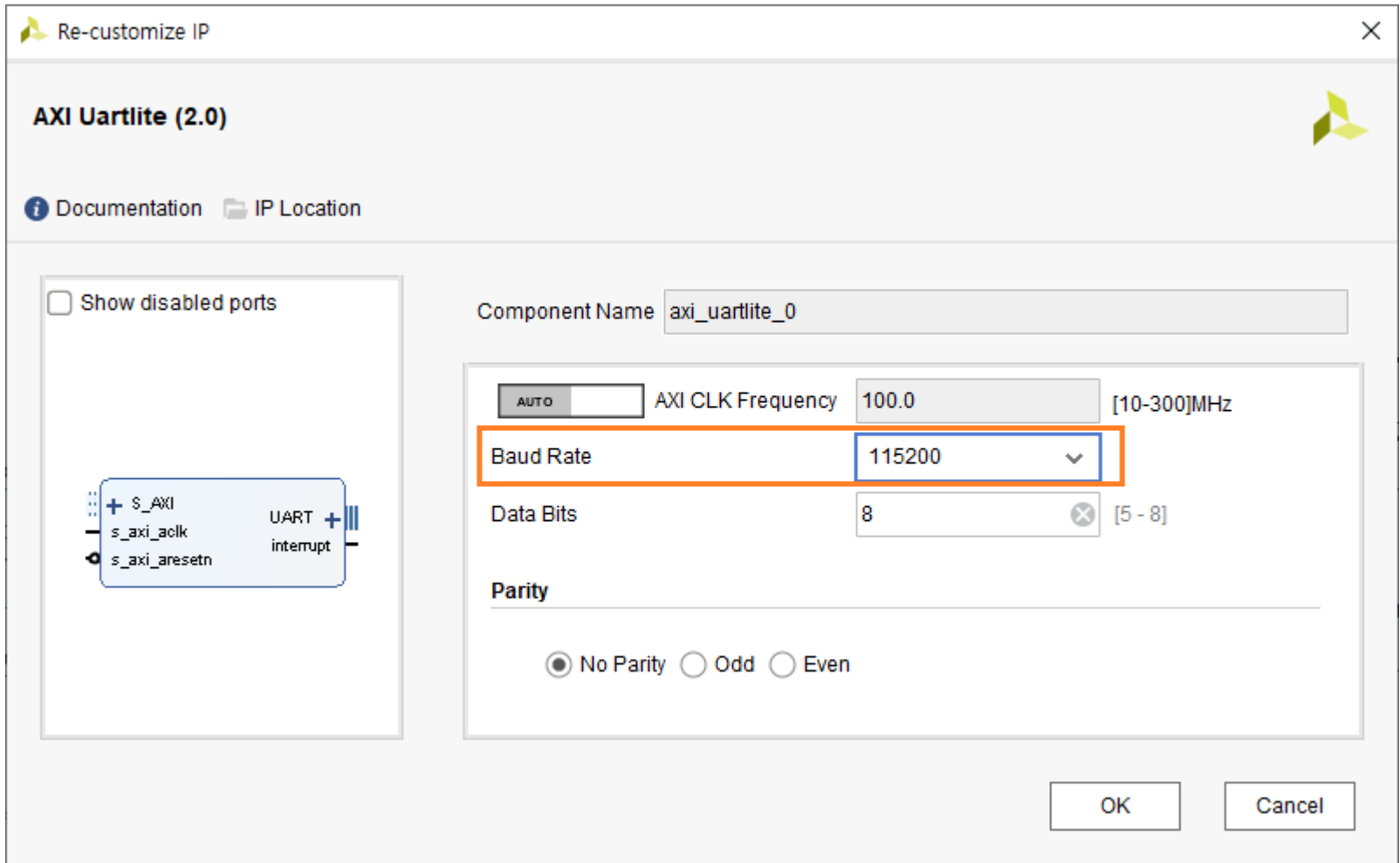


필요 없는 포트를 삭제하고 포트 이름을 변경합니다. 포트 이름 변경은 포트를 선택하면 왼쪽에 Port Properties 윈도우가 나타납니다. Name을 변경하면 됩니다.

- ✓ diff_clock_rtl_0 삭제
- ✓ reset_rtl_0 -> reset 으로 변경
- ✓ clk_100MHz -> sys_clock 으로 변경
- ✓ gpio_rtl_o -> gpio 로 변경
- ✓ uart_rtl_0 -> uart 로 변경



axi_uartlite_0 을 더블 클릭해서 속성을 변경합니다. Baud Rate : 115200 으로 변경하고 OK를 클릭합니다.



axi_gpio_0를 더블 클릭해서 속성을 변경합니다. gpio 핀은 총 4핀을 사용합니다. (각각 LD4 ~ LD7에 연결됩니다)

- ✓ All Outputs : check
- ✓ GPIO Width : 4
- ✓ Enable Dual Channel, Enable Interrupt : 모두 Uncheck

GPIO

☐ All Inputs

☒ All Outputs

GPIO Width [1 - 32]

Default Output Value [0x00000000,0xFFFFFFFF]

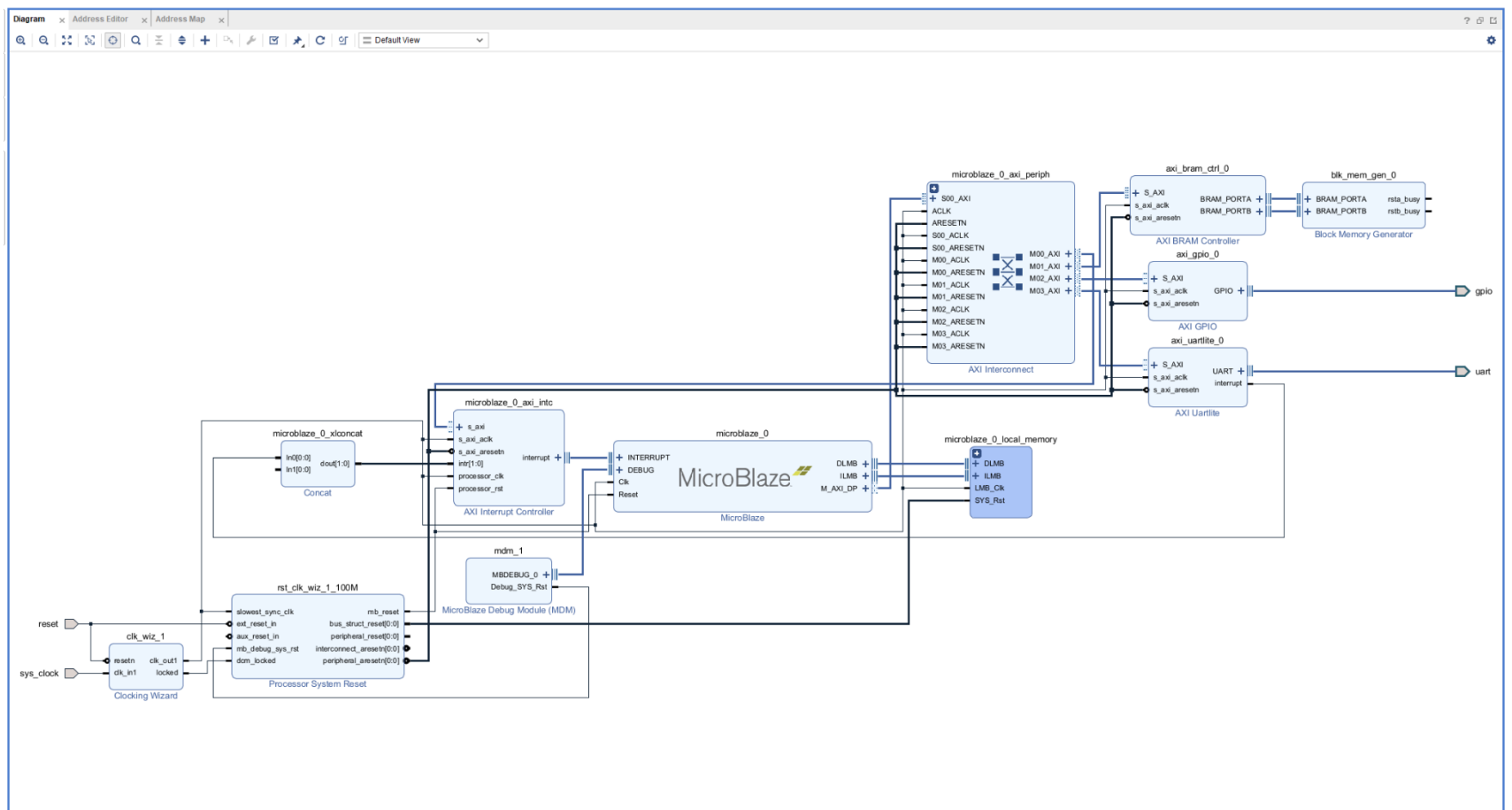
Default Tri State Value [0x00000000,0xFFFFFFFF]

☐ Enable Dual Channel

axi_uartlite_0의 interrupt 핀과 microblaze_0_xlconcat 의 In0[0:0]핀을 연결합니다.

Validate Design 아이콘을 클릭합니다. 에러가 없으면 Validation successful 메시지 창이 나타납니다. Regenerate Layout 아이콘을 클릭합니다.

아래 그림은 지금까지 구현된 Diagram을 보여줍니다. (자료실에 Diagram 파일(Diagram_ch10a.png)이 있으니 참조하시길 바랍니다)



Create HDL Wrapper... 를 실행하기 전에 추가된 AXI BRAM Controller와 Block Memory Generator의 속성을 확인해 보겠습니다. 여기서 설정된 속성은 다음장에서 사용자 로직에서 Block Memory Generator를 생성시 참고용으로 사용됩니다.

AXI BRAM Controller를 더블 클릭합니다. 속성을 보면 Number of BRAM interfaces : 2 로 설정되었습니다. 이는 Dual Port Ram을 위한 Interface 입니다. Dual Port로 해도 무관하지만, 좀더 간단하게 구현하기 위하여 Single Port Ram 으로 변경합니다. Number of BRAM Interfaces : 1로 수정합니다. AXI4 Protocol을 사용하고, Data Width : 32, Memory Depth : 2048 (총 8KB) 입니다. OK를 클릭합니다.

Component Name: axi_bram_ctrl_0

AXI Protocol: AXI4

Data Width: 32

Memory Depth (Auto): 2048

ID Width (Auto): 0

Support AXI Narrow Bursts: No

Read Latency: 1 [1 - 128]

Read Command Optimization: No

BRAM Options

BRAM Instance (Auto): External

Number of BRAM interfaces: 1

ECC Options

Enable ECC: No

ECC TYPE: Hamming

Enable Fault Injection: No

ECC Reset Value: 0

Block Memory Generator 를 더블 클릭합니다. Memory Type을 변경하고 OK를 클릭합니다.

- ✓ Mode : BRAM Controller
- ✓ Memory Type : Single Port RAM으로 변경
- ✓ Write Width, Read Width : 32
- ✓ Write Depth (Read Depth) : 2048 (Memory Size : 8KB)
- ✓ Operating Mode : Write First
- ✓ Enable Port Type : Use ENA Pin

Component Name: blk_mem_gen_0

Basic	Port A Options	Other Options	Summary
Mode	BRAM Controller	☒ Generate address interface with 32 bits	
Memory Type	Single Port RAM	☐ Common Clock	

Basic | Port A Options | Other Options | Summary

Memory Size (in words)

Write Width: 32 (Range: 32 to 1024 (bits))

Read Width: 32

Write Depth: 2048 (Range: 2 to 1048576)

Read Depth: 2048

Operating Mode: Write First

Enable Port Type: Use ENA Pin

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☒ RSTA Pin (set/reset pin) Output Reset Value (Hex): 0

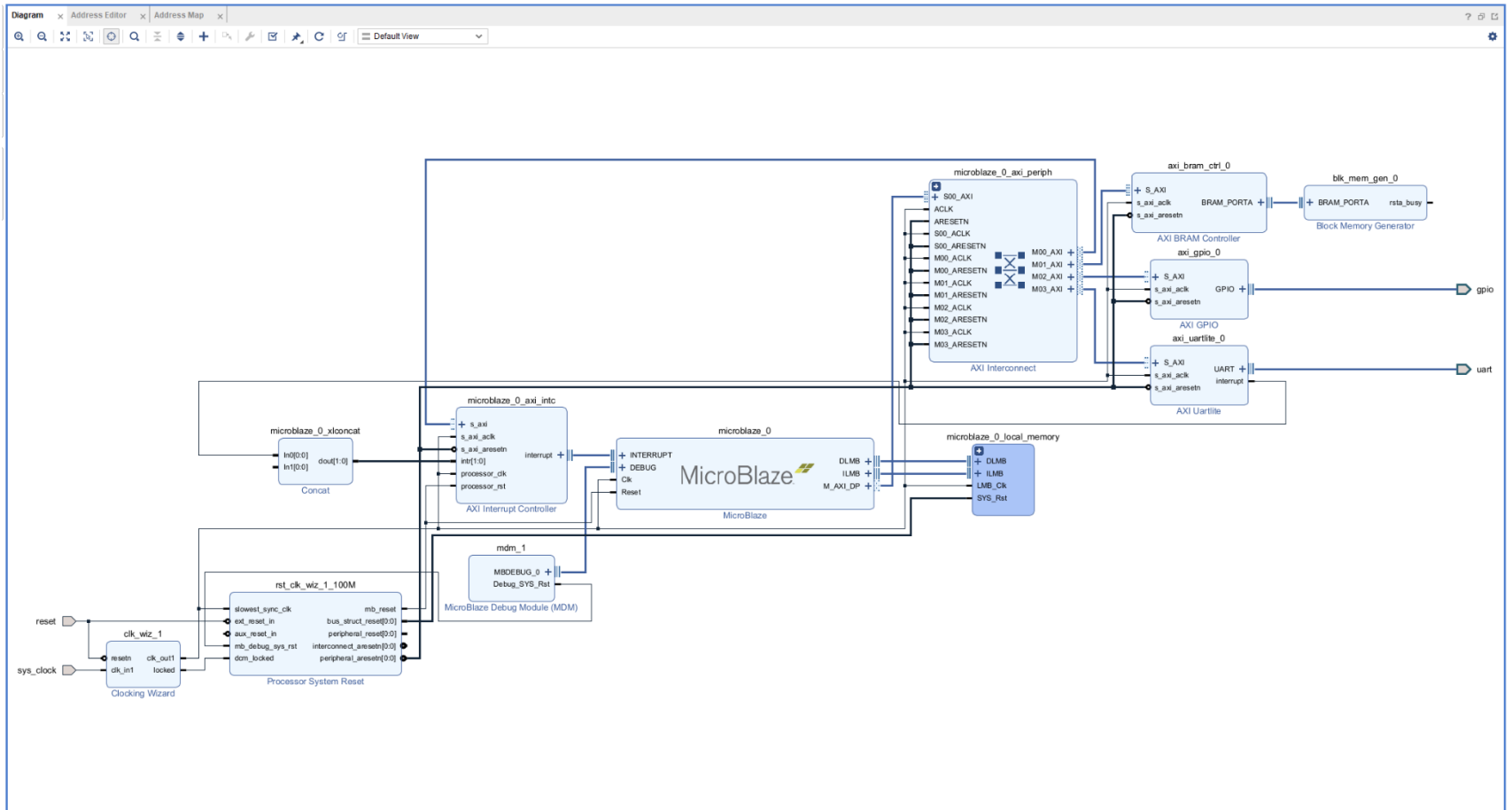
☐ Reset Memory Latch Reset Priority: CE (Latch or Register Enable)

READ Address Change A

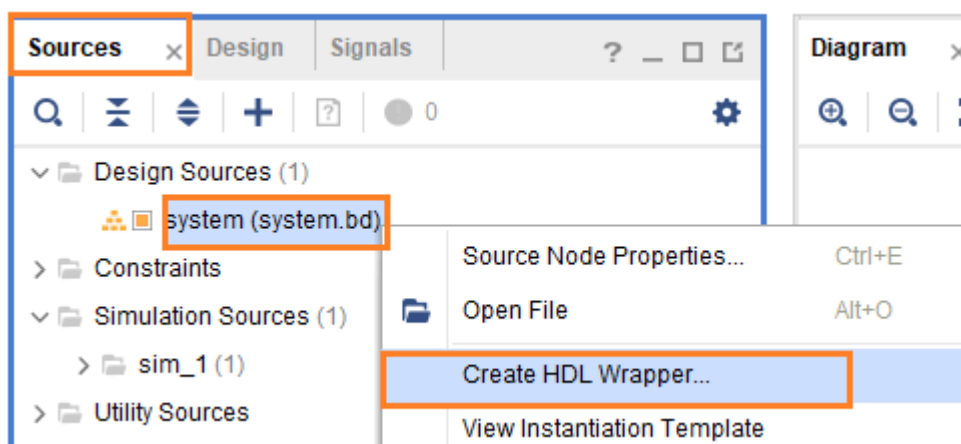
☐ Read Address Change A

Design이 변경되었습니다. Validate Design을 클릭합니다. 에러가 없으면 Validation Successful 메시지 윈도가 나타납니다.

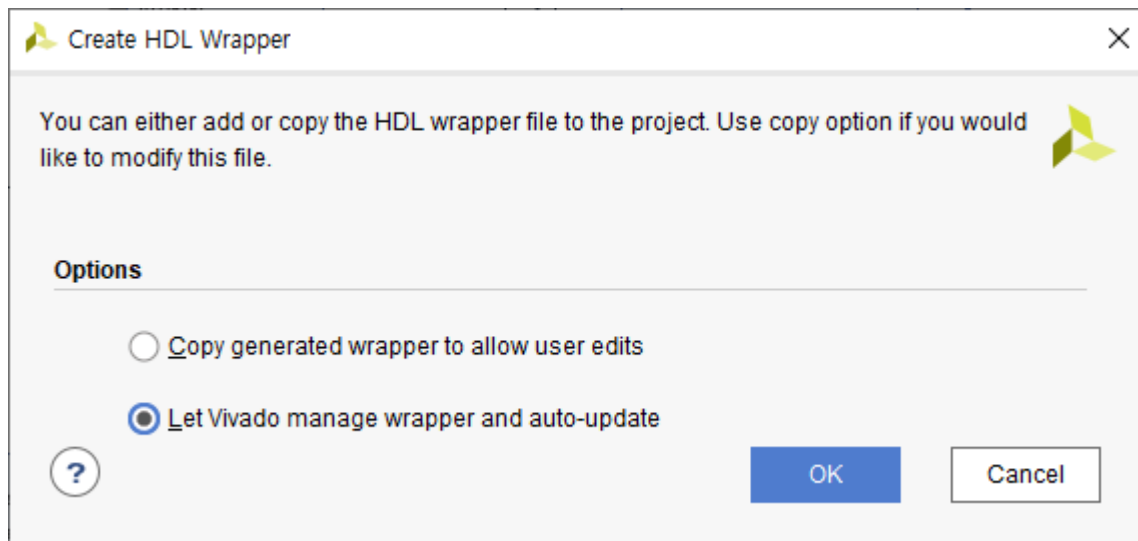
아래는 수정된 Diagram을 보여줍니다. (자료실에 Diagram_ch10b.png 파일을 참조하세요)



Sources 탭에서 Design Sources - system - 우클릭 - Create HDL Wrapper... 를 클릭합니다.



OK를 클릭합니다.



HDL Wrapper 파일이 생성되었습니다. (system_wrapper.v)

IHIL

10.3 Constraints 파일 추가

xdc 파일을 추가합니다. 자료실에서 다운로드 받은 파일을 복사한 후, Constraints - 우클릭 - Add Sources... 를 클릭해서 해당파일을 추가합니다. 파일명은 “bram02.xdc” 입니다.

아래는 bram02.xdc 파일을 보여줍니다.

```

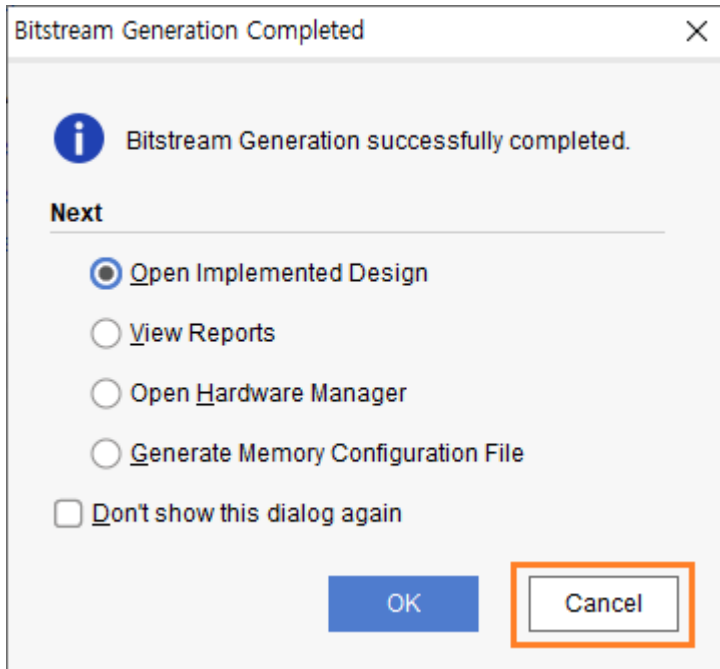
2 ## Clock signal
3 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clock }];
4 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clock }];
5
6 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
7
8 ## LEDs
9 set_property -dict { PACKAGE_PIN H5      IOSTANDARD LVCMOS33 } [get_ports { gpio_tri_o[0] }]; #LED4
10 set_property -dict { PACKAGE_PIN J5     IOSTANDARD LVCMOS33 } [get_ports { gpio_tri_o[1] }]; #LED5
11 set_property -dict { PACKAGE_PIN T9     IOSTANDARD LVCMOS33 } [get_ports { gpio_tri_o[2] }]; #LED6
12 set_property -dict { PACKAGE_PIN T10    IOSTANDARD LVCMOS33 } [get_ports { gpio_tri_o[3] }]; #LED7
13
14 ## Pmod Header JC
15 set_property -dict { PACKAGE_PIN V10     IOSTANDARD LVCMOS33 } [get_ports { uart_rxd }];
16 set_property -dict { PACKAGE_PIN V11     IOSTANDARD LVCMOS33 } [get_ports { uart_txd }];
17
18
19 #####
20 # MISC
21 #####
22 set_operating_conditions -ambient_temp 80.0
23
24 set_property CONFIG_MODE SPIx1 [current_design]
25 set_property CFGBVS VCCO [current_design]
26 set_property CONFIG_VOLTAGE 3.3 [current_design]
27
28 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
29 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
30 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
31 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

```

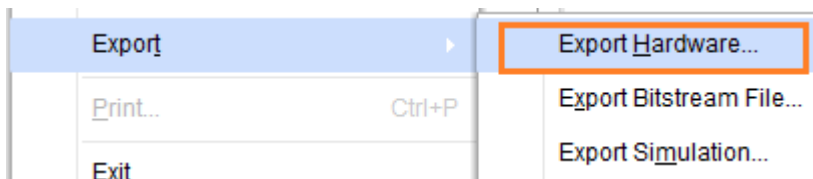
port name은 system_wrapper.v 파일을 참조해서 추가합니다.

PROGRAM AND DEBUG - Generate Bitstream 을 클릭해서 Bitstream을 생성합니다. 모든 IP들을 Generation 하고, Synthesis, Implementation, Generate Bitstream 까지 진행하므로 시간이 몇 분 정도 소요됩니다.

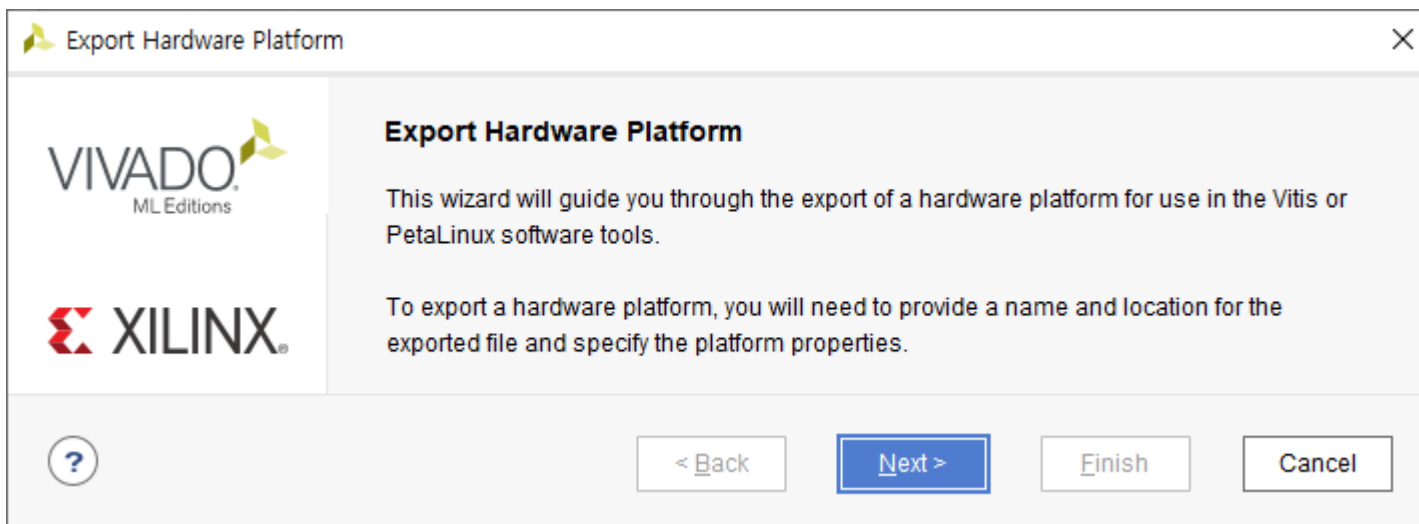
Bitstream이 생성되면, 완료 윈도우가 나타납니다. Cancel 을 클릭합니다.



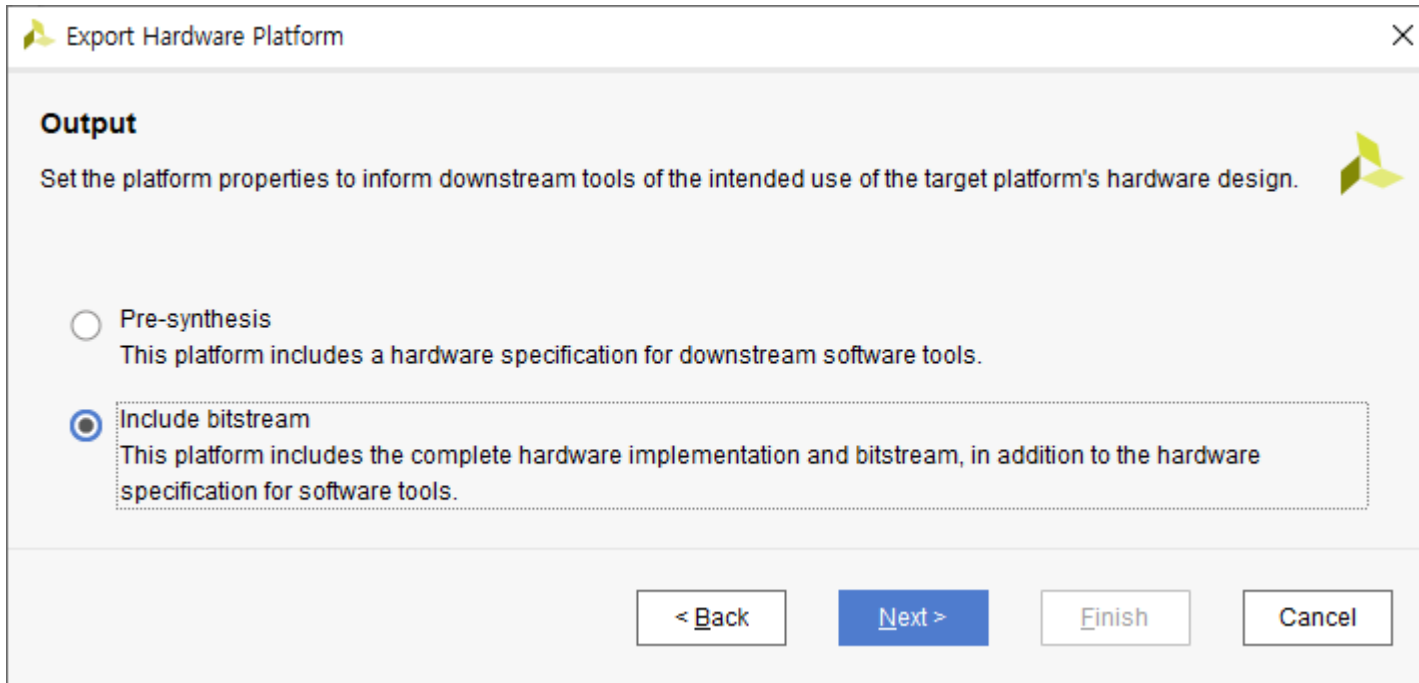
xsa 파일을 생성하기 위하여 File - Export - Export Hardware... 를 클릭합니다.



Next를 클릭합니다.

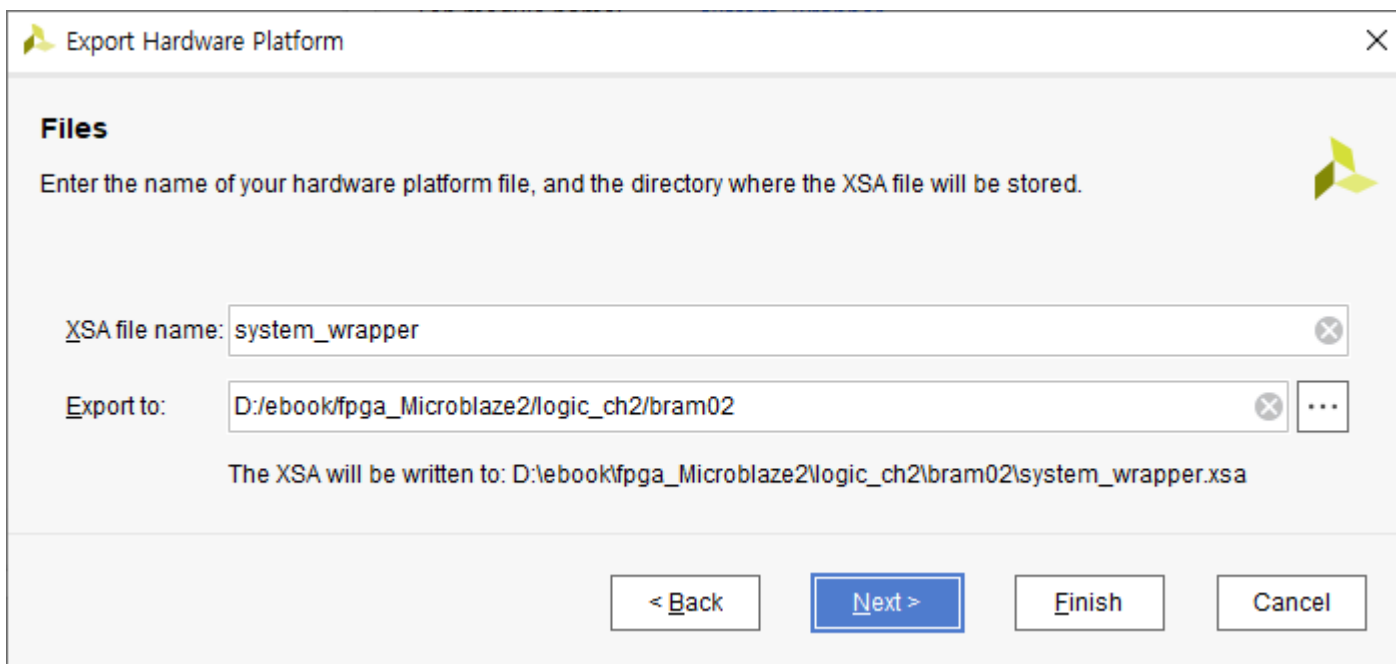


Include bitstream 을 선택하고 Next를 클릭합니다.



The dialog box is titled "Export Hardware Platform" and has a close button (X) in the top right corner. The "Output" tab is selected, showing the instruction: "Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design." There are two radio button options: "Pre-synthesis" (unselected) and "Include bitstream" (selected). The "Include bitstream" option is highlighted with a dashed border and includes the text: "This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools." At the bottom, there are four buttons: "< Back", "Next >" (highlighted in blue), "Finish", and "Cancel".

파일명, 위치를 확인하고 Next를 클릭합니다.

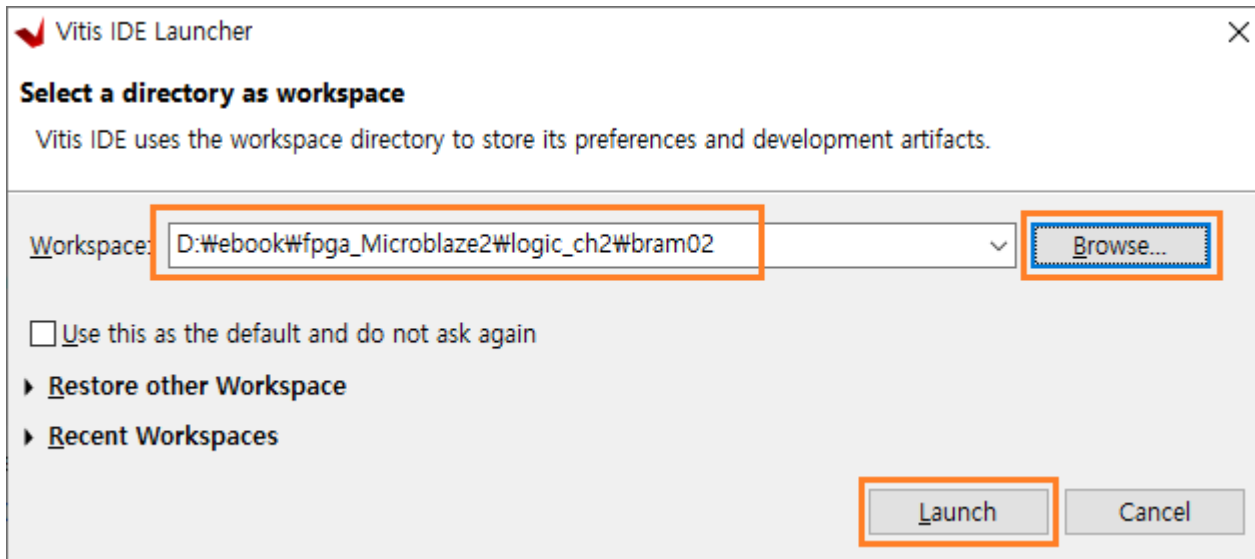


The dialog box is titled "Export Hardware Platform" and has a close button (X) in the top right corner. The "Files" tab is selected, showing the instruction: "Enter the name of your hardware platform file, and the directory where the XSA file will be stored." There are two input fields: "XSA file name:" with the text "system_wrapper" and "Export to:" with the text "D:/ebook/fpga_Microblaze2/logic_ch2/bram02". Both fields have a clear button (X) and a browse button (...). Below the fields, it says: "The XSA will be written to: D:\ebook\lpga_Microblaze2\logic_ch2\bram02\system_wrapper.xsa". At the bottom, there are four buttons: "< Back", "Next >" (highlighted in blue), "Finish", and "Cancel".

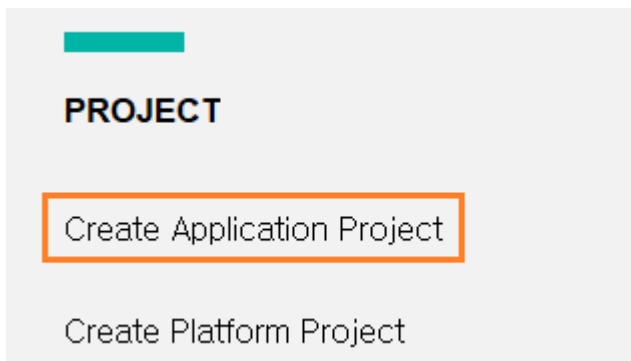
마지막으로 Finish를 클릭하면 xsa 파일이 생성됩니다.

10.4 응용 SW 구현

이번 장에서는 생성된 Block Memory를 Access 하는 응용 SW를 구현합니다. Tools - Launch Vitis IDE 를 클릭하여 Vitis 를 실행합니다. Browse... 를 클릭해서 프로젝트 폴더로 이동 후 Launch 를 클릭합니다.

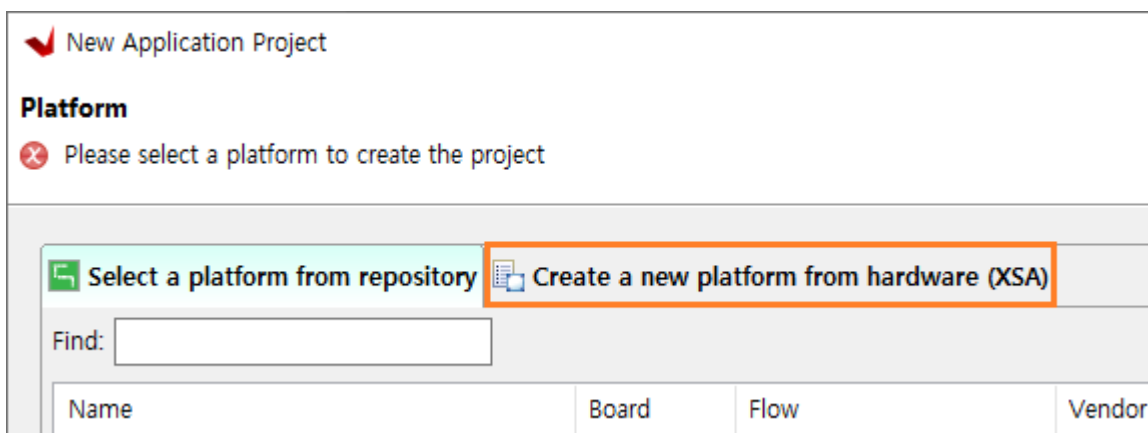


Vitis IDE가 실행됩니다. PROJECT - Create Application Project를 클릭합니다.

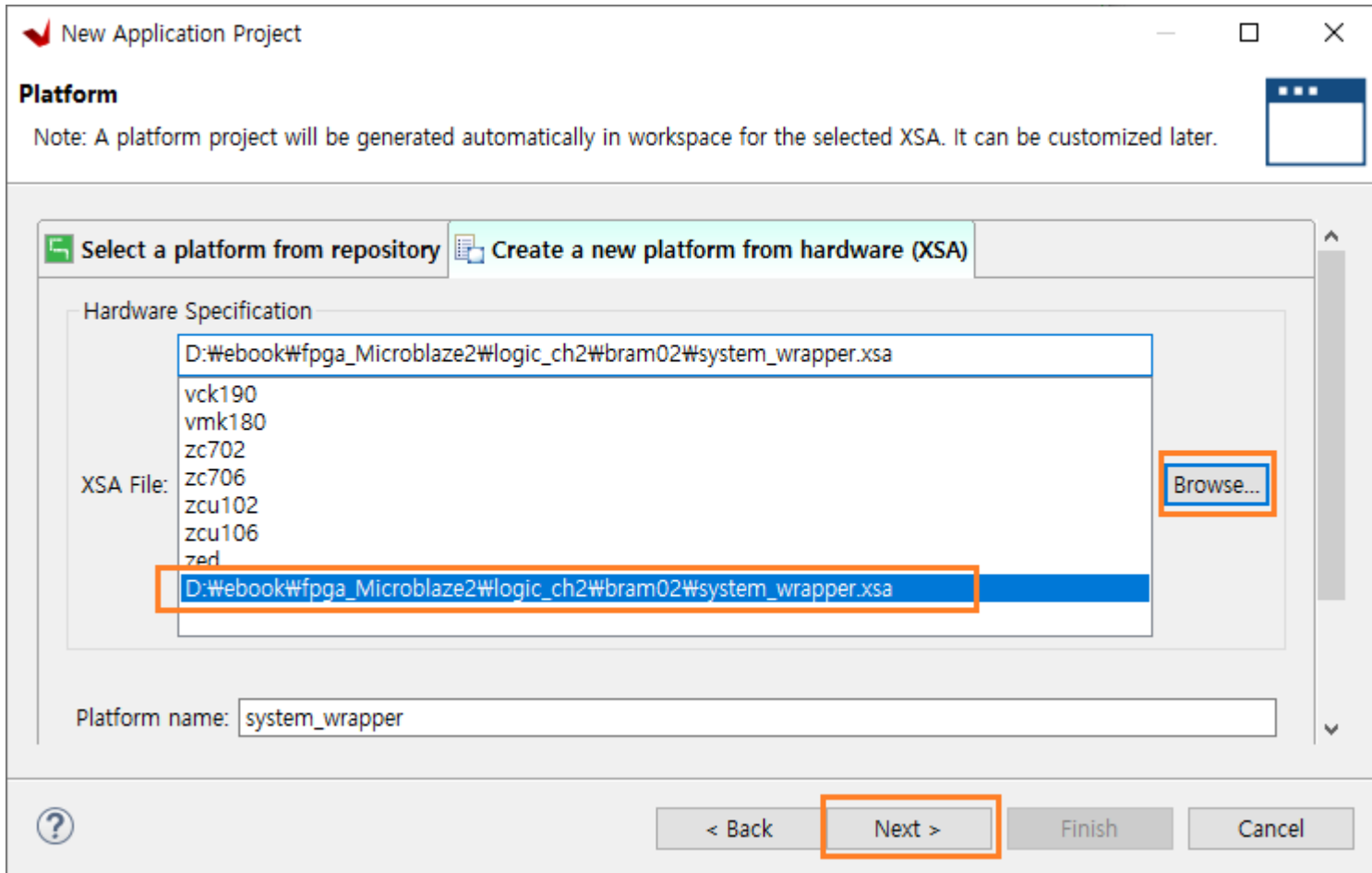


Create a New Application Project 윈도우에서 Next를 클릭합니다.

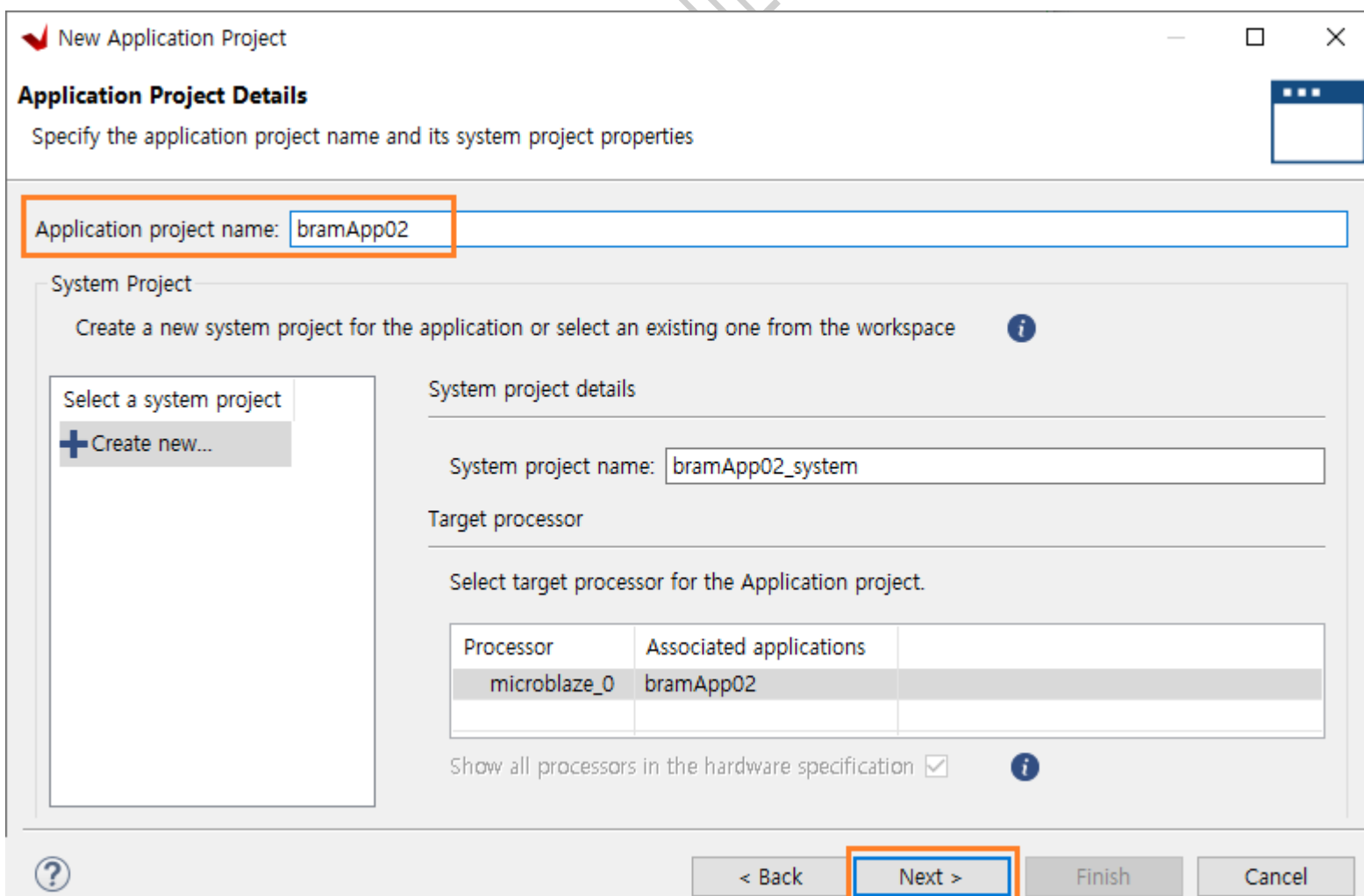
Platform 윈도우에서 Create a new platform from hardware (XSA)를 클릭합니다.



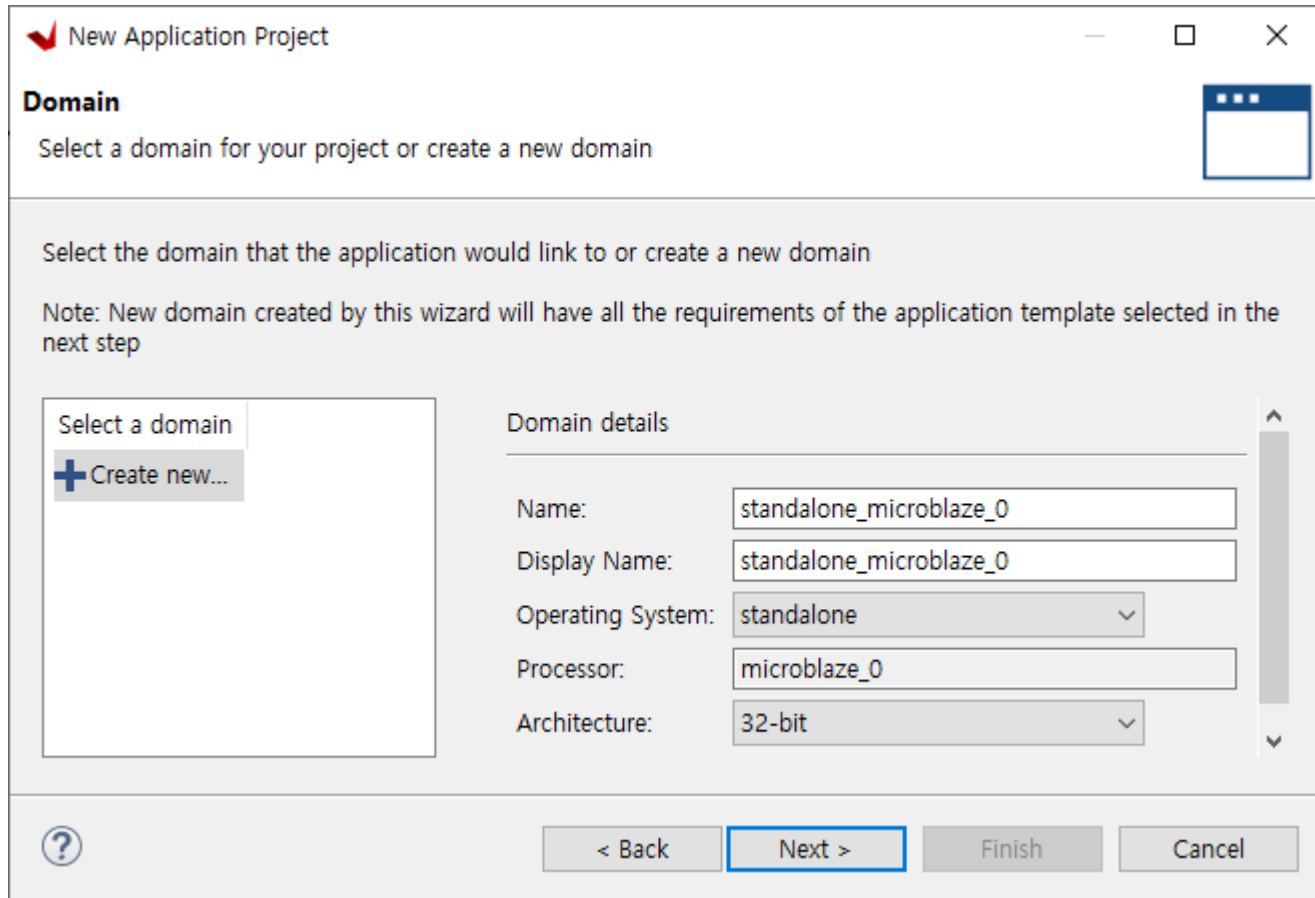
XSA File : 우측의 Browse... 를 클릭하여 생성한 system_wrapper.xsa 파일을 선택하고 Next 버튼을 클릭합니다.



Application Project name : bramApp02 입력하고(또는 사용자 지정) Next를 클릭합니다.



Next를 클릭합니다.



New Application Project

Domain
Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

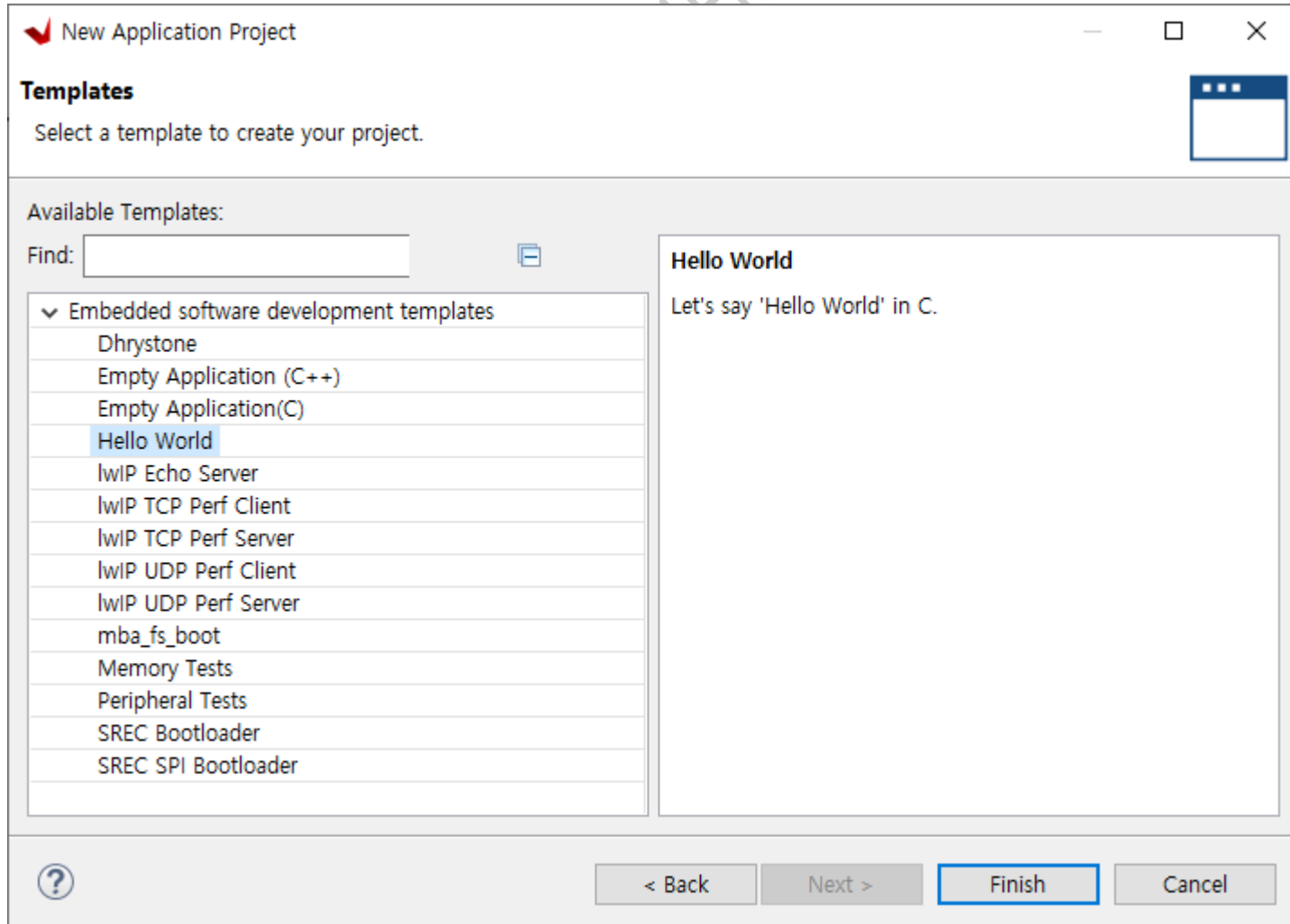
Select a domain
+ Create new...

Domain details

Name: standalone_microblaze_0
Display Name: standalone_microblaze_0
Operating System: standalone
Processor: microblaze_0
Architecture: 32-bit

< Back **Next >** Finish Cancel

Template 에서 Hello World를 선택하고 Finish를 클릭하면 프로젝트가 생성됩니다.



New Application Project

Templates
Select a template to create your project.

Available Templates:
Find:

- Embedded software development templates
 - Dhrystone
 - Empty Application (C++)
 - Empty Application(C)
 - Hello World**
 - lwIP Echo Server
 - lwIP TCP Perf Client
 - lwIP TCP Perf Server
 - lwIP UDP Perf Client
 - lwIP UDP Perf Server
 - mba_fs_boot
 - Memory Tests
 - Peripheral Tests
 - SREC Bootloader
 - SREC SPI Bootloader

Hello World
Let's say 'Hello World' in C.

< Back Next > **Finish** Cancel

helloworld.c 에 아래와 같이 코드를 추가합니다.

```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "sleep.h"
52
53
54 int main()
55 {
56     int i;
57     uint32_t rdata;
58     int err_cnt=0;
59
60     init_platform();
61
62     print("\r\n Hello World \r\n");
63     print("Successfully ran Hello World application\r\n");
64
65     xil_printf("write bram ..\r\n");
66     for(i=0; i<2048; i++)
67     {
68         Xil_Out32(XPAR_AXI_BRAM_CTRL_0_S_AXI_BASEADDR+(4*i), 0xaabb0000+i);
69     }
70
71     xil_printf("read bram ..\r\n");
72     for(i=0; i<2048; i++)
73     {
74         rdata = Xil_In32(XPAR_AXI_BRAM_CTRL_0_S_AXI_BASEADDR+(4*i));
75         if(rdata != 0xaabb0000+i) err_cnt++;
76         xil_printf("addr : %04x , w : %x r : %x \r\n", i, 0xaabb0000+i, rdata);
77     }
78     xil_printf(" error count : %d \r\n", err_cnt);
79
80     i = 0;
81     while(1)
82     {
83         sleep(100);
84         xil_printf("count : %d \r\n", i++);
85     }
86
87     cleanup_platform();
88     return 0;
89 }

```

- ✓ 라인 65 - 69 : 32bits(4바이트)씩 총 8KB를 write 합니다.
- ✓ 라인 71 - 78 : 32bits(4바이트)씩 총 8KB를 read 하고, write 한 데이터와 비교하여 error를 확인합니다.

Xil_In32(), Xil_Out32() 함수는 inline 함수로 정의된 메모리에 Access 하는 함수입니다. “xil_io.h”에 정의되어 있습니다.

```

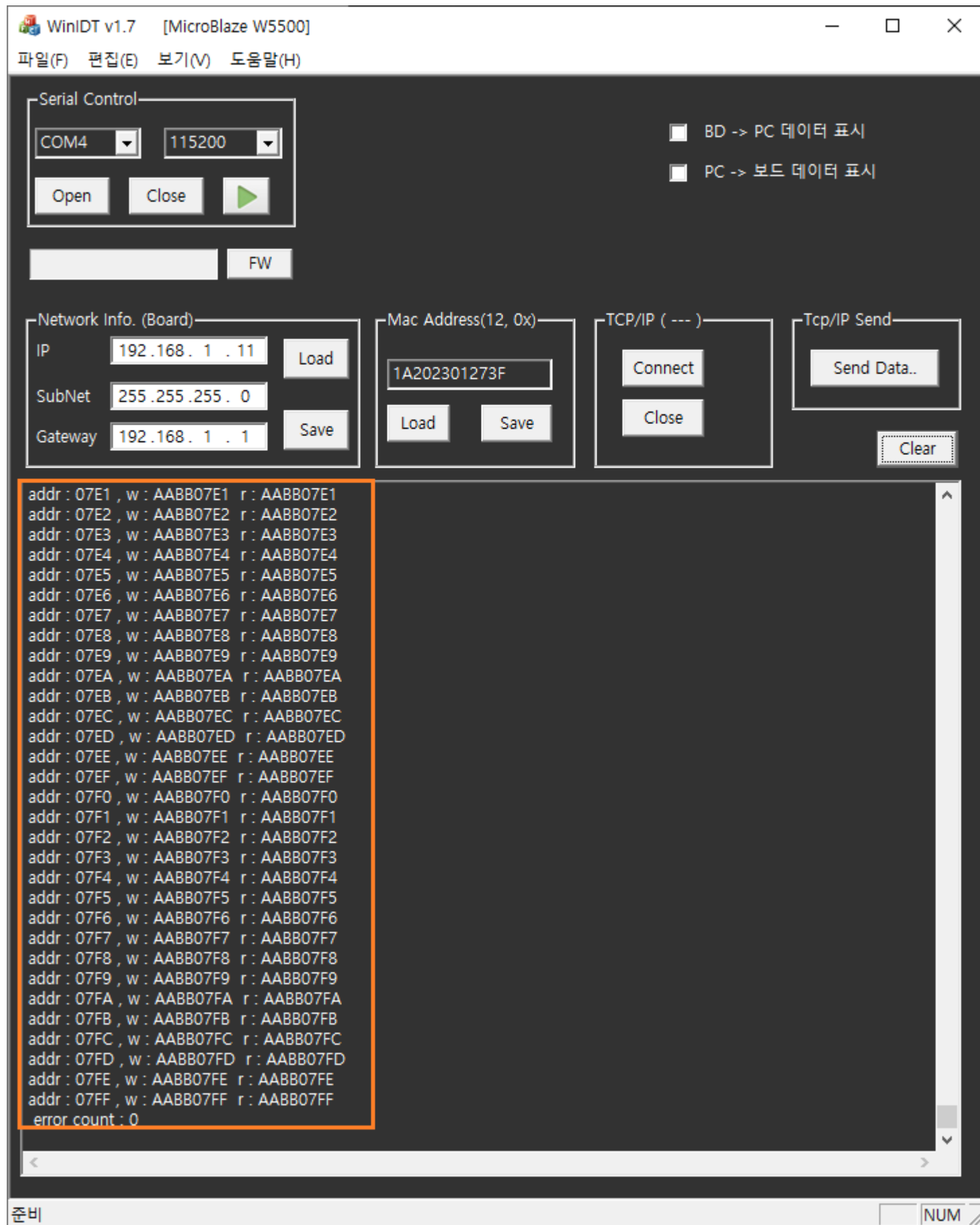
134 static INLINE u32 Xil_In32(UINTPTR Addr)
135 {
136     return *(volatile u32 *) Addr;
137 }
138
208 static INLINE void Xil_Out32(UINTPTR Addr, u32 Value)
209 {
210     /* write 32 bit value to specified address */
211     #ifndef ENABLE_SAFETY
212         volatile u32 *LocalAddr = (volatile u32 *)Addr;
213         *LocalAddr = Value;
214     #else
215         XStl_RegUpdate(Addr, Value);
216     #endif
217 }

```

프로그램을 Build 합니다. Project - Build All

프로그램을 다운로드 합니다. Xilinx - Program Device

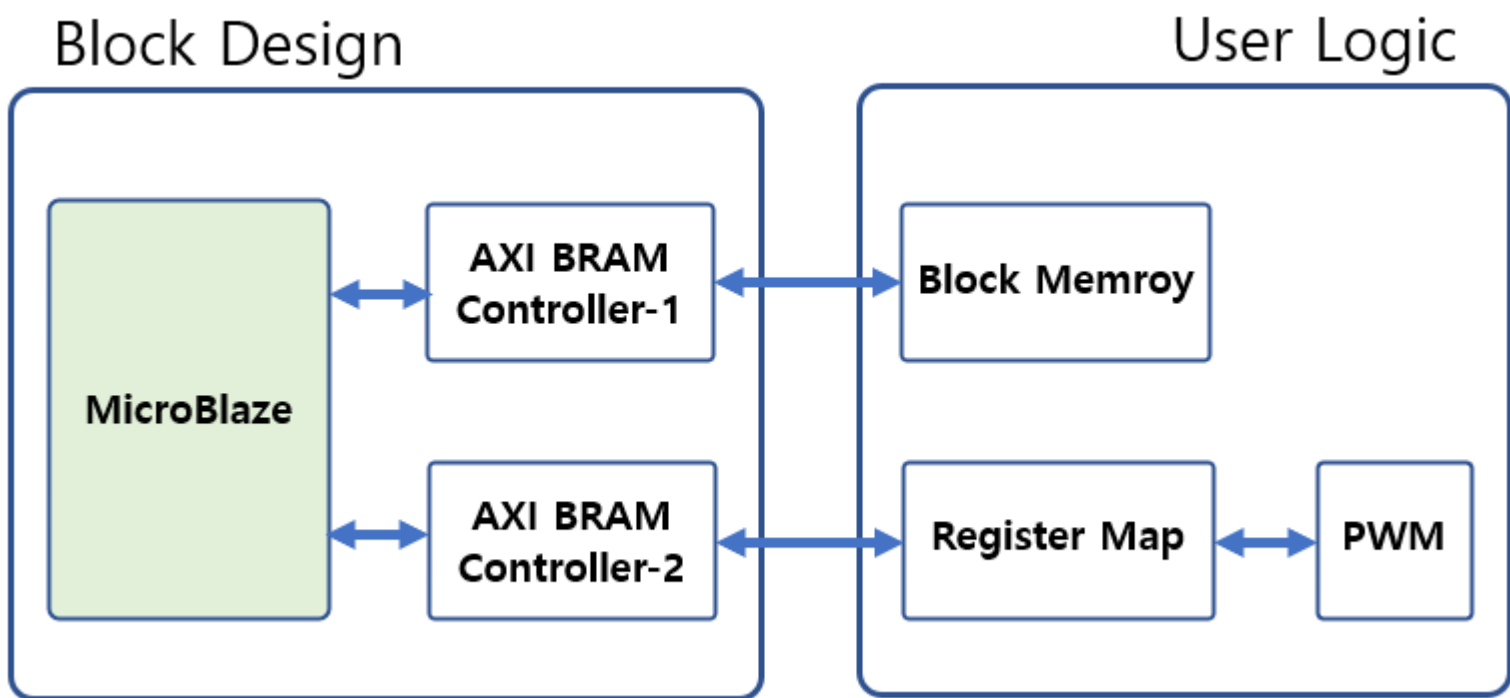
WinIDT를 이용하여 결과를 확인합니다.



11. Block Memory Interface - 2

이번 장에서는 Block Memory Interface를 이용한 User Logic Register Map을 구현합니다. 6장에서 SPI Interface (Master / Slave)를 통하여 User Logic Register Map을 구현하였지만, 속도나 효율면에서는 이번장에서 구현하는 방법이 더 좋은 것 같습니다. (사용자에 따라 달라질 수 있습니다)

이번장에서는 Block Memory Controller는 Block Design에서 구현하고, Block Memory Generator와 User Logic Register map은 사용자가 직접 구현하도록 하겠습니다. 아래는 Block Diagram을 보여줍니다. 응용 SW는 Microblaze에서 AXI BRAM Controller-2에 Memory를 Access 하듯이 Register Map에 Access하고, 결과적으로 PWM을 제어할 수 있습니다.



11.1 프로젝트 생성

vivado 2022.1을 실행하고, Create Project를 클릭합니다. (이번 장부터 상세한 설명을 생략합니다)

Create a New Vivado Project 윈도우에서 Next를 클릭합니다.

Project name : userMap으로 입력하고 Next를 클릭합니다. (프로젝트 위치를 설정합니다)

Project Type : RTL Project 를 선택하고 Next를 클릭합니다.

Add Sources, Add Constraints : Next를 클릭합니다. 파일은 나중에 추가합니다.

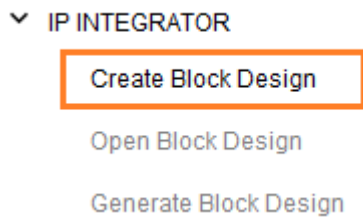
Part : xc7a35tcsg324-1을 선택하고 Next를 클릭합니다.

Finish를 클릭해서 프로젝트를 생성합니다.



11.2 Block Design

새로운 Block을 생성하기 위하여 PROJECT MANAGER에서 IP INTEGRATOR - Create Block Design을 클릭합니다.



Design Name : system 으로 하고 OK를 클릭합니다.

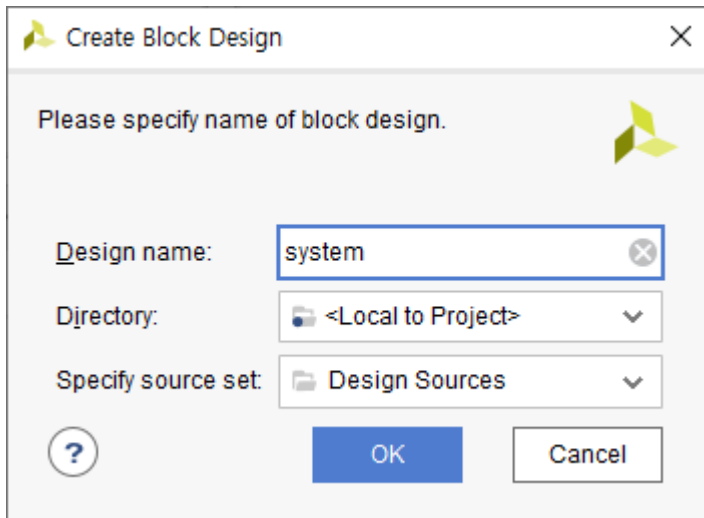


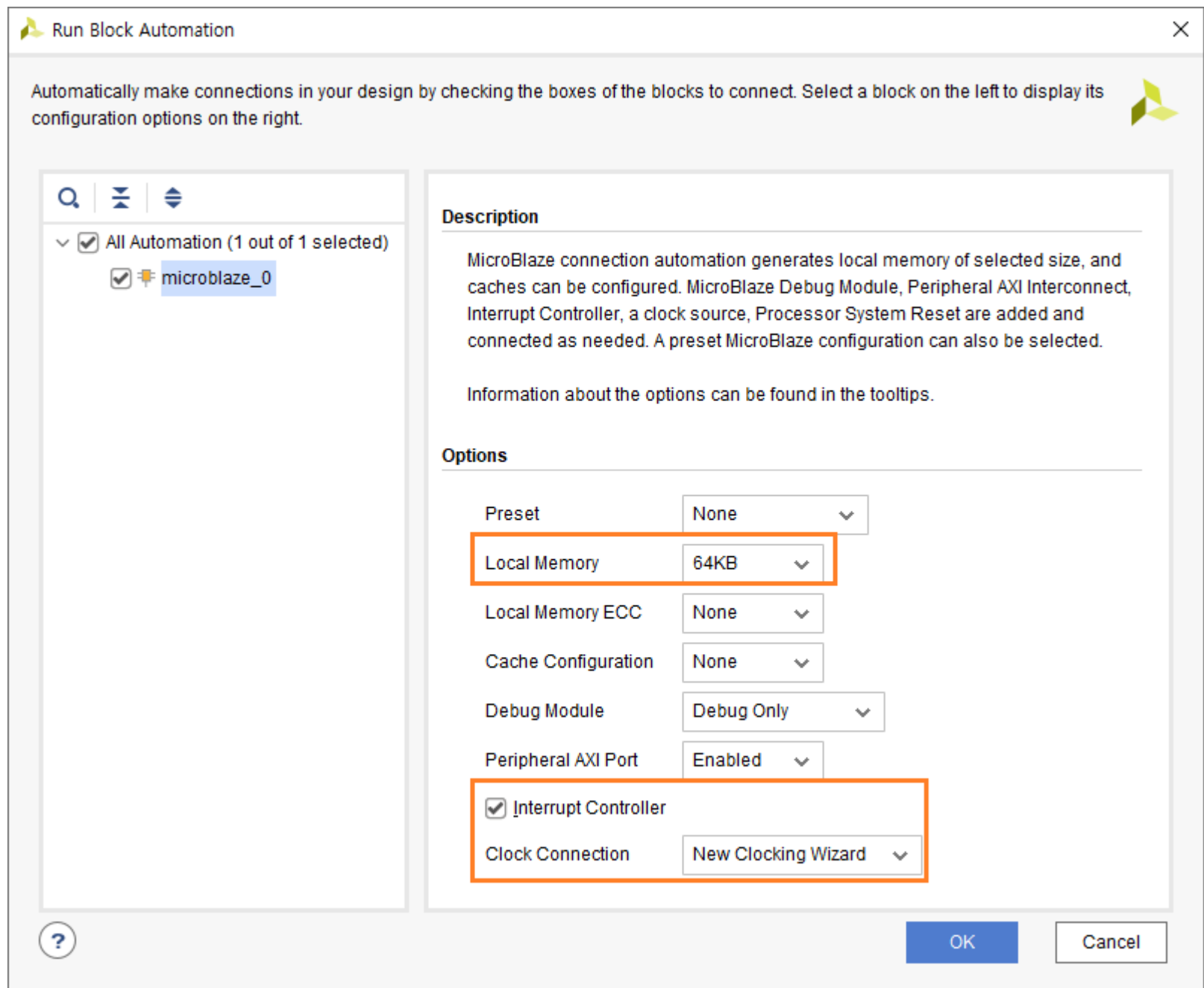
Diagram 윈도가 생성되었습니다. Add IP (+ 아이콘)을 클릭하여 5개의 IP를 추가합니다. Run Block Automation, Run Connection Automation은 5개를 모두 추가한 후에 진행합니다.

- ✓ MicroBlaze
- ✓ AXI Uartlite
- ✓ AXI GPIO
- ✓ AXI BRAM Controller - 1
- ✓ AXI BRAM Controller - 2

5개의 IP를 추가한 후에 Run Block Automation을 클릭합니다

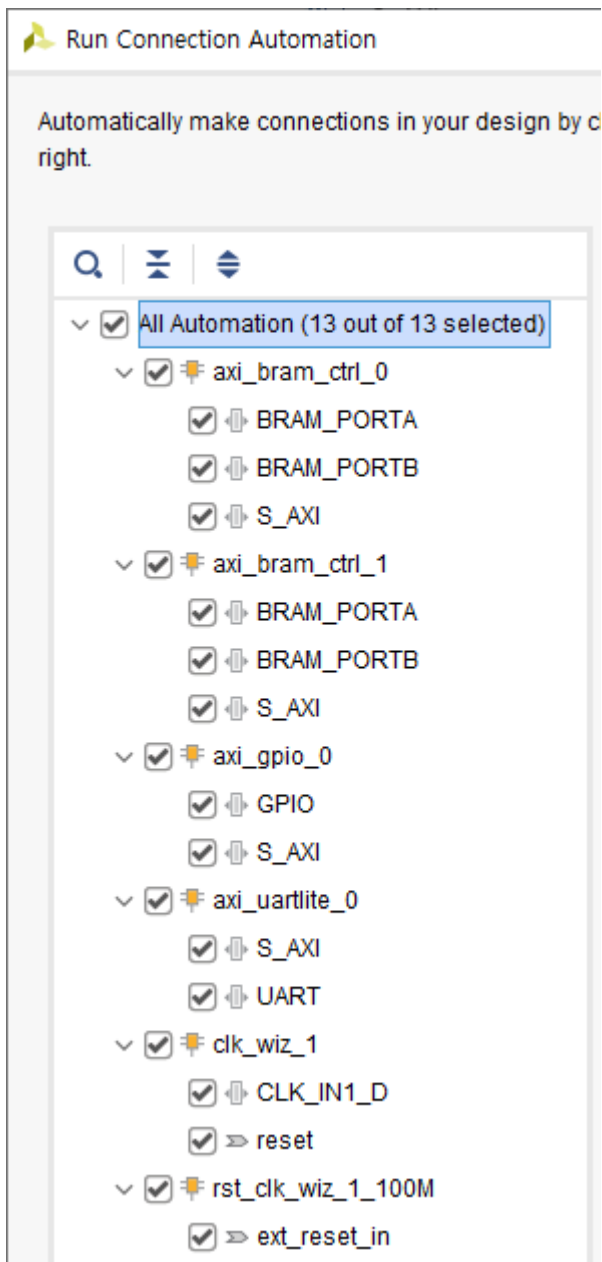
Microblaze의 속성을 설정합니다.

- ✓ Local Memory : 64KB (32KB도 가능하지만, 64KB로 넉넉히 설정하겠습니다)
- ✓ Interrupt Controller : check
- ✓ Clock Connection : New Clocking Wizard



OK를 클릭합니다.

Run Connection Automation을 클릭합니다. All Automation을 클릭해서 전체 모듈을 선택하고 OK를 클릭합니다.



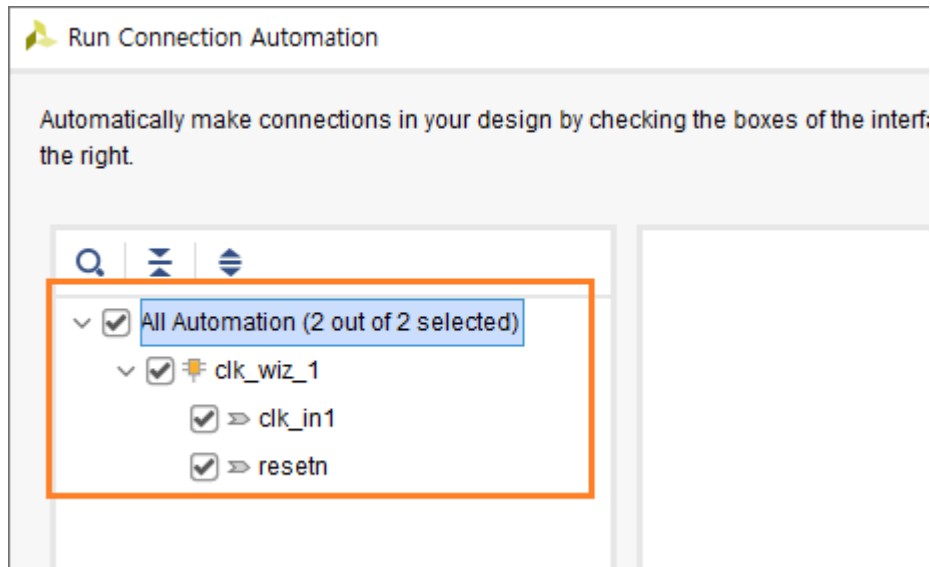
Clocking Wizard(clk_wiz_1)의 속성을 변경합니다. 해당 IP를 더블 클릭하면 속성창이 나타납니다.

- ✓ Input Clock Source : Single ended Clock capable pin
- ✓ Reset Type : Active Low

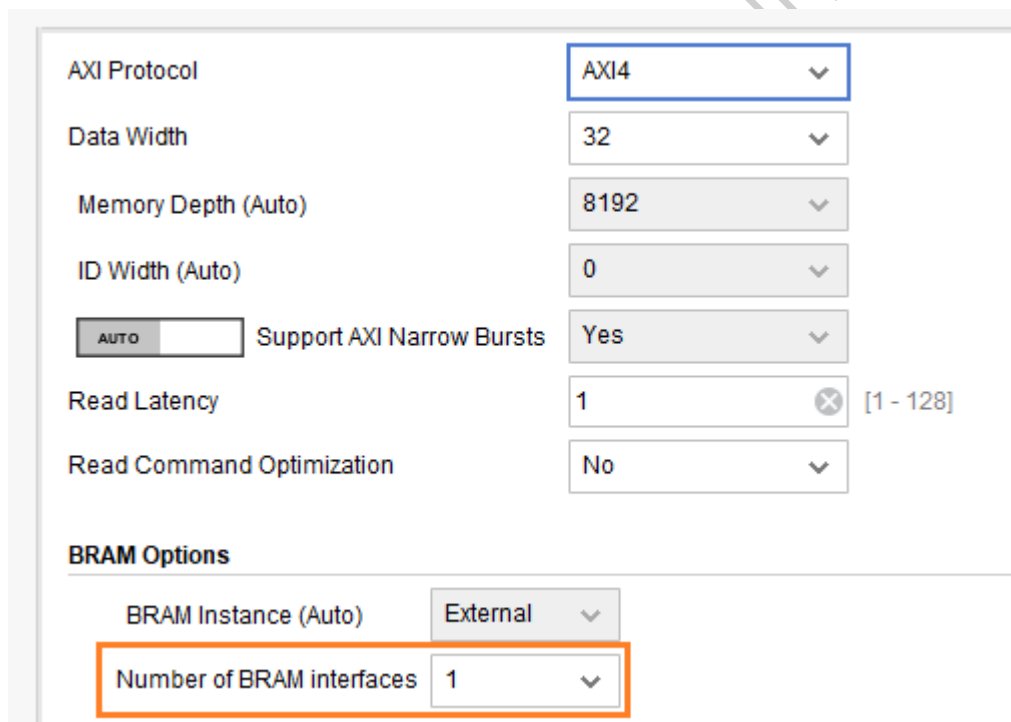
속성 변경 후 OK를 클릭합니다.

Run Connection Automation이 다시 활성화 되었습니다. reset_rtl_0 핀과 diff_clock_rtl_0 핀과 clk_wiz_1 을 서로 연결하기 위하여 Run Connection Automation을 클릭합니다. (수동으로 연결해도 됩니다)

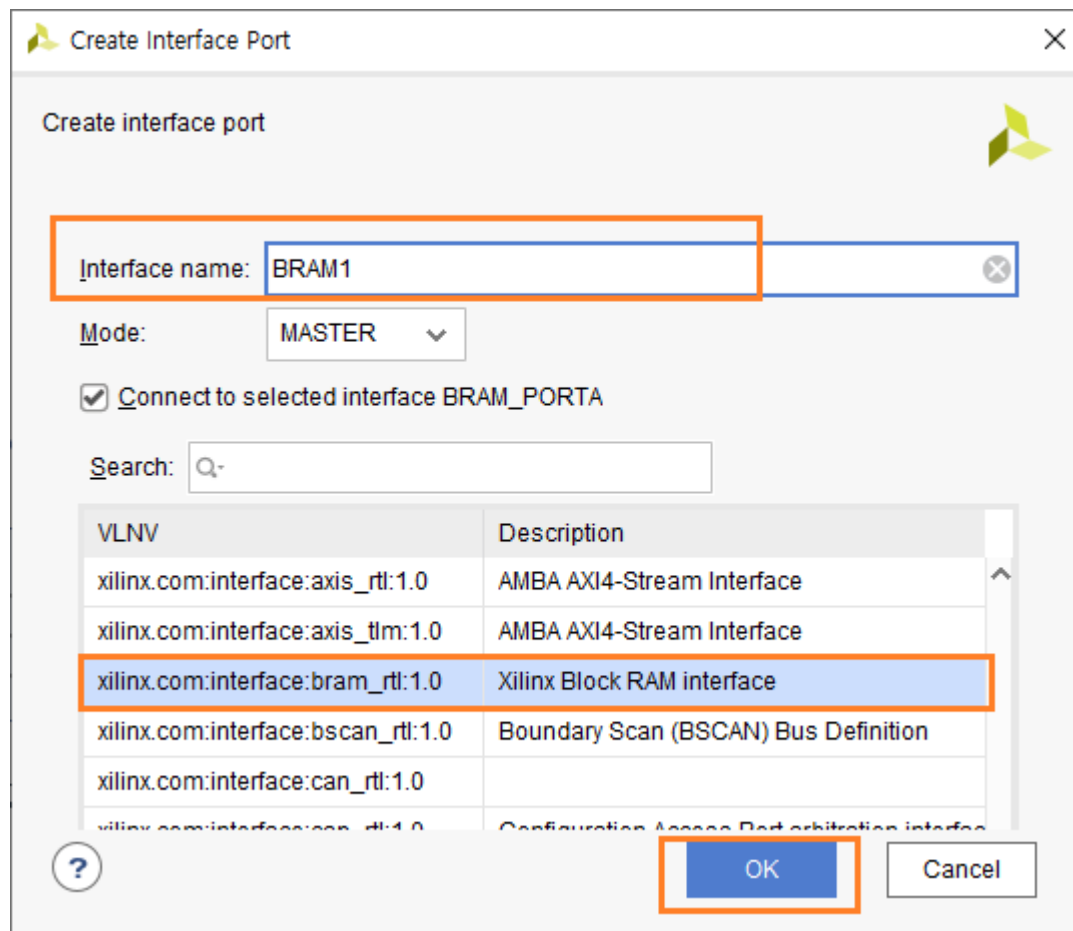
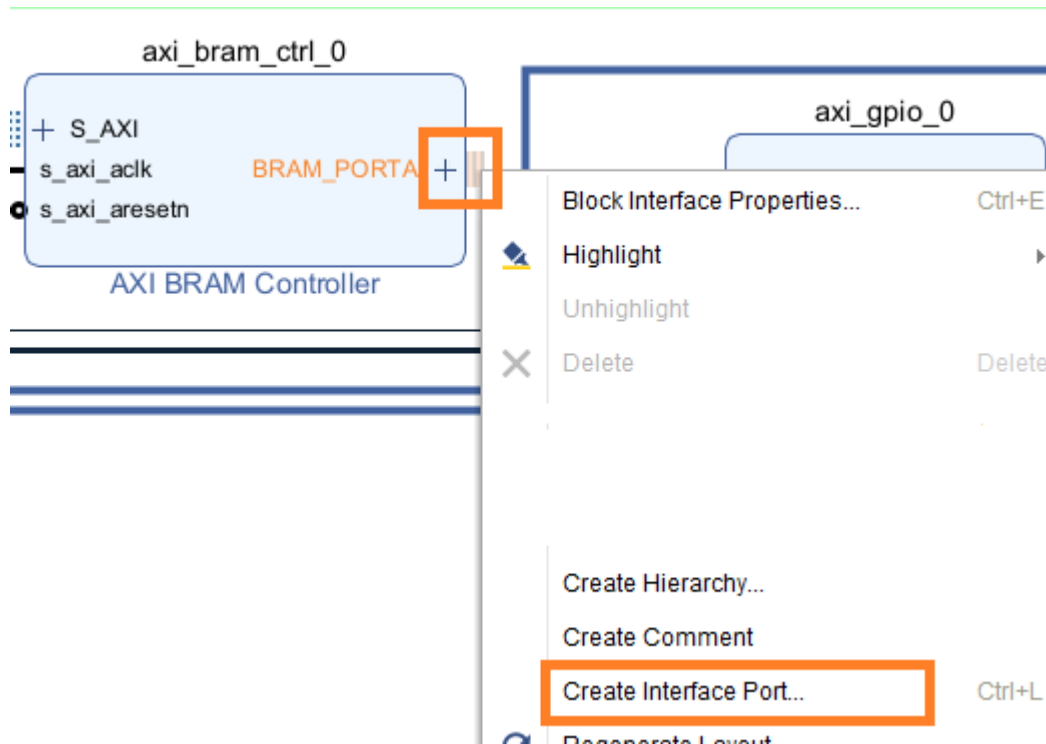
All Automation을 클릭하고 OK를 클릭합니다.



AXI BRAM Controller를 추가하면 자동으로 Block Memory Generator가 생성됩니다. 생성된 Block Memory Generator 2개 모두 삭제합니다. (axi_bram_ctrl_0_bram, axi_bram_ctrl_1_bram, 선택 후 Delete 키를 눌러 삭제 합니다) AXI BRAM Controller의 속성을 변경합니다. axi_bram_ctrl_0, 1을 각각 더블 클릭해서 아래와 같이 속성을 변경합니다. Dual Port대신 Single Port를 사용하기 위하여 Number of BRAM interfaces의 값을 1로 변경하고 OK를 클릭합니다.

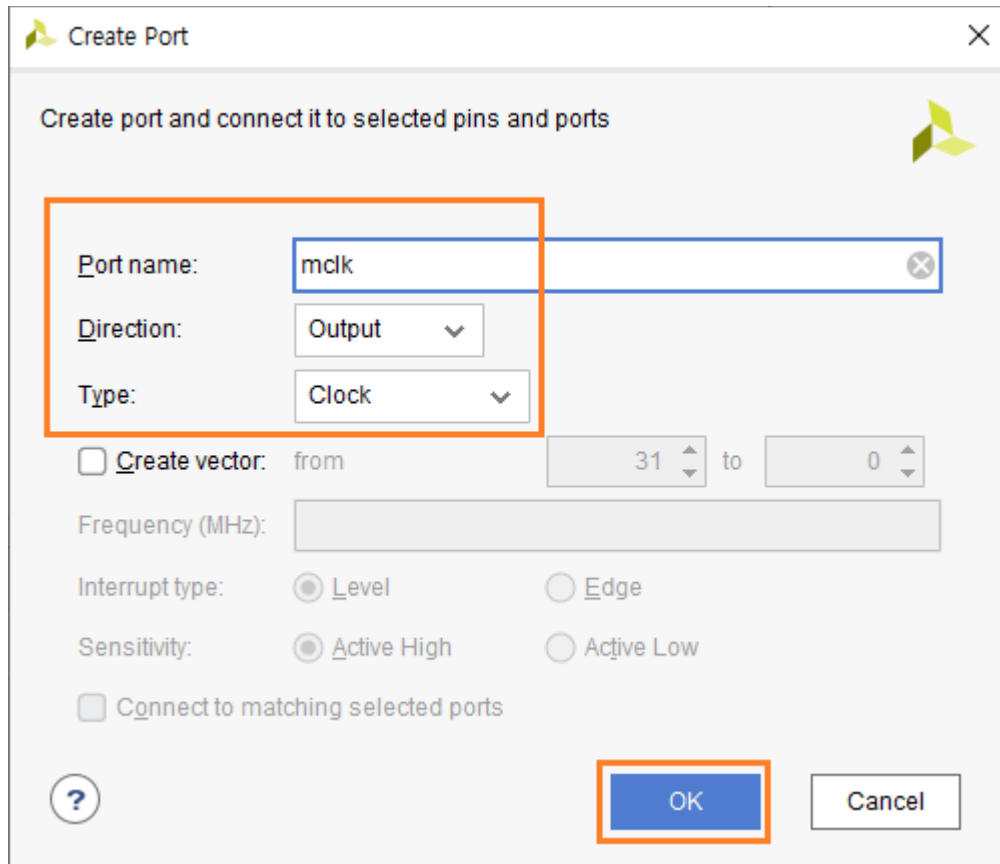


axi_bram_ctrl_0, 1의 BRAM_PORTA에 포트를 추가합니다. BRAM_PRTA - 우클릭 - Create Interface Port...를 클릭합니다. Xilinx Block RAM Interface 가 선택되어 있습니다. Interface name : BRAM1 으로 입력하고, OK를 클릭해서 Port를 추가합니다.



axi_bram_ctrl_1 에서는 Interface name : BRAM2로 입력하여 포트를 생성합니다.

User Logic에서 사용할 clock port를 추가합니다. Diagram 빈 곳 클릭 - 우클릭 - Create Port...를 클릭합니다. 아래와 같이 설정하고 OK를 클릭합니다.



포트가 생성되었습니다. mclk 포트를 clk_wiz_1의 clk_out1 핀과 연결합니다.

axi_gpio_0를 더블 클릭해서 아래와 같이 속성을 변경합니다.

- ✓ All Outputs : check
- ✓ GPIO Width : 4

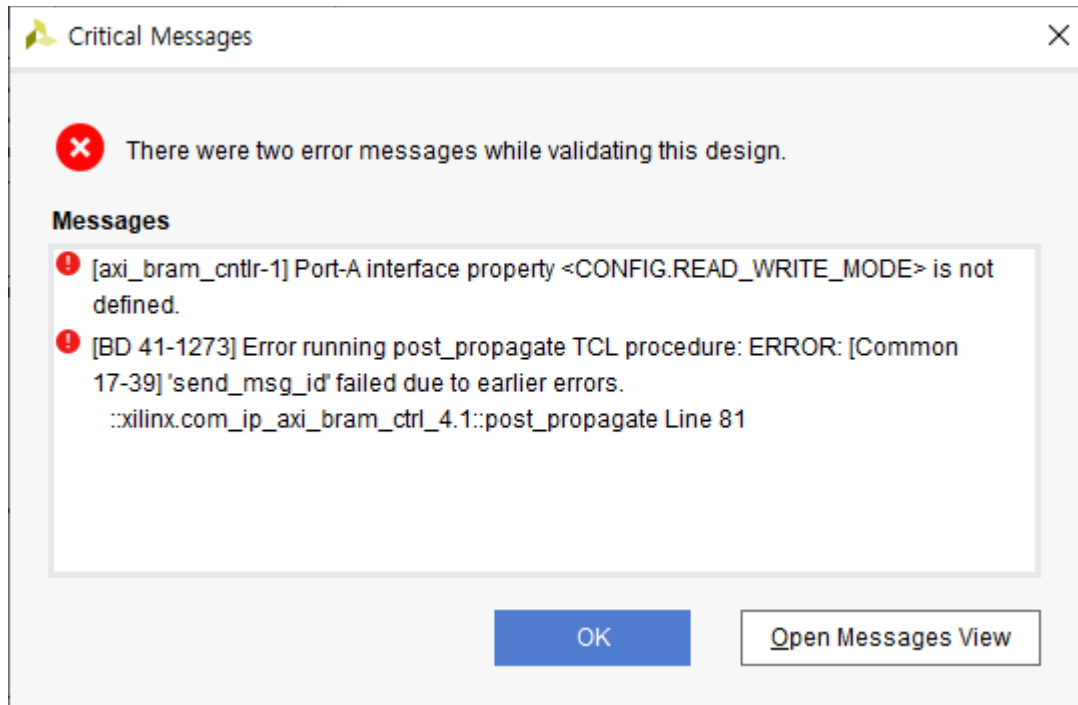
필요 없는 포트를 삭제하고 포트 이름을 변경합니다. 포트 이름 변경은 포트를 선택하면 왼쪽에 Port Properties 윈도우가 나타납니다. Name을 변경하면 됩니다.

- ✓ diff_clock_rtl_0 삭제
- ✓ reset_rtl_0 -> reset 으로 변경
- ✓ clk_100MHz -> sys_clock 으로 변경
- ✓ gpio_rtl_o -> gpio 로 변경
- ✓ uart_rtl_0 -> uart 로 변경

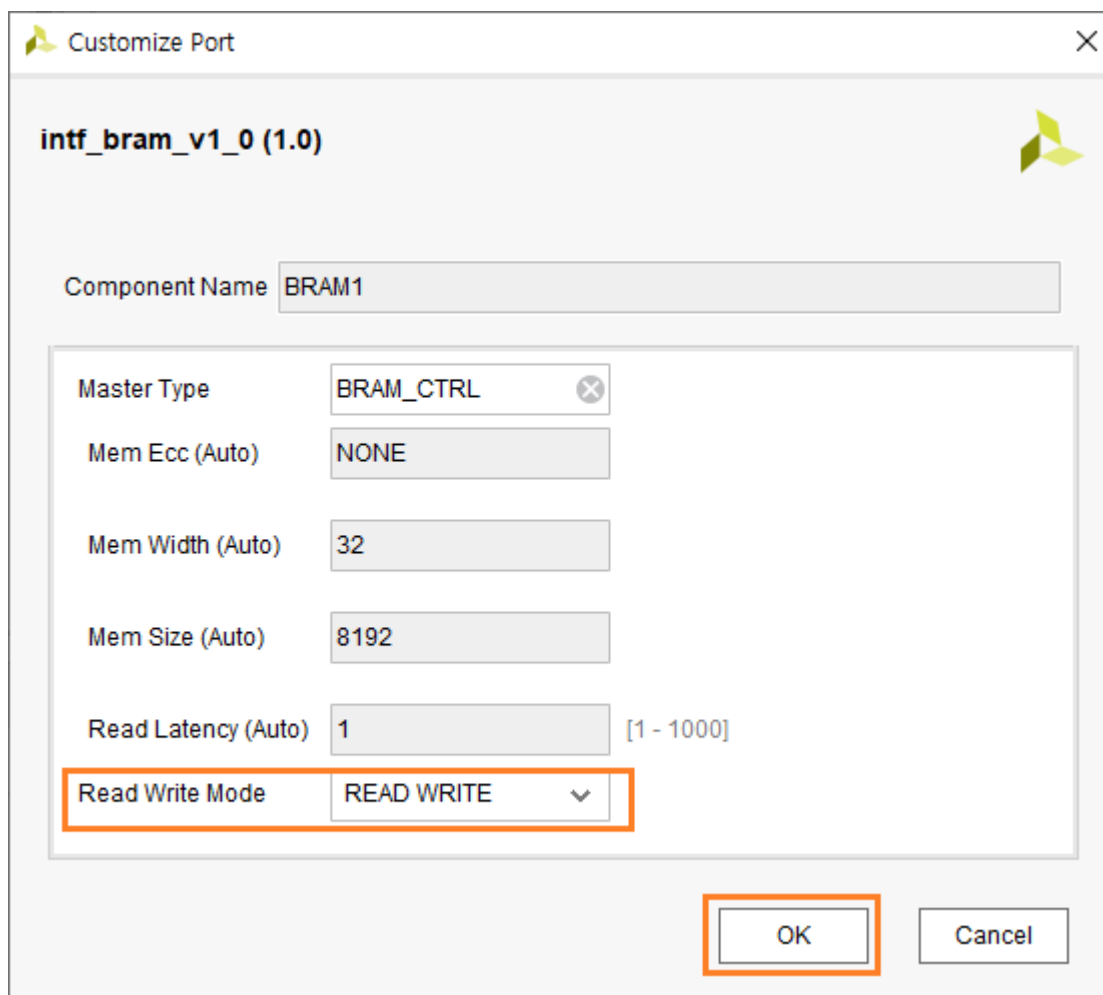
axi_uartlite_0의 interrupt 핀과 microblaze_0_xlconcat의 In0[0:0]핀을 서로 연결합니다.

axi_uartlite_0을 더블 클릭해서 Baud Rate : 115200으로 수정하고 OK를 클릭합니다.

모든 구성이 완료되었습니다. 저장하고 Validate Design 아이콘을 클릭해서 오류를 검사합니다. 아마 다음과 같은 오류가 발생할 것입니다.

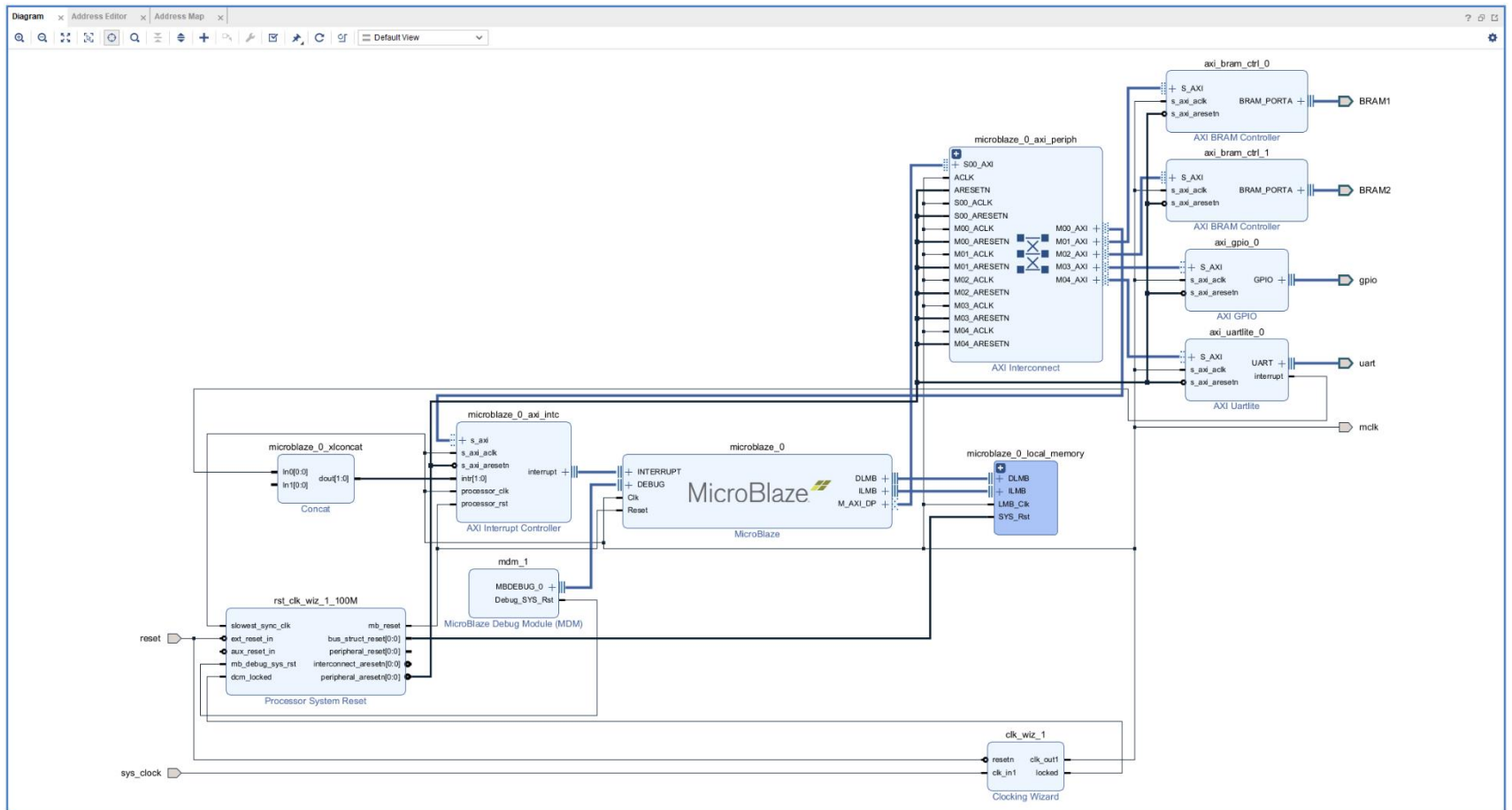


추가된 Port (BRAM1, BRAM2)의 Read/write 속성을 지정해 주어야 합니다. BRAM1 포트를 더블 클릭하면 포트 속성창이 나타납니다. Read Write Mode 를 READ WRITE 로 선택하고 OK를 클릭합니다. BRAM2 포트도 동일하게 설정합니다.

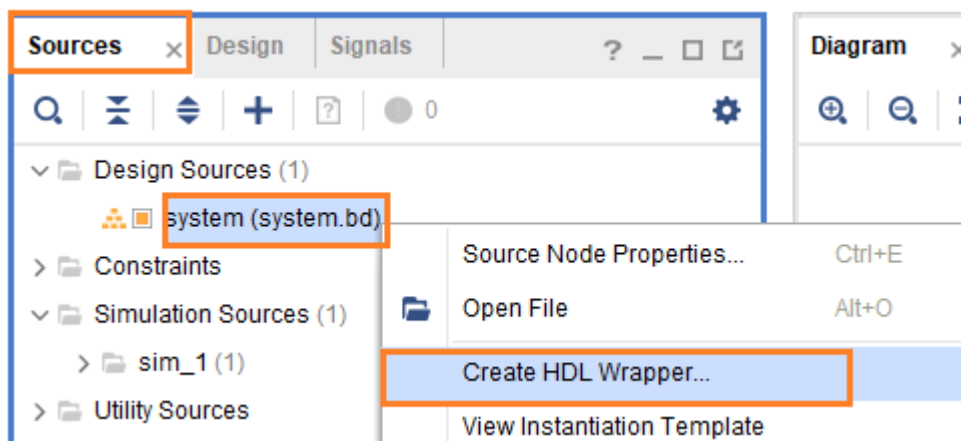


저장하고 다시 Validate Design 아이콘을 클릭합니다. 오류가 없으면 Validation Successful 창이 나타납니다. Regenerate Layout 아이콘을 클릭합니다.

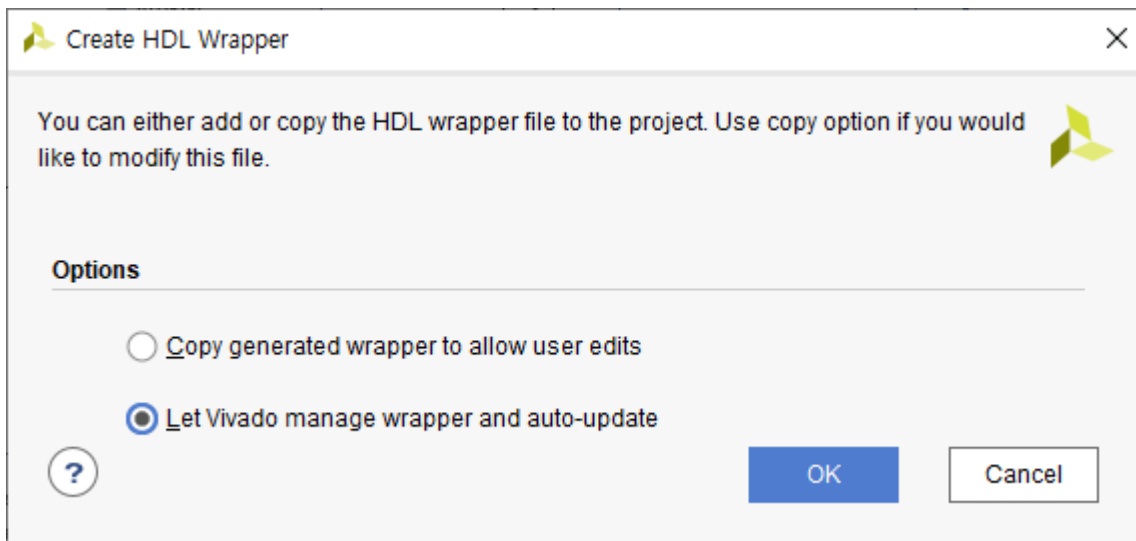
아래 그림은 최종 Diagram을 보여줍니다. (자료실에 Diagram_userMap_ch11.png 파일을 참조하세요)



HDL Wrapper를 생성합니다. Sources 탭에서 Design Sources - system - 우클릭 - Create HDL Wrapper... 를 클릭합니다.



OK를 클릭합니다.



HDL Wrapper 파일이 생성되었습니다. (system_wrapper.v)

IHIL

11.3 User Logic Design

이번 장에서는 User Logic을 추가합니다. 먼저 axi_bram_ctrl_0(BRAM1 포트)와 연결할 Block Memory를 생성합니다.

Diagram 윈도를 닫고, PROJECT MANGER - IP Catalog 를 클릭합니다. IP Catalog 탭에서 Memories & Storage Elements - RAMs & ROMs & BRAM - Block Memory Generator를 더블 클릭합니다.

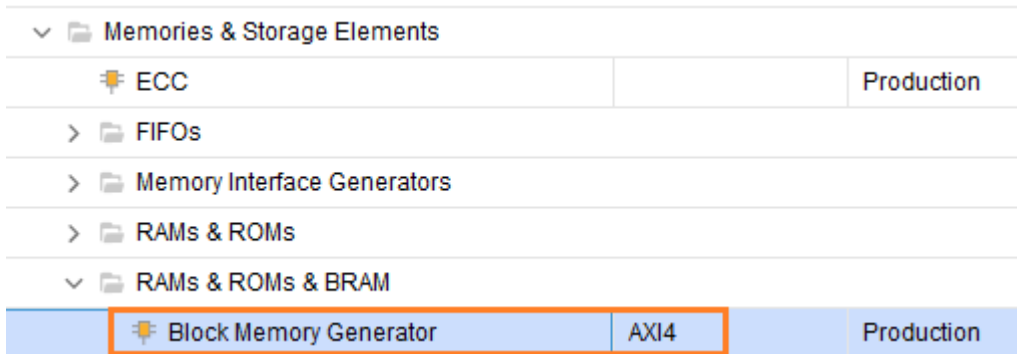
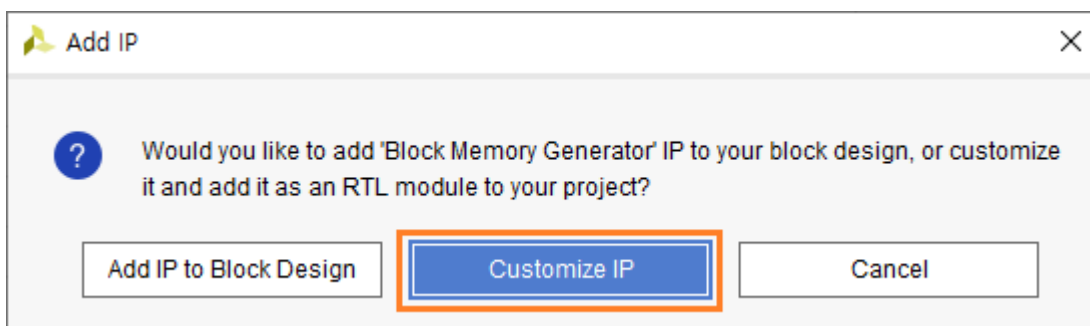
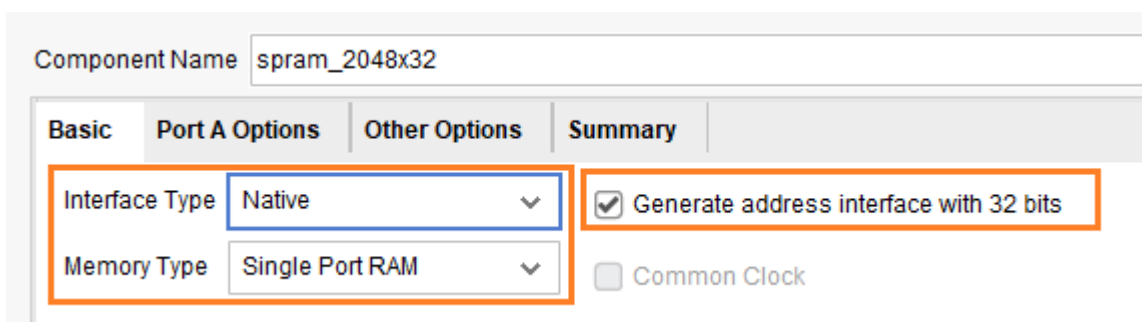


Diagram 윈도를 닫지 않고 진행하면 아래와 같이 윈도가 나타납니다. Block Memory를 Block Design에 추가할 것인지, 사용자 로직에 추가할 것인지를 묻습니다. Customize IP를 클릭합니다. (Diagram 윈도를 닫고 진행하면 아래 윈도가 나타나지 않습니다.)



Component Name : spram_2048x32 로 입력합니다.

Basic 탭에서는 아래와 같이 설정합니다.



Port A Options 탭에서는 아래와 같이 설정합니다.

- ✓ Write Depth, Read Depth : 2048
- ✓ Operating Mode : Write First
- ✓ Enable Port Type : Use ENA Pin
- ✓ Primitives Output Register : Unchecked
- ✓ RSTA Pin : Checked

Basic | **Port A Options** | Other Options | Summary

Memory Size

Write Width: 32 (Range: 32 to 1024 (bits))

Read Width: 32

Write Depth: 2048 (Range: 2 to 1048576)

Read Depth: 2048

Operating Mode: Write First

Enable Port Type: Use ENA Pin

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☒ RSTA Pin (set/reset pin) Output Reset Value (Hex): 0

☐ Reset Memory Latch Reset Priority: CE (Latch or Register Enable)

READ Address Change A

☐ Read Address Change A

나머지는 기본값을 사용하고 OK 를 클릭해서 Block Memory를 생성합니다.

User Register Map 모듈을 설계합니다. Block Design에서 설계했던 BRAM2 포트와 연결되는 모듈을 설계합니다. 먼저 system_wrapper.v 파일을 열어서 BRAM2 포트의 내용을 확인합니다. 아래는 system_wrapper 모듈의 input/output 포트를 보여줍니다.

```

33  output [12:0]BRAM1_addr;
34  output BRAM1_clk;
35  output [31:0]BRAM1_din;
36  input [31:0]BRAM1_dout;
37  output BRAM1_en;
38  output BRAM1_rst;
39  output [3:0]BRAM1_we;
40  output [12:0]BRAM2_addr;
41  output BRAM2_clk;
42  output [31:0]BRAM2_din;
43  input [31:0]BRAM2_dout;
44  output BRAM2_en;
45  output BRAM2_rst;
46  output [3:0]BRAM2_we;
47  output [3:0]gpio_tri_o;
48  output mclk;
49  input reset;
50  input sys_clock;
51  input uart_rxd;
52  output uart_txd;

```

- ✓ 라인 40 - 46까지의 포트가 BRAM2 와 연관된 포트입니다. 이 포트들을 이용하여 User Register Map 모듈을 설계합니다.

Design Sources - 우클릭 - Add Sources를 클릭해서 “ax_reg” 모듈을 추가합니다. (자료실에서 다운받은 파일을 그대로 사용하시면 됩니다)

아래는 코드를 보여줍니다.

```

22  `define      rPWM_FREQ1      13'h0020
23  `define      rPWM_FREQ2      13'h0024
24  `define      rPWM_FREQ3      13'h0028
25  `define      rPWM_FREQ4      13'h002c
26
27  `define      rPWM_DUTY1       13'h0030
28  `define      rPWM_DUTY2       13'h0034
29  `define      rPWM_DUTY3       13'h0038
30  `define      rPWM_DUTY4       13'h003c

```

- ✓ 라인 22 - 30 : pwm_top 에 정의된 Frequency, Duty를 설정하기 위한 Address 입니다. BRAM2_addr이 13bits 이므로 13bits로 정의되었습니다.

```
32 module ax_reg(  
33     reset      ,  
34     clock      ,  
35     ena        ,  
36     wea        ,  
37     addr       ,  
38     din        ,  
39     dout       ,  
40  
41     pwm_freq1  ,  
42     pwm_freq2  ,  
43     pwm_freq3  ,  
44     pwm_freq4  ,  
45     pwm_duty1  ,  
46     pwm_duty2  ,  
47     pwm_duty3  ,  
48     pwm_duty4  ,  
49 );  
50 input      reset      ;  
51 input      clock      ;  
52 input      ena        ;  
53 input [3:0] wea        ;  
54 input [12:0] addr      ;  
55 input [31:0] din       ;  
56 output [31:0] dout     ;  
57  
58 output [15:0] pwm_freq1 ;  
59 output [15:0] pwm_freq2 ;  
60 output [15:0] pwm_freq3 ;  
61 output [15:0] pwm_freq4 ;  
62 output [6:0]  pwm_duty1 ;  
63 output [6:0]  pwm_duty2 ;  
64 output [6:0]  pwm_duty3 ;  
65 output [6:0]  pwm_duty4 ;
```

- ✓ 라인 32 - 49 : ax_reg 모듈 정의
- ✓ 라인 50 - 56 : Block Design에서 생성한 BRAM2 포트와 연결되는 포트
- ✓ 라인 58 - 64 : User Register, pwm frequency, duty를 설정함.


```

67 reg      [15:0]  pwm_freq1 ;
68 reg      [15:0]  pwm_freq2 ;
69 reg      [15:0]  pwm_freq3 ;
70 reg      [15:0]  pwm_freq4 ;
71 reg      [6:0]   pwm_duty1 ;
72 reg      [6:0]   pwm_duty2 ;
73 reg      [6:0]   pwm_duty3 ;
74 reg      [6:0]   pwm_duty4 ;
75 always @(posedge clock or posedge reset)
76 begin
77     if (reset) begin
78         pwm_freq1 <= 16'd100;
79         pwm_freq2 <= 16'd100;
80         pwm_freq3 <= 16'd100;
81         pwm_freq4 <= 16'd100;
82         pwm_duty1 <= 7'd0;
83         pwm_duty2 <= 7'd0;
84         pwm_duty3 <= 7'd0;
85         pwm_duty4 <= 7'd0;
86
87     end
88 else begin
89     pwm_freq1 <= ((addr==`rPWM_FREQ1) & ena & (wea==4'b1111)) ? din[15:0] : pwm_freq1 ;
90     pwm_freq2 <= ((addr==`rPWM_FREQ2) & ena & (wea==4'b1111)) ? din[15:0] : pwm_freq2 ;
91     pwm_freq3 <= ((addr==`rPWM_FREQ3) & ena & (wea==4'b1111)) ? din[15:0] : pwm_freq3 ;
92     pwm_freq4 <= ((addr==`rPWM_FREQ4) & ena & (wea==4'b1111)) ? din[15:0] : pwm_freq4 ;
93
94     pwm_duty1 <= ((addr==`rPWM_DUTY1) & ena & (wea==4'b1111)) ? din[6:0] : pwm_duty1 ;
95     pwm_duty2 <= ((addr==`rPWM_DUTY2) & ena & (wea==4'b1111)) ? din[6:0] : pwm_duty2 ;
96     pwm_duty3 <= ((addr==`rPWM_DUTY3) & ena & (wea==4'b1111)) ? din[6:0] : pwm_duty3 ;
97     pwm_duty4 <= ((addr==`rPWM_DUTY4) & ena & (wea==4'b1111)) ? din[6:0] : pwm_duty4 ;
98 end
99 end
100
101 wire      [31:0]  dout  = (addr==`rPWM_FREQ1 ) ? {16'b0, pwm_freq1} :
102                          (addr==`rPWM_FREQ2 ) ? {16'b0, pwm_freq2} :
103                          (addr==`rPWM_FREQ3 ) ? {16'b0, pwm_freq3} :
104                          (addr==`rPWM_FREQ4 ) ? {16'b0, pwm_freq4} :
105                          (addr==`rPWM_DUTY1 ) ? {16'b0, pwm_duty1} :
106                          (addr==`rPWM_DUTY2 ) ? {16'b0, pwm_duty2} :
107                          (addr==`rPWM_DUTY3 ) ? {16'b0, pwm_duty3} :
108                          (addr==`rPWM_DUTY4 ) ? {16'b0, pwm_duty4} : 32'b0 ;
109
110 endmodule

```

- ✓ 라인 67 - 74 : user register 생성
- ✓ 라인 75 - 99 : register에 값을 write 하는 코드 구현, addr, ena, wea가 모두 active 일 때 해당 register write
- ✓ 라인 101 - 108 : register 값을 read 하는 코드 구현, addr 에 따라 register read.

pwm module은 기존에 구현되었던 코드를 사용합니다. 자료실에서 다운로드 받은 mcu_pwm.v, pwm_top.v 를 복사해서 프로젝트 소스 폴더(bram03\srcs\sources_1\new)에 추가합니다.

Top Module을 추가합니다. Souces - Design Sources - 우클릭 - Add Sources...를 클릭해서 “userMapTop.v”을 추가합니다.

아래는 userMapTop.v 모듈의 소스를 보여줍니다.

```

23 module userMapTop (
24     reset          ,
25     sys_clock      ,
26     gpio           ,
27     uart_rxd       ,
28     uart_txd       ,
29     pwm_out        ,
30 );
31 input      reset          ;
32 input      sys_clock      ;
33 output [3:0] gpio         ;
34 input      uart_rxd       ;
35 output      uart_txd      ;
36 output [3:0] pwm_out      ;

```

✓ 라인 23 - 30 : userMapTop 모듈 정의

✓ 라인 31 - 36 : input/output 포트 설정, pwm_out[3:0]는 LD7 - LD4 에 연결됩니다. 응용 SW에서 pwm의 duty 를 변화시키면서 LED의 밝기가 변하는지 확인합니다. gpio[3:0]는 테스트/디버깅 용도로 사용됩니다. 이번장에서는 사용하지 않습니다.

```

38 wire [12:0] BRAM1_addr  ;
39 wire      BRAM1_clk     ;
40 wire [31:0] BRAM1_din   ;
41 wire [31:0] BRAM1_dout  ;
42 wire      BRAM1_en      ;
43 wire      BRAM1_rst     ;
44 wire [3:0] BRAM1_we      ;
45 wire [12:0] BRAM2_addr  ;
46 wire      BRAM2_clk     ;
47 wire [31:0] BRAM2_din   ;
48 wire [31:0] BRAM2_dout  ;
49 wire      BRAM2_en      ;
50 wire      BRAM2_rst     ;
51 wire [3:0] BRAM2_we      ;
52 wire [3:0] gpio         ;
53 wire      mclk          ;
54 wire      uart_txd       ;
55 system_wrapper system_wrapper (
56     .BRAM1_addr (BRAM1_addr ),
57     .BRAM1_clk  (BRAM1_clk  ),
58     .BRAM1_din  (BRAM1_din  ),
59     .BRAM1_dout (BRAM1_dout ),
60     .BRAM1_en   (BRAM1_en   ),
61     .BRAM1_rst  (BRAM1_rst  ),
62     .BRAM1_we   (BRAM1_we   ),
63     .BRAM2_addr (BRAM2_addr ),
64     .BRAM2_clk  (BRAM2_clk  ),
65     .BRAM2_din  (BRAM2_din  ),
66     .BRAM2_dout (BRAM2_dout ),
67     .BRAM2_en   (BRAM2_en   ),
68     .BRAM2_rst  (BRAM2_rst  ),
69     .BRAM2_we   (BRAM2_we   ),
70     .gpio_tri_o (gpio       ),
71     .mclk       (mclk       ),
72     .reset      (reset      ),
73     .sys_clock  (sys_clock  ),
74     .uart_rxd   (uart_rxd   ),
75     .uart_txd   (uart_txd   ),
76 );

```

✓ 라인 38 - 77 : system_wrapper 모듈을 추가합니다.

```

78 wire    [31:0] bram_addr1 = {19'b0, BRAM1_addr};
79 wire    rsta_busr1;
80 spram_2048x32 mem1 (
81     .clka      (BRAM1_clk   ), // input wire clka
82     .rsta      (BRAM1_rst   ), // input wire rsta
83     .ena       (BRAM1_en    ), // input wire ena
84     .wea       (BRAM1_we    ), // input wire [3 : 0] wea
85     .addra     (bram_addr1  ), // input wire [31 : 0] addra
86     .dina      (BRAM1_din   ), // input wire [31 : 0] dina
87     .douta     (BRAM1_dout  ), // output wire [31 : 0] douta
88     .rsta_busy (rsta_busr1  )  // output wire rsta_busy
89 );

```

- ✓ 라인 78 - 89 : 사용자가 생성한 Block Memory를 추가합니다. 주의할 사항은 addra[31:0] 가 32bits로 설정되어 있습니다. 따라서 72라인에 {19'b0, BARM1_addr[12:0]} 를 생성해서 입력해 주었습니다.
- ✓ 생성한 Block Memory는 “userMap.gen\sources_1\wip\spram_2048x32\spram_2048x32.vco” 파일을 참조하여 추가하면 됩니다.

```

91 wire    [15:0] pwm_freq1 ;
92 wire    [15:0] pwm_freq2 ;
93 wire    [15:0] pwm_freq3 ;
94 wire    [15:0] pwm_freq4 ;
95 wire    [6:0]  pwm_duty1  ;
96 wire    [6:0]  pwm_duty2  ;
97 wire    [6:0]  pwm_duty3  ;
98 wire    [6:0]  pwm_duty4  ;
99 wire    ax_reg_rst = ~BRAM2_rst;
100 ax_reg  ax_reg (
101     .reset      (ax_reg_rst  ),
102     .clock      (BRAM2_clk   ),
103     .ena        (BRAM2_en    ),
104     .wea        (BRAM2_we    ),
105     .addr       (BRAM2_addr  ),
106     .din        (BRAM2_din   ),
107     .dout       (BRAM2_dout  ),
108
109     .pwm_freq1  (pwm_freq1   ),
110     .pwm_freq2  (pwm_freq2   ),
111     .pwm_freq3  (pwm_freq3   ),
112     .pwm_freq4  (pwm_freq4   ),
113     .pwm_duty1  (pwm_duty1   ),
114     .pwm_duty2  (pwm_duty2   ),
115     .pwm_duty3  (pwm_duty3   ),
116     .pwm_duty4  (pwm_duty4   )
117 );

```

- ✓ 라인 91 - 117 : ax_reg 모듈을 추가합니다.
- ✓ 라인 99 : BRAM controller의 reset(BRAM2_rst)은 High Active 이고, ax_reg의 reset은 Low Active 입니다. 따라서 BRAM2_rst의 반전된 신호를 ax_reg의 reset으로 입력합니다.

```
119 wire          pwm_out1 ;
120 wire          pwm_out2 ;
121 wire          pwm_out3 ;
122 wire          pwm_out4 ;
123 pwm_top        pwm_top (
124     .reset      (reset      ),
125     .mclk        (mclk        ),
126
127     .pwm_freq1   (pwm_freq1   ),
128     .pwm_freq2   (pwm_freq2   ),
129     .pwm_freq3   (pwm_freq3   ),
130     .pwm_freq4   (pwm_freq4   ),
131     .pwm_duty1   (pwm_duty1   ),
132     .pwm_duty2   (pwm_duty2   ),
133     .pwm_duty3   (pwm_duty3   ),
134     .pwm_duty4   (pwm_duty4   ),
135
136     .pwm_out1    (pwm_out1    ),
137     .pwm_out2    (pwm_out2    ),
138     .pwm_out3    (pwm_out3    ),
139     .pwm_out4    (pwm_out4    )
140 );
141
142 wire [3:0]      pwm_out = {pwm_out4, pwm_out3, pwm_out2, pwm_out1} ;
143
144 endmodule
```

- ✓ 라인 119 - 140 : pwm_top 모듈을 추가합니다.
- ✓ 라인 142 : pwm1 ~4 출력을 pwm_out[3:0]로 출력합니다.

이힐

xdc 파일을 추가합니다. Constraints - 우클릭 - Add Sources...를 클릭해서 “userMapTop.xdc” 파일을 추가합니다. 자료실에서 다운로드 받은 파일의 내용을 추가합니다. 아래는 코드를 보여줍니다.

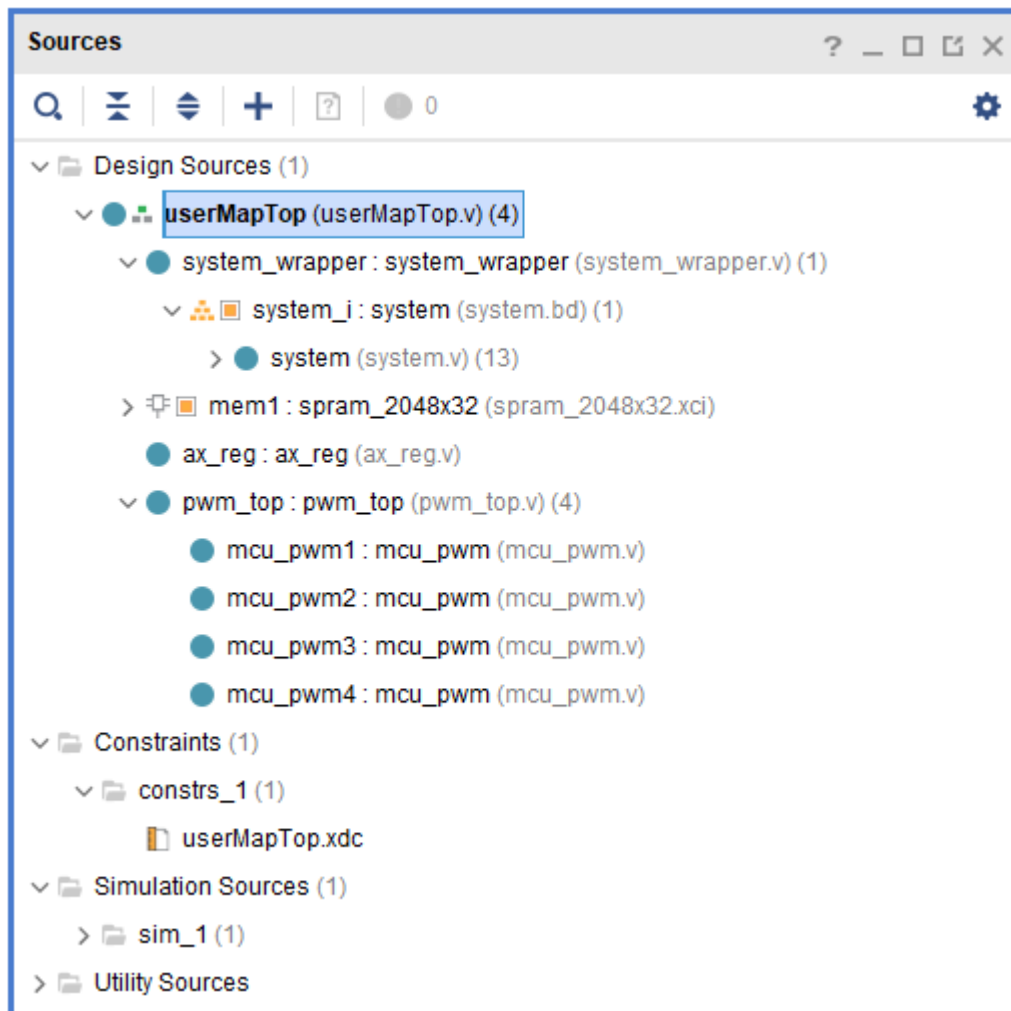
```

2 ## Clock signal
3 set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { sys_clock }];
4 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { sys_clock }];
5
6 set_property -dict { PACKAGE_PIN C2      IOSTANDARD LVCMOS33 } [get_ports { reset }];
7
8 ## LEDs
9 set_property -dict { PACKAGE_PIN H5      IOSTANDARD LVCMOS33 } [get_ports { pwm_out[0] }]; #LED4
10 set_property -dict { PACKAGE_PIN J5     IOSTANDARD LVCMOS33 } [get_ports { pwm_out[1] }]; #LED5
11 set_property -dict { PACKAGE_PIN T9     IOSTANDARD LVCMOS33 } [get_ports { pwm_out[2] }]; #LED6
12 set_property -dict { PACKAGE_PIN T10    IOSTANDARD LVCMOS33 } [get_ports { pwm_out[3] }]; #LED7
13
14 ## Pmod Header JD
15 set_property -dict { PACKAGE_PIN D4      IOSTANDARD LVCMOS33 } [get_ports { gpio[0] }];
16 set_property -dict { PACKAGE_PIN D3      IOSTANDARD LVCMOS33 } [get_ports { gpio[1] }];
17 set_property -dict { PACKAGE_PIN F4      IOSTANDARD LVCMOS33 } [get_ports { gpio[2] }];
18 set_property -dict { PACKAGE_PIN F3      IOSTANDARD LVCMOS33 } [get_ports { gpio[3] }];
19
20 ## Pmod Header JC
21 set_property -dict { PACKAGE_PIN V10     IOSTANDARD LVCMOS33 } [get_ports { uart_rxd }];
22 set_property -dict { PACKAGE_PIN V11     IOSTANDARD LVCMOS33 } [get_ports { uart_txd }];
23
24 #####
25 # MISC
26 #####
27 set_operating_conditions -ambient_temp 80.0
28
29 set_property CONFIG_MODE SPIx1 [current_design]
30 set_property CFGBVS VCCO [current_design]
31 set_property CONFIG_VOLTAGE 3.3 [current_design]
32
33 set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 1 [current_design]
34 set_property BITSTREAM.CONFIG.PERSIST NO [current_design]
35 set_property BITSTREAM.CONFIG.SPI_FALL_EDGE NO [current_design]
36 set_property BITSTREAM.CONFIG.CONFIGRATE 22 [current_design]

```

라인 9 - 12 : pwm 출력을 LED4 - LED7에 할당합니다. pwm duty를 변경하면 LED의 밝기가 변합니다.

아래는 전체 소스 구조를 보여줍니다.



userMapTop 모듈이 Top 모듈로 지정되어 있지 않으면, Top Module로 지정하고 Generate Bitstream 을 클릭하여 Bitstream을 생성합니다.

모든 IP들이 생성되고, Synthesis, Implementation을 진행하고 Bitstream을 생성합니다. 생성이 완료되면 Cancel 를 클릭합니다.

File - Export - Export Hardware...를 클릭하여 xsa 파일을 생성합니다.

Export Hardware Platform : Next를 클릭합니다.

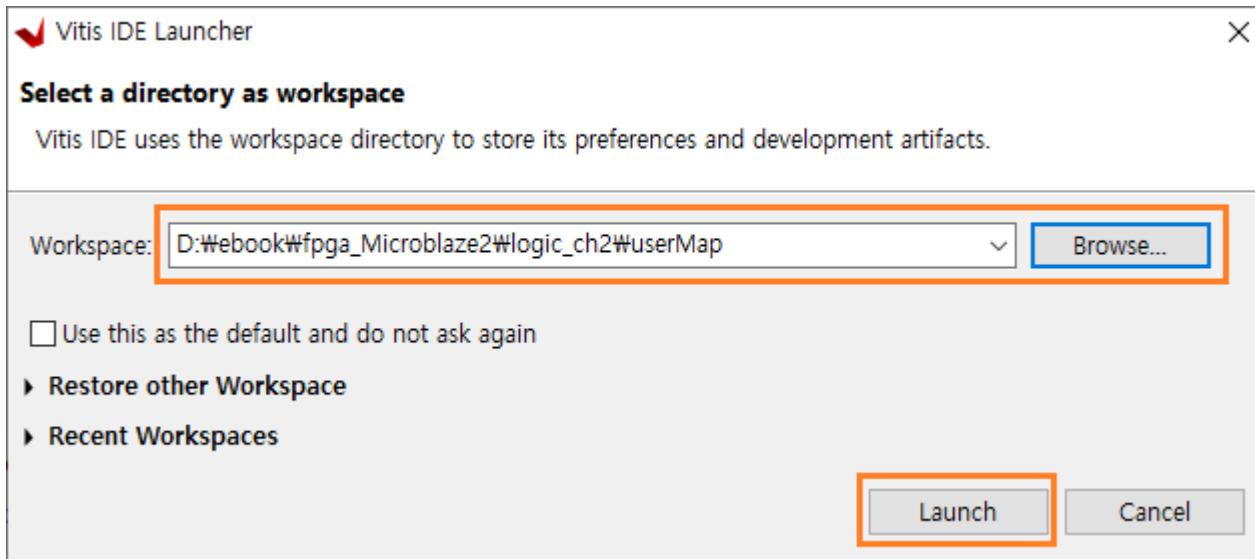
Output : Include bitstream 을 선택하고 Next를 클릭합니다.

Files : 생성되는 폴더 위치와 파일명 (userMapTop.xsa)을 확인하고 Next를 클릭합니다.

Exporting Hardware Platform : Finish를 클릭하면 xsa 파일이 생성됩니다.

11.4 응용 SW 구현

Tools - Launch Vitis IDE를 클릭하여 Vitis를 실행합니다. Browse를 클릭해서 프로젝트 폴더(bram03)를 선택하고 Launch를 클릭합니다.



Vitis에서 PROJECT - Create Application Project를 클릭합니다.

Create a New Application Project : Next를 클릭합니다.

Platform : Create a new platform from hardware (XSA)를 클릭합니다. “XSA File : “ 우측의 Browse... 버튼을 클릭하여 생성된 xsa (userTopTop.xsa) 파일을 선택합니다. Next 를 클릭합니다.

Application Project Details : Application project name 에 “userMapApp”을 입력하고 Next를 클릭합니다.

Domain : Next를 클릭합니다.

Templates : Hello World를 선택하고 Finish를 클릭하여 프로젝트를 생성합니다.

응용 SW는 다음과 같은 기능을 구현합니다.

- ✓ Customize IP로 생성한 Memory read/write 구현
- ✓ User Logic Register read/write 구현
- ✓ 1초마다 pwm duty를 변화하면서 led 밝기 변화 구현

아래는 helloworld.c 코드를 보여줍니다. (자료실에서 다운로드 받은 파일을 사용하시길 바랍니다)

```

48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51 #include "xparameters.h"
52 #include "sleep.h"
53
54 #define rPWM_FREQ1      0x20
55 #define rPWM_FREQ2      0x24
56 #define rPWM_FREQ3      0x28
57 #define rPWM_FREQ4      0x2c
58
59 #define rPWM_DUTY1      0x30
60 #define rPWM_DUTY2      0x34
61 #define rPWM_DUTY3      0x38
62 #define rPWM_DUTY4      0x3c

```

- ✓ 라인 48 - 52 : include 문
- ✓ 라인 54 - 62 : user register address (ax_reg.v 에 정의된 address)

```

78 xil_printf("\r\n1) customize bram test .. \r\n");
79 xil_printf("write bram-1 ..\r\n");
80 for(i=0; i<2048; i++)
81 {
82     Xil_Out32(XPAR_BRAM_0_BASEADDR+(4*i), 0xaabbcc00+i);
83 }
84
85 xil_printf("read bram-1 ..\r\n");
86 for(i=0; i<2048; i++)
87 {
88     rdata = Xil_In32(XPAR_BRAM_0_BASEADDR+(4*i));
89     if( rdata != (0xaabbcc00+i)) err_cnt++;
90     //xil_printf("i : %03d, w : %x, r : %x \r\n", i, (0xaabbcc00+i), rdata);
91 }
92 xil_printf("error_cnt : %d \r\n", err_cnt);

```

- ✓ 라인 78 - 92 : user block memory read/write test
- ✓ 라인 80 - 83 : 메모리의 전영역에 0xaabbcc00 ~ 0xaabbcc+2047까지 write
- ✓ 라인 86 - 91 : 메모리의 전영역의 read, write data와 read data를 비교하여 에러 카운팅
- ✓ 라인 92 : 에러값 표시

```

94 xil_printf("\r\n2) user register test.. \r\n");
95 xil_printf("write user register ..\r\n");
96 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ1, 0x1000);
97 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ2, 0x2000);
98 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ3, 0x3000);
99 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ4, 0x3000);
100
101 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY1, 10);
102 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY2, 30);
103 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY3, 50);
104 Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY4, 70);
105
106 xil_printf("read user register ..\r\n");
107 xil_printf("freq1 : %x \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ1) );
108 xil_printf("freq2 : %x \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ2) );
109 xil_printf("freq3 : %x \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ3) );
110 xil_printf("freq4 : %x \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_FREQ4) );
111
112 xil_printf("duty1 : %d \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY1) );
113 xil_printf("duty2 : %d \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY2) );
114 xil_printf("duty3 : %d \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY3) );
115 xil_printf("duty4 : %d \r\n", Xil_In32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY4) );

```


- ✓ 라인 94 - 115 : user register read/write test
- ✓ 라인 96 - 99 : freq1 ~ freq4에 0x1000, 0x2000, 0x3000, 0x4000 write
- ✓ 라인 101 - 104 : duty1 ~ duty4에 10, 30, 50, 70 write
- ✓ 라인 106 - 115 : freq1-4, duty1-4 값을 읽어서 화면에 출력

```

117     duty = 0;
118     while(1)
119     {
120         xil_printf("duty : %d \r\n", duty);
121         Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY1, duty);
122         Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY2, duty);
123         Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY3, duty);
124         Xil_Out32(XPAR_BRAM_1_BASEADDR+rPWM_DUTY4, duty);
125
126         duty++;
127         if(duty > 100) duty = 0;
128         sleep(1);
129     }
130
131     cleanup_platform();
132     return 0;
133 }

```

- ✓ 라인 117 - 129 : 1초마다 duty 값을 0 ~ 100까지 증가함. LD4 ~ LD7의 밝기가 1초마다 변하는 것을 알 수 있습니다.



11.5 다운로드 및 결과 확인

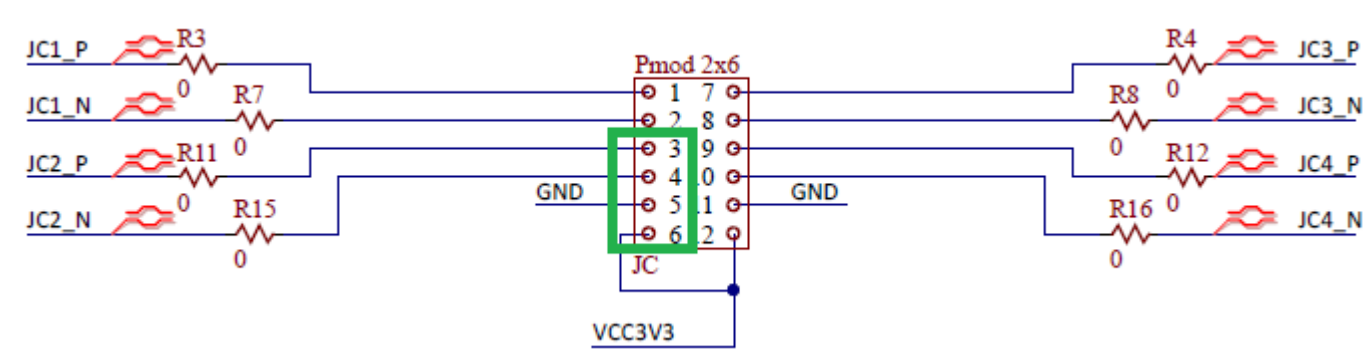
Vitis에서 userMapApp_system - 우클릭 - Build Project 를 클릭해서 Build 합니다.

보드에 USB 케이블을 연결하고, 자료실에서 다운받은 “WinIDT17₩Release₩WinIDT.exe”를 실행합니다. COM 포트를 확인하고 속도를 115200으로 하고 Open 를 클릭합니다.

이때 주의하실 사항이 있습니다. PC의 장치관리자를 보면 COM 포트가 2개가 잡힐 것입니다. 하나는 보드와 연결한 USB 케이블용 COM 포트, 다른 하나는 Max3232 모듈을 연결한 COM 포트입니다. xdc 파일을 만들 때, uart 포트를 USB 케이블을 사용하지 않고, 보드의 JC 커넥터로 연결하였습니다. 따라서 JC 커넥터에 Max3232 모듈을 연결하고, WinIDT 프로그램에서 COM 포트를 Open할 때, Max3232 모듈과 연결된 COM 포트를 Open해야 합니다.

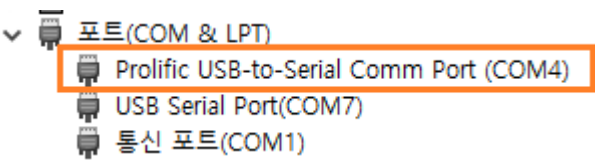
아래는 JC 커넥터와 Max3232 모듈의 핀맵입니다.

■ Max3232 Module 핀맵

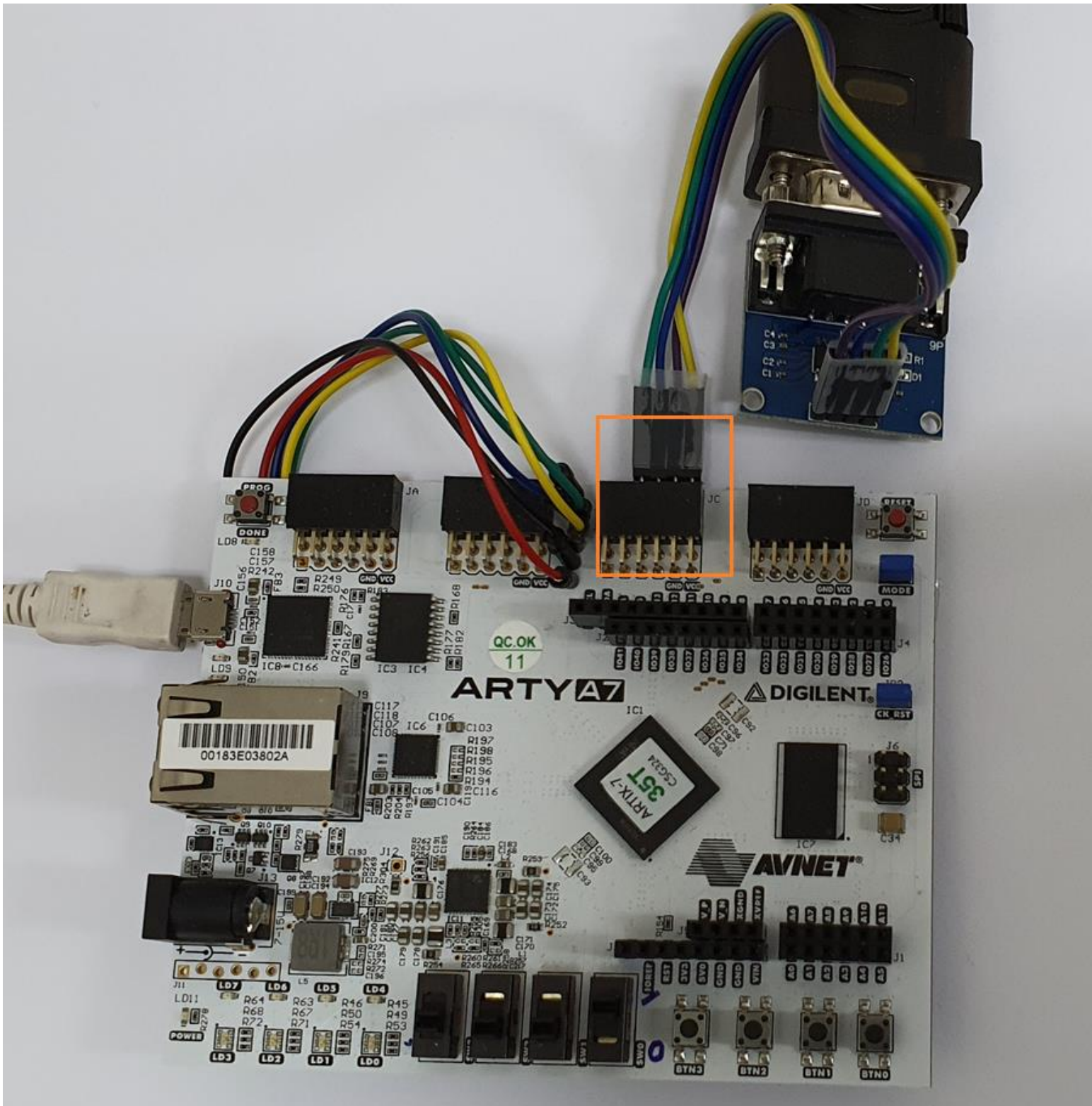


Arty A7 JC	Max3232 Module
3 (JC2_P)	2 (RXD)
4 (JC2_N)	3 (TXD)
5 (GND)	4 (GND)
6 (VCC3V3)	5 (VCC)

장치관리자의 COM port 입니다. 제 PC에서는 COM4가 Max3232 모듈과 연결된 포트입니다.



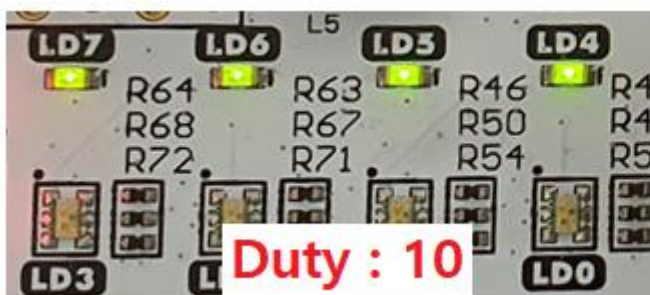
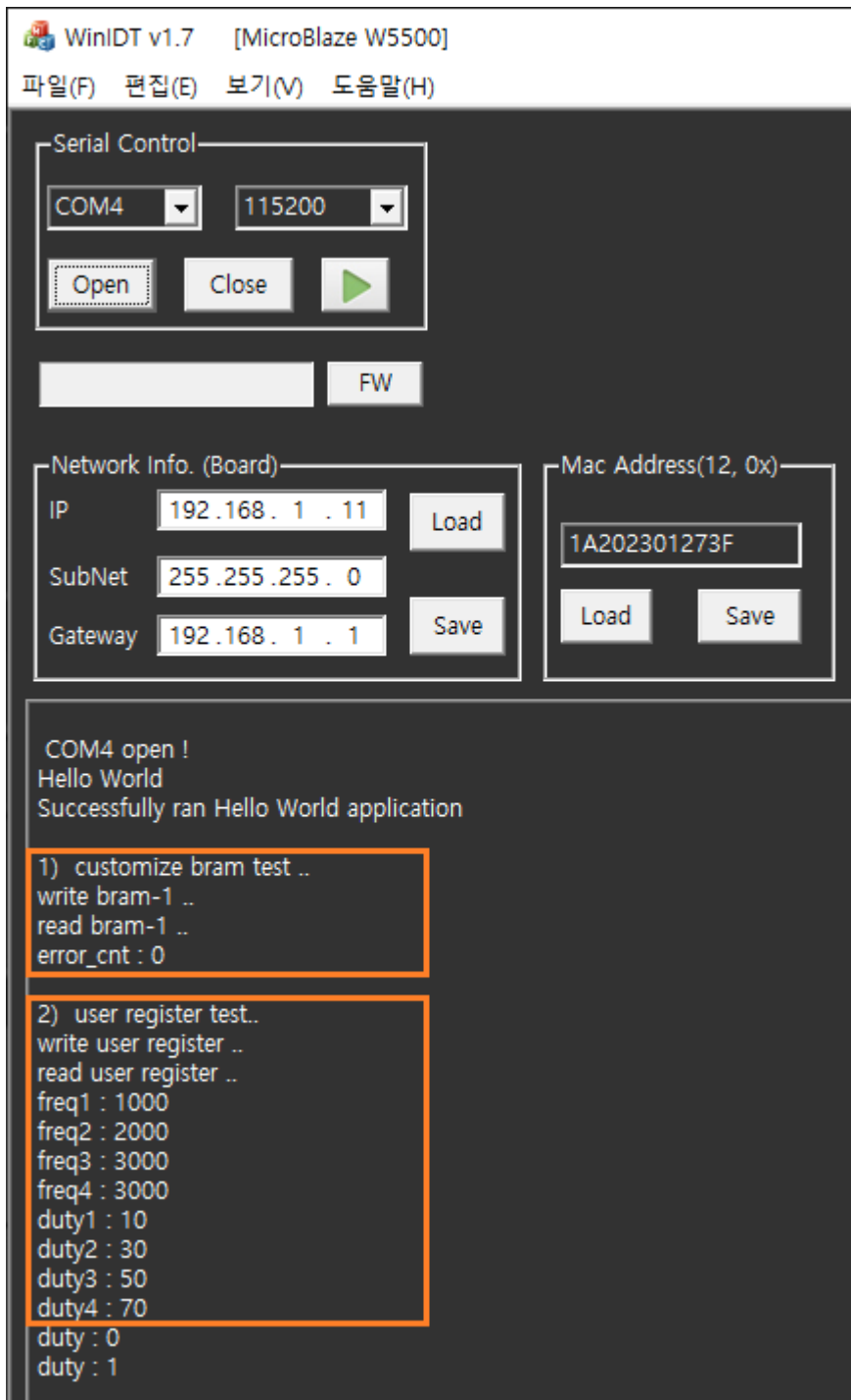
아래는 보드와 연결된 모습을 보여줍니다.



Vitis에서 Xilinx - Program Device를 클릭합니다.

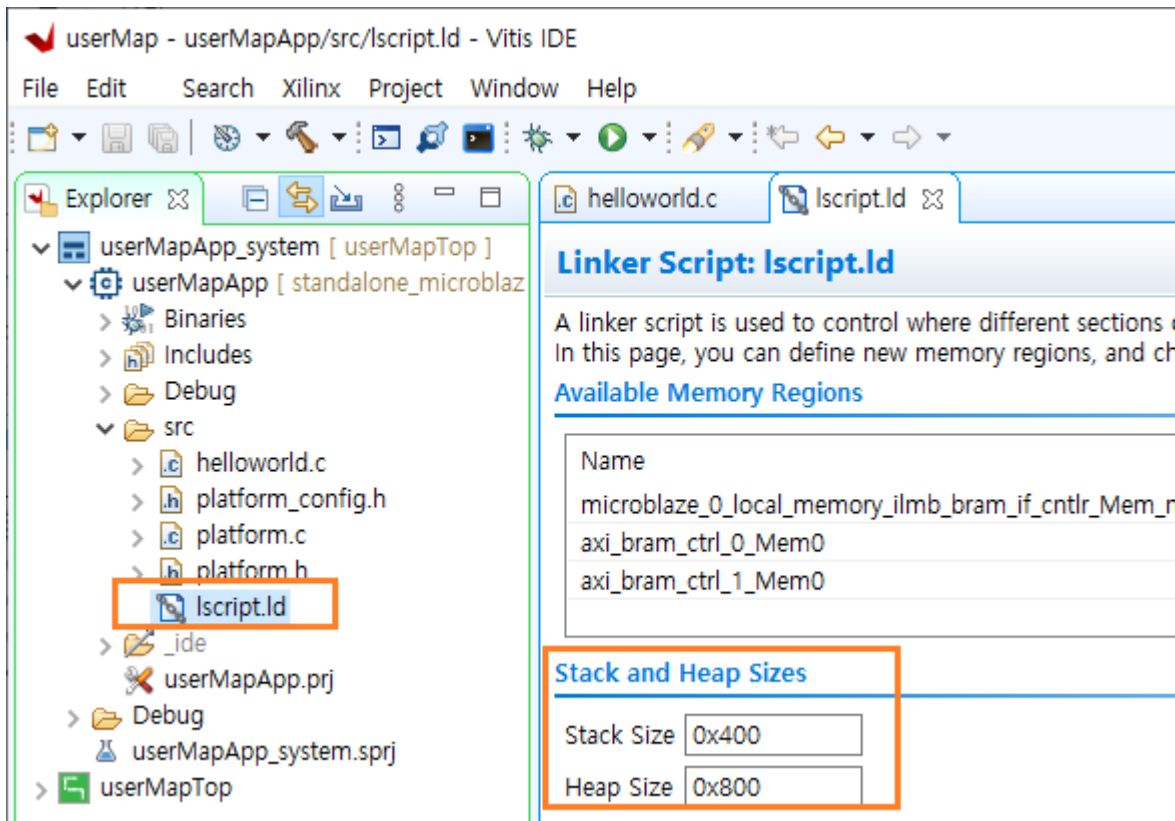
Program Device : bootloop 우측을 클릭해서 “userMapApp\Debug\userMapApp.elf”를 선택하고 Program 을 클릭 하면 프로그램이 다운로드 됩니다.

아래는 결과를 나타냅니다. bram read/write test 의 결과가 error_cnt : 0 임을 알 수 있습니다. 또한 user register에 write 후에 read한 값을 보면, write 한 값과 동일한 값이 읽히는 것을 알 수 있습니다. 1초마다 LED4-7까지의 duty가 바뀌면서 밝기가 변하는 것도 확인할 수 있습니다.

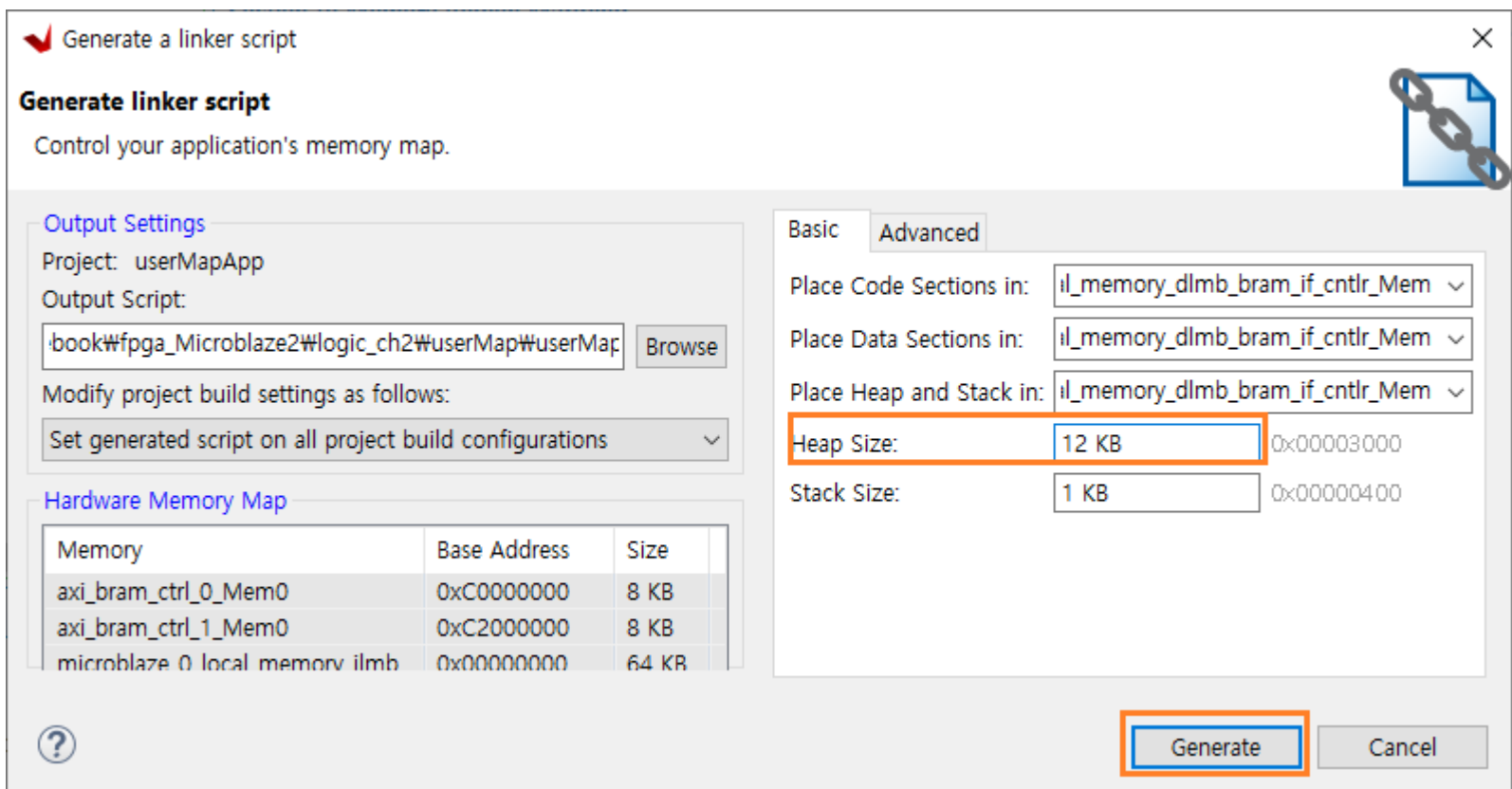


11.6 링크 스크립트 수정

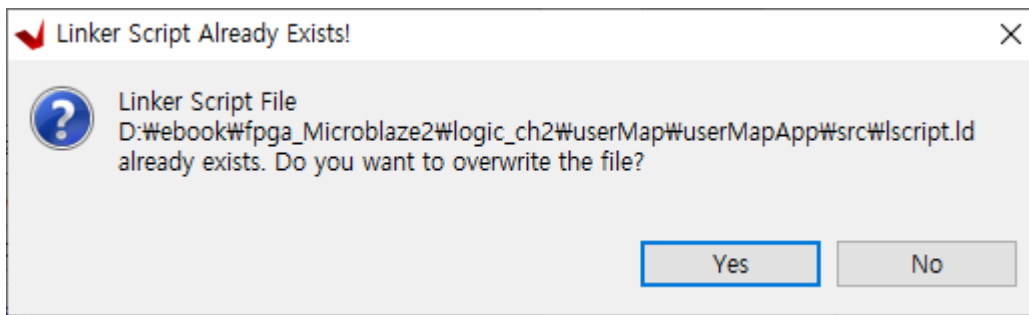
응용 SW에서 Heap 영역의 사이즈를 변경하려면 링크 스크립트를 수정해야 합니다. 현재 설정되어 있는 Stack, Heap Size를 확인하려면 src 아래 있는 lscript.ld 파일을 더블 클릭하면 됩니다. 현재는 기본으로 Stack Size : 0x400 (1KB), Heap Size : 0x800 (2KB)로 설정되어 있습니다.



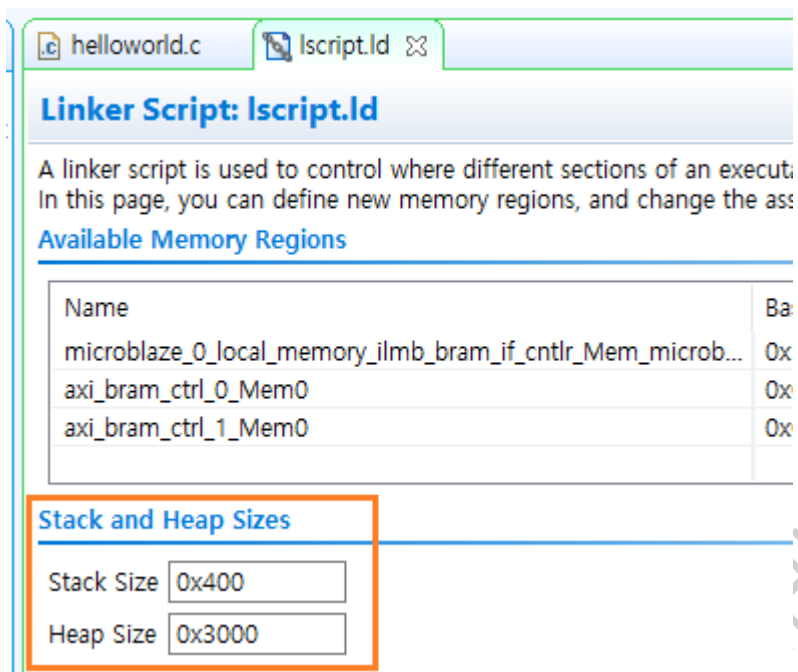
링크 스크립트를 수정하기 위해서는 Xilinx - Generate Linker Script 를 클릭합니다. Basic 탭에서 Heap Size를 변경하고 Generate 를 클릭합니다.



Yes를 클릭합니다.



변경된 내용을 확인하기 위해서 다시 lscript.ld를 더블 클릭합니다. Heap Size가 변경된 것을 확인할 수 있습니다.



12. 참고 자료

- 1) AXI GPIO v2.0 (PG144), <https://docs.xilinx.com/v/u/en-US/pg144-axi-gpio>
- 2) AXI Timer v2.0 (PG079), <https://docs.xilinx.com/v/u/en-US/pg079-axi-timer>
- 3) AXI UART Lite v2.0 (PG142), <https://docs.xilinx.com/v/u/en-US/pg142-axi-uartlite>
- 4) AXI Quad SPI v3.2 (PG153),

<https://docs.xilinx.com/viewer/book-attachment/S1BsQs9DwHr9jUb3xboS~A/pKXanuS1VbdLmAGuhoyknA>

- 5) lwIP Applications For the Arty Evaluation Board
- 6) LightWeight IP Application Examples (XAPP1026)
- 7) [STM32 HAL] LwIP TCP Echo Server, <https://blog.naver.com/eziya76/221854875861>
- 8) w5500 datasheet, v1.0.7
- 9) w5500 IOLIBRARY_BSD 를 MCU 8051 로 포팅하기, <https://midnightcow.tistory.com/4>
- 10) 기타.



<https://cafe.naver.com/worshippt> 카페의 전자문서 - MicroBlaze 방에 질문을 남겨 주시면 답변해 드리도록 하겠습니다.

감사합니다~

13. Revision History

Date	Version	Description
2022. 10. 04	1.0	Initial Release
2023. 01. 07	1.1	spl slave controller 코드 구현, simulation 업데이트 6.2.2.4 코드 구현 추가됨 6.2.4.5 simulation update
2023. 01. 14	1.2	7. LightWeight IP (lwIP) Echo Server 구현 추가됨. 8. lwIP 활용 추가됨.
2023. 01. 17	1.3	5장 내용 새롭게 작성함 6장 내용 새롭게 작성함
2023. 02. 16	1.4	9. W5500을 이용한 TCP/IP 구현 추가됨.
2023. 02. 24	1.5	10 - 11 장 추가됨 10장 Block Memory Interface - 1 11장 Block Memory Interface - 2
2023. 06. 08	1.6	오타 수정
2023. 08. 11	2.0	4장 내용 완전히 새롭게 업데이트 5장 내용 완전히 새롭게 업데이트
2023. 08. 12	2.1	6장 업데이트

[저작권 관련 내용]

이 자료는 대한민국 저작권법의 보호를 받습니다. 작성된 모든 내용의 권리는 작성자에게 있으며, 작성자의 동의 없는 사용이 금지됩니다.

본 자료의 일부 혹은 전체 내용을 무단으로 복제/배포하거나 2차적 저작물로 재편집하는 경우, 5년 이하의 징역 또는 5천만원 이하의 벌금과 민사상 손해배상을 청구합니다.

※ 저작권법 제 30조(사적이용을 위한 복제)

공표된 저작물을 영리를 목적으로 하지 아니하고, 개인적으로 이용하거나 가정 및 이에 준하는 한정된 범위 안에서 이용하는 경우에는 그 이용자는 이를 복제할 수 있다. 다만, 공중의 사용에 제공하기 위하여 설치된 복사기기에 의한 복제는 그러지 아니하다.

※ 저작권법 제 136조(벌칙)의 ① 다음 각 호의 어느 하나에 해당하는 자는 5년 이하의 징역 또는 5천만원 이하의 벌금에 처하거나 이를 병과할 수 있다.

지적재산권 및 이 법에 따라 보호되는 재산적 권리(제93조에 따른 권리는 제외한다)를 복제, 공연, 공중 송신, 전시, 배포, 대여, 2차적 저작물 작성의 방법으로 침해한 자

※ 민법 제750조(불법행위의 내용)

고의 또는 과실로 인한 위법행위로 타인에게 손해를 가한 자는 그 손해를 배상할 책임이 있다.